

Objective-C с нуля

От переменной до runtime и Foundation — разбираем каждую строку

Учебник

Содержание

Введение. Зачем эта книга и что такое Objective-C	1
Что такое Objective-C	1
Откуда он взялся	2
Жив ли Objective-C сегодня	2
Что мы построим за книгу	3
Как устроена каждая глава	3
Что тебе понадобится	4
Глава 1. Инструменты и первая программа	5
Что мы сделаем	5
Ставим инструменты	5
Пишем первую программу	6
Компилируем и запускаем	6
Разбираем каждую строку	7
#import <Foundation/Foundation.h>	7
int main(void)	7
@autoreleasepool { ... }	8
NSString *greeting = @"Привет, Objective-C!";	8
NSLog(@"%@", greeting);	8
return 0;	9
Проверяем	9
Частые ошибки — и что компилятор на них скажет	9
Упражнения	11
Что мы получили	11
Документация Apple	11
Глава 2. База Си: переменные, типы, ветвления, циклы	12
Что мы сделаем	12
Переменные и типы	12
Печать через printf и спецификаторы	14
Арифметика и ловушка целочисленного деления	15
Ввод с клавиатуры через scanf	16
Ветвления: if и else	18
Циклы: while и for	19
break и continue	20
Функции: выносим логику	21
Проверяем	23
Частые ошибки	23

Упражнения	24
Что мы получили	25
Документация Apple	25
Глава 3. Указатели и память	26
Что мы сделаем	26
Память — это длинная полка с ячейками	26
Шаг 1. Берём адрес переменной — оператор &	27
Шаг 2. Указатель — переменная для адреса	27
Шаг 3. Разыменование — та же звёздочка, другой смысл	28
Шаг 4. NULL — указатель в никуда	29
Шаг 5. Функция, которая не работает: наивный swap	30
Шаг 6. Чиним swap указателями	31
Шаг 7. Где живут данные: стек и куча	32
Шаг 8. Берём память руками: malloc, sizeof, free	33
Шаг 9. struct — склеиваем данные в один тип	34
Проверяем	36
Частые ошибки	36
Упражнения	37
Что мы получили	37
Документация Apple	37
Глава 4. Массивы и строки в Си	39
Что мы сделаем	39
Массив: набор значений под одним именем	39
Перебираем циклом	40
Выход за границу: Си не ловит беду	40
Имя массива — это адрес начала	41
Строка в Си — это массив символов	42
Зачем нужен '\0'	42
Строим свою функцию длины	44
Сравнение строк: ловушка ==	45
Копирование строк и опасность переполнения	47
Ввод строки: fgets, и почему gets запрещён	48
Кириллица в UTF-8: буква ≠ байт	49
Проверяем	50
Частые ошибки	51
Упражнения	51
Что мы получили	52
Документация Apple	52
Глава 5. Объекты и сообщения	53
Что мы сделаем	53
Класс и объект — на пальцах	53
Отправка сообщения	54
Что значит «сообщение», а не «вызов»	55
Шаг 1. Шлём строке несколько сообщений	55
Шаг 2. Сообщение с аргументом	56

Шаг 3. Вложенные сообщения	57
Шаг 4. Создаём объект: alloc и init	59
Шаг 5. Многоаргументный селектор	60
id — любой объект	61
nil — ноль для объектов	61
Сообщение к nil безопасно	62
Интроспекция — на вкус	63
Полная картина: что происходит при [obj msg:arg]	64
Проверяем	64
Частые ошибки	65
Упражнения	65
Что мы получили	66
Документация Apple	67
Глава 6. Свой первый класс	68
Что мы сделаем	68
Что такое класс	68
Разбираем задачу руками	69
Два блока: @interface и @implementation	69
Шаг 1. Объявляем класс	70
Шаг 2. Пишем реализацию	71
Шаг 3. Создаём объект и запускаем	72
Шаг 4. Метод report и self	73
Шаг 5. Метод класса +	74
Полная программа	75
Как это работает	77
Проверяем	77
Второй класс: Person	77
Немного про init	80
Частые ошибки	80
Упражнения	81
Что мы получили	82
Документация Apple	82
Глава 7. Создание объектов: alloc, init и dealloc	83
Что мы сделаем	83
Разбираем задачу руками	83
Шаг 1. Самый первый класс с init	84
@interface Person : NSObject { ... }	85
– (instancetype)init;	85
self = [super init];	86
if (self) { ... }	86
return self;	87
Шаг 2. Почему именноinstancetype, а не id	87
Шаг 3. Запускаем и смотрим на рождение объекта	88
Шаг 4. dealloc — момент смерти объекта	88
Шаг 5. Инициализатор с аргументами	90
Шаг 6. Назначенный и вторичные инициализаторы	90

Шаг 7. Фабричный метод класса	92
Как это работает	93
Проверяем	94
Частые ошибки	94
Упражнения	95
Что мы получили	96
Документация Apple	96
Глава 8. Свойства	97
Что мы сделаем	97
Разбираем задачу руками	97
@property — одна строка вместо четырёх	98
Точка — это сообщение, не поле	99
Атрибуты свойства	101
Атомарность: atomic vs nonatomic	101
Способ хранения: strong, weak, copy, assign	101
Доступность: readonly vs readwrite	102
Переименование: getter= и setter=	102
copy: зачем он нужен	102
А не дорого ли копировать каждую строку?	104
Свой геттер или сеттер поверх @property	105
@synthesize и @dynamic — пара слов для истории	107
_ivar или self.name — что когда	108
Проверяем	108
Частые ошибки	109
Упражнения	109
Что мы получили	110
Документация Apple	110
Глава 9. Наследование	111
Что мы сделаем	111
Разбираем задачу руками	111
Шаг 1. Базовый класс Animal	112
@interface Animal : NSObject	112
Шаг 2. Подкласс Dog	114
Шаг 3. Подкласс Cat	116
Шаг 4. Полиморфизм — один цикл, разные голоса	116
Почему вызовется метод фактического класса	117
Полная программа	118
Интроспекция: спросить объект, кто он такой	121
[obj class] и [obj superclass]	122
isKindOfClass: — этот класс или потомок	123
isMemberOfClass: — ровно этот класс	123
respondToSelector: — ответит ли на сообщение	123
Абстрактный базовый класс по соглашению	124
Как это работает	127
Проверяем	127
Частые ошибки	128

Упражнения	128
Что мы получили	129
Документация Apple	129
Глава 10. Протоколы и делегаты	131
Что мы сделаем	131
Зачем протокол, если есть наследование	131
Объявляем первый протокол	132
Класс принимает протокол	133
Тип <code>id<Protocol></code> — «любой, лишь бы умел»	135
Проверяем контракт: <code>conformsToProtocol:</code>	136
Собираем корзину	136
Паттерн делегирования: строим с нуля	137
Разбираем задачу руками	138
Шаг 1. Наивно — жёсткая связь	138
Шаг 2. Разворачиваем зависимость через протокол	139
Почему <code>delegate</code> — это <code>weak</code>	143
<code>@required</code> и <code>@optional</code>	143
Формальные и неформальные протоколы	146
Как это работает	147
Проверяем	147
Частые ошибки	148
Упражнения	149
Что мы получили	149
Документация Apple	150
Глава 11. Категории и расширения класса	151
Что мы сделаем	151
Разбираем задачу руками	152
Шаг 1. Объявляем категорию	152
<code>@interface NSString (MyUtils)</code>	152
Шаг 2. Реализуем методы категории	153
Шаг 3. Пользуемся — на обычных строках	154
Шаг 4. А теперь ломаем: эмодзи	156
Шаг 5. Чиним: разворот по составным символам	156
Зачем нужны категории	157
Ограничения категорий — важно	158
Расширение класса: приватная часть интерфейса	159
<code>@interface Counter ()</code>	159
Реализация	160
Пробуем	161
Ассоциированные объекты: «хранилище» для категории	162
Проверяем	165
Частые ошибки	166
Упражнения	166
Что мы получили	167
Документация Apple	167

Глава 12. Блоки	168
Что мы сделаем	168
Сначала — указатель на функцию в Си	168
Что такое блок	169
Блоки разной формы	170
Собираем первую программу	170
Захват окружения	171
typedef: короткое имя длинному типу	172
Программа с захватом	172
Изменяемый захват: __block	173
Программа с __block	173
Блок как аргумент метода: completion handler	174
Программа с completion handler	175
Блоки в Foundation	175
Ловушка блоков: цикл удержания	177
Чиним: weakSelf	178
Видим утечку вживую	178
Блоки и память: стек и куча	180
Полная программа	180
Проверяем	180
Частые ошибки	181
Упражнения	181
Что мы получили	182
Документация Apple	182
 Глава 13. Управление памятью: ARC и подсчёт ссылок	 183
Что мы сделаем	183
Разбираем задачу руками	184
Шаг 1. Модель подсчёта ссылок	184
Шаг 2. Прогоняем счётчик руками (без ARC)	185
Шаг 3. autorelease и пул автоосвобождения	188
Шаг 4. Включаем ARC — тот же пример, короче	189
Что ARC делает и чего НЕ делает	191
Шаг 5. Сильные и слабые ссылки	191
Шаг 6. Цикл удержания — главная ловушка	192
Шаг 7. Режем цикл словом weak	195
Шаг 8. autoreleasepool внутри циклов	197
Проверяем	198
Частые ошибки	199
Упражнения	199
Что мы получили	200
Документация Apple	200
 Глава 14. Среда выполнения и иерархия классов	 201
Что мы сделаем	201
Шаг 1. Объект знает свой класс	202
Шаг 2. Класс — это тоже объект	204
Шаг 3. Полная схема указателей	208

Шаг 4. Два корня дерева: NSObject и NSProxy	209
Шаг 5. Как сообщение находит метод: objc_msgSend	210
Шаг 6. Живая интроспекция: дампы иерархии и методов	212
Шаг 7. Динамическое разрешение метода	215
Шаг 8. Пересылка сообщения: три шанса	218
Шаг 9. doesNotRecognizeSelector: и падение	221
Шаг 10. Method swizzling: подмена реализации	222
Связь с KVO и тизер главы 20	224
Проверяем	224
Частые ошибки	225
Упражнения	226
Что мы получили	226
Документация Apple	227
Глава 15. Foundation: строки	228
Что мы сделаем	228
NSString: строка как объект	228
Почему NSString неизменяемый	229
Создаём строки	229
length: длина в UTF-16 code units	231
Правильный обход по «человеческим» символам	232
Сравнение строк	233
Порядок, регистр, префиксы	234
Поиск и подстроки	235
Регистр и обрезка	236
Разбиение, склейка и неизменяемая «дописка»	237
NSMutableString: строку можно менять	238
Число ↔ строка	239
NSAttributedString — строка с оформлением	240
Строим с нуля: нормализатор логина	240
Шаг 1. Наивная версия	240
Шаг 2. Срезаем пробелы	241
Шаг 3. Снимаем ведущую @	241
Шаг 4. Отвергаем пустой ввод	241
Полная функция	241
Проверяем	242
Частые ошибки	242
Упражнения	243
Что мы получили	243
Документация Apple	244
Глава 16. Foundation: числа и обёртки значений	245
Что мы сделаем	245
Разбираем задачу руками	245
NSNumber: упаковка числа в объект	246
Литералы: самый короткий способ	246
Фабричные методы: то же самое словами	247
Распаковка: достаём примитив обратно	247

Собираем первый пример	247
Сравнение NSNumber: почему == врёт	249
Ловушка кэширования: иногда == «работает», и это хуже всего	250
NSNumber: обёртка для любой C-структуры	251
NSDecimalNumber: точные деньги	253
Шаг 1. Наивная версия на double — и она врёт	253
Шаг 2. Чиним: NSDecimalNumber	254
Шаг 3. Складываем весь чек	255
NSNull: объект-«ничто» для коллекций	255
nil, NSNull и Nil — три разные вещи	256
Полные программы	257
Проверяем	257
Частые ошибки	258
Упражнения	258
Что мы получили	259
Документация Apple	259
Глава 17. Foundation: коллекции	260
Что мы сделаем	260
Три контейнера и зачем их три	260
NSArray: упорядоченный неизменяемый список	261
count, индексы, края	261
Поиск: containsObject: и indexOfObject:	262
componentsJoinedByString:	262
NSMutableArray: тот же массив, но изменяемый	262
nil класть нельзя	263
Перебор массива	264
Быстрый перебор for-in	264
enumerateObjectsUsingBlock: с индексом и stop	264
Обратный перебор	264
Проверяем (массивы)	265
NSDictionary: пары ключ → значение	266
Доступ по ключу	266
allKeys, allValues	266
NSMutableDictionary	266
Перебор словаря	267
Требования к ключам	267
Проверяем (словарь)	268
NSSet: уникальные элементы без порядка	268
Когда множество лучше массива	268
Операции над множествами	269
NSOrderedSet — уникальность плюс порядок	269
Проверяем (множества)	270
Сортировка и фильтрация	270
sortedArrayUsingComparator:	270
sortUsingDescriptors: кратко	271
filteredArrayUsingPredicate: и NSPredicate	271
valueForKey: на массиве — тизер KVC	272

Проверяем (сортировка и фильтр)	272
Ловушка: мутация во время перебора	272
Строим с нуля: учёт товаров за день	274
Шаг 1. Журнал и подсчёт	274
Шаг 2. Сколько разных товаров	275
Шаг 3. Категории через множество	275
Шаг 4. Рейтинг	275
Шаг 5. Чистка однодневок — аккуратно	275
Полная программа	276
Проверяем (учёт товаров)	278
Как это работает	278
coru vs mutableCoru у коллекций	279
Частые ошибки	279
Упражнения	280
Что мы получили	280
Документация Apple	280
Глава 18. Foundation: даты, данные, URL, ошибки	281
Что мы сделаем	281
NSDate: момент времени	281
Секунды: с эпохи и от «сейчас»	282
Арифметика дат	282
Сравнение дат	283
NSCalendar и NSDateComponents: год, месяц, день	283
Разложить дату на компоненты	284
Собрать дату из компонентов	284
Разница в днях — по календарю	285
Почему нельзя «просто прибавить 86400 секунд»	285
NSDateFormatter: дата ↔ строка	286
Две ловушки форматтера	287
NSData: мешок байтов	287
Строка ↔ байты	287
base64	288
NSData и файл	288
NSURL: ссылки и пути к файлам	289
URL ресурса	289
URL файла	289
NSUUID: уникальный идентификатор	290
NSError: как сообщают об ошибке	291
Что внутри NSError	291
Потребляем чужую ошибку: читаем несуществующий файл	291
Создаём свою ошибку	292
Строим: «сколько дней до дедлайна» и чтение файла с ошибкой	293
Проверяем	295
Частые ошибки	295
Упражнения	295
Что мы получили	296
Документация Apple	296

Глава 19. Foundation: файлы и сериализация	297
Что мы сделаем	297
NSFileManager: работа с файлами и папками	298
Куда вообще можно писать	298
Собираем путь из кусочков	299
Создаём папку	299
Существование, файл или папка	300
Список содержимого папки	300
Атрибуты файла	300
Удаление	300
Чтение и запись строк и данных	301
Записать строку	301
Прочитать строку	302
Сырые байты: NSData	302
Property list: сохраняем коллекции	303
JSON: NSJSONSerialization	304
Объект → JSON	304
JSON → объект	305
Ошибки — через возврат, не через краш	305
Строим с нуля: список задач в JSON-файл	306
Шаг 1. Данные	306
Шаг 2. В файл и обратно	306
Шаг 3. Работаем с прочитанным	306
Архивация объектов: NSData	307
Учим класс кодировать себя	308
Архивируем и восстанавливаем	309
Защита в действии	309
Какой формат когда	310
Проверяем	310
Частые ошибки	311
Упражнения	311
Что мы получили	312
Документация Apple	312
 Глава 20. KVC и KVO	 314
Что мы сделаем	314
Часть 1. KVC — доступ по строковому имени	315
Разбираем задачу руками	315
Шаг 1. Читаем и пишем свойство по имени	315
Шаг 2. Что KVC делает под капотом	316
Шаг 3. Key path — путь сквозь вложенные объекты	317
Шаг 4. KVC и коллекции	318
Зачем вообще KVC	321
Часть 2. KVO — наблюдение за изменением	321
Разбираем задачу руками	321
Шаг 5. Подписываемся и ловим изменение	321
Шаг 6. Запускаем и видим isa-swizzling	323
Шаг 7. Сеттер уведомляет, прямой ivar — нет	324

Шаг 8. Ручные уведомления и automaticallyNotifies...	325
Шаг 9. removeObserver — обязательно	325
А как в Swift: NSKeyValueObservation	326
Полная программа	326
Проверяем	326
Частые ошибки	327
Упражнения	327
Что мы получили	328
Документация Apple	328
Глава 21. Многопоточность, GCD и run loop	330
Что мы сделаем	330
Поток — это что	330
GCD: очереди задач	331
Шаг 1: долгая загрузка в фоне	332
dispatch_sync и дедлок	335
dispatch_after: сделать позже	335
Шаг 2: ждём несколько загрузок — dispatch_group	336
group_notify вместо ожидания	338
dispatch_barrier — кратко	338
Шаг 3: одноразовая инициализация — dispatch_once	338
Гонка данных вживую	341
NSOperation: объектная надстройка над GCD	344
NSThread — самый низкий уровень	345
Run loop: цикл ожидания событий	345
Шаг 4: таймер рассылает уведомления	346
Частые ошибки	350
Упражнения	351
Что мы получили	351
Документация Apple	352
Глава 22. Современный Objective-C	353
Что мы сделаем	353
Литералы и субскрипты — сводка	354
Свой субскрипт	355
NS_ENUM — типобезопасные перечисления	356
NS_OPTIONS — битовые флаги	357
Nullability — где допустим nil	359
Lightweight generics — дженерики у коллекций	360
instancetype вместо id	361
NS_DESIGNATED_INITIALIZER — назначенный инициализатор	362
@available — проверка версии ОС	363
Модули: @import вместо #import	364
KVO и dispatch блоками — пара слов	364
Строим с нуля: осовремениваем класс	365
Шаг 1. Что было	365
Шаг 2. Типы для статуса и прав	366
Шаг 3. Nullability и дженерики в интерфейсе	366

Шаг 4. Смотрим, как компилятор ловит ошибки	368
Шаг 5. Запускаем правильную версию	369
Проверяем	369
Частые ошибки	370
Упражнения	370
Что мы получили	371
Документация Apple	371
Глава 23. Взаимодействие со Swift	372
Что мы сделаем	372
Зачем это вообще нужно	372
Как Objective-C виден из Swift	373
Сценарий 1: один модуль (приложение) — bridging header	373
Сценарий 2: фреймворк — umbrella header и module map	374
Как Swift виден из Objective-C	374
@objc и @objcMembers	375
Что из Swift в принципе НЕ видно из Objective-C	375
Маппинг типов: что во что превращается	376
Собираем класс, который хорошо мостится	377
NS_ASSUME_NONNULL_BEGIN / END	378
Свойства: ownerName, items, promoCode	378
Инициализаторы: initWithOwner: и запрет init	379
Блок с NSError** → throws	379
Метод с NSError** → throws	379
Запускаем objc-сторону	380
Управляем именами: NS_SWIFT_NAME и компания	381
NS_SWIFT_NAME — переименовать для Swift	381
error: → throws и NS_SWIFT_NOTHROW	382
Запускаем второй файл	383
Что мостится плохо: подводные камни	383
Современная связка: как смешать языки в проекте	384
Проверяем	384
Частые ошибки	385
Упражнения	385
Что мы получили	386
Документация Apple	386
Глава 24. Что дальше	387
Что ты теперь умеешь	387
Куда идти по экосистеме Apple	388
Cocoa и AppKit — приложения для macOS	388
UIKit — приложения для iPhone и iPad	388
Core Foundation и toll-free bridging	389
Где пригодится знание runtime	389
Мост к Swift	389
Где сегодня применяют Objective-C	390
Как расти дальше	391
Документация Apple	391

Спасибо, что дошёл	393
Приложение А. Иерархия классов Foundation	394
Дерево классов Foundation	394
Пары «неизменяемый → изменяемый»	398
Ключевые протоколы Foundation	398
Предупреждение: toll-free bridging с Core Foundation	399
Документация Apple	400
Приложение В. Шпаргалка по синтаксису Objective-C	401
Компиляция	401
Скелет программы	402
Отправка сообщений	402
Объявление класса	402
Атрибуты @property	403
Создание объектов	404
Наследование	404
Протоколы	405
Категории и расширения	405
Блоки	406
Память	406
Литералы	407
Интроспекция	407
Частые типы	408
Паттерн NSError	409
Современный Objective-C	409
Спецификаторы формата NSLog	410
Частые грабли — одной строкой	411
Документация Apple	411
Приложение С. Как готовиться к собеседованию по Objective-C	412
Где сегодня собеседуют на Objective-C	412
Что проверяют по уровням	413
Junior — раздел D	413
Middle — раздел E	413
Senior — раздел F	414
Типичные форматы вопросов	414
Классические устные ловушки Objective-C	415
Как готовиться	417
Как разбирать задачи в этом разделе	418
Напоследок	419
Документация Apple	419
Приложение D. Задачи собеседования: Junior	421
Задача 1. Что выведет [nil length]	421
Задача 2. == или isEqualToString:	422
Задача 3. Два NSNumber и ==	423
Задача 4. Что делают alloc и init и что будет, если забыть init	424

Задача 5. strong против copy у свойства NSString	425
Задача 6. BOOL, YES, NO и ловушка с числом	426
Задача 7. addObject:nil — что произойдёт	427
Задача 8. objectAtIndex: против firstObject на пустом массиве	428
Задача 9. nil-значение в словаре и NSNull	429
Задача 10. Почему нельзя [array addObject:5]	429
Задача 11. Быстрый перебор for-in и мутация во время него	430
Задача 12. Полиморфизм: какой метод вызовется	431
Задача 13. isKindOfClass: против isKindOfClass:	432
Задача 14.instancetype или id в фабричном методе	433
Задача 15. Реализуй стек поверх NSMutableArray	434
Задача 16. Найди утечку: сильная ссылка на delegate	436
Задача 17. mutableCopy и copy — что можно менять	437
Задача 18. Объект-значение как ключ словаря	438
Задача 19. Свойство readonly и приватная изменяемость	439
Задача 20. Что такое self в методе и зачем [super init]	440
Что повторить перед собеседованием	441
Документация Apple	442

Приложение Е. Задачи собеседования: Middle 443

Задача 1. Что выведет блок: захват по значению и __block	443
Задача 2. weak после смерти объекта и ловушка unsafe_unretained	445
Задача 3. Найди утечку: команда и игроки	446
Задача 4. Цикл удержания объект ↔ блок: что выведет dealloc	447
Задача 5. Зачем autoreleasepool внутри длинного цикла	448
Задача 6. Поверхностная копия коллекции	449
Задача 7. copy-семантика у NSString-свойства	450
Задача 8. Простой кэш на NSMutableDictionary и copy-ключи	451
Задача 9. Спроектируй делегат с необязательными методами	452
Задача 10. Две категории с одинаковым методом — что вызовется	453
Задача 11. Категория не может добавить ivar — как обойти	454
Задача 12. KVC: доступ по строке, keyPath и операторы	455
Задача 13. KVO: что приходит в observeValueForKeyPath: и почему ivar молчит	456
Задача 14. Реализуй NSCopying	458
Задача 15. Реализуй isEqual: и hash вместе	459
Задача 16. isEqual: против isEqualToString:	460
Задача 17. Быстрый разбор: сработает ли dealloc?	460
Задача 18. Code review: найди и почини утечку в блоке	461
Что проверяет этот раздел	462
Документация Apple	463

Приложение F. Задачи собеседования: Senior 464

Задача 1. Куда указывает isa класса и где живут методы класса	464
Задача 2. objc_msgSend, SEL, IMP, Method — кто есть кто	466
Задача 3. Динамическое разрешение метода: сколько раз позовётся resolve	467
Задача 4. Логирующий прокси на forwardingTargetForSelector:	468
Задача 5. Рассылающий прокси: почему тут нужен forwardInvocation:	469
Задача 6. Ленивый прокси на NSProxy: зачем отдельный корень	470

Задача 7. Method swizzling правильно: +load, dispatch_once, class_addMethod	471
Задача 8. Найди тонкий баг: swizzling, который ломает KVO	472
Задача 9. Связанный объект: хранимое свойство в категории	473
Задача 10. KVO под капотом: подмена isa и NSObjectifying_	474
Задача 11. Две категории с одинаковым методом — какая победит	476
Задача 12. Гонка данных: atomic не делает коллекцию потокобезопасной	477
Задача 13. dispatch_sync на текущей последовательной очереди — дедлок	478
Задача 14. Потокобезопасный синглтон на dispatch_once	479
Задача 15. Потокобезопасный кэш: барьерная запись на concurrent-очереди	480
Задача 16. weak в многопоточности и пулы автоосвобождения в фоне	481
Задача 17. Исключение через границу @autoreleasepool и в фоновом потоке	482
Задача 18. Tagged pointers: почему == для маленьких NSNumber иногда прав	484
Задача 19. Найди и почини утечку: цикл удержания через NSTimer	485
Что повторить перед Senior-собеседованием	487
Документация Apple	488
Приложение G. Objective-C глазами C++-разработчика	490
1. Камертон: статика против динамики	490
2. Класс и файлы	491
3. Вызов метода против отправки сообщения	492
4. Типы и полиморфизм	493
5. Шаблоны против lightweight generics	494
6. Наследование	495
7. Расширение классов: категории	496
8. Память: ARC против RAII	497
9. Свойства	498
10. Перегрузка: её нет	499
11. Пространства имён: их тоже нет	500
12. Интроспекция против RTTI	501
13. Строки и контейнеры: семантика копирования	502
14. Исключения	503
15. Блоки против лямбд	504
16. const, ссылки и параметры	505
17. Objective-C++ (.mm): главное практическое	506
18. Когда что — и сводка	507
Проверяем	508
Документация Apple	509

Введение. Зачем эта книга и что такое Objective-C

Эта книга учит языку **Objective-C** с самого начала. Не «освежает знания», не «помогает перейти с другого языка» — ведёт с нуля. Если ты никогда не писал ни строчки кода, ты всё равно по адресу: мы начнём с того, что такое переменная, и дойдём до того, как во время выполнения программы один объект находит и зовёт метод у другого.

Книга — продолжение серии, в которой уже есть учебник по чистому Си. Objective-C не зря стоит рядом с ним: это **надстройка над Си**. Всё, что умеет Си, умеет и Objective-C, плюс сверху добавлен целый объектный мир — классы, объекты, сообщения. Поэтому первые главы дают компактную базу самого Си, а дальше мы шаг за шагом достраиваем объектную часть.

И ещё одно обещание с первой страницы: по ходу всей книги я буду **регулярно показывать, чем Objective-C отличается от обычного Си**. Только что разобрал кусок Си — и тут же видишь, что Objective-C делает иначе. Так новое ложится на знакомое, и язык перестаёт казаться магией.

Что такое Objective-C

Objective-C — язык программирования, который добавляет к языку Си объектную модель в стиле Smalltalk. «В стиле Smalltalk» здесь не украшение: именно оттуда взята главная идея языка — **отправка сообщений**. В большинстве языков ты *вызываешь метод*. В Objective-C ты *посылаешь объекту сообщение*, а тот уже сам, во время выполнения, решает, какой код на это сообщение выполнить. Эта мелкая на вид разница определяет почти всё в языке, и мы разберём её подробно.

Выглядит код так:

```
[label setText:@"Привет"];
```

Квадратные скобки — это и есть отправка сообщения `setText:` объекту `label` с аргументом `@\"Привет\"`. Пока просто запомни картинку; в главе 5 мы разложим её по косточкам.

Откуда он взялся

В начале 1980-х Брэд Кокс и Том Лав соединили Си со Smalltalk — так появился Objective-C. В 1988 году язык лицензировала компания **NeXT**, основанная Стивом Джобсом после ухода из Apple. На Objective-C была написана операционная система NeXTSTEP и её библиотеки — те самые, чьи имена и сегодня начинаются с букв NS (**NeXTSTEP**): `NSString`, `NSObject`, `NSArray`. Ты будешь видеть этот префикс в каждой главе про Foundation.

В 1996 году Apple купила NeXT, вернула Джобса и положила NeXTSTEP в основу будущей Mac OS X. Вместе с системой Apple унаследовала и Objective-C — на нём десятилетиями писали приложения для Mac, а с 2008 года и для iPhone. Целая эпоха мобильной разработки прошла на этом языке.

В 2014 году Apple представила **Swift** — новый язык, в который сейчас вкладывается основная энергия. Возникает законный вопрос.

Жив ли Objective-C сегодня

Да. На момент написания книги Apple не объявляла дату прекращения поддержки Objective-C. Компилятор `clang` собирает его без проблем, язык работает в самых свежих версиях Xcode, приложения на нём принимаются в App Store без ограничений.

Более того, Objective-C никуда не делся из-под капота. Огромные системные библиотеки — Foundation, AppKit, многое в UIKit — написаны на нём. Когда из Swift ты дёргаешь привычный API, очень часто под ним работает Objective-C-механика. Знать её — значит понимать, как платформа Apple устроена изнутри, а не только её современный фасад.

Где Objective-C реально нужен прямо сейчас:

- **Поддержка и развитие существующих проектов.** В мире живут миллионы строк рабочего Objective-C-кода, и их не переписывают одним движением.
- **Взаимодействие с Swift.** Мост между двумя языками двусторонний и официально поддерживается. Большие кодовые базы годами живут смешанными.
- **Понимание runtime.** Method swizzling, интроспекция, динамическая диспетчеризация, отладка системных вызовов — всё это и в Swift-проектах упирается в Objective-C-runtime.

Куда движется развитие — понятно: новые возможности платформы Apple выходят в первую очередь (а часто и только) для Swift. Поэтому честный ориентир такой: **новый проект с чистого листа сегодня обычно начинают на Swift**, а Objective-C учат, чтобы уверенно работать

с существующим кодом, понимать платформу изнутри и при необходимости связывать два языка. Эта книга даёт ровно такой фундамент.

Что мы построим за книгу

Маршрут — от азов до глубины:

- **База Си** (главы 2–4): переменные, типы, указатели, память, массивы и строки. Без этого объектная часть повисает в воздухе.
- **Объектная модель** (главы 5–12): объекты и сообщения, свой первый класс, свойства, наследование, протоколы, категории, блоки.
- **Память и runtime** (главы 13–14): сначала как объекты живут и умирают (ARC и ручной подсчёт ссылок), потом большая глава про сам runtime — isa, метаклассы, дерево классов, как objc_msgSend находит метод, как работает пересылка сообщений и интроспекция.
- **Foundation** (главы 15–21): строки, числа, коллекции, даты и данные, работа с файлами, KVC/KVO, многопоточность.
- **Современный Objective-C и Swift** (главы 22–23): то, как язык выглядит сегодня, и как он стыкуется со Swift. Глава 24 подскажет, куда двигаться дальше.

В приложениях — иерархия классов Foundation одним деревом (A), шпаргалка по синтаксису (B), подготовка к собеседованиям (C–F) и сравнение Objective-C с C++ (G).

Как устроена каждая глава

Мы не пересказываем справочник. Мы **строим**. В каждой главе:

1. ставим задачу простыми словами;
2. пишем самую наивную версию программы — пусть кривую;
3. компилируем и **смотрим, что реально вывелось**;
4. находим, что не так;
5. чиним и запускаем снова.

Ты будешь видеть программу в движении после каждого шага, а не получать готовый листинг «с неба». После каждого куска кода идёт разбор: что делает каждая новая строка, каждый знаковый знак, каждый метод.

В конце глав, где это уместно, стоит раздел «Документация Apple» со ссылками на официальные страницы. Привыкай в них заглядывать: всё в этой книге выверено по документации Apple, и тебе полезно уметь читать первоисточник.

Что тебе понадобится

- **Мас.** Objective-C и библиотека Foundation роднее всего чувствуют себя на macOS, и весь код книги рассчитан на неё.
- **Командная строка и компилятор clang.** Их ставят Xcode Command Line Tools — как именно, разберём в главе 1.
- **Любой текстовый редактор.** Xcode не обязателен: мы компилируем из терминала одной командой, чтобы видеть механику без лишней обвязки.

Готов? В главе 1 поставим инструменты и напишем первую программу — а заодно разберём в ней каждый символ.

Глава 1. Инструменты и первая программа

Прежде чем писать код, нужно, чтобы компьютер умел его переводить в работающую программу. В этой главе поставим инструменты, напомним первую программу на Objective-C, запустим её — и разберём в ней каждый символ. Не пропускай разбор: те шесть строк, что мы напишем, содержат почти все кирпичики, из которых сложена вся книга.

Что мы сделаем

- Поставим компилятор `clang` через Xcode Command Line Tools.
- Напишем программу, которая печатает строку.
- Скомпилируем её одной командой в терминале.
- Запустим и увидим вывод.
- Разберём каждую строку, каждый знак `@`, каждую скобку.

Ставим инструменты

Нам нужен компилятор — программа, которая превращает текст на Objective-C в исполняемый файл. На macOS его приносит набор **Xcode Command Line Tools**. Открой приложение «Терминал» и набери:

```
xcode-select --install
```

Появится окно с предложением установить инструменты — соглашайся и жди. Если они уже стоят, терминал скажет об этом — ничего страшного.

Проверим, что компилятор на месте:

```
clang --version
```

В ответ должно прийти что-то вроде `Apple clang version 21.0.0`. Точные цифры будут свои — важно, что команда отвечает, а не ругается «`command not found`».

`clang` — это и есть наш компилятор. Он понимает и Си, и Objective-C (второй ведь надстройка над первым).

Пишем первую программу

Создай файл `hello.m`. Расширение `.m` — это метка «здесь Objective-C» (по распространённому объяснению, буква `m` — от «`messages`», сообщения). Для сравнения: у чистого Си расширение `.c`. Набери в файле вот это:

```
#import <Foundation/Foundation.h>

int main(void) {
    @autoreleasepool {
        NSString *greeting = @"Привет, Objective-C!";
        NSLog(@"%@", greeting);
    }
    return 0;
}
```

Сохрани. Теперь скомпилируем.

Компилируем и запускаем

В терминале, в той папке, где лежит `hello.m`:

```
clang -fobjc-arc -framework Foundation hello.m -o hello
```

Разберём команду по частям:

- `clang` — зовём компилятор.
- `-fobjc-arc` — включаем **ARC** (Automatic Reference Counting, автоматический подсчёт ссылок). Это механизм, который сам следит за памятью объектов. Целая глава 13 посвящена тому, что он делает; пока просто включаем флаг и не думаем об освобождении памяти руками.
- `-framework Foundation` — подключаем библиотеку **Foundation**. В ней живут `NSString`, `NSLog` и сотни других базовых типов. Строго говоря, именно для Foundation современный `clang` справится и без флага (почему — разберём чуть ниже), но привычка писать его явно правильная: любой другой фреймворк без такой строки собран не будет.

- `hello.m` — наш исходный файл.
- `-o hello` — имя готовой программы (`-o` от *output*, вывод). Без него файл назвался бы `a.out`.

Если всё набрано верно, команда отработает молча — это хороший знак. Рядом появится файл `hello`. Запускаем его:

```
./hello
```

`./` означает «программа здесь, в текущей папке». Вывод:

```
2026-06-28 12:12:16.120 hello[60915:1250940] Привет, Objective-C!
```

Дата, время и числа в скобках будут свои — их подставляет `NSLog`. Нам важна последняя часть строки: `Привет, Objective-C!`. Программа работает.

Разбираем каждую строку

Теперь — самое ценное. Пройдём по программе сверху вниз.

`#import <Foundation/Foundation.h>`

Это **директива препроцессора**: она срабатывает до компиляции и подставляет в наш файл содержимое заголовочного файла `Foundation` — объявления всех его типов и функций. Без этой строки компилятор не знал бы, что такое `NSString` и `NSLog`.

Отличие от Си. В Си для подключения заголовков пишут `#include`. В Objective-C принят `#import` — он делает то же самое, но гарантирует, что один и тот же заголовок не вставится дважды. В Си от повторного включения защищаются вручную (через `#ifndef-обёртки`); `#import` снимает эту заботу.

Угловые скобки `< >` означают «ищи заголовок среди системных библиотек» (в отличие от `"..."` — «ищи рядом с моим файлом»).

`int main(void)`

Это **главная функция** — точка входа. С неё начинается выполнение любой программы. Слово `int` перед именем означает, что функция вернёт системе целое число — код завершения (0 = «всё хорошо»). `void` в скобках значит «функция не принимает аргументов». Фигурные скобки `{ }` обрамляют её тело — всё, что программа делает.

Отличие от Си. Здесь отличий нет вовсе: `int main(void)` — это чистый Си. Objective-C ничего не меняет в том, как программа запускается. Объектная надстройка начинается внутри тела.

```
@autoreleasepool { ... }
```

Первый чисто objc-элемент. Знак @ (мы будем встречать его постоянно) — это пометка «дальше идёт что-то из Objective-C, а не из Си». @autoreleasepool создаёт **пул автоосвобождения** — служебную область, в которой временные объекты дожидаются, пока их уберут из памяти. Пока прими как обязательную обёртку вокруг работы с объектами; полный смысл раскроется в главе 13, когда дойдём до памяти.

Отличие от Си. В Си такого блока нет и быть не может — там нет объектов, которыми надо управлять. @autoreleasepool существует только потому, что у Objective-C есть собственная модель памяти для объектов поверх си-шной.

```
NSString *greeting = @"Привет, Objective-C!";
```

Объявляем переменную greeting. Её тип — NSString *: указатель на объект-строку. **Звёздочка *** **обязательна**: в руках мы держим не сам объект, а указатель на него — адрес, по которому он лежит в памяти. Обычно объекты создаются в **куче** — общей области памяти, которую разберём в главе 3. Бывают и исключения: строковый литерал вроде нашего компилятор кладёт прямо внутрь готовой программы. Но и до него мы добираемся только по адресу, через *.

Справа — @"Привет, Objective-C!". Это **строковый литерал Objective-C**. Тот же знак @ превращает обычный текст в готовый объект NSString.

Отличие от Си. В Си строка — это char *s = "привет";, то есть просто массив байтов, заканчивающийся нулём. Никаких методов у неё нет. В Objective-C @"привет" — это полноценный объект NSString, который умеет сравнивать себя с другими, считать длину, переводить регистр и ещё сотню вещей. Маленький знак @ превращает мёртвый массив байтов в живой объект. Разницу мы подробно разберём в главе 15.

Точка с запятой ; в конце — конец инструкции. Она обязательна, как и в Си; забыть её — самая частая ошибка новичка.

```
NSLog(@"%@", greeting);
```

NSLog — функция Foundation для печати в журнал (лог). Первый аргумент — **строка формата @"%@"**. Внутри неё %@ — это «подставь сюда объект». Второй аргумент, greeting, и есть тот объект, который встанет на место %@. На экран попадёт его текстовое представление.

Отличие от Си. В Си печатают через printf("%s", s);, где %s ждёт си-строку (массив байтов). В Objective-C у NSLog есть свой спецификатор %@ — он умеет печатать любой объект, спрашивая у него самого, как тот хочет выглядеть. printf так не умеет. К тому же NSLog сам добавляет дату, имя программы и перевод строки в конце.


```
return 0;
```

Возвращаем системе 0 — «программа завершилась успешно». Это снова чистый Си, никакой объектной специфики.

Проверяем

Готовый файл главы — `code/01-hello.m`. Компиляция и запуск:

```
clang -fobjc-arc -framework Foundation code/01-hello.m -o hello
./hello
```

Ожидаемый вывод (дата и числа будут свои):

```
2026-06-28 12:12:16.120 hello[60915:1250940] Привет, Objective-C!
```

Частые ошибки — и что компилятор на них скажет

Ошибаться на первой программе нормально. Гораздо полезнее заранее услышать голос компилятора: тогда при встрече ты узнаешь сообщение, а не испугаешься его. Все тексты ниже — настоящие, я получил их, специально сломав `hello.m`.

Поставил `%s` вместо `%@`. Спецификатор `%s` ждёт си-строку — массив байтов, — а мы даём ему объект. Хорошая новость: `clang` это ловит ещё до запуска и даже подсказывает замену:

```
warning: format specifies type 'char *' but the argument has type
      'NSString *' [-Wformat]
NSLog(@"%s", greeting);
      ~~  ^
      %@
```

Это предупреждение, а не ошибка: программа всё равно соберётся и запустится. И вот тогда становится по-настоящему интересно — `%s` возьмёт указатель на объект и начнёт читать байты по этому адресу, как будто там текст:

```
2026-07-30 17:35:48.467 hello[66112:10996295] @jYXш
```

Не «пусто», не «ошибка», а мусор, который у тебя будет свой. Запомни эту картинку: если в логе вылезла абракадабра — почти наверняка объект попал в `%s`. Для объектов всегда `%@`.

Забыл `@` перед строкой. Тут никакого «предупреждения о несовпадении типов» — `clang` считает это прямой ошибкой и не собирает файл:

```
error: string literal must be prefixed by '@'
    NSString *greeting = "Привет, Objective-C!";
                          ^
                          @
```

Обрати внимание на подсказку под стрелкой: компилятор показывает ровно тот символ, которого не хватает. Для строк-объектов всегда @"...".

Потерял точку с запятой. Старая программистская примета «компилятор показывает на строку после пропущенной» для современного clang уже неверна — он указывает точно на конец той строки, где ; не хватило:

```
error: expected ';' at end of declaration
    NSString *greeting = @"Привет, Objective-C!"
                                   ^
                                   ;
```

Запускаешь без ./. Просто hello система в текущей папке искать не станет — она ходит только по каталогам из переменной PATH. Нужно ./hello.

А вот привычной ошибки «забыл -framework Foundation» ты, скорее всего, не увидишь. Убери флаг — и программа всё равно соберётся и запустится. Причина в том, что clang — не один только компилятор, а ещё и «водитель» (driver): он сам собирает команду для компоновщика. И когда среди входных файлов есть .m, то есть код на Objective-C, он добавляет в эту команду -framework Foundation и -lobjc за тебя. Убедиться можно флагом -### — он показывает, что clang собирается запустить, ничего при этом не собирая:

```
clang -fobjc-arc -### hello.m -o hello
```

В длинной простыне вывода найди строчки "-framework" "Foundation" и "-lobjc" — мы их не писали, а они там есть.

Второй способ проверить — посмотреть на **готовую** программу командой otool -L: она показывает, к каким библиотекам та привязана:

```
hello:
    /System/Library/Frameworks/Foundation.framework/Versions/C/Foundation
    /usr/lib/libobjc.A.dylib
    /usr/lib/libSystem.B.dylib
    /System/Library/Frameworks/CoreFoundation.framework/Versions/A/CoreFoundation
```

Четыре строки — и Foundation среди них, хотя флага мы не писали. Почему же тогда мы его пишем? Во-первых, чтобы намерение было видно в самой команде: читающий сразу понимает,

на чём стоит программа. Во-вторых, это любезность конкретного компилятора на конкретной системе, а не закон природы. И в-третьих — главное: стоит подключить любой **другой** фреймворк (AppKit, CoreData, AVFoundation), и его `-framework` придётся написать руками, иначе компоновщик выдаст ошибку вида `Undefined symbols`. Проще один раз завести привычку писать флаг всегда.

Упражнения

1. Поменяй текст приветствия на своё имя и пересобери программу.
2. Добавь вторую переменную `NSString *who = @"мир";` и выведи обе строки двумя вызовами `NSLog`.
3. Убери флаг `-framework Foundation` и убедись, что программа всё равно собралась. Потом запусти `otool -L` на обоих вариантах — с флагом и без — и сравни списки библиотек. Они должны совпасть.
4. Намеренно убери точку с запятой и посмотри, как звучит ошибка компилятора — потом тебе будет легче её узнавать.
5. Замени `%@` на `%s` и запусти. Свой мусор в логе не пугайся — это ровно тот случай, который мы разобрали выше.

Что мы получили

Инструменты стоят, первая программа собрана и запущена, и ты уже встретил почти весь алфавит будущих глав: `#import`, `@autoreleasepool`, `NSString *`, литерал `@"..."`, `NSLog` с `%@`. Дальше — короткий, но важный заход в базу самого Си: переменные, типы, ветвления и циклы. Без них объектная часть не на что опереть. С главы 5 начнём строить объекты.

Документация Apple

- Programming with Objective-C (общее введение в язык) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/Introduction/Introduction.html
- NSLog — developer.apple.com/documentation/foundation/nslog
- NSString — developer.apple.com/documentation/foundation/nsstring

Глава 2. База Си: переменные, типы, ветвления, циклы

В главе 1 ты собрал и запустил первую программу на Objective-C. В ней уже мелькали штуки из объектного мира — `NSString`, `NSLog`, знак `@`. Но под всем этим лежит обычный Си, и без него дальше двигаться нельзя: Objective-C — надстройка над Си, а не замена ему. Поэтому три главы подряд (2, 3, 4) мы потратим на сам Си. Эта — про самую основу: переменные, числа, выбор «если — иначе», повторения и функции.

Здесь почти всё будет чистым Си, и компилировать мы будем файлы с расширением `.c`, а не `.m`. Зато по краям я буду ставить врезки **«Отличие от Си»** — они смотрят вперёд и показывают, во что та или иная конструкция превратится, когда мы дойдём до объектов. Так база и объектная часть сразу свяжутся в голове.

Что мы сделаем

- Заведём переменные разных типов и научимся их печатать через `printf`.
- Посчитаем что-нибудь арифметикой и поймем классическую ловушку целочисленного деления.
- Прочитаем число с клавиатуры через `scanf`.
- Научим программу принимать решения — `if / else`.
- Заставим её повторять работу — циклы `while` и `for`.
- Вынесем кусок логики в собственную функцию.

Каждый шаг — компилируем и смотрим реальный вывод. Поехали.

Переменные и типы

Переменная — это коробка с именем, в которой лежит значение. Чтобы завести коробку, в Си надо сразу сказать, *что* в ней будет лежать — указать **тип**. Тип определяет, какие значения влезут и сколько места займут.

Четыре типа, которых пока хватит на всё:

- `int` — целое число: `-3`, `0`, `42`. Без дробной части.
- `double` — число с дробной частью: `3.14`, `199.90`. («double» — потому что под него отводится двойная точность.)
- `char` — один символ: `'A'`, `'?'`. В одинарных кавычках.
- `_Bool` — логическое: правда или ложь. Если в начале файла написать `#include <stdbool.h>`, появятся понятные имена `bool`, `true`, `false`.

Заведём по одной коробке каждого типа. Создай файл `02-vars.c`:

```
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    int age = 19;
    double price = 199.90;
    char grade = 'A';
    bool isStudent = true;

    printf("Возраст: %d\n", age);
    printf("Цена: %f\n", price);
    printf("Цена (2 знака): %.2f\n", price);
    printf("Оценка: %c\n", grade);
    printf("Студент: %d\n", isStudent);

    age = age + 1;
    printf("Через год: %d\n", age);

    return 0;
}
```

Разберём построчно.

`#include <stdio.h>` — подключаем заголовок «standard input/output». Оттуда берётся `printf` (печать на экран). Без него компилятор не знал бы, что это за функция. `#include <stdbool.h>` даёт слова `bool`, `true`, `false`.

Отличие от Си. В Си подключают заголовки через `#include`, как здесь. В Objective-C ты уже видел `#import <Foundation/Foundation.h>` — он делает то же самое, но не вставит один и тот же заголовок дважды. Сейчас, в чистом Си, остаёмся на `#include`.

`int main(void)` — главная функция, точка входа. Ровно та же, что в главе 1: тут отличий между Си и Objective-C нет вообще.

`int age = 19;` — заводим коробку `age` типа `int` и кладём в неё 19. Знак `=` здесь — **присваивание**, «положи правое в левое». Это не равенство из математики! Запомни сразу, потому что чуть позже встретится `==` — а вот это уже сравнение. Перепутать их — ошибка номер один у новичков.

`char grade = 'A';` — символ пишем в **одинарных** кавычках. Двойные кавычки `"A"` в Си означают совсем другое — строку (про них в главе 4).

Точка с запятой `;` в конце каждой инструкции обязательна.

Печать через `printf` и спецификаторы

`printf` печатает текст. Внутри строки прячутся **спецификаторы** — значки со знаком `%`, на место которых подставляются значения:

- `%d` — целое (`int`),
- `%f` — дробное (`double`),
- `%c` — один символ (`char`),
- `\n` — перевод строки (без него всё слиплось бы в одну строчку).

`printf("Возраст: %d\n", age);` — берёт `age`, подставляет вместо `%d`, переводит строку. `%.2f` — то же `%f`, но округлить до 2 знаков после точки.

`age = age + 1;` — читаем правую часть (`age + 1 = 19 + 1 = 20`) и кладём результат обратно в `age`. Теперь там 20.

Компилируем и запускаем. Команда — без `-framework Foundation`, он тут не нужен, это чистый Си:

```
clang -std=c11 -Wall -Wextra -O2 code/02-vars.c -o vars
./vars
```

Вывод:

```
Возраст: 19
Цена: 199.900000
Цена (2 знака): 199.90
Оценка: A
Студент: 1
Через год: 20
```

Обрати внимание на две вещи. `%f` без уточнения печатает кучу нулей — `199.900000`; для денег почти всегда хочется `%.2f`. А `bool` печатается как `1` (правда) или `0` (ложь) — отдельного «`true/false`» у `printf` нет.

Отличие от Си. `printf` печатает только примитивы — числа, символы, си-строки. Объект ему не скормишь. В Objective-C для печати есть `NSLog` со спецификатором `%@` — он умеет печатать любой объект, спрашивая у него самого, как тот выглядит (ты видел `NSLog(@"%@", ...)` в главе 1). Грубо: `%d/%f/%c` — мир Си, `%@` — мир объектов.

Арифметика и ловушка целочисленного деления

Складывать, вычитать и умножать Си умеет ожидаемо: `+`, `-`, `*`. А вот с делением есть подвох, на котором спотыкаются все. Разберём его методом «сначала наивно, потом чиним».

Задача: 7 яблок разделить на 2 человек. Руками понятно — по 3,5. Напишем в лоб. Файл `02-divide.c`, наивная первая версия:

```
#include <stdio.h>

int main(void) {
    int apples = 7;
    int people = 2;

    int bad = apples / people;
    printf("Целочисленно: 7 / 2 = %d\n", bad);

    return 0;
}
```

Запускаем:

```
Целочисленно: 7 / 2 = 3
```

Не 3,5, а **3**. Куда делась половина? Вот в чём дело: когда `/` делит **целое на целое**, результат тоже целое — дробная часть просто отбрасывается (не округляется, а отрезается). `7 / 2` даёт 3 и теряет остаток. Это и есть **целочисленное деление**.

Остаток, кстати, можно достать отдельным оператором `%` (он так и зовётся — «остаток от деления», или «модуло»):

```
int rest = apples % people;
printf("Остаток: 7 %% 2 = %d\n", rest);
```

(Внутри строки формата сам знак процента пишется как %% — иначе printf примет его за начало спецификатора.)

Теперь чиним главную проблему. Чтобы деление было дробным, хотя бы одно из чисел должно быть double. Превратим apples в дробное прямо в выражении — это называется **приведение типа**, пишется типом в скобках перед значением: (double)apples. Полный 02-divide.c:

```
#include <stdio.h>

int main(void) {
    int apples = 7;
    int people = 2;

    int bad = apples / people;
    printf("Целочисленно: 7 / 2 = %d\n", bad);

    int rest = apples % people;
    printf("Остаток: 7 %% 2 = %d\n", rest);

    double good = (double)apples / people;
    printf("Дробно: 7 / 2 = %f\n", good);
    printf("Дробно (2 знака): %.2f\n", good);

    return 0;
}
```

(double)apples говорит: «возьми apples и в этом выражении считай его дробным». Дальше double / int уже даёт double, и половина не теряется. Запускаем:

```
Целочисленно: 7 / 2 = 3
Остаток: 7 % 2 = 1
Дробно: 7 / 2 = 3.500000
Дробно (2 знака): 3.50
```

Запомни правило: **int / int — всегда целое**. Хочешь дробь — сделай дробным хотя бы один операнд. Эта ловушка будет преследовать тебя и в Objective-C, и в любом си-подобном языке.

Ввод с клавиатуры через scanf

Пока все числа были вшиты в код. Дадим программе прочитать число от пользователя. За ввод отвечает scanf — зеркало printf: тот печатает, этот читает. Файл 02-input.c:


```
#include <stdio.h>

int main(void) {
    int age = 0;

    printf("Сколько тебе лет? ");

    int got = scanf("%d", &age);
    if (got != 1) {
        printf("Это не число.\n");
        return 1;
    }

    printf("Через 10 лет тебе будет %d\n", age + 10);
    return 0;
}
```

Главная новая деталь — `scanf("%d", &age)`. Спецификатор `%d` тот же, что у `printf`: «жду целое». А вот знак `&` перед `age` обязателен и важен.

`&age` — это **адрес** переменной `age`, то есть «где в памяти лежит её коробка». `scanf` нужно не значение `age` (оно ему ни к чему — он сам его впишет), а место, куда положить прочитанное. `&` и означает «вот сюда». Забудешь `&` — программа, скорее всего, упадёт. Подробно адреса и знак `&` разберём в главе 3 про указатели; пока запомни правило: **в `scanf` перед переменной ставь `&`**.

`scanf` возвращает число успешно прочитанных значений. Мы просили одно — значит, при удаче вернётся 1. Поэтому проверяем: `if (got != 1)` — «если прочиталось не одно значение» (например, пользователь ввёл буквы) — сообщаем об ошибке и выходим с кодом 1 («что-то пошло не так»). Это хорошая привычка: всегда проверять, что ввод реально прочитался.

Запускаем и вводим 25:

```
Сколько тебе лет? 25
Через 10 лет тебе будет 35
```

А если ввести буквы вместо числа, например `abc`:

```
Сколько тебе лет? abc
Это не число.
```

Проверка сработала — программа не стала считать мусор.

Ветвления: if и else

Программа становится интересной, когда умеет **выбирать** путь. За это отвечает if («если»). Поставим оценку по баллу. Файл 02-branch.c:

```
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    int score = 75;

    if (score >= 90) {
        printf("Отлично\n");
    } else if (score >= 60) {
        printf("Зачёт\n");
    } else {
        printf("Пересдача\n");
    }
}
```

Читается почти как по-русски. if (score >= 90) — «если score больше или равно 90, выполни блок в фигурных скобках». Если условие ложно, проверяется следующий else if, и так далее. else без условия — «во всех остальных случаях».

Операторы **сравнения** — те самые, что дают «правду» или «ложь»:

- > больше, < меньше,
- >= больше или равно, <= меньше или равно,
- == равно, != не равно.

Вот оно, обещанное ==. Сравнение — это **два** знака равенства. Один знак = — присваивание (положить значение). Их легко перепутать, и компилятор с -Wall обычно предупредит, если в if затесалось одиночное =.

Условия можно соединять **логическими** операторами:

- && — «И»: истинно, когда истинны оба условия,
- || — «ИЛИ»: истинно, когда истинно хотя бы одно,
- ! — «НЕ»: переворачивает значение наоборот.

Допишем в тот же файл:

```
int age = 20;
bool hasTicket = true;
if (age >= 18 && hasTicket) {
```

```
    printf("Проходи\n");
}

if (!hasTicket || age < 18) {
    printf("Стоп\n");
} else {
    printf("Всё в порядке\n");
}

return 0;
}
```

`age >= 18 && hasTicket` — «совершеннолетний И с билетом». Оба верны (20 и true), значит печатается «Проходи». `!hasTicket || age < 18` — «нет билета ИЛИ младше 18»; оба ложны, поэтому срабатывает `else` со «Всё в порядке». Запускаем весь `02-branch.c`:

```
Зачёт
Проходи
Всё в порядке
```

`score` равен 75 — не дотянул до 90, но прошёл порог 60, отсюда «Зачёт».

Отличие от Си. Сравнение `==` отлично работает для чисел: `5 == 5` истинно. Но для **объектов** в Objective-C `==` сравнивает не содержимое, а адреса — «это буквально один и тот же объект в памяти?». Чтобы сравнить две строки *по тексту*, нужен метод `[a isEqualToString:b]`, а не `a == b`. Это классическая ловушка, и мы вернёмся к ней в главе 15 про `NSString`. Пока — для чисел `==` честно сравнивает значения.

Циклы: `while` и `for`

Когда работу надо повторять, на помощь приходят **циклы**. Посчитаем сумму чисел от 1 до 5 двумя способами. Файл `02-loops.c`.

Сначала `while` — «пока условие истинно, повторяй»:

```
#include <stdio.h>

int main(void) {
    int n = 5;

    int sum = 0;
```

```
int i = 1;
while (i <= n) {
    sum = sum + i;
    i = i + 1;
}
printf("Сумма от 1 до %d = %d\n", n, sum);
```

Заводим счётчик $i = 1$ и копилку $sum = 0$. Пока $i \leq n$, прибавляем i к сумме и увеличиваем i на единицу. Как только i станет 6, условие $6 \leq 5$ ложно — цикл прекращается. Важнейшая деталь: **внутри цикла счётчик обязан меняться** ($i = i + 1$), иначе условие никогда не станет ложным и программа заикнется навсегда.

Тот же расчёт короче пишется через `for`. Он собирает три части цикла в одну строку:

```
int sum2 = 0;
for (int k = 1; k <= n; k++) {
    sum2 = sum2 + k;
}
printf("То же через for = %d\n", sum2);
```

В скобках `for` три части через `;`:

1. `int k = 1` — что сделать **один раз** в начале (завести счётчик),
2. `k <= n` — условие, проверяется **перед каждым** проходом,
3. `k++` — что сделать **после каждого** прохода.

`k++` — это сокращение для `k = k + 1`, «увеличь на единицу». Очень частая запись; есть и брат `k--` — «уменьши на единицу». `for` удобен, когда заранее знаешь число повторений; `while` — когда повторяешь «пока что-то держится».

break и continue

Двумя словами можно управлять циклом изнутри:

- `break` — немедленно **выйти** из цикла целиком,
- `continue` — **пропустить остаток** текущего прохода и перейти к следующему.

Покажем оба — печатаем нечётные числа, но останавливаемся после 7:

```
for (int x = 1; x <= 10; x++) {
    if (x % 2 == 0) {
        continue; /* чётные пропускаем */
    }
    if (x > 7) {
```

```
        break;      /* после 7 выходим совсем */
    }
    printf("нечётное: %d\n", x);
}

return 0;
}
```

$x \% 2 == 0$ — «остаток от деления на 2 равен нулю», то есть число чётное; для него `continue` перескакивает к следующему `x`, печать не выполняется. Как только `x` переваливает за 7, `break` обрывает цикл. Текст в `/* ... */` — это **комментарий**, компилятор его игнорирует, он для человека. Запускаем весь `02-loops.c`:

```
Сумма от 1 до 5 = 15
То же через for = 15
нечётное: 1
нечётное: 3
нечётное: 5
нечётное: 7
```

Оба способа дали 15 ($1+2+3+4+5$). А нечётные напечатались только до 7 — `break` сделал своё дело.

Функции: выносим логику

Когда кусок логики хочется назвать и переиспользовать, его выносят в **функцию**. Функцию ты уже видел — `main` тоже функция. Заведём свои. Файл `02-functions.c`:

```
#include <stdio.h>

/* прототип: обещаем компилятору, что функция есть */
int square(int x);
int max2(int a, int b);

int main(void) {
    printf("square(5) = %d\n", square(5));
    printf("max2(3, 9) = %d\n", max2(3, 9));
    return 0;
}
```

```
/* определение: тело функции */
int square(int x) {
    return x * x;
}

int max2(int a, int b) {
    if (a > b) {
        return a;
    }
    return b;
}
```

Разберём по частям, потому что в одной строке функции много смысла.

Строка `int square(int x);` — это **прототип** (объявление). Она заканчивается точкой с запятой и не содержит тела. Прототип нужен, чтобы к моменту вызова в `main` компилятор уже знал: есть функция `square`, принимает один `int`, возвращает `int`. Прочитаем заголовок слева направо: `int` — тип возвращаемого значения; `square` — имя; в скобках `(int x)` — **параметр**: переменная `x` типа `int`, которую функция получит при вызове.

Ниже, после `main`, идёт **определение** — тот же заголовок, но с телом в фигурных скобках. `return x * x;` вычисляет квадрат и **возвращает** его наружу: слово `return` отдаёт значение тому, кто функцию вызвал, и тут же завершает её.

В `max2` два параметра, `a` и `b`. Внутри — знакомый `if`: если `a` больше, возвращаем `a`; иначе исполнение дойдёт до `return b`. Заметь — у функции может быть несколько `return`; срабатывает первый достигнутый.

Вызов выглядит как `square(5)`: пишем имя и в скобках передаём **аргумент** 5. Он попадёт в параметр `x`, функция отработает и вернёт 25, которое `printf` и напечатает. Запускаем:

```
square(5) = 25
max2(3, 9) = 9
```

Отличие от Си. `square(5)` — это **прямой вызов функции**: какой код выполнить, компилятор знает уже на этапе сборки. В Objective-C ты будешь писать иначе — `[obj square:5]`, в квадратных скобках. Это не вызов, а **отправка сообщения**: объекту `obj` шлётся сообщение `square:`, а нужный метод runtime ищет уже *во время выполнения*. Эта разница — сердце всего языка, ей посвящена глава 5. Пока запомни картинку: круглые скобки `foo(x)` — вызов функции Си; квадратные `[obj foo:x]` — сообщение объекту.

Проверяем

Все файлы главы лежат в `code/`. Каждый собирается одной командой (это чистый Си, флаг `-framework Foundation` не нужен):

```
clang -std=c11 -Wall -Wextra -O2 code/02-vars.c -o prog && ./prog
clang -std=c11 -Wall -Wextra -O2 code/02-divide.c -o prog && ./prog
clang -std=c11 -Wall -Wextra -O2 code/02-input.c -o prog && ./prog
clang -std=c11 -Wall -Wextra -O2 code/02-branch.c -o prog && ./prog
clang -std=c11 -Wall -Wextra -O2 code/02-loops.c -o prog && ./prog
clang -std=c11 -Wall -Wextra -O2 code/02-functions.c -o prog && ./prog
```

Флаги: `-std=c11` — версия языка Си, `-Wall -Wextra` — включить все предупреждения (пусть компилятор подсказывает), `-O2` — оптимизация. Всё должно собираться **без единого предупреждения**.

Частые ошибки

Две самые коварные ошибки этой главы компилятор ловит сам — но только если ты собираешь с `-Wall -Wextra`, как мы и договорились. Вот что он скажет.

= вместо == в условии. `if (x = 5)` ничего не сравнивает: оно кладёт в `x` пятёрку, а результатом присваивания становится сама пятёрка — то есть «истина». Условие всегда срабатывает, и ветка `else` никогда не выполняется. Clang подозревает неладное и даже предлагает оба возможных исправления:

```
warning: using the result of an assignment as a condition without
      parentheses [-Wparentheses]
      if (x = 5) {
          ~~~~
note: place parentheses around the assignment to silence this warning
note: use '==' to turn this assignment into an equality comparison
```

Второе примечание — про нас: почти всегда имелось в виду `==`. Первое пригодится в редком случае, когда присваивание в условии написано намеренно; тогда его оборачивают в лишние скобки — `if ((x = 5))`, — и компилятор понимает, что это не описка.

Забывтый & в scanf. `scanf("%d", age)` передаёт функции само значение `age`, а не адрес коробки. `scanf` примет это число за адрес и попытается писать по нему — с почти гарантированным падением. Компилятор предупреждает и тут:

```
warning: format specifies type 'int *' but the argument has type 'int'
    [-Wformat]
    scanf("%d", age);
           ~ ~ ^ ~
```

В `scanf` перед переменной всегда `&`.

Остальные грабли компилятор не увидит — их придётся ловить головой.

Целочисленное деление. `int avg = sum / count;` режет дробь молча. Нужна дробь — сделай операнд дробным: `(double)sum / count`.

Бесконечный цикл. В `while` забыл менять счётчик (`i = i + 1`) — условие никогда не станет ложным, программа висит. Прерывается `Ctrl+C`.

Перепутал кавычки. `'A'` (одинарные) — символ `char`. `"A"` (двойные) — строка. Для `char` нужны одинарные.

Потерял ;. Старый совет «смотри на строку выше той, где ругается компилятор» современному `clang` уже не нужен: он показывает точно то место, где точки с запятой не хватило, и рисует её под стрелкой:

```
error: expected ';' at end of declaration
    int age = 19
                ^
                ;
```

Упражнения

1. Заведи две переменные `double` (рост в метрах и вес в килограммах) и напечатай их через `%.2f`. Посчитай ИМТ (`вес / рост2`) и выведи результат.
2. Перепиши `02-divide.c` так, чтобы числа `apples` и `people` читались с клавиатуры через `scanf`. Не забудь `&` и проверку возврата.
3. Напиши цикл `for`, печатающий таблицу умножения на 7 (от `7×1` до `7×10`).
4. Сделай функцию `int isEven(int n)`, которая возвращает 1 для чётных и 0 для нечётных (подсказка: `n % 2`). Проверь её в цикле для чисел от 1 до 10.
5. Через `while` найди наименьшее число, квадрат которого больше 1000.
6. Перепиши пример с оценкой (`02-branch.c`) так, чтобы балл вводился через `scanf`, и добавь порог `>= 75` → «Хорошо».

Что мы получили

Теперь у тебя есть вся процедурная база Си: переменные и типы, арифметика с её ловушкой целочисленного деления, печать через `printf` и ввод через `scanf` со знаком `&`, ветвления `if/else` с операторами сравнения и логики, циклы `while/for` с `break/continue` и собственные функции с прототипами и `return`. Этого хватит, чтобы писать осмысленные программы — и этого же хватит, чтобы понять объектную часть, когда мы до неё дойдём.

Попутно мы не раз заглянули вперёд: `#include` против `#import`, `printf` против `NSLog` с `%@`, `==` против `isEqualToString:`, вызов `foo(x)` против сообщения `[obj foo:x]`. Держи эти контрасты в голове — они вернутся.

В главе 3 разберём то, на чём в Си держится буквально всё, включая объекты Objective-C, — **указатели и память**. Тот самый знак `&` из `scanf` и звёздочка `*` из `NSString *` оттуда родом.

Документация Apple

Эта глава — чистый Си, и собственно Apple-терминов здесь почти нет: они начнут появляться с главы 5, когда мы войдём в объектный мир. Если захочется заглянуть в первоисточник по самому языку Си:

- Справочник по стандартной библиотеке Си (`printf`, `scanf`, типы) — en.cppreference.com/w/c (раздел `C language` и `C standard library`).
- Документация компилятора Apple `clang` — clang.llvm.org/docs/ и developer.apple.com/documentation/xcode (Xcode и Command Line Tools).
- `Programming with Objective-C` — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/Introduction/Introduction.html: пригодится со следующих глав, когда поверх этой базы Си появятся объекты.

Глава 3. Указатели и память

Это самая важная глава из базовых. Если её понять — почти всё странное в Objective-C перестанет быть странным. Помнишь, в первой программе мы писали `NSString *greeting` со звёздочкой и я обещал объяснить, откуда она? Объяснение — здесь. Звёздочка — это **указатель**, и чтобы понять, почему объект в Objective-C всегда держат через указатель, надо сначала разобраться, что такое память и адрес.

Пойдём с самого низа: что такое адрес, что такое указатель, зачем он нужен, как с его помощью функция меняет чужую переменную, где живут данные (стек и куча) и что такое `struct`. Всё на чистом Си — но в конце каждого куска покажу, как это же отзовётся в объектной части книги.

Что мы сделаем

- Поймём, что у каждой переменной есть **адрес** в памяти, и научимся его брать оператором `&`.
- Заведём **указатель** — переменную, которая хранит адрес, — и пройдем по нему туда-обратно оператором `*`.
- Напишем функцию `swap`, которая сначала **не работает**, и починим её указателями.
- Разберём **стек** и **кучу**, возьмём память руками через `malloc` и вернём через `free`.
- Соберём свой составной тип через `struct` и сходим к его полям через `.` и `->`.

Память — это длинная полка с ячейками

Представь оперативную память компьютера как очень длинную полку, разбитую на пронумерованные ячейки. Каждая ячейка хранит один байт, у каждой — свой номер: первая, вторая, миллионная. Этот номер и называется **адресом** (address).

Когда ты пишешь `int age = 19;`, программа находит на полке свободное место под целое число, кладёт туда 19 и запоминает, по какому адресу оно лежит. Имя `age` — просто удобная бирка для тебя; машина работает с адресом.

Обычно адрес знать не нужно. Но иногда он нужен напрямую: чтобы дать функции право поменять твою переменную, чтобы взять память под данные неизвестного заранее размера, чтобы передать большой объект, не копируя его целиком. Для всего этого в Си есть две операции и один тип. С них и начнём.

Шаг 1. Берём адрес переменной — оператор &

Оператор & перед именем переменной читается «адрес такой-то». Он возвращает не значение переменной, а номер ячейки, где она лежит.

Напечатать адрес можно через `printf` со спецификатором `%p` (от *pointer*, указатель). Перепишем знакомый `printf` так, чтобы он показал и значение, и адрес:

```
int age = 19;
printf("Значение age:   %d\n", age);
printf("Адрес age:      %p\n", (void *)&age);
```

`%d` печатает значение 19, а `%p` — адрес `&age`. Запись `(void *)` перед `&age` — это **приведение типа**: `%p` хочет получить «адрес вообще, без уточнения, на что он смотрит», и такой обобщённый адрес в Си обозначается типом `void *`. Так требует стандарт Си. Clang с нашими флагами, правда, промолчит и без приведения (ругнётся только в строгом режиме `-Wformat-pedantic`), но правильно писать с ним. Пока прими `(void *)` как обязательный ритуал при печати адресов через `%p`.

Запустим (адрес у тебя будет свой — об этом ниже):

```
Значение age:   19
Адрес age:      0x7ff7b58e0aa4
```

Адрес печатается в шестнадцатеричном виде, с приставкой `0x`. Это просто большое число — номер ячейки. У тебя при каждом запуске оно будет **другим**: система кладёт переменные в разные места ради безопасности. Конкретное число запоминать бессмысленно — важно, что оно есть.

Шаг 2. Указатель — переменная для адреса

Адрес — это число, и его можно сохранить. Переменная, которая хранит адрес, называется **указателем** (*pointer*). Объявляется так:

```
int *p = &age;
```

Разберём по знакам, потому что здесь кроется главная путаница новичка.

- `int *p` — объявляем переменную `p`. Звёздочка `*` в объявлении значит «`p` — это **указатель на `int`**». То есть `p` хранит не само число, а адрес ячейки, где лежит `int`.
- `= &age` — кладём в `p` адрес переменной `age`. Типы сходятся: `&age` — это «адрес `int`», а `p` как раз хранит «адрес `int`».

Прочитать всю строку можно так: «`p` — это указатель на `int`, и сейчас он смотрит на `age`».

Важно сразу зафиксировать: звёздочка в объявлении `int *p` — это **часть типа**, она говорит «здесь будет адрес». Это не умножение и не действие. Через секунду мы встретим ту же звёздочку в совсем другом смысле — и надо их не перепутать.

```
printf("Значение p:      %p\n", (void *)p);
```

`p` сам по себе — это адрес, поэтому печатаем его через `%p`. И он совпадёт с адресом `age`, ведь именно туда `p` и смотрит:

```
Значение p:      0x7ff7b58e0aa4
```

Шаг 3. Разыменование — та же звёздочка, другой смысл

У нас есть `p` — адрес ячейки. Как добраться до самого числа в этой ячейке? Поставить перед указателем звёздочку:

```
printf("Куда смотрит p: %d\n", *p);
```

`*p` читается «значение по адресу, который лежит в `p`». Эта операция называется **разыменование** (dereference): «пойди по адресу и возьми то, что там лежит». Результат — обычный `int`, поэтому печатаем через `%d`:

```
Куда смотрит p: 19
```

Вот и обещанная развилка. Звёздочка встречается в двух разных ролях, и различать их — половина успеха:

```
int *p = &age;    ← звёздочка в ОБЪЯВЛЕНИИ: «p — указатель на int»
*p = 25;          ← звёздочка в ВЫРАЖЕНИИ: «пойди по адресу p и положи туда»
```

Простое правило: если звёздочка стоит там, где объявляется переменная (рядом с типом) — это «тип-указатель». Если звёздочка стоит в обычном действии над уже существующим указателем — это «сходи по адресу».

Проверим вторую роль. Сейчас `p` смотрит на `age`. Запишем через `p` новое значение — и сама `age` изменится, хотя её имя мы не трогали:

```
*p = 25;
printf("Теперь age:      %d\n", age);
```

```
Теперь age:      25
```

*p = 25 означает «по адресу, на который смотрит p, положи 25». А по этому адресу лежит age. Мы поменяли переменную *через её адрес*. Это и есть сверхспособность указателей — и сейчас мы пустим её в дело.

Шаг 4. NULL — указатель в никуда

Указатель обязан куда-то смотреть. Но иногда он пока ни на что не должен смотреть — например, мы ещё не решили, на что. Для «пустого» указателя есть специальное значение **NULL**:

```
int *empty = NULL;
if (empty == NULL) {
    printf("empty пока никуда не смотрит\n");
}
```

NULL — это «адрес-ноль», заведомо невалидный, который означает «здесь ничего нет». Проверять указатель на NULL перед использованием — добрая привычка, и скоро мы увидим, зачем она нужна.

Самая страшная ошибка с указателями — разыменовывать тот, который никуда толком не смотрит. Если объявить `int *p`; и **не** присвоить ему адрес, в p окажется мусор — случайное число, оставшееся в ячейке. Команда `*p = 25` тогда запишет 25 по случайному адресу, и программа упадёт с `Segmentation fault` или незаметно испортит чужие данные. Поэтому правило: **указатель либо смотрит на реальную переменную, либо равен NULL** — третьего не дано.

Соберём всё в один файл `code/03-pointers.c` и запустим целиком:

```
#include <stdio.h>

int main(void) {
    int age = 19;

    printf("Значение age:      %d\n", age);
    printf("Адрес age:        %p\n", (void *)&age);

    int *p = &age;
    printf("Значение p:        %p\n", (void *)p);
    printf("Куда смотрит p:    %d\n", *p);
```

```
*p = 25;
printf("Теперь age:      %d\n", age);

int *empty = NULL;
if (empty == NULL) {
    printf("empty пока никуда не смотрит\n");
}

return 0;
}
```

Реальный вывод (адреса будут свои):

```
Значение age:      19
Адрес age:         0x7ff7b58e0aa4
Значение p:        0x7ff7b58e0aa4
Куда смотрит p:    19
Теперь age:        25
empty пока никуда не смотрит
```

Отличие от Си. Вот тут и кроется секрет звёздочки из первой программы. Когда ты пишешь `NSString *greeting`, ты заводишь **точно такой же указатель**, как `int *p`, только смотрит он не на число, а на объект-строку. Сам объект `NSString` лежит где-то в памяти, а в `greeting` хранится его адрес. Поэтому объект в Objective-C *всегда* держат через *: `NSString *`, `NSObject *`, `NSArray *`. Звёздочка там не каприз синтаксиса — это буквально «здесь адрес объекта». К механике объектов вернёмся в главе 5, к строкам — в главе 15.

Шаг 5. Функция, которая не работает: наивный swap

Возьмём классическую задачу — поменять местами значения двух переменных. «Руками» это так: завести временную копию, чтобы не потерять первое значение, и переложить по кругу. Запишем функцию в лоб:

```
void swapBad(int a, int b) {
    int tmp = a;
    a = b;
    b = tmp;
}
```

Логика верная: tmp запоминает a, в a кладём b, в b — спасённое tmp. Зовём её на $x = 1$ и $y = 2$:

```
int x = 1, y = 2;
printf("До:          x = %d, y = %d\n", x, y);
swapBad(x, y);
printf("После swapBad: x = %d, y = %d\n", x, y);
```

Запускаем — и вот сюрприз:

```
До:          x = 1, y = 2
После swapBad: x = 1, y = 2
```

Ничего не поменялось! Почему? Потому что в Си функция получает аргументы **по значению** — то есть копии. Когда мы зовём `swapBad(x, y)`, создаются две новые переменные `a` и `b`, в них копируются значения `x` и `y`. Внутри функции мы старательно меняем местами `a` и `b` — но это копии. Настоящие `x` и `y` в `main` функция даже не видит. Копии живут до конца функции и исчезают, а оригиналы остаются нетронутыми.

Шаг 6. Чиним swap указателями

Чтобы функция меняла **наши** переменные, дадим ей не копии значений, а их адреса. По адресу она доберётся до оригиналов и перепишет их:

```
void swapGood(int *a, int *b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
```

Читаем по знакам:

- `int *a, int *b` — функция принимает два **указателя на int**, то есть два адреса.
- `int tmp = *a` — разыменовываем `a`: берём значение, лежащее по этому адресу, и сохраняем в `tmp`.
- `*a = *b` — по адресу `a` кладём значение, которое лежит по адресу `b`.
- `*b = tmp` — по адресу `b` кладём спасённое значение.

Звёздочки здесь — это разыменование (роль из шага 3): мы ходим по адресам и переписываем чужие ячейки. Зовём функцию, передавая **адреса** переменных через `&`:

```
swapGood(&x, &y);
printf("После swapGood: x = %d, y = %d\n", x, y);
```

`&x` и `&y` — адреса наших переменных. Теперь функция знает, где лежат оригиналы. Обе функции и вызовы собраны в `code/03-swap.c`. Реальный вывод:

До: `x = 1, y = 2`

После `swapBad`: `x = 1, y = 2`

После `swapGood`: `x = 2, y = 1`

`swapBad` не сделал ничего, `swapGood` поменял значения местами. Разница — только в том, что одному дали копии, а другому адреса. Это главный приём указателей: **передаёшь адрес — даёшь функции право менять оригинал**.

Шаг 7. Где живут данные: стек и куча

Пора понять, откуда вообще берётся память под переменные. У программы её два больших источника.

Стек (stack) — память под локальные переменные функций. Работает как стопка тарелок: вызвали функцию — на стек лёг её «поднос» с переменными; функция закончилась — поднос убрали, переменные исчезли. Поэтому `a` и `b` внутри `swapBad` пропали, как только функция вернулась. Стек быстрый и автоматический, но переменная на нём живёт ровно столько, сколько работает её функция, и размер должен быть известен заранее.

Куча (heap) — большой общий склад, у которого память просишь сам, сколько надо, и сам же возвращаешь, когда закончил. Данные на куче живут, пока ты их не освободишь, — хоть после конца функции, которая их создала. За эту свободу ты платишь ответственностью: что взял с кучи, то обязан вернуть.

СТЕК (автоматически)	КУЧА (вручную)
main: x, y функция: a, b ← исчезнет	блок под 5 int ← malloc дал 10 20 30 40 50 ← free вернёт
живёт пока работает функция	живёт пока сам не освободишь

Шаг 8. Берём память руками: malloc, sizeof, free

Когда заранее не знаешь, сколько данных понадобится, память берут с кучи функцией `malloc` (от *memory allocate*, выделить память). Она живёт в заголовке `<stdlib.h>`. Возьмём место под пять чисел:

```
int count = 5;
int *arr = malloc(count * sizeof(int));
```

- `malloc(...)` принимает **число байтов** и возвращает адрес начала выделенного блока — то есть указатель. Мы сохраняем его в `arr`.
- `sizeof(int)` — оператор, который сообщает, сколько байтов занимает тип. У `int` это обычно 4 байта. Писать `sizeof(int)` вместо числа 4 правильнее: на разных машинах размер может отличаться, а `sizeof` всегда даст верный.
- `count * sizeof(int)` — значит «пять чисел по 4 байта = 20 байт».

`malloc` отдаёт «адрес вообще» (тип `void *`), а присваиваем мы его в `int *arr` — компилятор сам поймёт, что блок будем читать как числа.

Теперь — обязательная проверка. `malloc` может вернуть `NULL`, если памяти в системе не осталось. Разыменовывать `NULL` — это падение, поэтому проверяем сразу:

```
if (arr == NULL) {
    printf("Память не выделилась\n");
    return 1;
}
```

Вот зачем мы в шаге 4 познакомились с `NULL`: проверка `arr == NULL` ловит неудачу до того, как она обрушит программу. Если память дали, работаем с блоком как с массивом — через квадратные скобки. Массивы подробно разберём в главе 4, а пока достаточно знать: `arr[i]` — это ячейка номер `i` в блоке, причём счёт идёт с нуля (`arr[0]` — первая, `arr[4]` — пятая):

```
for (int i = 0; i < count; i++) {
    arr[i] = (i + 1) * 10;
}
```

Когда блок больше не нужен, его **обязательно** возвращают системе функцией `free`:

```
free(arr);
```

`free(arr)` говорит «этот кусок кучи мне больше не нужен, забирай». Если этого не сделать, блок останется занятым до конца программы, хотя им уже никто не пользуется. Такая забытая, недоступная, но не освобождённая память называется **утечкой памяти** (memory leak). Одна

утечка незаметна; тысячи в долгоживущей программе постепенно съедают всю память. Правило кучи простое: **на каждый malloc — свой free**.

Все куски собраны в code/03-malloc.c. Спецификатор %zu в printf — для размеров, которые возвращает sizeof (у них особый беззнаковый тип); запомни его в паре с sizeof. Реальный вывод:

```
arr[0] = 10
arr[1] = 20
arr[2] = 30
arr[3] = 40
arr[4] = 50
sizeof(int) = 4 байт
Всего заняли = 20 байт
Память освобождена
```

Отличие от Си. В чистом Си кучей управляешь руками: сам malloc, сам free, забыл free — получил утечку. В Objective-C объекты тоже живут на куче (потому ты и держишь их через *), но освобождать их вручную ты не будешь: за это отвечает **ARC** — Automatic Reference Counting, тот самый флаг -fobjc-arc из главы 1. ARC сам расставляет освобождение там, где объект больше не нужен. Механика та же — «объект на куче, в руках адрес», — а вот рутину free с тебя сняли. Как именно ARC считает ссылки и почему иногда всё же протекает (retain cycle), разберём в большой главе 13.

Шаг 9. struct — склеиваем данные в один тип

Пока у нас были одиночные значения. Но часто данные ходят вместе: у точки есть x и y, у человека — имя и возраст. Связать несколько полей в один тип позволяет **struct** (структура):

```
struct Point {
    int x;
    int y;
};
```

Это объявление нового типа struct Point с двумя **полями** — x и y, оба int. Само по себе оно ничего не создаёт, а лишь описывает форму. Точка с запятой после } обязательна — это особенность struct.

Заведём переменную такого типа и заполним её поля. К полю обращаемся через **точку** .:

```
struct Point a;  
a.x = 3;  
a.y = 4;  
printf("a = (%d, %d)\n", a.x, a.y);
```

`a.x` читается «поле `x` структуры `a`». Запускаем:

```
a = (3, 4)
```

Теперь возьмём указатель на структуру. Как и раньше, `&a` — её адрес, а `struct Point *p` — указатель на `struct Point`:

```
struct Point *p = &a;
```

И вот тонкость. У нас в руках не сама структура, а указатель. Чтобы дойти до поля через указатель, по-честному надо сначала разыменовать его, а потом взять поле: `(*p).x`. Скобки тут обязательны из-за приоритета операций, и читать это неудобно. Поэтому в Си придумали короткую запись — **стрелку** `->`:

```
p->x = 10;
```

`p->x` — это ровно то же самое, что `(*p).x`: «пойди по адресу `p` и возьми поле `x`». Стрелка `->` так и расшифровывается: «к полю через указатель». Правило простое:

```
у тебя сама структура → точка:    a.x  
у тебя указатель на неё → стрелка:  p->x    (то же, что (*p).x)
```

Так как `p` смотрит на `a`, запись `p->x = 10` меняет настоящую `a` — тот же приём, что в `swap`. Все куски собраны в `code/03-struct.c`. Реальный вывод:

```
a = (3, 4)  
После p->x = 10: a = (10, 4)  
Через стрелку:  p->y = 4  
Через (*p).y =  4
```

Последние две строки доказывают, что `p->y` и `(*p).y` — одно и то же.

Отличие от Си. `struct` в Си — это только данные, мешок полей. Никакого поведения: у `struct Point` нет методов, она не умеет ничего, кроме как хранить `x` и `y`. Объект в Objective-C устроен поверх ровно такой же `struct` — это `struct`, у которой первым, скрытым полем идёт указатель `isa`. Этот `isa` показывает на **класс** объекта — описание того, какие у объекта методы и как он себя ведёт. Когда ты шлешь объекту сообщение, runtime берёт его `isa`, по нему находит класс, а в классе — нужный метод. Поэтому объект — это «`struct` с данными плюс стрелка к своему поведению»,

и держат его через указатель ровно как `struct Point *p`. Эту кухню — `isa`, классы, `objc_msgSend` — мы во всех деталях разберём в главе 14 про runtime. Пока запомни картинку: объект = данные + `isa` → класс.

Проверяем

Файлы главы — `code/03-pointers.c`, `code/03-swap.c`, `code/03-malloc.c`, `code/03-struct.c`. Это чистый Си, поэтому компилируем как Си-программы (без Foundation и ARC):

```
clang -std=c11 -Wall -Wextra -O2 code/03-pointers.c -o t && ./t
clang -std=c11 -Wall -Wextra -O2 code/03-swap.c      -o t && ./t
clang -std=c11 -Wall -Wextra -O2 code/03-malloc.c    -o t && ./t
clang -std=c11 -Wall -Wextra -O2 code/03-struct.c    -o t && ./t
```

Флаги `-Wall -Wextra` включают все предупреждения — код главы собирается без единого. Вывод должен совпасть с показанным выше (кроме адресов в `03-pointers` — они у каждого свои).

Частые ошибки

- **Перепутал две роли звёздочки.** `int *p` — это объявление указателя. `*p = 5` — это запись по адресу. Первое заводит переменную, второе ходит по ней. Смотри, рядом ли звёздочка с типом.
- **Разыменовал неинициализированный указатель.** `int *p;` без присваивания содержит мусор; `*p = 5` пишет в случайное место и роняет программу. Указатель либо смотрит на реальную переменную, либо `NULL`.
- **Передал значение вместо адреса.** `swapGood(x, y)` вместо `swapGood(&x, &y)` — и это не предупреждение, а **ошибка**: современный clang такой код не соберёт. Зато сразу покажет, чего не хватает:

```
error: incompatible integer to pointer conversion passing 'int' to
      parameter of type 'int *'; take the address with & [-Wint-conversion]
      swapGood(x, y);
                ^
                &
```

Амперсанд под стрелкой — это подсказка компилятора: поставь `&`. Функции, которая ждёт адрес, давай адрес.

- **Забыл free.** Каждый `malloc` обязан получить парный `free`, иначе утечка. И наоборот: освободил — больше по этому указателю не ходи, блок уже не твой.

- **free дважды.** Освободить один и тот же блок два раза — ошибка, программа может упасть. Один `malloc` — один `free`.
- **Точка вместо стрелки (и наоборот).** У структуры — `a.x`. У указателя на структуру — `p->x`. Перепутаешь — компилятор укажет на ошибку типа.

Упражнения

1. Заведи `double price = 99.5;`, возьми на неё указатель `double *pp` и через `*pp` сделай скидку: `*pp = *pp - 10;`. Напечатай `price` и убедись, что она изменилась.
2. Напиши функцию `void addTax(double *value)`, которая увеличивает число по переданному адресу на 12%. Проверь её на переменной в `main`.
3. В `03-malloc.c` поменяй `count` на 10 и заполни массив квадратами: `arr[i] = (i + 1) * (i + 1)`. Не забудь, что `free` остаётся один.
4. Добавь в `struct Point` третье поле `int z;`. Заполни его через `a.z`, а потом измени через `p->z` и напечатай.
5. Намеренно убери `free(arr)` из `03-malloc.c`, пересобери и запусти. Программа отработает как ни в чём не бывало — утечку глазами не видно. Верни `free` обратно: привыкай возвращать память, даже когда «и так работает».
6. Напиши функцию `void swap3(int *a, int *b, int *c)`, которая сдвигает три значения по кругу: `a→b→c→a`. Проверь на трёх переменных.

Что мы получили

Теперь ты знаешь, что такое адрес, указатель и разыменованье, умеешь менять чужие переменные через их адреса, понимаешь разницу между стеком и кучей и владеешь `struct`. И — главное для всей книги — ты понял, почему `NSString *` пишется со звёздочкой: объект живёт на куче, а в руках у тебя его адрес, как `int *p` смотрит на `age`. Эта картинка — «объект по указателю» — будет с тобой до последней главы.

В главе 4 достроим си-базу до конца: разберём массивы и си-строки (`char *`) — те самые «мёртвые массивы байтов», которым `@"..."` противопоставляет живой `NSString`. После неё мы наконец войдём в объектный мир и начнём с главы 5 строить свои классы.

Документация Apple

Эта глава — про базу Си, и Apple отдельной страницы ей не посвящает. Но два опережающих ориентира полезно знать уже сейчас:

- Programming with Objective-C, раздел «Working with Objects» — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/WorkingwithObjects/WorkingwithObjects.html. Объясняет, почему объекты держат по указателю; читай его после главы 5.
- Objective-C Runtime → developer.apple.com/documentation/objectivec — здесь живёт описание isa и того, как устроен объект «под капотом». Вернётся к этой странице в главе 14.

Глава 4. Массивы и строки в Си

В прошлых главах мы работали с одиночными значениями: одно число, один символ, один указатель. Но программам постоянно нужны *наборы* — список оценок, последовательность букв в слове, строка текста. В Си для этого есть массивы. А строка в Си — это вообще просто массив символов с маленькой хитростью на конце.

Это последняя глава нашего захода в чистый Си. Она важная вдвойне: мы не только разберём массивы и строки, но и **почувствуем, как примитивно с ними в Си** — без длины, без проверок, без методов. Именно эта примитивность через несколько глав заставит нас полюбить NSString и NSArray. Чтобы оценить удобный инструмент, полезно сначала помучиться ручным.

Что мы сделаем

- Объявим массив, разложим по нему числа и переберём циклом.
- Увидим, что выход за границу массива Си не ловит — и чем это грозит.
- Поймём, что имя массива — это адрес его начала (привет, глава 3).
- Разберём си-строку: `char s[] = "hi"` и завершающий нулевой байт `'\0'`.
- Напишем **свою** функцию длины строки, потом сверим со `strlen`.
- Попробуем сравнить строки через `==`, удивимся и перейдём на `strcmp`.
- Скопируем строку через `strcpy` и поговорим о переполнении буфера.
- Прочитаем строку с клавиатуры через `fgets` (и узнаем, почему `gets` под запретом).
- Споткнёмся о кириллицу в UTF-8 — и это станет мостиком к NSString.

Массив: набор значений под одним именем

Представь журнал с пятью оценками. Можно завести пять переменных `g1...g5`, но перебрать их циклом не выйдет, и каждую надо называть руками. Массив решает проблему: одно имя, а под ним — пронумерованные ячейки.

Объявляется массив так:

```
int a[5];
```

Читается: «массив `a` из пяти `int`». Компилятор выделяет под него пять ячеек `int` подряд в памяти. Сейчас в них лежит мусор — мы их ещё не заполнили.

Доступ к ячейке — через квадратные скобки и номер:

```
a[0] = 10;  
a[1] = 20;  
a[2] = 30;  
a[3] = 40;  
a[4] = 50;
```

Вот первое, обо что спотыкаются все новички: **нумерация начинается с нуля**. Первый элемент — `a[0]`, а не `a[1]`. У массива из пяти элементов индексы идут от 0 до 4. Последний — `a[4]`, а `a[5]` уже **не существует**. Запомни это сразу, сэкономишь себе часы.

Почему с нуля? Индекс — это не «порядковый номер», а **смещение от начала**. `a[0]` — «ноль ячеек от начала», то есть само начало; `a[3]` — «три ячейки вперёд». Вернёмся к этому, когда свяжем массив с указателем.

Перебираем циклом

Главная сила массива — его можно пройти циклом `for`, который ты уже видел в главе 2:

```
int sum = 0;  
for (int i = 0; i < 5; i++) {  
    printf("a[%d] = %d\n", i, a[i]);  
    sum = sum + a[i];  
}  
printf("Сумма: %d\n", sum);
```

Переменная `i` пробегает значения 0, 1, 2, 3, 4, и на каждом шаге `a[i]` — это очередная ячейка. Условие `i < 5` (а не `i <= 5`!) — ровно потому, что последний живой индекс это 4. Поставишь `<=` — залезешь в `a[5]`, которого нет.

Выход за границу: Си не ловит беду

Вот что отличает Си от языков посовременнее. Если ты напишешь:

```
int a[5];  
a[7] = 999;           /* ячейки a[7] не существует! */
```


то по правилам языка это **неопределённое поведение** (undefined behavior): гарантий нет никаких. Си просто запишет 999 в чужую память. Может, там лежала другая твоя переменная, и ты молча её испортил; может, программа рухнет через десять строк, и ты не поймёшь почему.

Справедливости ради: в этом простейшем виде clang тебя всё-таки окликнет, потому что видит и размер массива, и номер прямо в коде:

```
warning: array index 7 is past the end of the array (that has type
      'int[5]') [-Warray-bounds]
      a[7] = 999;
      ^ ~
note: array 'a' declared here
```

Но стоит спрятать номер в переменную — и компилятор умывает руки:

```
int i = 7;
a[i] = 999;      /* ни единого предупреждения */
```

Собери этот вариант с `-Wall -Wextra` — тишина. Запусти — программа спокойно доработает до конца и вернёт 0, как будто ничего не случилось. А 999 тем временем легло куда-то за край массива. Вот это и есть настоящее лицо проблемы: страшен не тот выход за границу, который видно глазами в листинге, а тот, где индекс приходит из счётчика цикла, из `scanf` или из чужой функции. Такой не поймает никто.

В массиве нет «длины», которую можно спросить. Сколько в нём элементов, знаешь только ты, и следить за границами обязан сам. Запомни это ощущение беспомощности: через несколько глав NSArray снимет эту заботу.

Отличие от Си. Си-массив фиксированного размера ничего о себе не знает: ни длины, ни проверки границ. Залез за край — порча памяти молча. В Objective-C коллекция NSArray — это объект: он помнит свою длину (спросишь `[arr count]`), а на выход за границу честно бросает исключение вместо тихой порчи памяти. А NSMutableArray ещё и **растёт сам** — добавляешь элементы, и места всегда хватает. Никакого [5] наперёд. Подробно — в главе 17.

Имя массива — это адрес начала

В главе 3 мы разобрали указатели: переменная, в которой лежит адрес. У массива и указателя тесная связь. **Имя массива в выражении — это адрес его первого элемента.** То есть `a` и `&a[0]` — это один и тот же адрес.

Проверим это глазами, напечатав оба адреса спецификатором `%p` (он печатает указатель):

```
printf("Адрес a:   %p\n", (void *)a);  
printf("Адрес a[0]: %p\n", (void *)&a[0]);
```

Запись `(void *)` — приведение типа: `%p` хочет «указатель вообще», и мы говорим «считай это просто адресом». Оба `printf` напечатают одно число.

Вот почему индекс — это смещение: `a[3]` компилятор читает как «возьми адрес `a`, отступи на три `int` вперёд, прочитай оттуда». Массив и арифметика указателей — две стороны одной монеты, и это пригодится прямо сейчас, когда дойдём до строк.

Строка в Си — это массив символов

А теперь — главное в этой главе. В Си **нет отдельного типа «строка»**. Строка — это просто массив `char`, в конце которого стоит особый невидимый символ: нулевой байт `'\0'`.

Объявим строку:

```
char s[] = "hi";
```

Мы не указали размер в скобках — компилятор посчитал его сам по тексту. Сколько ячеек он выделил? Не две, а **три**:

индекс:	0	1	2
символ:	'h'	'i'	'\0'

Третья ячейка — тот самый завершающий ноль `'\0'`. Это не цифра «0» (у неё код 48), а символ с кодом 0. Его называют *нуль-терминатором*.

Зачем нужен `'\0'`

Массив не знает своей длины. Так откуда `printf("%s", s)` узнаёт, **где строка кончается**? По нуль-терминатору. Все строковые функции Си идут по массиву символ за символом и останавливаются на `'\0'`. Этот байт — единственная метка конца. Нет его — функция пойдёт читать дальше, в чужую память, пока случайно не встретит где-нибудь ноль. Снова неопределённое поведение.

Поэтому строка `"hi"` из двух букв занимает три байта. За этот «лишний» байт надо всегда помнить, выделяя место под строку руками.

Соберём первую программу: посмотрим размер массива и пройдем по символам сами.

```
#include <stdio.h>  
#include <string.h>
```

```
int main(void) {
    char s[] = "hi";

    /* Сколько ячеек заняла строка? Два символа плюс нулевой байт. */
    printf("Размер массива s: %zu\n", sizeof s);

    /* Печатаем посимвольно, пока не встретим завершающий ноль. */
    for (int i = 0; s[i] != '\0'; i++) {
        printf("s[%d] = '%c' (код %d)\n", i, s[i], s[i]);
    }

    /* А вот кириллица в UTF-8: букв 6, а байтов вдвое больше. */
    char ru[] = "Привет";
    printf("Букв глазами: 6, а strlen даёт: %zu\n", strlen(ru));

    return 0;
}
```

Разберём новое:

- `#include <string.h>` — подключаем заголовок стандартных строковых функций (`strlen`, `strcmp`, `strcpy`...). В чистом Си это `#include`, а не `#import` — `#import` появится только в Objective-C (см. главу 1).
- `sizeof s` — оператор, который даёт размер в байтах. Для нашего массива это 3. Спецификатор `%zu` печатает значение типа `size_t` (беззнаковый целый тип для размеров) — для размеров всегда используем именно его.
- Цикл идёт не «`i < длина`», а «**пока `s[i]` не равен `'\0'`**». Длины у нас нет — ориентируемся на ноль-терминатор, как и все си-функции.
- `'%c'` печатает символ, `%d` — его числовой код. Увидим коды букв.

Скомпилируем и запустим:

```
clang -std=c11 -Wall -Wextra -O2 code/04-strings.c -o strings
./strings
```

Вывод:

```
Размер массива s: 3
s[0] = 'h' (код 104)
s[1] = 'i' (код 105)
Букв глазами: 6, а strlen даёт: 12
```

Размер — 3, как и считали. Цикл напечатал две буквы и остановился на нуле, не показав его. А последняя строка — наша мина, к которой мы вернёмся в конце главы. Пока заметь: «Привет» — это шесть букв, а Си насчитал двенадцать.

Отличие от Си. `char *` или `char[]` — это **мёртвый массив байтов**: у него нет ни длины, ни единого метода. Всё, что с ним можно сделать, — звать внешние функции из `<string.h>`, и те ползают по массиву до `'\0'`. В Objective-C `@"Привет"` — это **объект** `NSString`: он сам знает свою длину, умеет сравниваться, искать подстроки, менять регистр и корректно работать с юникодом. Тот же знак `@` (см. главу 1) превращает голый массив байтов в живой объект. Подробно — в главе 15.

Строим свою функцию длины

Чтобы почувствовать, как устроена строка, напишем `strlen` сами. Задача: дана строка, вернуть число символов до `'\0'` (сам нуль не считаем).

Алгоритм руками простой: ставим счётчик на 0, шагаем по массиву и пока текущий символ не нуль — увеличиваем счётчик.

```
/* Наша собственная длина строки: шагаем до завершающего '\0'. */
static size_t my_strlen(const char *str) {
    size_t n = 0;
    while (str[n] != '\0') {
        n++;
    }
    return n;
}
```

В `main` (полный файл — `code/04-strlen.c`) сверим её с библиотечной:

```
char word[] = "Objective";
printf("my_strlen: %zu\n", my_strlen(word));    /* 9 */
printf("strlen:    %zu\n", strlen(word));      /* 9 */

char empty[] = "";
printf("Длина пустой строки: %zu\n", my_strlen(empty)); /* 0 */
```

Что здесь нового:

- Параметр `const char *str` — указатель на символы. Имя массива — это адрес начала, поэтому при вызове `my_strlen(word)` внутрь передаётся адрес первого символа, а не копия строки. Функция ходит по ней `str[n]`.

- `const` — обещание «строку только читаю, не меняю»; компилятор за этим проследит.
- `static` значит «функция видна только в этом файле» — аккуратная мелочь, не отвлекайся.
- Пустая строка `""` — массив из одного байта `'\0'`. Цикл не сделает ни шага, длина — 0. Хороший крайний случай.

Собираем и запускаем:

```
clang -std=c11 -Wall -Wextra -O2 code/04-strlen.c -o strlen_demo
./strlen_demo
```

```
my_strlen: 9
strlen:    9
Длина пустой строки: 0
```

Наша функция дала ровно то же, что библиотечная, — и на обычном слове, и на пустой строке. Мы своими руками сделали то, что в Си приходится делать постоянно: ползать по массиву до нуля. Обрати внимание, что это работа **линейная** — чем длиннее строка, тем дольше считается её длина, и так каждый раз, когда ты спрашиваешь `strlen`. В `NSString` длина будет просто свойством, которое объект уже знает про себя.

Сравнение строк: ловушка ==

Теперь типичная сцена. Новичок хочет проверить, что две строки одинаковы, и пишет, как с числами, — через `==`. Давай сделаем эту ошибку нарочно и посмотрим, что выйдет.

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char buf1[] = "cat";
    char buf2[] = "cat";
    const char *a = buf1;    /* указывает на первый "cat" */
    const char *b = buf2;    /* указывает на второй "cat" */

    /* Наивная попытка: сравнить строки через ==. */
    if (a == b) {
        printf("a == b: строки равны\n");
    } else {
        printf("a == b: строки РАЗНЫЕ (сравнили адреса, не текст!)\n");
    }
}
```

```
/* Правильно: strcmp возвращает 0, когда строки совпадают. */
if (strcmp(a, b) == 0) {
    printf("strcmp: строки равны\n");
} else {
    printf("strcmp: строки разные\n");
}

/* Знак результата говорит о порядке. */
printf("strcmp(\"apple\", \"banana\") = %d\n", strcmp("apple", "banana"));
printf("strcmp(\"banana\", \"apple\") = %d\n", strcmp("banana", "apple"));

/* Копирование: буфер заведомо больше источника. */
char dst[16];
strcpy(dst, "cat");
printf("После strcpy: %s\n", dst);

return 0;
}
```

Запускаем:

```
clang -std=c11 -Wall -Wextra -O2 code/04-strcmp.c -o strcmp_demo
./strcmp_demo
```

```
a == b: строки РАЗНЫЕ (сравнили адреса, не текст!)
strcmp: строки равны
strcmp("apple", "banana") = -1
strcmp("banana", "apple") = 1
После strcpy: cat
```

Вот сюрприз: текст в `a` и `b` одинаковый — `"cat"` и `"cat"`, — `a ==` говорит «разные». Почему?

Потому что `a` и `b` — это **указатели**, то есть адреса. У нас два отдельных массива `buf1` и `buf2` в разных местах памяти. Сравнивая `a == b`, мы сравниваем **адреса**, а они разные; содержимое `==` вообще не смотрит. Это как сравнить два дома по почтовым адресам, а не по тому, что внутри: адреса разные — «дома разные», хотя обстановка одинаковая.

Чтобы сравнить **содержимое**, есть `strcmp`. Она идёт по обеим строкам символ за символом и возвращает:

- 0 — строки совпадают полностью;
- отрицательное число — первая «меньше» (раньше по алфавиту/кодам);

- положительное — первая «больше».

Поэтому проверка на равенство выглядит непривычно: `strcmp(a, b) == 0`. `Не == 1, а == 0` — «разница нулевая, строки одинаковы». `apple` против `banana` дал `-1` (`a` идёт раньше `b`), наоборот — `1`.

Только не привязывайся к самим числам `-1` и `1`: стандарт обещает лишь **знак** результата. Библиотека `macOS`, например, возвращает разность первых несовпавших байтов, и `strcmp` для `"apple"` и `"cherry"` во время работы программы даст `-2` (`'a'` — `'c'`). Поэтому сравнивают всегда со знаком: `< 0, == 0, > 0`.

Отличие от Си. Си-строки сравнивают по содержимому через `strcmp(a, b) == 0`. В Objective-C голый `==` для объектов-строк — та же ловушка, только ещё коварнее: он сравнивает **адреса объектов**, а не текст. Две строки с одинаковыми буквами, созданные по отдельности, обычно дают `==` → «не равны» (а иногда случайно «равны» — и это ещё хуже, почему так бывает, увидим в главе 15). Поэтому строки-объекты сравнивают методом `[a isEqualToString:b]` — он смотрит именно на текст. Запомни связку: в Си — `strcmp`, в Objective-C — `isEqualToString`; `a ==` для строк не доверяй ни там, ни там. Разберём в главе 15.

Копирование строк и опасность переполнения

В той же программе была строка `strcpy(dst, "cat")`. `strcpy` (string copy) копирует строку из второго аргумента в первый, **вместе с завершающим нулём**. Результат — `dst` теперь содержит `"cat"`.

Но у `strcpy` знаменитая опасность: она **не проверяет**, влезает ли источник в приёмник. Сколько байт до `'\0'` в источнике — столько и запишет, не глядя на размер `dst`. Если приёмник меньше, `strcpy` вылезет за его границы и затрёт соседнюю память. Это классическое *переполнение буфера* (buffer overflow) — источник бесчисленных багов и дыр в безопасности.

```
char dst[3];
strcpy(dst, "cat");    /* "cat" + '\0' = 4 байта, а места 3 — БЕДА */
```

Здесь `"cat"` требует четыре байта (три буквы и ноль), а `dst` вмещает три. По правилам языка `strcpy` запишет четвёртый байт за краем массива — то самое неопределённое поведение.

На `macOS` тебе в этом месте повезло. Заголовки системы подменяют `strcpy` проверяющей версией (механизм называется `FORTIFY_SOURCE`), и когда компилятор знает размер приёмника, он сначала предупредит при сборке:

```
warning: 'strcpy' will always overflow; destination buffer has size 3,  
      but the source string has length 4 (including NUL byte)  
      [-Wfortify-source]
```

А если всё же запустить, программа аварийно остановится ровно на копировании, вместо того чтобы тихо испортить соседнюю память. Но это любезность конкретной системы, а не гарантия языка: если размер приёмника компилятору неизвестен (скажем, адрес буфера пришёл из другой части программы), проверять будет нечего, и переполнение пройдёт молча.

Правило простое: **под приёмник выделяй не меньше места, чем длина источника плюс единица на '\0', а лучше с запасом**. Мы так и сделали — взяли `char dst[16]` под трёхбуквенное слово, с огромным запасом.

Отличие от Си. В Си ты сам стережёшь каждый байт буфера, и одна забытая проверка размера превращается в дыру безопасности. В Objective-C копию строки делает сам `NSString`: `[str copy]` возвращает строку-копию (а у неизменяемой строки — и вовсе тот же объект: копировать нечего, менять её всё равно никто не сможет). Память выделяет Foundation, ручного буфера нет — и переполнять нечего. Память объектов разберём в главе 13, копирование строк — в главе 15.

Ввод строки: `fgets`, и почему `gets` запрещён

Чтобы прочитать строку с клавиатуры, начинающие тянутся к функции `gets`. **Не делай этого.** `gets` не принимает размер буфера — ей всё равно, сколько ты ввёл; введёшь больше, чем влезает, — то же переполнение буфера. Функция настолько опасна, что её **убрали из языка**: в современном стандарте Си `gets` просто нет.

Правильный инструмент — `fgets` (file get string). Она **знает размер буфера**, поэтому за него никогда не вылезет — лишнее просто не прочитает. Рабочий пример:

```
#include <stdio.h>  
#include <string.h>  
  
int main(void) {  
    char name[32];  
  
    printf("Как тебя зовут? ");  
    if (fgets(name, sizeof name, stdin) == NULL) {  
        printf("Ничего не ввёл.\n");  
        return 1;  
    }  
}
```



```
name[strcspn(name, "\n")] = '\0'; /* срезаем перевод строки */
printf("Привет, %s! В имени %zu байт.\n", name, strlen(name));
return 0;
}
```

Разберём:

- `fgets(name, sizeof name, stdin)` — три аргумента: куда читать (`name`), сколько байт максимум (`sizeof name`, то есть 32), откуда (`stdin` — клавиатура). Больше 31 байта не возьмёт (последний бережёт под `'\0'`). Вот защита, которой нет у `gets`.
- Вернёт `NULL`, если читать нечего (ввод закончился) — это мы проверяем.
- `fgets` оставляет в строке символ перевода строки `'\n'` от нажатия Enter. Строка `name[strcspn(name, "\n")] = '\0';` находит этот `'\n'` и заменяет нулём, обрезая строку. Без этого вывод съехал бы на новую строку.

Файл лежит в `code/04-fgets.c`. Соберём и скормим ввод не с клавиатуры, а через конвейер — так удобнее проверять: `printf` слева печатает строку, а `|` отправляет её нашей программе вместо клавиатуры.

```
clang -std=c11 -Wall -Wextra -O2 code/04-fgets.c -o fgetsdemo
printf 'Серик\n' | ./fgetsdemo
```

Как тебя зовут? Привет, Серик! В имени 10 байт.

Вопрос и ответ оказались в одной строке — это нормально: `printf` в программе не переводил строку после «зовут?», ждал ввода в той же.

И снова мина! Имя «Серик» — это **пять** букв, а программа говорит **десять** байт. Та же история, что и с «Привет» в начале главы. Пора разобраться, что происходит.

Кириллица в UTF-8: буква ≠ байт

Си-строка — это массив `char`, а `char` — это **один байт**. Пока мы писали латиницей (`hi`, `Objective`, `cat`), всё совпадало: одна буква — один байт. Латинские буквы, цифры и знаки помещаются в один байт по старой таблице ASCII.

С кириллицей иначе. Современный текст хранят в кодировке **UTF-8**, и в ней русская буква занимает **два байта**, а не один. Эмодзи — четыре, а составные эмодзи (флаги, семьи) — и того больше. Поэтому:

- «Привет» — 6 букв → `strlen` даёт 12 (6×2 байта);
- «Серик» — 5 букв → `strlen` даёт 10 (5×2 байта).

`strlen` честно посчитала **байты до нуля** — это всё, что она умеет. Про «буквы» она ничего не знает; для неё строка — поток байтов. И стоит тебе взять `s[0]` у строки «Привет», ты получишь не букву «П», а только её *первый байт* — половину символа. Перебрать кириллицу посимвольно простым циклом `for (i...) s[i]` не выйдет: ты будешь резать буквы пополам.

Вот стена, в которую упирается Си со строками. Считать длину в буквах, взять третий символ, перевернуть слово, сменить регистр — для не-латиницы всё это в чистом Си превращается в ручной разбор байтов UTF-8. Мучительно и легко ошибиться.

Отличие от Си. Здесь `char *` сдаётся окончательно: он видит байты, а не буквы, и про юникод не знает ничего. `NSString` устроен иначе — он хранит текст в юникоде, а не как сырые байты UTF-8. Длина «Привет» для него — 6, а не 12. Сравнение, поиск, смена регистра работают для любого языка (у эмодзи с длиной есть своя тонкость — её разберём в главе 15). Ровно ради этого Objective-C и не использует си-строки в обычной работе, а везде гоняет `NSString`. Полностью разберём в главе 15 — и ты оценишь, от какой боли он избавляет.

Проверяем

Готовые файлы главы лежат в `code/`. Собираются как чистый Си:

```
clang -std=c11 -Wall -Wextra -O2 code/04-arrays.c -o t && ./t
clang -std=c11 -Wall -Wextra -O2 code/04-strings.c -o t && ./t
clang -std=c11 -Wall -Wextra -O2 code/04-strlen.c -o t && ./t
clang -std=c11 -Wall -Wextra -O2 code/04-strcmp.c -o t && ./t
clang -std=c11 -Wall -Wextra -O2 code/04-fgets.c -o t
printf 'Серик\n' | ./t
```

Ожидаемый вывод `04-arrays.c` (адреса будут свои):

```
Первый: 10
Последний: 50
a[0] = 10
a[1] = 20
a[2] = 30
a[3] = 40
a[4] = 50
Сумма: 150
Адрес a: 0x7ff7ba47ca70
Адрес a[0]: 0x7ff7ba47ca70
```

Главное в нём — две последние строки: адрес `a` и адрес `a[0]` совпадают. Имя массива и есть адрес его начала.

Частые ошибки

- **Считать с единицы или условие $i \leq N$.** Индексы массива из N элементов идут от 0 до $N-1$; последний — `a[N-1]`. Условие цикла — $i < N$, иначе залезешь в несуществующий `a[N]` (классический «off-by-one»).
- **Забыть про `'\0'`.** Под строку из N символов нужно минимум $N+1$ байт. Забудешь байт под ноль — функции уползут за край.
- **Сравнивать строки через `==`.** `==` сравнивает адреса, а не текст; для содержимого — `strcmp(a, b) == 0`, причём именно `== 0` (положительное число значит лишь «первая больше»). А для `NSString` позже — `isEqualToString:`.
- **`strcpy` в маленький буфер.** Источник плюс ноль должны влезать в приёмник; не уверен — бери буфер с запасом.
- **`gets`.** Не используй никогда: переполнение гарантировано, и из языка её убрали. Только `fgets` с размером буфера.
- **Резать кириллицу по `s[i]`.** В UTF-8 русская буква — два байта, эмодзи — четыре и больше; посимвольный перебор байтов разрежет буквы. Это работа для `NSString`.

Упражнения

1. Заведи массив `int a[5]` со своими числами и выведи их сумму и среднее (сумму делить на 5.0, чтобы получить дробное).
2. Допиши в `04-arrays.c` поиск максимального элемента: заведи переменную `max = a[0]` и в цикле сравнивай с каждым следующим.
3. Напиши функцию `my_strcmp(const char *x, const char *y)`, которая, как `strcmp`, возвращает 0 при совпадении строк. Идею возьми из `my_strlen`: шагай по обеим строкам, сравнивая `x[i]` и `y[i]`.
4. Напиши функцию, которая печатает строку задом наперёд: сначала узнай длину через `my_strlen`, потом иди циклом от последнего символа к первому. Проверь на латинском слове — и посмотри, что выйдет на кириллице (буквы поедут — это ожидаемо).
5. Прочитай через `fgets` слово и посчитай в нём гласные латинские буквы (`a`, `e`, `i`, `o`, `u`), перебирая символы циклом.
6. Намеренно объяви `char dst[3]` и скопируй в него `strcpy(dst, "cat")`. Прочитай предупреждение компилятора, потом запусти программу и убедись, что macOS остановила её на копировании. Подумай, что случилось бы на системе без такой защиты.

Что мы получили

Мы прошли последнюю главу о чистом Си — и собрали список претензий к нему. Массив не знает своей длины и не ловит выход за границу. Строка — голый массив байтов с нулём на конце, без единого метода: сравнение через `strcmp`, копирование через рискованный `strcpy`, ввод через осторожный `fgets`. А кириллица в UTF-8 ломает наивный «буква = байт».

Каждая из этих болей — будущая радость от Foundation. `NSArray` помнит длину и растёт сам. `NSString` для кириллицы считает длину в *буквах*, корректно сравнивается через `isEqualToString:`, работает с юникодом и не даёт переполнить буфер. Мы помучились вручную, чтобы оценить, что именно берёт на себя объектный мир.

База Си закончена: переменные, типы, указатели, память, массивы и строки у нас в руках. С главы 5 начинается самое интересное — объекты и сообщения. Тот самый знак `@` и квадратные скобки `[ ]`, ради которых всё и затевалось.

Документация Apple

- `NSString` — developer.apple.com/documentation/foundation/nsstring (строки-объекты: длина, сравнение, юникод; разберём в главе 15).
- `isEqualToString:` — [developer.apple.com/documentation/foundation/nsstring/isequal\(to:\)](http://developer.apple.com/documentation/foundation/nsstring/isequal(to:)) (правильное сравнение строк по содержимому).
- `NSArray` — developer.apple.com/documentation/foundation/nsarray и `NSMutableArray` — [.../nsmutablearray](http://developer.apple.com/documentation/foundation/nsmutablearray) (коллекции вместо си-массивов; глава 17).
- Строковые функции Си (`strlen`, `strcmp`, `strcpy`, `fgets`) — часть стандартной библиотеки Си; их описание есть в `man string` и `man fgets` прямо в терминале.

Глава 5. Объекты и сообщения

Вот и началась объектная часть. Всё, что было до этого — переменные, указатели, функции, массивы, — это фундамент. А теперь на нём вырастает главная идея Objective-C: **объекты, которые шлют друг другу сообщения**.

В главе 1 слово «сообщение» мелькнуло мимоходом — в названии расширения .m. Здесь мы разберём, что оно значит, по косточкам: каждую квадратную скобку, каждое двоеточие. Своих классов мы пока писать не будем: это в главе 6. Сейчас возьмём готовые объекты из библиотеки Foundation (NSString, NSMutableArray) и научимся с ними разговаривать. Когда ты поймёшь, как послать сообщение чужому объекту, свой собственный класс покажется мелочью.

Что мы сделаем

- Разберёмся, что такое **класс** и **объект** на простом примере.
- Научимся **отправлять сообщение**: [получатель сообщение].
- Передадим сообщению **аргумент**: [s hasPrefix:@"При"].
- Поймём **многоаргументные** селекторы: setObject:forKey:.
- Разберём **вложенные** сообщения: [[NSMutableArray alloc] init].
- Встретим id, nil — и увидим, что **сообщение к nil безопасно**.
- Заглянем в BOOL, YES, NO.
- Попробуем на вкус **интроспекцию**: isKindOfClass:, respondsToSelector:.

И по дороге — много врезок «Отличие от Си». Эта глава про них в первую очередь: именно здесь Objective-C расходится с Си сильнее всего.

Класс и объект — на пальцах

Представь чертёж дома. На чертеже написано: стены, окна, дверь, крыша. Сам по себе чертёж — не дом, в нём не живут. Но по этому одному чертежу можно построить сто домов. Они будут одинаковой конструкции, но это сто разных домов: в одном горит свет, в другом нет; у одного зелёная дверь, у другого синяя.

- **Класс** — это чертёж. Он описывает, что объект умеет и что в нём хранится. В коде класс один.
- **Объект** — это конкретный дом, построенный по чертежу. Их может быть много, и каждый живёт своей жизнью. Объект ещё называют **экземпляром** (instance) класса.

NSString — это класс, чертёж «текстовой строки». А @"привет" — это один конкретный объект NSString, построенный по этому чертежу. Рядом может жить @"пока" — другой объект того же класса. Класс один, объектов много.

Объект умеет две вещи: **хранить данные** (строка хранит свои буквы) и **отвечать на сообщения** (строку можно спросить её длину, попросить перевести себя в верхний регистр). Сейчас займёмся вторым.

Отправка сообщения

Вот центральная конструкция языка:

```
[receiver message]
```

Читается так: «послать объекту receiver сообщение message». Три слова, которые надо запомнить — это термины самой Apple:

- **получатель** (receiver) — объект, которому шлём. Стоит сразу после открывающей скобки.
- **сообщение** (message) — что мы у него просим. Стоит вторым.
- **селектор** (selector) — имя сообщения, то есть message. Пока считай «селектор» и «имя сообщения» синонимами; чуть позже увидим, что селектор — это ещё и отдельная сущность, которую можно подержать в руках.

Возьмём строку и спросим её длину:

```
NSString *s = @"привет, objective-c";  
NSUInteger len = [s length];
```

Здесь s — получатель, length — сообщение (селектор). Квадратные скобки означают «отправь». Объект s получает сообщение length, сам считает свои символы и возвращает число. Это число мы кладём в len.

Тип NSUInteger — это просто целое без знака (длина не бывает отрицательной). Считай его «большим unsigned int»; подробно типы Foundation разберём позже, сейчас важны скобки.

Отличие от Си. В Си длину строки берут функцией: strlen(s) — имя функции, в скобках аргумент. В Objective-C — [s length]: получатель, потом сообщение. Это не вызов функции «снаружи» над данными, а **просьба к самому объекту** что-то про себя сообщить. Объект и его поведение живут вместе.

Отличие от Си. Квадратные скобки тут — **не индексация массива**. В Си `a[i]` означает «элемент массива `a` под номером `i`». В Objective-C `[s length]` — это «послать `s` сообщение `length`». Та же пара символов `[ ]`, совсем другой смысл. Внутри скобок objc сообщения слева стоит объект-получатель, а не индекс.

Что значит «сообщение», а не «вызов»

В большинстве языков (и в Си тоже) ты *вызываешь функцию*: компилятор заранее, ещё при сборке, знает точный адрес кода, на который надо прыгнуть. В Objective-C всё иначе. Когда ты пишешь `[s length]`, в готовой программе не зашит адрес метода `length`. Вместо этого во время выполнения срабатывает механизм `runtime`: он берёт объект `s`, смотрит, какого тот класса, находит у класса метод с именем `length` и только тогда его запускает.

Этот «поиск метода по имени во время работы программы» называется **динамической диспетчеризацией** (`dynamic dispatch`). Именно поэтому в Objective-C говорят «отправить сообщение», а не «вызвать метод»: ты не прыгаешь по известному адресу, ты *посылаешь объекту имя*, а он сам в рантайме разбирается, какой код на это имя выполнить.

Отличие от Си. `foo(obj, 5)` в Си — адрес функции `foo` известен уже на этапе компиляции; программа прыгает прямо на него. `[obj foo:5]` в Objective-C — отправка сообщения: `runtime` берёт `obj`, смотрит его класс и **ищет метод `foo`: во время выполнения**. Поэтому объект одного типа и объект другого могут ответить на одно и то же сообщение по-разному — решает сам объект, а не компилятор.

Звучит как лишняя работа — и да, она чуть медленнее прямого вызова. Но именно из этой гибкости вырастает половина возможностей языка: полиморфизм, интроспекция, категории, пересылка сообщений. До них дойдём; пока просто запомни: **скобки `[ ]` — это отправка имени, а не прыжок по адресу**.

Шаг 1. Шлём строке несколько сообщений

Хватит теории — построим программу. Возьмём одну строку и зададим ей несколько вопросов, каждый раз глядя на реальный вывод.

```
#import <Foundation/Foundation.h>

int main(void) {
    @autoreleasepool {
        NSString *s = @"привет, objective-c";

        NSUInteger len = [s length];
```

```
    NSLog(@"длина: %lu", (unsigned long)len);

    NSString *big = [s uppercaseString];
    NSLog(@"в верхнем регистре: %@", big);
}
return 0;
}
```

Разберём новое:

- [s length] — спросили длину. Вернулось целое, положили в len.
- %lu в NSLog — спецификатор для unsigned long. Мы привели len к unsigned long через (unsigned long)len, чтобы тип точно совпал со спецификатором и компилятор не ругался. NSUInteger на разных системах бывает разной ширины, и такое приведение — стандартная мера предосторожности.
- [s uppercaseString] — попросили строку отдать **новую** строку в верхнем регистре. Важно: исходная s не меняется. NSString неизменяемая (immutable), сообщение uppercaseString не трогает s, а возвращает свежий объект. Его мы кладём в big.
- %@ (помнишь из главы 1) печатает объект — здесь нашу строку.

Соберём и запустим:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/05-messages.m -o /tmp/t && /tmp/t
```

```
2026-06-28 12:16:46.661 t[61103:1251002] длина: 19
```

```
2026-06-28 12:16:46.661 t[61103:1251002] в верхнем регистре: ПРИВЕТ, OBJECTIVE-C
```

(Дата, время и числа в скобках у тебя будут свои — их добавляет NSLog, см. главу 1. Дальше я буду показывать вывод так же.)

Длина 19 — посчитай символы в привет, objective-c, включая запятую, пробел и дефис, выйдет ровно 19. Верхний регистр сработал и для русских букв, и для латиницы.

Шаг 2. Сообщение с аргументом

Сообщение length пустое — мы ничего объекту не передаём. Но часто объекту нужно что-то сообщить. Тогда после имени ставится **двоеточие**, а за ним — аргумент.

Спросим строку, начинается ли она с «при»:


```
B00L starts = [s hasPrefix:@"при"];
NSLog(@"начинается с \"при\": %d", starts);
```

Смотри на форму записи: `[s hasPrefix:@"при"]`.

- `s` — получатель.
- `hasPrefix:` — селектор. **Двоеточие — часть имени.** Метод называется именно `hasPrefix:` (с двоеточием), а не `hasPrefix`. Двоеточие говорит: «этому сообщению нужен один аргумент».
- `"при"` — сам аргумент, объект-строка. Стоит сразу после двоеточия.

Вернётся `B00L` — это objс-шный «да/нет». У него два значения: `YES` (истина) и `NO` (ложь). Под капотом `YES` — это 1, `NO` — это 0, поэтому печатаем через `%d` (целое). Полноценные `if (starts)` `{ ... }` тоже работают.

Отличие от Си. В Си логическое значение — это `bool` из `<stdbool.h>` со значениями `true/false` (мы им пользовались в главе 2), а в условиях по-прежнему работает старое правило «0 — ложь, всё остальное — истина». Objective-C завёл свою пару имён: тип `B00L` со значениями `YES` и `NO`. И это не просто переименованный `bool`: чем именно окажется `B00L`, зависит от платформы — на Apple Silicon это настоящий `bool`, а на 64-битном Intel — `signed char`, то есть обычный байт, куда влезает не только 0 и 1. Отсюда правило, верное везде: пиши `if (ok)`, а не `if (ok == YES)`. Проверка на точное равенство единице подведёт, если в переменную каким-то образом попало другое ненулевое число. Эту ловушку подробно разбираем в приложении D с задачами собеседований.

Двоеточие в имени селектора — ключевая деталь языка, привыкай видеть её как часть названия:

Отличие от Си. В Си имя функции цельное: `hasPrefix(s, "при")`. Имя — это `hasPrefix`, аргументы отдельно в скобках. В Objective-C имя метода **«разрезано» двоеточием** прямо по месту аргумента: `hasPrefix:`. Двоеточие — буквально часть имени. Нет аргумента — нет и двоеточия (`length`). Один аргумент — одно двоеточие (`hasPrefix:`).

Шаг 3. Вложенные сообщения

Результат одного сообщения — это объект. Значит, ему можно тут же послать следующее сообщение. Для этого внешние скобки оборачивают внутренние.

Спросим длину строки **после** перевода в верхний регистр:

```
NSUInteger n = [[s uppercaseString] length];
```

Читаем изнутри наружу, как в математике скобки:

1. `[s uppercaseString]` — внутренние скобки выполняются первыми. Строка `s` отдаёт новый объект — себя в верхнем регистре.
2. Этот новый объект и становится получателем внешнего сообщения: `[ ... length]`. У него спрашиваем длину.

То есть `[[s uppercaseString] length]` — это «возьми верхний регистр `s` и спроси у результата длину». Никакой промежуточной переменной заводить не пришлось.

Ещё пример — приклеим к строке восклицательный знак:

```
NSString *full = [s stringByAppendingString:@"!"];
```

`stringByAppendingString:` берёт получатель `s`, дописывает к нему строку-аргумент и возвращает **новую** строку (исходная `s` снова не тронута). Имя метода длинное и говорящее — это норма для Objective-C: методы называют так, чтобы строка кода читалась почти как фраза.

И сравнение строк — отдельная важная тема:

```
BOOL same = [s isEqualToString:@"привет, objective-c"];
```

`isEqualToString:` сравнивает строки **по содержимому** — буква за буквой. Запомни это сразу, потому что тут новички падают:

Отличие от Си. Для строк в Objective-C **никогда не сравнивай через `==`**. `==` сравнивает указатели — то есть «это один и тот же объект в памяти?», а не «одинаковый ли текст?». Две разные строки с одинаковыми буквами `==` посчитает разными. Сравнивай содержимое сообщением `isEqualToString:`. (В Си строки сравнивали `strcmp` — та же логика «сравнивай содержимое, а не адреса».)

Полный файл этого раздела — `code/05-messages.m`. Запуск даёт:

```
длина: 19
в верхнем регистре: ПРИВЕТ, OBJECTIVE-C
начинается с "при": 1
длина верхнего: 19
с восклицанием: привет, objective-c!
строки равны: 1
```

(Префикс с датой я для краткости убрал — у тебя он будет.) 1 в строках про префикс и равенство — это YES.

Шаг 4. Создаём объект: alloc и init

До сих пор объекты-строки нам дарил литерал @"...". Но обычно объект надо **построить руками**. Делается это в два сообщения, и оба ты уже почти понимаешь.

Возьмём NSMutableArray — изменяемый массив (mutable = изменяемый), в который можно докладывать элементы. Создаём так:

```
NSMutableArray *list = [[NSMutableArray alloc] init];
```

Это вложенные сообщения, читаем изнутри:

1. [NSMutableArray alloc] — посылаем сообщение alloc **самому классу** NSMutableArray. Да, класс тоже может быть получателем! alloc (от *allocate*, выделить) выделяет в памяти место под новый, пустой, ещё не настроенный объект и возвращает его.
2. [... init] — этому сырому объекту посылаем init (от *initialize*, инициализировать). init приводит объект в рабочее состояние: обнуляет счётчики, готовит внутренности. Возвращает готовый к работе объект, его и кладём в list.

Связка alloc/init — стандартный способ родить объект в Objective-C. Почти всегда они идут вместе и именно вложенной парой. Запомни картинку [[Класс alloc] init] целиком.

Отличие от Си. В Си объект (структуру) в куче создавали так: сначала malloc(sizeof(struct ...)) выделяет память, потом ты вручную заполняешь поля. В Objective-C это два сообщения: alloc выделяет память, init заполняет — но делает это сам объект по своему чертежу. Никакого ручного malloc/free: память объектов считает ARC (глава 13). Ты не освобождаешь list руками — нужный код освобождения за тебя вставит компилятор.

Теперь наполним массив и спросим его:

```
[list addObject:@"яблоко"];
[list addObject:@"банан"];
[list addObject:@"вишня"];

NSUInteger c = [list count];
NSString *first = [list objectAtIndex:0];
NSString *last = [list objectAtIndex:[list count] - 1];
```

- addObject: — сообщение с одним аргументом-объектом. Кладёт объект в **конец** массива. Каждый вызов добавляет один элемент.
- count — сообщение без аргументов, возвращает число элементов.
- objectAtIndex: — берёт элемент по номеру. Нумерация **с нуля**, как и в массивах Си: первый элемент — индекс 0. Поэтому последний — это count - 1.

Обрати внимание на `[list objectAtIndex:[list count] - 1]`: внутри аргумента опять вложенное сообщение `[list count]`, у которого вычитаем 1. Аргументом может быть любое выражение, лишь бы получился нужный тип.

Отличие от Си. В Си массив — это кусок памяти фиксированного размера: объявил `int a[5]` — пять и останется, добавить шестой некуда. `NSMutableArray` растёт сам: шлёшь `addObject:` сколько нужно, он сам выделяет память под новые элементы. И берёшь элемент не через `a[i]`, а сообщением `objectAtIndex:` — снова разговор с объектом, а не прямой доступ к памяти.

Шаг 5. Многоаргументный селектор

Часто сообщению нужно сразу несколько данных. Тогда **каждый** аргумент получает свою именованную часть с двоеточием. Имя метода складывается из всех этих частей.

Возьмём словарь `NSMutableDictionary` — хранилище пар «ключ → значение». Положим в него пару:

```
NSMutableDictionary *dict = [[NSMutableDictionary alloc] init];
[dict setObject:@"красный" forKey:@"яблоко"];
```

Разбираем `[dict setObject:@"красный" forKey:@"яблоко"]`:

- `dict` — получатель.
- `setObject:` — первая часть имени, после неё первый аргумент `@ "красный"` (значение).
- `forKey:` — вторая часть имени, после неё второй аргумент `@ "яблоко"` (ключ).
- Полное имя метода (селектор) — **`setObject:forKey:`**, обе части вместе, оба двоеточия. Не `setObject`, не `forKey`, а именно `setObject:forKey:` целиком.

Прочитай вслух: «у `dict` установить объект красный для ключа яблоко». Именованные аргументы делают строку самодокументируемой: видно, что где. Достать значение по ключу — обратное сообщение `objectForKey:`:

```
NSLog(@"цвет яблока: %@", [dict objectForKey:@"яблоко"]); // красный
```

Отличие от Си. В Си имя функции цельное и неделимое — назвали бы `setObjectForKey(dict, value, key)`, и из самого вызова не видно, где значение, а где ключ: просто два аргумента подряд, легко перепутать местами. В Objective-C имя **разрезано** двоеточиями ровно по аргументам: `setObject:forKey:`. Каждый аргумент подписан своим словом прямо в месте вызова. Переставить сами подписи местами уже не выйдет: метода `forKey:setObject:` нет, и компилятор такую строку не примет.

Селектор — это всё имя со всеми двоеточиями. Сосчитать аргументы метода можно по двоеточиям в его имени:

- `length` — двоеточий нет → 0 аргументов.
- `hasPrefix:` — одно двоеточие → 1 аргумент.
- `setObject:forKey:` — два двоеточия → 2 аргумента.

Полный файл — `code/05-array.m`. Вывод:

```
в массиве элементов: 3
первый: яблоко
последний: вишня
цвет яблока: красный
[0] = яблоко
[1] = банан
[2] = вишня
```

id — любой объект

Иногда заранее не известно, объект какого именно класса будет в руках. Для таких случаев есть тип `id` — «какой угодно объект». Переменная типа `id` может указывать на строку, на массив, на словарь — на что угодно.

```
id thing = @"я строка";
```

Заметь: у `id` **нет звёздочки**. Пишут `id obj`, а не `id *obj`. `id` уже сам по себе означает «указатель на какой-то объект» — звёздочка встроена в него. А вот у конкретного типа звёздочка нужна: `NSString *s`.

`id` пригодится, когда мы дойдём до коллекций (в массиве могут лежать объекты разных классов) и до `runtime`. Пока запомни: `id` — это «объект, тип которого мы не уточняем».

nil — ноль для объектов

В Си «ничего» для указателя — это `NULL`. В Objective-C «никакого объекта» — это `nil`. Переменная-объект, которой ещё ничего не присвоили, или которую обнулили, равна `nil`.

```
NSString *empty = nil; // в руках нет никакого объекта
```

И вот теперь — самая удивительная для новичка из Си вещь во всей главе.

Сообщение к nil безопасно

В Си попытка пройти по NULL-указателю — это крах программы (segmentation fault). Разыменовал нулевой адрес — приложение упало. В Objective-C **послать сообщение nil совершенно безопасно**: ничего не выполнится, программа не упадёт, а в ответ ты получишь «нулевой» результат — nil, 0 или NO, смотря что метод должен был вернуть.

Проверим это на деле:

```
NSString *empty = nil;

NSUInteger len = [empty length];           // не падает! вернёт 0
NSString *up   = [empty uppercaseString]; // вернёт nil
BOOL b        = [empty hasPrefix:@"При"]; // вернёт NO (0)
```

Запускаем code/05-nil.m:

```
длина nil-строки: 0
uppercase от nil: (null)
hasPrefix у nil: 0
сообщение к id-nil: (null)
имя не задано
```

Разбор:

- [empty length] дало 0. Сообщение к nil тихо «проглатывается», а число-результат выходит нулём.
- [empty uppercaseString] дало nil. NSLog печатает nil-объект как (null).
- [empty hasPrefix:@"При"] дало NO, то есть 0.

Программа спокойно доработала до конца. Никакого падения.

Отличие от Си. В Си вызов через нулевой/неизвестный адрес — мгновенный крах: ((void(*)())NULL)() или разыменование NULL роняет программу. В Objective-C [nil любоеСообщение] — **безопасная пустая операция**: runtime видит, что получатель nil, и просто ничего не делает, возвращая нулевое значение. Это сознательное решение языка, а не случайность.

Это удобно: можно не обвешивать каждый объект проверкой if (x != nil) перед каждым сообщением — отправка nil и так не навредит. Но у медали есть обратная сторона: если объект случайно оказался nil, ошибка не выстрелит сразу, а «тихо» даст 0 или nil, и баг всплывёт позже и не там. Поэтому, когда результат важен, проверять на nil всё-таки стоит:

```
NSString *name = nil;
if (name == nil) {
    NSLog(@"имя не задано");
} else {
    NSLog(@"имя: %@", name);
}
```

Здесь `==` к месту: мы сравниваем не текст, а сам факт «есть объект или нет». Для проверки на `nil ==` — это правильно. (А вот сравнивать *содержимое* строк через `==` нельзя — про это была врезка выше.)

Интроспекция — на вкус

Раз объект сам в рантайме разбирается с сообщениями, его можно об этом **спросить**: «ты какого класса?», «ты ответишь на такое сообщение?». Это называется **интроспекция** (introspection) — самопознание объекта. Полно разберём в главе 14, тут только попробуем.

Два сообщения, которые понимает любой объект:

```
id obj = @"я строка";

if ([obj isKindOfClass:[NSString class]]) {
    NSLog(@"obj — это NSString");
}

if ([obj respondsToSelector:@selector(uppercaseString)]) {
    NSLog(@"строка умеет uppercaseString → %@", [obj uppercaseString]);
}
```

- `isKindOfClass:` — спрашивает: «объект относится к этому классу (или его потомку)?». Аргумент — сам класс. Класс как объект-значение даёт сообщение `[NSString class]`: «дай мне объект, представляющий класс `NSString`».
- `respondsToSelector:` — спрашивает: «ты ответишь на такое сообщение?». Аргумент — селектор. А чтобы получить селектор из имени метода, есть конструкция `@selector(...)`: `@selector(uppercaseString)` — это «возьми имя `uppercaseString` как селектор». Вот тот случай, когда селектор — отдельная сущность, которую держишь в руках и передаёшь аргументом, а не просто посылаешь.

Запускаем `code/05-introspection.m`:

```
obj — это NSString
obj — НЕ NSArray
строка умеет uppercaseString → Я СТРОКА
строка НЕ умеет addObject:
```

Строка честно призналась, что она NSString, а не NSArray, что умеет uppercaseString, но не умеет addObject: (это сообщение для массивов). respondsToSelector: — прямой способ заранее узнать, безопасно ли слать объекту то или иное сообщение, не нарываясь на ошибку.

Отличие от Си. В Си у данных нет «самосознания»: struct не знает своего типа и не может сказать, какие функции с ним работают — это всё в голове у программиста и в коде компилятора. Объект Objective-C носит знание о себе с собой: в рантайме он способен ответить, какого он класса и на какие сообщения откликнется. Си так не умеет в принципе.

Полная картина: что происходит при [obj msg:arg]

Соберём механику воедино. Когда исполняется строка [s hasPrefix:@"при"]:

```
[s hasPrefix:@"при"]
|         |         |
|         |         |└─ аргумент: объект-строка @"при"
|         |         |└─ селектор (имя сообщения): hasPrefix:
|         |         |└─ получатель: объект s
```

1. Программа доходит до этой строки **во время выполнения**.
2. Runtime смотрит на получателя *s*. Если *s* — nil, всё прекращается, результат «нулевой». Иначе идём дальше.
3. Runtime узнаёт класс объекта *s* (это NSString).
4. В этом классе ищется метод с именем-селектором hasPrefix:.
5. Найденный метод запускается с аргументом @"при"; объект *s* — текущий получатель внутри метода.
6. Метод возвращает BOOL, он становится значением всего выражения.

Шаги 3–4 — та самая динамическая диспетчеризация. Метод выбирается **в момент выполнения**, по имени, а не зашит в программу заранее. Это и делает [] «отправкой сообщения», а не обычным вызовом функции.

Проверяем

Все четыре программы главы:


```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/05-messages.m      -o /tmp/t && /tmp/t
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/05-nil.m           -o /tmp/t && /tmp/t
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/05-array.m         -o /tmp/t && /tmp/t
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/05-introspection.m -o /tmp/t && /tmp/t
```

Каждая должна собраться **без предупреждений** (флаги `-Wall -Wextra` включают почти все предупреждения) и выдать показанный выше вывод — с твоими датой и временем в начале строк NSLog.

Частые ошибки

- **Сравнил строки через `==`.** `[a isEqualToString:b]` сравнивает текст; `a == b` — адреса. Две разные строки с одинаковыми буквами `==` сочтёт неравными. Для содержимого — всегда `isEqualToString:`.
- **Потерял двоеточие в имени метода.** `hasPrefix` и `hasPrefix:` — разные имена. Метод с аргументом всегда с двоеточием. Без него компилятор скажет, что такого метода нет.
- **Забыл вложить `alloc/init`.** Объект родится связкой `[[Класс alloc] init]`. Один `[Класс alloc]` без `init` даёт недонастроенный объект — так не пользуются.
- **Написал `id *obj`.** У `id` звёздочка уже внутри. Пиши `id obj`. А у конкретного класса звёздочка нужна: `NSString *s`.
- **Понадеялся, что `nil` выстрелит ошибкой.** Сообщение к `nil` молча даёт `0/nil/NO`, программа не падает. Если объект неожиданно оказался `nil`, баг проявится позже. Где результат важен — проверяй `if (x == nil)`.
- **Перепутал индекс с длиной.** Элементы нумеруются с 0, поэтому последний — `[arr objectAtIndex:[arr count] - 1]`. Запрос `objectAtIndex:` за пределами массива роняет программу (это уже не `nil`-случай, а реальная ошибка диапазона).
- **`%@` против `%lu`.** Объект печатают через `%@`, число — через `%d/%lu`. `NSUInteger` безопаснее привести к `(unsigned long)` и печатать `%lu`.

Упражнения

1. Возьми строку `@\"Objective-C\"` и выведи её длину, нижний регистр (`lowercaseString`) и ответ на `[s hasSuffix:@\"-C\"]`.

2. Склеить три строки в одну через `stringByAppendingString:` (вложи сообщения друг в друга) и выведи результат.
3. Заведи `NSMutableArray`, добавь в него пять чисел-строк, выведи количество и средний по индексу элемент.
4. Создай `NSMutableDictionary` с тремя парами «город → страна» через `setObject:forKey:` и достань одно значение по ключу.
5. Присвой `NSArray *arr = nil;` и пошли ему `[arr count]`. Убедись, что программа не падает и печатает 0. Объясни словами, почему.
6. Через `respondsToSelector:` и `@selector(...)` проверь, отвечает ли на `addObject:` объект-массив (созданный через `[[NSMutableArray alloc] init]`), а затем — объект-строка. Выведи оба ответа. Спрашивай именно объекты: `[NSMutableArray respondsToSelector:...]` спросит сам класс, и ответ будет другим.

Что мы получили

Это была опорная глава: теперь ты понимаешь главный механизм языка.

- **Класс** — чертёж, **объект** — экземпляр по этому чертежу.
- [получатель сообщение] — отправка сообщения. Получатель, селектор (имя со всеми двоеточиями), аргументы после двоеточий.
- Аргументы именованные: `setObject:forKey:` — это полное имя метода; по числу двоеточий видно число аргументов.
- Объект рождается связкой `[[Класс alloc] init]`; сообщения можно **вкладывать** друг в друга.
- `id` — любой объект; `nil` — отсутствие объекта; **сообщение к `nil` безопасно** и даёт нулевой результат.
- Метод выбирается **во время выполнения** (динамическая диспетчеризация) — поэтому это «сообщение», а не «вызов функции».
- Объект знает себя: `isKindOfClass:`, `respondsToSelector:`.

И главный контраст с Си: там код вызывает функции над пассивными данными по заранее известным адресам; здесь активные объекты получают сообщения и сами в рантайме решают, как ответить.

Пока мы пользовались чужими объектами Foundation. В главе 6 напомним **свой первый класс** — то есть начертим собственный чертёж и научим его объекты отвечать на наши сообщения.

Документация Apple

- Programming with Objective-C → Working with Objects (объекты, отправка сообщений, nil, id) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/WorkingwithObjects/WorkingwithObjects.html
- NSString — developer.apple.com/documentation/foundation/NSString
- NSArray / NSMutableArray — developer.apple.com/documentation/foundation/NSArray и developer.apple.com/documentation/foundation/NSMutableArray
- NSObject Protocol (методы isKindOfClass:, respondsToSelector:) — developer.apple.com/documentation/objectivec/NSObjectProtocol

Глава 6. Свой первый класс

В главе 5 мы посылали сообщения чужим объектам — строкам `NSString`, которые кто-то написал за нас. Теперь напишем свой собственный тип. Мы построим класс `Counter` — маленький счётчик, который умеет увеличиваться, уменьшаться и сообщать своё значение. Соберём его с нуля, запустим, найдём в нём шероховатость и поправим. По дороге разберём каждое новое слово: `@interface`, `@implementation`, `@end`, минус и плюс перед методом, `self` и фигурные скобки с переменными.

Что мы сделаем

- Объявим свой класс двумя блоками: `@interface` (что класс умеет) и `@implementation` (как он это делает).
- Заведём в нём внутреннюю переменную — **ivar** (instance variable).
- Напишем **методы экземпляра** (со знаком `-`) и **метод класса** (со знаком `+`).
- Научимся вызывать собственный метод объекта через `[self ...]`.
- Создадим объект `Counter` и погоняем его в `main`.
- Сделаем заодно класс `Person` с именем и приветствием.

Что такое класс

Объект из главы 5 — это конкретная строка `@привет`. А `NSString` — это **класс**: общий чертёж, по которому такие строки делаются. Класс описывает две вещи сразу:

- какие **данные** хранит каждый объект этого типа (у строки — её байты);
- какое **поведение** у него есть, то есть на какие сообщения он отвечает (`length`, `uppercaseString`, `hasPrefix:`).

Наш `Counter` будет хранить одно число и отвечать на сообщения «увеличься», «уменьшись», «назови значение».

Отличие от Си. В Си данные и поведение живут порознь. Данные — это `struct Counter { int value; };`, отдельный тип. Поведение — это свободные функции `void increment(struct Counter *c)`, которые лежат где-то рядом и сами по себе со структурой никак не связаны. Класс Objective-C **склеивает** их в один тип: число и методы, которые с ним работают, объявлены в одном месте и принадлежат друг другу. Это и есть главная мысль объектного программирования.

Разбираем задачу руками

Представь счётчик на турникете. У него внутри одно число. Снаружи к нему три действия: «+1», «-1» и «сколько сейчас». Само число наружу не торчит — к нему добиваются только через эти действия. Вот это «внутри одно число, снаружи три действия» мы и переведём на Objective-C.

Класс мы опишем двумя отдельными блоками. Так в Objective-C принято всегда, и сразу объясню зачем.

Два блока: `@interface` и `@implementation`

Описание класса делится на две части.

`@interface` — это **объявление**: список того, что класс умеет. Своего рода оглавление. Здесь перечислены имена методов и типы, но нет их тел — нет кода, который реально что-то делает. По этому блоку и компилятор, и другой программист понимают, какие сообщения объекту можно слать, ничего не зная про внутреннюю кухню.

`@implementation` — это **реализация**: тела всех методов, настоящий код. Здесь написано, что именно происходит, когда объект получает то или иное сообщение.

Оба блока обязательно закрываются словом **`@end`**.

Зачем делить на два? Чтобы отделить «что» от «как». Кто пользуется классом — читает только `@interface`, короткий список возможностей; кто меняет класс изнутри — лезет в `@implementation`. На практике блоки кладут в **разные файлы**: `@interface` — в заголовок `.h`, `@implementation` — в `.m`; заголовок подключают через `#import`. Но никто не запрещает положить оба блока в **один .m** — для маленькой программы так удобнее. Именно так мы и сделаем всю эту главу.

Отличие от Си. Деление на «объявление в `.h` и реализацию в `.c`» тебе знакомо по Си: там в заголовке пишут *прототипы* функций, а тела — в `.c`. Идея та же. Новое — что в Objective-C объявление и тела сгруппированы вокруг класса (`@interface Counter / @implementation Counter`), а не разбросаны как независимые функции.

Шаг 1. Объявляем класс

Начнём с блока `@interface`. Создай файл `06-counter.m` и впиши:

```
#import <Foundation/Foundation.h>

@interface Counter : NSObject
{
    NSInteger _value;    /* внутреннее число счётчика */
}
- (void)increment;
- (void)decrement;
- (NSInteger)value;
@end
```

Разберём построчно.

`@interface Counter : NSObject` — открываем объявление класса с именем **Counter**. Двоеточие и `NSObject` после него значат: «Counter наследует от `NSObject`». `NSObject` — это **базовый класс** почти всех объектов в Objective-C; от него наш счётчик бесплатно получает умение создаваться в памяти (`alloc`) и инициализироваться (`init`). Подробно наследование — в главе 9; пока просто всегда пиши : `NSObject` после имени своего класса, без этого объект не из чего будет создать.

Дальше — фигурные скобки `{ }`, а в них объявления переменных:

```
{
    NSInteger _value;
}
```

Это **инстанс-переменные**, или **ivars** (instance variables) — данные, которые хранит каждый объект класса. У каждого отдельного счётчика будет своя личная копия `_value`. `NSInteger` — это целочисленный тип Foundation (на современных Mac — 64-битное знаковое целое, то есть `long`, «большой `int`»). Имя `_value` начинается с подчёркивания: это давняя традиция Objective-C — пометать `ivars` ведущим `_`, чтобы их сразу было видно в коде. Подчёркивание — не магия, просто часть имени.

Отличие от Си. Эти фигурные скобки с переменными — почти буквально си-шный `struct`: поля внутри `{ }`. Разница в том, что в Си структура хранит **только** данные, а здесь рядом, в том же `@interface`, объявлено и поведение — методы. В Си `struct` и функции, которые его обрабатывают, ничем формально не связаны; здесь они — части одного класса.

Теперь три строки с минусом:

```
- (void)increment;  
- (void)decrement;  
- (NSInteger)value;
```

Это **объявления методов**. Прочитаем первое: `-(void)increment`;

- Ведущий знак `-` (минус) говорит: это **метод экземпляра** (instance method). Он работает с конкретным объектом — конкретным счётчиком. Слать его будем так: `[c increment]`.
- `(void)` в скобках — **тип возвращаемого значения**. `void` значит «ничего не возвращает»: метод просто делает дело. У третьего метода в скобках стоит `(NSInteger)` — он вернёт число.
- `increment` — **имя метода** (он же селектор из главы 5).
- Точка с запятой `;` закрывает объявление. Тела пока нет — мы только обещаем, что такой метод будет.

Отличие от Си. Этот ведущий `-` — не вычитание и не «минус число». Это часть синтаксиса объявления метода, которой в Си нет вообще. В Си функцию объявляют так: `void increment(void);` — без всякого знака впереди. Минус (для методов экземпляра) и плюс (для методов класса, увидим ниже) — чисто objc-пометки, кто получатель сообщения.

`@end` закрывает блок. Объявление готово: мы пообещали миру, что у `Counter` есть три метода. Но тел ещё нет — если сейчас собрать, методы будут пустыми обещаниями. Добавим реализацию.

Шаг 2. Пишем реализацию

Сразу под `@end` от интерфейса добавь второй блок:

```
@implementation Counter  
  
- (void)increment {  
    _value = _value + 1;  
}  
  
- (void)decrement {  
    _value = _value - 1;  
}  
  
- (NSInteger)value {  
    return _value;  
}
```

```
}
```

```
@end
```

`@implementation Counter` — открываем реализацию того же класса. Имя то же самое; `ivars` второй раз перечислять не надо — они уже объявлены в `@interface`.

Дальше — те же три метода, но теперь с **телами** в фигурных скобках:

- `increment` берёт `_value`, прибавляет 1 и кладёт обратно. Заметь: внутри метода мы обращаемся к `_value` **напрямую по имени**, без всяких точек и стрелок. Метод экземпляра всегда работает «изнутри» своего объекта, и его `ivars` ему видны просто по имени.
- `decrement` — то же, но минус 1.
- `value` ничего не меняет, а возвращает текущее число: `return _value;`. Тип возврата (`NSInteger`) совпадает с тем, что мы обещали в `@interface` — иначе компилятор поругается.

`@end` закрывает реализацию.

Шаг 3. Создаём объект и запускаем

Класс описан. Теперь сделаем по нему настоящий объект и погоняем. Ниже второго `@end` добавь `main`:

```
int main(void) {
    @autoreleasepool {
        Counter *c = [[Counter alloc] init];
        [c report];
        [c increment];
        [c increment];
        [c increment];
        [c report];
    }
    return 0;
}
```

Стоп — я нарочно вписал `[c report]`, метода `report` у нас ещё нет. Сначала разберём строку создания объекта, потом соберём и увидим, как компилятор поймает нашу оплошность.

`Counter *c = [[Counter alloc] init];` — здесь рождается объект. Читается изнутри наружу:

- `[Counter alloc]` — посылаем сообщение **alloc** самому классу `Counter`. `alloc` (от *allocate*, выделить) находит в памяти место под один новый счётчик и обнуляет его. Все ivars при этом становятся нулём — наш `_value` стартует с 0 бесплатно.
- `[... init]` — новорождённому объекту шлём **init** (от *initialize*, привести в исходное состояние). Он возвращает готовый к работе объект.
- `Counter *c = ...` — кладём указатель на этот объект в переменную `c`. Звёздочка `*` обязательна: как и со строками, в руках мы держим не сам объект, а указатель на него (см. главу 5).

Пару `alloc/init` всегда пишут вместе: `[[Counter alloc] init]` — «выдели и инициализируй». Что именно делает `init` и как написать свой — разберём в главе 7; пока эта связка просто наш способ получить объект.

Отличие от Си. В Си объект в куче ты создавал так: `struct Counter *c = malloc(sizeof *c);` — и сам отвечал за `free(c)` в конце. Здесь `alloc/init` делают выделение и обнуление, а память уберёт за тебя ARC (флаг `-fobjc-arc`), без ручного `free`. Никакого `malloc` для объектов ты больше не пишешь.

Скомпилируем как есть, с ошибочным `report` (собираем твой файл `06-counter.m`; готовый `code/06-counter.m` уже исправлен и соберётся чисто):

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    06-counter.m -o /tmp/t
```

Компилятор остановится с ошибкой:

```
error: no visible @interface for 'Counter'
      declares the selector 'report'
[c report];
~ ^~~~~~
```

Он честно говорит: у класса `Counter` не объявлено сообщение `report` — посылать нечего. Так бывает, когда получатель имеет конкретный тип (`Counter *`): компилятор сверяется с `@interface` и не пропускает неизвестный метод. Это полезно — опечатку в имени метода ловят сразу, до запуска. Добавим `report` — заодно увидим `self`.

Шаг 4. Метод `report` и `self`

Допишем в `@interface` строку:

```
- (void)report;
```

А в `@implementation` — тело:

```
- (void)report {
    NSLog(@"счётчик = %ld", (long)[self value]);
}
```

Самое интересное здесь — **self**. Это указатель на **текущий объект** — на тот самый счётчик, которому прислали report. Когда мы пишем [self value], объект посылает сообщение value **сам себе** и получает своё же число. То есть метод может звать другие методы того же объекта через self.

Можно было написать и просто _value — ivar виден напрямую. Но через [self value] мы лиш- ний раз показываем приём «объект зовёт свой метод»; он понадобится постоянно.

Про (long) и %ld: NSLog со спецификатором %ld ждёт тип long. NSInteger совпадает с long по размеру, но чтобы не зависеть от платформы и не ловить предупреждений, число приводят к long явно — (long)[self value]. Это та же осторожность, что и в главе 5 с (unsigned long).

Отличие от Си. self похож на то, как в Си ты вручную передавал указатель на струк- туру первым аргументом: void increment(struct Counter *self), и внутри работал через self->value. Идея та же — «вот объект, над которым я работаю». Разница: в Objective-C self передаётся **неявно**, сам собой, в каждый метод экземпляра. Ты его не объявляешь и не передаёшь руками — он просто есть внутри любого метода.

Теперь программа соберётся без предупреждений. Запускаем /tmp/t:

```
2026-06-28 12:17:25.920 t[61148:1254679] счётчик = 0
2026-06-28 12:17:25.920 t[61148:1254679] счётчик = 3
```

Сначала 0 (только что созданный счётчик), потом 3 (три раза increment). Класс работает. Доба- вим ещё уменьшение и удобный способ создания.

Шаг 5. Метод класса +

Каждый раз писать [[Counter alloc] init] длинновато. Заведём короткий способ создать го- товый счётчик — **метод класса**. Допиши в @interface:

```
+ (Counter *)counter;
```

Знак + (плюс) вместо минуса означает: это **метод класса** (class method). Его шлют не объекту, а **самому классу** — слову Counter, ровно как alloc. Метод экземпляра (-) спрашивает у кон- кретного счётчика, метод класса (+) — у всего класса в целом, когда отдельного объекта ещё нет.

Тело в @implementation:

```
+ (Counter *)counter {  
    return [[Counter alloc] init];  
}
```

Метод создаёт объект и возвращает его. Тип возврата (Counter *) — «указатель на Counter». Теперь вместо [[Counter alloc] init] можно писать коротко [Counter counter]. Такие методы-«фабрики» очень распространены в Foundation: [NSString string], [NSArray array] — ровно тот же приём, метод класса со знаком +.

Отличие от Си. В Си нет понятия «метод, привязанный к типу, но не к конкретному значению». Ближайшая аналогия — обычная функция-фабрика struct Counter *counter_create(void), которая внутри делает malloc и возвращает указатель. Знак + в Objective-C делает её формально частью класса: вызов [Counter counter] адресован самому типу.

Полная программа

Вот итоговый code/06-counter.m целиком:

```
#import <Foundation/Foundation.h>  
  
/* Объявление класса: что Counter умеет. */  
@interface Counter : NSObject  
{  
    NSInteger _value;    /* ivar — внутреннее число счётчика */  
}  
- (void)increment;        /* метод экземпляра: увеличить на 1 */  
- (void)decrement;        /* метод экземпляра: уменьшить на 1 */  
- (NSInteger)value;        /* метод экземпляра: вернуть число */  
- (void)report;            /* метод экземпляра: напечатать число */  
+ (Counter *)counter;    /* метод класса: создать готовый счётчик */  
@end  
  
/* Реализация класса: как Counter это делает. */  
@implementation Counter  
  
- (void)increment {  
    _value = _value + 1;  
}
```

```
- (void)decrement {
    _value = _value - 1;
}

- (NSInteger)value {
    return _value;
}

- (void)report {
    NSLog(@"счётчик = %ld", (long)[self value]);
}

+ (Counter *)counter {
    return [[Counter alloc] init];
}

@end

int main(void) {
    @autoreleasepool {
        Counter *c = [[Counter alloc] init];
        [c report];           /* 0 */
        [c increment];
        [c increment];
        [c increment];
        [c report];           /* 3 */
        [c decrement];
        [c report];           /* 2 */

        Counter *c2 = [Counter counter];
        [c2 increment];
        [c2 report];          /* 1 */
    }
    return 0;
}
```

Как это работает

Пройдём по main. `[[Counter alloc] init]` создаёт объект с `_value = 0` и кладёт указатель в `c`. `[c report]` шлёт счётчику сообщение `report`; внутри объект зовёт `[self value]` (сам себя), получает 0 и печатает счётчик = 0. Три `increment` поднимают `_value` до 3, `decrement` опускает до 2 — `report` печатает их. Наконец `[Counter counter]` — сообщение **классу**: метод `+counter` создаёт второй, независимый счётчик `c2` со своим `_value = 0`, а `increment` поднимает его до 1.

Важное: `c` и `c2` — два разных объекта, у каждого свой `_value`. Изменение одного не трогает другой. В этом и смысл `ivars` — личные данные каждого экземпляра.

Проверяем

Файл главы — `code/06-counter.m`. Компиляция и запуск:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/06-counter.m -o /tmp/t && /tmp/t
```

Ожидаемый вывод (дата и числа в скобках будут свои):

```
2026-06-28 12:17:25.920 t[61148:1254679] счётчик = 0
2026-06-28 12:17:25.920 t[61148:1254679] счётчик = 3
2026-06-28 12:17:25.920 t[61148:1254679] счётчик = 2
2026-06-28 12:17:25.920 t[61148:1254679] счётчик = 1
```

Второй класс: Person

Закрепим на втором примере — классе `Person`. У него два `ivar` (имя и возраст) и методы, чтобы их задать, прочитать и поздороваться. Заодно увидим методы с параметром.

```
#import <Foundation/Foundation.h>

@interface Person : NSObject
{
    NSString *_name;    /* имя */
    NSInteger _age;     /* возраст */
}
- (void)setName:(NSString *)newName;
- (NSString *)name;
- (void)setAge:(NSInteger)newAge;
- (void)sayHello;
```

```
- (void)greet:(Person *)other;
@end
```

Новое тут — методы с **параметром**. Посмотри на `-(void)setName:(NSString *)newName;`:

- `-(void)` уже знакомы: метод экземпляра, ничего не возвращает.
- `setName:` — имя метода. Обрати внимание на **двоеточие** в конце: оно означает «дальше будет аргумент». Двоеточие — часть имени метода. Имя целиком так и звучит: «setName:» с двоеточием (см. главу 5 про селекторы).
- `(NSString *)` — **тип параметра** в скобках: метод ждёт строку.
- `newName` — **имя параметра**: так аргумент зовётся внутри тела метода.

То есть строка читается: «метод `setName:`, принимает аргумент типа `NSString *` (внутри он доступен под именем `newName`), ничего не возвращает». Метод `greet:` так же принимает другого человека — `-(void)greet:(Person *)other`, тип параметра `Person *` (указатель на наш же класс), чтобы поздороваться с объектом того же типа.

Реализация:

```
@implementation Person

- (void)setName:(NSString *)newName {
    _name = newName;
}

- (NSString *)name {
    return _name;
}

- (void)setAge:(NSInteger)newAge {
    _age = newAge;
}

- (void)sayHello {
    NSLog(@"Привет! Меня зовут %@, мне %ld.", _name, (long)_age);
}

- (void)greet:(Person *)other {
    NSLog(@"%@ говорит: привет, %@!", [self name], [other name]);
}
```

```
@end
```

В `setName:` приходящий аргумент `newName` мы кладём в `ivar _name`. В `name` возвращаем его обратно. Пара «задать/прочитать» (`setName:` и `name`) — это ручные «аксессоры». В главе 8 мы узнаем про `@property` — он генерирует такие пары сам, и писать их руками больше не придётся. Пока полезно увидеть их вживую.

В `greet:` объект зовёт `[self name]` (своё имя) и `[other name]` (имя гостя) и печатает приветствие. Здесь хорошо видно: `self` — это «я», а `other` — «другой объект», переданный аргументом. Оба одного класса, оба отвечают на `name`.

`main:`

```
int main(void) {
    @autoreleasepool {
        Person *anna = [[Person alloc] init];
        [anna setName:@"Анна"];
        [anna setAge:30];

        Person *boris = [[Person alloc] init];
        [boris setName:@"Борис"];
        [boris setAge:25];

        [anna sayHello];
        [boris sayHello];
        [anna greet:boris];
        [boris greet:anna];
    }
    return 0;
}
```

Создаём двух людей, каждому задаём имя и возраст, затем они здороваются. Запуск `code/06-person.m` даёт (дату, время и служебные числа в скобках я убрал, чтобы строки влезли):

```
Привет! Меня зовут Анна, мне 30.
Привет! Меня зовут Борис, мне 25.
Анна говорит: привет, Борис!
Борис говорит: привет, Анна!
```

Немного про `init`

Пока мы пользовались `init`, который достался от `NSObject`: он просто оставляет все `ivars` нулевыми. Часто этого хватает. Но иногда хочется, чтобы объект рождался уже настроенным. Тогда пишут **свой** `init`. Минимальный вид такой (файл `code/06-counter-init.m`):

```
- (instancetype)init {
    self = [super init];    /* спросили базовый класс */
    if (self) {
        _value = 100;       /* наше стартовое значение */
    }
    return self;
}
```

Не пугайся, это пока без разбора — полностью `init` мы разложим в главе 7. Сейчас прочитай его как заклинание-шаблон и запомни форму:

- `(instancetype)` как тип возврата значит «вернётся объект этого же класса». Удобный точный тип для инициализаторов; подробнее — в главе 7.
- `self = [super init]` просит базовый класс (`NSObject`) сделать свою часть подготовки; результат кладём в `self`. Слово `super` («базовый класс») разберём в главе 9.
- `if (self) { ... }` — если подготовка удалась, настраиваем свои `ivars` (тут ставим `_value = 100`).
- `return self;` — отдаём готовый объект.

С таким `init` строка `[[Counter alloc] init]` даст счётчик, стартующий со 100. Проверь сам: `code/06-counter-init.m` печатает `старт = 100`, а после одного `increment` — после `++` = 101.

Частые ошибки

- **Забыл `@end`.** Каждый из блоков `@interface` и `@implementation` закрывается своим `@end`. Пропустишь — компилятор посыплет странными ошибками, часто указывая совсем не на ту строку.
- **Перепутал `-` и `+`.** Метод экземпляра — минус, метод класса — плюс. Объявил `+(void)increment`, а зовёшь `[c increment]` у объекта — компилятор не найдёт метод.
- **Не совпали типы в `@interface` и `@implementation`.** Объявил `-(NSInteger)value`, а в реализации написал `-(void)value` — компилятор поругается на несоответствие. Сигнатуры должны совпадать точь-в-точь.

- **Объявил метод, но забыл реализовать.** Тогда @interface обещает метод, которого в @implementation нет. Это не ошибка, а предупреждение — и указывает оно на строку @implementation, а не на место вызова:

```
warning: method definition for 'report' not found
      [-Wincomplete-implementation]
@interface Counter
    ^
note: method 'report' declared here
- (void)report;
^
```

Заметь: собраться такой файл всё-таки сможет — это всего лишь предупреждение. Зато при отправке [с report] программа рухнет уже во время работы:

```
*** Terminating app due to uncaught exception 'NSInvalidArgumentException',
reason: '-[Counter report]: unrecognized selector sent to instance 0x600000270070'
```

Вот почему предупреждения не игнорируют: тут компилятор буквально предсказал падение за один шаг до него. Кстати, запомни формулировку unrecognized selector sent to instance — это самый частый краш в Objective-C, и означает он ровно одно: объекту прислали сообщение, которого он не знает.

- **Шлешь несуществующее сообщение.** [с report] до того, как report добавлен в @interface, у типизированного получателя (Counter *) даёт ошибку no visible @interface ... declares the selector 'report'. Читай её — она прямо называет пропущенный метод.
- **Забыл * у объектного типа.** Counter с вместо Counter *с — ошибка: объект держат только через указатель.

Упражнения

1. Добавь в Counter метод – (void)reset;, который ставит _value в 0. Вызови его в main после нескольких increment и проверь вывод.
2. Добавь метод класса + (Counter *)counterAtThree;, который создаёт счётчик и сразу делает ему три increment. (Подсказка: создай объект, пошли ему сообщения, верни его.)
3. В Person сделай метод – (BOOL)isAdult;, возвращающий YES, если _age не меньше 18, иначе NO. Напечатай результат для Анны и для ребёнка 10 лет.
4. Добавь в Person метод – (void)haveBirthday;, который увеличивает _age на 1, а внутри зовёт [self sayHello] через self. Проверь, что возраст вырос.

5. Создай в `main` массив из трёх счётчиков (обычный `си`-массив `Counter *arr[3]`), создай каждый через `[Counter counter]` и в цикле увеличь i -й счётчик i раз. Распечатай все три значения.

Что мы получили

Мы написали первый собственный класс — с нуля и в движении. Теперь ты знаешь, что класс — это объявление (`@interface`) плюс реализация (`@implementation`), оба закрыты `@end`; что данные хранятся в `ivars` внутри фигурных скобок, а поведение — в методах; что `-` помечает метод экземпляра, `+` — метод класса; что двоеточие в имени метода вводит параметр; и что `self` — это «текущий объект», через который объект зовёт свои же методы. Объект мы пока создаём связкой `[[Класс alloc] init]`.

Слабое место осталось: пары методов вроде `setName:/name` мы писали руками, а `init` пощупали только краем. В главе 7 разберём `init` и `dealloc` по-настоящему — как объект правильно рождается и что происходит при его уходе. А в главе 8 увидим `@property`, который избавит от рукописных аксессоров.

Документация Apple

- Programming with Objective-C → «Defining Classes» (объявление классов, `@interface/@implementation`, методы экземпляра и класса) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/DefiningClasses/DefiningClasses.html
- `NSObject` — developer.apple.com/documentation/objectivec/nsobject (базовый класс, `alloc`, `init`).
- `NSInteger` — developer.apple.com/documentation/objectivec/nsinteger

Глава 7. Создание объектов: alloc, init и dealloc

В прошлой главе ты написал свой класс: объявил его в `@interface`, описал методы в `@implementation`, спрятал внутри данные в `ivars`. Но один вопрос мы там обошли стороной: **как объект вообще появляется на свет?** Откуда берётся `self`, в который мы складываем данные? И что происходит, когда объект больше не нужен — кто и когда убирает его из памяти?

Эта глава — про рождение и смерть объекта. Мы разберём два шага создания (`alloc` и `init`), научимся писать собственный инициализатор — и без аргументов, и с аргументами, — поймём странную на вид строчку `self = [super init]`, добавим удобные фабричные методы и заглянем в `dealloc`, чтобы своими глазами увидеть момент, когда объект исчезает.

Что мы сделаем

- Поймём, почему объект создаётся в **два шага**: `alloc` и `init`.
- Разберём паттерн `self = [super init]; if (self) { ... } return self;` по знакам — зачем присваивать `self`, зачем проверка, что делает `super`.
- Напишем инициализатор без аргументов, потом с аргументами (`initWithName:age:`).
- Узнаем про тип `instancetype` и почему он лучше `id` для `init`-методов.
- Введём **назначенный инициализатор** (`designated initializer`) и вторичные инициализаторы, которые к нему сводятся.
- Добавим **фабричный метод класса** `+personWithName:age:`.
- Заглянем в `dealloc` и увидим в выводе, в какой именно момент объект уничтожается.

Разбираем задачу руками

Представь, что объект — это бланк анкеты. Чтобы получить заполненную анкету, нужно сделать две разные вещи.

Сначала — **взять чистый бланк**: лист нужного размера с готовыми графами «имя», «возраст», пустой, но правильной формы. В мире объектов это делает alloc: просит у системы кусок памяти под объект и обнуляет его — все поля становятся нулями.

Потом — **заполнить бланк**: вписать имя, проставить возраст, выставить значения по умолчанию. Это делает init: берёт сырой объект от alloc и доводит его до рабочего состояния.

Два дела — два шага. Поэтому полное создание объекта в Objective-C всегда выглядит как два сообщения подряд:

```
Person *p = [[Person alloc] init];
```

Читается изнутри наружу: сначала [Person alloc] (взять бланк), потом этому бланку посылается init (заполнить). Результат — готовый объект, адрес которого мы кладём в переменную p.

Отличие от Си. В Си «создать объект» — это два ручных действия, которые ты делаешь сам и легко забываешь. Сначала malloc(sizeof(...)) выделяет память (и не обнуляет её — внутри мусор), потом ты вручную расставляешь значения в полях структуры. В Objective-C обе операции формализованы и названы: alloc гарантированно обнуляет память, init расставляет значения. А освободить память тебе вообще не придётся — этим займётся ARC. Меньше шагов руками — меньше споров ошибиться.

Шаг 1. Самый первый класс с init

Возьмём знакомого из прошлой главы человека. У него два поля-ivar: имя и возраст. Свойств (@property) мы ещё не проходили — это следующая глава, — поэтому работаем с ivars напрямую, как в главе 6.

Начнём с инициализатора **без аргументов**: пусть свежий человек получает имя «Без имени» и возраст 0.

```
#import <Foundation/Foundation.h>
```

```
@interface Person : NSObject {
    NSString *_name;
    NSInteger _age;
}
- (instancetype)init;
- (void)describe;
@end
```

```
@implementation Person

- (instancetype)init {
    self = [super init];
    if (self) {
        _name = @"Без имени";
        _age = 0;
    }
    return self;
}

- (void)describe {
    NSLog(@"Человек: %@, возраст %ld", _name, (long)_age);
}

@end
```

Здесь много нового синтаксиса. Разберём построчно.

@interface Person : NSObject { ... }

Объявляем класс Person, наследник NSObject (двоеточие = «наследует от»). В фигурных скобках — ivars: `_name` типа `NSString *` и `_age` типа `NSInteger`. Подчёркивание в начале имени — общепринятое соглашение для ivars; так их видно с первого взгляда. Всё это мы разбирали в главе 6.

`NSInteger` — это целочисленный тип Foundation, подстраивающийся под разрядность системы: на 64-битных это `long`, на 32-битных — `int`. Под него в `NSLog` идёт спецификатор `%ld`, и значение принято приводить к `(long)` — чтобы тип точно совпал на любой платформе и компилятор не ругался.

- (instancetype)init;

Объявляем метод экземпляра (минус — означает «метод объекта, не класса»). Имя метода — `init`. Возвращаемый тип — `instancetype`. На нём остановимся отдельно, чуть ниже; пока читай его как «вернётся объект этого же класса».

```
self = [super init];
```

Самая загадочная строка. Разберём её по частям, потому что ты будешь писать её в каждом инициализаторе до конца жизни.

`super` — это **тот же объект, но рассматриваемый как его родитель**. Пишешь `[super init]` — посылаешь сообщение `init` родительскому классу (для нас — `NSObject`). Зачем? Родитель тоже настраивает свою часть объекта: у `NSObject` внутри есть служебные поля (например, `isa` — указатель на класс, дойдём до него в главе 14), и только сам `NSObject` знает, как их правильно выставить. Поэтому первым делом даём родителю сделать свою работу.

Отличие от Си. В Си нет ни наследования, ни `super`: если структура «расширяет» другую, ты сам копируешь поля и сам помнишь, что инициализировать. В Objective-C `[super init]` встроен в язык — цепочка инициализации поднимается от твоего класса до `NSObject`, каждый слой настраивает себя сам. Тебе достаточно вежливо позвать `init` родителя первым.

`[super init]` **возвращает** настроенный объект. И вот тонкость: иногда родитель может вернуть **не тот же самый** объект, что пришёл от `alloc`, а другой (или вообще `nil`, если настройка не удалась). Поэтому результат обязательно нужно сохранить обратно в `self`:

```
self = [super init];
```

`self` — это указатель на «текущий объект», на котором выполняется метод. Присваивая ему результат `[super init]`, мы говорим: «дальше работаем именно с тем объектом, который вернул родитель». Если этого не сделать и продолжить настраивать старый `self`, в редких случаях получишь работу с неправильным объектом. Поэтому правило железное: **результат `[super init]` всегда кладём в `self`**.

```
if (self) { ... }
```

Проверка: а вернул ли родитель вообще объект? Если `[super init]` отдал `nil` (настройка провалилась), то `self` равен `nil`, и лезть внутрь — писать в его поля — бессмысленно и опасно. Запись `if (self)` — сокращение для `if (self != nil)`: непустой указатель в условии считается «истиной», `nil` — «ложью».

Только когда объект действительно есть, мы заполняем его поля:

```
_name = @"Без имени";  
_age = 0;
```

```
return self;
```

Возвращаем готовый объект тому, кто нас вызвал — именно `self`, тот объект, что прошёл через `[super init]`. Писавший `[[Person alloc] init]` получит этот указатель в свою переменную.

Собранный воедино, шаблон почти не меняется от класса к классу — заучи его как стихотворение, дальше будем лишь подставлять внутрь разную настройку:

```
- (instancetype)init {
    self = [super init];
    if (self) {
        /* настройка своих ivars */
    }
    return self;
}
```

Шаг 2. Почему именно `instancetype`, а не `id`

Мы написали `-(instancetype)init`. Почему не `id`, который ты уже знаешь из главы 5 как «указатель на любой объект»?

`id` означает «какой-то объект, какого именно класса — неизвестно». `instancetype` означает строго «объект **того же класса**, у которого вызвали метод». Раз `init` возвращает `instancetype`, компилятор знает: в `Person *p = [[Person alloc] init]` лежит именно `Person`, разрешит звать у `p` методы `Person` и предупредит об опечатке.

Ещё важнее это с наследованием. Если `Student` — наследник `Person`, то `[[Student alloc] init]` благодаря `instancetype` вернёт тип `Student`, а не `Person`.

Честная оговорка: именно для `init` (и для `alloc`, `new`) clang выводит тип сам, даже если написать `-(id)init` — эти семейства методов он знает по имени. А вот у своих фабрик вроде `+(id)personNamed:` с `id` проверка пропадает: `NSArray *a = [Person personNamed:@"Аня"]` соберётся молча. Поэтому правило простое — пиши `instancetype` везде, и у `init`, и у фабрик; так тип виден и компилятору, и читателю.

Отличие от Си. В Си тип возврата жёстко записан в объявлении: функция объявлена как `struct Person *` — её и вернёт, у кого бы её ни звали. `instancetype` — тип «по месту вызова»: компилятор подставляет класс того, кому послали сообщение. `[Person alloc] → Person *`, `[Student alloc] → Student *`. Работает это целиком **при компиляции** — в готовой программе никакого `instancetype` нет, runtime про него не знает. В Си такой подстановки нет, как нет и `id`.

Правило простое: для **init-методов и фабричных методов всегда пишиinstancetype**. Это и есть рекомендация Apple. id оставь для случаев, когда тип объекта действительно заранее неизвестен.

Шаг 3. Запускаем и смотрим на рождение объекта

Добавим к классу простой main, создадим человека и попросим описать себя. Полный файл — code/07-person.m (с dealloc, до которого дойдём через абзац). Компилируем и запускаем:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/07-person.m -o person
./person
```

Вывод:

```
--- до создания объекта ---
Человек: Без имени, возраст 0
dealloc: объект Без имени уничтожен
--- объект уже уничтожен ---
```

(Дату, имя программы и числа в квадратных скобках от NSLog я здесь и дальше убираю, чтобы не мешали; у тебя они будут свои.)

Вторая строка — человек описал себя: имя и возраст ровно как мы выставили в init. Объект родился, ожил, ответил на describe. А что за строчка dealloc перед последней? К ней и переходим.

Шаг 4. dealloc — момент смерти объекта

У рождения есть пара — уничтожение. Когда объект больше никому не нужен, система убирает его из памяти. Прямо перед этим она посылает объекту сообщение dealloc — последний шанс прибраться за собой (закрыть файл, освободить какой-то ресурс). Это зеркало init: init настраивает, dealloc сворачивает.

Добавим dealloc в наш класс — пусть он печатает строчку, чтобы мы видели момент смерти:

```
- (void)dealloc {
    NSLog(@"dealloc: объект %@ уничтожен", _name);
}
```

Тип возврата — void: dealloc ничего не возвращает. Внутри мы просто логируем. И тут важнейший момент про ARC.

Под ARC ты не зовёшь release руками и не зовёшь [super dealloc]. В старом Objective-C (без ARC) в dealloc приходилось вручную отпускать свои объекты и в конце звать [super dealloc]. С ARC компилятор вставляет это за тебя. Твой dealloc нужен, только если есть что прибрать **сверх** обычных ivar; часто он не нужен совсем — мы пишем его ради учебного NSLog. (Полная история про ARC, retain/release — глава 13; пока достаточно знать, что память объектов под ARC ведёт компилятор.)

Отличие от Си. В Си ты сам зовёшь free(p), когда решишь, и сам отвечаешь за каждый забытый или повторный free. Под ARC момент уничтожения выбирает система — когда на объект не осталось ссылок, — а ты лишь можешь вклиниться через dealloc. Освобождение памяти (аналог free) отобрали ради безопасности и отдали компилятору.

Когда же именно срабатывает dealloc? Когда на объект не остаётся ни одной ссылки. В нашей программе хитрость в том, как устроен main:

```
int main(void) {
    @autoreleasepool {
        NSLog(@"--- до создания объекта ---");

        @autoreleasepool {
            Person *p = [[Person alloc] init];
            [p describe];
        } /* здесь p выходит из области видимости → dealloc */

        NSLog(@"--- объект уже уничтожен ---");
    }
    return 0;
}
```

Объект создаётся внутри **вложенного** @autoreleasepool. Переменная p живёт только до закрывающей } этого блока. Как только выполнение доходит до неё, на объект не остаётся ссылок, ARC его освобождает — и мы видим dealloc в выводе **между** двумя соседними строками --- ... --- (см. вывод выше).

Без вложенного блока объект дождался бы до конца main — менее наглядно. Работает здесь именно пара фигурных скобок, ограничивающая жизнь p: обычный блок { ... } без слова @autoreleasepool дал бы тот же вывод. Вложенный блок (или присваивание p = nil) — стандартный учебный способ **увидеть** момент уничтожения. В реальном коде ты редко думаешь о dealloc вообще: ARC всё делает молча.

Шаг 5. Инициализатор с аргументами

«Без имени» — скучно. Хочется создавать человека сразу с конкретным именем и возрастом. Для этого пишут инициализатор **с аргументами**:

```
- (instancetype)initWithName:(NSString *)name age:(NSInteger)age {
    self = [super init];
    if (self) {
        _name = [name copy];
        _age = age;
    }
    return self;
}
```

Разберём имя метода: `initWithName:age:`. В Objective-C аргументы вплетаются прямо в имя. Двоеточие `:` — метка «здесь будет аргумент». После `initWithName:` идёт `(NSString *)name` — параметр `name` типа `NSString *`. После `age:` идёт `(NSInteger)age`. Полное имя метода (селектор) — `initWithName:age:`, и зовут его так:

```
Person *p = [[Person alloc] initWithName:@"Аружан" age:24];
```

Имя по соглашению начинается с `init`, дальше `WithName:` — это словесная подпись к первому аргументу, чтобы на месте вызова было читаемо.

Тело — тот же знакомый шаблон: `self = [super init]`, проверка, настройка, `return self`. Но визна только в одной строке:

```
_name = [name copy];
```

Почему `copy`, а не просто `_name = name`? `NSString` бывает изменяемым (`NSMutableString` — дойдём до строк в главе 15). Сохрани мы чужую ссылку, а строку снаружи поменяй — наше `_name` молча изменилось бы вместе с ней. `copy` снимает независимую копию, которую снаружи уже не тронут. Стандартная предосторожность для строк; подробно — в главе 8 про свойства.

Шаг 6. Назначенный и вторичные инициализаторы

Часто у класса несколько инициализаторов — для удобства: и с именем и возрастом, и только с именем, и вообще без всего. Но если в каждом дублировать `[super init]` и настройку полей, легко наплодить расхождений: в одном забыл скопировать строку, в другом выставил не тот возраст.

Решение — **назначенный инициализатор** (designated initializer). Это главный, «настоящий» инициализатор класса: только он делает всю работу — зовёт `[super init]` и настраивает **все**

ivars. Все остальные инициализаторы — **вторичные** (secondary): они ничего не настраивают сами, а лишь подставляют значения по умолчанию и зовут назначенный.

У нашего человека назначенный — самый полный, initWithName:age:. А вторичные сводятся к нему:

```
/* Назначенный: единственный, кто зовёт [super init] и трогает ivars. */
- (instancetype)initWithName:(NSString *)name age:(NSInteger)age {
    self = [super init];
    if (self) {
        _name = [name copy];
        _age = age;
    }
    return self;
}

/* Вторичный: только имя, возраст по умолчанию 0. */
- (instancetype)initWithName:(NSString *)name {
    return [self initWithName:name age:0];
}

/* Вторичный: совсем без данных. */
- (instancetype)init {
    return [self initWithName:@"Без имени" age:0];
}
```

Главное отличие в одном слове. Назначенный зовёт **[super init]** — поднимается к родителю. Вторичные зовут **[self initWithName:age:]** — идут «вбок», к назначенному инициализатору **своего же** класса. Заметь: [self init...], не [super init...]. Вторичный перенаправляет работу на назначенный, и только назначенный — вверх, к родителю.

Правило цепочки (так его формулирует Apple):

- **вторичный инициализатор всегда зовёт другой инициализатор того же класса** ([self ...]), в итоге дотягиваясь до назначенного;
- **назначенный инициализатор зовёт назначенный инициализатор родителя** ([super init]).

Так вся настройка стекается в одну точку — назначенный инициализатор. Надо поменять правило заполнения полей? Меняешь в одном месте. Это убирает дублирование и расхождения.

Пока «назначенный» — только наша с тобой договорённость: из кода не видно, какой из трёх инициализаторов главный, и компилятор за этим не следит. Современный Objective-C умеет это исправить — назначенный инициализатор помечают в @interface макросом `NS_DESIGNATED_INITIALIZER`:

```
- (instancetype)initWithName:(NSString *)name
                        age:(NSInteger)age NS_DESIGNATED_INITIALIZER;
```

После такой пометки компилятор начинает проверять правило цепочки за тебя и ругается, если вторичный инициализатор полез к `[super init]` в обход назначенного. Разбираем этот макрос вместе с остальными современными пометками в главе 22 — там он встанет в один ряд с `nullable`, `NS_ENUM` и дженериками.

Полная программа — `code/07-person-name.m`. Запускаем:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/07-person-name.m -o person-name
./person-name
```

Вывод:

```
Человек: Аружан, возраст 24
Человек: Бекжан, возраст 0
Человек: Без имени, возраст 0
dealloc: объект Без имени уничтожен
dealloc: объект Бекжан уничтожен
dealloc: объект Аружан уничтожен
--- пул закрыт, все объекты освобождены ---
```

Три человека созданы тремя разными инициализаторами, но настроены через один назначенный. «Бекжан» получил возраст 0 от вторичного `initWithName:`, «Без имени» — оба значения по умолчанию от `init`. А когда вложенный пул закрылся, все три объекта по очереди получили `dealloc` — три строчки смерти подряд.

Шаг 7. Фабричный метод класса

`[[Person alloc] initWithName:@"Аружан" age:24]` — правильно, но многословно. Foundation любит давать короткие **фабричные методы класса**: вспомни `[NSString stringWithFormat:...]` — там нет ни `alloc`, ни `init`, объект приходит готовым. Сделаем такой же для `Person`. Фабричный метод — это метод **класса** (знак `+`, а не `-`), который сам внутри делает `alloc` и `init` и отдаёт готовый объект:

```
+ (instancetype)personWithName:(NSString *)name age:(NSInteger)age {
    return [[self alloc] initWithName:name age:age];
}
```

Знак + означает «метод класса»: его шлют самому классу Person, не экземпляру. Внутри — `[[self alloc] initWithName:name age:age]`. В методе класса `self` означает **сам класс**, поэтому `[[self alloc]` выделяет память под объект этого класса. Почему `[[self alloc]`, а не `[Person alloc]`? Чтобы у наследника фабрика порождала наследника: если `Student` унаследует метод, `[Student personWithName:...]` создаст `Student`. Именно поэтому тип возврата — `instancetype`.

Зовут фабрику без `alloc` — коротко и читаемо:

```
Person *b = [Person personWithName:@"Бекжан" age:31];
```

По соглашению Apple имя фабричного метода начинается с имени типа в нижнем регистре: `Person` → `personWithName:`, `NSString` → `stringWithFormat:`. Так на месте вызова сразу видно, какой объект получишь.

Полная программа — `code/07-person-factory.m`. Запуск:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/07-person-factory.m -o person-factory
./person-factory
```

Вывод:

```
Человек: Аружан, возраст 24
Человек: Бекжан, возраст 31
dealloc: объект Бекжан уничтожен
dealloc: объект Аружан уничтожен
--- готово ---
```

а создан длинной записью, `b` — фабрикой; результат одинаковый. Оба объекта живут до конца вложенного пула и там же уничтожаются.

Как это работает

Соберём всю картину рождения объекта в одну схему:

```
[[Person alloc] init]
├─ [Person alloc] → обнулённая память под Person
│                   (все ivars = 0/nil)
└─ init этому объекту:
```

```

self = [super init]    → NSObject настраивает свой слой
if (self) { ... }      → если объект есть, настраиваем свой
return self            → готовый объект уходит в переменную

```

А с назначенным/вторичным инициализатором цепочка вызовов идёт сначала «вбок» (вторичный зовёт назначенный через [self ...]), а назначенный — «вверх» к NSObject через [super init]:

```

init (вторичный)
→ [self initWithName:... age:...] (назначенный)
→ [super init] (назначенный у NSObject)

```

И смерть — зеркально: когда на объект не остаётся ссылок, ARC шлёт ему dealloc, тот выполняется, и память возвращается системе.

Проверяем

Файлы главы и команды сборки:

```

clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/07-person.m -o person && ./person

clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/07-person-name.m -o person-name && ./person-name

clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/07-person-factory.m -o person-factory && ./person-factory

```

Все три собираются без единого предупреждения и печатают вывод, показанный выше (даты и числа в начале строк будут своими).

Частые ошибки

- **Забыл присвоить self.** Написал [super init]; отдельной строкой, а не self = [super init];. Под ARC это даже не предупреждение, а **ошибка компиляции** — правило зашито в язык:

```

error: the result of a delegate init call must be immediately returned
      or assigned to 'self'
[super init];
      ^~~~~~

```

Так что забыть присваивание попросту не дадут — и это хорошо.

- **Нет проверки if (self).** Если родитель вернул nil, а ты лезешь писать в _name (то есть в self->_name), это запись по нулевому указателю — та самая ошибка из Си, программа упадёт. Безопасны только *сообщения* к nil, а прямой доступ к ivar — нет. Привычка if (self) { ... } спасает.
- **Вторичный зовёт [super init] вместо [self ...].** Тогда настройка идёт мимо назначенного инициализатора, поля не заполняются правильно, и смысл цепочки теряется. Вторичный всегда зовёт [self initWithName:age:].
- **id вместоinstancetype у init.** Скомпилируется, но компилятор перестанет строго проверять тип результата, и с наследниками возможны сюрпризы. Для init/фабрик — всегдаinstancetype.
- **Под ARC написал в dealloc [super dealloc] или release.** Такой код просто не соберётся:

```
error: ARC forbids explicit message send of 'dealloc'
    [super dealloc];
    ~~~~~ ^
```

dealloc под ARC содержит только твою собственную уборку, если она вообще нужна.

- **Создал объект на один шаг.** [Person alloc] без init или [Person init] без alloc — обе половинки нужны вместе: всегда [[Person alloc] init] (или фабрику).

Упражнения

1. Добавь человеку третий ivar _city (NSString *). Заведи назначенный инициализатор initWithName:age:city: и сделай прежний initWithName:age: вторичным, подставляющим город @"не указан".
2. Добавь фабричный метод +personWithName: (только имя). Внутри он должен звать [self personWithName:name age:0] или соответствующий инициализатор. Проверь вывод.
3. Убери из main вложенный @autoreleasepool и вместо него после describe напиши p = nil;. Срабатывает ли dealloc и в какой момент? Объясни, почему.
4. Намеренно верни из вторичного инициализатора [super init] вместо [self initWithName:age:]. Запусти и посмотри, как изменится вывод (имя/возраст). Верни как было.
5. Создай наследника Student : Person с ivar _university. Убедись, что [Student personWithName:@"Дана" age:19] благодаряinstancetype и [self alloc] создаёт именно Student. Проверь через [obj isKindOfClass:[Student class]] (метод из главы 5).

Что мы получили

Теперь ты умеешь управлять всей жизнью объекта. Создание идёт в два шага — alloc выделяет и обнуляет память, init настраивает, — а вместе они пишутся как `[[Class alloc] init]`. Ты освоил шаблон `self = [super init]; if (self) { ... } return self;` и понимаешь каждый его знак, пишешь инициализаторы с аргументами, различаешь назначенный и вторичные, сводишь настройку в одну точку, даёшь классу фабричные методы с + и `instancetype`. И видел своими глазами, как `dealloc` срабатывает в момент уничтожения объекта под ARC.

Внутри инициализаторов мы складывали данные прямо в `ivars` и вручную писали `_name = [name copy]`. В следующей главе свойства (`@property`) сами сгенерируют `ivar`, геттер и сеттер — и заодно возьмут на себя тот самый `copy`. А большую тему ARC и ручного управления памятью, которую мы здесь пощупали через `dealloc`, развернём целиком в главе 13.

Документация Apple

- Programming with Objective-C → **Working with Objects** (раздел «Objects Are Created Dynamically», `alloc/init`) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/WorkingwithObjects/WorkingwithObjects.html
- Concepts in Objective-C Programming → **Object Initialization** (designated и secondary initializers, правило цепочки `[self init...]/[super init]`) — developer.apple.com/library/archive/documentation/General/Conceptual/CocoaEncyclopedia/Initialization/Initialization.html
- Adopting Modern Objective-C (`instancetype`, `NS_DESIGNATED_INITIALIZER`) — developer.apple.com/library/archive/releasenotes/ObjectiveC/ModernizationObjC/AdoptingModernObjectiveC/AdoptingModernObjectiveC.html
- `NSObject` — `alloc`, `init`, `dealloc` — developer.apple.com/documentation/objectivec/nsobject

Глава 8. Свойства

В главе 6 ты сделал свой первый класс с переменными экземпляра (ivar), а в главе 7 научил его рождаться через `alloc/init` и прибираться в `dealloc`. Но ivar спрятан внутри объекта: снаружи к нему не дотянуться. Чтобы дать к нему доступ, приходится руками писать пару методов — геттер (читалку) и сеттер (писалку). Для одного поля терпимо. Для пяти — уже скучно, и легко ошибиться.

В этой главе мы избавимся от рутины. Познакомимся с `@property` — объявлением, по которому компилятор **сам** напишет геттер, сеттер и даже скрытый ivar. Разберём точечную запись `obj.name` и докажем, что точка здесь — не доступ к полю, как в Си, а замаскированная отправка сообщения. И разберём атрибуты свойства: `strong`, `weak`, `copy`, `nonatomic` и прочие — что каждый из них означает и когда какой брать.

Что мы сделаем

- Напишем класс с ручными геттером и сеттером — почувствуем рутину.
- Заменим всё это одной строкой `@property` и посмотрим, что сгенерировал компилятор.
- Разберём точечную запись `obj.name` и докажем, что это сообщение.
- Пройдём по всем атрибутам свойства и поймём, когда какой нужен.
- На живом примере увидим, зачем `NSString`-свойствам нужен `copy`.
- Напишем свой геттер вручную поверх `@property`.

Разбираем задачу руками

Возьмём класс `Person` с одним полем — именем. Как в главе 6, объявим ivar и напишем к нему доступ руками. Геттер — метод без аргументов, который возвращает значение. Сеттер — метод с одним аргументом, который значение записывает. По соглашению Objective-C геттер называется как само поле (`name`), а сеттер — `setИмя`: с большой буквы и двоеточием (`setName:`).

```
@interface Person : NSObject {
    NSString *_name;           // ivar, подчёркивание — давняя традиция
```

```
}
- (NSString *)name;           // геттер
- (void)setName:(NSString *)newName; // сеттер
@end

@implementation Person
- (NSString *)name {
    return _name;
}
- (void)setName:(NSString *)newName {
    _name = newName;
}
@end
```

Работает. Но посмотри, сколько ручного труда ради одного поля: строка в блоке `ivar`, две строки объявлений в `@interface`, два метода в `@implementation`. Добавь возраст, город, телефон — и ты переписываешь этот шаблон снова и снова, меняя только имена. Каждая копия — шанс опечататься: написать `getrun`, перепутать тип аргумента, забыть подчёркивание. Скудная механическая работа, которую так и просится отдать машине.

Машина согласна.

@property — одна строка вместо четырёх

Всю эту обязанку заменяет одно объявление в `@interface`:

```
@interface Person : NSObject
@property (nonatomic, copy) NSString *name;
@end
```

И всё. Блок `ivar` не нужен, методы в `@implementation` не нужны. Разберём строку по знакам:

- `@property` — ключевое слово (знак `@`, как всегда, помечает «это Objective-C, а не Си»). Оно говорит компилятору: «объяви свойство».
- `(nonatomic, copy)` — **атрибуты** свойства в круглых скобках через запятую. Они настраивают, как свойство хранит значение и ведёт себя в многопоточности. Их мы разберём ниже; пока прими как есть.
- `NSString *name` — тип и имя свойства, ровно как при объявлении переменной: указатель на строку по имени `name`.
- `;` — конец объявления.

Что компилятор делает по этой строке (это называется **автосинтез**, autosynthesis):

1. заводит скрытый ivar с именем `_name` (то же имя, что у свойства, но с подчёркиванием спереди);
2. синтезирует **getter** `-(NSString *)name`, возвращающий `_name`;
3. синтезирует **setter** `-(void)setName:`, записывающий в `_name`.

То есть те самые четыре куска кода, что мы писали руками, теперь рождаются из одной строки. Имена не случайны: getter зовётся `name`, setter — `setName:`, ivar — `_name`. Эту тройку компилятор выводит из имени свойства автоматически.

Отличие от Си. В Си, чтобы дать доступ к полю структуры, ничего писать не надо — поле `s.name` и так открыто всем, читай и пиши напрямую. Но именно поэтому в Си нельзя вклиниться: проверить значение при записи, залогировать чтение, сделать поле «только для чтения». `@property` в Objective-C — это поле, доступ к которому всегда идёт через методы. Снаружи похоже на простое поле, а внутри — полноценные getter и setter, в которые можно вмешаться.

Точка — это сообщение, не поле

Раз у свойства под капотом есть getter `name` и setter `setName:`, читать и писать его можно обычными сообщениями, как ты уже умеешь:

```
[p setName:@"Айгерим"];           // запись
NSString *n = [p name];           // чтение
```

Но есть короче — **точечная запись** (dot syntax):

```
p.name = @"Айгерим";              // запись
NSString *n = p.name;              // чтение
```

Эти две формы делают абсолютно одно и то же. Точка — чистый синтаксический сахар. Компилятор смотрит, с какой стороны от `=` стоит выражение:

- `p.name = ...` слева от `=` — значит запись, разворачивается в `[p setName:...]`;
- `... = p.name` справа (или просто читаем) — значит чтение, разворачивается в `[p name]`.

Это критично понять правильно, потому что запись `p.name` выглядит точь-в-точь как обращение к полю структуры из Си — а это совсем не оно.

Отличие от Си. В Си `p->name` или `p.name` — это прямой доступ к полю структуры: процессор просто берёт байты по фиксированному смещению внутри struct. Никаких функций не вызывается. В Objective-C `p.name` — это **отправка сообщения**: компилятор превращает её в `[p name]` или `[p setName:...]`, и во время выполнения runtime

ищет нужный метод. Выглядит как поле, работает как вызов. Если ты переопределишь геттер name, точка p.name позовёт твой код — со struct в Си такое невозможно в принципе.

Соберём первую программу. Класс с одним свойством-строкой и одним свойством-числом, и оба способа доступа рядом:

```
#import <Foundation/Foundation.h>

@interface Person : NSObject
@property (nonatomic, copy) NSString *name;
@property (nonatomic, assign) NSInteger age;
@end

@implementation Person
@end

int main(void) {
    @autoreleasepool {
        Person *p = [[Person alloc] init];

        p.name = @"Айгерим";           // = [p setName:@"Айгерим"]
        p.age = 30;                     // = [p setAge:30]
        NSLog(@"через точку:   %@, %ld", p.name, (long)p.age);

        [p setName:@"Болат"];           // то же самое сообщениями
        [p setAge:42];
        NSLog(@"через сообщения: %@, %ld", [p name], (long)[p age]);
    }
    return 0;
}
```

Обрати внимание: @implementation Person пустой — ни одного метода. Всё сгенерировал авто-синтез. Компилируем и запускаем:

```
2026-06-28 12:17:38.793 t[61165:1255010] через точку:   Айгерим, 30
2026-06-28 12:17:38.793 t[61165:1255010] через сообщения: Болат, 42
```

Обе пары строк отработали одинаково — потому что под капотом это один и тот же геттер и сеттер. Точка и квадратные скобки — два написания одного действия.

Маленькая деталь про `%ld` и `(long):NSInteger` на 64-битном Mac — это и есть `long`, так что здесь компилятор промолчал бы и без приведения. Но на 32-битных платформах `NSInteger` — это `int`, и там `%ld` уже не совпал бы с типом. Поэтому Apple советует всегда приводить `NSInteger` к `long` явным `(long)` и печатать `%ld` — тогда строка формата верна везде.

Атрибуты свойства

Скобки (`nonatomic`, `copy`) — это и есть атрибуты. Они делятся на три независимые группы: атомарность, способ хранения и доступность на чтение/запись. Плюс можно переименовать аксессоры. Разберём по группам.

Атомарность: `atomic` vs `nonatomic`

- `atomic` (по умолчанию, если ничего не указать) — геттер и сеттер защищены блокировкой, чтобы из двух потоков одновременно не получить «полузаписанное» значение. За это платишь скоростью.
- `nonatomic` — без блокировки, быстрее. Никакой потокобезопасности не даёт.

На практике почти везде пишут `nonatomic`. Многопоточную безопасность объекта обычно строят на уровне выше, а `atomic` гарантирует слишком мало, чтобы на него полагаться, и при этом тормозит. Привыкай ставить `nonatomic` по умолчанию.

Способ хранения: `strong`, `weak`, `copy`, `assign`

Это самая важная группа — она про то, что сеттер делает с переданным значением и как объект держится в памяти.

- `strong` (по умолчанию для объектных типов) — **сильная ссылка**. Свойство удерживает объект в памяти: пока ссылка жива, объект не умрёт. Берёшь для большинства объектов, которыми твой объект «владеет».
- `weak` — **слабая ссылка**. Свойство ссылается на объект, но не держит его. Если все сильные ссылки исчезнут, объект умрёт, а слабое свойство само станет `nil`. Нужно, чтобы разорвать **цикл удержания** (`retain cycle`): когда два объекта держат друг друга `strong`, ни один не может умереть — оба зависят в памяти навсегда. Классический случай — родитель держит ребёнка `strong`, а ребёнок ссылается на родителя `weak`. Подробно про циклы — в главе 12 (блоки) и главе 13 (ARC); пока запомни правило: «обратная ссылка на владельца — `weak`».
- `copy` — при установке сеттер **делает копию** объекта и хранит её, а не то, что ему дали. Зачем — разберём отдельным примером ниже. Берут для `NSString`, `NSArray`, `NSDictionary` и других классов, у которых есть изменяемые подклассы (`NSMutableString` и т.п.).

- `assign` — просто записывает значение «как есть», без всякого подсчёта ссылок. Для **примитивов**: `NSInteger`, `double`, `B00L`, `CGFloat`, указатели на си-структуры. У числа нет «времени жизни», удерживать нечего — поэтому `assign`. (Для объектов `assign` тоже бывает, но это опасный «висячий указатель»; для них есть `weak`.)

Памятка-шпаргалка:

примитив (число, <code>B00L</code>)	→ <code>assign</code>
обычный объект, которым владеем	→ <code>strong</code>
обратная ссылка на владельца	→ <code>weak</code>
строка/массив/словарь	→ <code>copy</code>

Доступность: `readwrite` vs `readonly`

- `readwrite` (по умолчанию) — есть и геттер, и сеттер. Читай и пиши.
- `readonly` — только геттер. Сеттер не синтезируется, снаружи свойство не записать. Удобно для вычисляемых значений (например, `fullName`, собранное из имени и фамилии) или для полей, которые объект меняет только сам внутри себя.

Переименование: `getter=` и `setter=`

Имена аксессоров можно задать вручную:

```
@property (nonatomic, assign, getter=isPaid) B00L paid;
```

Здесь геттер будет называться `isPaid`, а не `paid`. Для свойств-флагов типа `B00L` это общепринятый стиль в Сосоа: читается как вопрос — `if (account.isPaid)`. Записывать всё равно через `account.paid = YES` (сеттер остался `setPaid`).

сору: зачем он нужен

Самый неочевидный атрибут — `copy`. Покажем на живом примере, чем он отличается от `strong`, и почему для строк берут именно его.

Заведём класс с двумя свойствами-строками: одно `strong`, другое `copy`. Положим в оба **одну и ту же** изменяемую строку `NSMutableString` — это `NSString`, который, в отличие от обычного, разрешено менять после создания. А потом испортим исходную строку и посмотрим на свойства.

```
#import <Foundation/Foundation.h>

@interface Label : NSObject
```

```
@property (nonatomic, strong) NSString *strongName;
@property (nonatomic, copy)   NSString *safeName;
@end

@implementation Label
@end

int main(void) {
    @autoreleasepool {
        Label *label = [[Label alloc] init];

        NSMutableString *text =
            [NSMutableString stringWithString:@"Привет"];

        label.strongName = text;      // strong: хранит сам text
        label.safeName   = text;      // copy:   хранит КОПИЮ text

        NSLog(@"до правки:");
        NSLog(@" strongName = %@", label.strongName);
        NSLog(@" safeName   = %@", label.safeName);

        [text appendString:@" мир!"]; // меняем исходную строку

        NSLog(@"после правки исходной строки:");
        NSLog(@" strongName = %@", label.strongName);
        NSLog(@" safeName   = %@", label.safeName);
    }
    return 0;
}
```

[NSMutableString stringWithString:@"Привет"] создаёт изменяемую строку со стартовым содержанием «Привет». [text appendString:@" мир!"] дописывает к ней текст — меняет тот самый объект, на который мы уже дали ссылки обоим свойствам. Запускаем:

```
2026-06-28 12:17:48.516 t[61196:1255120] до правки:
2026-06-28 12:17:48.517 t[61196:1255120] strongName = Привет
2026-06-28 12:17:48.517 t[61196:1255120] safeName   = Привет
2026-06-28 12:17:48.517 t[61196:1255120] после правки исходной строки:
```

```
2026-06-28 12:17:48.517 t[61196:1255120] strongName = Привет, мир!  
2026-06-28 12:17:48.517 t[61196:1255120] safeName = Привет
```

Вот она, разница. До правки оба свойства показывали «Привет». После того как мы испортили исходную строку:

- `strongName` тоже изменилось на «Привет, мир!» — потому что `strong` хранил **тот же самый** объект `text`. Поменяли `text` — поменялось и свойство, хотя `label` никто не трогал. Объект подменили у него за спиной.
- `safeName` осталось «Привет» — потому что `copy` в момент установки снял **независимую копию** и хранит её. Что бы ни делали с исходной строкой, копия не дрогнет.

Поэтому правило: **свойства-строки (и коллекции) объявляй `copy`**. Иначе кто-то передаст тебе `NSMutableString`, сохранит на неё ссылку и потом будет менять содержимое — а твой объект об этом не узнает и будет показывать не то, что ему дали изначально. `copy` отрезает эту ниточку.

А не дорого ли копировать каждую строку?

Законный вопрос: если каждое присваивание `p.name = ...` делает копию, не станет ли программа тратить время на копирование строк на ровном месте? Нет — и это стоит увидеть своими глазами. Дело в том, что `copy` у **неизменяемой** строки не копирует ничего: копировать нечего, объект и так никогда не изменится, поэтому он просто возвращает сам себя. Проверим на адресах (`code/08-copy-identity.m`):

```
NSString *a = @"Привет";  
NSString *b = [a copy];  
NSLog(@"неизменяемая: a=%p b=%p — тот же объект: %@",  
      (__bridge void *)a, (__bridge void *)b,  
      (a == b) ? @"да" : @"нет");  
  
NSMutableString *m = [NSMutableString stringWithString:@"Привет"];  
NSString *c = [m copy];  
NSLog(@"изменяемая: m=%p c=%p — тот же объект: %@",  
      (__bridge void *)m, (__bridge void *)c,  
      (m == c) ? @"да" : @"нет");  
  
NSLog(@"класс копии изменяемой строки: %@",  
      NSStringFromClass([c class]));
```


Спецификатор `%p` печатает адрес и формально ждёт си-шный `void *`. Поэтому объектный указатель мы приводим к `void *`. Под ARC простое `(void *)a` не скомпилируется: компилятор требует пометку `__bridge` — «просто смотрю на адрес, владение объектом не трогаю». Это слово ещё встретится в главе 14 и в главе 24 (мост к CoreFoundation). Запускаем:

```
неизменяемая: a=0x10fc5b048 b=0x10fc5b048 — тот же объект: да
изменяемая:   m=0x600000d80000 c=0x600000d80030 — тот же объект: нет
класс копии изменяемой строки: __NSCFString
```

Адреса говорят всё. Копия неизменяемой строки — **тот же самый объект** по тому же адресу: работа нулевая. А вот у `NSMutableString` адрес другой — здесь копия настоящая и независимая. И главное: `copy` всегда отдаёт **неизменяемую** строку, даже когда ему дали изменяемую. Попробуй привести результат обратно к `NSMutableString` и что-нибудь дописать — получишь падение:

```
*** Terminating app due to uncaught exception 'NSInvalidArgumentException',
reason: 'Attempt to mutate immutable object with appendString:'
```

Кстати, о печатаемом классе `__NSCFString`. По нему нельзя судить, изменяемая строка или нет: этот же класс носят и `NSMutableString`, и неизменяемая копия — внутри Foundation они устроены на общей основе CoreFoundation. Даже `[c isKindOfClass:[NSMutableString class]]` ответит да, хотя менять `c` нельзя, что мы только что и увидели. А копия короткой латинской строки, скажем `@"abc"`, и вовсе окажется классом `NSTaggedPointerString`. Мораль: изменяемость строки проверяют не интроспекцией класса, а тем, откуда объект пришёл. Именно поэтому `copy` в свойстве и нужен — он снимает вопрос раз и навсегда.

Отсюда практический вывод: `copy` для строкового свойства бесплатен в обычном случае и спасает в опасном. Не экономь на нём.

Отличие от Си. В Си присваивание указателя `a = b`; копирует только **адрес** — оба указателя смотрят на одну и ту же память, и правка через один видна через другой. Чтобы получить настоящую независимую копию, в Си зовут `strcpy/memcpy` руками. `@property (copy)` встраивает это копирование прямо в сеттер: каждое присваивание `obj.name = ...` незаметно шлёт значению `copy` — и изменяемая строка копируется, а неизменяемая, как мы видели, просто отдаёт саму себя. Поведение, которое в Си надо прописывать вручную, здесь зашито в свойство.

Свой геттер или сеттер поверх `@property`

Автосинтез — удобство, а не тюрьма. Если для свойства написать геттер или сеттер руками, компилятор увидит это и **не будет** синтезировать тот, что ты написал (второй, если не тронут,

синтезирует как обычно). Так в доступ можно вставить любую логику: проверку, лог, вычисление на лету.

Сделаем класс Account. Баланс хранится внутри в тиынах (целым числом, 1 тенге = 100 тиын), а наружу свойство balance отдаёт тенге дробным числом. Это вычисляемое свойство: ставим readonly и пишем геттер сами.

```
#import <Foundation/Foundation.h>

@interface Account : NSObject
@property (nonatomic, assign, getter=isPaid) BOOL paid;
@property (nonatomic, readonly) double balance;
@end

@implementation Account {
    NSInteger _tiyn;    // свой ivar: 1 тенге = 100 тиын
}

- (double)balance {      // ручной геттер для balance
    return _tiyn / 100.0;
}

- (void)depositTenge:(double)tenge {
    _tiyn += (NSInteger)(tenge * 100);
}

@end

int main(void) {
    @autoreleasepool {
        Account *a = [[Account alloc] init];

        [a depositTenge:150.50];
        a.paid = YES;

        NSLog(@"баланс: %.2f тг", a.balance);
        NSLog(@"оплачено: %@", a.isPaid ? @"да" : @"нет");
    }
}
```

```
    return 0;
}
```

Разберём, что нового:

- `@implementation Account { NSInteger _tiyn; }` — фигурные скобки сразу после `@implementation` объявляют **приватный ivar** `_tiyn`, видимый только внутри реализации класса. Это поле не связано ни с каким свойством, мы храним в нём данные по-своему.
- `-(double)balance { return _tiyn / 100.0; }` — наш геттер. Раз мы написали его сами, компилятор свой не синтезирует. А у `readonly`-свойства, кроме геттера, генерировать нечего, поэтому и скрытый `ivar _balance` не появится — он нам и не нужен. Деление `/ 100.0` (с точкой!) даёт `double`; написали бы `/ 100` — получили бы целочисленное деление и потеряли тиыны.
- `getter=isPaid` — поэтому читаем флаг как `a.isPaid`. Записали мы его через `a.paid = YES` — сеттер остался стандартным `setPaid:`.

Запуск:

```
2026-06-28 12:17:40.565 t[61179:1255044] баланс: 150.50 тг
2026-06-28 12:17:40.565 t[61179:1255044] оплачено: да
```

Снаружи `a.balance` выглядит как обычное поле, но за точкой прячется наш геттер, который на лету пересчитывает тиыны в тенге. Ровно то, чего в Си со `struct`-полем не сделать.

@synthesize и @dynamic — пара слов для истории

Раньше (примерно до 2012 года) автосинтеза не было, и для каждого свойства приходилось писать в `@implementation` строку:

```
@synthesize name = _name;
```

Она и означала «сгенерируй мне геттер/сеттер для `name`, а `ivar` назови `_name`». Сегодня компилятор делает это сам, и `@synthesize` почти не нужен. Изредка его пишут, чтобы задать `ivar` нестандартного имени или когда автосинтез по какой-то причине отключился (например, ты переопределил оба аксессора у `readwrite`-свойства).

Есть и противоположность — `@dynamic name;`. Она говорит: «не генерируй ничего, аксессоры появятся во время выполнения». Так делают фреймворки, которые создают методы динамически (например, Core Data). В обычном коде тебе это не понадобится — просто знай, что значит слово, если встретишь.

`_ivar` или `self.name` — что когда

Внутри методов класса к данным свойства можно обращаться двумя путями:

- напрямую к `ivar` — `_name`;
- через свойство — `self.name` (это `[self name]` / `[self setName:...]`).

Разница та же, что снаружи: `_name` лезет в память напрямую, `self.name` проходит через аксессор. Правила простые:

- **В обычных методах** пользуйся `self.name`. Тогда любая логика, которую ты (или подкласс) вложил в геттер/сеттер, отработает. Для `copy`-свойства это важно: `self.name = arg` сделает копию, а `_name = arg` — нет.
- **В `init` и `dealloc`** принято обращаться напрямую к `_name`. Объект ещё (или уже) не полностью готов, а сеттер подкласса может рассчитывать на доделанный объект и сломаться. Поэтому в инициализаторе пишут `_name = [value copy];`, минуя `self.name`. Это прямая рекомендация Apple из «Programming with Objective-C».

```
- (instancetype)initWithName:(NSString *)value {
    self = [super init];
    if (self) {
        _name = [value copy];    // напрямую в ivar, и copy руками
    }
    return self;
}
```

Заметь `[value copy]`: раз мы обходим сеттер (а он бы скопировал сам), копию для `copy`-свойства делаем своими руками. Обходишь аксессор — берёшь на себя и его работу.

Проверяем

Файлы главы — `code/08-property.m`, `code/08-copy.m`, `code/08-copy-identity.m`, `code/08-custom-accessor.m`. Каждый собирается и запускается так:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/08-property.m -o /tmp/t && /tmp/t
```

Ожидаемый вывод `08-property` (дата и числа будут свои):

```
через точку:    Айгерим, 30
через сообщения: Болат, 42
```

08-copу наглядно показывает, как strong-свойство меняется вслед за исходной строкой, а copу — нет:

после правки исходной строки:

```
strongName = Привет, мир!  
safeName   = Привет
```

Частые ошибки

- **strong у строки вместо copу.** Передали NSMutableString, потом поменяли её снаружи — и значение свойства «уехало» само собой. Для NSString/NSArray/NSDictionary всегда copу.
- **strong в обе стороны → retain cycle.** Если объект А держит В сильно, а В держит А сильно, оба никогда не освободятся. Обратную ссылку делай weak. Подробно — главы 12 и 13.
- **assign для объекта.** Объект могут освободить, а указатель останется смотреть в никуда — «висячий указатель» и падение. Для объектов вместо assign бери weak (он сам обнулится в nil). assign — только для примитивов: чисел, BOOL, си-структур.
- **Думать, что точка — это поле.** obj.name — это сообщение. Если объект nil, obj.name не упадёт (сообщение к nil безопасно, см. главу 5), а вернёт нулевое значение. Со struct-полем такого «прощения» нет.
- **Целочисленное деление в геттере.** _tiyn / 100 отбросит дробную часть. Чтобы получить дробь, хотя бы один операнд должен быть дробным: / 100.0.
- **Менять copу-свойство в обход сеттера.** Записал _name = arg; напрямую — копию никто не сделал, защита copу потеряна. Вне init пользуйся self.name = arg.

Упражнения

1. Добавь классу Person свойство city (NSString, copу) и выведи все три поля одной строкой NSLog.
2. Сделай у Person свойство fullName с атрибутом readonly и ручным геттером, который возвращает имя + пробел + фамилия. Добавь ivar или свойство для фамилии.
3. Повтори эксперимент из 08-copу.m, но с NSArray и NSMutableArray: положи изменяемый массив в strong- и copу-свойства, добавь в исходный массив элемент и сравни.
4. Объяви свойство BOOL hidden с getter=isHidden. Установи через obj.hidden = YES, прочитай через obj.isHidden. Убедись, что obj.isHidden и [obj isHidden] дают одно и то же.
5. Напиши сеттер setAge: вручную так, чтобы отрицательный возраст он превращал в 0. Проверь, что p.age = -5; приводит к нулю.

Что мы получили

Ты заменил гору ручных аксессоров одной строкой `@property` и понял, что точечная запись `obj.name` — это замаскированная отправка сообщения, а не доступ к полю, как в Си. Разобрал атрибуты: `nonatomic` по умолчанию, `strong/weak/copy/assign` для разного хранения, `readonly` для вычисляемых полей, `getter=` для красивых имён. Увидел вживую, зачем строкам `copy`, и научился вставлять свой геттер поверх автосинтеза.

Свойства — это аккуратный фасад объекта: снаружи простые «поля», внутри — методы, в которые ты волен вмешаться. В следующей главе мы дадим объектам **наследование**: один класс будет перенимать свойства и методы другого, а `super` позволит дотянуться до родителя.

Документация Apple

- Programming with Objective-C → **Encapsulating Data** (раздел «Properties Encapsulate an Object's Values») — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/EncapsulatingData/EncapsulatingData.html
- `@property` и атрибуты — там же, подразделы «Use Attributes to Define Properties», «Copy Properties Maintain Their Own Copies».
- `NSString` / `NSMutableString` — developer.apple.com/documentation/foundation/nsstring, developer.apple.com/documentation/foundation/nsmutablestring

Глава 9. Наследование

У нас уже есть классы: мы умеем описывать тип объекта, давать ему методы и свойства (главы 6–8). Но классы редко живут поодиночке. Часто один тип — это «частный случай» другого: собака и кошка — оба животные; кнопка и поле ввода — оба элемента интерфейса. У них масса общего и немного своего. Переписывать общее в каждом классе заново — глупо.

Наследование позволяет сказать: «этот класс — как тот, плюс своё». Новый класс берёт все поля, свойства и методы родителя бесплатно, а потом добавляет или меняет то, что нужно. В этой главе мы соберём базовый класс `Animal`, наследуем от него `Dog` и `Cat`, переопределим у каждого голос — и сложим всех в один массив, чтобы они «заговорили» каждый по-своему. Это и есть полиморфизм, и в Objective-C он работает удивительно просто.

Что мы сделаем

- Научимся объявлять подкласс: `@interface Dog : Animal`.
- Поймём, что именно наследуется и откуда берётся `NSObject`.
- Переопределим метод и вызовем версию родителя через `[super ...]`.
- Достроим цепочку инициализаторов через `[super initWithName:...]`.
- Увидим полиморфизм вживую: один цикл, разные голоса.
- Научимся спрашивать объект о его классе: `isKindOfClass:`, `isMemberOfClass:`, `respondsToSelector:`, `class`, `superclass`.
- Сделаем абстрактный базовый класс «по соглашению».

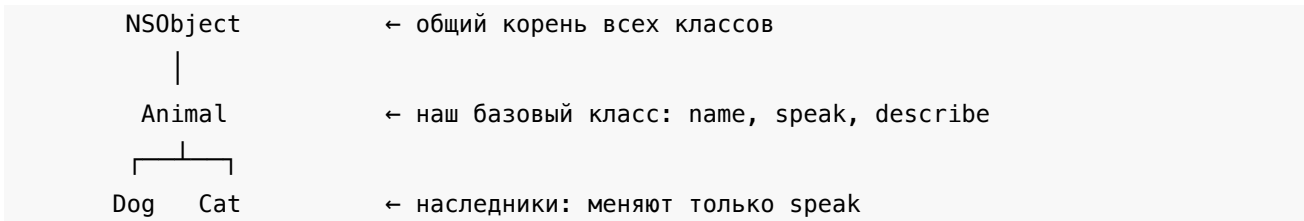
Разбираем задачу руками

Представь зоопарк. В нём собаки, кошки и какие-то ещё звери. У каждого есть имя. Каждый умеет издавать звук — но звук у всех разный: собака лает, кошка мяукает.

Без наследования пришлось бы написать класс `Dog` с полем `name` и методом «представься», потом класс `Cat` — с тем же полем `name` и почти тем же методом, отличается только звук. И так для каждого нового зверя. Общее дублируется снова и снова.

Наследование разворачивает задачу иначе. Выделяем общее в один класс `Animal`: у него есть `name` и метод `speak` (сказать свой звук). А `Dog` и `Cat` объявляем его наследниками. Они автоматически получают `name` и весь «общий» код, и им остаётся переопределить ровно одно — `speak`.

Нарисуем это деревом:



Стрелка снизу вверх читается «является частным случаем»: `Dog` — это `Animal`, а `Animal` — это `NSObject`. Каждый класс знает своего **суперкласса** (родителя ровно на ступень выше) и получает в наследство всё, что есть выше по дереву.

Отличие от Си. В чистом Си наследования нет вовсе. Есть `struct` — набор полей, и всё. Если хочешь «как тот тип, плюс своё», ты вручную кладёшь старую структуру первым полем новой и руками протаскиваешь указатели на функции, имитируя методы. Получается громоздко и хрупко. Objective-C встроил наследование прямо в язык: одно двоеточие — и подкласс получает родителя целиком.

Шаг 1. Базовый класс `Animal`

Начнём с родителя. Создай файл `09-animals.m`:

```
#import <Foundation/Foundation.h>

@interface Animal : NSObject
@property (nonatomic, copy) NSString *name;
- (instancetype)initWithName:(NSString *)name;
- (NSString *)speak;
- (void)describe;
@end
```

Разберём первую строку — она новая и в ней вся суть главы.

@interface `Animal` : `NSObject`

Это объявление класса `Animal`, как в главе 6. Новое — **двоеточие и имя после него**: `: NSObject`. Двоеточие читается «наследует от». То есть «`Animal` наследует от `NSObject`».

NSObject — это **корневой класс** почти всего в Objective-C. От него, прямо или через цепочку родителей, происходит подавляющее большинство классов: NSString, NSArray, твой Person из прошлых глав — все они в итоге восходят к NSObject. Именно NSObject даёт объектам базовые умения, без которых они вообще не работают:

- alloc — выделить память под объект;
- init — базовая настройка;
- class, isKindOfClass:, respondsToSelector: — самопознание (разберём ниже);
- участие в подсчёте ссылок ARC.

Поэтому когда ты пишешь свой класс «с нуля», ты никогда не начинаешь с пустого места — ты наследуешь от NSObject и получаешь весь этот фундамент даром. Вот почему в каждом нашем @interface справа от двоеточия стоял NSObject.

Дальше идёт уже знакомое:

- @property (nonatomic, copy) NSString *name; — свойство «имя» (глава 8). copy берёт независимую копию строки, чтобы её не подменили снаружи.
- - (instancetype)initWithName:(NSString *)name; — инициализатор с аргументом. `instancetype` означает «вернёт объект того же класса, у которого вызвали» (глава 7).
- - (NSString *)speak; — вернуть звук животного.
- - (void)describe; — представиться вслух.

Теперь реализация:

```
@implementation Animal

- (instancetype)initWithName:(NSString *)name {
    self = [super init];
    if (self) {
        _name = [name copy];
    }
    return self;
}

- (NSString *)speak {
    return @"...(непонятный звук)";
}

- (void)describe {
    NSLog(@"%@ говорит: %@", self.name, [self speak]);
}
```

```
@end
```

В `initWithName:` мы снова видим `[super init]` — ту самую строку из главы 7. Теперь, после разговора про дерево классов, её смысл виден полностью: `super` — это «родитель», то есть `NSObject`. Мы просим родителя сначала настроить свою часть объекта (общую базу), он возвращает готовый `self`, и только потом мы задаём своё поле `_name`. Сначала фундамент, потом надстройка — поэтому `[super init]` идёт первой строкой.

`speak` пока возвращает невнятную заглушку — это значение «по умолчанию», которое подклассы заменят. А `describe` интересен: он зовёт `[self speak]`. Запомни эту строчку, к ней вернёмся, когда дойдём до полиморфизма — в ней спрятана вся магия.

Шаг 2. Подкласс Dog

Теперь наследник. Допишем в тот же файл:

```
@interface Dog : Animal
@property (nonatomic, copy) NSString *breed;
- (instancetype)initWithName:(NSString *)name breed:(NSString *)breed;
@end
```

`@interface Dog : Animal` — Dog наследует от `Animal`. Что это значит на практике? Dog **бесплатно получает всё**, что есть у `Animal`:

- поле `_name` и свойство `name` (с готовыми геттером и сеттером);
- метод `initWithName::`
- метод `speak`;
- метод `describe`.

И всё, что `Animal` сам унаследовал от `NSObject` (`alloc`, `class` и прочее), тоже достаётся Dog. Наследство складывается по всей цепочке.

Сверх наследства Dog добавляет **своё**: свойство `breed` (порода) и свой инициализатор с двумя аргументами. Реализуем:

```
@implementation Dog

- (instancetype)initWithName:(NSString *)name breed:(NSString *)breed {
    self = [super initWithName:name];
    if (self) {
        _breed = [breed copy];
    }
}
```

```

        return self;
    }

    - (NSString *)speak {
        return @"Гав-гав!";
    }

    - (void)describe {
        [super describe];
        NSLog(@"    (порода: %@)", self.breed);
    }

@end

```

Здесь сразу три важные вещи.

Цепочка инициализаторов. В своём `initWithName:breed:` мы зовём не `[super init]`, а `[super initWithName:name]` — инициализатор родителя `Animal`. Родитель настроит `name`, вернёт готовый объект, и мы добавим своё поле `breed`. А внутри `Animal`-овского `initWithName:` ещё раньше вызывался `[super init]` — то есть `NSObject`. Получается лесенка вызовов снизу вверх по дереву:

```

Dog initWithName:breed:
└─ super → Animal initWithName:
    └─ super → NSObject init

```

Каждый уровень настраивает свою часть и передаёт работу выше. Это и есть **цепочка инициализации**: объект собирается слоями, от корня к листу.

Переопределение метода. У `Dog` есть свой `speak`, и он **заменяет** родительский. У `Animal` он возвращал заглушку, а у `Dog` — `@"Гав-гав!"`. Имя метода то же, тело новое. Это и называется **переопределение (override)**: подкласс даёт свою версию унаследованного метода. Ключевых слов вроде `override` в Objective-C для этого не нужно — достаточно объявить метод с тем же именем.

Вызов версии родителя через `[super ...]`. В `describe` мы не стали переписывать родительский код заново. Мы вызвали `[super describe]` — «сделай то, что делал бы `Animal`», — а потом дописали строчку про породу. Так подкласс **дополняет** поведение родителя, не теряя его. Сравни: в `speak` мы родителя полностью заменили (не звали `super`), а в `describe` — расширили (позвали `super` и добавили своё). Оба приёма законны; выбираешь по задаче.

Отличие от Си. Ни переопределения, ни `super` в Си нет в принципе. Если бы ты имитировал методы указателями на функции в структуре, то «переопределение»

означало бы вручную перезаписать поле-указатель, а «вызов родителя» — где-то сохранить старый указатель и не потерять его. Здесь же всё это — встроенная механика языка: `super` сам знает, чью реализацию звать.

Что такое `super`. `super` — это **не объект**. Это указание компилятору и runtime: «при поиске метода начни не с моего класса, а на ступень выше — у суперкласса». `self` и `super` указывают на один и тот же объект; разница лишь в том, *откуда начинать искать* нужный метод. Подробно эту механику поиска мы разберём в главе 14 про runtime.

Шаг 3. Подкласс Cat

Кошке хватит переопределить один метод:

```
@interface Cat : Animal
@end

@implementation Cat
- (NSString *)speak {
    return @"Мяу!";
}
@end
```

`@interface Cat : Animal` без единой новой строчки внутри — и `Cat` уже полноценный класс: у него есть `name`, `initWithName:`, `describe` — всё от `Animal`. Мы лишь переопределили `speak`, чтобы кошка мяукала. Меньше кода — больше пользы; вот ради чего наследование и придумали.

Шаг 4. Полиморфизм — один цикл, разные голоса

Теперь главное. Напишем `main`:

```
int main(void) {
    @autoreleasepool {
        Animal *someone = [[Dog alloc] initWithName:@"Рекс"
                                                breed:@"овчарка"];
        NSLog(@"--- одно животное в переменной типа Animal* ---");
        [someone describe];

        NSLog(@"--- зоопарк: один цикл, разные голоса ---");
        NSArray *zoo = @[
```

```
        [[Dog alloc] initWithName:@"Пекс" breed:@"овчарка"],
        [[Cat alloc] initWithName:@"Мурка"],
        [[Animal alloc] initWithName:@"Нечто"],
    ];
    for (Animal *a in zoo) {
        [a describe];
    }
}
return 0;
}
```

Останови взгляд на первой строке:

```
Animal *someone = [[Dog alloc] initWithName:@"Пекс" breed:@"овчарка"];
```

Тип переменной — `Animal *`. А справа создаётся `Dog`. Так можно: `Dog` является `Animal` (наследник), поэтому указатель на `Animal` спокойно держит собаку. Переменная «видит» объект как животное, но внутри лежит именно собака.

И вот вопрос на миллион: когда мы зовём `[someone describe]`, а внутри `describe` срабатывает `[self speak]` — **чей** `speak` выполнится? Родительский (`Animal`, ведь тип переменной такой) или собачий (`Dog`, ведь объект — собака)?

Ответ: **собачий**. И это ключевая идея главы.

Почему вызовется метод фактического класса

Вспомни главу 5: `[obj speak]` — это не вызов функции, а **отправка сообщения**. Получатель сообщения — сам объект, а не тип переменной. Когда сообщение `speak` приходит к объекту, runtime смотрит на **фактический класс объекта** (а он зашит в самом объекте, не в типе переменной) и ищет метод именно там. Объект — собака, значит найдётся `Dog`-овский `speak`, и неважно, что переменная объявлена как `Animal *`.

Эта подмена нужного метода по фактическому типу объекта и называется **полиморфизм** (от греческого «много форм»): одно и то же сообщение `describe` даёт разное поведение в зависимости от того, кто его получил.

Отличие от Си. В Си *ты сам* решаешь, какую функцию звать. Если бы у тебя был массив разных «зверей», пришлось бы хранить рядом тег типа и писать `switch (animal->type) { case DOG: dog_speak(); ... }` — длинную ручную развилку, которую надо править при каждом новом типе. В Objective-C развилки нет: ты шлешь сообщение

speak, а runtime сам подбирает реализацию по фактическому классу получателя. Добавил новый класс — старый код менять не нужно.

Цикл `for (Animal *a in zoo)` — это **быстрый перебор** (fast enumeration): он по очереди достаёт из массива каждый элемент в переменную `a` типа `Animal *`. Сам массив `zoo` хранит объекты как `id` (любой объект), и в нём вперемешку лежат собака, кошка и «нечто». Цикл один и тот же, сообщение `[a describe]` одно и то же — а голоса разные, потому что каждый объект получает своё сообщение и отвечает своим методом.

Полная программа

Весь `09-animals.m` целиком:

```
#import <Foundation/Foundation.h>

@interface Animal : NSObject
@property (nonatomic, copy) NSString *name;
- (instancetype)initWithName:(NSString *)name;
- (NSString *)speak;
- (void)describe;
@end

@implementation Animal

- (instancetype)initWithName:(NSString *)name {
    self = [super init];
    if (self) {
        _name = [name copy];
    }
    return self;
}

- (NSString *)speak {
    return @"...(непонятный звук)";
}

- (void)describe {
    NSLog(@"%@ говорит: %@", self.name, [self speak]);
}
```

```
@end

@interface Dog : Animal
@property (nonatomic, copy) NSString *breed;
- (instancetype)initWithName:(NSString *)name breed:(NSString *)breed;
@end

@implementation Dog

- (instancetype)initWithName:(NSString *)name breed:(NSString *)breed {
    self = [super initWithName:name];
    if (self) {
        _breed = [breed copy];
    }
    return self;
}

- (NSString *)speak {
    return @"Гав-гав!";
}

- (void)describe {
    [super describe];
    NSLog(@"    (порода: %@)", self.breed);
}

@end

@interface Cat : Animal
@end

@implementation Cat
- (NSString *)speak {
    return @"Мяу!";
}
@end
```

```
int main(void) {
    @autoreleasepool {
        Animal *someone = [[Dog alloc] initWithName:@"Рекс"
                                                    breed:@"овчарка"];
        NSLog(@"--- одно животное в переменной типа Animal* ---");
        [someone describe];

        NSLog(@"--- зоопарк: один цикл, разные голоса ---");
        NSArray *zoo = @[
            [[Dog alloc] initWithName:@"Рекс" breed:@"овчарка"],
            [[Cat alloc] initWithName:@"Мурка"],
            [[Animal alloc] initWithName:@"Нечто"],
        ];
        for (Animal *a in zoo) {
            [a describe];
        }
    }
    return 0;
}
```

Компилируем и запускаем:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/09-animals.m -o animals
./animals
```

Вывод (префикс NSLog с датой и номерами опущен):

```
--- одно животное в переменной типа Animal* ---
Рекс говорит: Гав-гав!
    (порода: овчарка)
--- зоопарк: один цикл, разные голоса ---
Рекс говорит: Гав-гав!
    (порода: овчарка)
Мурка говорит: Мяу!
Нечто говорит: ...(непонятный звук)
```

Разбор вывода:

- «Рекс говорит: Гав-гав!» — переменная была `Animal *`, но объект-собака, и `describe` через `[self speak]` нашёл собачий голос. Полиморфизм в действии.

- «(порода: овчарка)» — это вторая строка из Dog-овского describe, где после [super describe] мы дописали породу.
- У кошки — «Мяу!», свой переопределённый speak.
- У чистого Animal («Нечто») переопределять было нечего, сработала заглушка ... (непонятный звук).

Один метод describe, один цикл — три разных результата. Ровно то, ради чего затевалось наследование.

Интроспекция: спросить объект, кто он такой

NSObject даёт каждому объекту набор методов **самопознания (introspection)** — спросить его прямо во время выполнения, какого он класса и что умеет. Соберём отдельный файл 09-introspection.m:

```
#import <Foundation/Foundation.h>

@interface Animal : NSObject
- (NSString *)speak;
@end

@implementation Animal
- (NSString *)speak { return @"..."; }
@end

@interface Dog : Animal
- (void)fetch;
@end

@implementation Dog
- (NSString *)speak { return @"Гав!"; }
- (void)fetch { NSLog(@"    %@: принёс мячик", [self class]); }
@end

int main(void) {
    @autoreleasepool {
        Animal *a = [[Dog alloc] init];

        NSLog(@"[a class]    = %@", [a class]);
    }
}
```

```

NSLog(@"[a superclass] = %@", [a superclass]);

NSLog(@"kind of Dog?      %@",
      [a isKindOfClass:[Dog class]]      ? @"да" : @"нет");
NSLog(@"kind of Animal?   %@",
      [a isKindOfClass:[Animal class]]   ? @"да" : @"нет");
NSLog(@"kind of NSObject? %@",
      [a isKindOfClass:[NSObject class]] ? @"да" : @"нет");

NSLog(@"member of Dog?    %@",
      [a isKindOfClass:[Dog class]]      ? @"да" : @"нет");
NSLog(@"member of Animal? %@",
      [a isKindOfClass:[Animal class]]   ? @"да" : @"нет");

NSLog(@"умеет fetch?      %@",
      [a respondsToSelector:@selector(fetch)]      ? @"да" : @"нет");
NSLog(@"умеет addObject?  %@",
      [a respondsToSelector:@selector(addObject:)] ? @"да" : @"нет");

if ([a respondsToSelector:@selector(fetch)]) {
    [(Dog *)a fetch];
}
}
return 0;
}

```

Разберём каждый инструмент.

[obj class] и [obj superclass]

`[a class]` возвращает **класс объекта** как значение (тип `Class`). Это не строка, но `NSLog` с `%@",` умеет печатать его именем. `[a superclass]` — класс родителя. Для нашей собаки `class` даст `Dog`, `a superclass` — `Animal`.

Откуда объект вообще знает свой класс? Внутри каждого объекта есть скрытое служебное поле — указатель **isa**, который ведёт на его класс. Именно по `isa runtime` и понимает, кто перед ним, и именно `isa` делает возможным полиморфизм. Само поле `isa`, метаклассы и устройство класса — большая тема главы 14; пока достаточно знать, что объект «носит свой класс с собой».

isKindOfClass: — ЭТОТ КЛАСС ИЛИ ПОТОМОК

[a isKindOfClass:[Animal class]] спрашивает: «объект a — это Animal **или любой его потомок?**». Для собаки ответ «да» по всем трём строкам: собака — это и Dog, и (как наследник) Animal, и (ещё выше) NSObject. Метод проверяет принадлежность ко **всей ветке** дерева вверх.

Обрати внимание на аргумент: [Dog class]. Сами классы — тоже значения, и [Dog class] даёт объект-класс Dog, который мы передаём для сравнения.

isMemberOfClass: — РОВНО ЭТОТ КЛАСС

[a isMemberOfClass:[Dog class]] строже: «объект — **ровно** Dog, не потомок, не предок?». Для собаки member of Dog? — «да», а member of Animal? — уже «нет»: объект наследует от Animal, но *является* именно Dog. Разница в одном: isKindOfClass: учитывает наследников, isMemberOfClass: — нет. На практике почти всегда нужен первый.

respondsToSelector: — ОТВЕТИТ ЛИ НА СООБЩЕНИЕ

[a respondsToSelector:@selector(fetch)] спрашивает: «есть ли у объекта метод fetch?». Это проверка не по типу переменной, а по фактическому классу: переменная объявлена как Animal *, у которого fetch нет, но объект-собака fetch умеет — значит «да». А addObject: (метод массивов) собака не знает — «нет».

@selector(fetch) превращает имя метода в значение-селектор (мы встречали его в главе 5): компактный идентификатор сообщения, который можно передать как аргумент. Двоеточия внутри важны: @selector(fetch) и @selector(addObject:) — разные селекторы, второй ждёт аргумент.

respondsToSelector: — главный страховочный приём перед отправкой сообщения, в котором не уверен: сначала спроси, потом шли. В нашем коде именно так:

```
if ([a respondsToSelector:@selector(fetch)]) {  
    [(Dog *)a fetch];  
}
```

Тип переменной — Animal *, в нём fetch не объявлен, поэтому компилятор не дал бы написать [a fetch] напрямую. Мы приводим указатель к Dog * — (Dog *)a — и тем самым обещаем компилятору: «верь, тут собака». А проверка respondsToSelector: гарантирует, что обещание не пустое.

Соберём и запустим:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/09-introspection.m -o introspection
./introspection
```

Вывод:

```
[a class]      = Dog
[a superclass] = Animal
kind of Dog?   да
kind of Animal? да
kind of NSObject? да
member of Dog? да
member of Animal? нет
умеет fetch?   да
умеет addObject? нет
    Dog: принёс мячик
```

Всё сходится: фактический класс — `Dog`, родитель — `Animal`; по `kind` собака принадлежит всей ветке, по `member` — только своему классу; `fetch` умеет, `addObject`: — нет.

Абстрактный базовый класс по соглашению

`Animal` из первого примера можно было создать напрямую (`[[Animal alloc] init...]`), и он давал невнятный звук. Иногда же базовый класс — это чистый «каркас», который сам по себе бессмыслен, и создавать его экземпляры нельзя. Такой класс называют **абстрактным**.

В некоторых языках есть ключевое слово `abstract`, запрещающее создание экземпляра. В Objective-C **такого слова нет**. Абстрактность здесь — это **соглашение**, а не запрет компилятора. Приём такой: базовый метод, который обязан переопределить подкласс, делаем заглушкой, которая громко падает, если её всё-таки вызвали напрямую. Тогда забывчивость вылезет сразу при запуске, а не тихо даст неверный результат.

Файл `09-abstract.m` — фигуры с площадью:

```
#import <Foundation/Foundation.h>

@interface Shape : NSObject
- (double)area;
@end

@implementation Shape
```

```
- (double)area {
    [NSException raise:NSInternalInconsistencyException
                 format:@"%@" обязан переопределить -area", [self class]];
    return 0;    /* сюда выполнение уже не дойдёт */
}
@end

@interface Circle : Shape
@property (nonatomic) double radius;
- (instancetype)initWithRadius:(double)radius;
@end

@implementation Circle
- (instancetype)initWithRadius:(double)radius {
    self = [super init];
    if (self) {
        _radius = radius;
    }
    return self;
}
- (double)area {
    return M_PI * self.radius * self.radius;
}
@end

@interface Square : Shape
@property (nonatomic) double side;
- (instancetype)initWithSide:(double)side;
@end

@implementation Square
- (instancetype)initWithSide:(double)side {
    self = [super init];
    if (self) {
        _side = side;
    }
    return self;
}
}
```

```
- (double)area {
    return self.side * self.side;
}
@end

int main(void) {
    @autoreleasepool {
        NSArray *shapes = @[ [[Circle alloc] initWithRadius:2.0],
                             [[Square alloc] initWithSide:3.0] ];
        for (Shape *s in shapes) {
            NSLog(@"%@: площадь = %.2f", [s class], [s area]);
        }

        @try {
            Shape *raw = [[Shape alloc] init];
            [raw area];
        } @catch (NSException *ex) {
            NSLog(@"Поймали исключение: %@", ex.reason);
        }
    }
    return 0;
}
```

Что здесь нового:

- Shape — абстрактный базовый класс. Его area не считает площадь, а бросает **исключение** через [NSException raise:format:]. Сообщение включает [self class] — фактический класс «нарушителя», чтобы сразу понять, кто забыл переопределить метод.
- Circle и Square наследуют от Shape и **обязаны** дать свой area — что и делают. Площадь круга — $M_PI * r * r$, квадрата — $side * side$. M_PI — константа числа π из заголовка `<math.h>` (Foundation подключает его сам).
- В цикле снова полиморфизм: перебираем фигуры через тип базы Shape *, но [s area] уходит фактическому классу — каждый считает по-своему.
- @try { ... } @catch (NSException *ex) { ... } — **перехват исключения**. Внутри @try мы намеренно создаём «голую» Shape и зовём area. Заглушка бросает исключение, выполнение прыгает в @catch, и мы печатаем ex.reason — причину. Без @try программа бы здесь упала.

Отличие от Си. Механизма исключений (`@try/@catch`) в чистом Си нет — там об ошибках сигналижат кодами возврата и проверяют их вручную. Исключения и `NSException` — часть объектной надстройки Objective-C.

Соберём и запустим:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/09-abstract.m -o abstract
./abstract
```

Вывод:

```
Circle: площадь = 12.57
Square: площадь = 9.00
Поймали исключение: Shape обязан переопределить -area
```

Круг и квадрат посчитали площадь, а попытка спросить площадь у «голой» `Shape` честно сообщила, что так делать нельзя. Абстрактный класс защитил себя сам — без всякого ключевого слова `abstract`, одним соглашением.

Как это работает

Соберём картину полиморфизма в одну мысль. Когда ты пишешь `[someone describe]`:

1. `someone` — переменная типа `Animal *`, но внутри лежит объект `Dog`. Объект носит свой класс с собой (через `isa`).
2. Отправляется сообщение `describe`. Runtime смотрит на фактический класс объекта — `Dog`.
3. У `Dog` есть свой `describe` — берётся он. Внутри он зовёт `[super describe]` (родительский) и дописывает породу.
4. Родительский `describe` зовёт `[self speak]`. `self` — всё тот же объект-собака, поэтому `speak` снова ищется с `Dog` — находится `@"Гав-гав!"`.

Тип переменной не участвует в выборе метода ни разу. Решает только фактический класс объекта, и решает это runtime во время выполнения. В этом сила и гибкость Objective-C — и в этом его отличие от языков, где метод выбирается по типу переменной ещё на компиляции.

Проверяем

Файлы главы:

- `code/09-animals.m` — наследование и полиморфизм (зоопарк);
- `code/09-introspection.m` — `class`, `superclass`, `isKindOfClass:`, `isMemberOfClass:`, `respondsToSelector:`;

- `code/09-abstract.m` — абстрактный базовый класс и исключение.

Каждый собирается и запускается одинаково:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/09-animals.m -o animals && ./animals
```

Ожидаемый вывод приведён выше под каждым файлом (дата, имя программы и числа в реальном выводе NSLog будут свои).

Частые ошибки

- **Забыл `[super init]` (или вызвал не тот `init`).** Если в своём инициализаторе не позвать инициализатор родителя, базовая часть объекта останется ненастроенной. Всегда начинай `self = [super initWithName:...]` (или `[super init]`), проверяй `if (self)`, потом задавай своё.
- **Думаешь, что вызовется метод по типу переменной.** Нет: вызывается метод **фактического класса** объекта. `Animal *a`, в котором лежит `Dog`, на `speak` ответит по-собачьи. Тип переменной влияет только на то, какие сообщения тебе разрешит написать компилятор, а не на то, чей метод сработает.
- **Зовёшь `[a fetch]`, когда `fetch` нет в типе `Animal *`.** Компилятор откажется: в `Animal` такого метода не объявлено. Либо приводи тип `((Dog *)a)` после проверки `respondsToSelector:`, либо объявляй метод там, где он реально нужен.
- **Путаешь `isKindOfClass:` и `isMemberOfClass:`.** Первый — «этот класс или потомок» (нужен почти всегда), второй — «ровно этот класс». `[dog isKindOfClass:[Animal class]]` — это «нет», хоть собака и животное.
- **Бесконечная рекурсия через `super`.** `[super describe]` зовёт версию родителя. Если перепутать и написать `[self describe]` внутри самого `describe` — метод позовёт сам себя без конца, стек вызовов переполнится, и программа упадёт: терминал напишет `segmentation fault`.
- **Создаёшь экземпляр абстрактного класса.** Если базовый класс — каркас с методами-заглушками, не создавай его напрямую. Заглушка с `NSException` напомнит об этом падением при первом же вызове.

Упражнения

1. Добавь третий подкласс `Cow` (корова) с голосом `@“Му-у!”` и положи её в зоопарк. Убедись, что в общем цикле она «замычала» без правок цикла.

2. Дай Cat свойство `indoor` (домашняя ли) и свой инициализатор `initWithName:indoor:`, который зовёт `[super initWithName:]`. Допиши `describe` у Cat через `[super describe]`, добавив строку про домашность.
3. В `09-introspection.m` создай ещё переменную `id obj = @"строка";` и проверь на ней `isKindOfClass:[NSString class]` и `respondsToSelector:@selector(uppercaseString)`. Объясни результат.
4. В `09-abstract.m` добавь `Triangle` с основанием и высотой и площадью $0.5 * base * height$. Проверь его в общем цикле.
5. Намеренно убери у `Square` метод `area` и запусти программу. Прочитай текст исключения — теперь ты знаешь, как выглядит «забыл переопределить».
6. Сделай у `Animal` метод `legs` (число ног), верни в базовом 4, а у воображаемой `Bird` переопредели на 2. Выведи число ног у каждого зверя в зоопарке.

Что мы получили

Мы научились строить семьи классов. Подкласс объявляется одним двоеточием (`Dog : Animal`) и бесплатно получает поля, свойства и методы родителя — и всё, что тот сам унаследовал вплоть до общего корня `NSObject`. Метод можно переопределить (дать свою версию) или дополнить, позвав родительскую через `[super ...]`; инициализаторы выстраиваются в цепочку снизу вверх по дереву. Полиморфизм оказался прямым следствием модели сообщений из главы 5: метод всегда выбирается по фактическому классу объекта, а не по типу переменной, и выбирает его runtime. Мы научились расспрашивать объект о нём самом — `class`, `superclass`, `isKindOfClass:`, `isMemberOfClass:`, `respondsToSelector:` — и сделали абстрактный базовый класс по соглашению.

Дальше — глава 10 про **протоколы**: способ договориться о наборе методов, который класс обязуется поддерживать, *не наследуясь* ни от кого. Если наследование отвечает на вопрос «кто этот объект», то протокол — на вопрос «что он умеет», и эти два механизма отлично дополняют друг друга.

Документация Apple

- Programming with Objective-C → **Defining Classes** — объявление классов, суперклассы, переопределение методов, `[super ...]` — [developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ ProgrammingWithObjectiveC/DefiningClasses/DefiningClasses.html](http://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/DefiningClasses/DefiningClasses.html)
- Programming with Objective-C → **Working with Objects** — отправка сообщений, `self`, динамическая диспетчеризация.
- `NSObject` — `class`, `superclass`, `isKindOfClass:`, `isMemberOfClass:`, `respondsToSelector:`, developer.apple.com/documentation/objectivec/nsobject-swift.class

- `NSException` — developer.apple.com/documentation/foundation/nsexception

Глава 10. Протоколы и делегаты

В прошлой главе мы строили семьи классов: `Dog` — это `Animal`, а тот — `NSObject`. Наследование отвечает на вопрос «кто этот объект». Но часто важно другое: «что он **умеет**». Кнопке всё равно, кто именно нажал, — ей важно, чтобы этот «кто-то» умел принять сообщение «меня нажали». Загрузчику файла всё равно, кто слушает результат, — лишь бы тот умел ответить на «готово» и «ошибка».

Для таких договорённостей в Objective-C есть **протокол** (`protocol`) — список методов без реализации, который класс обязуется поддерживать. В этой главе мы разберём протоколы по косточкам и соберём на них самый важный паттерн всего Сосоа — **делегирование** (`delegation`): объект сообщает о событиях «кому-то на стороне», не зная и не желая знать, кто этот «кто-то».

Что мы сделаем

- Поймём, что такое `@protocol` и зачем он, когда наследование не годится.
- Объявим протокол и заставим класс его принять.
- Разберём `@required` и `@optional` методы и проверку `respondsToSelector:`.
- Освоим тип `id<Protocol>` — «любой объект, лишь бы умел эти сообщения».
- Проверим принятие протокола: `conformsToProtocol:` и `@protocol(Name)`.
- Построим делегирование с нуля: наивный жёсткий вариант → гибкий через протокол-делегат.
- Узнаем, почему свойство `delegate` делают `weak`.

Зачем протокол, если есть наследование

Наследование — мощная штука, но у него два жёстких ограничения.

Во-первых, родитель один. В Objective-C класс наследует ровно от одного суперкласса. Если объект должен «уметь» сразу две независимые вещи — скажем, и сериализоваться в текст, и сравниваться с другими, — в одного родителя это не уложить.

Во-вторых, наследование связывает родственников. Унаследовать общий метод можно только у общего предка. А что, если умение нужно совершенно не родственным классам? Книга и подписка на музыку — это разные вещи без общего осмысленного предка. Но у обеих есть цена, и обе можно положить в корзину магазина. Заводить ради этого искусственный класс-прародитель «ПродаваемаяСущность» — натягивание совы на глобус.

Протокол решает обе беды. Он описывает **набор умений** отдельно от дерева классов. Любой класс — хоть Book, хоть Subscription, хоть твой собственный — может объявить: «я поддерживаю этот протокол», и обязуется реализовать перечисленные в нём методы. Классы при этом остаются чужими друг другу, а умение у них общее.

Отличие от Си. В Си «контракт» имитируют вручную: заводят struct с указателями на функции — самодельную таблицу методов (vtable). Каждый «тип» обязан заполнить эту таблицу нужными функциями, и никто не проверит, что он сделал это правильно: забыл указатель — получишь NULL и падение в рантайме. @protocol — это контракт, **встроенный в язык**: компилятор сам следит, что класс, объявивший поддержку протокола, реализовал все обязательные методы, и предупредит, если нет.

Объявляем первый протокол

Протокол объявляется ключевым словом @protocol, дальше имя, потом список методов, и в конце @end — как у класса, только без тел методов. Опишем контракт «продаваемой вещи»:

```
@protocol Sellable <NSObject>
- (NSString *)title;
- (double)price;
@end
```

Разберём по знакам.

- @protocol Sellable — объявляем протокол с именем Sellable. Имена протоколов, как и классов, пишут с большой буквы.
- <NSObject> в угловых скобках — этот протокол **сам наследует** от протокола NSObject. Да, у NSObject есть не только класс, но и одноимённый протокол: в нём перечислены базовые умения любого объекта — respondsToSelector:, conformsToProtocol:, isEqual:, class и прочие. Указывать <NSObject> в своих протоколах — стандарт Apple: так гарантируется, что объект, принявший твой протокол, точно умеет отвечать на эти базовые сообщения. Почти все протоколы пишут именно так.

- `– (NSString *)title; и – (double)price;` — два метода контракта. Минус – означает метод экземпляра (как в главе 6). Тел нет — только объявления: протокол говорит, *что* должно быть, а *как* — дело принявшего класса.
- `@end` закрывает объявление.

Этот протокол не несёт ни байта кода. Это чистое обещание: «у того, кто меня принял, есть методы `title` и `price`».

Класс принимает протокол

Чтобы класс заявил «я поддерживаю `Sellable`», имя протокола ставят в угловых скобках после суперкласса в `@interface`:

```
@interface Book : NSObject <Sellable>
@property (nonatomic, copy) NSString *name;
@property (nonatomic, copy) NSString *author;
@property (nonatomic) double cost;
– (instancetype)initWithName:(NSString *)name
    author:(NSString *)author
    cost:(double)cost;
@end
```

`Book : NSObject <Sellable>` читается так: «класс `Book` наследует от `NSObject` **и принимает** протокол `Sellable`». Двоеточие — про наследование (родитель), угловые скобки — про протокол (контракт). Это разные вещи, и стоят они рядом.

Приняв протокол, `Book` берёт на себя обязательство реализовать `title` и `price`. Если забудешь — компилятор предупредит. Реализуем:

```
@implementation Book
– (instancetype)initWithName:(NSString *)name
    author:(NSString *)author
    cost:(double)cost {
    self = [super init];
    if (self) {
        _name = [name copy];
        _author = [author copy];
        _cost = cost;
    }
    return self;
}
```

```
- (NSString *)title {
    return [NSString stringWithFormat:@"«%@», %@", self.name, self.author];
}
- (double)price {
    return self.cost;
}
@end
```

Здесь нет ничего нового по сравнению с главой 8, кроме того, что методы `title` и `price` появились **не просто так**, а во исполнение контракта `Sellable`.

Теперь — главный фокус. Заведём **совсем другой** класс, не родственник `Book`, который тоже принимает `Sellable`:

```
@interface Subscription : NSObject <Sellable>
@property (nonatomic, copy) NSString *service;
@property (nonatomic) int months;
@property (nonatomic) double perMonth;
- (instancetype)initWithService:(NSString *)service
                        months:(int)months
                        perMonth:(double)perMonth;
@end

@implementation Subscription
- (instancetype)initWithService:(NSString *)service
                        months:(int)months
                        perMonth:(double)perMonth {
    self = [super init];
    if (self) {
        _service = [service copy];
        _months = months;
        _perMonth = perMonth;
    }
    return self;
}
- (NSString *)title {
    return [NSString stringWithFormat:@"Подписка %@ (%d мес.)",
        self.service, self.months];
}
- (double)price {
```

```

    return self.months * self.perMonth;
}
@end

```

Book и Subscription не связаны наследованием — у них нет общего предка, кроме NSObject. Но обе умеют title и price, потому что обе приняли Sellable. Это ровно тот случай, где наследование бессильно, а протокол подходит идеально.

Тип id<Protocol> — «любой, лишь бы умел»

Теперь самое интересное. Раз Book и Subscription оба умеют title и price, можно работать с ними **единообразно**, не зная их класса. Для этого есть особый тип:

```

id<Sellable> book = [[Book alloc] initWithName:@"Чистый код"
                    author:@"P. Мартин"
                    cost:5900];

```

id<Sellable> читается так: «какой угодно объект (id), **при условии** что он принял протокол Sellable». Вспомни главу 5: id — это «любой объект». Угловые скобки сужают: не любой вообще, а только тот, что умеет сообщения из Sellable. Компилятор после этого разрешит слать такой переменной title и price, и не разрешит ничего лишнего.

Отличие от Си. id<Sellable> — это динамический тип «что угодно, лишь бы умело эти сообщения». В Си тип переменной жёстко привязан к конкретной структуре: struct Book * — это только Book и ничего другого. Сказать «указатель на любой тип, у которого есть поля-функции title и price» язык Си не умеет — пришлось бы вручную городить void-указатели и таблицы функций, теряя всякую проверку компилятора. Здесь же компилятор и проверит контракт, и разрешит ровно те сообщения, что в него входят.

Есть и второй вид записи — когда тип объекта известен, но хочется дополнительно подчеркнуть протокол:

```

Book *b = ...;           // это точно Book
NSObject<Sellable> *x = ...; // это NSObject, и он умеет Sellable

```

Запись Тип<Protocol> * означает «объект данного типа, и вдобавок он принимает протокол». На практике чаще всего пишут именно id<Protocol> — когда конкретный класс не важен, важно лишь умение.

Проверяем контракт: conformsToProtocol:

Иногда нужно спросить объект прямо во время выполнения: «а ты вообще принял такой-то протокол?». Для этого есть метод `conformsToProtocol:` (он достался нам как раз от протокола `NSObject`):

```
[book conformsToProtocol:@protocol(Sellable)]
```

Возвращает BOOL — YES, если класс объекта (или любой его предок) объявил поддержку протокола. Обрати внимание на аргумент: нельзя написать просто Sellable — это имя само по себе не значение. Нужна обёртка @protocol(Sellable): она превращает имя протокола в значение (объект типа Protocol *), которое можно передать в метод. Сравни с @selector(name) из главы 9, где имя метода так же превращалось в значение-селектор, и с [Book class], где имя класса — в объект-класс. В Objective-C почти всё можно «опредметить»: и метод, и класс, и протокол.

Собираем корзину

Сложим обе вещи в массив и посчитаем сумму, не глядя на классы:

```
static double cartTotal(NSArray *cart) {
    double total = 0;
    for (id<Sellable> item in cart) {
        NSLog(@" %@ - %.0f ¯", [item title], [item price]);
        total += [item price];
    }
    return total;
}
```

Функция `cartTotal` принимает массив и перебирает его быстрым циклом (глава 9). Каждый элемент она берёт в переменную типа `id<Sellable>` — ей безразлично, книга это или подписка, важно лишь, что объект умеет `title` и `price`. Шлёт оба сообщения, печатает строку, копит сумму.

Все куски выше вместе с main и составляют файл 10-protocol.m. main в нём такой:

[illegible]


```

perMonth:1490];

NSLog(@"Book принял Sellable?      %@",
      [book conformsToProtocol:@protocol(Sellable)] ? @"да" : @"нет");
NSLog(@"Subscription принял Sellable? %@",
      [sub conformsToProtocol:@protocol(Sellable)] ? @"да" : @"нет");

NSLog(@"--- корзина ---");
NSArray *cart = @[ book, sub ];
double total = cartTotal(cart);
NSLog(@"Итого: %.0f ₴", total);
}
return 0;
}

```

Компилируем и запускаем:

```

clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/10-protocol.m -o protocol
./protocol

```

Вывод (префикс NSLog с датой и номерами опущен):

```

Book принял Sellable?      да
Subscription принял Sellable? да
--- корзина ---
«Чистый код», Р. Мартин — 5900 ₴
Подписка Музыка (12 мес.) — 17880 ₴
Итого: 23780 ₴

```

Два чужих друг другу класса легли в один массив и обработались одним циклом — потому что у них общий контракт, а не общий предок. Это и есть то, чего наследование дать не может.

Паттерн делегирования: строим с нуля

Теперь главное применение протоколов. Возьмём типичную задачу.

Разбираем задачу руками

Представь объект-«загрузчик» (Downloader): он получает адрес, что-то качает и должен **сообщить о результате** — «готово, вот данные» или «не вышло, вот ошибка». Вопрос: кому сообщить?

Загрузчик не должен этого знать. Сегодня результат печатают в консоль, завтра — показывают в окне, послезавтра — пишут в файл. Если зашить конкретного получателя внутрь загрузчика, мы привяжем его к одному сценарию намертво. А хочется один раз написать Downloader и переиспользовать его где угодно.

Шаг 1. Наивно — жёсткая связь

Сделаем в лоб: пусть Downloader сам держит внутри объект-печатник ConsoleLogger и зовёт его.

```
@interface ConsoleLogger : NSObject
- (void)printResult:(NSString *)data;
@end

@implementation ConsoleLogger
- (void)printResult:(NSString *)data {
    NSLog(@"  [+] готово: %@", data);
}
@end

@interface Downloader : NSObject
@property (nonatomic, copy) NSString *url;
@property (nonatomic, strong) ConsoleLogger *logger;    // жёстко прибит
- (instancetype)initWithURL:(NSString *)url;
- (void)start;
@end

@implementation Downloader
- (instancetype)initWithURL:(NSString *)url {
    self = [super init];
    if (self) {
        _url = [url copy];
        _logger = [[ConsoleLogger alloc] init];
    }
    return self;
}

- (void)start {
```

```

    NSLog(@"Downloader: качаю %@ ...", self.url);
    NSString *data = [NSString stringWithFormat:@"<содержимое %@>", self.url];
    [self.logger printResult:data];
}
@end

```

Наивный вариант целиком лежит в `code/10-delegate-naive.m` — собери и запусти его, он честно работает:

```

clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/10-delegate-naive.m -o naive && ./naive

```

```

Downloader: качаю http://site.kz/file.txt ...
[+] готово: <содержимое http://site.kz/file.txt>

```

Печатает. Но приглядишься к строке `@property ... ConsoleLogger *logger`. Загрузчик намертво знает про `ConsoleLogger`. Он умеет докладывать **только** ему. Захотел показать результат в окне вместо консоли — лезь и переписывай сам `Downloader`. А ещё мы вообще не предусмотрели случай ошибки. Связь жёсткая, и это плохо: класс, который должен быть универсальным, прибит гвоздями к одному способу вывода.

Шаг 2. Разворачиваем зависимость через протокол

Идея делегирования: пусть `Downloader` не знает конкретного получателя. Он знает лишь **контракт** — набор сообщений, которые шлёт «по результату». Кто их примет — не его забота. Этот контракт и есть протокол, а получателя называют **делегатом** (`delegate`) — тем, кому загрузчик «делегировать» (передаёт, поручает) реакцию на события.

Объявим контракт:

```

@class Downloader; /* анонс: имя есть, описание ниже */

@protocol DownloaderDelegate <NSObject>
- (void)downloader:(Downloader *)downloader
  didFinishWithData:(NSString *)data;
- (void)downloader:(Downloader *)downloader
  didFailWithError:(NSString *)reason;
@end

```

Два момента.

`@class Downloader;` — это **предварительный анонс** (`forward declaration`): «есть такой класс `Downloader`, подробности ниже». Он нужен потому, что протокол ссылается на тип `Downloader`

* в своих методах, а полное @interface Downloader идёт ниже по файлу. Анонс успокаивает компилятор: имя существует, детали подождут.

Имена методов делегата — downloader:didFinishWithData: и downloader:didFailWithError: — построены по **стилю Apple**: первым параметром идёт сам отправитель (downloader), дальше суть события. Так делегат, обслуживающий сразу несколько загрузчиков, поймёт, от кого именно пришло сообщение. Этому шаблону следуют все системные делегаты Cocoa (tableView:didSelectRowAtIndexPath: и подобные).

Теперь Downloader хранит не конкретный класс, а **делегата по контракту**:

```
@interface Downloader : NSObject
@property (nonatomic, copy) NSString *url;
@property (nonatomic, weak) id<DownloaderDelegate> delegate;
- (instancetype)initWithURL:(NSString *)url;
- (void)start;
@end
```

Свойство delegate имеет тип id<DownloaderDelegate> — «любой объект, принявший протокол DownloaderDelegate». Загрузчику всё равно, кто это. Про атрибут weak — через минуту, сначала допишем логику:

```
@implementation Downloader
- (instancetype)initWithURL:(NSString *)url {
    self = [super init];
    if (self) {
        _url = [url copy];
    }
    return self;
}
- (void)start {
    NSLog(@"Downloader: качаю %@ ...", self.url);
    if ([self.url containsString:@"secret"]) {
        [self.delegate downloader:self
                    didFailWithError:@"403 Доступ запрещён"];
    } else {
        NSString *data = [NSString stringWithFormat:@"<содержимое %@>",
                                                    self.url];
        [self.delegate downloader:self didFinishWithData:data];
    }
}
```

```
}  
@end
```

start имитирует сеть: адреса со словом secret «падают», остальные «успешны». В обоих случаях загрузчик не печатает результат сам — он **сообщает делегату**: [self.delegate downloader:self didFinishWithData:data] или ...didFailWithError:.... Метод containsString: у NSString возвращает YES, если строка содержит подстроку.

Кто будет делегатом? Любой класс, принявший DownloaderDelegate:

```
@interface ConsoleLogger : NSObject <DownloaderDelegate>  
@end  
  
@implementation ConsoleLogger  
- (void)downloader:(Downloader *)downloader  
  didFinishWithData:(NSString *)data {  
    NSLog(@"  [+] %@ готово: %@", downloader.url, data);  
}  
- (void)downloader:(Downloader *)downloader  
  didFailWithError:(NSString *)reason {  
    NSLog(@"  [-] %@ ошибка: %@", downloader.url, reason);  
}  
@end
```

Связываем загрузчик с делегатом в main:

```
int main(void) {  
    @autoreleasepool {  
        ConsoleLogger *logger = [[ConsoleLogger alloc] init];  
  
        Downloader *d1 =  
            [[Downloader alloc] initWithURL:@"http://site.kz/file.txt"];  
        d1.delegate = logger;  
        [d1 start];  
  
        Downloader *d2 =  
            [[Downloader alloc] initWithURL:@"http://site.kz/secret.txt"];  
        d2.delegate = logger;  
        [d2 start];  
  
        /* Делегат не назначен (nil). start не упадёт. */  
    }  
}
```

```
Downloader *d3 =
    [[Downloader alloc] initWithURL:@"http://site.kz/orphan.txt"];
    [d3 start];
}
return 0;
}
```

Назначение `d1.delegate = logger` — обычное присваивание свойства. Загрузчик и делегат соединены, но связаны **слабо**: загрузчик видит в делегате лишь «объект, умеющий два метода контракта».

Собираем `10-delegate.m` и запускаем:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/10-delegate.m -o delegate
./delegate
```

Вывод:

```
Downloader: качаю http://site.kz/file.txt ...
[+] http://site.kz/file.txt готово: <содержимое http://site.kz/file.txt>
Downloader: качаю http://site.kz/secret.txt ...
[-] http://site.kz/secret.txt ошибка: 403 Доступ запрещён
Downloader: качаю http://site.kz/orphan.txt ...
```

Разберём вывод.

- Первый загрузчик доложил делегату об успехе — `[+]`.
- Второй (адрес с `secret`) доложил об ошибке — `[-]`.
- Третий (`orphan.txt`) напечатал «качаю...» и **замолчал**. Делегат ему не назначили, значит `self.delegate` равен `nil`. А сообщение к `nil` в Objective-C — это **тихий no-op**: ничего не происходит, программа не падает (см. главу 5). Поэтому отсутствие делегата абсолютно безопасно.

Теперь `Downloader` ничего не знает про `ConsoleLogger`. Захотим выводить результат в окно — напишем `WindowReporter : NSObject <DownloaderDelegate>` и подсунем его как делегата, **не трогая Downloader**. Жёсткая связь из шага 1 разорвана. Это и есть реальная выгода паттерна: один универсальный класс событий и сколько угодно сменных получателей.

Почему `delegate` — это `weak`

Вернёмся к строке `@property (nonatomic, weak) id<DownloaderDelegate> delegate;`. Почему `weak`, а не `strong`?

Атрибут `strong` (по умолчанию для объектов) означает «держу объект живым, пока ссылаюсь» — это сильная ссылка (`strong reference`). `weak` — «ссылаюсь, но не держу»: слабая ссылка (`weak reference`) не мешает объекту умереть, а когда тот умирает, сама автоматически становится `nil`.

В делегировании это важно из-за **взаимных ссылок**. Очень часто делегат сам владеет загрузчиком (создал его, держит сильной ссылкой), а загрузчик ссылается на делегата обратно. Если бы обе ссылки были сильными, получилось бы кольцо: делегат держит загрузчика, загрузчик держит делегата. Каждый не даёт другому умереть — и оба зависают в памяти навсегда, даже когда больше никому не нужны. Это **цикл удержания** (`retain cycle`), классическая утечка памяти под ARC.

Лекарство — разорвать кольцо: ссылку «назад», от загрузчика к делегату, делают слабой. Делегат держит загрузчика сильно, загрузчик держит делегата слабо — кольца нет, всё освобождается вовремя. Поэтому **свойство `delegate` почти всегда объявляют `weak`**. Полностью циклы удержания, `strong/weak` и устройство ARC мы разберём в главе 13; пока запомни правило: `delegate` → `weak`.

Отличие от Си. В Си нет ни подсчёта ссылок, ни «слабых указателей», которые сами обнуляются. Указатель в Си — это голый адрес: на кого показывал, на того и показывает, даже если объект по этому адресу уже освобождён через `free`. Связка `strong/weak` — часть объектной модели памяти Objective-C, которой в чистом Си попросту нет.

`@required` и `@optional`

В наших протоколах все методы были **обязательными**: принял протокол — реализуй всё. По умолчанию методы протокола именно такие — `@required`. Но часть умений можно сделать **необязательной**, пометив `@optional`. Тогда делегат волен реализовать метод или нет.

Соберём `10-optional.m`. Пусть задача (`Task`) сообщает делегату о трёх событиях: «начала» и «прогресс» — по желанию, «завершила» — обязательно:

```
@class Task;

@protocol TaskDelegate <NSObject>
@required
```

```

- (void)taskDidFinish:(Task *)task;          /* обязан реализовать каждый */
@optional
- (void)taskDidStart:(Task *)task;          /* по желанию */
- (void)task:(Task *)task didProgress:(int)percent; /* по желанию */
@end

```

Слова `@required` и `@optional` — это **переключатели**: всё, что идёт после `@required`, обязательно; всё после `@optional` — нет; действуют до следующего переключателя или до `@end`. Здесь `taskDidFinish:` обязателен, а `taskDidStart:` и `task:didProgress:` — нет.

Сама задача устроена так же, как `Downloader`: имя и слабая ссылка на делегата.

```

@interface Task : NSObject
@property (nonatomic, copy) NSString *name;
@property (nonatomic, weak) id<TaskDelegate> delegate;
- (instancetype)initWithName:(NSString *)name;
- (void)run;
@end

```

`initWithName:` пишется точно как у `Downloader` (`[super init]`, потом `_name = [name copy]`), а весь интерес — в методе `run`.

Вот тонкость, без которой будет падение. Обязательный метод можно звать смело: если класс делегата его забыл, компилятор предупредил об этом ещё при сборке. А **опциональный метод сначала надо проверить**: вдруг делегат его не реализовал? Если послать сообщение, которого у объекта нет, программа упадёт с `unrecognized selector`. Спасает знакомый по главе 9 `respondsToSelector::`

```

- (void)run {
    /* Опциональный метод — сначала спрашиваем, умеет ли делегат. */
    if ([self.delegate respondsToSelector:@selector(taskDidStart:)]) {
        [self.delegate taskDidStart:self];
    }
    for (int p = 50; p <= 100; p += 50) {
        if ([self.delegate respondsToSelector:@selector(task:didProgress:)]) {
            [self.delegate task:self didProgress:p];
        }
    }
    /* Обязательный метод — зовём напрямую, он гарантированно есть. */
    [self.delegate taskDidFinish:self];
}

```


Перед каждым опциональным вызовом стоит `if ([self.delegate respondsToSelector:@selector(...)])` — «а делегат вообще умеет это сообщение?». Если да — шлём, если нет — спокойно пропускаем. Обязательный `taskDidFinish`: зовём без проверки. Это **стандартный шаблон** работы с делегатом: `required` — напрямую, `optional` — через `respondsToSelector`.

Заведём двух делегатов — минимального и полного:

```
@interface QuietWatcher : NSObject <TaskDelegate>
@end

@implementation QuietWatcher
- (void)taskDidFinish:(Task *)task {
    NSLog(@"[тихий] «%@» завершена", task.name);
}
@end

@interface ChattyWatcher : NSObject <TaskDelegate>
@end

@implementation ChattyWatcher
- (void)taskDidStart:(Task *)task {
    NSLog(@"[болтливый] «%@» стартовала", task.name);
}

- (void)task:(Task *)task didProgress:(int)percent {
    NSLog(@"[болтливый] «%@»: %d%%", task.name, percent);
}

- (void)taskDidFinish:(Task *)task {
    NSLog(@"[болтливый] «%@» завершена", task.name);
}
@end
```

`QuietWatcher` реализовал только обязательный метод — и это законно, протокол выполнен. `ChattyWatcher` реализовал всё. В `main` дадим каждому свою задачу:

```
int main(void) {
    @autoreleasepool {
        QuietWatcher *quiet = [[QuietWatcher alloc] init];
        Task *t1 = [[Task alloc] initWithName:@"Резервная копия"];
        t1.delegate = quiet;
        NSLog(@"--- тихий наблюдатель (только required) ---");
        [t1 run];

        ChattyWatcher *chatty = [[ChattyWatcher alloc] init];
```

```
Task *t2 = [[Task alloc] initWithName:@"Импорт данных"];
t2.delegate = chatty;
NSLog(@"--- болтливый наблюдатель (required + optional) ---");
[t2 run];
}
return 0;
}
```

Обрати внимание: делегат лежит в **отдельной сильной** переменной (quiet, chatty), а уже потом присваивается слабому свойству t1.delegate. Так и надо: свойство weak объект не удерживает, поэтому кто-то другой обязан держать его сильно, иначе под ARC он умрёт сразу после создания и делегат окажется nil. Это частая ловушка — вернёмся к ней в «Частых ошибках».

Собираем и запускаем:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/10-optional.m -o optional
./optional
```

Вывод:

```
--- тихий наблюдатель (только required) ---
[тихий] «Резервная копия» завершена
--- болтливый наблюдатель (required + optional) ---
[болтливый] «Импорт данных» стартовала
[болтливый] «Импорт данных»: 50%
[болтливый] «Импорт данных»: 100%
[болтливый] «Импорт данных» завершена
```

Тихий наблюдатель выдал одну строку: опциональные taskDidStart: и task:didProgress: он не реализовал, и respondsToSelector: честно их пропустил — без падения. Болтливый получил все четыре уведомления. Один и тот же Task, один и тот же run — поведение делегатов разное, потому что опциональные методы у них разные. Программа не упала ни разу.

Формальные и неформальные протоколы

Маленькая историческая справка, чтобы термины не застали врасплох в чужом коде. То, что мы делали, — **формальный протокол** (formal protocol): объявленный через @protocol, проверяемый компилятором.

До появления @optional (а это сравнительно позднее добавление) все методы протокола были обязательными, и «необязательные» умения оформляли иначе — через **неформальный про-**

токол (informal protocol). Это была просто **категория** на `NSObject` (категории — тема главы 11): набор методов, которые любой объект *мог бы* реализовать, но не обязан. Проверяли их всё тем же `respondsToSelector:`. Формальной гарантии компилятора не было — отсюда и «неформальный».

Сегодня неформальные протоколы почти не нужны: `@optional` решает ту же задачу формально и под присмотром компилятора. Встретишь термин в старом коде или документации — знай, что это устаревший предшественник `@optional`-методов. Пиши формальные протоколы.

Как это работает

Соберём картину делегирования в одну мысль. Когда `Downloader` хочет доложить о результате:

1. Он шлёт сообщение **своему делегату** — `[self.delegate downloader:self didFinishWithData:data]`. Тип делегата — `id<DownloaderDelegate>`, конкретный класс загрузчику неизвестен.
2. Если делегат `nil` (не назначен) — сообщение тихо пропадает, ничего не происходит.
3. Если делегат назначен — runtime по фактическому классу делегата (через `isa`, глава 9) находит нужный метод и выполняет его. У `ConsoleLogger` это печать в консоль; у другого делегата был бы другой код.
4. Опциональные методы загрузчик/задача предварительно проверяет `respondsToSelector:` — шлёт, только если делегат их умеет.

Загрузчик и делегат соединены **тонкой ниткой контракта**, а не жёстким знанием друг о друге. Заменить одну сторону можно, не трогая другую. Вот почему делегирование — самый частый способ «обратной связи» во всём Cocoa: от таблиц и текстовых полей до сетевых сессий.

Проверяем

Файлы главы:

- `code/10-protocol.m` — объявление протокола, принятие двумя чужими классами, `id<Sellable>`, `conformsToProtocol:`, `@protocol(...)`;
- `code/10-delegate.m` — паттерн делегирования, `weak`-свойство `delegate`, безопасность сообщения к `nil`-делегату;
- `code/10-optional.m` — `@required/@optional` и проверка `respondsToSelector:` перед опциональным вызовом.

Каждый собирается и запускается одинаково:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/10-protocol.m -o protocol && ./protocol
```

Ожидаемый вывод приведён выше под каждым файлом (дата, имя программы и числа в реальном выводе NSLog будут свои).

Частые ошибки

- **Забыл реализовать обязательный метод протокола.** Объявил `<Sellable>`, но не написал `price`? Компилятор предупредит: `method 'price' in protocol 'Sellable' not implemented`. Прочитай предупреждение и допиши метод — протокол на то и контракт.
- **Зовёшь опциональный метод напрямую.** Если послать опциональный метод, которого у делегата нет, программа упадёт с `unrecognized selector sent to instance`. Перед каждым опциональным вызовом — `if ([delegate respondsToSelector:@selector(...)])`.
- **Сделал delegate сильным (strong).** Очень вероятен цикл удержания (`delegate` ↔ объект), и оба зависнут в памяти. Свойство `delegate` объявляй `weak`.
- **Присвоил weak-делегату временный объект.** Строка `obj.delegate = [[Watcher alloc] init];` под ARC бесполезна: у нового объекта нет сильного владельца, `weak`-свойство его не держит — он умирает немедленно, и `obj.delegate` оказывается `nil`. Хорошая новость в том, что компилятор это видит:

```
warning: assigning retained object to weak property; object will be
    released after assignment [-Warc-unsafe-retained-assign]
11 |         obj.delegate = [[Watcher alloc] init];
    |                        ^ ~~~~~
```

«object will be released after assignment» — буквально «объект будет освобождён сразу после присваивания». Запусти такой код, и печать делегата даст `(null)`. Держи делегата в отдельной сильной переменной (или свойстве), как мы сделали с `quiet` и `chatty`.

- **Передал в conformsToProtocol: голое имя.** Нужен `@protocol(Name)`, а не `Name`: имя протокола само по себе не значение, его надо «опредметить» обёрткой `@protocol(...)`.
- **Забыл @class перед протоколом, который ссылается на твой класс.** Если протокол упоминает тип `Downloader` * раньше, чем объявлен сам `Downloader`, поставь выше `@class Downloader`; — предварительный анонс.

Упражнения

1. Добавь третий класс, принимающий `Sellable`, — например `GiftCard` (подарочная карта) с фиксированным номиналом. Положи его в ту же корзину и убедись, что `cartTotal` посчитал сумму без единой правки самой функции.
2. В `10-delegate.m` напиши второй делегат `SilentCounter : NSObject <DownloaderDelegate>`, который ничего не печатает, а считает, сколько загрузок завершилось успехом и сколько ошибкой, и выводит итог в конце. Подключи его вместо `ConsoleLogger`, не трогая класс `Downloader`.
3. Добавь в `DownloaderDelegate` опциональный метод `downloaderDidStart:` и зови его из `start` через `respondsToSelector:`. Реализуй его только в одном из делегатов и проверь, что второй продолжает работать без падения.
4. Проверь `[d1 conformsToProtocol:@protocol(DownloaderDelegate)]` для загрузчика и для делегата. Объясни, почему один отвечает «нет», а другой «да».
5. Намеренно убери у `ConsoleLogger` один из обязательных методов протокола и собери файл. Прочитай предупреждение компилятора — теперь ты знаешь, как выглядит «забыл выполнить контракт».
6. Сделай делегат-свойство `strong` вместо `weak` и устрой цикл удержания (делегат держит загрузчик, загрузчик — делегата). К этому примеру вернёмся в главе 13: запомни его, там мы увидим утечку «в цифрах».

Что мы получили

Протокол — это контракт: список методов без реализации, который класс обещает поддерживать. Он отвечает на вопрос «что объект умеет», тогда как наследование — на вопрос «кто он». Протокол выручает там, где наследование бессильно: когда умение нужно нескольким **не родственным** классам или когда умений несколько сразу. Класс принимает протокол записью `<Name>` после суперкласса и обязуется реализовать его методы; компилятор за этим следит. Тип `id<Protocol>` означает «любой объект, лишь бы умел эти сообщения», `conformsToProtocol:` и `@protocol(Name)` позволяют проверить принятие во время выполнения, а `@required/ @optional` делят методы на обязательные и необязательные (последние зовём только после `respondsToSelector:`).

На протоколах стоит важнейший паттерн Сосоа — делегирование: объект сообщает о событиях делегату по контракту, не зная его класса. Мы прошли путь от жёсткой связи к гибкой и поняли, почему свойство `delegate` делают `weak` (тизер цикла удержания — подробно в главе 13).

Дальше — глава 11 про **категории**: способ дописать методы к уже существующему классу, даже к чужому вроде `NSString`, не трогая его исходники и не наследуясь от него.

Документация Apple

- Programming with Objective-C → **Working with Protocols** — объявление протоколов, @required/@optional, id<Protocol>, делегирование — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/WorkingwithProtocols/WorkingwithProtocols.html
- NSObject (протокол) — conformsToProtocol:, respondsToSelector:, developer.apple.com/documentation/objectivec/nsobjectprotocol
- Cocoa Core Competencies → **Delegation** — паттерн делегирования и его роль в Cocoa — developer.apple.com/library/archive/documentation/General/Conceptual/DevPedia-CocoaCore/Delegation.html
- Protocol — представление протокола как объекта, developer.apple.com/documentation/objectivec/protocol

Глава 11. Категории и расширения класса

До сих пор, когда нам не хватало методов у класса, мы либо писали свой класс (глава 6), либо наследовали от чужого (глава 9). Но иногда хочется другого: добавить пару методов к **уже готовому** классу — да хоть к системному `NSString` — и при этом не наследовать, не оборачивать, не иметь его исходников. В чистом Си это невозможно в принципе: дописать функцию в чужой готовый тип нельзя, не тронув его код. Objective-C это умеет, и инструмент называется **категория**.

В этой главе мы добавим к `NSString` метод «развернуть строку задом наперёд» и метод «это палиндром?» — и они станут работать на любой строке в программе, включая литералы вроде `@"шалаш"`. Потом разберём **расширение класса** (class extension): с его помощью класс прячет приватное состояние. А в конце заглянем в **ассоциированные объекты** — трюк runtime, которым обходят главное ограничение категорий.

Что мы сделаем

- Объявим **категорию** на `NSString`: `@interface NSString (MyUtils)`.
- Добавим методы `reversedString` и `isPalindrome` — и вызовем их прямо на строковых литералах.
- Поймём, **зачем** нужны категории и какие у них **ограничения** (нельзя добавить хранилище, опасны конфликты имён).
- Освоим **расширение класса** `@interface Counter ()` — приватные свойства и методы, плюс приём «снаружи `readonly`, внутри `readwrite`».
- Кратко познакомимся с **ассоциированными объектами** — как категория всё-таки «хранит» данные через runtime.

Разбираем задачу руками

Возьмём строку @"кефир". Мы хотим уметь спросить у неё: «а как ты выглядишь задом наперёд?» — и получить @"рифек". И ещё: «ты палиндром?» (читаешься ли одинаково в обе стороны) — для @"шалаш" ответ «да», для @"кефир" — «нет».

Класс NSString написан в Apple. Его исходников у нас нет, наследовать от него ради двух методов — стрельба из пушки по воробьям (да и NSString устроен хитро, наследоваться от него не стоит). Хочется просто сказать: «NSString, вот тебе ещё два метода, пользуйся».

Ровно это и делает категория. Мы пишем отдельный кусочек — «довесок» к чужому классу — и компилятор приклеивает наши методы к NSString. После этого **любая** строка в программе умеет reversedString и isPalindrome, как будто они были там всегда.

Отличие от Си. В Си тип закрыт для дополнения. Есть struct point в чужой библиотеке — и всё, что ты можешь, это писать **отдельные** функции point_distance(p), которые принимают структуру параметром. Они не становятся «частью» типа: вызов всегда point_distance(p), а не p.distance. Категория же дописывает методы **внутри** чужого класса: после неё [str reversedString] — настоящий метод NSString, неотличимый от родных. Так в Си нельзя.

Шаг 1. Объявляем категорию

Создай файл 11-category.m. Начнём с объявления категории:

```
#import <Foundation/Foundation.h>

@interface NSString (MyUtils)
- (NSString *)naiveReversedString;    // первая попытка, с изъяном
- (BOOL)isPalindrome;                // читается одинаково в обе стороны?
@end
```

Имя naiveReversedString выбрано не случайно: первая версия разворота будет наивной. Мы найдём в ней брак и рядом напишем правильную — как всегда в этой книге, сначала честно ошибаемся, потом чиним.

Эта конструкция похожа на @interface из главы 6, но с одним новым элементом — **именем в круглых скобках**. Разберём по знакам.

@interface NSString (MyUtils)

- @interface — то же ключевое слово, что и при объявлении класса.

- NSString — имя класса, который мы **расширяем**. Не объявляем заново, а именно дополняем уже существующий.
- (MyUtils) — **имя категории** в круглых скобках. Это не новый класс и не подкласс; это «ярлык» нашей группы методов. Имя выбираем сами — обычно по смыслу довеска (MyUtils, JSON, Validation). Внутри одного класса имена категорий должны быть разными.

Дальше идут привычные объявления методов с минусом (метод экземпляра) и @end. Никакого : NSObject, никакого списка свойств с хранением — мы не создаём тип, мы дополняем чужой.

Отличие от Си. Круглые скобки (MyUtils) здесь — не вызов функции и не приведение типа, как ты привык в Си. Это синтаксис Objective-C: «категория с таким именем для такого класса». Знак @ перед interface как раз и предупреждает: дальше objc-конструкция, читай её по правилам Objective-C, а не Си.

Шаг 2. Реализуем методы категории

Теперь напишем тела методов. Реализация категории — это @implementation с тем же именем категории в скобках:

```
@implementation NSString (MyUtils)

- (NSString *)naiveReversedString {
    NSUInteger length = self.length;
    NSMutableString *result =
        [NSMutableString stringWithCapacity:length];
    // Идём с конца к началу и доклеиваем по одному unichar.
    for (NSUInteger i = length; i > 0; i--) {
        unichar c = [self characterAtIndex:i - 1];
        [result appendFormat:@"%C", c];
    }
    return [result copy];
}

- (BOOL)isPalindrome {
    return [self isEqualToString:[self naiveReversedString]];
}

@end
```

Разберём построчно.

`@implementation NSString (MyUtils)` — открываем реализацию **той же категории**. Имя в скобках обязано совпасть с тем, что в `@interface`.

Внутри `naiveReversedString` ключевое слово — `self`. Здесь `self` — это **та самая строка**, у которой вызвали метод. Если написать `[@"кефир" naiveReversedString]`, то внутри метода `self` будет равно `@"кефир"`. Категория работает «изнутри» класса, поэтому ей доступны все публичные методы `NSString` через `self`.

- `self.length` — длина строки в `unichar`-ах (про них чуть ниже; для кириллицы и латиницы это просто число букв). Тип `NSUInteger` — «беззнаковое целое» (количество не бывает отрицательным).
- `[NSMutableString stringWithCapacity:length]` — создаём **изменяемую** строку (в неё можно дописывать). `stringWithCapacity:` — подсказка, сколько примерно символов планируем, чтобы не перевыделять память зря.
- `for (NSUInteger i = length; i > 0; i--)` — идём от конца к началу. Берём индекс `i - 1`, потому что у строки длины 5 символы стоят на позициях 0...4, а не 1...5.
- `[self characterAtIndex:i - 1]` — один элемент строки по индексу. Возвращается `unichar` — 16-битное целое, один «кусочек» текста в кодировке UTF-16. Для обычных букв кусочек равен символу; в шаге 4 увидим, где это не так.
- `[result appendFormat:@"%C", c]` — дописываем символ в конец `result`. Спецификатор `%C` (заглавная) печатает один `unichar`. Не путай с `%c` (строчная) — та для обычного си-символа `char`.
- `return [result copy]` — возвращаем **неизменяемую** копию. Мы строили результат в `NSMutableString`, а наружу честнее отдать обычный `NSString`, который снаружи уже не переделают (подробно про `copy` и изменяемость — в главе 15).

Метод `isPalindrome` короткий: строка — палиндром, если она равна себе же, развёрнутой. Сравниваем через `isEqualToString:` — правильный способ сравнивать строки по содержимому (а не `==`, которое сравнило бы адреса; см. главу 5). Внутри одной категории методы свободно зовут друг друга: `isPalindrome` пользуется нашим же разворотом через `self`.

Шаг 3. Пользуемся — на обычных строках

Самое приятное: вызывать наши методы можно на **любой** строке, даже на литерале `@"..."`. Добавим `main`:

```
int main(void) {
    @autoreleasepool {
        NSString *word = @"кефир";
        NSLog(@"%@ наоборот: %@", word, [word naiveReversedString]);
    }
}
```

```

NSString *a = @"шалаш";
NSString *b = @"кефир";
NSLog(@"%@ – палиндром? %@", a,
      a.isPalindrome ? @"да" : @"нет");
NSLog(@"%@ – палиндром? %@", b,
      b.isPalindrome ? @"да" : @"нет");

// Метод работает и на строковом литерале, и на результате
// других методов Foundation – это всё тот же NSString.
NSString *upper = [@"abcba" uppercaseString];
NSLog(@"%@ – палиндром? %@", upper,
      upper.isPalindrome ? @"да" : @"нет");
}
return 0;
}

```

Здесь `a.isPalindrome` — это точечная запись вызова метода без аргументов, то же самое, что `[a isPalindrome]` (мы разбирали эту форму в главе 8). Конструкция условие ? «да» : «нет» — тернарный оператор из Си: если `isPalindrome` вернул YES, берём первую строку, иначе вторую.

Обрати внимание на последнюю пару строк. `[@"abcba" uppercaseString]` возвращает новый `NSString @"ABCBA"`, и на этом результате тут же работает наш `isPalindrome`. Категория приклеилась к классу `NSString` целиком — значит, к **каждому** объекту этого класса, откуда бы он ни взялся: из литерала, из метода Foundation, из файла.

Скомпилируем и запустим:

```

clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    11-category.m -o 11-category && ./11-category

```

Реальный вывод (дата, время и числа в скобках будут свои):

```

2026-09-27 15:46:34.849 11-category[28287:210204] кефир наоборот: рифек
2026-09-27 15:46:34.849 11-category[28287:210204] шалаш – палиндром? да
2026-09-27 15:46:34.849 11-category[28287:210204] кефир – палиндром? нет
2026-09-27 15:46:34.849 11-category[28287:210204] ABCBA – палиндром? да

```

Перед каждой строкой `NSLog` ставит дату, время, имя программы и в квадратных скобках номер процесса и потока. Дальше в главе этот префикс опускаем и показываем только сам текст.

Если собрать готовый `code/11-category.m`, строк будет больше: там уже есть продолжение из шагов 4 и 5.

кефир развернулся в рифек, шалаш опознан как палиндром, кефир — нет, а АВСВА — да. Два метода, которых у `NSString` отродясь не было, теперь работают на любой строке программы.

Шаг 4. А теперь ломаем: эмодзи

Радоваться рано. Наш разворот идёт по `unichar` — по 16-битным «кусочкам» Unicode. Для кириллицы и латиницы один символ — это ровно один `unichar`, и всё сходится. Но эмодзи в UTF-16 хранится **парой** таких кусочков (это называется суррогатной парой), и наш цикл разворачивает пару задом наперёд, превращая её в бессмыслицу.

Проверим на строке `@"привет 🙌 мир"` — допиши в конец `main`:

```
NSString *withEmoji = @"привет 🙌 мир";
NSString *broken = [withEmoji naiveReversedString];
NSData *utf8 = [broken dataUsingEncoding:NSUTF8StringEncoding];
NSLog(@"наивный разворот переводится в UTF-8: %@",
      utf8 ? @"да" : @"нет — строка битая");
NSLog(@"наивно:   %@", broken);
```

`dataUsingEncoding`: пытается перевести строку в байты UTF-8 и возвращает `nil`, если это невозможно. Вот что печатается:

```
наивный разворот переводится в UTF-8: нет — строка битая
```

И это **всё**: строки «наивно: ...» в выводе просто нет. `NSLog` получил строку с разорванной суррогатной парой, не смог перевести её в байты — и, по нашим экспериментам на macOS, молча пропустил всю запись целиком. Ни ошибки, ни падения, ни пустой строки. Такие «исчезающие» логи в реальной жизни отлаживают часами, поэтому запомни симптом: **пропала целая строка лога — ищи битый юникод**.

Шаг 5. Чиним: разворот по составным символам

Правильный инструмент у Foundation есть — метод `enumerateSubstringsInRange:options:usingBlock`: с опцией `NSStringEnumerationByComposedCharacterSequences`. Он перебирает строку не по `unichar`, а по **составным символам**: эмодзи с модификатором цвета кожи, буква с диакритикой, флаг из двух региональных индикаторов — каждый приходит одним куском:

```
- (NSString *)reversedString {
    NSMutableString *result =
        [NSMutableString stringWithCapacity:self.length];
    [self enumerateSubstringsInRange:NSMakeRange(0, self.length)
```

```

options:NSUTFStringEnumerationByComposedCharacterSequences
usingBlock:^(NSString *piece, NSRange substringRange,
              NSRange enclosingRange, BOOL *stop) {
    (void)substringRange; (void)enclosingRange; (void)stop;
    [result insertString:piece atIndex:0];    // каждый кусок — в начало
}];
return [result copy];
}

```

Приём внутри простой: вместо того чтобы идти с конца, мы идём с начала, но каждый очередной кусок **вставляем в позицию 0** — то есть перед всем, что уже накопилось. Получается разворот.

Про `^(...) { ... }` в аргументе не переживай: это **блок** — кусок кода, который передают как значение. Ему посвящена следующая глава, а здесь достаточно понимать, что Foundation вызывает наш блок по разу на каждый составной символ. Строчки `(void)substringRange;` нужны, чтобы компилятор с `-Wextra` не ругался на неиспользованные параметры: блок обязан принять все четыре, хотя нам нужен только первый.

Добавь объявление — `(NSString *)reversedString;` в `@interface NSString (MyUtils)`, а в `isPalindrome` и в первой строке `main` замени `naiveReversedString` на `reversedString`. В конец `main` допиши:

```
NSLog(@"аккуратно: %@", [withEmoji reversedString]);
```

Теперь `isPalindrome` зовёт уже правильную версию, и разворот работает на чём угодно:

```
аккуратно: рим 🤚 тевирп
```

Эмодзи уцелел и встал на своё место. Это тот случай, когда наивный код работает на всех примерах из учебника и разваливается на первом же реальном сообщении пользователя.

Зачем нужны категории

Два главных применения.

1. Расширить чужой класс. Системный `NSString`, `NSArray`, `NSDate` — их исходников у тебя нет, а пары удобных методов не хватает. Категория добавляет их аккуратно, не плодя обёрток и подклассов. Так делают сами Apple: методы рисования строки (`drawAtPoint:...`) живут не в самом `NSString`, а в категории `NSStringDrawing` из `AppKit` (на iOS — из `UIKit`). Базовый класс остаётся компактным, а «довески» подключаются по необходимости.

2. Разнести большой класс по файлам. Если у твоего собственного класса разрослись методы, их группируют по категориям в разные файлы: `Person+Networking.m`, `Person+Validation.m`. Принятое имя файла — `Класс+Категория.m`. Класс читается легче, каждый файл — про одну тему.

Ограничения категорий — важно

Категория мощная, но не всесильна. Три правила, которые надо помнить.

Категория не добавляет хранилище. Через категорию нельзя завести новый `ivar` (поле-переменную экземпляра) или обычное **хранимое** `@property`. Можно объявить методы — и всё. Причина в том, как устроена память объекта: её размер фиксируется, когда класс компилируется, а категория приклеивается позже и «раздвинуть» уже созданный объект не может. Если в категории написать `@property (nonatomic) NSInteger n;`, компилятор **не** создаст для него хранилище и геттер/сеттер автоматически — он лишь объявит методы `n/setN:`, а писать их тело придётся самому. Куда писать значение — отдельный вопрос; ответ (ассоциированные объекты) — в конце главы.

Конфликт имён непредсказуем. Если две категории (или категория и сам класс) объявляют метод с **одинаковым** именем, в рантайме победит какой-то один — и какой именно, не гарантируется (зависит от порядка загрузки). Никакого `super`, никакого «обе версии» — просто одна молча затирает другую. Поэтому методам категорий на чужих классах принято давать **префикс**, чтобы случайно не столкнуться: не `reversedString`, а `xyz_reversedString`. В учебных примерах мы префикс опускаем для читаемости, но в реальном коде на системных классах он спасает от тихих багов.

Не переопределяй существующие методы через категорию. Технически категория может объявить метод, который у класса уже есть, и «перебить» его. Но это плохая идея: ты ломаешь поведение класса для **всей** программы, включая системный код, который на этот метод рассчитывает, и сделать `[super ...]` оттуда нельзя. Хочешь изменить поведение — наследуй (глава 9) или используй честные приёмы `runtime` (глава 14). Категория — для **добавления** методов, не для подмены.

Отличие от Си. В Си «конфликт имён» ловится на этапе компоновки: две функции `reversedString` дадут ошибку `duplicate symbol`, и программа просто не соберётся — ты сразу узнаешь о проблеме. У категорий же конфликт методов компилятор обычно пропускает молча, а «побеждает» один из методов уже во время выполнения. Поэтому дисциплина с префиксами тут на тебе, а не на компиляторе.

Расширение класса: приватная часть интерфейса

Теперь — родственная конструкция, которая выглядит как категория **без имени**, но решает противоположную задачу. Если категория добавляет методы чужому классу снаружи, то **расширение класса** (class extension) описывает **приватную** часть **своего** класса — то, что не должно торчать наружу.

Построим простой счётчик. Снаружи у него должно быть видно текущее значение, но **менять** его извне нельзя — только через методы `increment` и `reset`. Внутри же счётчику нужно своё приватное состояние: шаг и метод-помощник для лога.

Создай файл `11-extension.m`. Сначала **публичный** интерфейс:

```
#import <Foundation/Foundation.h>

@interface Counter : NSObject
@property (nonatomic, readonly) NSInteger value;
- (void)increment;
- (void)reset;
@end
```

Здесь `value` объявлен с атрибутом **readonly** — «только чтение». Снаружи можно прочитать `c.value`, но нельзя написать `c.value = 5` — компилятор не даст. Сеттера у такого свойства нет. Из публичного интерфейса виден лишь геттер и два метода-команды. Это и есть контракт класса для внешнего мира.

Теперь — **расширение класса** прямо в этом же `.m`-файле:

```
@interface Counter ()
@property (nonatomic, readwrite) NSInteger value;
@property (nonatomic) NSInteger step;
- (void)logChange;
@end
```

@interface Counter ()

Это и есть расширение класса. Внешне — категория, но скобки **пустые**: `()`, без имени. Такую анонимную категорию компилятор трактует особо.

Что мы тут делаем:

- `@property (nonatomic, readwrite) NSInteger value;` — **переобъявляем** то же свойство `value`, но уже как `readwrite` (чтение и запись). Это классический приём: **снаружи**

readonly, внутри readonly. Внешний код видит только геттер, а сам класс получает ещё и сеттер и спокойно меняет значение через `self.value = ...`.

- `@property (nonatomic) NSInteger step;` — приватное свойство «шаг». Снаружи о нём вообще никто не знает: его нет в публичном `@interface`.
- `– (void)logChange;` — приватный метод-помощник, тоже невидимый снаружи.

Главное отличие от обычной категории — вот в чём. Расширение класса компилятор присоединяет к классу **в момент компиляции самого класса** (оно лежит рядом с его `@implementation`), а не «приклеивает позже». А раз так, расширение **может** добавлять `ivar` и настоящие хранимые свойства — в отличие от категории. Поэтому `step` — полноценное хранимое свойство со своим хранилищем и сеттером/геттером, всё как обычно.

Отличие от Си. В Си приватность имитируют через `static`: функцию или глобальную переменную помечают `static` в `.c`-файле, и за пределы этого файла она не видна. Расширение класса тоже прячет «по файлам»: его видит только код, который стоит ниже него в том же `.m`. Но прячет оно не отдельную функцию, а **часть интерфейса класса**: публичную половину все видят через `.h`, приватную — только `Counter.m`. И это проверка компилятора, не runtime: сообщение `logChange` чужой код всё равно может послать в обход, runtime его не остановит.

Реализация

Дальше — обычный `@implementation`, который пользуется и публичными, и приватными членами:

```
@implementation Counter

– (instancetype)init {
    self = [super init];
    if (self) {
        _value = 0;
        _step = 1;    // приватное состояние, наружу не торчит
    }
    return self;
}

– (void)increment {
    self.value += self.step;    // пишем в «readonly» свойство изнутри
    [self logChange];
}
```



```
- (void)reset {
    self.value = 0;
    [self logChange];
}

- (void)logChange {           // приватный метод
    NSLog(@" значение теперь: %ld", (long)self.value);
}

@end
```

В `init` пишем напрямую в `ivar`-ы `_value` и `_step` (подчёркивание — автоматическое хранилище свойства, глава 8). Строка `self.value += self.step` работает только потому, что расширение класса дало `readwrite`-версию `value`: будь она `readonly`, компилятор бы не пропустил. `logChange` — приватный, зовём его из любого метода класса.

`%ld` в `NSLog` — формат для `long`, поэтому `value` приводим к `long` через `(long)`. Так советует Apple: у `NSInteger` на разных платформах разный размер (на 32-битных это `int`), а с приведением формат верен везде.

Пробуем

```
int main(void) {
    @autoreleasepool {
        Counter *c = [[Counter alloc] init];
        NSLog(@"старт, value = %ld", (long)c.value);
        [c increment];
        [c increment];
        [c increment];
        NSLog(@"читаем снаружи: value = %ld", (long)c.value);
        [c reset];
        // В файле, который видит только публичный @interface, строка
        // ниже не скомпилируется: error: assignment to readonly property.
        // Здесь main стоит ниже расширения и видит readwrite-версию.
        // c.value = 100;
    }
    return 0;
}
```

main только **читает** `c.value` и шлёт команды. Строка `c.value = 100` закомментирована, и тут есть тонкость. Компилятор читает файл сверху вниз, и main стоит **ниже** расширения — значит, видит и `readwrite`-версию `value`, и `step`, и `logChange`. Раскомментируй строку — она соберётся. В настоящем проекте публичный `@interface` лежит в `Counter.h`, расширение и `@implementation` — в `Counter.m`, а чужой код подключает только `Counter.h`. Вот там запись в `value` уже не пройдёт.

Проверить это можно и в одном файле: перенеси main сразу под публичный `@interface Counter`, выше расширения, и добавь в него `c.value = 100;`, `c.step = 2;` и `[c logChange];`. Компилятор ответит:

```
error: assignment to readonly property
error: property 'step' not found on object of type 'Counter *'
error: no visible @interface for 'Counter' declares the selector
      'logChange'
```

Верни main на место. Собираем и запускаем:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/11-extension.m -o 11-extension && ./11-extension
```

Реальный вывод (префикс `NSLog` с датой опущен):

```
старт, value = 0
    значение теперь: 1
    значение теперь: 2
    значение теперь: 3
читаем снаружи: value = 3
    значение теперь: 0
```

Счётчик дошёл до 3 шагами по 1, снаружи его значение читается, а вот поменять напрямую код, которому виден только публичный интерфейс, не может — он не знает ни про `setValue:`, ни про `step`. Класс прячет своё устройство и открывает наружу ровно тот минимум, что нужен для работы.

Ассоциированные объекты: «хранилище» для категории

Вернёмся к ограничению категорий: они не добавляют `ivar`. Но иногда очень хочется прицепить к чужому объекту ещё одно значение. Например, пометить произвольную строку ярлыком (`tag`), не наследуя `NSString`.

Выход есть — **ассоциированные объекты** (`associated objects`). Это функции runtime, которые умеют «привязать» к любому объекту дополнительный объект по ключу и потом достать его

обратно. Внутри runtime держит общую скрытую таблицу «объект → его довески», а когда объект умирает, стирает его строчку вместе с довесками.

Это уже территория runtime (ему посвящена глава 14). Здесь — только рабочий рецепт, чтобы снять ограничение категории.

Создай 11-associated.m. Новое — подключаем заголовок runtime:

```
#import <Foundation/Foundation.h>
#import <objc/runtime.h> // objc_setAssociatedObject и компания
```

<objc/runtime.h> — заголовок самого Objective-C-runtime, та низкоуровневая кухня, на которой стоит весь язык. Из него нам нужны две функции.

Объявляем категорию с «мнимым» свойством и реализуем геттер/сеттер вручную:

```
@interface NSString (Tag)
@property (nonatomic, copy) NSString *tag; // мнимое «хранилище»
@end

@implementation NSString (Tag)

// Уникальный ключ. Адрес самой статической переменной никогда не
// повторится — этого достаточно, чтобы пометить наше значение.
static const void *kTagKey = &kTagKey;

- (void)setTag:(NSString *)tag {
    objc_setAssociatedObject(self, kTagKey, tag,
                            OBJC_ASSOCIATION_COPY_NONATOMIC);
}

- (NSString *)tag {
    return objc_getAssociatedObject(self, kTagKey);
}

@end
```

Разберём по частям.

@property ... NSString *tag; в категории, как мы выяснили, **не** создаёт хранилище само — только обещает методы tag и setTag:. Их тела мы пишем руками ниже, и внутри них прячется вся хитрость.

`static const void *kTagKey = &kTagKey;` — это **ключ**, под которым runtime запомнит наше значение. Трюк: переменной присваивают её собственный адрес `&kTagKey`. Адрес уникален и стабилен на всё время работы программы — идеальный «несгораемый» ключ, который точно не совпадёт ни с чьим. Значение самого `kTagKey` неважно, важен его адрес.

`objc_setAssociatedObject(self, kTagKey, tag, ...)` — привязывает к `self` значение `tag` под ключом `kTagKey`. Последний аргумент — **политика хранения**: `OBJC_ASSOCIATION_COPY_NONATOMIC` означает «храни **копию**, без потокобезопасности» — как `(nonatomic, copy)` у обычного свойства. Всего политик пять: `OBJC_ASSOCIATION_ASSIGN` (не удерживать, как `assign`), `OBJC_ASSOCIATION_RETAIN_NONATOMIC` и `OBJC_ASSOCIATION_RETAIN` (удерживать, как `strong`), `OBJC_ASSOCIATION_COPY_NONATOMIC` и `OBJC_ASSOCIATION_COPY` (хранить копию). Варианты без `_NONATOMIC` — потокобезопасные, как `atomic`.

`objc_getAssociatedObject(self, kTagKey)` — достаёт обратно то, что привязали под этим ключом. Если ничего не привязывали, вернётся `nil`.

Пробуем:

```
int main(void) {
    @autoreleasepool {
        NSMutableString *note =
            [NSMutableString stringWithString:@"молоко"];
        NSLog(@"до:    note = \"%@\"", tag = @"", note, note.tag);

        note.tag = @"покупки";    // вызывает наш setTag:
        NSLog(@"после: note = \"%@\"", tag = @"", note, note.tag);
    }
    return 0;
}
```

Почему `NSMutableString`, а не литерал `@"молоко"`? Одинаковые строковые литералы в программе — это **один и тот же объект**. Повесь ярлык на `@"молоко"`, и он появится у каждого `@"молоко"` во всём коде. Свежесозданная изменяемая строка — отдельный объект, ярлык достанется только ей.

`note.tag = @"покупки"` — точечная запись, которая под капотом зовёт наш `setTag:`. А `note.tag` в `NSLog` зовёт геттер `tag`. Внутри строки формата `\"` — это экранированная кавычка (чтобы напечатать сам символ `"`), приём из Си. Собираем и запускаем:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/11-associated.m -o 11-associated && ./11-associated
```

Реальный вывод (префикс `NSLog` опущен):

```
до:    note = "молоко", tag = (null)
после: note = "молоко", tag = покупки
```

До установки `tag` геттер вернул `nil` (печатается как `(null)`) — мы ещё ничего не привязывали. После присваивания у строки появился ярлык покупки, хотя у класса `NSString` никакого поля `tag` нет и в помине. Категория «обзавелась хранилищем» — руками `runtime`.

Пользуйся этим приёмом **с осторожностью** и только когда наследование не подходит: чаще честнее завести свой класс.

Проверяем

Три готовых файла главы — `code/11-category.m` (категория с `reversedString/isPalindrome`), `code/11-extension.m` (расширение класса `Counter` () с приватным состоянием) и `code/11-associated.m` (категория с ассоциированным объектом вместо хранилища). Каждый собирается и запускается одной строкой:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/11-category.m -o /tmp/t && /tmp/t
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/11-extension.m -o /tmp/t && /tmp/t
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/11-associated.m -o /tmp/t && /tmp/t
```

Все три собираются **без единого предупреждения** даже с `-Wall -Wextra`. Вывод — тот, что показан выше по главе (без префикса `NSLog` с датой и номерами). Для `11-category.m` он целиком выглядит так:

```
кефир наоборот: рифек
шалаш — палиндром? да
кефир — палиндром? нет
АВСВА — палиндром? да
наивный разворот переводится в UTF-8: нет — строка битая
аккуратно: рим 🙌 тевирп
```

Шесть строк вывода на семь `NSLog` в программе — и это не опечатка: седьмую, с наивно развёрнутой строкой, `NSLog` проглотил, как мы и разобрали в шаге 4.

Частые ошибки

- **Имя категории не совпадает в @interface и @implementation.** @interface NSString (MyUtils), а реализация — @implementation NSString (Utils). Компилятор промолчит, и программа даже заработает: runtime ищет метод по имени, а не по категории. Но теперь это две разные категории, и объявление в одной не сверяется с реализацией в другой — забытый метод никто не заметит до падения. Имена в скобках держи буква в букву одинаковыми.
- **Ждёшь, что @property в категории сам заведёт хранилище.** Не заведёт: получишь предупреждение property 'n' requires method 'n' to be defined, а в рантайме — падение с unrecognized selector. В категории @property — только обещание методов; хранилище делай через ассоциированные объекты.
- **Одинаковые имена методов в разных категориях.** Молча победит один — и не угадаешь, какой. На системных классах всегда давай методам префикс (abc_...).
- **Переопределяешь готовый метод класса через категорию.** Ломается поведение для всей программы, super недоступен. Для замены поведения — наследование или runtime (глава 14), не категория.
- **Кладёшь расширение класса () в .h-файл.** Тогда его «приватные» члены видны всем — смысл теряется. Расширение класса живёт в .m.
- **Забыл #import <objc/runtime.h> для ассоциированных объектов.** Компилятор не найдёт objc_setAssociatedObject: Foundation.h этот заголовок не подключает, его нужно импортировать явно.

Упражнения

1. Добавь в категорию NSString (MyUtils) метод — (NSUInteger)countVowels — сколько в строке гласных. Проверь на нескольких словах.
2. Сделай категорию на NSArray с методом — (id)secondObject, возвращающим второй элемент (или nil, если его нет). Подумай, что вернуть для пустого массива.
3. В Counter добавь в расширение класса приватное свойство NSDate *lastChange и обновляй его в logChange. Снаружи время последнего изменения видно быть не должно.
4. Перенеси main выше расширения (как в опыте из раздела «Пробуем») и добавь c.value = 100; — сборка упадёт. Теперь сделай публичное свойство value readonly и убедись, что строка компилируется. Верни readonly и почувствуй разницу в контракте.
5. Дай методам категории MyUtils префикс (sm_reversedString) и обнови вызовы. Так выглядит безопасный код для системных классов.

Что мы получили

Теперь ты умеешь дополнять **любой** класс, даже системный, не имея его исходников: **категория** приклеивает к классу новые методы, работающие на всех его объектах. Знаешь её границы — нет своего хранилища, опасны конфликты имён, не для подмены методов. Освоил **расширение класса** — приватную часть интерфейса в .m, с приёмом «снаружи readonly, внутри readwrite» и возможностью добавлять настоящие ivar-ы. И одним глазом увидел **ассоциированные объекты** — как категория обходит запрет на хранилище через runtime.

В следующей главе возьмём ещё один краеугольный инструмент языка — **блоки** (blocks): кусочки кода, которые можно положить в переменную, передать в метод и выполнить позже. А устройство самого runtime — isa, поиск метода, подмену реализаций — разберём в главе 14.

Документация Apple

- Programming with Objective-C → **Customizing Existing Classes** (Categories) — [developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ ProgrammingWithObjectiveC/ CustomizingExistingClasses/ CustomizingExistingClasses.html](http://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/CustomizingExistingClasses/CustomizingExistingClasses.html)
- Class extensions описаны там же, в разделе «Customizing Existing Classes → Class Extensions Extend the Internal Implementation».
- Objective-C Runtime — objc_setAssociatedObject, objc_getAssociatedObject — [developer.apple.com/documentation/objectivec/objc_setassociatedobject\(::::\)](http://developer.apple.com/documentation/objectivec/objc_setassociatedobject(::::))
- NSString — developer.apple.com/documentation/foundation/NSString
- NSMutableString — developer.apple.com/documentation/foundation/NSMutableString

Глава 12. Блоки

До сих пор мы передавали друг другу *данные*: числа, строки, объекты. В этой главе научимся передавать *действие* — кусок кода, упакованный в переменную. В Objective-C такая упаковка называется **блок** (block). Блок можно положить в переменную, отдать в метод, сохранить на потом и вызвать когда угодно. А ещё блок умеет то, чего обычная функция не умеет: он запоминает переменные из того места, где был написан. Это делает его **замыканием** (closure) — и именно ради этого блоки и придуманы.

Мы построим всё по нарастающей: простейший блок в переменной → блок, который захватывает окружение → изменяемый захват через `__block` → блок как «обработчик завершения» у собственного метода → и, наконец, самая частая ловушка блоков — цикл удержания (retain cycle) и его починка. Каждый шаг компилируем и смотрим на живой вывод.

Что мы сделаем

- Разберём, что такое блок и зачем он нужен.
- Научимся читать и писать тип блока `void (^name)(int)` и литерал `^(int x){ ... }`.
- Дадим типам блоков короткие имена через `typedef`.
- Поймём, как блок захватывает переменные окружения и почему по умолчанию их нельзя менять внутри.
- Введём `__block` для изменяемого захвата.
- Передадим блок в метод — паттерн completion handler.
- Встретим retain cycle вживую (через `dealloc`) и починим его слабой ссылкой `weakSelf`.

Сначала — указатель на функцию в Си

Чтобы оценить блок, посмотрим, что было до него. В Си действие тоже можно положить в переменную — через **указатель на функцию**. Звучит страшно, на деле просто адрес: переменная хранит адрес кода функции, и через эту переменную функцию можно вызвать.

Объявляется так:


```
int (*f)(int);
```

Читается изнутри наружу: `f` — это указатель (*) на функцию, которая принимает `int` и возвращает `int`. Скобки вокруг `*f` обязательны: без них `int *f(int)` означало бы совсем другое — функцию, возвращающую `int *`.

Присваиваем и зовём:

```
int doubleC(int x) { return x * 2; }

int (*f)(int) = doubleC; // f показывает на код doubleC
int r = f(21);           // вызов через указатель: r == 42
```

Указатель на функцию умеет одно: показать на код. Он **не помнит** ничего о месте, где его создали. Запомни это — через минуту блок сделает ровно то, чего здесь не хватает.

Что такое блок

Блок — это **объект-замыкание**: кусок кода вместе с переменными, которые он «утащил» из окружения. В отличие от функции, блок можно написать прямо посреди другого кода, без отдельного имени, и тут же положить в переменную.

Сравним два синтаксиса блок о блок:

```
// Си: указатель на функцию
int (*f)(int) = doubleC;

// Objective-C: переменная-блок
int (^doubleB)(int) = ^(int x) {
    return x * 2;
};
```

Разница в одном символе: вместо звёздочки * у блока стоит **домик** ^ (caret). Где у указателя на функцию (*f), у блока (^doubleB). Всё остальное в типе читается одинаково: `int (^doubleB)(int)` — это блок, принимающий `int` и возвращающий `int`.

Справа от = стоит **литерал блока**:

```
^(int x) {
    return x * 2;
}
```

Он начинается с того же домика `^`, дальше — список параметров (`int x`) как у функции, затем тело в фигурных скобках. Имени у этого кода нет: блок безымянный, он живёт в переменной `doubleB`.

Вызывают блок так же, как функцию — имя и круглые скобки с аргументами:

```
doubleB(21); // вернёт 42
```

Отличие от Си. Указатель на функцию `int (*f)(int)` умеет только показать на готовый код — он ничего не знает об окружении. Блок `int (^b)(int)` тоже хранит код, но вдобавок **захватывает** переменные того места, где написан. Это и делает его замыканием: код плюс прихваченное окружение. У си-функции окружения нет вовсе.

Блоки разной формы

Не у каждого блока есть аргументы и возвращаемое значение. Самый частый блок — «сделай что-нибудь», без входа и выхода:

```
void (^hello)(void) = ^{
    NSLog(@"привет из блока");
};
hello();
```

Тип `void (^hello)(void)` читается так: блок, который ничего не принимает (`void` в скобках) и ничего не возвращает (`void` слева). Обрати внимание: если у блока нет параметров, список (`void`) в литерале можно опустить — пишем просто `^{ ... }`. Этот короткий вид `^{ }` ты уже мельком видел в главе 11; теперь понятно, что это блок без аргументов.

Собираем первую программу

Сложим всё в один файл `code/12-basic.m` — функция `doubleC`, указатель `f` на неё, блок `doubleB`, блок `hello` без аргументов и несколько вызовов `doubleB`. Полный листинг — в репозитории книги. Компилируем и запускаем:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/12-basic.m -o /tmp/t && /tmp/t
```

Вывод (дата, время и числа в квадратных скобках будут свои):

```
2026-06-28 12:27:46.636 t[61960:1255120] функция через указатель: 42
2026-06-28 12:27:46.636 t[61960:1255120] блок: 42
2026-06-28 12:27:46.636 t[61960:1255120] привет из блока
2026-06-28 12:27:46.636 t[61960:1255120] 2, 10, 20
```

Функция и блок дали одинаковый 42. Пока блок выглядит как более многословная функция. Его сила раскрывается на следующем шаге.

Захват окружения

Вот что блок умеет, а функция — нет. Напиши блок рядом с локальной переменной, упомяни её внутри — и блок **захватит** (capture) её значение.

```
int factor = 3;

int (^multiply)(int) = ^(int x) {
    return x * factor;    // factor взят из окружения
};

NSLog(@"3 * 5 = %d", multiply(5));    // 15
```

Переменной `factor` нет среди параметров блока. Блок «дотянулся» до неё наружу, в тело `main`. В момент создания блока он снял **копию** значения `factor` — то есть число 3 — и положил внутрь себя.

Слово «копию» здесь ключевое. Поменяем `factor` после создания блока:

```
factor = 100;
NSLog(@"блок помнит 3 * 5 = %d", multiply(5));    // всё ещё 15
```

Блок по-прежнему считает с тройкой. Он захватил *значение* 3, а не саму переменную `factor`. Дальнейшие изменения `factor` его не трогают.

Отсюда и правило: **захваченные значения внутри блока менять нельзя**. Если написать в теле `factor = 7;` или `factor++`, компилятор откажется:

```
error: variable is not assignable (missing __block type specifier)
```

Для блока это чужая константа, копия только для чтения. Так сделано специально — чтобы не было путаницы, чьё значение меняется, внешнее или внутреннее. Через минуту мы снимем это ограничение через `__block`, но сперва запомним поведение по умолчанию: **захват — это копия только для чтения**.

Отличие от Си. Указатель на функцию не способен «увидеть» локальную `factor` — функция знает только свои параметры. Чтобы передать ей `factor`, в Си его пришлось бы явно отдать аргументом: `multiply(5, factor)`. Блок прихватывает `factor` сам, молча, в момент написания. Это и есть замыкание.

typedef: короткое имя длинному типу

Тип блока громоздкий. Когда он встречается часто (в свойстве, в параметре метода), его удобно спрятать за коротким именем через typedef:

```
typedef void (^Greeter)(NSString *);
```

Читается так: Greeter — это имя для типа «блок, принимающий NSString * и ничего не возвращающий». Имя блока в typedef (Greeter) встаёт ровно туда, где в объявлении переменной стояло бы (^имя). Теперь вместо длинного типа пишем коротко:

```
NSString *prefix = @"Привет";
Greeter greet = ^(NSString *name) {
    NSLog(@"%@, %@!", prefix, name);    // prefix захвачен
};
greet(@"Мир");
```

Блок заодно захватил и prefix — объект захватывается так же, как число. Без typedef то же объявление выглядело бы как `void (^greet)(NSString *) = ^(NSString *name){ ... };` — глазам тяжелее. typedef особенно выручает, когда тип блока — это тип параметра метода: в сигнатуре метода длинная конструкция в скобках читается совсем плохо. Скоро увидим это на completion handler.

Программа с захватом

Все эти куски собраны в файл `code/12-capture.m` (полный листинг — в репозитории книги). Запуск:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/12-capture.m -o /tmp/t && /tmp/t
```

Вывод:

```
2026-06-28 ... 3 * 5 = 15
2026-06-28 ... factor теперь 100, но блок помнит 3 * 5 = 15
2026-06-28 ... Привет, Мир!
2026-06-28 ... Привет, Серик!
```

factor стал сотней, а блок всё считает с тройкой — захват зафиксировал значение в момент создания. Один и тот же greet с захваченным prefix сработал дважды, под разные имена.

Изменяемый захват: `__block`

Иногда блок должен не просто читать внешнюю переменную, а **менять** её — например, считать, сколько раз его вызвали. Обычный захват это запрещает. Решение — пометить переменную ключевым словом `__block` (два подчёркивания в начале):

```
__block int count = 0;

void (^tick)(void) = ^{
    count++;    // теперь можно: count — общая ячейка
};

tick();
tick();
tick();
NSLog(@"%d", count);    // 3
```

`__block` меняет саму природу захвата. Без него блок брал копию значения. С ним переменная `count` превращается в **общую ячейку**: и внешний код, и блок смотрят на одну и ту же `count`. Блок инкрементирует её — изменение видно снаружи. Внешний код меняет её — изменение видно блоку.

Отличие от Си. Чтобы функция в Си смогла изменить чужую переменную, ей передают **указатель** на эту переменную: `void tick(int *count) { (*count)++; }`, а зовут `tick(&count)`. Функция пишет по адресу — и снаружи видно. `__block` делает то же самое, но скрытно: компилятор сам заворачивает `count` в общую ячейку и сам организует доступ по ссылке. Тебе не нужно ни `&`, ни `*` — пишешь `count++`, как с обычной переменной.

Программа с `__block`

Оба блока собраны в файл `code/12-block-var.m`: первый (`tick`) считает вызовы, второй (`add`) накапливает сумму. Запуск и вывод:

```
2026-06-28 ... блок сработал 3 раза
2026-06-28 ... сумма = 100
```

Второй блок `add` накапливает сумму в `__block`-переменной `sum`, вызываясь по разу на каждый элемент массива. Это типичный приём: блок — накопитель, `__block`-переменная — копилка.

Блок как аргумент метода: completion handler

Главное применение блоков — отдать кусок кода *другому коду*, чтобы тот вызвал его в нужный момент. Самый частый случай: метод делает работу, а в конце зовёт переданный ему блок с результатом. Такой блок называют **completion handler** — обработчик завершения.

Заведём класс `Downloader` с методом, который «качает» имя и по завершении зовёт блок. Сначала через `typedef` опишем тип блока — иначе сигнатура метода будет нечитаемой:

```
typedef void (^DownloadHandler)(NSString *result);

@interface Downloader : NSObject
- (void)downloadName:(NSString *)name
      completion:(DownloadHandler)completion;
@end
```

Метод `downloadName:completion:` берёт два аргумента: имя и блок-обработчик. Реализация делает «работу» и в конце вызывает блок, отдавая ему результат:

```
@implementation Downloader
- (void)downloadName:(NSString *)name
      completion:(DownloadHandler)completion {
    NSLog(@"качаю %@ ...", name);
    NSString *result = [name uppercaseString];
    completion(result);    // зовём переданный блок
}
@end
```

`completion(result)` — обычный вызов блока: имя параметра и круглые скобки с аргументом. Метод не знает и не хочет знать, *что* за код внутри блока; его дело — позвать его с результатом.

А вызывающий код передаёт блок прямо на месте, последним аргументом:

```
Downloader *d = [[Downloader alloc] init];
[d downloadName:@"file.txt" completion:^(NSString *result) {
    NSLog(@"готово, получил: %@", result);
}];
```

Литерал блока встал на место аргумента `completion:`. Он работает не сразу, а тогда, когда метод дойдёт до `completion(result)`.

Отличие от Си. В Си «обратный вызов» (callback) делают через указатель на функцию: метод принимает `void (*cb)(const char *)`, а вызывающий передаёт имя заранее объявленной функции. Но функция-callback не видит локальных переменных

места вызова — нужные данные ей прописывают через дополнительный аргумент `void *context`. Блок носит своё окружение с собой, поэтому completion handler в Objective-C пишут прямо по месту, и он сразу видит все окрестные переменные.

Программа с completion handler

Интерфейс, реализация и два вызова с разными обработчиками собраны в файл `code/12-completion.m`. Вывод:

```
2026-06-28 ... качаю file.txt ...
2026-06-28 ... готово, получил: FILE.TXT
2026-06-28 ... качаю photo.png ...
2026-06-28 ... длина результата: 9
```

Один и тот же метод вызван дважды с разными обработчиками: первый печатает результат, второй — его длину. Метод одинаков, поведение «на финише» задаёт вызывающий. Так блоки делают код гибким.

Блоки в Foundation

Многие методы Foundation принимают блок и зовут его для каждого элемента или для сравнения. Два частых примера.

enumerateObjectsUsingBlock: — обойти массив, вызвав блок на каждом элементе. Блок получает сам объект, его индекс и указатель `BOOL *stop`, через который можно досрочно прервать обход:

```
NSArray *fruits = @[@"яблоко", @"груша", @"слива"];
[fruits enumerateObjectsUsingBlock:^(NSString *item,
                                     NSUInteger idx,
                                     BOOL *stop) {
    NSLog(@"%lu: %@", (unsigned long)idx, item);
    if (idx == 1) { *stop = YES; } // остановиться после второго
}];
```

`*stop` — это запись по указателю: метод проверит ячейку после каждого шага и, увидев `YES`, остановится. Вывод этого куска будет чуть ниже, вместе с сортировкой — обе программы у нас в одном файле.

sortedArrayUsingComparator: — отсортировать массив, сравнивая пары элементов блоком-компаратором. Блок возвращает `NSComparisonResult`: «меньше», «равно» или «больше». Отсортируем слова по длине:

```

NSArray *words = @[@"кит", @"муравей", @"слон", @"ёж"];
NSArray *sorted = [words sortedArrayUsingComparator:
    ^NSComparisonResult(NSString *a, NSString *b) {
        return [(a.length) compare:(b.length)];
    }];
NSLog(@"по длине: %@", sorted);

```

Здесь `^NSComparisonResult(...)` — блок с явно указанным типом возвращаемого значения сразу после `^`. Внутри `[(a.length) compare:(b.length)]` заворачивает длину в `NSNumber`, а `compare:` сравнивает два числа и сам выдаёт нужный `NSComparisonResult`.

Обе программы лежат в `code/12-foundation-blocks.m`. Запусти — и увидишь неожиданное:

```

2026-06-28 ... 0: яблоко
2026-06-28 ... 1: груша
2026-06-28 ... по длине: (
    "\U0451\U0436",
    "\U043a\U0438\U0442",
    "\U0441\U043b\U043e\U043d",
    "\U043c\U0443\U0440\U0430\U0432\U0435\U0439"
)
2026-06-28 ... по длине через componentsJoinedByString: ёж, кит,
слон, муравей

```

(Последняя строка в терминале идёт одной строкой, здесь она перенесена, чтобы влезть в строку.)

Обход прервался после «груши», как мы и просили, — до «сливы» дело не дошло. А вот сортировка напечаталась абракадаброй, хотя отработала правильно. Дело не в блоке и не в кодировке файла: так печатает себя сам `NSArray`. Его метод `description` (тот, что зовёт `%@",`) экранирует все не-ASCII символы в вид `\U0451`. `\U0451` — это код буквы «ё», `\U0436` — «ж», то есть первый элемент это «ёж».

Расшифровывать такое глазами невозможно, поэтому запомни приём из последней строки: если надо посмотреть на массив строк по-человечески, склей его в одну строку методом `componentsJoinedByString:`. У `NSString` никакого экранирования нет, и кириллица печатается как есть. Порядок, кстати, верный: «ёж» (2), «кит» (3), «слон» (4), «муравей» (7).

Ловушка блоков: цикл удержания

Теперь — самое важное про блоки и память. Блок **захватывает объекты сильно** (strong): пока жив блок, жив и захваченный объект, потому что блок держит на него крепкую ссылку. Обычно это удобно. Но иногда оборачивается бедой.

Сначала — два слова про «сильно» и «слабо», подробности будут в главе 13 про ARC. Под ARC каждый объект живёт, пока на него есть хотя бы одна **сильная ссылка** (strong reference). Не осталось сильных ссылок — объект уничтожается, срабатывает его dealloc. **Слабая ссылка** (weak reference) на объект указывает, но «не считается»: она не удерживает объект в живых.

Проблема возникает, когда два объекта держат друг друга сильно. Тогда ни один из них никогда не дойдёт до нуля сильных ссылок — оба зависнут в памяти навсегда. Это и есть **цикл удержания** (retain cycle).

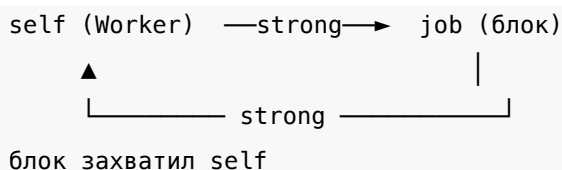
Самый частый цикл — объект хранит блок в своём свойстве, а блок внутри себя обращается к self:

```
@interface Worker : NSObject
@property (nonatomic, copy) void (^job)(void);
@property (nonatomic, copy) NSString *name;
@end
```

Свойство job объявлено copy — для блоков это правильный атрибут (почему copy, разберём ниже в разделе про память). Теперь напишем метод, который кладёт блок в job, а блок упоминает self.name:

```
– (void)setupBad {
    self.job = ^{
        NSLog(@"работаю как %@", self.name);    // захват self!
    };
}
```

Стоит блоку упомянуть self.name, как он захватывает **сам self** — сильно. Получается кольцо:



self держит job, job держит self. Кольцо замкнулось — никто не отпустит другого, dealloc не сработает. Утечка памяти.

Кстати, clang это кольцо **видит** и предупреждает. Если собрать setupBad как есть, компилятор скажет:

```
warning: capturing 'self' strongly in this block is likely to lead
to a retain cycle [-Warc-retain-cycles]
```

В нашем учебном файле мы это предупреждение намеренно глушим прагмой, чтобы показать сам эффект утечки. В рабочем коде предупреждение надо не глушить, а чинить.

Чиним: weakSelf

Лекарство — захватить не сам self, а **слабую ссылку** на него. Тогда блок на self смотрит, но в живых его не держит, и кольцо рвётся:

```
- (void)setupGood {
    __weak typeof(self) weakSelf = self;
    self.job = ^{
        NSLog(@"работаю как %@", weakSelf.name);
    };
}
```

Разберём первую строку по частям:

- `__weak` — слабая ссылка: указывает на объект, но не удерживает его.
- `typeof(self)` — «тип, как у self», то есть `Worker *`. Можно было написать и `Worker *` прямо, но `typeof(self)` не привязан к имени класса и переживёт переименования — это устойчивая идиома.
- `weakSelf = self` — заводим слабую копию ссылки на текущий объект.

Внутри блока обращаемся к `weakSelf`, а не к `self`. Блок захватывает `weakSelf` (слабую ссылку), `self` блок больше сильно не держит — кольца нет. Когда внешний код отпустит `Worker`, тот спокойно уничтожится.

Отличие от Си. В Си такой проблемы нет, потому что нет ни ARC, ни сильных ссылок: память ты освобождаешь руками через `free`, и кольцевых зависимостей подсчёта ссылок просто не существует. Цикл удержания — плата за подсчёт ссылок: объект живёт, пока его кто-то держит. `__weak` подсказывает ARC: «эту ссылку при подсчёте не учитывай».

Видим утечку вживую

Чтобы цикл удержания не остался теорией, поймаем его через `dealloc`. Добавим в `Worker` метод `dealloc` с печатью — он сработает только если объект реально уничтожен. Полный файл —

code/12-retain-cycle.m. К классу Worker (с показанными выше setupBad и setupGood) добавлен dealloc, а в setupBad стоит та самая прагма, глушащая -Warc-retain-cycles:

```
- (void)dealloc {
    NSLog(@"dealloc: Worker %@ уничтожен", self.name);
}
```

Главная функция создаёт два объекта в отдельных вложенных пулах — «плохой» и «хороший» — и смотрит, для кого сработает dealloc:

```
int main(void) {
    @autoreleasepool {
        NSLog(@"--- цикл удержания (setupBad) ---");
        @autoreleasepool {
            Worker *w = [[Worker alloc] init];
            w.name = @"bad";
            [w setupBad];
        } /* w вышел из области, но dealloc НЕ сработает: цикл */
        NSLog(@"(dealloc для bad так и не вызвался)");

        NSLog(@"--- починка (setupGood) ---");
        @autoreleasepool {
            Worker *w = [[Worker alloc] init];
            w.name = @"good";
            [w setupGood];
        } /* здесь dealloc сработает — цикла нет */
        NSLog(@"(объект good освобождён выше)");
    }
    return 0;
}
```

Запуск:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/12-retain-cycle.m -o /tmp/t && /tmp/t
```

Вывод:

```
2026-06-28 ... --- цикл удержания (setupBad) ---
2026-06-28 ... (dealloc для bad так и не вызвался)
2026-06-28 ... --- починка (setupGood) ---
```

```
2026-06-28 ... dealloc: Worker good уничтожен
2026-06-28 ... (объект good освобождён выше)
```

Вот доказательство. Для объекта bad строки `dealloc: Worker bad` уничтожен **нет** — он завис в памяти, цикл не дал ему умереть. А для good строка `dealloc: Worker good` уничтожен появилась ровно там, где объект вышел из своего пула: слабая ссылка разорвала кольцо, и ARC освободил объект. Один и тот же класс, одна разница — `self` против `weakSelf`.

Блоки и память: стек и куча

Когда ты пишешь литерал `^{ ... }`, захватывающий переменные, блок сначала рождается **на стеке** — в кадре текущей функции — и исчезает, как только функция вернётся. Сохранить такой блок «на потом» (в свойство, в массив) нельзя: после выхода из функции там мусор. Поэтому блок при необходимости **копируют в кучу** — операцией `Block_copy` (парная ей `Block_release`); копия в куче живёт, пока на неё есть ссылки, как обычный объект. (Блок, который ничего не захватывает, компилятор кладёт в постоянную память программы: ему нечего терять при выходе из функции.)

Под ARC это происходит **прозрачно**: присваиваешь блок сильной переменной или свойству, возвращаешь из метода — ARC сам копирует его в кучу и сам следит за временем жизни, руками `Block_copy/Block_release` писать не нужно. Поэтому блочное свойство объявляют с атрибутом `copy(@property (nonatomic, copy) void (^job)(void);)` — он подчёркивает «храни в куче». `strong` сработал бы так же, но `copy` остаётся принятым стилем.

Полная программа

Итоговых файлов в главе шесть, каждый — самостоятельная программа:

- `code/12-basic.m` — блок в переменной, рядом с указателем на функцию.
- `code/12-capture.m` — захват окружения и `typedef`.
- `code/12-block-var.m` — изменяемый захват через `__block`.
- `code/12-completion.m` — блок как `completion handler` у своего метода.
- `code/12-retain-cycle.m` — цикл удержания и починка через `weakSelf`.
- `code/12-foundation-blocks.m` — блоки в самом Foundation: обход коллекции с досрочной остановкой и сортировка компаратором.

Проверяем

Каждый файл собирается и запускается одной командой (подставь имя):

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/12-basic.m -o /tmp/t && /tmp/t
```

Все шесть собираются **без предупреждений**. Ожидаемые выводы показаны выше под каждым листингом. Где вместо префикса NSLog стоит ..., у тебя будут дата, время, имя программы и числа в квадратных скобках — их подставляет NSLog.

Частые ошибки

- **Перепутал ^ и *.** Тип блока — `void (^name)(int)` с домиком, а не `void (*name)(int)` со звёздочкой (это указатель на функцию). Перепутаешь — компилятор будет ругаться на несовместимость типов.
- **Меняешь захваченную переменную без `__block`.** Тело блока считает обычный захват константой: `count++` внутри не скомпилируется, пока не пометишь `count` как `__block`.
- **Думаешь, что блок видит свежее значение.** Захват снимает копию в момент *создания* блока. Поменял переменную потом — блок этого не заметит (если только она не `__block`).
- **Цикл удержания через `self`.** Блок в свойстве объекта, упоминающий `self` (в том числе скрыто, через `ivar` или `self.property`), создаёт кольцо. Лекарство — `__weak typeof(self) weakSelf = self;` и обращение к `weakSelf`. Не глуши `-Warc-retain-cycles` — чини причину.
- **Забыл, что блок может не вызваться.** Completion handler вызывает *чужой* код. Если он по какой-то ветке забудет позвать твой блок, твой «финальный» код не выполнится. Проектируя метод с блоком, вызывай его на всех путях.
- **Возвращаешь стековый блок без ARC.** Под ARC безопасно; в ручном режиме (глава 13) вернуть захватывающий блок из функции без `Block_copy` — обращение к мусору после выхода из функции. Прямое `return ^{ ... x ... };` clang поймает сам (`returning block that lives on the local stack`), а вот через промежуточную переменную — уже нет.

Упражнения

1. Напиши блок `int (^maxB)(int, int)`, возвращающий большее из двух чисел. Вызови его с разными парами и выведи результат.
2. Заведи `typedef double (^MathOp)(double, double)` и три переменные этого типа: сложение, вычитание, умножение. Положи их в `NSArray` и пройди циклом, применяя каждую к паре 10 и 3.
3. Сделай блок-счётчик с `__block`: каждый вызов увеличивает счётчик и печатает его. Вызови пять раз.

4. Добавь классу `Downloader` метод `processNames:(NSArray *)names completion:(void (^)(NSInteger))completion`, который «обрабатывает» все имена и в конце зовёт `completion` с их количеством.
5. Возьми массив строк и отсортируй его `sortedArrayUsingComparator:` по убыванию длины (поменяй порядок аргументов в `compare:`).
6. Сломай `setupGood:` верни внутри блока `self` вместо `weakSelf`, собери и убедись, что `dealloc` перестал срабатывать. Потом верни `weakSelf` обратно.

Что мы получили

Теперь ты умеешь упаковывать действие в переменную. Разобрали тип блока `void (^)(name)(int)` и литерал `^(int x){ ... }`, домик `^` против звёздочки указателя на функцию, захват окружения (копия только для чтения) и изменяемый захват через `__block`, `typedef` для читаемых типов, блок как аргумент метода (completion handler) и методы Foundation с блоками. И главное — встретили цикл удержания вживую через `dealloc` и починили его слабой ссылкой `weakSelf`.

Блоки вплотную подвели нас к теме памяти: сильные и слабые ссылки, кто кого удерживает, когда срабатывает `dealloc`. Ровно об этом — следующая глава 13 про ARC, где «сильно/слабо», `strong/weak`, `copy` и весь подсчёт ссылок разбираются с самого начала и до конца.

Документация Apple

- Programming with Objective-C → «Working with Blocks» (как объявлять, передавать и захватывать) — <https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/WorkingwithBlocks/WorkingwithBlocks.html>
- Blocks Programming Topics (полный справочник по блокам, `__block`, `Block_copy`) — https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/Blocks/Articles/00_Introduction.html
- Transitioning to ARC Release Notes (сильные и слабые ссылки, захват `self` в блоках) — <https://developer.apple.com/library/archive/releasenotes/ObjectiveC/RN-TransitioningToARC/Introduction/Introduction.html>
- `NSArray` (`enumerateObjectsUsingBlock:`, `sortedArrayUsingComparator:`) — <https://developer.apple.com/documentation/foundation/nsarray>

Глава 13. Управление памятью: ARC и подсчёт ссылок

В главе 3 ты выделял память руками: `malloc` просил у системы байты, `free` их возвращал, и всю бухгалтерию ты вёл сам. В главе 7 мы увидели вторую половину жизни объекта — `dealloc`, момент смерти, — но честно отложили вопрос: а кто и по какому правилу решает, что объект пора уничтожить? Пришло время собрать полную картину.

Эта глава — про то, как объекты Objective-C живут и умирают. Сначала руками, на счётчике ссылок, чтобы прочувствовать механику (это код **без ARC**). Потом включим ARC и увидим, что компилятор делает ровно ту же работу за нас. А под конец встретим главного врага памяти — **цикл удержания** — и научимся резать его одной слабой ссылкой.

Что мы сделаем

- Поймём проблему: объект в куче надо когда-то освободить, но **слишком рано** → крах по висячему указателю, **слишком поздно или никогда** → утечка.
- Разберём **подсчёт ссылок** (reference counting): счётчик, владение, правила `alloc/retain/release`.
- Своими руками прогоним цикл `retain/release/dealloc` — это файл `.mrr.m`, который собирается **без ARC**.
- Поймём `autorelease` и `@autoreleasepool` из главы 1 — наконец до конца.
- Включим **ARC** и увидим тот же пример короче и безопаснее. Разберём, что ARC делает и чего **не** делает.
- Разложим **strong / weak / unsafe_unretained** и спецификаторы `__strong / __weak`.
- Построим **цикл удержания**, увидим, что `dealloc` молчит, и починим его словом `weak`.
- В конце — практический совет про `@autoreleasepool` внутри циклов.

Разбираем задачу руками

Объект живёт в куче (heap) — той самой большой области памяти из главы 3, которую `malloc` раздаёт по запросу. Память конечна. Значит, когда объект больше не нужен, его место надо вернуть системе. Вопрос на миллион: **когда именно «больше не нужен»?**

Тут две ошибки, обе дорогие.

Освободить слишком рано. Ты вернул память системе, но в другой переменной ещё лежит адрес этого объекта. Адрес есть — а объекта по нему уже нет (или там поселился кто-то другой). Это **висячий указатель** (dangling pointer). Пошлешь по нему сообщение — программа в лучшем случае рухнет, в худшем тихо испортит чужие данные.

Освободить слишком поздно или никогда. Ты забыл вернуть память. Объект больше не нужен, но занимает место. Один такой — пустяк. Но если это происходит в цикле или при каждом действии пользователя, память течёт, пока приложение не съест всё и система его не убьёт. Это **утечка** (memory leak).

Отличие от Си. В Си ты решаешь обе проблемы вручную: сам зовёшь `malloc`, сам зовёшь `free`, сам следишь за каждым байтом и за каждым указателем на него. Забыл `free` — утечка. Позвал `free` дважды или обратился после него — крах. В Objective-C объект **сам считает ссылки на себя**, а ARC автоматизирует освобождение. Но это **не** сборщик мусора, как в Java или C#: момент уничтожения определён **детерминированно**, по счётчику, а не выбирается фоновым процессом когда-нибудь потом.

Как же объект понимает, что он больше не нужен? Через **счётчик ссылок**.

Шаг 1. Модель подсчёта ссылок

Идея простая до гениальности. Внутри каждого объекта есть число — **счётчик ссылок** (reference count, он же retain count). Это «сколько владельцев у меня прямо сейчас».

Правила игры (исторически их называют правилами **MRR** — Manual Retain Release, ручного управления):

- Получил объект от метода, чьё имя начинается с `alloc`, `new`, `copy` или `mutableCopy`, — ты стал владельцем. У только что созданного объекта счётчик равен 1. (`new` — это `alloc` и `init` одним сообщением.)
- Хочешь стать ещё одним владельцем уже существующего объекта — пошли ему `retain`, счётчик +1.
- Закончил с объектом — пошли `release`, счётчик -1. Ты больше не владелец.
- Когда счётчик доходит до **нуля**, владельцев не осталось — система посылает объекту `dealloc` и возвращает память.

Метафора — комната с лампой и людьми. Кто входит в комнату, тому нужен свет: он включает свою «долю» (счётчик +1). Кто выходит — гасит свою долю (счётчик -1). Пока в комнате хоть один человек, лампа горит. Вышел последний — счётчик 0 — лампа гаснет (dealloc). Никто не гасит свет, пока в комнате остались люди; никто не сидит в темноте при живых посетителях.

Ключевое слово здесь — **владение** (ownership). Не «кто видит объект», а «кто за него отвечает». Можно держать указатель на объект и **не** быть владельцем (мы дойдём до таких ссылок — weak). Владелец — тот, кто сделал alloc/retain/copy и обязан в ответ сделать release.

Правило-мнемоника от Apple: **на каждый твой alloc/new/copy/retain должен найтись ровно один release (или autorelease)**. Балансируй плюсы и минусы — и счётчик сойдётся к нулю в нужный момент.

Шаг 2. Прогоняем счётчик руками (без ARC)

Хватит слов — посмотрим на счётчик живьём. Напишем класс Person и вручную поведем его счётчик вверх-вниз, печатая значение на каждом шаге.

Это код **БЕЗ ARC**. Файл назовём code/13-mrr-refcount.mrr.m — двойное расширение .mrr.m для нашего сборщика метка: «компилируй меня с -fno-objc-arc». Под ARC всё, что мы тут напишем руками, запрещено — компилятор не даст.

```
#import <Foundation/Foundation.h>

@interface Person : NSObject {
    NSString *_name;
}
- (instancetype)initWithName:(NSString *)name;
@end

@implementation Person

- (instancetype)initWithName:(NSString *)name {
    self = [super init];
    if (self) {
        _name = [name copy];    /* copy даёт нам ВЛАДЕНИЕ строкой */
    }
    return self;
}

- (void)dealloc {
```

```
    NSLog(@"dealloc: %@ уничтожен", _name);
    [_name release];          /* отпускаем то, чем владели */
    [super dealloc];          /* под MRR обязательно и последней строкой */
}

@end
```

Прежде чем перейти к `main`, разберём два новых для тебя места в `dealloc` — они есть **только без ARC**.

`[_name release]` — мы в `init` сделали `[name copy]` и тем самым стали владельцем строки `_name`. Правило требует баланса: раз был плюс (`copy`), нужен минус (`release`). Делаем его в `dealloc` — последний момент жизни объекта.

`[super dealloc]` — в самом конце отдаём управление родителю, чтобы `NSObject` освободил уже свою часть и вернул память системе. **Без ARC эта строка обязательна и идёт последней.**

Отличие от Си. В главе 7 под ARC мы писали `dealloc` без единой из этих строк — компилятор вставлял их сам. Здесь, без ARC, ты снова, как в Си с `free`, отвечаешь за освобождение руками: каждый свой `retain/ copy` уравновесить `release`, в конце позвать `[super dealloc]`. Забудешь — утечка; позовёшь лишний раз — крах. ARC ровно эту рутину и отбирает.

Теперь `main` — будем двигать счётчик и печатать его:

```
int main(void) {
    @autoreleasepool {
        /* alloc даёт владение: счётчик ссылок = 1. */
        Person *p = [[Person alloc] initWithName:@"Аружан"];
        NSLog(@"после alloc/init: retainCount = %lu",
              (unsigned long)[p retainCount]);

        /* retain: ещё +1. Счётчик = 2. */
        [p retain];
        NSLog(@"после retain:      retainCount = %lu",
              (unsigned long)[p retainCount]);

        /* release: -1. Счётчик = 1. Объект ещё жив. */
        [p release];
        NSLog(@"после release:    retainCount = %lu",
              (unsigned long)[p retainCount]);
    }
}
```

```
    /* Последний release: -1. Счётчик = 0 → система шлёт dealloc. */  
    NSLog(@"шлём последний release...");  
    [p release];  
    NSLog(@"объект больше не существует");  
}  
return 0;  
}
```

[p retainCount] — сообщение, которое возвращает текущее значение счётчика. Тип результата — NSUInteger (беззнаковое целое), под него в NSLog идёт %lu, и значение приводим к (unsigned long), чтобы тип совпал на любой платформе (та же история, что с %ld и (long) для NSInteger в главе 7).

Собираем командой **без ARC** и запускаем:

```
clang -fno-objc-arc -framework Foundation -Wall -Wextra -O2 \  
    code/13-mrr-retaincount.mrr.m -o mrr  
./mrr
```

Вывод (дату, имя программы и числа в скобках от NSLog убираю — у тебя будут свои):

```
после alloc/init: retainCount = 1  
после retain:      retainCount = 2  
после release:     retainCount = 1  
шлём последний release...  
dealloc: Аружан уничтожен  
объект больше не существует
```

Вот он, счётчик в движении. alloc поднял его до 1. retain — до 2. Первый release опустил до 1 — объект жив, dealloc молчит. И только **второй** release, обнуливший счётчик, мгновенно вызвал dealloc: строчка смерти встала ровно между «шлём последний release...» и «объект больше не существует». Уничтожение **детерминировано** — оно случилось в ту же секунду, на той же строке, где счётчик стал нулём.

Заметь баланс: один alloc и один retain — это два плюса; два release — два минуса. Сошлось в ноль. Если бы мы забыли второй release, объект бы завис в памяти навсегда (утечка). Если бы позвали release третий раз — обратились бы к уже мёртвому объекту (крах по всяческому указателю). Ровно те две беды из начала главы.

Про retainCount начистоту. Мы печатаем счётчик только ради учёбы — увидеть механику глазами. В реальном коде на retainCount **не опираются**: у системных объектов он бывает неожиданным (строки-литералы, кеши, оптимизации). Apple прямо

не советует принимать решения по его значению. Доверяй не числу, а правилу баланса плюсов и минусов.

Шаг 3. autorelease и пул автоосвобождения

С release есть неудобный случай. Представь фабричный метод (как +personWithName: из главы 7), который создаёт объект и **возвращает** его. Внутри он сделал alloc — значит, владеет. Но отдать наружу он должен **не свой** объект, а «готовый, бери». Кто делает release? Сам метод не может — он вернёт объект уже мёртвым. А вызывающий не делал alloc, по правилам он и не обязан звать release.

Выход — **отложенный release**, он же autorelease. Послать объекту autorelease значит: «не отпускай прямо сейчас, но запиши, что один release за тобой — отдашь его позже сам». Этим «позже» заведует **пул автоосвобождения** (autorelease pool).

```
/* Фабричный метод без ARC возвращает autoreleased-объект. */
+ (instancetype)personWithName:(NSString *)name {
    Person *p = [[Person alloc] initWithName:name]; /* +1, мы владеем */
    return [p autorelease]; /* «-1 позже»: отдаём владение пулу */
}
```

[p autorelease] возвращает тот же объект p, но помечает его: «когда текущий пул закроется, пошли мне release». Объект остаётся живым на всё время, пока вызывающий с ним работает, а лишний release случится автоматически при закрытии пула. Баланс соблюден, и никому не достался мёртвый объект.

Вот теперь понятно, **зачем @autoreleasepool из главы 1**. Эта обёртка и есть тот самый пул:

```
@autoreleasepool {
    /* ... код, создающий временные объекты ... */
} /* на этой скобке пул шлёт release всем autoreleased-объектам внутри */
```

Открывающая { создаёт пул. Закрывающая } его **сливает** (drain): каждому объекту, помеченному autorelease внутри блока, прилетает отложенный release. Без обёртки autoreleased-объектам некому было бы послать этот release — они бы потекли. Поэтому в главе 1 мы и завернули всё в @autoreleasepool: даже простой NSLog мог по дороге создать временные autoreleased-строки, и им нужен пул, который их приберёт.

Отличие от Си. В Си нет ни autorelease, ни пулов — нечему откладывать освобождение, объектов нет. @autoreleasepool существует только из-за объектной модели Objective-C: это «корзина», в которую складывают временные объекты, чтобы освободить их разом на выходе из блока, а не отслеживать каждый поодиночке.

Важно: autorelease никуда не делся и под ARC. Просто теперь release, retain и autorelease за тебя расставляет компилятор. А @autoreleasepool ты по-прежнему пишешь руками — он управляет **временем** слива, и в одном месте главы это окажется практически важно (см. последний шаг).

Шаг 4. Включаем ARC — тот же пример, короче

ARC (Automatic Reference Counting, автоматический подсчёт ссылок) — это не новая модель памяти. Это **тот же** подсчёт ссылок, только retain/release/autorelease расставляет **компилятор** на этапе сборки, по тем же правилам владения, что мы только что гоняли руками. Включается флагом -fobjc-arc (мы ставим его с главы 1).

Перепишем Person под ARC. Файл — code/13-arc-basic.m, обычное расширение .m:

```
#import <Foundation/Foundation.h>

@interface Person : NSObject
@property (nonatomic, copy) NSString *name;
- (instancetype)initWithName:(NSString *)name;
@end

@implementation Person

- (instancetype)initWithName:(NSString *)name {
    self = [super init];
    if (self) {
        _name = [name copy];
    }
    return self;
}

- (void)dealloc {
    NSLog(@"dealloc: %@ уничтожен", _name);
}

@end
```

Сравни этот dealloc с MRR-версией. Исчезли [_name release] и [super dealloc] — под ARC они **запрещены**, компилятор вставит их сам. Остался только наш учебный NSLog. Если бы не он, dealloc можно было бы вообще не писать.

Теперь main. Никаких retain/release — двигаем счётчик **через сами ссылки**: присвоил указатель → ARC сделал retain, обнулil в nil → ARC сделал release.

```
int main(void) {
    @autoreleasepool {
        /* Сильная ссылка p: ARC удерживает объект, счётчик = 1. */
        Person *p = [[Person alloc] initWithName:@"Аружан"];
        NSLog(@"объект жив: %@", p.name);

        /* Вторая сильная ссылка на тот же объект: счётчик = 2. */
        Person *q = p;
        NSLog(@"q указывает на того же человека: %@", q.name);

        /* Обнуляем p: счётчик = 1. Объект ещё держит q — dealloc НЕ будет. */
        p = nil;
        NSLog(@"p = nil, но q ещё держит объект — dealloc не было");

        /* Обнуляем q: счётчик = 0 → ARC шлёт dealloc прямо здесь. */
        q = nil;
        NSLog(@"q = nil — объект уже уничтожен (dealloc выше)");
    }
    return 0;
}
```

Собираем уже **с ARC** и запускаем:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/13-arc-basic.m -o arcbasic
./arcbasic
```

Вывод:

```
объект жив: Аружан
q указывает на того же человека: Аружан
p = nil, но q ещё держит объект — dealloc не было
dealloc: Аружан уничтожен
q = nil — объект уже уничтожен (dealloc выше)
```

Та же механика, что в MRR-примере, но мы не написали ни одного retain и ни одного release. ARC увидел `Person *q = p;` и вставил retain (счётчик 2). Увидел `p = nil;` — вставил release (счётчик 1, объект жив, тишина). Увидел `q = nil;` — вставил последний release (счётчик 0), и dealloc сработал ровно на этой строке. Детерминированно, как по часам.

Что ARC делает и чего НЕ делает

Делает: расставляет retain при появлении новой сильной ссылки, release при её исчезновении (переприсвоение, выход из области видимости, = nil), autorelease где нужно вернуть объект, и [super dealloc] в конце dealloc. Всё по правилам владения, на этапе компиляции, без накладных расходов в рантайме сверх самих retain/release.

Не делает:

- **ARC — не сборщик мусора.** Нет фонового потока, который ходит по памяти и решает, что собрать. Освобождение по-прежнему происходит **в точный момент** обнуления счётчика, в твоём коде, предсказуемо.
- **ARC не ловит циклы удержания.** Если два объекта сильно держат друг друга, их счётчики никогда не дойдут до нуля сами — и ARC честно их **не** освободит. Это утечка, и отвечаешь за неё ты. Ровно об этом — следующий шаг.
- **ARC не управляет не-объектной памятью.** malloc/free из главы 3, CFRetain/CFRelease для Core Foundation, файловые дескрипторы — всё это по-прежнему на тебе. ARC считает ссылки только на объекты Objective-C.

Отличие от Си. В Си ты пишешь malloc и free и отвечаешь за каждый байт лично. Под ARC освобождение объектов автоматизировано — но это **не** магия и **не** сборщик мусора как в Java: всё та же детерминированная арифметика счётчика, просто retain/release пишет компилятор, а не ты. Память объекта возвращается системе в строго определённый момент, а не «когда-нибудь, когда дойдут руки у GC».

Шаг 5. Сильные и слабые ссылки

Мы всё время говорили «сильная ссылка». Пора назвать виды ссылок явно — это атрибуты, которые ты уже видел у @property в главе 8, теперь с полным смыслом.

strong (сильная) — ссылка-владелец. Присвоение в сильную переменную делает retain (+1), обнуление — release (−1). Объект жив, пока на него указывает хоть одна сильная ссылка. Это **поведение по умолчанию** для объектных свойств и переменных под ARC: если не написал ничего, ссылка сильная.

weak (слабая) — ссылка-наблюдатель. Она указывает на объект, но **не владеет** им: счётчик не трогает. Объект может умереть, пока ты на него слабо ссылаешься. И вот ключевая способность: когда объект умирает, ARC **сам обнуляет** все слабые ссылки на него в nil. Поэтому слабая ссылка либо показывает живой объект, либо честный nil — но **никогда** не висячий указатель.

unsafe_unretained / assign — тоже не владеет (счётчик не трогает), **но и не обнуляется**. Объект умер — ссылка осталась указывать на освобождённую память. Это и есть висячий указатель, и обращение по нему — крах. Название говорит само: `unsafe` — небезопасно. Использовать только когда точно знаешь, что делаешь (или ради совместимости со старым кодом). Атрибут `assign` исторически означает то же для свойств.

Сводка:

ссылка	владеет?	счётчик	когда объект умер
<code>strong</code>	да	+1 / -1	(не даст умереть, пока держит)
<code>weak</code>	нет	не трогает	сама становится <code>nil</code>
<code>unsafe_unretained</code>	нет	не трогает	остаётся ВИСЯЧЕЙ → крах

Те же слова есть и для **локальных переменных** — как спецификаторы с двумя подчёркиваниями: `__strong` (по умолчанию), `__weak`, `__unsafe_unretained`:

```
Person *owner = somebody; /* __strong по умолчанию: владеет */
__weak Person *watcher = somebody; /* слабая локальная ссылка */
```

Между `@property (... , weak)` и `__weak Person *x` разница только в месте: первое — свойство объекта, второе — локальная переменная или `ivar`. Смысл один.

Отличие от Си. Слабая ссылка, которая **сама превращается в `nil`**, когда объект умер, — в Си такого механизма нет в принципе. Освободил память через `free`, а все указатели, что на неё смотрели, остались висеть: язык про них не знает и не обнулит. Поймать обращение к освобождённой памяти в Си — отдельная боль (`valgrind`, санитайзеры). `weak` в Objective-C закрывает целый класс ошибок: висячий указатель просто не возникает — рантайм следит за объектом и гасит ссылки на него при смерти.

Шаг 6. Цикл удержания — главная ловушка

Подсчёт ссылок прекрасен, но у него есть слепое пятно. Что если два объекта держат друг друга **сильно**?

Заведём родителя и ребёнка. Пусть родитель сильно держит ребёнка (логично — владеет), а ребёнок сильно держит родителя (чтобы знать, чей он). Файл — `code/13-retain-cycle.m`:

```
#import <Foundation/Foundation.h>

@class Child;
```



```
@interface Parent : NSObject
@property (nonatomic, copy) NSString *name;
@property (nonatomic, strong) Child *child;    /* сильная: владеет ребёнком */
@end

@interface Child : NSObject
@property (nonatomic, copy) NSString *name;
@property (nonatomic, strong) Parent *parent; /* сильная ← вот проблема */
@end

@implementation Parent
- (void)dealloc { NSLog(@"dealloc Parent (%@)", self.name); }
@end

@implementation Child
- (void)dealloc { NSLog(@"dealloc Child (%@)", self.name); }
@end
```

Строка `@class Child;` — **предварительное объявление** (forward declaration): «класс `Child` существует, подробности будут ниже». Без неё `Parent` не смог бы упомянуть `Child` раньше, чем тот объявлен. Оба `dealloc` печатают строчку — это наши датчики смерти.

`main` связывает их в кольцо:

```
int main(void) {
    @autoreleasepool {
        Parent *mom = [Parent new];
        mom.name = @"Мама";
        Child *kid = [Child new];
        kid.name = @"Дочь";

        mom.child = kid;    /* мама держит дочь (+1 к счётчику kid) */
        kid.parent = mom;   /* дочь держит маму (+1 к счётчику mom): цикл */

        NSLog(@"выходим из блока — ждём два dealloc...");
    } /* mom и kid выходят из видимости, но держат друг друга */

    NSLog(@"блок закрыт. Видел dealloc выше? Нет. Это утечка памяти.");
    return 0;
}
```

[Parent new] — короткая форма [[Parent alloc] init]: метод класса +new унаследован от NSObject и делает оба шага разом. Имя начинается с new, значит, по правилам шага 1 он тоже даёт владение, счётчик 1.

Посчитаем счётчики руками. После mom.child = kid у kid две сильные ссылки: локальная kid и свойство mom.child — счётчик 2. После kid.parent = mom у mom тоже две: локальная mom и kid.parent — счётчик 2.

Теперь блок закрывается. ARC обнуляет локальные mom и kid — по release каждому. Счётчик mom падает с 2 до 1 (его ещё держит kid.parent). Счётчик kid падает с 2 до 1 (его ещё держит mom.child). **Ни один не дошёл до нуля.** Оба живы, но снаружи на них уже **нет ни одной** ссылки — добраться до них нельзя, а память занята. Классическая утечка.

Собираем и запускаем:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/13-retain-cycle.m -o cycle
./cycle
```

Вывод:

выходим из блока — ждём два dealloc...

блок закрыт. Видел dealloc выше? Нет. Это утечка памяти.

Между двумя строками **должны** были стоять два dealloc — и их нет. Объекты не умерли. ARC ничем не помог: с его точки зрения у каждого счётчик 1, владелец есть, освобождать нельзя. Он не видит, что эти двое держат друг друга по кругу и больше не нужны никому. Это и есть то, чего **ARC не делает** — не разрывает циклы.

Где такие циклы встречаются на практике:

- **parent ↔ child**, как у нас: родитель держит ребёнка, ребёнок — родителя.
- **объект ↔ его блок**. Блок захватывает self (глава 12), а объект сильно держит блок в свойстве. self держит блок, блок держит self. Лекарство ты уже видел в главе 12 — __weak-копия self (weakSelf).
- **delegate**. Объект А держит объект В, а В держит А через свойство delegate. Поэтому свойства-делегаты почти всегда объявляют weak.

Во всех трёх случаях лечение одно: **одну из двух ссылок сделать слабой**.

Шаг 7. Режим цикл словом weak

Кто должен уступить и стать слабым? Тот, кто **не владеет** по смыслу. Родитель логично владеет ребёнком — пусть `Parent.child` остаётся `strong`. А ребёнок родителем не владеет, лишь ссылается на него «вверх» — значит `Child.parent` должно быть **weak**. Меняем один атрибут.

Заодно покажем в этом же файле второй талант weak — автообнуление. Файл — `code/13-weak-fix.m`:

```
#import <Foundation/Foundation.h>

@class Child;

@interface Parent : NSObject
@property (nonatomic, copy) NSString *name;
@property (nonatomic, strong) Child *child;    /* сильная: родитель владеет */
@end

@interface Child : NSObject
@property (nonatomic, copy) NSString *name;
@property (nonatomic, weak) Parent *parent;    /* WEAK: не владеет */
@end

@implementation Parent
- (void)dealloc { NSLog(@"dealloc Parent %@", self.name); }
@end

@implementation Child
- (void)dealloc { NSLog(@"dealloc Child %@", self.name); }
@end
```

Единственное изменение против прошлого файла — `strong` → `weak` у `Child.parent`. Теперь `kid.parent = mom` **не** увеличивает счётчик `mom`. Цикла нет.

`main` с двумя сценариями:

```
int main(void) {
    @autoreleasepool {
        /* --- Сценарий 1: цикл починен, оба dealloc срабатывают --- */
        NSLog(@"=== Сценарий 1: цикл разорван weak ===");
    }
}
```

```

    Parent *mom = [Parent new];
    mom.name = @"Мама";
    Child *kid = [Child new];
    kid.name = @"Дочь";

    mom.child = kid;    /* сильная: +1 к счётчику kid */
    kid.parent = mom;   /* weak:   счётчик mom НЕ меняется */

    NSLog(@"выходим из вложенного блока – ждём два dealloc...");
}   /* mom и kid освобождаются: ничто не держит их по кругу */
NSLog(@"оба dealloc были выше – утечки нет");

/* ---- Сценарий 2: weak сам обнуляется в nil ---- */
NSLog(@"=== Сценарий 2: weak обнуляется автоматически ===");
Child *observer = [Child new];
observer.name = @"Наблюдатель";
@autoreleasepool {
    Parent *temp = [Parent new];
    temp.name = @"Времянка";
    observer.parent = temp;    /* weak-ссылка на temp */
    NSLog(@"пока temp жив: observer.parent = %@",
           observer.parent.name);
}   /* temp выходит из видимости и умирает */
/* weak-ссылка сама стала nil – всяческого указателя НЕТ */
NSLog(@"temp умер: observer.parent = %@", observer.parent);
}
return 0;
}

```

Собираем и запускаем:

```

clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/13-weak-fix.m -o weakfix
./weakfix

```

Вывод:

```

=== Сценарий 1: цикл разорван weak ===
выходим из вложенного блока – ждём два dealloc...
dealloc Parent (Мама)

```

```
dealloc Child (Дочь)
оба dealloc были выше — утечки нет
=== Сценарий 2: weak обнуляется автоматически ===
пока temp жив: observer.parent = Времянка
dealloc Parent (Времянка)
temp умер: observer.parent = (null)
dealloc Child (Наблюдатель)
```

Сравни с прошлым шагом. Там между строками не было ни одного `dealloc` — тут в сценарии 1 их **два**: `Parent` и `Child` спокойно умерли на выходе из вложенного блока. Одно слово `weak` разорвало кольцо. Теперь, когда исчезли локальные `mom` и `kid`, счётчик `mom` сразу упал до 0 (`kid` держал его лишь слабо — счётчик не трогал), за ним освободился и `kid`.

Сценарий 2 показывает автообнуление. Пока `temp` жив, `observer.parent` печатает `Времянка`. Как только `temp` вышел из видимости и получил `dealloc`, слабая `observer.parent` **сама** стала `nil` — в выводе `(null)`. Никакого висячего указателя: мы обратились к умершей ссылке и получили честный `nil`, а не крах.

Отличие от Си. Повтори про себя сценарий 2 на языке Си. Ты бы сделал `free(temp)`, а указатель `observer->parent` остался бы смотреть на освобождённую память — висячим. Печать по нему — неопределённое поведение: мусор или падение. В Objective-C `weak` превратил эту мину в аккуратный `nil`. Механизма «обнули за меня все ссылки на умерший объект» в Си нет.

Шаг 8. autoreleasepool внутри циклов

Последний практический сюжет — и он снова про `@autoreleasepool` из главы 1, теперь с пользой по делу.

Многие методы Foundation возвращают **autoreleased**-объекты (временные, которые освободятся при сливе пула). В обычном коде это незаметно: пул закрывается в конце `main` или итерации события — и всё прибирается. Но если ты в **длинном цикле** создаёшь тысячи временных объектов, они **копятся** в пуле до самого его слива. Память пухнет, хотя на каждом шаге тебе нужен лишь один объект.

```
/* Плохо: все временные строки копятся в пуле до конца цикла. */
for (NSInteger i = 0; i < 1000000; i++) {
    NSString *line = [NSString stringWithFormat:@"строка %ld", (long)i];
    обработать(line);
    /* line — autoreleased, release ей придёт только в конце main */
}
```

Лекарство — завести **свой** пул **внутри** цикла. Тогда временные объекты каждой итерации освобождаются сразу, на её закрывающей скобке, и память не растёт:

```
/* Хорошо: внутренний пул сливается на каждой итерации. */
for (NSInteger i = 0; i < 1000000; i++) {
    @autoreleasepool {
        NSString *line = [NSString stringWithFormat:@"строка %ld", (long)i];
        обработать(line);
    } /* здесь release всем временным объектам ЭТОЙ итерации */
}
```

Правило-памятка: **тяжёлый цикл, плодящий много временных объектов, — оберни тело в @autoreleasepool**. Под ARC это твой главный рычаг управления **временем** освобождения временных объектов: сами retain/release компилятор расставил, а вот когда сливать пул — решаешь ты. Это та же обёртка из главы 1, просто поставленная в нужное место.

Проверяем

Файлы главы и команды сборки. Обрати внимание: MRR-файл собирается с `-fno-objc-arc`, остальные — с `-fobjc-arc`.

```
# Ручной подсчёт ссылок — БЕЗ ARC:
clang -fno-objc-arc -framework Foundation -Wall -Wextra -O2 \
    code/13-mrr-refcount.mrr.m -o mrr && ./mrr

# Тот же пример под ARC:
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/13-arc-basic.m -o arcbasic && ./arcbasic

# Цикл удержания (dealloc НЕ срабатывает):
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/13-retain-cycle.m -o cycle && ./cycle

# Починка слабой ссылкой (dealloc срабатывает + weak → nil):
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/13-weak-fix.m -o weakfix && ./weakfix
```

Все четыре собираются без единого предупреждения и печатают вывод, показанный выше (даты и числа в начале строк будут своими).

Частые ошибки

- **Думать, что ARC — сборщик мусора.** Нет. Освобождение детерминировано: ровно в момент, когда счётчик стал нулём. Никакого фонового сборщика, никакого «когда-нибудь потом».
- **Забывать про retain cycle.** ARC его **не** ловит. Два объекта, сильно держащие друг друга, утекут молча. Признак: ты ждёшь dealloc, а его нет. Лекарство — одну ссылку сделать weak.
- **Свойство-делегат объявить strong.** Почти всегда это половина цикла. Делегаты и обратные ссылки «вверх» (child → parent, cell → tableView) делай weak.
- **Захватить self в блоке сильно.** Объект держит блок, блок держит self — цикл (глава 12). Используй __weak-копию self.
- **Под ARC написать retain/release/autorelease/[super dealloc].** Запрещено — компилятор выдаст ошибку. Эти вызовы только в MRR-коде (.mrr.m, флаг -fno-objc-arc).
- **Опираться на retainCount.** Значение бывает неожиданным у системных объектов. Это учебный индикатор, не основа для решений.
- **Тяжёлый цикл без внутреннего @autoreleasepool.** Временные объекты копят, память пухнет. Оберни тело цикла в свой пул.
- **unsafe_unretained там, где нужен weak.** Объект умер — ссылка повисла, обращение к ней роняет программу. Если не уверен — бери weak.

Упражнения

1. В 13-mrr-refcount.mrr.m добавь ещё один [p retain] сразу после первого. Сколько теперь должно быть release, чтобы объект корректно умер? Допиши их и проверь по выводу, что dealloc сработал ровно один раз.
2. В том же файле **намеренно** убери последний [p release]. Что пропадёт из вывода? Объясни, почему это утечка. Верни строку обратно.
3. В 13-arc-basic.m после q = nil; добавь NSLog(@"%@", p.name);. Что напечатается и почему это безопасно (вспомни про nil из главы 5)?
4. Возьми 13-retain-cycle.m и почини его **другим** концом: оставь Child.parent сильным, а Parent.child сделай weak. Сработают ли оба dealloc? Подумай, какой из двух вариантов починки правильнее по смыслу «кто кем владеет».
5. В 13-weak-fix.m поменяй в сценарии 2 атрибут weak на unsafe_unretained и запусти. Что выведет последняя строка (и выведет ли вообще)? Почему weak безопаснее.
6. Напиши крошечную программу: в цикле на 5 итераций создавай NSString stringWithFormat: и логируй. Оберни тело в @autoreleasepool и убедись, что всё работает; объясни, что меняет эта обёртка при миллионе итераций.

Что мы получили

Теперь у тебя полная картина памяти Objective-C. Объект носит внутри **счётчик ссылок**: `alloc/new/copy/retain` его поднимают, `release` опускает, на нуле — `dealloc` и возврат памяти. Ты прогнал этот счётчик руками (код **без ARC**) и увидел, что **ARC** делает ровно то же, только `retain/release/autorelease` пишет за тебя компилятор — это детерминированный подсчёт, а **не** сборщик мусора. Ты различаешь **strong** (владеет, +1), **weak** (не владеет, сам станет `nil`) и опасный **unsafe_unretained** (повиснет). Ты построил **цикл удержания**, увидел, как `dealloc` молчит, и разрезал кольцо одним словом `weak`. И понял, зачем `@autoreleasepool` — и как им гасить рост памяти в тяжёлых циклах.

Главное, что унеси: ARC снял с тебя рутину `retain/release`, но **ответственность за владение оставил**. Думай не «кто видит объект», а «кто им владеет» — и циклы удержания перестанут быть для тебя неожиданностью.

В следующей главе мы заглянем под капот ещё глубже — в сам **runtime Objective-C**: указатель `isa`, метаклассы, дерево классов и то, как `objc_msgSend` находит метод при каждой отправке сообщения. Память мы закрыли — пора понять, как вообще работает диспетчеризация, на которой держится весь язык.

Документация Apple

- Programming with Objective-C → **Encapsulating Data** (владение и `strong/weak/copy` у свойств) — <https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/EncapsulatingData/EncapsulatingData.html>
- **Advanced Memory Management Programming Guide** — полная модель владения, правила `retain/release/autorelease`, пулы автоосвобождения — <https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/MemoryMgmt/Articles/MemoryMgmt.html>
- **Transitioning to ARC Release Notes** — что делает ARC, что под ним запрещено, `__strong/__weak/__unsafe_unretained`, циклы удержания и их разрыв — <https://developer.apple.com/library/archive/releasenotes/ObjectiveC/RN-TransitioningToARC/Introduction/Introduction.html>
- `NSObject` — `alloc`, `retain`, `release`, `autorelease`, `dealloc`, `retainCount` — <https://developer.apple.com/documentation/objectivec/nsobject-swift.class>
- `NSAutoreleasePool` и директива `@autoreleasepool` — <https://developer.apple.com/documentation/foundation/nsautoreleasepool>

Глава 14. Среда выполнения и иерархия классов

Эта глава — сердце книги. До сих пор мы пользовались объектами как готовыми: создаём, шлём сообщения, наследуем. Но что такое объект на самом деле? Что такое класс? Куда уходит сообщение `[obj foo]` в момент выполнения и как оно находит нужный код?

Ответ даёт **среда выполнения** Objective-C (Objective-C runtime) — небольшая библиотека `libobjc`, которая живёт рядом с каждой твоей программой и всё время работает: хранит классы, ищет методы, пересылает сообщения. В Си ничего подобного нет — там после компиляции от типов не остаётся и следа. В Objective-C классы — это **живые объекты в памяти**, у которых во время выполнения можно спросить имя, суперкласс, список методов и даже дорастить им новый метод на лету.

Мы пойдём снизу вверх: от факта «объект знает свой класс» — к тому, что класс это тоже объект, к метаклассам, к полной схеме указателей, к тому, как `objc_msgSend` ищет метод, и закончим тремя механизмами, которые делают язык по-настоящему динамическим: динамическим разрешением, пересылкой сообщений и подменой методов (swizzling).

Что мы сделаем

- Увидим через runtime, что объект — это структура, чьё первое поле `isa` указывает на класс.
- Поймём, что **класс — тоже объект**, и что его `isa` указывает на **метакласс**.
- Нарисуем и разберём классическую схему указателей `isa/superclass`.
- Разберём два корня иерархии: `NSObject` и `NSProxy`.
- Проследим, как `objc_msgSend` ищет реализацию по селектору.
- Напишем живой дамп иерархии и списка методов через функции runtime.
- Реализуем динамическое разрешение метода (`+resolveInstanceMethod:`).
- Сделаем рабочий объект-прокси через `-forwardingTargetForSelector:`.
- Подменим реализацию метода через `method_exchangeImplementations`.

- Увидим настоящее падение «unrecognized selector».

Весь runtime живёт в двух заголовках. Их мы и подключаем поверх Foundation:

```
#import <Foundation/Foundation.h>
#import <objc/runtime.h>      /* object_getClass, class_getName, ... */
#import <objc/message.h>     /* objc_msgSend и компания */
```

<objc/runtime.h> — это набор Си-функций для работы с классами, методами, селекторами и полями. <objc/message.h> — объявления самих функций отправки сообщения. Дальше по тексту, когда функция нужна, я буду указывать, из какого заголовка она пришла.

Шаг 1. Объект знает свой класс

Вспомни из главы 3, что такое **структура** (struct): несколько полей, лежащих в памяти подряд. Так вот, объект Objective-C — это обычная си-структура, у которой **самое первое поле** называется isa и хранит указатель на класс этого объекта. Само имя «isa» читается как «is a» — «является»: rex is a Dog, rex является собакой.

Упрощённо объект в памяти выглядит так:

```
объект rex в памяти:
+-----+      <- адрес объекта (то, что лежит в Dog *)
| isa  -> указатель на Class |  первое поле, размер указателя (8 байт)
+-----+
| ... поля экземпляра (ivar) |  дальше идут _name, _age и т.п.
+-----+
```

Раз isa — это просто поле, его можно прочесть. Высокоуровневое сообщение [rex class] (см. главу 9) и низкоуровневая функция runtime object_getClass(rex) обычно дают одно и то же: класс из поля isa. Проверим, что это буквально один и тот же указатель.

Оговорка на будущее. «Первое поле — указатель на класс» — модель, которой учит документация Apple, и думать о isa так удобно. В современном 64-битном runtime в это слово памяти заодно упакованы служебные биты (например, часть счётчика ссылок), поэтому читать его руками бессмысленно: класс из него достаёт object_getClass. По нашему эксперименту на Intel-маке сырое значение первого слова rex было 0x11d800104d63261, а адрес класса — 0x104d63260: младшие разряды совпадают, остальное — служебные биты.

```
#import <Foundation/Foundation.h>
#import <objc/runtime.h>

@interface Dog : NSObject
```

```

@end

@implementation Dog
@end

int main(void) {
    @autoreleasepool {
        Dog *rex = [[Dog alloc] init];

        Class c1 = [rex class];           /* через сообщение */
        Class c2 = object_getClass(rex);  /* читаем isa напрямую */

        NSLog(@"[rex class]          = %s", class_getName(c1));
        NSLog(@"object_getClass(rex) = %s", class_getName(c2));
        NSLog(@"это один и тот же класс? %@",
              (c1 == c2) ? @"да" : @"нет");

        NSLog(@"адрес класса Dog    = %p", (__bridge void *)c1);

        Class super = class_getSuperclass(c1);
        NSLog(@"суперкласс Dog      = %s", class_getName(super));

        Dog *barsik = [[Dog alloc] init];
        NSLog(@"isa у rex и barsik совпадает? %@",
              (object_getClass(rex) == object_getClass(barsik))
              ? @"да" : @"нет");
    }
    return 0;
}

```

Разберём новые элементы.

- `Class` — это тип «класс». Под капотом это указатель на структуру класса (`struct objc_class *`). Переменная типа `Class` хранит адрес класса, ровно как `Dog *` хранит адрес объекта.
- `object_getClass(rex)` — функция из `<objc/runtime.h>`. Принимает любой объект, возвращает его `isa`, то есть класс. Это и есть «прочитать первое поле структуры» — с учётом упакованных битов из оговорки выше.
- `class_getName(c)` — принимает `Class`, возвращает **си-строку** (`const char *`) именем класса. Поэтому печатаем через `%s`, а не `%@`: это обычный массив байтов, а не `NSString`.

- `(__bridge void *)c1` — приведение указателя-класса к «сырому» указателю, чтобы напечатать его адрес через `%p`. Слово `__bridge` нужно из-за ARC: оно говорит «просто перетолкуй указатель, владение не меняется». Без `__bridge` под ARC компилятор откажется приводить объектный указатель к `void *`.
- `class_getSuperclass(c)` — возвращает класс-родитель. Для `Dog` это `NSObject`.

Компилируем и запускаем:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/14-isa.m -o t && ./t
```

Здесь и дальше в главе префикс `NSLog` (дата, время, имя программы) в выводах опущен, а адреса у тебя будут свои:

```
[rex class]          = Dog
object_getClass(rex) = Dog
это один и тот же класс? да
адрес класса Dog     = 0x10dbc80b8
суперкласс Dog       = NSObject
isa у rex и barsik совпадает? да
```

Два главных факта отсюда:

1. Класс `Dog` существует как **значение** — у него есть адрес в памяти (`0x10dbc80b8`). Это не абстракция компилятора, а реальный объект.
2. Все экземпляры одного класса делят **один** указатель `isa`. У `rex` и `barsik` он совпадает: класс на всех один, в каждый объект просто вложен адрес этого единственного класса.

Отличие от Си. В Си у структуры нет никакого «знания о себе». Если ты объявил `struct Dog`, то после компиляции от имени `Dog` не останется ничего — в исполняемом файле это просто блок байтов нужного размера. Спросить у си-структуры «как тебя зовут?» невозможно: спрашивать некого. В Objective-C первое поле объекта — `isa` — постоянно держит связь с живым классом, и потому объект всегда знает, кто он.

Шаг 2. Класс — это тоже объект

Раз у класса есть адрес и поля, логично спросить: а у класса самого есть `isa`? Есть. **Класс — это объект**, и его первое поле тоже `isa`.

Зачем это нужно. Подумай про методы. Методы экземпляра (с минусом, `-`) хранятся в классе: когда ты шлешь сообщение объекту, runtime идёт в его класс и ищет метод там. А где тогда хранятся методы класса (с плюсом, `+`) — `alloc`, `new`, твои фабричные методы? Их нужно где-то

держат по той же схеме. Решение элегантное: у каждого класса есть свой невидимый «класс класса» — **метакласс** (metaclass). Методы экземпляра лежат в классе; методы класса лежат в метаклассе. Одна и та же машина поиска работает на обоих уровнях.

объект	---isa-->	класс	---isa-->	метакласс
(rex)		(Dog)		(Dog meta)
		хранит -методы		хранит +методы

isa объекта — это его класс. isa класса — это его метакласс. Прочитать метакласс можно той же функцией `object_getClass`, передав ей не объект, а класс (ведь класс тоже объект):

```
Class dogClass = object_getClass(rex);      /* Dog      */
Class dogMeta  = object_getClass(dogClass); /* Dog (meta) */
```

Метакласс невидим в коде — ты не пишешь его имя, не создаёшь его руками. Компилятор и runtime заводят его автоматически для каждого класса. Отличить класс от метакласса помогает `class_isMetaClass(c)` — вернёт YES, если это метакласс. Вот программа, которая проходит по цепочке isa и печатает её:

```
#import <Foundation/Foundation.h>
#import <objc/runtime.h>

@interface Animal : NSObject
@end
@implementation Animal
@end

@interface Dog : Animal
@end
@implementation Dog
@end

static void describe(Class c) {
    NSString *kind = class_isMetaClass(c) ? @"метакласс" : @"класс";
    Class super = class_getSuperclass(c);
    NSLog(@"%@ %-18s isa=%p super=%s",
          kind,
          class_getName(c),
          (__bridge void *)object_getClass(c),
          super ? class_getName(super) : "(none)");
}
```

```
int main(void) {
    @autoreleasepool {
        Dog *rex = [[Dog alloc] init];

        Class dogClass = object_getClass(rex);          /* Dog          */
        Class dogMeta  = object_getClass(dogClass);      /* Dog (meta) */

        NSLog(@"--- классы ---");
        describe(dogClass);
        describe([Animal class]);
        describe([NSObject class]);

        NSLog(@"--- метаклассы ---");
        describe(dogMeta);
        describe(object_getClass([Animal class]));
        describe(object_getClass([NSObject class]));

        Class rootMeta = object_getClass([NSObject class]);
        NSLog(@"--- замыкание ---");
        NSLog(@"isa корневого метакласса == он сам? %@",
              (object_getClass(rootMeta) == rootMeta) ? @"да" : @"нет");
        NSLog(@"superclass корневого метакласса = %s",
              class_getName(class_getSuperclass(rootMeta)));

        NSLog(@"Dog          – метакласс? %@",
              class_isMetaClass(dogClass) ? @"да" : @"нет");
        NSLog(@"Dog(meta) – метакласс? %@",
              class_isMetaClass(dogMeta)  ? @"да" : @"нет");
    }
    return 0;
}
```

Пара деталей синтаксиса печати:

- `%-18s` в строке формата — это «напечатай си-строку, выровняв по левому краю в поле шириной 18 символов». Минус означает выравнивание влево. Так колонки в выводе становятся ровными.

- `describe` берёт `Class` и печатает его вид, имя, `isa` (адрес его метакласса) и суперкласс. Заметь: для класса `isa` — это адрес метакласса, и мы это сейчас увидим в числах.

Запускаем:

```
--- классы ---
класс      Dog            isa=0x10e24a170  super=Animal
класс      Animal        isa=0x10e24a120  super=NSObject
класс      NSObject       isa=0x7ff84d568530  super=(none)
--- метаклассы ---
метакласс  Dog            isa=0x7ff84d568530  super=Animal
метакласс  Animal        isa=0x7ff84d568530  super=NSObject
метакласс  NSObject       isa=0x7ff84d568530  super=NSObject
--- замыкание ---
isa корневого метакласса == он сам? да
superclass корневого метакласса = NSObject
Dog          — метакласс? нет
Dog(meta) — метакласс? да
```

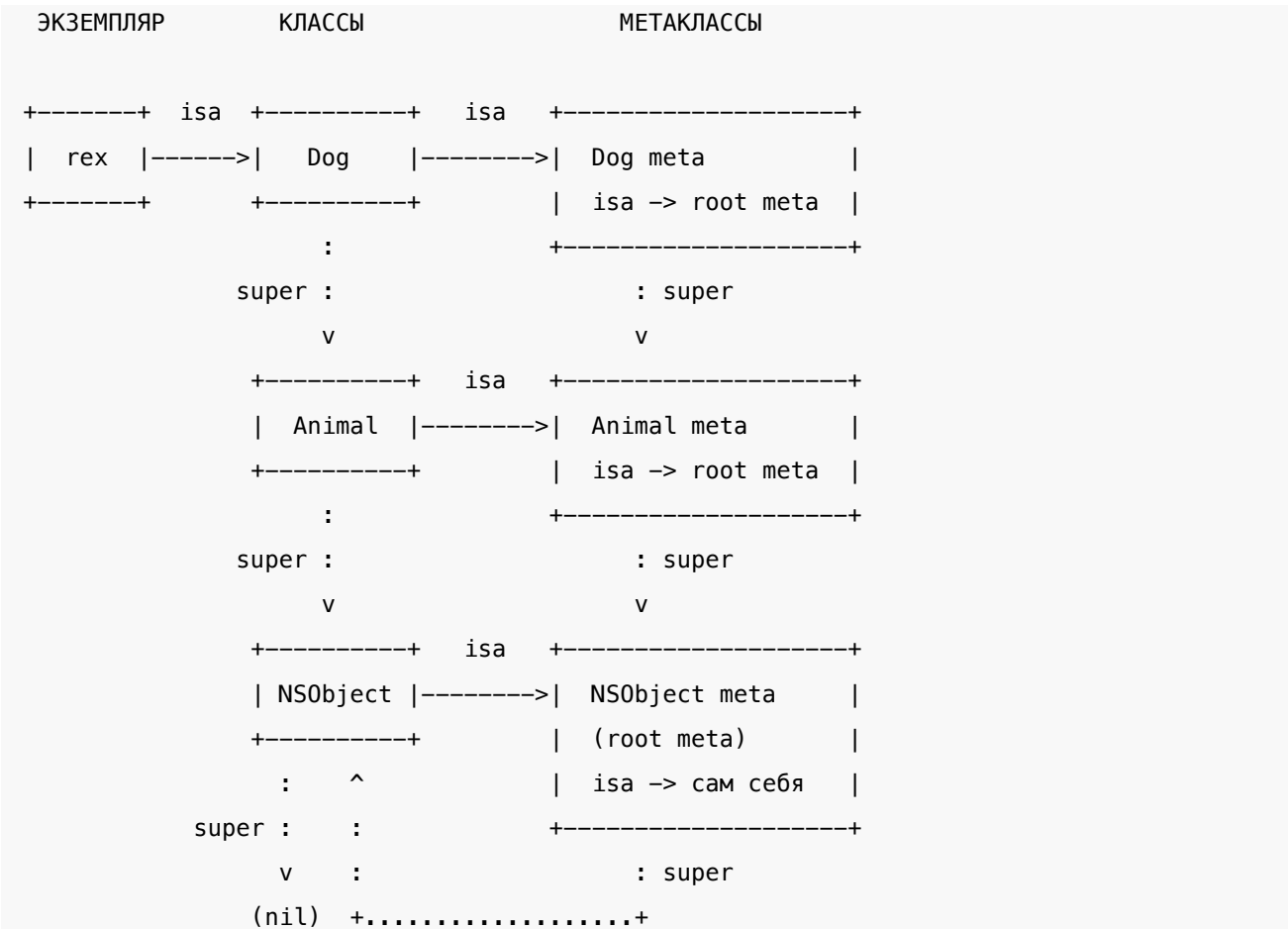
Вчитайся в адреса — в них вся картина:

- `isa` класса `Dog` = ...170, а `isa` класса `Animal` = ...120. Это адреса их метаклассов: у каждого класса свой метакласс.
- `isa` **всех** метаклассов одинаков: 0x7ff84d568530. Это адрес **корневого метакласса** — метакласса `NSObject`. А сам метакласс `NSObject` своим `isa` указывает на себя же (`isa` корневого метакласса == он сам? да). Цепочка `isa` замкнулась.
- `superclass` метакласса `Dog` = `Animal`. `Runtime` тут печатает имя через `class_getSuperclass`, и для метакласса это метакласс родителя — но имя у метакласса то же, что у класса, поэтому в выводе видно `Animal`. Идея: иерархия метаклассов повторяет иерархию классов.
- `superclass` корневого метакласса (метакласса `NSObject`) — это сам **класс** `NSObject`. Так метаклассовая ветка «впадает» обратно в обычную, и поиск метода класса в самом конце доходит до методов экземпляра `NSObject`.

Отличие от Си. В Си тип — это инструкция компилятору на этапе сборки, и только. У него нет ни адреса, ни «класса своего типа», ни возможности спросить о себе что-либо во время работы программы. Здесь же класс — полноценный объект, у которого есть свой класс (метакласс), а у того — свой (корневой метакласс). Целая живая структура там, где в Си пустота.

Шаг 3. Полная схема указателей

Соберём всё в одну диаграмму — ту самую классическую картинку, которую рисуют в документации Apple. Возьмём нашу тройку `Dog : Animal : NSObject` и один объект `rex` класса `Dog`. Сплошные стрелки `isa`, пунктирные `superclass`.



Стрелки `isa` из метаклассов в корневой метакласс вписаны прямо в прямоугольники, чтобы схема не превратилась в паутину. Нижняя пунктирная линия — `superclass` корневого метакласса: он ведёт обратно в **класс** `NSObject`.

Прочитаем по стрелкам, медленно — это карта всего runtime:

- `rex.isa -> Dog`. Объект знает свой класс.
- `Dog.isa -> Dog meta`. Класс знает свой метакласс.
- `Dog.superclass -> Animal`, `Animal.superclass -> NSObject`, `NSObject.superclass -> nil`. Это обычное дерево наследования; у корня родителя нет.
- `Dog meta.isa -> root metaclass`, `Animal meta.isa -> root metaclass`, `NSObject meta.isa -> сам NSObject meta`. Все метаклассы своим `isa` показывают на корневой метакласс, а тот — на себя. Цепочка `isa` конечна и замкнута.

- `Dog meta.superclass -> Animal meta, Animal meta.superclass -> NSObject meta`. Иерархия метаклассов параллельна иерархии классов.
- `NSObject meta.superclass -> NSObject (класс)`. Самый красивый узел: суперкласс корневого метакласса — это обычный класс `NSObject`.

Зачем последняя стрелка. Когда ты шлешь сообщение **класса** (`[Dog new]`), runtime ищет +-метод по метаклассовой ветке вверх: `Dog meta -> Animal meta -> NSObject meta`. Если не нашёл и там — поиск переходит в класс `NSObject` и продолжается по его --методам. Проверить мостик просто: добавь категорией (глава 11) метод экземпляра — `(void)hi` в `NSObject` и пошли его **классу**: `[Dog hi]`. Метода класса `+hi` нет нигде, но вызов сработает — поиск через мостик найдёт `-hi` у класса `NSObject`, а `self` внутри будет классом `Dog`.

Запомни короткую формулу: **isa отвечает на вопрос «какого я вида»** (объект -> класс -> метакласс), **superclass отвечает на вопрос «от кого я унаследован»** (вверх по дереву до `nil`).

Шаг 4. Два корня дерева: NSObject и NSProxy

Почти всё в Foundation и UIKit растёт из одного корня — класса **NSObject**. Он даёт базовое поведение: `alloc`, `init`, подсчёт ссылок, `isKindOfClass:`, `respondsToSelector:`, `description`, участие в пересылке сообщений. Когда пишешь `@interface Dog : NSObject`, ты встаёшь в это дерево.

Но корень не один. Есть второй, отдельный — **NSProxy**. Это тоже абстрактный базовый класс, но «тонкий»: он создан специально для объектов-заместителей, которые не делают работу сами, а пересылают почти все сообщения куда-то ещё. `NSProxy` реализует совсем мало и заставляет наследника определить пересылку. На нём строят, например, удалённые прокси и ленивую загрузку.

корни иерархии классов Objective-C

`NSObject` (основной корень — почти всё дерево)

|

+-- `NSString` --- `NSMutableString`

+-- `NSArray` --- `NSMutableArray`

+-- `NSNumber`

+-- `NSDictionary`

+-- ... сотни классов Foundation/AppKit/UIKit ...

+-- твои классы (`Dog`, `Animal`, ...)

`NSProxy` (отдельный корень — заместители/прокси)

```
|
+-- твои прокси-классы
```

Оба класса реализуют корневой протокол `NSObject` (одноимённый протокол, не путать с классом) — поэтому у объектов обоих деревьев есть `class`, `isKindOfClass:`, `respondToSelector:` и так далее. Просто `NSObject` тащит за собой ещё гору готового поведения, а `NSProxy` оставляет тебя наедине с пересылкой. В быту ты почти всегда наследуешь `NSObject`; о `NSProxy` достаточно знать, что второй корень существует.

Шаг 5. Как сообщение находит метод: `objc_msgSend`

Теперь — главный механизм. Когда ты пишешь

```
[rex speak];
```

компилятор не вставляет прямой переход к коду метода. Он превращает эту строку в **вызов функции отправки сообщения**:

```
objc_msgSend(rex, @selector(speak));
```

Разберём участников.

- **`objc_msgSend`** — функция из `<objc/message.h>`. Её первый аргумент — получатель (*receiver*), второй — селектор, дальше идут аргументы метода. Она находит реализацию и передаёт ей управление.
- **SEL (селектор)** — это **уникализированное имя сообщения**. Не строка, а её зарегистрированный в runtime «номерок»: для имени "speak" во всей программе существует ровно один SEL. Поэтому селекторы можно сравнивать через `==`, а не посимвольно. Получить селектор из имени: `@selector(speak)` на этапе компиляции или `sel_registerName("speak")` в рантайме. Обратно имя: `sel_getName(sel)`.
- **IMP (реализация, implementation)** — это **указатель на си-функцию**, которая и есть тело метода. У всякой IMP первые два скрытых аргумента — `self` (получатель) и `_cmd` (селектор), а дальше параметры метода.
- **Метод (Method)** — это **пара (SEL, IMP)**: имя сообщения и адрес кода, который на него отвечает (плюс строка с кодировкой типов, о ней в шаге 6). Класс хранит таблицу таких методов.

Алгоритм `objc_msgSend` по шагам:

```
objc_msgSend(receiver, selector, аргументы...)
```

```
1. receiver == nil? -> ничего не делаем, возвращаем 0/nil. (см. главу 5)
```

```

2. cls = класс из receiver->isa
3. смотрим в КЭШ cls: этот селектор уже вызывали?
   да -> сразу прыгаем на запомненную IMP (самый частый случай)
4. ищем selector в таблице методов cls
   нашли -> кладём (selector -> IMP) в кэш класса получателя
   и прыгаем на IMP, передав self, _cmd, аргументы
   не нашли -> cls = cls->superclass, повторяем шаг 4
5. дошли до конца (superclass == nil, выше NSObject)?
   -> динамическое разрешение и ПЕРЕСЫЛКА (шаги 7–9 ниже)

```

Картинка поиска для нашей тройки, если у rex зовём speak, а метод определён только в Animal:

```

[rex speak]
|
v isa
Dog ---- нет speak? --+
| superclass          \
v                      (поиск идёт вверх)
Animal -- ЕСТЬ speak! --> прыгаем на его IMP, выполняем
|
v (если бы не нашли — дальше)
NSObject -- нет --> superclass == nil --> ПЕРЕСЫЛКА

```

Отличие от Си. В Си `foo(rex, 5)` — это переход по **фиксированному адресу**: какой именно `foo` вызвать, решает компоновщик на этапе сборки, и в готовой программе там стоит конкретный адрес. В Objective-C `[rex foo:5]` — это `objc_msgSend(rex, @selector(foo:), 5)`: реальный адрес кода находится **во время выполнения**, проходом по дереву классов от `isa` получателя вверх по `superclass`. Один и тот же `[obj speak]` может выполнить разный код в зависимости от того, какой объект лежит в `obj` прямо сейчас. Этого Си не умеет в принципе.

Строка `objc_msgSend(rex, @selector(speak))` выше — это то, во что компилятор превращает скобки; сама по себе в таком виде она не соберётся. Можно ли вызвать `objc_msgSend` руками? Да, но осторожно: в SDK она объявлена как `void objc_msgSend(void)`, и прямой вызов с аргументами компилятор отвергает ошибкой `too many arguments to function call, expected 0, have 2`. Звать её можно, только приведя к точной сигнатуре метода. Поэтому в обычном коде мы её **не** дёргаем напрямую — за нас это делают квадратные скобки. А для интроспекции и динамики пользуемся высокоуровневыми функциями `runtime`, к которым и переходим.

Шаг 6. Живая интроспекция: дамп иерархии и методов

Раз класс — объект в памяти, спросим у него всё: имя, цепочку суперклассов, список методов, список полей. Соберём маленький «рентген» класса.

```
#import <Foundation/Foundation.h>
#import <objc/runtime.h>

@interface Vehicle : NSObject {
    int _wheels;          /* ivar базового класса */
}
- (void)move;
- (int)wheels;
@end

@implementation Vehicle
- (void)move { NSLog(@"еду"); }
- (int)wheels { return _wheels; }
@end

@interface Car : Vehicle {
    NSString *_brand;     /* ivar потомка */
}
- (void)honk;
- (void)refuel:(int)liters;
@end

@implementation Car
- (void)honk          { NSLog(@"би-бип"); }
- (void)refuel:(int)l { NSLog(@"залил %d л", l); }
@end

static void dumpMethods(Class c) {
    unsigned int count = 0;
    Method *methods = class_copyMethodList(c, &count);
    NSLog(@"методы класса %s (%u шт.):", class_getName(c), count);
    for (unsigned int i = 0; i < count; i++) {
        SEL name = method_getName(methods[i]);
        NSLog(@"    - %s (аргументов: %u)",
```

```
        sel_getName(name),
        method_getNumberOfArguments(methods[i]));
    }
    free(methods);          /* список выделен malloc – освобождаем сами */
}

static void dumpIvars(Class c) {
    unsigned int count = 0;
    Ivar *ivars = class_copyIvarList(c, &count);
    NSLog(@"поля класса %s (%u шт.):", class_getName(c), count);
    for (unsigned int i = 0; i < count; i++) {
        NSLog(@"    - %s : %s",
              ivar_getName(ivars[i]),
              ivar_getTypeEncoding(ivars[i]));
    }
    free(ivars);
}

static void dumpHierarchy(Class c) {
    NSLog(@"иерархия:");
    NSMutableString *line = [NSMutableString string];
    while (c) {
        [line appendString:[NSString stringWithUTF8String:class_getName(c)]];
        c = class_getSuperclass(c);
        if (c) [line appendString:@" -> "];
    }
    NSLog(@"    %@", line);
}

int main(void) {
    @autoreleasepool {
        Car *car = [[Car alloc] init];
        Class c = object_getClass(car);

        dumpHierarchy(c);
        dumpMethods(c);
        dumpIvars(c);
        dumpMethods([Vehicle class]);
    }
}
```

```

    NSLog(@"Car отвечает на move (от родителя)? %@",
          class_respondsToSelector(c, @selector(move)) ? @"да" : @"нет");
    NSLog(@"Car отвечает на honk? %@",
          class_respondsToSelector(c, @selector(honk)) ? @"да" : @"нет");
    NSLog(@"Car отвечает на fly? %@",
          class_respondsToSelector(c, @selector(fly)) ? @"да" : @"нет");
}
return 0;
}

```

Новые функции runtime, по порядку:

- `class_copyMethodList(c, &count)` — возвращает **массив** методов, объявленных **именно в этом классе** (унаследованные не входят — они лежат в суперклассах). В `count` кладёт длину. Имя функции содержит `copy`: память под массив выделяется через `malloc`, и освободить её обязан ты сам через `free`. Это си-уровень, ARC сюда не дотягивается.
- `method_getName(m)` — селектор метода. `method_getNumberOfArguments(m)` — сколько у него аргументов, **считая** скрытые `self` и `_cmd`. Поэтому у метода без видимых параметров их два, а у `refuel`: — три.
- `class_copyIvarList(c, &count)` — массив полей (`ivar`) класса. Тоже требует `free`.
- `ivar_getName(ivar)` — имя поля, `ivar_getTypeEncoding(ivar)` — закодированный тип. Кодировка типов — это маленький «алфавит» runtime: `i` это `int`, `@` это объект, `@NSString` — объект конкретного класса. Тот же язык кодировки мы используем в `class_addMethod` (шаг 7).
- `class_respondsToSelector(c, sel)` — умеет ли **экземпляр** класса отвечать на сообщение, **с учётом наследования** (в отличие от `class_copyMethodList`, который смотрит только свой класс).
- `[NSString stringWithUTF8String:...]` — превращает си-строку (`const char *` от `class_getName`) в `NSString`, чтобы склеивать иерархию в одну читаемую строку.

Запускаем:

иерархия:

```
Car -> Vehicle -> NSObject
```

методы класса Car (3 шт.):

```
- refuel:    (аргументов: 3)
- honk      (аргументов: 2)
- .cxx_destruct  (аргументов: 2)
```

поля класса Car (1 шт.):

```
- _brand : @"NSString"
```

методы класса Vehicle (2 шт.):

```

- wheels    (аргументов: 2)
- move      (аргументов: 2)

```

Car отвечает на move (от родителя)? да

Car отвечает на honk? да

Car отвечает на fly? нет

Смотри, что мы вытащили из работающей программы:

- Полную цепочку наследования Car → Vehicle → NSObject — просто шагая по superclass.
- Список методов каждого класса по отдельности. Методы родителя move и wheels в списке Car не появились — они в Vehicle. Это прямое подтверждение того, что метод живёт ровно в том классе, где определён, а поиск при вызове идёт вверх.
- Загадочный .cxx_destruct в Car. Его добавил **ARC**: это автоматический «деструктор», который освобождает объектные поля (наш _brand) при уничтожении объекта. Ты его не писал — здесь это работа ARC, и runtime честно показывает его в списке. (Если у класса есть поля C++-типов, компилятор заводит этот метод и без ARC — чтобы вызвать их деструкторы; отсюда и cxx в имени.) Метка «по нашему эксперименту»: Apple не обещает имя .cxx_destruct, мы просто наблюдаем его в выводе.
- Поле _brand с кодировкой типа @"NSString".
- class_respondsToSelector учёл наследование: move достался Car от Vehicle — «да»; fly не существует нигде — «нет».

Отличие от Си. Это и есть та пропасть между языками. В Си нельзя во время выполнения спросить у типа список его функций или полей — типа в готовой программе уже нет, как нет и «списка функций структуры»: функции вообще не привязаны к структурам. В Objective-C класс — живой объект, и весь его «паспорт» — имя, родители, методы, поля — доступен прямо во время работы программы.

Шаг 7. Динамическое разрешение метода

Пойдём ещё дальше. Метод можно не просто *прочитать*, но и *добавить* классу прямо во время выполнения. Самый аккуратный момент для этого — когда runtime ищет метод, не находит, и **спрашивает сам класс**: «может, доопределишь?» Для методов экземпляра это вызов +resolveInstanceMethod: (для методов класса есть парный +resolveClassMethod:).

Сделаем класс Robot, который не реализует метод greet, но в момент первого обращения допустит его себе через class_addMethod.

```

#import <Foundation/Foundation.h>
#import <objc/runtime.h>

```

```
/* Обычная Си-функция. Её и подставим как реализацию метода.
   Первые два скрытых аргумента любой IMP – self и _cmd. */
static void dynamicGreet(id self, SEL _cmd) {
    NSLog(@" привет от %@, селектор был :%s",
          [self class], sel_getName(_cmd));
}

@interface Robot : NSObject
@end

@implementation Robot

+ (BOOL)resolveInstanceMethod:(SEL)sel {
    if (sel == @selector(greet)) {
        /* class_addMethod(класс, селектор, функция, кодировка-типов).
           "v@" = void-результат(v), self(@), _cmd(:). */
        class_addMethod(self, sel, (IMP)dynamicGreet, "v@:");
        NSLog(@" [resolve] добавили реализацию для %s", sel_getName(sel));
        return YES;
    }
    return [super resolveInstanceMethod:sel];
}

@end

int main(void) {
    @autoreleasepool {
        Robot *r = [[Robot alloc] init];

        NSLog(@"первый вызов greet:");
        [r performSelector:@selector(greet)]; /* запустит resolve */

        NSLog(@"второй вызов greet:");
        [r performSelector:@selector(greet)]; /* resolve уже НЕ нужен */
    }
    return 0;
}
```


Разбор по частям:

- `dynamicGreet` — обычная Си-функция, **не** метод. Но у неё та самая сигнатура IMP: первый параметр `id self`, второй `SEL _cmd`. Это и есть скрытые аргументы, которые каждому методу передаёт `objc_msgSend`. Внутри можно пользоваться `self`, как в обычном методе.
- `+resolveInstanceMethod:(SEL)sel` — метод **класса** из `NSObject`, который runtime зовёт, не найдя реализацию `sel` обычным поиском. Мы его переопределяем. Если узнаём селектор `greet`, добавляем реализацию и возвращаем `YES` — это сигнал runtime: «я доопределил, повтори поиск». Для всех остальных селекторов вызываем `super`, чтобы не сломать поведение по умолчанию.
- `class_addMethod(self, sel, (IMP)dynamicGreet, "v@:")` — главная функция. Внутри `+resolveInstanceMethod:` параметр `self` — это сам класс `Robot`. Мы добавляем ему метод: имя `sel`, код — наша функция, приведённая к IMP. Последний аргумент — **кодировка типов**: `"v@:"` читается как «возвращает `void` (`v`), принимает `self` — объект (`@`) и `_cmd` — селектор (`:`)». Та же кодировка, что мы видели в дампе полей.
- `[r performSelector:@selector(greet)]` — отправляем сообщение `greet`. Метода в классе нет, поэтому запустится `+resolveInstanceMethod:`. Мы пользуемся `performSelector:`, а не `[r greet]`, потому что метод `greet` нигде не объявлен — компилятор о нём не знает, а `performSelector:` отправляет сообщение по селектору без объявления.

Запускаем:

```
первый вызов greet:
[resolve] добавили реализацию для greet
привет от Robot, селектор был :greet
второй вызов greet:
привет от Robot, селектор был :greet
```

`[resolve]` напечаталось **ровно один раз** — при первом вызове. Дальше метод уже встроен в класс, и второй вызов идёт напрямую, минуя разрешение. Класс буквально вырос на один метод во время работы программы.

Отличие от Си. В Си набор функций фиксируется при компоновке: какие функции есть в программе, то и есть, новую во время выполнения «дорастить» структуре нельзя. Здесь же `class_addMethod` добавляет классу метод на ходу, и со следующего сообщения объекты уже умеют то, чего минуту назад не умели.

Шаг 8. Пересылка сообщения: три шанса

Что, если объект так и не нашёл метод и не доопределил его в `resolveInstanceMethod:`? Сообщение не пропадает сразу. У runtime есть **три последовательных шанса** обработать «непонятное» сообщение, прежде чем сдаться и уронить программу:

```
[obj foo] -> поиск в классе и вверх по superclass -> НЕ НАЙДЕНО
|
ШАНС 1: +resolveInstanceMethod:(foo)
        можно добавить метод на лету (class_addMethod)
        | не разрешили
ШАНС 2: -forwardingTargetForSelector:(foo)
        вернуть ДРУГОЙ объект, который умеет foo
        -> runtime повторяет отправку ему (быстро)
        | вернули nil/self
ШАНС 3: -methodSignatureForSelector:(foo) + -forwardInvocation:
        полный объект NSInvocation: можно изменить, залогировать,
        переслать нескольким адресатам, вернуть значение
        | не обработали
-> -doesNotRecognizeSelector: -> ИСКЛЮЧЕНИЕ, падение
```

Первый шанс мы уже разобрали. Самый практичный — **второй**: `-forwardingTargetForSelector:`. Ты возвращаешь другой объект, и runtime просто повторяет отправку уже ему — без копирования аргументов, дёшево. На этом строят **прокси**: объект, который сам почти ничего не умеет, а все сообщения переадресует «настоящему» работнику.

```
#import <Foundation/Foundation.h>
#import <objc/runtime.h>

@interface Worker : NSObject
- (NSString *)greet:(NSString *)name;
- (int)square:(int)x;
@end

@implementation Worker
- (NSString *)greet:(NSString *)name {
    return [NSString stringWithFormat:@"Worker приветствует %@", name];
}
- (int)square:(int)x { return x * x; }
@end
```

```
@interface Proxy : NSObject
- (instancetype)initWithTarget:(id)target;
@end

@implementation Proxy {
    id _target;    /* кому пересылаем */
}

- (instancetype)initWithTarget:(id)target {
    self = [super init];
    if (self) { _target = target; }
    return self;
}

- (id)forwardingTargetForSelector:(SEL)sel {
    NSLog(@" [forward] %s -> %@", sel_getName(sel), [_target class]);
    if ([_target respondsToSelector:sel]) {
        return _target;
    }
    return [super forwardingTargetForSelector:sel];
}

@end

int main(void) {
    @autoreleasepool {
        Worker *worker = [[Worker alloc] init];

        /* Тип переменной – id, поэтому компилятор разрешает слать
           проху любые объявленные селекторы. */
        id proxy = [[Proxy alloc] initWithTarget:worker];

        NSString *hi = [proxy greet:@"Серик"];
        NSLog(@"ответ: %@", hi);

        int sq = [proxy square:9];
        NSLog(@"9 в квадрате = %d", sq);
    }
}
```

```

        NSLog(@"proxy сам умеет greet:? %@",
              [proxy respondsToSelector:@selector(greet:)] ? @"да" : @"нет");
    }
    return 0;
}

```

Ключевые места:

- Proxy наследует NSObject и **не реализует** ни `greet:`, ни `square:`. Внутри хранит `_target` — настоящего работника.
- `-forwardingTargetForSelector:(SEL)sel` — метод из NSObject. runtime зовёт его для каждого нераспознанного сообщения. Возвращаем `_target`, если тот умеет `sel`; иначе передаём дальше в `super` (там сообщение пойдёт на третий шанс и, скорее всего, упадёт).
- `id proxy = ...` — мы намеренно объявили переменную как `id`, а не `Proxy *`. Под `id` компилятор разрешает слать любые **объявленные где угодно** селекторы (`greet:`, `square:` объявлены в `Worker`) и не ругается, что Proxy их не знает. Это та самая динамика главы 5.
- `[proxy greet:@"Серик"]` возвращает NSString, `[proxy square:9]` — int. Пересылка сохраняет и аргументы, и тип возврата: для `forwardingTargetForSelector:` это бесплатно, ведь сообщение буквально повторяется другому получателю.

Запускаем:

```

[forward] greet: -> Worker
ответ: Worker приветствует Серик
[forward] square: -> Worker
9 в квадрате = 81
proxy сам умеет greet:? нет

```

Снаружи proxy ведёт себя как Worker: ответил на `greet:` и `square:`. А последняя строка честно говорит «proxy сам не умеет `greet:`» — `respondsToSelector:` смотрит на реальные методы класса Proxy, а пересылка работает уровнем ниже, уже после того, как обычный поиск провалился.

Про **третий шанс** — `-forwardInvocation:` — достаточно знать идею. Если переопределить пару `-methodSignatureForSelector:` (вернуть описание сигнатуры метода) и `-forwardInvocation:(NSInvocation *)inv`, то runtime упакует всё сообщение целиком в объект `NSInvocation`: получатель, селектор, все аргументы. С ним можно делать что угодно — изменить аргументы, переслать **нескольким** адресатам, записать в лог, отложить. Это мощнее `forwardingTargetForSelector:`, но и дороже: построение `NSInvocation` не бесплатно. Эскиз метода (фрагмент, не целая программа):

```

- (void)forwardInvocation:(NSInvocation *)inv {
    if ([_target respondsToSelector:[inv selector]]) {
        [inv invokeWithTarget:_target];    /* выполнить на другом объекте */
    } else {
        [super forwardInvocation:inv];
    }
}
}

```

Шаг 9. doesNotRecognizeSelector: и падение

Если ни один из трёх шансов не сработал, runtime зовёт у объекта `-doesNotRecognizeSelector:`, а тот бросает исключение `NSInvalidArgumentException`. Это и есть знаменитое падение, которое рано или поздно видит каждый. Спровоцируем его намеренно:

```

#import <Foundation/Foundation.h>

@interface Cat : NSObject
@end

@implementation Cat
@end

int main(void) {
    @autoreleasepool {
        Cat *c = [[Cat alloc] init];
        [c performSelector:@selector(fly)];    /* Cat не умеет fly */
    }
    return 0;
}

```

Запускаем и видим аварийное завершение:

```

*** Terminating app due to uncaught exception 'NSInvalidArgumentException',
reason: '-[Cat fly]: unrecognized selector sent to instance 0x600003ad8070'
*** First throw call stack:
(
    0   CoreFoundation    __exceptionPreprocess + 241
    1   libobjc.A.dylib    objc_exception_throw + 62
    ...

```

```
)
libc++abi: terminating due to uncaught exception of type NSException
```

(Первые две строки в терминале — одна длинная строка; колонку адресов в стеке мы убрали.)

Расшифруй сообщение — оно очень информативно:

- `-[Cat fly]` — минус говорит, что искали **метод экземпляра** `fly` у класса `Cat` (для метода класса был бы плюс: `+[Cat fly]`).
- `unrecognized selector sent to instance 0x...` — селектор не нашёлся и ни один шанс пересылки его не подобрал; в конце адрес самого объекта.
- `objc_exception_throw` в стеке — это runtime бросил исключение.

Встречая такое падение, читай первую строку: класс и селектор почти всегда сразу показывают, кто кому послал что-то, чего тот не умеет. Частая причина — опечатка в имени метода или вызов метода объекта не того класса (например, послал `addObject:` строке).

Шаг 10. Method swizzling: подмена реализации

Последний приём — самый острый инструмент. Мы умеем читать пары (SEL, IMP) и добавлять новые. А ещё их можно **менять местами**: `method_exchangeImplementations` обменивает реализации двух методов. После обмена старое имя ведёт на чужой код, и наоборот. Это называют **method swizzling**.

```
#import <Foundation/Foundation.h>
#import <objc/runtime.h>

@interface Greeter : NSObject
- (NSString *)hello;
@end

@implementation Greeter
- (NSString *)hello { return @"привет"; }
@end

@interface Greeter (Swizzle)
@end

@implementation Greeter (Swizzle)
- (NSString *)my_hello {
    NSString *original = [self my_hello];    /* после обмена: старый hello */
```

```
        return [NSString stringWithFormat:@"[лог] %@!", original];
    }
@end

int main(void) {
    @autoreleasepool {
        Greeter *g = [[Greeter alloc] init];
        NSLog(@"до обмена: %@", [g hello]);

        Method orig = class_getInstanceMethod([Greeter class],
                                                @selector(hello));
        Method swiz = class_getInstanceMethod([Greeter class],
                                                @selector(my_hello));
        method_exchangeImplementations(orig, swiz);

        NSLog(@"после обмена: %@", [g hello]);

        Greeter *other = [[Greeter alloc] init];
        NSLog(@"и для нового объекта: %@", [other hello]);
    }
    return 0;
}
```

Что здесь происходит:

- В категории Greeter (Swizzle) (категории — глава 11) мы добавляем метод my_hello. Внутри он зовёт... [self my_hello]. На вид это бесконечная рекурсия, но это не она: мы пишем код **до** обмена, а выполнится он **после**, когда имя my_hello уже будет указывать на исходный hello. То есть [self my_hello] в теле вызовет старый hello. Хитрый, но классический приём.
- class_getInstanceMethod(cls, sel) — возвращает Method (пару SEL+IMP) для селектора, с **учётом наследования**.
- method_exchangeImplementations(orig, swiz) — меняет IMP у двух методов местами. Теперь сообщение hello выполняет код, который мы написали как my_hello, а тот внутри зовёт настоящий старый hello.

Запускаем:

```
до обмена:  привет
после обмена: [лог] привет!
и для нового объекта: [лог] привет!
```

Метод `hello` после обмена возвращает `[лог] привет!` — мы «обернули» чужой метод, не трогая его исходник. И главное — изменение коснулось **всех** экземпляров `Greeter`, включая созданный позже `other`. В этом и сила, и опасность.

Чем `swizzling` опасен:

- Подмена глобальна: ты меняешь поведение класса для **всей** программы, включая код, который об этом не знает (в том числе системный).
- Порядок и момент подмены важны. Если два места засвиззлят один метод, результат зависит от того, кто успел первым, — ловить такие баги тяжело.
- Имя `my_hello` не должно случайно совпасть с чужим методом — иначе сломаешь и его. Поэтому `swizzling`-методам дают редкие префиксы.

Правило простое: `swizzling` — это инструмент для отладки, аналитики и обхода чужих багов, а не для повседневной логики. В обычном коде ему не место.

Связь с KVO и тизер главы 20

Подменять можно не только реализации методов, но и сам `isa` объекта: функция `runtime_object_setClass(obj, cls)` переселяет объект в другой класс. На этом Apple строит **KVO** (Key-Value Observing, наблюдение за значением по ключу). Когда ты начинаешь наблюдать за свойством объекта, Foundation на лету создаёт **динамический подкласс** этого объекта, переопределяет в нём сеттеры (а они шлют уведомления наблюдателям) и **подменяет `isa`** объекта на этот новый подкласс. Объект продолжает выглядеть как прежний класс (—class специально это маскирует), но его реальный `isa` теперь указывает на сгенерированный подкласс (по нашему эксперименту для класса `Dog` он называется `NSKVONotifying_Dog`). Это и есть «isa-swizzling».

Теперь, зная, что объект — это структура с полем `isa`, ты понимаешь, как такое вообще возможно: достаточно переписать класс в первом поле объекта (через `object_setClass`, а не руками — вспомни про упакованные биты), и он начинает вести себя как другой класс. Подробно KVO разберём в главе 20 — там этот фокус заиграет красками.

Проверяем

Готовые файлы главы лежат в `code/`. Собираются и запускаются одинаково:


```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/14-isa.m -o t && ./t
```

Список файлов и что они показывают:

- `code/14-isa.m` — объект знает свой класс (`object_getClass` vs `[obj class]`), общий `isa` у экземпляров.
- `code/14-metaclass.m` — класс это объект, цепочка `isa` до корневого метакласса, замыкание `isa` на себя.
- `code/14-introspect.m` — дампы иерархии, методов и полей через `runtime`.
- `code/14-resolve.m` — динамическое добавление метода в `+resolveInstanceMethod:`.
- `code/14-forwarding.m` — рабочий прокси на `-forwardingTargetForSelector:`.
- `code/14-swizzle.m` — обмен реализациями через `method_exchangeImplementations`.

Все шесть собираются **без предупреждений** под `-Wall -Wextra`. В выводах главы префикс `NSLog` опущен: у тебя перед каждой строкой будут дата, время, имя программы и числа в квадратных скобках, а адреса — свои. Важна содержательная часть строк.

Частые ошибки

- **Прямой вызов `objc_msgSend` без приведения типа.** Компилятор отвергает его ошибкой: в SDK `objc_msgSend` объявлена без параметров, и звать её надо, приведя к точному типу метода. В обычном коде её не трогают вовсе — пользуйся квадратными скобками или функциями `runtime`.
- **Забыл `free` после `class_copy`....** Функции с `copy` в имени (`class_copyMethodList`, `class_copyIvarList`) выделяют память через `malloc`. ARC до неё не дотягивается — освобождай руками, иначе утечка.
- **Перепутал `isa` и `superclass`.** `isa` — «какого я вида» (объект -> класс -> метакласс). `superclass` — «от кого унаследован» (вверх по дереву). Разные оси, не смешивай.
- **Неверная кодировка типов в `class_addMethod`.** Если метод возвращает не `void` или принимает другие аргументы, строка кодировки должна это отражать ("`i@:`" — вернёт `int`; "`v@:i`" — примет ещё `int`). Ошибка тут даёт порчу памяти, а не понятную диагностику.
- **`respondsToSelector:` на прокси возвращает «нет».** Это нормально: прокси и правда не реализует метод, он его *пересылает*. Если нужно, чтобы прокси честно отвечал «да», переопредели `respondsToSelector:`.
- **Swizzling в повседневной логике.** Глобальная подмена ради обычной фишки — источник трудноуловимых багов. Оставь её для отладки и обходных путей.
- **`[obj fly]` к объекту, который не умеет `fly`.** Получишь `unrecognized selector` и падение. Перед динамическим вызовом проверяй `respondsToSelector:` (глава 5).

Упражнения

1. Добавь в 14-metaclass.m класс `Puppy : Dog` и распечатай для него и класс, и метакласс. Убедись, что `isa` его метакласса по-прежнему ведёт к тому же корневому метаклассу.
2. Расширь дампы из 14-introspect.m: для каждого метода печатай ещё и его закодированный тип возврата через `method_copyReturnType` (не забудь `free`).
3. В 14-resolve.m добавь обработку второго селектора `bye` с собственной функцией-реализацией. Проверь, что `resolve` срабатывает по разу на каждый.
4. Преврати `Proxy` из 14-forwarding.m в «логирующий прокси»: пусть он ещё и считает, сколько сообщений переслал, и печатает счётчик.
5. В 14-swizzle.m сначала напиши классу `Greeter` собственный `-description`, а потом засвиizzi его так, чтобы он возвращал текст в верхнем регистре. Проследи, что `NSLog(@"%q", obj)` печатает новый текст. Подумай, почему без своего `-description` обмен задел бы `description` самого `NSObject` — то есть всех объектов программы.
6. Поймай `unrecognized selector` в свой код: оберни рискованный вызов в `@try { ... } @catch (NSException *e) { ... }` и напечатай `e.reason` вместо падения. (Исключения — обзорно; в проде так не «лечат» баги.)

Что мы получили

Мы добрались до самого низа объектной модели и увидели, что магии нет — есть стройная механика. Объект — это структура, чьё первое поле `isa` ведёт к классу. Класс — тоже объект, его `isa` ведёт к метаклассу, и вся конструкция замыкается на корневом метаклассе `NSObject`. Сообщение `[obj foo]` — это `objc_msgSend`, который ищет метод от `isa` получателя вверх по `superclass`; не найдя — даёт три шанса пересылки и лишь потом падает с «`unrecognized selector`». А поскольку классы живые, их можно читать (интроспекция), дополнять (`class_addMethod`, `resolveInstanceMethod`), переадресовывать (`forwardingTargetForSelector`) и переписывать (`swizzling`) прямо во время выполнения.

Это и есть та динамика, которой Си лишён в принципе: там после компиляции от типов не остаётся ничего, а вызовы прибиты к фиксированным адресам. Понимание `runtime` — ключ ко всему, что дальше: к KVC/KVO (глава 20), к тому, как `Foundation` устроен изнутри, и к отладке самых загадочных падений.

Со следующей главы мы поднимаемся обратно на прикладной уровень и осваиваем `Foundation` — строки, числа, коллекции, — но уже зная, что именно крутится под капотом у каждого [].

Документация Apple

- Objective-C Runtime Programming Guide (Messaging, Dynamic Method Resolution, Message Forwarding, Type Encodings) — <https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ObjCRuntimeGuide/Introduction/Introduction.html>
- Objective-C Runtime (справочник функций `object_getClass`, `class_getName`, `class_copyMethodList`, `class_addMethod`, `method_exchangeImplementations`, `sel_getName`) — <https://developer.apple.com/documentation/objectivec/objective-c-runtime>
- `NSObject` (`class`, `superclass`, `isKindOfClass:`, `respondsToSelector:`, `+resolveInstanceMethod:`, `-forwardingTargetForSelector:`, `-forwardInvocation:`, `-doesNotRecognizeSelector:`) — <https://developer.apple.com/documentation/objectivec/nsobject-swift.class>
- `NSProxy` (второй корневой класс, абстрактный суперкласс для прокси) — <https://developer.apple.com/documentation/foundation/nsproxy>
- `NSInvocation` (упакованное сообщение для `-forwardInvocation:`) — <https://developer.apple.com/documentation/foundation/nsinvocation>

Глава 15. Foundation: строки

Мы дошли до Foundation — библиотеки, ради которой во многом и существует вся объектная машинерия из прошлых глав. И первое, с чего начинают почти любую программу, — это текст. Имя пользователя, путь к файлу, ответ сервера, цена в чеке: всё это строки.

В главе 4 ты намучился с си-строками: `char *`, ручной поиск `'\0'`, `strlen`, `strcmp`, и финальный позор — кириллица, где `strlen("Привет")` выдала 12 вместо 6. Тогда я обещал: придёт `NSString` и всё станет по-человечески. Глава пришла.

Что мы сделаем

- Разберём `NSString` — строку-объект: чем он богаче голого `char *`.
- Научимся создавать строки пятью способами и переводить их в си-строку и обратно.
- Поймём `length` — и почему он считает не байты и не «человеческие» буквы, а UTF-16 code units (и где это больно бьёт по эмодзи).
- Сравним строки правильно: `isEqualToString:`, а не `==`.
- Пройдёмся по поиску, подстрокам, регистру, обрезке, разбиению и склейке.
- Возьмём `NSMutableString` — строку, которую можно менять на месте.
- Построим с нуля нормализатор логина и прогоним его на живых данных.

NSString: строка как объект

В Си строка — это кусок памяти с байтами и нулём в конце. Она ничего не знает о себе: ни длины, ни умения сравниться с соседкой. Всё это ты писал руками.

`NSString` — наоборот, полноценный **объект**. Он хранит текст внутри себя и несёт с собой десятки методов: посчитать длину, сравнить, найти подстроку, перевести регистр, разбить по разделителю. Ты не лазаешь по байтам — ты посылаешь строке сообщения, как любому другому объекту из прошлых глав.

Отличие от Си. `char *s = "hi"` — это адрес первого байта и больше ничего: ни длины за $O(1)$, ни единого метода. `NSString *s = @"hi"` — это объект, который знает свою длину и умеет десятки операций. Маленький знак @ (см. главу 1) превращает мёртвый массив байтов в живую строку.

Почему NSString неизменяемый

`NSString` **неизменяемый** (immutable): раз создан — его текст менять нельзя. Нет метода «допиши сюда букву» или «замени третий символ». Любая операция, которая на вид «меняет» строку, в действительности возвращает **новую** строку, а исходная остаётся как была.

Звучит как ограничение, но это подарок. Раз строку нельзя испортить, её можно безбоязненно раздавать: передал `NSString` в три разных места — и уверен, что никто тайком не перепишет твой текст под ногами. Поэтому в главе 8 свойства-строки мы помечали сору — чтобы зафиксировать неизменяемый снимок. А когда менять текст всё-таки нужно, для этого есть отдельный класс `NSMutableString`, наследник `NSString`; к нему дойдём в конце главы.

Создаём строки

Откроем `15-create.m` и пройдем по способам создания.

```
#import <Foundation/Foundation.h>

int main(void) {
    @autoreleasepool {
        NSString *hello = @"Привет";
        NSLog(@"литерал:      %@", hello);

        NSString *name = @"Серик";
        int age = 40;
        NSString *line = [NSString stringWithFormat:@"%@", тебе %d",
                                                                    name, age];
        NSLog(@"формат:      %@", line);

        NSString *line2 = [[NSString alloc] initWithFormat:@"%@", name];
        NSLog(@"alloc/init:   %@", line2);

        const char *c = "raw C string";
        NSString *fromC = [NSString stringWithUTF8String:c];
        NSLog(@"из C-строки:  %@", fromC);
    }
}
```

```
const char *back = [fromC UTF8String];
NSLog(@"в C-строку:    %s (это уже char*)", back);

NSLog(@"length 'Привет': %lu", (unsigned long)[hello length]);
}
return 0;
}
```

Разберём по способам.

Литерал @"Привет". Самый частый. Знак @ перед кавычками говорит компилятору: «собери из этого готовый объект NSString». Такая строка живёт всё время работы программы, освобождать её не нужно.

stringWithFormat: — собрать строку из кусков по образцу, ровно как printf, только результат не печатается, а возвращается новой строкой. Принимает строку формата и значения для подстановки. Спецификаторы те же, что у printf: %d для int, %.2f для double, плюс особый %@ — «подставь сюда объект» (см. главу 1). Это **фабричный метод класса** (+), поэтому зовём его у самого NSString, а не у экземпляра.

initWithFormat: — то же самое, но в два шага: alloc создаёт пустой объект, initWithFormat: его наполняет. Результат идентичен stringWithFormat:; под ARC разницы в памяти ты не почувствуешь. Знать обе формы полезно: фабрики stringWith... и пары alloc/initWith... идут парой почти у каждого класса Foundation.

stringWithUTF8String: — мост из мира Си. Принимает const char * (си-строку в кодировке UTF-8) и строит из неё NSString. Вот зачем тебе эти страдания с char * из главы 4: иногда строки приходят из си-функций, и их надо втянуть в объектный мир.

UTF8String — мост в обратную сторону. Возвращает const char *: си-строку в UTF-8. Нужна, когда objc-строку надо отдать в функцию, которая понимает только char * (многие системные вызовы такие).

Подводный камень UTF8String. Возвращённый указатель **временный**: он живёт не дольше самой строки, а может и меньше — до конца текущего пула автоосвобождения. Не сохраняй его «на потом» в долгую переменную — используй тут же. Нужна долгоживущая копия — скопируй байты в свой буфер (strdup или malloc + strcpy из главы 4) и потом освободи его через free.

Запускаем:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/15-create.m -o /tmp/t && /tmp/t
```

```
литерал:      Привет
формат:       Серик, тебе 40
alloc/init:   Серик!
из C-строки:  raw C string
в C-строку:   raw C string (это уже char*)
length 'Привет': 6
```

Обрати внимание на последнюю строку: `length` для «Привет» дал 6 — ровно столько букв. В главе 4 `strlen` той же «Привет» выдавала 12. Объект считает иначе, чем `си`-строка. Что именно он считает — следующий разговор.

length: длина в UTF-16 code units

`length` возвращает количество символов в строке. Тип результата — `NSUInteger` (беззнаковое целое). На 64-битном Mac это `unsigned long`, а на старых 32-битных платформах было `unsigned int`. Поэтому в `NSLog` его печатают как `%lu` с явным приведением (`unsigned long`) — такая запись верна на любой платформе, и так советует Apple. Возьмёшь не тот спецификатор, например `%d`, — компилятор предупредит и сам подскажет приведение.

Но «количество символов» здесь означает не то, что подсказывает интуиция. `NSString` показывает текст наружу как последовательность **UTF-16** (внутри он вправе хранить его компактнее, но это его личное дело), и `length` считает **code units UTF-16** — 16-битные кусочки, из которых текст собран. Для латиницы и кириллицы один `code unit` = одна буква, поэтому «Привет» дал честные 6. Но так везёт не всегда.

Отличие от Си. `strlen` считает **байты** до `'\0'`: «Привет» = 12. `length` считает **code units UTF-16**: «Привет» = 6. Ни то, ни другое не равно числу «человеческих» символов, как только в тексте появляется эмодзи. Запомни три разные величины — байты, `code units`, символы — это не одно и то же.

Разберём ловушку на `15-unicode.m`.

```
#import <Foundation/Foundation.h>

int main(void) {
    @autoreleasepool {
        NSString *en = @"café";
        NSLog(@"'café' length = %lu", (unsigned long)[en length]);
    }
}
```

```

NSString *ru = @"Привет";
NSLog(@"'Привет' length = %lu, UTF-8 байт = %lu",
      (unsigned long)[ru length],
      (unsigned long)strlen([ru UTF8String]));

NSString *emoji = @"🍎";
NSLog(@"'🍎' length = %lu (а человек видит 1 символ)",
      (unsigned long)[emoji length]);

NSString *flag = @"🇰🇿";
NSLog(@"'🇰🇿' length = %lu (а человек видит 1 флаг)",
      (unsigned long)[flag length]);
}
return 0;
}

```

Вывод:

```

'cafe' length = 4
'Привет' length = 6, UTF-8 байт = 12
'🍎' length = 2 (а человек видит 1 символ)
'🇰🇿' length = 4 (а человек видит 1 флаг)

```

Смотри внимательно. 🍎 — один значок для человека, а length говорит 2. Так вышло потому, что это символ за пределами базового набора, и UTF-16 кодирует его двумя code units (так называемой суррогатной парой). Флаг 🇰🇿 ещё хуже: он составлен из двух региональных символов, каждый по два code unit, итого length = 4 при одном видимом флаге.

Мораль: **length нельзя путать с «числом символов, которые видит человек»**. Для латиницы и кириллицы они совпадают, и можно расслабиться. Но стоит появиться эмодзи или экзотике — и наивный обход «по индексам от 0 до length» разрежет эмодзи пополам и испортит текст.

Правильный обход по «человеческим» символам

Когда нужно пройти именно по тому, что человек считает символами, существует `enumerateSubstringsInRange:options:usingBlock:` с опцией `NSStringEnumerationByComposedCharacterSequences`. Длинное имя, но смысл простой: «разбей строку на составные последовательности символов и дай мне каждую по очереди». Допишем в `15-unicode.m`:


```
NSString *mix = @"a🍏🚩b";
NSLog(@"'a🍏🚩b' length = %lu", (unsigned long)[mix length]);

__block NSUInteger humanCount = 0;
NSStringEnumerationOptions opts =
    NSStringEnumerationByComposedCharacterSequences;
[mix enumerateSubstringsInRange:NSMakeRange(0, [mix length])
    options:opts
    usingBlock:^(NSString *sub, NSRange subRange,
                NSRange enclosing, BOOL *stop) {
    (void)subRange; (void)enclosing; (void)stop;
    humanCount++;
    NSLog(@" символ #%lu = %@", (unsigned long)humanCount, sub);
}];
NSLog(@"человеческих символов: %lu", (unsigned long)humanCount);
```

Здесь работает **блок** `^{ ... }` из главы 12: метод зовёт его для каждой найденной последовательности и передаёт саму подстроку `sub` и её диапазон `subRange`. Переменную `humanCount` мы поместили `__block`, чтобы блок мог её менять (см. главу 12). Параметры `subRange`, `enclosing`, `stop` нам не нужны — глушим предупреждение `-Wextra` об их неиспользовании строкой `(void)subRange;`. Опцию мы вынесли в переменную `opts` только ради того, чтобы длинное имя не растягивало строку. Вывод:

```
'a🍏🚩b' length = 8
символ #1 = a
символ #2 = 🍏
символ #3 = 🚩
символ #4 = b
человеческих символов: 4
```

`length` сказал 8, а человеческих символов оказалось 4 — и эмодзи с флагом не порезаны. Запомни этот метод как надёжный способ ходить по тексту посимвольно, когда туда может попасть юникод.

Сравнение строк

Самая частая и самая болезненная ошибка новичка. В главе 4 мы пробовали сравнить си-строки через `==` и получили чушь, потому что `==` сравнивал адреса, а не буквы. С `NSString` — ровно та же ловушка, и по той же причине.

Откроем 15-compare.m:

```
NSString *a = @"яблоко";
NSString *b = [NSString stringWithFormat:@"%@", @"яблоко"];

NSLog(@"a == b:           %@", (a == b) ? @"YES" : @"NO");
NSLog(@"isEqualToString: %@",
      [a isEqualToString:b] ? @"YES" : @"NO");
```

a и b несут один и тот же текст «яблоко», но это **два разных объекта**: b мы собрали через stringWithFormat:, и он лежит в памяти по другому адресу. Вывод:

```
a == b:           NO
isEqualToString:  YES
```

== дал NO, потому что сравнил **адреса** двух объектов, а они разные. isEqualToString: дал YES, потому что сравнил **содержимое** — букву к букве.

Отличие от Си. Это та же история, что с char * в главе 4: == сравнивает указатели (адреса), а не текст. В Си спасал strcmp. В Objective-C для текста **всегда** isEqualToString:. Писать == для строк — значит спрашивать «это буквально один и тот же объект в памяти?», а почти всегда тебе нужно «одинаковый ли текст?».

isEqualToString: принимает другую строку и возвращает BOOL: YES, если тексты совпадают посимвольно. Это самый быстрый и точный способ проверить равенство. (Есть ещё общий isEqual: от NSObject, но для строк бери специальный isEqualToString: — он быстрее, ведь не тратит время на проверку типов.)

Порядок, регистр, префиксы

```
NSComparisonResult r = [@"apple" compare:@"banana"];
NSLog(@"compare apple/banana: %ld (-1 = первый раньше)", (long)r);

NSComparisonResult ci = [@"Apple" caseInsensitiveCompare:@"apple"];
NSLog(@"caseInsensitive: %ld (0 = равны)", (long)ci);
```

compare: отвечает не «равны/нет», а **как они соотносятся по порядку**. Возвращает NSComparisonResult — одно из трёх значений: NSOrderedAscending (-1, получатель идёт раньше), NSOrderedSame (0, равны), NSOrderedDescending (1, получатель идёт позже). Это то, что нужно для сортировки списка строк по алфавиту.

caseInsensitiveCompare: — то же сравнение, но без учёта регистра: «Apple» и «apple» для него равны, результат 0.

```
NSString *file = @"report-2026.pdf";
NSLog(@"hasPrefix report-:    %@",
      [file hasPrefix:@"report-"] ? @"YES" : @"NO");
NSLog(@"hasSuffix .pdf:      %@",
      [file hasSuffix:@".pdf"] ? @"YES" : @"NO");
NSLog(@"containsString 2026:  %@",
      [file containsString:@"2026"] ? @"YES" : @"NO");
```

`hasPrefix:` — начинается ли строка с заданной (удобно проверять схему `https://` или префикс имени файла). `hasSuffix:` — заканчивается ли (проверка расширения `.pdf`). `containsString:` — есть ли подстрока где-то внутри. Все три возвращают `BOOL`. Вывод:

```
compare apple/banana:  -1 (-1 = первый раньше)
caseInsensitive:       0 (0 = равны)
hasPrefix report-:     YES
hasSuffix .pdf:        YES
containsString 2026:   YES
```

Поиск и подстроки

`hasPrefix:/containsString:` отвечают «да/нет». А когда нужно знать *где* именно нашлась подстрока, берут `rangeOfString:`. Он возвращает `NSRange` — простую структуру из двух полей: `location` (с какого индекса) и `length` (сколько символов). Это чистый Си-struct, не объект.

```
NSRange range = [file rangeOfString:@"-"];
if (range.location == NSNotFound) {
    NSLog(@"дефис не найден");
} else {
    NSLog(@"дефис на позиции:    %lu", (unsigned long)range.location);
}
```

Подводный камень. Если подстрока не найдена, `rangeOfString:` возвращает `{NSNotFound, 0}`: в `location` кладёт особое значение `NSNotFound` (это огромное число, `NSIntegerMax`, а не ноль!). Проверять надо именно `range.location == NSNotFound`. Забудешь проверку — отдашь этот «индекс» в `substringToIndex:` и уронишь программу исключением.

Зная индекс, режем строку на части:

```
NSString *namePart = [file substringToIndex:range.location];
NSString *tailPart = [file substringFromIndex:range.location + 1];
```

`substringToIndex:` берёт кусок **от начала до** указанного индекса (сам индекс не включается). `substringFromIndex:` — **от индекса и до конца**. Есть и `substringWithRange:`, берущий произвольный диапазон `NSRange`. Каждый из них, по правилу неизменяемости, возвращает **новую** строку. Вывод:

```
дефис на позиции:      6
substringToIndex:       report
substringFromIndex:     2026.pdf
```

Подводный камень индексов. Индексы здесь — это позиции в UTF-16 code units, те самые, что считает `length`. Режь так строку, где есть эмодзи, по «человеческим» позициям — и можешь рассечь суррогатную пару. Для текста с юникодом ищи границы через `rangeOfComposedCharacterSequenceAtIndex:` или обход из раздела про `length`.

Регистр и обрезка

```
NSLog(@"uppercaseString:    %@", [@"привет" uppercaseString]);
NSLog(@"lowercaseString:    %@", [@"ПРИВЕТ" lowercaseString]);
NSLog(@"capitalizedString:  %@", [@"мир добра" capitalizedString]);

NSString *messy = @"  hello world  ";
NSCharacterSet *ws = [NSCharacterSet whitespaceAndNewlineCharacterSet];
NSString *clean = [messy stringByTrimmingCharactersInSet:ws];
NSLog(@"trim:                %@", clean);
```

`uppercaseString` и `lowercaseString` возвращают новую строку целиком в верхнем или нижнем регистре. `capitalizedString` делает заглавной первую букву **каждого слова**.

`stringByTrimmingCharactersInSet:` срезает с **обоих краёв** строки все символы из переданного набора `NSCharacterSet` — и останавливается, как только встретит символ не из набора. Внутренние символы не трогает. `whitespaceAndNewlineCharacterSet` — готовый набор «пробелы, табы и переводы строк»; это самый частый случай — почистить ввод пользователя от случайных пробелов по краям. Вывод:

```
uppercaseString:    ПРИВЕТ
lowercaseString:    привет
capitalizedString:   Мир Добра
trim:               'hello world'
```

Кавычки вокруг 'hello world' я поставил нарочно, чтобы было видно: края вычищены, а пробел между словами остался на месте.

Разбиение, склейка и неизменяемая «дописка»

Переходим к 15-mutate.m. Сначала — операции, которые рожают новые строки (получатель остаётся прежним).

```
NSString *csv = @"яблоко,банан,груша";
NSArray<NSString *> *parts = [csv componentsSeparatedByString:@","];
NSLog(@"частей после split:    %lu", (unsigned long)[parts count]);
NSLog(@"вторая часть:        %@", parts[1]);

NSString *joined = [parts componentsJoinedByString:@" | "];
NSLog(@"joined:                %@", joined);
```

`componentsSeparatedByString:` режет строку по разделителю и возвращает NSArray кусков (массивы — тема главы 17, пока достаточно знать, что `count` даёт число элементов, а `parts[1]` — второй из них). Запись `NSArray<NSString *> *` в угловых скобках — подсказка компилятору «внутри лежат строки»; без неё код работает так же, подробно — в главе 22. Обратная операция `componentsJoinedByString:` живёт уже на массиве: склеивает его элементы в одну строку, вставляя между ними разделитель.

```
NSString *base = @"Лог";
NSString *one = [base stringByAppendingString:@": старт"];
NSString *two = [one stringByAppendingFormat:@" (код %d)", 200];
NSLog(@"appendString:        %@", one);
NSLog(@"appendFormat:        %@", two);
NSLog(@"base не изменилась:   %@", base);
```

`stringByAppendingString:` приклеивает другую строку в конец и возвращает **новую** строку. `stringByAppendingFormat:` — то же, но хвост собирается по образцу формата. Ключевая деталь в последней строке вывода:

```
частей после split:    3
вторая часть:          банан
joined:                 яблоко | банан | груша
appendString:          Лог: старт
appendFormat:          Лог: старт (код 200)
base не изменилась:     Лог
```

base так и осталась «Лог». Мы её не меняли — `stringByAppendingString`: создал отдельную строку, а base неизменяема по определению. Это и есть неизменяемость в действии.

NSMutableString: строку можно менять

Собирать строку из десяти кусков через десять `stringByAppending...` — расточительно: каждый шаг плодит новый объект, а промежуточные тут же выбрасываются. Для случаев, когда строку надо собирать или править по месту, есть `NSMutableString` — **изменяемый** наследник `NSString`.

Отличие от Си. В Си «изменяемость» строки — это просто наличие места в буфере `char buf[64]`, и ты сам стережёшь, чтобы не вылезти за край (глава 4, привет `strcpy` и переполнению). `NSMutableString` растёт сам: добавляешь текст — он сам выделит память. Никаких буферов и подсчёта байтов руками.

```
NSMutableString *log = [NSMutableString stringWithString:@""];
[log appendString:@"GET"];
[log appendFormat:@" /users?page=%d", 2];
[log appendString:@""];
NSLog(@"mutable собрали:      %@", log);

NSUInteger n = [log replaceOccurrencesOfString:@"users"
                                     withString:@"clients"
                                     options:0
                                     range:NSMakeRange(0, [log length])];
NSLog(@"замен сделано:      %lu", (unsigned long)n);
NSLog(@"после замены:      %@", log);

[log insertString:@"HTTP " atIndex:1];
NSLog(@"после вставки:      %@", log);
```

Разбираем методы, которых нет у неизменяемого `NSString` — они меняют **сам объект** и обычно ничего не возвращают (`void`):

- `appendString`: — дописывает строку в конец прямо в этот объект.
- `appendFormat`: — то же, но хвост по образцу формата.
- `replaceOccurrencesOfString:withString:options:range:` — заменяет все вхождения одной подстроки на другую **в указанном диапазоне**. `options:0` — без особых правил поиска; `range` обычно вся строка, поэтому пишем `NSMakeRange(0, [log length])`. Этот метод как раз **возвращает** число сделанных замен — единственный с возвратом.

- `insertString:atIndex:` — вставляет строку начиная с указанной позиции, раздвигая остальное.

Вывод:

```
mutable собрали:      [GET /users?page=2]
замен сделано:        1
после замены:         [GET /clients?page=2]
после вставки:        [HTTP GET /clients?page=2]
```

Заметь: `stringWithString:` создаёт изменяемую копию из обычной строки — удобный старт. И ещё: `NSMutableString` — наследник `NSString`, его можно передавать всюду, где ждут `NSString`. Обратное неверно: где ждут `NSMutableString`, обычную `NSString` не подсунешь, у неё нет `appendString:`.

Частая ошибка. Свойство объявлено `@property (copy) NSString *name;` (глава 8), ты присвоил ему `NSMutableString`, продолжаешь менять его снаружи и ждёшь, что правки появятся в свойстве. Не появятся: `copy` сделал неизменяемый снимок в момент присвоения. И это не баг, а ровно та причина, по которой строковые свойства помечают `copy`, а не `strong`: никто снаружи не перепишет текст объекта у него за спиной.

Число ↔ строка

Текст из поля ввода приходит строкой, а считать надо числом. И наоборот — число надо показать строкой. Оба перехода есть в конце `15-create.m`:

```
NSString *priceText = @"199.90";
NSString *countText = @"7 штук";
NSLog(@"doubleValue:   %.2f", [priceText doubleValue]);
NSLog(@"intValue:     %d", [countText intValue]);

double total = [priceText doubleValue] * [countText intValue];
NSString *totalText = [NSString stringWithFormat:@"%s.2f тенге", total];
NSLog(@"итог строкой:  %@", totalText);
```

`[priceText doubleValue]` читает из строки дробное, `[countText intValue]` — целое, а обратно число превращаем в текст знакомым `stringWithFormat:` через `%d/%f`. `intValue/doubleValue` терпимы к мусору: «7 штук» даст 7, разбор пропускает пробелы в начале и останавливается на первом нечисловом символе. Вывод:

```
doubleValue:    199.90
intValue:      7
итог строкой:  1399.30 тенге
```

Подводный камень intValue. Если строка вообще не начинается с цифры («abc»), вернётся 0 — и ты не отличишь «там был ноль» от «там была не цифра». Когда важно знать, удался ли разбор, бери `NSScanner` или `NSNumberFormatter` (оба описаны в документации Foundation, в книге мы их не разбираем); `intValue` хорош для быстрых прикидок, где 0-по-умолчанию устраивает.

NSAttributedString — строка с оформлением

Кратко, для общей картины. `NSString` хранит только сам текст, без вида: шрифта, цвета, жирности. Когда тексту нужно оформление — например, часть слова красная и жирная, — берут `NSAttributedString`: это текст плюс набор **атрибутов** (`attributes`), привязанных к диапазонам символов.

Им рисуют форматированные подписи, ссылки внутри текста, цветные куски в интерфейсе. У него тоже есть изменяемый собрат — `NSMutableAttributedString`. Глубоко мы его здесь не трогаем: он живёт ближе к `UIKit/AppKit`, где есть что раскрашивать. Просто знай: если спросят «как сделать слово в подписи красным», ответ — `NSAttributedString`, а не `NSString`.

Строим с нуля: нормализатор логина

Соберём из изученного маленькую честную задачу. Пользователь вводит логин как попало: с пробелами по краям, в разном регистре, иногда с ведущей @ (привычка из соцсетей). Нам нужно привести всё к единому виду: без пробелов, в нижнем регистре, без @ в начале. Пустой ввод — отвергнуть.

Шаг 1. Наивная версия

Первая мысль — просто перевести в нижний регистр:

```
NSString *normalizeUsername(NSString *raw) {
    return [raw lowercaseString];
}
```

Прогоним мысленно на « @Ivan_KZ ». Получим « @ivan_kz ». Регистр поправили, но пробелы по краям остались, и @ тоже. Логин « @ivan_kz » и «ivan_kz» система сочтёт разными людьми. Плохо.

Шаг 2. Срезаем пробелы

Добавим обрезку краёв перед сменой регистра:

```
NSMutableCharacterSet *ws = [NSMutableCharacterSet whitespaceAndNewlineCharacterSet];
NSString *s = [raw stringByTrimmingCharactersInSet:ws];
s = [s lowercaseString];
```

Теперь « @Ivan_KZ » → «@ivan_kz». Пробелы ушли, регистр в норме. Осталась ведущая @.

Шаг 3. Снимаем ведущую @

Проверим префикс и, если он есть, отрежем первый символ:

```
if ([s hasPrefix:@"@"]) {
    s = [s substringFromIndex:1];
}
```

hasPrefix:@"@" отвечает, начинается ли строка с собачки. substringFromIndex:1 берёт всё начиная со второго символа — то есть выбрасывает @. «@ivan_kz» → «ivan_kz».

Шаг 4. Отвергаем пустой ввод

Если пользователь ввёл одни пробелы, после обрезки строка станет пустой. Вернём nil — пусть вызывающий код решит, что делать:

```
if ([s length] == 0) {
    return nil;
}
```

Полная функция

Складываем шаги в одну функцию:

```
NSString *normalizeUsername(NSString *raw) {
    NSMutableCharacterSet *ws = [NSMutableCharacterSet whitespaceAndNewlineCharacterSet];
    NSString *s = [raw stringByTrimmingCharactersInSet:ws];
    s = [s lowercaseString];
    if ([s hasPrefix:@"@"]) {
        s = [s substringFromIndex:1];
    }
    if ([s length] == 0) {
        return nil;
    }
}
```

```
    }  
    return s;  
}
```

В main (целиком — в 15-normalize.m) прогоняем её по списку грязных вводов через for (NSString *input in inputs) — это **быстрый перебор** (fast enumeration): пройтись по элементам массива по очереди, без индексов (подробно — в главе 17). Запускаем:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \  
    code/15-normalize.m -o /tmp/t && /tmp/t
```

```
' @Ivan_KZ ' -> 'ivan_kz'  
'SERIK' -> 'serik'  
'@Astana2026' -> 'astana2026'  
' ' -> ОТКЛОНЁН (пусто)
```

Четыре разных «грязных» ввода — четыре правильных исхода. Заметь, что вся функция собрана из методов, которые мы разобрали по дороге, и ни в одном месте мы не лазали в байты руками. Это и есть выигрыш NSString над char *: задача читается как описание словами.

Проверяем

Все программы главы лежат в code/ и собираются одинаково (подставь имя файла):

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \  
    code/15-create.m -o /tmp/t && /tmp/t
```

Файлы: 15-create.m (создание и числа), 15-compare.m (сравнение, поиск, регистр), 15-mutate.m (разбиение, склейка, NSMutableString), 15-unicode.m (ловушка length и обход по символам), 15-normalize.m (нормализатор логина). Все собираются без единого предупреждения.

Частые ошибки

- **== вместо isEqualToString:.** == сравнивает адреса объектов, а не текст. Иногда «срабатывает» случайно (на одинаковых литералах компилятор может вернуть один и тот же объект) — и тем коварнее: код работает на тестах и падает на данных из сети. Для текста всегда isEqualToString:.
- **length как «число символов».** Для эмодзи и составных символов length больше, чем видит человек. Не нарежь по таким индексам строку с юникодом — порежешь символ пополам.

- **Забыл проверку `NSNotFound`.** `rangeOfString:` при неудаче кладёт в `location` гигантское `NSNotFound`, а не 0. Используешь его как индекс без проверки — падение.
- **Меняешь `immutable`-строку.** `NSString` менять нельзя; `appendString:` у него просто нет. Нужно менять по месту — бери `NSMutableString`.
- **Сохранил `UTF8String` надолго.** Указатель временный, привязан к пулу автоосвобождения. Используй сразу или копируй байты.
- **`NSUInteger` без приведения.** `length/count` — это `NSUInteger`. Печатай как `(unsigned long)[s length]` с `%lu`. С `%d` компилятор предупредит, а голый `%lu` без приведения на 64-битном Mac пройдёт молча, но на платформе, где `NSUInteger` — `unsigned int`, формат разъедется.

Упражнения

1. Напиши функцию, которая принимает email и возвращает его доменную часть (после @). Используй `rangeOfString:` и `substringFromIndex:.` Не забудь про `NSNotFound`, если @ нет.
2. Дополни `normalizeUsername:` отвергай логин, если в нём остались пробелы внутри (подсказка — `rangeOfString:@" "`).
3. Собери из массива слов `@[@"Objective", @"C", @"c", @"нуля"]` одну строку через пробел двумя способами: `componentsJoinedByString:` и циклом с `NSMutableString`. Сравни код.
4. Возьми строку с эмодзи и посчитай в ней «человеческие» символы через `enumerateSubstringsInRange:options:usingBlock:` с `NSStringEnumerationByComposedCharacterSequences`. Сверх с `length`.
5. Напиши функцию-«цензор»: замени в тексте все вхождения слова на звёздочки той же длины через `NSMutableString` и `replaceOccurrencesOfString:withString:options:range:.`
6. Проверь на себе ловушку `==`: создай две строки с одинаковым текстом через `stringWithFormat:` и сравни их и `==`, и `isEqualToString:.`

Что мы получили

Голый `char *` из главы 4 уступил место `NSString` — объекту, который сам знает длину, умеет сравниваться, искать, резать, менять регистр и склеиваться. Мы разобрались, почему строки неизменяемы по умолчанию и зачем рядом живёт `NSMutableString`, научились правильно сравнивать (`isEqualToString:`, не `==`) и обходить юникод по-человечески, не доверяя `length` слепо.

Дальше — числа. В главе 16 возьмём `NSNumber` и `NSValue`: как упаковать обычный `int` или `BOOL` в объект, чтобы положить его в коллекцию, и как считать деньги без потери копеек через `NSDecimalNumber`.

Документация Apple

- NSString — developer.apple.com/documentation/foundation/nsstring
- NSMutableString — developer.apple.com/documentation/foundation/nsmutablestring
- NSAttributedString — developer.apple.com/documentation/foundation/nsattributedString
- NSCharacterSet — developer.apple.com/documentation/foundation/nscharacterset
- String Programming Guide (создание, сравнение, поиск, кодировки) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/Strings/introStrings.html

Глава 16. Foundation: числа и обёртки значений

В прошлой главе мы работали со строками — `NSString` — и привыкли, что в Objective-C всё интересное является объектом. Но есть исключение, доставшееся от Си: числа. `int`, `double`, `BOOL`, `char` — это **примитивы**, кусочки памяти, а не объекты. Сообщения им слать нельзя, в массив-объект положить — тоже. И вот тут начинается отдельная история.

Сейчас разберёмся, как Foundation выдаёт примитивам «объектное гражданство»: упаковывает число в `NSNumber`, любую C-структуру — в `NSValue`, а для денег, где `double` тихо врёт, даёт точный `NSDecimalNumber`. Под конец встретим `NSNull` — специальный объект-«ничто» для коллекций. Всё построим руками на одной сквозной задаче: сложить цены в чеке и не потерять копейку.

Что мы сделаем

- Поймём, почему примитив нельзя положить в коллекцию-объект.
- Научимся **упаковывать** (boxing) число в `NSNumber` и распаковывать обратно.
- Узнаем, почему `NSNumber` нельзя сравнивать через `==`, и чем это заменить.
- Завернём произвольную C-структуру в `NSValue`.
- Увидим, как `double` теряет копейки, и починим это `NSDecimalNumber`.
- Разберём `NSNull` и отличие `nil` / `NSNull` / `Nil`.

Разбираем задачу руками

Представь корзину интернет-магазина. Есть список цен: 199.99, 49.50, 3.30, 0.01. Нужно сложить их в итог и где-то хранить весь список.

В Си ты бы завёл массив `double prices[4]` и прошёл циклом — там и числа, и хранилище примитивны. В Objective-C хранилище — это `NSArray`, объект, и он держит **только объекты**. Число

199.99 туда напрямую не положишь: массив ждёт указатель на объект, а `double` — это не указатель. Значит, число надо сначала **завернуть в объект**. Это и называется упаковкой. А когда дойдём до сложения денег, выяснится, что и сам `double` для этой задачи негоден — но по порядку.

Отличие от Си. В Си `int` — это `int`: кладёшь его в массив, передаёшь в функцию, печатаешь `printf("%d", x)` напрямую. В Objective-C, чтобы положить число в коллекцию-объект (`NSArray`, `NSDictionary`), его обязательно упаковать в `NSNumber`. Запись `@(x)` — это и есть упаковка примитива в объект. Коллекции Foundation хранят указатели на объекты и ничего другого принять не могут.

NSNumber: упаковка числа в объект

`NSNumber` — это объект, единственная задача которого — носить внутри себя одно число любого примитивного типа. Завернул примитив в `NSNumber` — получил полноценный объект, которому можно слать сообщения и который можно класть в коллекции.

Литералы: самый короткий способ

Знакомый знак `@` (мы видели его у строк `@"..."`) умеет делать и числовые объекты. Поставь `@` перед числом — получишь готовый `NSNumber`:

```
NSNumber *a = @42;           /* число */
NSNumber *pi = @3.14;        /* дробь */
NSNumber *yes = @YES;         /* BOOL */
NSNumber *letter = @'A';      /* символ */
```

Разберём знаки:

- `@42` — упаковка целого. Внутри объекта лежит `int` со значением 42.
- `@3.14` — упаковка `double`.
- `@YES` — упаковка `BOOL`. Внутри будет 1 (а `@NO` — 0).
- `@'A'` — одиночные кавычки, как в Си, означают `char`. Внутри окажется код символа A — число 65.

А чтобы упаковать **результат выражения**, а не записанную руками константу, выражение берут в скобки после `@`:

```
int x = 7, y = 5;
NSNumber *sum = @(x + y); /* посчитает 12 и завернёт в объект */
```

@(...) — общая форма. @42 — это просто частный короткий случай @(42). Внутри скобок может стоять любое числовое выражение: переменная, сумма, вызов функции — компилятор посчитает значение и завернёт его в NSNumber.

Отличие от Си. В Си $7 + 5$ — это просто число 12, лежащее в регистре процессора. В Objective-C $@(7 + 5)$ — это уже объект в куче, у которого есть адрес, методы и собственная жизнь под управлением ARC. Знак @ превращает голое число в объект, ровно как @"..." превращает массив байтов в NSString.

Фабричные методы: то же самое словами

До литералов @(...) (они появились в 2012 году) числа упаковывали через методы-фабрики. Они и сейчас работают, и иногда читаются понятнее:

```
NSNumber *big = [NSNumber numberWithInt:2.5];
```

numberWithDouble: — это **сообщение классу** NSNumber (+-метод, как alloc). Оно создаёт и возвращает NSNumber, внутри которого double 2.5. Таких методов целое семейство, по одному на тип: numberWithInt:, numberWithDouble:, numberWithBool:, numberWithChar: и так далее. @2.5 и [NSNumber numberWithInt:2.5] дают один и тот же результат.

Распаковка: достаём примитив обратно

Завернули — рано или поздно надо развернуть: чтобы посчитать, сравнить или напечатать через %d. У NSNumber есть парные методы-«геттеры», каждый возвращает значение в нужном примитивном типе:

```
int ai = [a intValue];          /* 42 */
double pid = [pi doubleValue]; /* 3.14 */
BOOL yb = [yes boolValue];     /* YES (1) */
```

intValue достаёт значение как int, doubleValue — как double, boolValue — как BOOL. Есть и floatValue, longValue, charValue, unsignedIntValue и прочие. NSNumber помнит, каким типом его создали (это можно узнать через objCType: у @42 это "i", у @3.14 — "d"), но не держится за него: ты просишь то представление, которое тебе сейчас нужно, и он сконвертирует. Спросишь intValue у @3.99 — получишь 3 (дробную часть отбросит).

Собираем первый пример

```
#import <Foundation/Foundation.h>

int main(void) {
```

```

@autoreleasepool {
    NSNumber *a = @42;
    NSNumber *pi = @3.14;
    NSNumber *yes = @YES;
    NSNumber *letter = @'A';

    int x = 7, y = 5;
    NSNumber *sum = @(x + y);
    NSNumber *big = [NSNumber numberWithDouble:2.5];

    NSLog(@"a=%@ pi=%@ yes=%@ letter=%@", a, pi, yes, letter);
    NSLog(@"sum=%@ big=%@", sum, big);

    int ai = [a intValue];
    double pid = [pi doubleValue];
    BOOL yb = [yes boolValue];
    NSLog(@"распаковка: ai=%d pid=%.2f yb=%d", ai, pid, yb);

    /* А вот и ради чего всё затевалось: число — в массив. */
    NSArray *nums = @[ @10, @20, @(x * y) ];
    NSLog(@"массив чисел: %@", nums);

    NSNumber *third = nums[2];
    NSLog(@"третий как int: %d", [third intValue]);
}
return 0;
}

```

Обрати внимание на %@ в NSLog. Мы печатаем сам **объект** NSNumber, поэтому спецификатор — %@ («спроси у объекта, как он выглядит»), а не %d. А в последней строке мы сначала распаковали объект в int через intValue и печатаем примитив — поэтому там %d. Запускаем:

```

a=42 pi=3.14 yes=1 letter=65
sum=12 big=2.5
распаковка: ai=42 pid=3.14 yb=1
массив чисел: (
    10,
    20,
    35
)

```



```
)  
третий как int: 35
```

Всё сошлось. @YES напечатался как 1, @'A' — как 65 (код символа), а @[@10, @20, @(x * y)] — массив из трёх упакованных чисел. Без упаковки он бы не собрался: на @[10, 20] компилятор ответит ошибкой `numeric literal must be prefixed by '@' in a collection`.

Сравнение NSNumber: почему == врёт

Естественное желание — сравнить два числа через ==. С примитивами в Си так и делают: `if (x == y)`. Но NSNumber — это **объект**, а с объектами == сравнивает не значения, а адреса в памяти. Мы это уже проходили на строках: два разных NSString с одинаковым текстом — не равны по ==. С числами та же ловушка, только коварнее.

Отличие от Си. Для примитивов == сравнивает значения: `1000 == 1000` — истина, всегда. Для NSNumber (как и для любого объекта) == сравнивает **указатели**: «это буквально один и тот же объект в памяти?». Два разных объекта с числом 1000 внутри по == могут оказаться неравны. Значения сравнивают другим инструментом.

Этот инструмент — `isEqualToNumber:`. Он лезет **внутрь** объектов и сравнивает сами числа:

```
NSNumber *a = @(1000);  
NSNumber *b = [NSNumber numberWithInt:1000];  
  
if (a == b)           { /* про адреса */ }  
if ([a isEqualToNumber:b]) { /* про значения */ }
```

А чтобы узнать не просто «равны / не равны», а кто больше, есть `compare:`. Он возвращает не BOOL, а одно из трёх значений типа `NSComparisonResult`:

- `NSOrderedAscending` (значение -1) — получатель **меньше** аргумента;
- `NSOrderedSame` (0) — равны;
- `NSOrderedDescending` (1) — получатель **больше**.

```
NSComparisonResult r = [small compare:large];  
if (r == NSOrderedAscending) { /* small < large */ }
```

Те же `NSOrderedAscending/Same/Descending` мы видели у строк (`compare:` у `NSString`) — это общий для всего Foundation язык сравнения, его понимают и методы сортировки коллекций.

Ловушка кэширования: иногда == «работает», и это хуже всего

Тут история тоньше, чем со строками. Соберём программу, которая в лоб проверяет == на разных числах:

```
#import <Foundation/Foundation.h>

int main(void) {
    @autoreleasepool {
        NSNumber *a = @(1000);
        NSNumber *b = [NSNumber numberWithInt:1000];
        if (a == b)
            NSLog(@"== : а и b это один объект");
        else
            NSLog(@"== : а и b разные объекты (адреса не совпали)");

        if ([a isEqualToNumber:b])
            NSLog(@"isEqualToNumber: значения равны");

        NSNumber *small = @5, *large = @9;
        if ([small compare:large] == NSOrderedAscending)
            NSLog(@"compare: 5 меньше 9");

        NSNumber *p = @1, *q = @1;
        NSLog(@"@1 == @1 ? %@", (p == q) ? @"да" : @"нет");

        NSNumber *m = @100000, *n = @100000;
        NSLog(@"@100000 == @100000 ? %@", (m == n) ? @"да" : @"нет");
    }
    return 0;
}
```

Запускаем:

```
== : а и b разные объекты (адреса не совпали)
isEqualToNumber: значения равны
compare: 5 меньше 9
@1 == @1 ? да
@100000 == @100000 ? да
```

Вглядиись в результат — он на первый взгляд противоречив:

- а и b (оба со значением 1000) по == — **разные** объекты;
- но @1 и @1 по == — **один и тот же**;
- и @100000 с @100000 — тоже **один и тот же**.

Как так? По нашим экспериментам (Apple clang 21, macOS 26) здесь работают два разных механизма.

- **Литералы-константы.** @1, @100000 и @(1000) с постоянным значением компилятор собирает в готовые объекты прямо внутри программы (их класс — `NSConstantIntegerNumber`), причём одинаковые литералы склеивает в **один** объект. Поэтому @1 == @1 и @100000 == @100000 дали «да»: это буквально один и тот же объект. Уровень оптимизации (-O0 или -O2) на это не влияет.
- **Числа, созданные в рантайме.** `numberWithInt:1000` строится во время работы. Небольшие значения runtime ради экономии памяти вообще не кладёт в кучу, а упаковывает прямо внутрь самого указателя (этот приём называют *tagged pointer*, «помеченный указатель»). Два таких числа 1000 дадут одинаковый «указатель», а вот с константой @(1000) из программы он, конечно, не совпал. Большое число, которое внутрь указателя не влезает, runtime честно создаёт в куче — и два таких объекта по == уже разные.

Вывод простой и жёсткий: **поведение == на NSNumber непредсказуемо**. Оно зависит от величины числа, способа создания и версии компилятора и ОС. Иногда == случайно «работает» — и это самое опасное: на маленьких числах при отладке всё совпадает, а на проде с большими или иначе созданными числами тихо разваливается. Поэтому правило без исключений:

Никогда не сравнивай NSNumber через ==. Для равенства — `isEqualToNumber:`, для порядка — `compare:`.

NSNumber: обёртка для любой C-структуры

NSNumber заворачивает одно число. А если надо положить в коллекцию не число, а **структуру** — например `NSRange` (диапазон: с какой позиции и сколько, встречали у строк) или точку с координатами? NSNumber тут бессилён: он умеет только числа. Для произвольных C-структур есть `NSNumber` — старший брат `NSNumber` (формально `NSNumber` — наследник `NSNumber`).

`NSNumber` копирует **байты** структуры внутрь себя. Создают его так:

```
NSRange range = NSRange(2, 5); /* location=2, length=5 */
NSNumber *boxed = [NSNumber valueWithBytes:&range
                               objCType:@encode(NSRange)];
```

Два аргумента:

- `valueWithBytes:` — **адрес** структуры (`&range` — «дай адрес переменной `range`»). `NSValue` копирует оттуда байты к себе. Передаём адрес, а не саму структуру, потому что метод должен знать, *откуда* копировать.
- `objCType:` — **тип-код**, описание формы структуры. Его выдаёт директива `@encode(NSRange)`: компилятор превращает тип в короткую строку-код, по которой `NSValue` знает, сколько байт и как устроено хранимое. Это нужно, чтобы при распаковке вернуть всё на место без искажений.

Отличие от Си. В Си структуру просто кладут в массив структур или копируют присваиванием — она и так значение, не объект. В Objective-C, чтобы структура попала в `NSArray/NSDictionary`, её надо завернуть в объект `NSValue`. `@encode(Тип)` — чисто objc-директива: в Си нет понятия «строка-описание типа для рантайма».

Распаковка — метод `getValue:`. Ему дают адрес пустой структуры, куда вернуть байты:

```
NSRange back;
[boxed getValue:&back];    /* скопирует байты из объекта в back */
```

Для самых частых структур Foundation даёт готовые удобные обёртки, чтобы не возиться с `@encode`. Для `NSRange` это пара `valueWithRange:` / `rangeValue:`:

```
NSValue *r2 = [NSValue valueWithRange:NSMakeRange(10, 3)];
NSRange got = [r2 rangeValue];
```

(Для точек и прямоугольников на macOS есть `valueWithPoint:/pointValue` и `valueWithRect:/rectValue`, а в UIKit на iOS — `valueWithCGPoint:/CGPointValue` и `valueWithCGRect:/CGRectValue`.)

Соберём:

```
NSRange range = NSMakeRange(2, 5);
NSValue *boxed = [NSValue valueWithBytes:&range objCType:@encode(NSRange)];

NSArray *ranges = @[ boxed ];
NSLog(@"в массиве лежит: %@", ranges[0]);

NSRange back;
[boxed getValue:&back];
NSLog(@"распаковали: location=%lu length=%lu",
      (unsigned long)back.location, (unsigned long)back.length);

NSValue *r2 = [NSValue valueWithRange:NSMakeRange(10, 3)];
NSRange got = [r2 rangeValue];
```

```
NSLog(@"valueWithRange: location=%lu length=%lu",  
      (unsigned long)got.location, (unsigned long)got.length);
```

Вывод:

```
в массиве лежит: NSRange: {2, 5}  
распаковали: location=2 length=5  
valueWithRange: location=10 length=3
```

Структура благополучно съездила в объект и вернулась — и длинным путём через `valueWithBytes:objCType:`, и коротким через `valueWithRange:`. Заметь первую строку: `NSValue` умеет описать себя человеку — `NSRange: {2, 5}`, — поэтому в отладке содержимое видно сразу. Когда тебе мало одного числа и нужна целая структура в коллекции — это работа для `NSValue`.

NSDecimalNumber: точные деньги

Теперь главное в этой главе — деньги. Вернёмся к чеку из четырёх цен. Наивно сложим их через `double`, как сделали бы в Си.

Шаг 1. Наивная версия на `double` — и она врёт

```
double p1 = 0.10;  
double p2 = 0.20;  
double total = p1 + p2;  
NSLog(@"double: 0.10 + 0.20 = %.17f", total);  
  
if (total == 0.30)  
    NSLog(@"double: сумма точно равна 0.30");  
else  
    NSLog(@"double: сумма НЕ равна 0.30 (потеряли копейку)");
```

`%.17f` просит напечатать число с 17 знаками после запятой — чтобы увидеть правду целиком. Запускаем:

```
double: 0.10 + 0.20 = 0.30000000000000004  
double: сумма НЕ равна 0.30 (потеряли копейку)
```

Вот он, знаменитый «хвост»: `0.30000000000000004` вместо `0.30`. Это не баг Objective-C и не баг твоего Mac. Это устройство типа `double`.

`double` хранит число в двоичной системе. А в двоичной дроби `0.1` и `0.2` записываются **бесконечной** периодической дробью, ровно как $1/3 = 0.333...$ в десятичной. В конечную ячейку

памяти бесконечное не влезает, его обрубают — и появляется крошечная погрешность. Сложил два обрубка — погрешности сложились и вылезли на семнадцатом знаке.

Отличие от Си. Здесь Objective-C ни при чём — `double` ведёт себя совершенно одинаково в Си и в Objective-C, потому что это один и тот же си-шный тип с одним и тем же двоичным представлением. Разница в другом: в чистом Си **готового решения нет** — точную десятичную арифметику пришлось бы писать руками или тащить стороннюю библиотеку. Foundation же даёт объектный тип `NSDecimalNumber`, который считает в десятичной системе и копеек не теряет.

Для денег погрешность на семнадцатом знаке кажется ерундой, но она копится: на тысячах операций «копейка туда, копейка сюда» расходится баланс, и бухгалтерия этого не прощает. Деньги нельзя считать в `double`. Точка.

Шаг 2. Чиним: `NSDecimalNumber`

`NSDecimalNumber` хранит число в **десятичной** системе — как мы его и пишем, — поэтому `0.1` для него ровно одна десятая, без двоичных хвостов. Создают его лучше всего **из строки**, а не из `double` (иначе погрешность просочится ещё на входе):

```
NSDecimalNumber *d1 = [NSDecimalNumber decimalNumberWithString:@"0.10"];
NSDecimalNumber *d2 = [NSDecimalNumber decimalNumberWithString:@"0.20"];
```

`decimalNumberWithString:` разбирает текст `"0.10"` и строит точное десятичное число. Складывать примитивным `+` объекты нельзя (это объекты, а не числа), поэтому у арифметики свои методы-сообщения:

- `decimalNumberByAdding:` — сложить;
- `decimalNumberBySubtracting:` — вычесть;
- `decimalNumberByMultiplyingBy:` — умножить;
- `decimalNumberByDividingBy:` — разделить.

Каждый возвращает **новый** `NSDecimalNumber` с результатом, не меняя исходные.

```
NSDecimalNumber *dsum = [d1 decimalNumberByAdding:d2];
NSLog(@"decimal: 0.10 + 0.20 = %@", dsum);

NSDecimalNumber *exact = [NSDecimalNumber decimalNumberWithString:@"0.30"];
if ([dsum isEqualToNumber:exact])
    NSLog(@"decimal: сумма ТОЧНО равна 0.30");
```

`NSDecimalNumber` — наследник `NSNumber`, поэтому сравнивается ровно теми же `isEqualToNumber:` и `compare:` (и снова **не** через `==`).

Шаг 3. Складываем весь чек

```
NSArray *prices = @[ @"199.99", @"49.50", @"3.30", @"0.01" ];
NSDecimalNumber *cart = [NSDecimalNumber zero];
for (NSString *priceText in prices) {
    NSDecimalNumber *price =
        [NSDecimalNumber decimalNumberWithString:priceText];
    cart = [cart decimalNumberByAdding:price];
}
NSLog(@"итого по чеку: %@", cart);

NSDecimalNumber *one = [NSDecimalNumber decimalNumberWithString:@"19.99"];
NSDecimalNumber *three = [one decimalNumberByMultiplyingBy:
    [NSDecimalNumber decimalNumberWithString:@"3"]];
NSLog(@"три по 19.99 = %@", three);
```

[NSDecimalNumber zero] — готовый ноль, с него удобно начинать накопление. Цикл for (NSString *priceText in prices) — перебор коллекции (*fast enumeration*): на каждом витке priceText берёт очередной элемент. Внутри превращаем строку цены в точное число и прибавляем к cart.

Полный вывод всей программы про деньги:

```
double: 0.10 + 0.20 = 0.30000000000000004
double: сумма НЕ равна 0.30 (потеряли копейку)
decimal: 0.10 + 0.20 = 0.3
decimal: сумма ТОЧНО равна 0.30
итого по чеку: 252.8
три по 19.99 = 59.97
```

NSDecimalNumber дал 0.3 без хвоста и точное 252.8 по чеку. 59.97 за три товара по 19.99 — тоже копейка в копейку. Задача из начала главы решена: цены сложены и не потеряно ни копейки.

NSNull: объект-«ничто» для коллекций

Последний герой главы. В коллекцию иногда нужно положить «здесь значения нет» — пустую ячейку. Логично было бы сунуть nil. Но nil в коллекции **запрещён**: для методов вроде array withObjects: nil означает «конец списка». Литерал @[a, nil, b] с написанным руками nil компилятор не пропустит, а если nil пришёл в переменной, программа упадёт при запуске с исключением attempt to insert nil object from objects[1]. Класть nil некуда.

Решение — `NSNull`: специальный класс с единственным объектом, который и играет роль «официального ничто». Берут его так:

```
id maybeName = [NSNull null];
NSArray *row = @[ @"Иван", maybeName, @42 ];
```

`[NSNull null]` всегда возвращает **один и тот же** общий объект-пустышку (*singleton*, одиночка). Его-то и кладут в коллекцию вместо отсутствующего значения. Когда позже читаешь ячейку, проверяешь, не пустышка ли там:

```
id cell = row[1];
if (cell == [NSNull null])
    NSLog(@"в ячейке 1 – NSNull (значения нет)");
```

Здесь `==` уместен и правилен — потому что `[NSNull null]` это всегда буквально один и тот же объект, и мы как раз спрашиваем про тождество, а не про значение.

Запустим весь кусок с `NSNull`:

```
строка с дыркой: (
    "\U0418\U0432\U0430\U043d",
    "<null>",
    42
)
в ячейке 1 – NSNull (значения нет)
nil это NSNull? нет
```

`NSNull` в массиве печатается как `<null>`. (Кириллица «Иван» вылезла кодами `\U0418...` — это особенность того, как `NSArray` печатает себя в `NSLog`, к теме главы отношения не имеет.)

nil, NSNull и Nil — три разные вещи

Их легко перепутать, поэтому разложим по полочкам:

nil	пустой указатель на ОБЪЕКТ (значение 0)	«переменной не на что указывать»; в коллекцию класть нельзя
Nil	пустой указатель на КЛАСС (тип Class)	то же, но для переменной типа Class
NSNull	настоящий	«значение есть, и оно —

	ОБЪЕКТ-пустышка	осознанное ничто»; в	
	[NSNull null]	коллекцию класть можно	

Ключевая разница: `nil` — это **отсутствие объекта** (ноль вместо адреса), а `NSNull` — **присутствующий объект**, означающий «ничего». Поэтому в выводе `nil == [NSNull null]` оказалось нет: слева пустой указатель, справа живой объект по реальному адресу. `Nil` — тот же `nil`, но для переменных-классов (тип `Class`).

Отличие от Си. В Си есть только `NULL` — нулевой указатель, и всё. В Objective-C нулей три: `nil` (нет объекта), `Nil` (нет класса) и `NULL` (нет си-шного указателя, для не-объектов). А `NSNull` — вообще не «ноль», а полноценный объект, придуманный специально потому, что в коллекции настоящий ноль положить нельзя.

Полные программы

В папке `code/` лежат четыре готовых файла этой главы, каждый собирается и запускается отдельно:

- `16-nsnumber.m` — упаковка и распаковка `NSNumber`, число в массиве;
- `16-compare.m` — `==` против `isEqualToNumber:/compare:` и ловушка кэширования;
- `16-decimal.m` — ошибка `double` и точный `NSDecimalNumber`, сумма чека;
- `16-nsvalue.m` — `NSValue` для `NSRange` и разбор `NSNull`.

Проверяем

Собери и запусти любой из них одной командой:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/16-decimal.m -o /tmp/decimal
/tmp/decimal
```

Ожидаемый вывод `16-decimal.m` (строки `NSLog` идут с датой и временем — они будут свои):

```
double: 0.10 + 0.20 = 0.30000000000000004
double: сумма НЕ равна 0.30 (потеряли копейку)
decimal: 0.10 + 0.20 = 0.3
decimal: сумма ТОЧНО равна 0.30
итого по чеку: 252.8
три по 19.99 = 59.97
```

–Wall –Wextra включают все предупреждения; код главы собирается **без единого** из них.

Частые ошибки

- **Кладёшь примитив в коллекцию напрямую.** `@[ 10, 20 ]` или `[arr addObject:5]` не скомпилируются: коллекции хранят объекты. Упакуй — `@[ @10, @20 ], [arr addObject:@5]`.
- **Сравниваешь NSNumber через ==.** Иногда «работает» на маленьких числах и тихо ломается на больших. Всегда `isEqualToNumber:` (равенство) или `compare:` (порядок).
- **Считаешь деньги в double.** Копейки уплывут на семнадцатом знаке и накопятся. Для денег — `NSDecimalNumber`, создавай его из строки.
- **Создаёшь NSDecimalNumber из double.** `decimalNumberWithString:@"0.1"` — точно; а если завести его из `double 0.1`, двоичная погрешность просочится ещё на входе. Подавай число строкой.
- **Кладёшь nil в массив как «пусто».** Литерал `@[ a, x ]`, где `x` оказался `nil`, упадёт, а `arrayWithObjects:` молча обрежется на `nil`. Для пустой ячейки — `[NSNumber null]`.
- **Печатаешь NSNumber через %d.** `NSLog(@"%d", @42)` даст мусор (и компилятор об этом предупредит): `%d` ждёт примитив, а ты дал объект. Либо печатай объект через `%@`, либо сперва распакуй: `NSLog(@"%d", [num intValue])`.

Упражнения

1. Заведи `NSNumber` со значением 255 и напечатай его тремя способами: как объект (`%@`), как `int` (`intValue`) и как `double` (`doubleValue`).
2. Создай два `NSNumber` с одним и тем же большим значением разными способами (`@(...)` и `numberWithInt:`) и сравни их `==` и `isEqualToNumber:`. Объясни результат словами.
3. Добавь в чек из `16-decimal.m` ещё две цены и пересчитай итог. Проверь сумму на калькуляторе — копейка в копейку?
4. Посчитай через `NSDecimalNumber` стоимость 7 товаров по 12.45 и вычти из неё скидку 5.00. Используй `decimalNumberByMultiplyingBy:` и `decimalNumberBySubtracting:`.
5. Заверни в `NSNumber` структуру `NSRange` со значениями по своему выбору, положи в массив, достань обратно через `rangeValue` и напечатай поля.
6. Сделай массив `@[ @"a", [NSNumber null], @"c" ]` и пройди его циклом, печатая для пустых ячеек слово «пусто», а для остальных — само значение.

Что мы получили

Примитивы Си получили объектное гражданство. Теперь ты умеешь упаковывать число в `NSNumber` (`@42`, `@(x)`, `numberWithDouble:`) и распаковывать обратно (`intValue`, `doubleValue`), знаешь, что сравнивать их надо `isEqualToNumber:` и `compare:`, а не `==`. Любая C-структура заворачивается в `NSValue`. Деньги считаются точно через `NSDecimalNumber`, потому что `double` теряет копейки. А пустую ячейку в коллекции занимает `NSNull` — не путать с `nil`.

Всё это — подготовка к следующей главе. Мы наконец возьмёмся за коллекции всерьёз: `NSArray`, `NSDictionary`, `NSSet` и их изменяемые версии. И там `NSNumber` с `NSNull` сразу пригодятся: ведь класть в коллекции, как мы теперь знаем, можно только объекты.

Документация Apple

- `NSNumber` — developer.apple.com/documentation/foundation/nsnumber
- `NSValue` — developer.apple.com/documentation/foundation/nsvalue
- `NSDecimalNumber` — developer.apple.com/documentation/foundation/nsdecimalnumber
- `NSNull` — developer.apple.com/documentation/foundation/nsnull
- Number and Value Programming Topics (упаковка чисел и значений) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/NumbersandValues/NumbersandValues.html

Глава 17. Foundation: коллекции

В главе 4 ты собирал данные в си-массив: `int prices[10];`. Удобно, пока знаешь размер заранее и хранишь примитивы. Но стоит захотеть «список имён, который растёт сам», «телефонную книгу имя→номер» или «набор без повторов» — и голый Си заставляет писать всё это руками.

Foundation даёт три готовых контейнера на все случаи: **NSArray** (упорядоченный список), **NSDictionary** (пары ключ→значение) и **NSSet** (набор уникальных элементов). У каждого есть изменяемый брат — `NSMutableArray...`. Разберём их по методам, а в конце построим маленький учёт товаров: посчитаем продажи, выделим уникальные категории и составим рейтинг.

Что мы сделаем

- Создадим неизменяемый массив литералом `@[...]` и переберём его.
- Научимся менять `NSMutableArray`: добавлять, вставлять, удалять.
- Соберём словарь `@{...}` и достанем значения по ключу.
- Сложим уникальные элементы в множество и пересечём два множества.
- Отсортируем массив блоком-компаратором и отфильтруем предикатом.
- Споткнёмся о классический краш «мутация во время перебора» и починим.
- Построим сквозной пример — учёт товаров за день.

Три контейнера и зачем их три

Представь, что ведёшь маленькую кофейню. Тебе нужно:

- **порядок продаж за день** — что за чем пробили. Список с порядком и повторами → массив;
- **остаток на складе** — «кофе: 12, чай: 5». Связь имя→число, ищем по имени → словарь;
- **какие категории сегодня продавались** — «горячее», «холодное», без повторов и порядка → множество.

Один и тот же товар может лежать во всех трёх — контейнеры дополняют друг друга.

Отличие от Си. В Си из коробки есть только массив, да и тот фиксированной длины и хранит примитивы по значению. «Словаря» и «множества» в языке нет вовсе — их пишут руками (хеш-таблицы, деревья) или тащат стороннюю библиотеку. В Foundation это готовые, вылизанные десятилетиями классы: бери и пользуйся.

Ещё одно общее правило, важное с самого начала: **коллекции Foundation хранят объекты, а не примитивы**. Положить туда `int` напрямую нельзя — сначала упакуй его в `NSNumber` (глава 16). И **`nil` класть нельзя** — для «пустого места» есть особый объект `NSNull` (тоже глава 16). Почему так — увидим по ходу.

NSArray: упорядоченный неизменяемый список

Самый частый способ создать массив — **литерал** `@[...]`. Тот же знак `@`, что превращал текст в `NSString`, а число в `NSNumber`, здесь строит готовый `NSArray`:

```
NSArray *fruits = @[ @"яблоко", @"банан", @"вишня" ];
```

Внутри — три объекта `NSString`. Запятые разделяют элементы, и можно ставить запятую даже после последнего — компилятор не против.

Разберём основные сообщения массиву.

count, индексы, края

```
NSLog(@"всего: %lu", (unsigned long)[fruits count]);

NSString *first = [fruits objectAtIndex:0];
NSString *second = fruits[1];           /* субскрипт */
```

- `count` — сколько элементов. Возвращает `NSUInteger` (беззнаковое целое). Печатаем его как `%lu`, приведя к `(unsigned long)` — это снимает предупреждение о размере типа на разных платформах.
- `objectAtIndex:` — взять элемент по номеру. **Нумерация с нуля**, как в Си: первый элемент — индекс 0.
- `fruits[1]` — то же самое, но короче. Квадратные скобки с числом компилятор разворачивает в вызов `objectAtIndexedSubscript:`, который работает как `objectAtIndex:`. Это **субскрипт** (`subscript`), привычная по Си запись `arr[i]`.

Отличие от Си. Запись `fruits[1]` выглядит как сишный доступ к массиву, но это не разыменование адреса по смещению. Это вызов метода `objectAtIndexedSubscript:`, который сам проверит границы. Выйдешь за край — получишь аккуратное исклю-

чение с понятным текстом, а не молчаливый выход за пределы памяти и мусор, как в Си.

Чтобы безопасно взять края, есть два метода, которым не нужен индекс:

```
NSLog(@"перв=%@ посл=%@", [fruits firstObject], [fruits lastObject]);
```

`firstObject` и `lastObject` возвращают первый и последний элементы. Их прелесть в том, что на **пустом** массиве они вернут `nil`, а не упадут. Сравни: `[empty objectAtIndex:0]` на пустом массиве — это исключение и краш.

Поиск: `containsObject:` и `indexOfObject:`

```
BOOL hasBanana = [fruits containsObject:@"банан"];      /* 1 */
NSUInteger idx = [fruits indexOfObject:@"вишня"];      /* 2 */
NSUInteger miss = [fruits indexOfObject:@"манго"];      /* NSNotFound */
NSLog(@"манго найдено? %d", miss != NSNotFound);        /* 0 */
```

- `containsObject:` — есть ли такой элемент. Возвращает `BOOL`.
- `indexOfObject:` — на каком он месте. Если нет — возвращает особое значение `NSNotFound` (это просто очень большое число). Сравнивай результат именно с `NSNotFound`, не с `-1`.

Важная деталь: оба метода ищут не «тот же самый объект в памяти», а **равный по содержанию**. Они зовут у элементов `isEqual:`. Поэтому строка `"банан"`, набранная заново, находит свою копию в массиве, хотя это, возможно, другой объект в памяти. Для строк это `isEqualToString:` под капотом (см. главу 15 про `==` против `isEqual:`).

`componentsJoinedByString:`

```
NSLog(@"список: %@", [fruits componentsJoinedByString:@" "]);
// → яблоко, банан, вишня
```

`componentsJoinedByString:` склеивает все элементы в одну строку, вставляя между ними разделитель. Работает, когда элементы — строки. Очень удобно для печати: иначе `NSLog(@"%@", fruits)` выведет массив в служебном формате со скобками и (для кириллицы) с `\U...`-кодами.

NSMutableArray: тот же массив, но изменяемый

`NSArray` после создания не меняется — ни добавить, ни удалить. Когда нужно менять, берём `NSMutableArray` (`mutable` — изменяемый). Его обычно создают пустым и наполняют:

```
NSMutableArray *cart = [[NSMutableArray alloc] init];
[cart addObject:@"хлеб"];           /* в конец */
[cart addObject:@"молоко"];
[cart insertObject:@"соль" atIndex:0]; /* вставить на позицию 0 */
[cart replaceObjectAtIndex:1 withObject:@"батон"];
[cart removeObject:@"молоко"];      /* удалить по значению */
[cart removeObjectAtIndex:0];      /* удалить по индексу */
```

- `addObject:` — добавить в конец. Массив растёт сам, заранее размер знать не нужно.
- `insertObject:atIndex:` — вставить на конкретную позицию; всё, что правее, сдвигается.
- `replaceObjectAtIndex:withObject:` — заменить элемент на месте.
- `removeObject:` — убрать **все** элементы, равные данному (по `isEqual:`).
- `removeObjectAtIndex:` — убрать один элемент по номеру.

Есть ещё конструктор-подсказка:

```
NSMutableArray *big = [NSMutableArray arrayWithCapacity:100];
```

`arrayWithCapacity:` намекает, что элементов будет около сотни, чтобы массив сразу выделил память и реже перевыделял её при росте. Это **не предел**: добавишь больше — массив спокойно вырастет. Чисто оптимизация; если не уверен — просто `init`.

Отличие от Си. Си-массив `int a[10]` — это десять ячеек подряд, фиксированных навсегда, и хранят они числа **по значению**. `NSMutableArray` растёт и сжимается сам, а хранит **ссылки на объекты**. Числа в нём лежат упакованными в `NSNumber` (глава 16): сам массив про `int` ничего не знает, он знает только объекты.

nil класть нельзя

Попытка `[cart addObject:nil]` — это краш с исключением (если `nil` написан прямо в коде, компилятор ещё и предупредит: `null passed to a callee that requires a non-null argument`). `nil` значит «объекта нет», а в методах вроде `arrayWithObjects:` он ещё и служит концом списка аргументов, поэтому как значение он запрещён. Если по смыслу нужна «дырка» или «значение отсутствует» — клади объект-заглушку `NSNull`:

```
[cart addObject:[NSNull null]]; /* законная «пустышка» */
```

`[NSNull null]` всегда возвращает один и тот же общий объект. Доставая его, ты сравниваешь: `if (obj == [NSNull null]) { ... }`. Подробно `NSNull` разбирали в главе 16.

Перебор массива

Быстрый перебор for-in

Главный способ пройти по коллекции — **быстрый перебор** (fast enumeration), цикл for-in:

```
for (NSString *f in fruits) {  
    NSLog(@"for-in: %@", f);  
}
```

Читается как «для каждого f из fruits». На каждом витке f становится очередным элементом. Не нужен ни счётчик, ни count, ни objectAtIndex: — компилятор и runtime берут это на себя, и делают это быстро (отсюда «быстрый»).

Отличие от Си. Сишный for (int i = 0; i < n; i++) крутит счётчик и сам лезет по индексу. for-in — другое: ты получаешь сразу **значения**, а не номера, и не управляешь шагом. Платишь за удобство одним ограничением: **коллекцию нельзя менять прямо во время такого перебора** (об этом — отдельный раздел ниже, там и краш покажем).

enumerateObjectsUsingBlock: с индексом и stop

Когда нужен не только элемент, но и его номер, помогает перебор блоком (блоки — глава 12):

```
[fruits enumerateObjectsUsingBlock:^(id obj, NSUInteger i, BOOL *stop) {  
    NSLog(@"блок [%lu] = %@", (unsigned long)i, obj);  
    if (i == 1) {  
        *stop = YES;    /* прерываем перебор досрочно */  
    }  
}];
```

Блок получает три параметра:

- obj — текущий элемент;
- i — его индекс;
- stop — указатель на флаг. Записав *stop = YES, ты говоришь «дальше не надо», и перебор останавливается. Это замена сишному break.

В нашем примере перебор оборвётся после индекса 1 — третий элемент (вишня) уже не попадёт в лог.

Обратный перебор

Чтобы пройти массив с конца, добавим опцию:


```
[fruits enumerateObjectsWithOptions:NSEnumerationReverse
                        usingBlock:^(id obj, NSUInteger i, BOOL *stop) {
    NSLog(@"обратно [%lu] = %@", (unsigned long)i, obj);
    (void)stop;
}];
```

`NSEnumerationReverse` — флаг «идти от последнего к первому». Индексы при этом остаются на-стоящими (2, 1, 0), просто порядок обхода обратный. Строка `(void)stop;` нужна, только чтобы компилятор не ругался на неиспользованный параметр под `-Wextra`; никакого действия она не несёт.

Проверяем (массивы)

Весь код выше собран в `code/17-array.m`:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/17-array.m -o /tmp/t && /tmp/t
```

Реальный вывод (даты и числа в скобках будут свои):

```
всего: 3
первый=яблоко второй=банан
перв=яблоко посл=вишня
банан внутри? 1
вишня на позиции 2
манго найдено? 0
список: яблоко, банан, вишня
корзина: соль, батон
после удаления[0]: батон
в big элементов: 1
for-in: яблоко
for-in: банан
for-in: вишня
блок [0] = яблоко
блок [1] = банан
обратно [2] = вишня
обратно [1] = банан
обратно [0] = яблоко
```

Видно: после `*stop = YES` блок остановился на индексе 1, а обратный перебор честно пошёл 2 → 1 → 0.

NSDictionary: пары ключ → значение

Словарь хранит связи «ключ → значение»: по ключу мгновенно достаёшь значение. Литерал — `@{ ключ : значение, ... }`:

```
NSDictionary *colors = @{
    @"яблоко" : @"красный",
    @"банан"  : @"жёлтый",
    @"слива"  : @"синий",
};
```

Слева от двоеточия — ключ, справа — значение. Здесь и ключи, и значения — строки, но значением может быть любой объект (число, массив, другой словарь).

Доступ по ключу

```
NSString *a = [colors objectForKey:@"яблоко"]; /* метод */
NSString *b = colors[@"банан"];               /* subscript */
NSString *miss = colors[@"манго"];             /* нет ключа → nil */
```

- `objectForKey:` — взять значение по ключу.
- `colors[@"банан"]` — то же короче (субскрипт по ключу разворачивается в `objectForKeyedSubscript:`).
- Нет такого ключа — вернётся `nil`, а **не** падение. В логе `nil` печатается как `(null)`.

`count` у словаря — это число пар.

`allKeys`, `allValues`

```
NSLog(@"ключи: %@", [[colors allKeys] componentsJoinedByString:@" "]);
```

`allKeys` отдаёт массив всех ключей, `allValues` — массив всех значений. Порядок в этих массивах **не определён**: словарь хранит пары так, как ему удобно для быстрого поиска, а не в порядке добавления.

NSMutableDictionary

Изменяемый словарь наполняют после создания:

```
NSMutableDictionary *stock = [[NSMutableDictionary alloc] init];
[stock setObject:@10 forKey:@"яблоко"]; /* метод */
stock[@"банан"] = @5;                  /* subscript-присваивание */
```

```
stock[@"слива"] = @3;
stock[@"яблоко"] = @12;           /* тот же ключ → замена */
[stock removeObjectForKey:@"слива"]; /* убрать пару */
```

- `setObject:forKey:` — положить значение под ключ.
- `stock[@"банан"] = @5;` — то же самое короче.
- Записать по **существующему** ключу — значит **заменить** значение, а не добавить вторую пару. Ключи в словаре уникальны.
- `removeObjectForKey:` — удалить пару по ключу.

Перебор словаря

`for-in` по словарю идёт **по ключам**:

```
for (NSString *fruit in colors) {
    NSLog(@"%@ → %@", fruit, colors[fruit]);
}
```

Если нужны сразу ключ и значение — есть блочный вариант:

```
[stock enumerateKeysAndObjectsUsingBlock:^(id key, id obj, BOOL *stop) {
    NSLog(@"склад: %@ = %@ шт.", key, obj);
    (void)stop;
}];
```

Требования к ключам

Ключ словаря — не любой объект. Он должен:

- **поддерживать `isEqual:` и `hash`** — словарь по хешу раскладывает пары по «корзинам» и сравнивает ключи на равенство. У `NSString` и `NSNumber` это уже сделано правильно, поэтому они — идеальные ключи;
- **копироваться** — при добавлении словарь делает копию ключа и хранит копию. Это защита: если ты потом изменишь свой объект-ключ, словарь не «потеряет» пару. Поэтому ключом обычно берут неизменяемые объекты — те же строки.

Значение копироваться не обязано — оно хранится по ссылке, как в массиве.

Отличие от Си. Чтобы получить такое в Си, ты бы писал хеш-таблицу сам: функцию хеширования, разрешение коллизий, рост таблицы. `NSDictionary` всё это уже умеет, а от твоих ключей просит лишь корректные `hash/isEqual:` — у стандартных классов они уже есть.

Проверяем (словарь)

Файл code/17-dictionary.m. Вывод:

```
пар: 3
яблоко=красный банан=жёлтый
манго=(null) (nil печатается как (null))
ключи: банан, слива, яблоко
яблоко на складе: 10
после довоза яблок: 12
после списания слив пар: 2
банан → жёлтый
слива → синий
яблоко → красный
склад: яблоко = 12 шт.
склад: банан = 5 шт.
```

Обрати внимание: порядок ключей в выводе не совпадает с порядком, в котором мы их писали в литерале. Это нормально — словарь неупорядочен. Последние две строки — это блочный обход `enumerateKeysAndObjectsUsingBlock`: по складу, где после довоза и списания осталось две позиции.

NSSet: уникальные элементы без порядка

Множество хранит **уникальные** объекты и не помнит порядок. Его сила — в одном: мгновенная проверка «есть ли такой элемент». Часто множество делают из массива, и дубликаты сами схлопываются:

```
NSArray *raw = @[ @"кофе", @"чай", @"кофе", @"сок", @"чай" ];
NSSet *drinks = [NSSet setWithArray:raw];
// в raw 5 элементов, в drinks — 3

NSLog(@"есть чай? %d", [drinks containsObject:@"чай"]); /* 1 */
NSLog(@"есть морс? %d", [drinks containsObject:@"морс"]); /* 0 */
```

`containsObject`: у множества работает по тому же `isEqual:/hash`, но по скорости это совсем другое дело, чем у массива.

Когда множество лучше массива

`containsObject`: у массива в худшем случае перебирает все элементы по очереди — чем длиннее массив, тем дольше. У множества проверка идёт по хешу и почти не зависит от размера.

Правило простое: **если главная операция — «а есть ли тут такой?», и порядок не важен, бери множество**. Список «кого уже обработали», «какие id видели», «множество тегов» — классические задачи для NSMutableSet.

Операции над множествами

NSMutableSet умеет складывать множества как в школьной теории:

```
NSMutableSet *cafeA = [NSMutableSet arrayWithObjects:@"кофе", @"чай",
                                                    @"какао" ];
NSSet *cafeB = [NSSet arrayWithObjects:@"чай", @"сок", @"какао" ];

NSMutableSet *common = [cafeA mutableCopy];
[common intersectSet:cafeB]; /* пересечение: только общие */

NSMutableSet *all = [cafeA mutableCopy];
[all unionSet:cafeB]; /* объединение: всё вместе */

NSMutableSet *only = [cafeA mutableCopy];
[only minusSet:cafeB]; /* разность: A без того, что есть в B */
```

- intersectSet: — оставить только элементы, которые есть **и там, и там**.
- unionSet: — добавить всё из второго множества.
- minusSet: — выкинуть из первого всё, что встречается во втором.

Эти методы меняют сам объект, поэтому мы каждый раз работаем с копией: mutableCopy возвращает новую **изменяемую** копию коллекции (подробнее — в разделе про copy и mutableCopy ниже). Так исходный cafeA остаётся нетронутым. И addObject: у множества молча игнорирует дубликат — добавить «чай» второй раз ничего не изменит.

NSMutableSet — уникальность плюс порядок

Иногда нужны обе вещи разом: и без повторов, и с сохранением порядка. Это NSMutableSet:

```
NSMutableSet *menu = [NSMutableSet initWithObjects:
                    @"суп", @"салат", @"суп", @"чай" ];
// → суп, салат, чай (второй «суп» отброшен, порядок сохранён)
NSLog(@"первый в меню: %@", [menu firstObject]);
```

Он как массив (есть firstObject, индексы, array), но с гарантией уникальности. Платишь за это памятью и чуть большей сложностью — поэтому берут его реже, по реальной нужде.

Проверяем (множества)

Файл `code/17-set.m`. Вывод (порядок строк внутри множества может отличаться — он не определён):

```
в массиве 5, в множестве 3 (дубли убраны)
есть чай? 1
есть морс? 0
напиток: кофе
напиток: сок
напиток: чай
общие: чай, какао
вместе: какао, кофе, сок, чай
только в A: кофе
после повторного добавления чая: 3
меню по порядку: суп, салат, чай
первый в меню: суп
```

Сортировка и фильтрация

`sortedArrayUsingComparator:`

Отсортировать массив можно блоком-**компаратором** — он сравнивает два элемента и говорит, кто раньше:

```
NSArray *words = @[ @"груша", @"ёж", @"апельсин",
                    @"кот", @"баклажан" ];

NSArray *byLength = [words sortedArrayUsingComparator:
    ^NSComparisonResult(id a, id b) {
        NSUInteger la = [(NSString *)a length];
        NSUInteger lb = [(NSString *)b length];
        if (la < lb) return NSOrderedAscending;
        if (la > lb) return NSOrderedDescending;
        return NSOrderedSame;
    }];
```

Блок получает два элемента `a` и `b` и обязан вернуть `NSComparisonResult` — одно из трёх значений:

- `NSOrderedAscending` — `a` должен идти **раньше** `b`;
- `NSOrderedDescending` — `a` должен идти **позже** `b`;

- `NSOrderedSame` — порядок между ними не важен.

`sortedArrayUsingComparator:` возвращает **новый** отсортированный массив, исходный не трогает. Здесь мы сортируем по длине строки.

Для частых случаев ничего изобретать не надо: у строк уже есть готовый компаратор `compare:`, который сравнивает по алфавиту. Тогда блок сводится к одной строке:

```
NSArray *byAbc = [words sortedArrayUsingComparator:
    ^NSComparisonResult(id a, id b) {
        return [(NSString *)a compare:b];
    }];
```

sortUsingDescriptors: кратко

Когда сортируешь массив объектов (или словарей) по какому-то их полю, удобнее **дескриптор сортировки** `NSSortDescriptor` — ты просто называешь ключ и направление:

```
NSMutableArray *people = @[
    @{ @"name" : @"Аня", @"age" : @30 },
    @{ @"name" : @"Боря", @"age" : @22 },
    @{ @"name" : @"Вера", @"age" : @27 },
] mutableCopy];

NSSortDescriptor *byAge =
    [NSSortDescriptor sortDescriptorWithKey:@"age" ascending:YES];
[people sortUsingDescriptors:@[ byAge ]];
```

`sortDescriptorWithKey:ascending:` — «сортируй по полю `age`, по возрастанию». `sortUsingDescriptors:` принимает **массив** дескрипторов: первый задаёт основной порядок, второй разруливает совпадения, и так далее. В отличие от `sortedArray...`, метод `sortUsingDescriptors:` меняет сам мутабельный массив на месте.

filteredArrayUsingPredicate: и NSPredicate

Отобрать из массива подходящие элементы помогает **предикат** `NSPredicate` — условие, записанное строкой:

```
NSPredicate *longOnes =
    [NSPredicate predicateWithFormat:@"length > 3"];
NSArray *filtered = [words filteredArrayUsingPredicate:longOnes];
// останутся только слова длиннее трёх букв
```

`predicateWithFormat:` строит условие из текста. `filteredArrayUsingPredicate:` прогоняет каждый элемент через это условие и возвращает новый массив из тех, для кого оно истинно. Здесь `length` — свойство строки; предикат сам спрашивает его у каждого элемента. Язык предикатов богат (`>`, `<`, `==`, `BEGINSWITH`, `CONTAINS`, `AND`, `OR`) — это отдельная большая тема, пока достаточно почувствовать идею.

valueForKey: на массиве — тизер KVC

И последний приём, который выглядит как магия:

```
NSArray *names = [people valueForKey:@"name"];  
// → массив из имён: Аня, Боря, Вера (в текущем порядке people)
```

`valueForKey:`, посланный **массиву**, идёт по всем его элементам, спрашивает у каждого значение под ключом `name` и собирает результаты в новый массив. Это первое прикосновение к **KVC** (Key-Value Coding, кодирование «ключ-значение») — целому механизму доступа к данным по строковому имени. Полноценно разберём его в главе 20; пока запомни, что из массива объектов одной строкой достаётся «колонка» их значений.

Проверяем (сортировка и фильтр)

Файл `code/17-sort-filter.m`. Вывод:

```
по длине: ёж, кот, груша, апельсин, баклажан  
по алфавиту: апельсин, баклажан, груша, ёж, кот  
длиннее 3 букв: груша, апельсин, баклажан  
по возрасту: Боря, Вера, Аня  
возрасты: (  
    22,  
    27,  
    30  
)
```

Последняя строка показывает, как `NSArray` печатает себя через `%@`: в скобках и по одному элементу на строку. Числа видны как есть; будь там строки, кириллица вышла бы `\U...`-кодами — потому для людей мы и склеиваем через `componentsJoinedByString:`.

Ловушка: мутация во время перебора

Самый частый краш новичка с коллекциями — изменить её прямо посреди `for-in`. Выглядит невинно: «иду по списку и удаляю ненужное».


```
NSMutableArray *a = [@[ @"a", @"b", @"c" ] mutableCopy];
for (NSString *s in a) {
    if ([s isEqualToString:@"b"]) {
        [a removeObject:s];    /* МЕНЯЕМ во время перебора! */
    }
}
```

Запусти — и программа упадёт с исключением:

```
*** Terminating app due to uncaught exception 'NSGenericException',
reason: '*** Collection <__NSArrayM: 0x...> was mutated while being
enumerated.'
```

Текст прямой: «коллекция была изменена во время перебора». Быстрый перебор работает по внутреннему состоянию коллекции, и если из-под него выдернуть элемент, состояние ломается — runtime честно это ловит и останавливает программу, чтобы ты не получил тихо испорченные данные.

Отличие от Си. В Си цикл по массиву ничего о «целостности коллекции» не знает — удалишь элемент на ходу, и в лучшем случае пропустишь соседний, в худшем залезешь в чужую память без всякого предупреждения. `for-in` в Objective-C, наоборот, специально следит и падает громко и сразу. Громкий краш лучше тихого повреждения.

Правильных способов два.

Первый — собрать «кандидатов на удаление» в отдельный список, а удалять уже потом, отдельным проходом:

```
NSMutableArray *toRemove = [NSMutableArray array];
for (NSString *s in a) {
    if ([s isEqualToString:@"b"]) {
        [toRemove addObject:s];
    }
}
for (NSString *s in toRemove) {
    [a removeObject:s];    /* а во время этого цикла не перебирается */
}
```

Второй — идти по индексам **с конца**. При проходе от последнего к первому удаление элемента не сдвигает те, до которых ты ещё не дошёл:

```
for (NSInteger i = (NSInteger)[a count] - 1; i >= 0; i--) {
    if ([a[i] isEqualToString:@"b"]) {
        [a removeObjectAtIndex:i];
    }
}
```

То же правило действует и для словаря с множеством: не удаляй из них ключи/элементы, пока перебираешь их в `for-in`. Собирай ключи на удаление в массив, потом удаляй.

Строим с нуля: учёт товаров за день

Соберём всё в одну маленькую программу. Дано — журнал продаж кофейни за день: просто список проданных товаров, с повторами. Нужно:

1. посчитать, сколько раз продан каждый товар;
2. узнать, сколько **разных** товаров продано;
3. собрать **категории** проданного без повторов;
4. составить рейтинг — от самого ходового к редкому;
5. вычистить «однодневок» (товары, проданные меньше двух раз).

Здесь пригодятся все три контейнера: массив для журнала, словарь для подсчёта, множество для уникальных категорий.

Шаг 1. Журнал и подсчёт

```
NSArray *sales = @[ @"кофе", @"чай", @"кофе", @"кофе",
                    @"сок", @"чай", @"кофе" ];

NSMutableDictionary *counts = [NSMutableDictionary dictionary];
for (NSString *item in sales) {
    NSNumber *prev = counts[item];          /* nil, если впервые */
    NSInteger n = prev ? [prev integerValue] : 0;
    counts[item] = @(n + 1);
}
```

Идём по журналу. Для каждого товара достаём текущий счётчик. Если товара ещё не было, `counts[item]` вернёт `nil` — тогда считаем, что было 0. Прибавляем единицу, упаковываем обратно в `NSNumber` через `@(n + 1)` и кладём в словарь. Тернарный оператор `prev ? ... : 0` аккуратно обрабатывает «первую встречу».

Шаг 2. Сколько разных товаров

```
NSSet *unique = [NSSet setWithArray:[counts allKeys]];
NSLog(@"разных товаров: %lu", (unsigned long)[unique count]);
```

Ключи словаря — это и есть все различные товары. Завернём их в множество (хотя ключи и так уникальны, это явно подчёркивает смысл «набор разных») и спросим count.

Шаг 3. Категории через множество

```
NSDictionary *category = @{
    @"кофе" : @"горячее",
    @"чай"  : @"горячее",
    @"сок"  : @"холодное",
};
NSMutableSet *soldCats = [NSMutableSet set];
for (NSString *item in unique) {
    [soldCats addObject:category[item]];
}
```

У нас есть справочник «товар → категория». Идём по уникальным товарам и складываем их категории в множество. «Кофе» и «чай» оба «горячее», но в множестве «горячее» окажется один раз — ровно то, что нужно.

Шаг 4. Рейтинг

```
NSArray *top = [[counts allKeys] sortedArrayUsingComparator:
    ^NSComparisonResult(id a, id b) {
        return [counts[b] compare:counts[a]]; /* по убыванию */
    }];
```

Сортируем ключи по их значениям в counts. Хитрость в порядке сравнения: мы сравниваем counts[b] с counts[a] (а не наоборот) — этим переворачиваем сортировку, и более продаваемые встают вперёд. У NSNumber есть готовый compare:, который вернёт нужный NSComparisonResult.

Шаг 5. Чистка однодневок — аккуратно

```
NSMutableArray *toRemove = [NSMutableArray array];
for (NSString *item in counts) {
```

```
    if ([counts[item] integerValue] < 2) {
        [toRemove addObject:item];
    }
}
for (NSString *item in toRemove) {
    [counts removeObjectForKey:item];
}
```

Помним ловушку: удалять из `counts` прямо в `for-in` нельзя — упадёт. Поэтому сначала собираем ключи-кандидаты в `toRemove`, и только вторым проходом удаляем их из словаря.

Полная программа

Все пять шагов вместе — это файл `code/17-inventory.m`:

```
#import <Foundation/Foundation.h>

int main(void) {
    @autoreleasepool {
        /* Журнал продаж за день: имена товаров, с повторами. */
        NSArray *sales = @[ @"кофе", @"чай", @"кофе", @"кофе",
                             @"сок", @"чай", @"кофе" ];
        NSLog(@"всего продаж: %lu", (unsigned long)[sales count]);

        /* Шаг 1. Подсчёт: имя товара → сколько раз продан. */
        NSMutableDictionary *counts = [NSMutableDictionary dictionary];
        for (NSString *item in sales) {
            NSNumber *prev = counts[item];          /* nil, если впервые */
            NSInteger n = prev ? [prev integerValue] : 0;
            counts[item] = @(n + 1);
        }
        for (NSString *item in counts) {
            NSLog(@"  %@: %@ шт.", item, counts[item]);
        }

        /* Шаг 2. Уникальные товары — это просто ключи словаря. */
        NSSet *unique = [NSSet setWithArray:[counts allKeys]];
        NSLog(@"разных товаров: %lu", (unsigned long)[unique count]);
    }
}
```

```
/* Шаг 3. Категории проданного – множество без повторов. */
NSDictionary *category = @{
    @"кофе" : @"горячее",
    @"чай" : @"горячее",
    @"сок" : @"холодное",
};
NSMutableSet *soldCats = [NSMutableSet set];
for (NSString *item in unique) {
    [soldCats addObject:category[item]];
}
NSLog(@"категории: %@",
      [[soldCats allObjects] componentsJoinedByString:@" "]);

/* Шаг 4. Рейтинг: ключи, отсортированные по убыванию продаж. */
NSArray *top = [[counts allKeys] sortedArrayUsingComparator:
    ^NSComparisonResult(id a, id b) {
        return [counts[b] compare:counts[a]]; /* по убыванию */
    }];
NSLog(@"--- рейтинг ---");
for (NSString *item in top) {
    NSLog(@"  %@ – %@", item, counts[item]);
}

/* Шаг 5. Убрать «однодневков» (продано < 2). Менять словарь
прямо в for-in нельзя – сначала собираем ключи, потом
удаляем отдельным проходом. */
NSMutableArray *toRemove = [NSMutableArray array];
for (NSString *item in counts) {
    if ([counts[item] integerValue] < 2) {
        [toRemove addObject:item];
    }
}
for (NSString *item in toRemove) {
    [counts removeObjectForKey:item];
}
NSLog(@"осталось ходовых: %@",
      [[counts allKeys] componentsJoinedByString:@" "]);
}
```

```
    return 0;
}
```

Проверяем (учёт товаров)

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/17-inventory.m -o /tmp/t && /tmp/t
```

Реальный вывод:

```
всего продаж: 7
    кофе: 4 шт.
    чай: 2 шт.
    сок: 1 шт.
разных товаров: 3
категории: горячее, холодное
--- рейтинг ---
    кофе — 4
    чай — 2
    сок — 1
осталось ходовых: кофе, чай
```

Семь продаж превратились в подсчёт по товарам, три уникальных товара свелись к двум категориям, рейтинг встал по убыванию, а «сок» с единственной продажей вылетел из списка ходовых. Три контейнера сделали ровно то, для чего каждый придуман.

Как это работает

Чем три контейнера отличаются внутри:

NSArray	[0]→объект [1]→объект [2]→объект	порядок важен, повторы можно
NSDictionary	hash(ключ) → корзина → значение	поиск по ключу, ключи уникальны
NSSet	hash(объект) → «есть/нет»	только наличие, без порядка

Массив — пронумерованные ячейки со ссылками на объекты. Словарь и множество построены на **хешировании**: спрашивают у ключа/элемента число hash, по нему находят нужную «корзину» и там сравнивают через isEqual:. Поэтому проверка членства у них почти мгновенна и не зависит от размера, а взамен они не хранят порядок — оттого allKeys и перебор множества выдают элементы «как попало».

copy vs mutableCopy у коллекций

Коллекции, как и строки (про copy — глава 8), отвечают на copy и mutableCopy:

- copy у коллекции возвращает **неизменяемую** версию (NSArray, NSDictionary, NSSet);
- mutableCopy — **изменяемую** (NSMutableArray и т.д.), даже если оригинал был неизменяемым. Именно так мы выше делали [safeA mutableCopy], чтобы получить множество, которое не жалко портить.

Важнейшая оговорка: копия **поверхностная** (shallow). Копируется сам контейнер и его ссылки на элементы — но **не сами элементы**. После mutableCopy у тебя два массива, но объекты внутри — одни и те же:

```
NSArray *orig = @[ person1, person2 ];
NSMutableArray *dup = [orig mutableCopy];
// dup — другой массив, но dup[0] и orig[0] — ОДИН и тот же person1
```

Изменишь сам person1 — изменение «видно» через оба массива, ведь это один объект. Добавишь объект в dup — orig не дрогнет, это разные контейнеры. Для ключей словаря это безопасно (ключи и так копируются), а вот за значениями и элементами помни: их разделяют, не дублируют. Если нужна копия с дублированием элементов, есть [[NSArray alloc] initWithArray:orig copyItems:YES] — он пошлёт copy каждому элементу, но только на один уровень вглубь. Полную **глубокую** копию вложенных коллекций делают через архивацию (глава 19).

Частые ошибки

- **Положил nil в коллекцию.** addObject:nil, setObject:nil forKey:... — мгновенный краш. Для «пусто» используй [NSNull null].
- **Меняешь коллекцию в for-in.** NSGenericException ... was mutated while being enumerated. Собирай изменения в отдельный список или иди по индексам с конца.
- **Пробуешь изменить immutable.** [someNSArray addObject:...] не скомпилируется (no visible @interface for 'NSArray' declares the selector 'addObject:'), а если тип скрыт за id — упадёт при запуске. NSArray неизменяем. Нужен NSMutableArray.
- **Сравниваешь элементы через ==.** Поиск и членство в коллекциях идут по isEqual:. Свой класс-элемент без правильного isEqual:/hash будет «не находиться» в множестве и словаре. Для строк/чисел всё уже сделано.
- **Ждёшь порядок от словаря или множества.** Они неупорядочены. Нужен порядок — сортируй allKeys/allObjects или бери массив / NSOrderedSet.
- **Печатаешь кириллицу через NSLog(@"%@", array).** Получишь \U...-коды. Для людей склеивай componentsJoinedByString:.

Упражнения

1. Заведи `NSMutableArray` из пяти чисел (`NSNumber`). Удали все нечётные — сначала наивно, прямо в `for-in` (увидь краш), потом правильно, через сбор кандидатов.
2. Построй словарь «город → население» из пяти пар. Выведи только города с населением больше миллиона (перебери и сравни `integerValue`).
3. Даны два массива тегов статьи. Через `NSSet` найди общие теги, теги только первой статьи и все теги вместе.
4. Отсортируй массив строк по **убыванию** длины через `sortedArrayUsingComparator:`. Подсказка: переверни сравнение.
5. Возьми массив словарей `@{ "name":..., "price":... }`. Отфильтруй предикатом `@("price" < 100)`, затем отсортируй по `price` через `NSSortDescriptor`.
6. Через `valueForKey:` достань из массива словарей задачи 5 массив всех имён одной строкой кода.

Что мы получили

Теперь у тебя в руках три рабочих контейнера Foundation. **`NSArray`** — когда важен порядок и допустимы повторы. **`NSDictionary`** — когда ищешь по ключу. **`NSSet`** — когда главное «есть или нет» и порядок не нужен. Ты умеешь перебирать их тремя способами, сортировать блоком и дескриптором, фильтровать предикатом, и обходишь главный краш-капкан — мутацию во время перебора. И ты увидел, что коллекции хранят только объекты, копируются поверхностно и не терпят `nil`.

Дальше — глава 18 про даты, `NSData`, `NSURL` и `NSError`: как Foundation представляет время, сырые байты, адреса и ошибки. А KVC, мелькнувший здесь в `valueForKey:`, развернётся в полную силу в главе 20.

Документация Apple

- `NSArray` — developer.apple.com/documentation/foundation/nsarray
- `NSMutableArray` — developer.apple.com/documentation/foundation/nsmutablearray
- `NSDictionary` — developer.apple.com/documentation/foundation/nsdictionary
- `NSSet` — developer.apple.com/documentation/foundation/nsset
- `NSOrderedSet` — developer.apple.com/documentation/foundation/nsorderedset
- `NSPredicate` — developer.apple.com/documentation/foundation/nspredicate
- Collections Programming Topics (массивы, словари, множества, быстрый перебор и предикаты одним обзором) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/Collections/Collections.html

Глава 18. Foundation: даты, данные, URL, ошибки

Строки, числа и коллекции у нас уже есть. Но реальная программа почти сразу упирается в четыре вещи, которых в коллекциях нет: **время** («сколько дней до дедлайна?»), **сырые байты** («прочитай файл»), **адреса** («где этот файл лежит, что это за ссылка») и **ошибки** («а если файла нет?»).

Foundation закрывает всё это четвёркой классов-значений: `NSDate`, `NSData`, `NSURL` и `NSError` — плюс помощники вокруг них: `NSCalendar`, `NSDateComponents`, `NSDateFormatter`, `NSUUID`. Разберём каждый по методам, а в конце построим две живые задачи: «сколько дней до дедлайна» и «прочитай текст из файла, а если файла нет — внятно сообщи об ошибке».

Что мы сделаем

- Возьмём момент времени `NSDate`, сравним две даты и сдвинем дату.
- Разложим дату на год/месяц/день через `NSCalendar` и соберём обратно.
- Поймём, почему «прибавить 86400 секунд» — это не всегда «прибавить день».
- Превратим дату в строку и строку в дату через `NSDateFormatter` — и узнаем, почему форматтер дорого создавать.
- Поработаем с сырыми байтами `NSData`: строка ↔ байты, `base64`, файл.
- Разберём `NSURL`: ссылку на сайт и путь к файлу, их компоненты.
- Научимся принимать и создавать ошибку `NSError` через `NSError **`.
- Выдадим уникальный идентификатор `NSUUID`.

NSDate: момент времени

`NSDate` — это **один момент времени**, точка на оси. Не «25 декабря» и не «14:30» — а конкретное мгновение, одинаковое для всей планеты. Внутри это просто число: сколько секунд прошло от опорной точки.

Самый частый способ получить дату — спросить «сейчас»:

```
NSDate *now = [NSDate date];
NSLog(@"сейчас: %@", now);
```

+date — это **фабричный метод класса** (плюс + перед именем, как мы помним с главы 7): возвращает новый объект-момент «прямо сейчас».

```
сейчас: 2026-06-28 07:39:19 +0000
```

Дата печатается в формате UTC (+0000) — всемирное время, без поправки на твой часовой пояс. Примеры в главе запускались в Алматы (UTC+5): на часах было 12:39, а напечаталось 07:39. Это не ошибка, просто пояс смещён на пять часов; у тебя разница будет своя. Понятную локальную строку мы получим позже через форматтер.

Отличие от Си. В Си время живёт в <time.h>: time_t — это целое число секунд, а struct tm — разбитая на поля дата, и всё это ты крутишь руками функциями time(), localtime(), mktime(). Ни объекта, ни методов, ни понятия часового пояса «из коробки». Foundation даёт NSDate как полноценный объект и отдельный NSCalendar, который знает про пояса, високосные годы и переход на летнее время.

Секунды: с эпохи и от «сейчас»

У момента можно спросить два числа.

```
NSLog(@"с 1970: %.0f секунд", [now timeIntervalSince1970]);
```

- timeIntervalSince1970 — сколько секунд прошло с **1 января 1970 года**, это опорная точка Unix (так называемая «эпоха»). Возвращает NSTimeInterval — а это просто double, число секунд с дробной частью. Печатаем как %.0f (вещественное без знаков после запятой).
- timeIntervalSinceNow — сколько секунд от текущего момента до этой даты. Для будущей даты число положительное, для прошлой — отрицательное.

```
с 1970: 1782632360 секунд
```

Арифметика дат

Сдвинуть момент вперёд или назад — это dateByAddingTimeInterval:. Он **не меняет** исходную дату (NSDate неизменяем), а возвращает новую.

```
NSDate *inHour = [now dateByAddingTimeInterval:3600];
NSDate *inDay = [now dateByAddingTimeInterval:24 * 3600];
```

Аргумент — секунды. Час это 3600 секунд, сутки — 24 * 3600. Отрицательное число сдвигает в прошлое.

А чтобы узнать, сколько между двумя моментами, есть `timeIntervalSinceDate::`

```
NSTimeInterval gap = [inHour timeIntervalSinceDate:now];  
NSLog(@"разница: %.0f секунд", gap);    /* 3600 */
```

Сравнение дат

Сравнивать даты оператором `<` нельзя — это объекты, в руках у нас указатели (см. главу 16, где мы то же говорили про `NSNumber`). Для сравнения есть `compare::`

```
if ([now compare:inDay] == NSOrderedAscending) {  
    NSLog(@"now раньше, чем inDay");  
}
```

`compare:` возвращает одно из трёх значений перечисления:

- `NSOrderedAscending` — получатель **раньше** аргумента (`now < inDay`);
- `NSOrderedDescending` — получатель **позже**;
- `NSOrderedSame` — моменты совпадают.

Это те же три константы, которыми сравнивались строки и числа — единый язык сравнения во всём Foundation. Есть и короткие помощники:

```
NSDate *earlier = [now earlierDate:inDay];    /* вернёт более раннюю */
```

`earlierDate:` отдаёт ту из двух дат, что раньше; есть парный `laterDate:`. Для проверки «это тот же момент?» — `isEqualToDate:`.

Всё это лежит в файле `code/18-date.m` (там же — форматтер, до него дойдём ниже, его две строки вывода покажем там). Реальный вывод (дата и время будут твои — это «сейчас» твоего запуска):

```
сейчас: 2026-06-28 07:39:19 +0000  
с 1970: 1782632360 секунд  
через час: 2026-06-28 08:39:19 +0000  
через сутки: 2026-06-29 07:39:19 +0000  
разница: 3600 секунд  
now раньше, чем inDay  
раньше из двух: 2026-06-28 07:39:19 +0000
```

NSCalendar и NSDateComponents: год, месяц, день

`NSDate` ничего не знает про «месяцы» и «дни недели» — он просто число секунд. Чтобы разложить момент на привычные поля или собрать дату из полей, нужен **календарь** `NSCalendar`. Он

знает правила: сколько дней в феврале этого года, когда переводят часы, с какого дня начинается неделя.

```
NSCalendar *cal = [NSCalendar currentCalendar];
NSDate *now = [NSDate date];
```

+currentCalendar — календарь с настройками системы (григорианский, твой часовой пояс, ТВОЯ локаль).

Разложить дату на компоненты

```
NSCalendarUnit units = NSCalendarUnitYear | NSCalendarUnitMonth |
    NSCalendarUnitDay | NSCalendarUnitHour |
    NSCalendarUnitMinute;
NSDateComponents *c = [cal components:units fromDate:now];
NSLog(@"сегодня: %ld-%02ld-%02ld %02ld:%02ld",
    (long)c.year, (long)c.month, (long)c.day,
    (long)c.hour, (long)c.minute);
```

- NSCalendarUnit... — это **флаги-биты**, перечисляющие, какие поля нам нужны. Складываем их через | (побитовое ИЛИ) — получается «дай год, месяц, день, час и минуту». Запрашивай только нужное: лишние поля считать дороже.
- components:fromDate: возвращает NSDateComponents — простой объект- контейнер с полями .year, .month, .day и т.д. Каждое поле — NSInteger, поэтому печатаем через %ld с приведением (long). %02ld дополняет число до двух цифр нулём (получаем 06, а не 6).

```
сегодня: 2026-06-28 12:39
```

Обрати внимание: здесь уже **12:39** — местное время, потому что календарь учитывает часовой пояс. Голый NSDate печатал 07:39 в UTC.

Собрать дату из компонентов

Обратный путь: заполняем поля и просим календарь построить момент.

```
NSDateComponents *deadlineParts = [[NSDateComponents alloc] init];
deadlineParts.year = 2027;
deadlineParts.month = 1;
deadlineParts.day = 1;
deadlineParts.hour = 9;
NSDate *deadline = [cal dateFromComponents:deadlineParts];
NSLog(@"дедлайн: %@", deadline);
```

`dateFromComponents`: собирает `NSDate` из заполненных полей. Незаданные поля (минуты, секунды) считаются нулями.

```
дедлайн: 2027-01-01 04:00:00 +0000
```

Снова 04:00 вместо 09:00: мы задавали **местные** 9 утра, а `NSLog` печатает момент в UTC, и пять часов разницы пояса видны. Это один и тот же момент — просто записан в разных поясах.

Разница в днях — по календарю

Главный вопрос задачи: сколько дней между двумя датами? Не считаем секунды руками — спрашиваем календарь.

```
NSDateComponents *diff = [cal components:NSCalendarUnitDay
                                     fromDate:now
                                     toDate:deadline
                                     options:0];
NSLog(@"до дедлайна дней: %ld", (long)diff.day);
```

`components:fromDate:toDate:options:` берёт две даты и возвращает разницу в запрошенных единицах. Мы попросили только дни — получаем целое число дней между «сейчас» и дедлайном. `options:0` — без особых правил (0 значит «настроек нет»).

```
до дедлайна дней: 186
```

Почему нельзя «просто прибавить 86400 секунд»

Кажется логичным: день — это 86400 секунд, прибавил — получил завтра. Чаще всего сработает, но **в дни перехода на летнее/зимнее время сутки длятся не 24 часа**, а 23 или 25. Прибавив ровно 86400 секунд, ты промахнёшься на час, а иногда и на целую дату. Правильно — прибавлять **календарную** единицу:

```
NSDateComponents *oneDay = [[NSDateComponents alloc] init];
oneDay.day = 1;
NSDate *tomorrow = [cal dateByAddingComponents:oneDay
                                     toDate:now
                                     options:0];
```

`dateByAddingComponents:toDate:options:` говорит календарю «прибавь один день» — и тот сам учтёт перевод часов. Результат — завтра в то же местное время, даже если сегодня сутки были «кривой» длины.

```
завтра в это же время: 2026-06-29 07:39:46 +0000
```

Отличие от Си. В Си прибавление дня — это `t += 86400`; к `time_t`, и про переход на летнее время язык не думает: получишь сдвиг на час. Чтобы сделать честно, в Си надо лезть в `struct tm`, менять `tm_mday` и звать `mktime()`, который пересчитает с учётом местных правил. `NSDate` берёт всю эту бухгалтерию на себя.

NSDateFormatter: дата ↔ строка

`NSDate` печатается коряво и в UTC. Чтобы показать дату человеку или, наоборот, разобрать дату из строки, нужен **форматтер**.

```
NSDateFormatter *f = [[NSDateFormatter alloc] init];
f.locale = [NSLocale localeWithLocaleIdentifier:@"ru_RU"];
f.dateFormat = @"dd.MM.yyyy HH:mm";
NSString *text = [f stringFromDate:now];
NSLog(@"отформатировано: %@", text);
```

- `dateFormat` — шаблон. Буквы в нём — это **placeholder’ы**: `dd` — день двумя цифрами, `MM` — месяц, `yyyy` — год, `HH` — часы 0–23, `mm` — минуты. Регистр важен: `mm` это минуты, а `MM` — месяц; `HH` это 24-часовой формат, `hh` — 12-часовой. Всё, что не буква (точки, пробелы, двоеточия), вставляется как есть.
- `locale` — локаль, набор национальных правил. Без неё форматтер берёт системную, и на чужой машине результат «поедет». Для стабильного разбора фиксированных форматов задавай локаль явно; для машинных форматов (обмен с сервером, файлы) Apple советует особую локаль `en_US_POSIX` — она не зависит от настроек пользователя.
- `stringFromDate:` — превращает дату в строку по шаблону.

```
отформатировано: 28.06.2026 12:39
```

Обратный путь — `dateFromString::`

```
NSDate *parsed = [f dateFromString:@"31.12.2025 23:59"];
NSLog(@"разобрано из строки: %@", parsed);
```

```
разобрано из строки: 2025-12-31 18:59:00 +0000
```

Снова UTC при печати: местные 23:59 это 18:59 по UTC. Момент верный.

Две ловушки форматтера

1. **Создавать форматтер дорого.** Под капотом он поднимает тяжёлые локализационные данные. Если ты в цикле форматируешь тысячу дат, не создавай тысячу форматтеров — сделай один до цикла и переиспользуй.
2. **Не меняй форматтер из разных потоков.** Форматировать даты одним настроенным `NSDateFormatter` из нескольких потоков можно (с macOS 10.9 и iOS 7 это потокобезопасно), а вот менять его свойства (`dateFormat`, `locale`), пока другие потоки им пользуются, — источник плавающих багов. Настрой один раз и больше не трогай его свойства, либо заведи свой форматтер на поток.

NSData: мешок байтов

`NSData` — это **неизменяемый блок сырых байтов**. Не текст, не число — просто байты: содержимое файла, картинка, ответ из сети. Изменяемый брат — `NSMutableData`.

Строка ↔ байты

Текст превращается в байты в выбранной кодировке:

```
NSString *hello = @"Привет, NSData!";
NSData *bytes = [hello dataUsingEncoding:NSUTF8StringEncoding];
NSLog(@"строка заняла %lu байт", (unsigned long)[bytes length]);
```

- `dataUsingEncoding:` — отдаёт байты строки в указанной кодировке. `:NSUTF8StringEncoding` — UTF-8, самая распространённая кодировка текста сегодня.
- `length` — сколько байтов в блоке. Возвращает `NSUInteger`, печатаем как `%lu` с приведением (`unsigned long`).

```
строка заняла 21 байт
```

В строке 15 символов, а байтов 21. Посчитаем: «Привет» — шесть русских букв по два байта каждая, это 12, плюс девять символов «, NSData!», которые в UTF-8 занимают по одному байту. Итого 21. Это та же тема, что в главе 15: символ и байт — не одно и то же.

Обратно — `initWithData:encoding::`

```
NSString *back = [[NSString alloc] initWithData:bytes
                                     encoding:NSUTF8StringEncoding];
NSLog(@"раскодировали обратно: %@", back); /* Привет, NSData! */
```

```
раскодировали обратно: Привет, NSData!
```

Кодировка при чтении должна совпадать с той, в которой записывали — иначе получишь либо мусор, либо `nil`.

base64

base64 — способ записать любые байты обычными буквами и цифрами, чтобы их можно было вставить в текст, URL или JSON.

```
NSString *b64 = [bytes base64EncodedStringWithOptions:0];
NSData *decoded = [[NSData alloc] initWithBase64EncodedString:b64
                                                             options:0];
```

`base64EncodedStringWithOptions:` кодирует байты в текст, `initWithBase64EncodedString:options:` декодирует обратно. `options:0` — без переносов строк. Результат:

```
base64: 0J/RgNC40LLQtdGCLCB0U0RhGEh
из base64: Привет, NSData!
```

Двадцать один байт превратился в 28 символов текста — base64 всегда раздувает данные примерно на треть (каждые 3 байта кодируются четырьмя символами). Платим объёмом за возможность проташить любые байты там, где разрешён только текст.

NSData и файл

NSData умеет читать и писать файлы целиком:

```
NSString *path = @"/tmp/18-data-demo.txt";
BOOL ok = [bytes writeFile:path atomically:YES];

NSData *loaded = [NSData dataWithContentsOfFile:path];
NSLog(@"прочитали %lu байт", (unsigned long)[loaded length]);
```

- `writeFile:atomically:` — записать байты в файл. Вернёт `BOOL: YES` — получилось, `NO` — нет. `atomically:YES` означает «пиши во временный файл, потом переименуй» — так на диске никогда не окажется наполовину записанного файла.
- `dataWithContentsOfFile:` — прочитать весь файл в `NSData`. Если файла нет, вернёт `nil` (а версия с `error:` ещё и расскажет почему — об этом ниже).

```
запись в файл удалась: да
прочитали 21 байт из файла
```


NSURL: ссылки и пути к файлам

NSURL хранит адрес. Адрес бывает двух родов: ссылка на сетевой ресурс (`https://...`) и путь к файлу на диске (`file://...`). Создаются они **разными** методами — это важно.

URL ресурса

```
NSURL *site = [NSURL URLWithString:
    @"https://developer.apple.com/documentation/foundation?q=1"];
NSLog(@"scheme: %@", site.scheme);    /* https */
NSLog(@"host:   %@", site.host);      /* developer.apple.com */
NSLog(@"path:   %@", site.path);      /* /documentation/foundation */
NSLog(@"query:  %@", site.query);     /* q=1 */
```

`URLWithString:` разбирает готовую строку-ссылку и раскладывает её на части: `scheme` (протокол), `host` (домен), `path` (путь), `query` (параметры после `?`). Если строку не получается разобрать как URL, метод вернёт `nil` — проверяй результат. (Недопустимые символы вроде пробела начиная с macOS 14 и iOS 17 он уже не отвергает, а сам кодирует в `%20`; на старых системах такая строка давала `nil`.)

URL файла

Для пути на диске — отдельный метод:

```
NSURL *file = [NSURL fileURLWithPath:@"tmp/notes/today.txt"];
NSLog(@"file scheme: %@", file.scheme);    /* file */
NSLog(@"file path:   %@", file.path);      /* tmp/notes/today.txt */
NSLog(@"имя файла:   %@", file.lastPathComponent); /* today.txt */
NSLog(@"расширение:  %@", file.pathExtension); /* txt */
```

`fileURLWithPath:` помечает адрес как **файловый** — у него схема `file`. Он умеет аккуратно до-страивать пути:

```
NSURL *base = [NSURL fileURLWithPath:@"tmp/notes"];
NSURL *child = [base URLByAppendingPathComponent:@"june.txt"];
NSLog(@"составной путь: %@", child.path); /* tmp/notes/june.txt */
```

`URLByAppendingPathComponent:` приклеивает компонент пути и сам ставит нужную косую черту — лучше, чем склеивать строки руками и гадать, есть ли уже `/` на конце.

Все три куска собраны в `code/18-url.m`; в конце файл ещё печатает UUID — о нём в следующем разделе. Запускаем:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/18-url.m -o /tmp/t && /tmp/t
```

```
scheme: https
host:    developer.apple.com
path:    /documentation/foundation
query:   q=1
file scheme: file
file path:    /tmp/notes/today.txt
имя файла:    today.txt
расширение:   txt
составной путь: /tmp/notes/june.txt
UUID: 318506AF-0CC9-4967-9452-A9535DB7B901
```

Обрати внимание на `file path: /tmp/notes/today.txt` — свойство `path` отдаёт **обычный путь без схемы**, а не строку `file:///tmp/...`. Если нужна именно строка URL целиком, спрашивай `absoluteString`. Это частая путаница: `path` — для передачи в файловые API, `absoluteString` — для показа и хранения.

Почему два разных метода? `URLWithString:` ждёт строку-ссылку и относится к ней как к URL; `fileURLWithPath:` берёт обычный путь файла, сам экранирует пробелы и спецсимволы и предоставляет схему `file`. Подсунешь путь `@"/tmp/my notes/a.txt"` в `URLWithString:` — получишь URL **без схемы** (`scheme` вернёт `nil`), то есть не файловый адрес, и методы, которые ждут файловый URL, с ним работать откажутся. Для файлов всегда `fileURLWithPath:`.

NSUUID: уникальный идентификатор

Иногда нужен идентификатор, который гарантированно не совпадёт ни с каким другим — для записи в базе, имени временного файла, ключа. Это `NSUUID`:

```
NSUUID *uuid = [NSUUID UUID];
NSLog(@"UUID: %@", uuid.UUIDString);
```

`+UUID` создаёт новый случайный идентификатор, `UUIDString` отдаёт его в виде строки. Каждый запуск — свой:

```
UUID: 4B78567C-21D7-4A54-A746-369FEE76E675
```

Вероятность совпадения двух UUID настолько мала, что её считают нулевой.

NSError: как сообщают об ошибке

Дошли до самого важного — ошибок. В Foundation у метода, который может не справиться, два способа доложить о провале:

1. **Возврат** говорит, удалось или нет: метод отдаёт `BOOL` (`NO` — провал) или объект (`nil` — провал).
2. **Объект NSError** через параметр-указатель `NSError **` рассказывает *подробности*: что именно случилось.

Железное правило: **сначала смотри на возврат, и только если он сказал «провал» — читай error**. Не наоборот. После успешного вызова в `error` может лежать что угодно.

Что внутри NSError

У ошибки три части:

- `domain` — строка-домен, область, к которой относится ошибка (например `NSCocoaErrorDomain` для файлов). Делит коды на «вселенные», чтобы код 260 из одного домена не путали с 260 из другого.
- `code` — целочисленный код конкретной ошибки внутри домена.
- `userInfo` — словарь с деталями. Самый полезный ключ — `NSLocalizedStringKey`: по нему лежит человекочитаемое описание, которое отдаёт `localizedDescription`.

Отличие от Си. В Си об ошибке докладывают кодом возврата и глобальной переменной `errno`: функция вернула `-1` или `NULL`, а ты бежишь смотреть `errno` и переводишь его через `strerror()`. Один общий `errno` на всё, никакого контекста. Стил Objective-C — вернуть `BOOL/nil` и заполнить **объект NSError** по переданному указателю `NSError **`: в нём и домен, и код, и готовое текстовое описание.

Потребляем чужую ошибку: читаем несуществующий файл

Самый частый сценарий — принять ошибку от системного метода. Прочитаем файл, которого заведомо нет:

```
NSError *readErr = nil;
NSString *text = [NSString stringWithContentsOfFile:@" /tmp/нет-такого.txt"
                                     encoding:NSUTF8StringEncoding
                                     error:&readErr];

if (text == nil) {
    NSLog(@"чтение не удалось: %@", readErr.localizedDescription);
    NSLog(@" домен: %@, код: %ld", readErr.domain, (long)readErr.code);
}
```

```

} else {
    NSLog(@"прочитали: %@", text);
}

```

Разберём по косточкам:

- `NSError *readErr = nil;` — заводим переменную под ошибку, пока пустую.
- `error:&readErr` — передаём её **адрес** (значок `&` — «адрес переменной», см. главу 3). Тип параметра — `NSError **`, указатель на указатель: метод через него запишет нам объект ошибки. Если ошибка тебе не нужна, сюда можно передать `NULL`.
- Проверяем **возврат**: `text == nil` значит «не прочиталось». Только внутри этой ветки лезем в `readErr`.
- `localizedDescription` — готовая фраза для человека.

```
чтение не удалось: The file "нет-такого.txt" couldn't be opened because there is no such file.  
домен: NSCocoaErrorDomain, код: 260
```

Система сама заполнила всё: домен `NSCocoaErrorDomain`, код 260 (нет файла) и понятное описание. Нам осталось его показать.

Создаём свою ошибку

Когда твоя собственная функция хочет сообщить о провале тем же способом, ты создаёшь NSError сам и кладёшь его по переданному указателю. Договорённость по сигнатуре стандартная: последний параметр — NSError **error, возврат — BOOL или объект.

[illegible]

```

        return NO;                                /* провал: NO + заполнили error */
    }

    NSDateComponents *diff = [cal components:NSCalendarUnitDay
                                     fromDate:now toDate:target options:0];
    if (outDays) { *outDays = diff.day; }
    return YES;                                    /* успех: YES, error не трогаем */
}

```

Что здесь важно:

- `errorWithDomain:code:userInfo:` — фабрика готовой ошибки. Передаём свой домен (своя строка-«вселенная»), свой код и словарь с описанием.
- `if (error)` перед записью — обязательная проверка. Вызвавший мог передать `NULL`, если ошибка ему не нужна; писать по `NULL` нельзя.
- При провале — `*error = ...; return NO;`. При успехе — просто `return YES;`, **не трогая** `error`. В этом вся договорённость.

Строим: «сколько дней до дедлайна» и чтение файла с ошибкой

Соберём обе задачи в одну программу. Функцию `daysUntil` мы уже написали выше — берём её целиком. В `main` вызовем её с датой в прошлом (ждём ошибку) и с будущей датой (ждём число дней), потом прочитаем несуществующий файл. Полный файл — `code/18-error.m`; вот его `main`:

```

int main(void) {
    @autoreleasepool {
        /* 1. Дата в прошлом → ждём ошибку. */
        NSDate *past = [NSDate dateWithTimeIntervalSinceNow:-3600];
        NSError *err = nil;
        NSInteger days = 0;
        if (daysUntil(past, &days, &err)) {
            NSLog(@"дней до дедлайна: %ld", (long)days);
        } else {
            NSLog(@"ошибка: %@", err.localizedDescription);
            NSLog(@" домен: %@, код: %ld", err.domain, (long)err.code);
        }

        /* 2. Будущая дата → ждём число дней. */
        NSDate *future =
    }
}

```

```
        [NSDate dateWithTimeIntervalSinceNow:5 * 24 * 3600];
    NSError *err2 = nil;
    if (daysUntil(future, &days, &err2)) {
        NSLog(@"дней до будущего дедлайна: %ld", (long)days);
    }

    /* 3. Чтение несуществующего файла. */
    NSError *readErr = nil;
    NSString *text =
        [NSString stringWithContentsOfFile:@"tmp/нет-такого.txt"
                                     encoding:NSUTF8StringEncoding
                                     error:&readErr];

    if (text == nil) {
        NSLog(@"чтение не удалось: %@", readErr.localizedDescription);
        NSLog(@" домен: %@, код: %ld",
              readErr.domain, (long)readErr.code);
    } else {
        NSLog(@"прочитали: %@", text);
    }
}
return 0;
}
```

Здесь `dateWithTimeIntervalSinceNow`: — ещё один способ создать дату: сдвиг в секундах от «сейчас» (–3600 — час назад, $5 * 24 * 3600$ — пять суток вперёд). Реальный вывод:

ошибка: Дедлайн уже в прошлом.

домен: com.objcbook.deadline, код: 1

дней до будущего дедлайна: 4

чтение не удалось: The file “нет-такого.txt” couldn’t be opened because there is no such file.

домен: NSCocoaErrorDomain, код: 260

Заметь хитрость в третьей строке: мы сдвинули дату ровно на пять суток вперёд, а календарь насчитал 4 полных дня. Дело в долях: пока шла программа, «сейчас» внутри `daysUntil` отщёлкало на доли секунды вперёд, и до цели осталось чуть меньше пяти полных суток. `NSCalendarUnitDay` считает **целые** дни и округляет вниз. Хочешь честных пять — считай не моменты, а календарные дни: прогони обе даты через `[cal startOfDayForDate:]` (он вернёт полночь того же дня) и уже эти полуночи отдай в `components:fromDate:toDate:options:.`

Проверяем

Файлы главы — `code/18-date.m`, `code/18-calendar.m`, `code/18-data.m`, `code/18-url.m`, `code/18-error.m`. Каждый собирается и запускается так (подставь нужное имя):

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/18-error.m -o /tmp/t && /tmp/t
```

Весь показанный вывод — настоящий, с реального запуска. Даты, время и строка UUID у тебя будут свои: `NSDate` берёт текущий момент, а `NSUUID` выдаёт новый идентификатор при каждом запуске.

Частые ошибки

- **Сравнил даты через `<` или `==`.** Это объекты, в руках указатели. `==` сравнит адреса, а не моменты. Для порядка — `compare:`, для равенства — `isEqualToDate:`.
- **Прибавил день как `+ 86400`.** В день перевода часов сутки делятся 23 или 25 часов — промахнёшься. Прибавляй календарный день через `dateByAddingComponents:toDate:options:`.
- **Создаёшь форматтер в цикле.** Это дорого. Сделай один до цикла и переиспользуй. И не дёргай свойства одного форматтера из разных потоков.
- **Читаешь `error` без проверки возврата.** После успешного вызова в `error` может лежать мусор. Сначала `if (result == nil)` (или `== NO`), и только потом смотри ошибку.
- **Пишешь в `*error`, не проверив `error != NULL`.** Вызвавший мог не захотеть ошибку и передать `NULL` — запись по нему уронит программу.
- **Путаешь `URLWithString:` и `fileURLWithPath:`.** Путь файла через `URLWithString:` даст URL без схемы `file`, а не файловый адрес. Для файлов — `fileURLWithPath:`.
- **Раскодировал `NSData` не той кодировкой.** Записал в UTF-8, читаешь как что-то другое — получишь `nil` или кракозябры. Кодировки должны совпадать.

Упражнения

1. Возьми `[NSDate date]`, прибавь к ней 90 минут через `dateByAddingTimeInterval:` и напечатай обе даты форматтером в виде `HH:mm`.
2. Собери календарём дату своего дня рождения в этом году и через `components:fromDate:toDate:options:` посчитай, сколько дней до него (или сколько прошло).
3. Заведи один `NSDateFormatter` с форматом `yyyy-MM-dd` и в цикле отформатируй им пять дат подряд (например, пять ближайших дней). Один форматтер на все пять.
4. Закодируй строку со своим именем в `NSData`, переведи в `base64`, напечатай, потом декодируй обратно и убедись, что совпало.

5. Разбери `URLWithString:` ссылку `https://example.com:8080/a/b?x=1` и напечатай `scheme`, `host`, `port`, `path`, `query` по отдельности.
6. Напиши функцию `BOOL readFirstLine(NSString *path, NSString **outLine, NSError **error):` читает файл, кладёт первую строку в `outLine`, а при ошибке заполняет `error`. Проверь её на существующем и несуществующем файле.

Что мы получили

Теперь у тебя есть четыре рабочие лошадки Foundation. `NSDate` — момент времени, который складывается, сравнивается и сдвигается. `NSCalendar` с `NSDateComponents` раскладывает момент на год/месяц/день и собирает обратно, честно учитывая часовые пояса и перевод часов — поэтому день прибавляем календарём, а не «плюс 86400». `NSDateFormatter` переводит дату в строку и обратно — но создавать его дорого, и потоки он не любит. `NSData` — мешок байтов: строка ↔ байты, `base64`, файлы. `NSURL` хранит и сетевые ссылки, и пути файлов (помни про два разных конструктора), а `NSUUID` выдаёт неповторимый идентификатор.

И главное — `NSError`. Ты увидел обе стороны: как **принять** чужую ошибку (проверь возврат, потом читай `error.localizedDescription`) и как **создать** свою (`errorWithDomain:code:userInfo:`, заполни `*error`, верни `N0`). Этот паттерн `BOOL/nil + NSError **` пронизывает весь Foundation — дальше он будет встречаться постоянно. В следующей главе возьмёмся за файлы и файловую систему всерьёз, и `NSData`, `NSURL` и `NSError` сразу пойдут в дело вместе.

Документация Apple

- `NSDate` — developer.apple.com/documentation/foundation/nsdate
- `NSCalendar` — developer.apple.com/documentation/foundation/nscalendar
- `NSDateComponents` — developer.apple.com/documentation/foundation/nsdatecomponents
- `NSDateFormatter` — developer.apple.com/documentation/foundation/dateformatter
- `NSData` — developer.apple.com/documentation/foundation/nsdata
- `NSURL` — developer.apple.com/documentation/foundation/nsurl
- `NSError` — developer.apple.com/documentation/foundation/nSError
- `NSUUID` — developer.apple.com/documentation/foundation/nsuuid
- Date and Time Programming Guide (`NSDate`, `NSCalendar`, форматтеры) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/DatesAndTimes/DatesAndTimes.html
- Error Handling Programming Guide (паттерн `NSError` и `NSError **`) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ErrorHandlingCocoa/ErrorHandling/ErrorHandling.html

Глава 19. Foundation: файлы и сериализация

В главе 18 ты держал данные в NSData — сыром мешке байтов — и в коллекциях. Но как только программа закрылась, всё это исчезло: жило только в памяти. Чтобы данные пережили запуск, их надо положить **на диск**, в файл, а потом прочитать обратно.

В чистом Си файл — это поток байтов: открыл `fopen`, накидал байтов `fwrite`, закрыл `fclose`, а формат придумай и разбери сам. Foundation поднимает планку: он умеет сохранять **целые объекты и коллекции** одним вызовом — в plist, в JSON, в архив. Ты говоришь «запиши этот массив словарей в файл» — и он сам решает, как разложить его по байтам, и сам соберёт обратно.

Разберём четыре слоя работы с диском: управление файлами (`NSFileManager`), чтение/запись строк и данных, сериализацию коллекций (plist и JSON) и архивацию своих объектов (`NSSecureCoding`). В конце построим список задач, сохраним его в JSON-файл и поднимем назад, а потом научим свой класс `Task` сохранять самого себя.

Что мы сделаем

- Найдём временную папку и поработаем в ней через `NSFileManager`: создадим папку, файлы, спросим существование и атрибуты, уберём за собой.
- Запишем строку в файл и прочитаем обратно; то же самое с сырыми байтами `NSData`. Разберёмся, зачем `atomically`.
- Сохраним массив словарей в **property list** и в **JSON**, прочитаем назад, поймем битый JSON ошибкой, а не крахом.
- Сделаем класс `Task`, поддерживающий `NSSecureCoding`, заархивируем его в `NSData` и в файл и распакуем обратно.

Все примеры пишут только во временную директорию и сами удаляют за собой файлы — после них на диске ничего не остаётся.

Отличие от Си. В Си работа с файлом — это `FILE *f = fopen(...)`, затем `fread/fwrite` байтами и `fclose`. Ты сам придумываешь формат: где число, где строка, где конец записи. Foundation идёт выше: один вызов `writeToFile:` или `dataWithJSONObject:` — и целая коллекция объектов ложится на диск в готовом формате, а обратный вызов поднимает её как живые объекты. Байтами вручную ворочать почти не приходится.

NSFileManager: работа с файлами и папками

`NSFileManager` — класс, через который мы спрашиваем файловую систему: существует ли файл, создай папку, удали, перечисли содержимое, дай атрибуты. Сам по себе он не хранит файлы — это посредник между твоим кодом и диском.

Берут его почти всегда так:

```
NSFileManager *fm = [NSFileManager defaultManager];
```

`defaultManager` — **общий экземпляр** (shared instance): один на всю программу, создавать свой обычно незачем. Это знакомый по Foundation приём — как `[NSNotificationCenter defaultCenter]`.

Куда вообще можно писать

Приложению для iPhone или из Mac App Store нельзя писать куда попало: оно живёт в песочнице, и система разрешает только определённые папки. Наша консольная программа песочницы не имеет, но хорошая привычка одна на всех — спрашивать нужную папку у системы. Две, которые нужны чаще всего:

```
NSString *tmp = NSTemporaryDirectory();
```

`NSTemporaryDirectory()` — это **функция** (не метод!), возвращает путь к временной папке текущего пользователя (у приложения в песочнице — своей для этого приложения). Туда можно свободно писать черновики; система вправе их подчистить, когда захочет, поэтому ничего ценного там не храни. Для учебных примеров это идеальное место: намусорил — не страшно.

Для данных, которые должны жить долго, есть папка **Documents**:

```
NSArray<NSString*> *dirs = NSSearchPathForDirectoriesInDomains(  
    NSDocumentDirectory, NSUserDomainMask, YES);  
NSString *docs = dirs.firstObject;
```

`NSSearchPathForDirectoriesInDomains` — функция, которая ищет системные папки. Первый аргумент — *какую* (`NSDocumentDirectory` — Documents), второй — *где* (`NSUserDomainMask` — в домаш-

ней папке пользователя), третий — раскрывать ли ~ в полный путь (YES). Возвращает массив путей; берём первый.

Современный способ — через URL и сам `NSFileManager`:

```
NSURL *docsURL = [fm URLForDirectory:NSDocumentDirectory
                               inDomain:NSUserDomainMask
                               appropriateForURL:nil
                               create:YES
                               error:NULL];
```

`URLForDirectory:...` возвращает `NSURL` вместо строки-пути и при `create:YES` создаёт папку, если её нет. В реальных приложениях предпочитают именно URL-вариант, но для примеров нам хватит временной папки и строк-путей.

Отличие от Си. В Си ты просто пишешь `fopen("/tmp/file.txt", "w")` — любой путь, какой указал, и за права отвечаешь ты сам. На iOS и в Mac App Store приложение живёт в **песочнице** (sandbox): чужие папки для него закрыты, а «правильные» (Documents, tmp) надо спросить у системы этими функциями, а не хардкодить путь.

Собираем путь из кусочков

Склеивать пути вручную через / — плохая привычка: легко получить двойной слэш или потерять разделитель. У `NSString` для этого есть методы работы с путями:

```
NSString *dir = [tmp stringByAppendingPathComponent:@"objc-fm-demo"];
NSString *file = [dir stringByAppendingPathComponent:@"a.txt"];
```

`stringByAppendingPathComponent:` приклеивает компонент пути, сам ставя ровно один разделитель. Обратные методы: `lastPathComponent` (имя файла из пути), `pathExtension` (расширение), `stringByDeletingLastPathComponent` (папка-родитель).

Создаём папку

```
NSError *err = nil;
BOOL ok = [fm createDirectoryAtPath:dir
                      withIntermediateDirectories:YES
                      attributes:nil
                      error:&err];
```

- `createDirectoryAtPath:` — путь создаваемой папки.
- `withIntermediateDirectories:YES` — создать заодно все недостающие промежуточные папки. С NO метод упадёт, если родителя нет.

- `attributes:nil` — права/атрибуты по умолчанию.
- `error:&err` — знакомый по главе 18 приём: метод вернёт `BOOL`, а при `NO` положит причину в `err`. Проверяем именно возвращённый `BOOL`, а на `err` смотрим только когда он `NO`.

Существование, файл или папка

```
BOOL there = [fm fileExistsAtPath:file];

BOOL isDir = NO;
[fm fileExistsAtPath:dir isDirectory:&isDir];
```

`fileExistsAtPath:` отвечает YES/NO — есть ли что-то по пути. Версия `fileExistsAtPath:isDirectory:` заодно через указатель `&isDir` скажет, папка это (YES) или обычный файл (NO).

Список содержимого папки

```
NSArray<NSString *> *items =
    [fm contentsOfDirectoryAtPath:dir error:NULL];
```

Возвращает массив **имён** (не полных путей!) внутри папки, без `.` и `...`. Порядок не гарантирован — если нужен предсказуемый, сортируй сам (`sortedArrayUsingSelector:@selector(compare:)`)).

Атрибуты файла

```
NSDictionary<NSFileAttributeKey, id> *attrs =
    [fm attributesOfItemAtPath:file error:NULL];
unsigned long long size = [attrs[NSFileSize] unsignedLongLongValue];
NSDate *mtime = attrs[NSFileModificationDate];
```

`attributesOfItemAtPath:error:` возвращает словарь, где по ключам-константам лежат сведения о файле: `NSFileSize` (размер в байтах, завернут в `NSNumber`), `NSFileModificationDate` (дата изменения, `NSDate`), `NSFileType`, `NSFilePosixPermissions` и другие.

Удаление

```
[fm removeItemAtPath:dir error:&err];
```

Удаляет файл или **папку целиком вместе с содержимым**. Возвращает `BOOL`. После него можно проверить `fileExistsAtPath:` — вернёт `NO`.

Соберём всё это в один пример. Он создаёт во временной папке свою подпапку, кладёт три файла, перечисляет их, смотрит атрибуты и убирает за собой.

Полный файл — `code/19-filemanager.m`. Реальный вывод (пути и время — свои):

```
временная папка: /var/folders/km/.../T/
наша папка:      /var/folders/km/.../T/objc-fm-demo
a.txt на месте? 1
z.txt на месте? 0
наш путь — папка? 1
в папке: a.txt, b.txt, note.md
a.txt: размер=15 байт, изменён=2026-06-28 07:40:12 +0000
папку удалили
папка ещё есть? 0
```

1 и 0 — это BOOL в формате %d (YES=1, NO=0). Видно, что после `removeItemAtPath:` папки больше нет.

Чтение и запись строк и данных

Самый частый случай — сохранить строку текста и прочитать её обратно. У `NSString` это один метод в каждую сторону.

Записать строку

```
NSString *text = @"Первая строка\nВторая строка\n";
NSError *err = nil;
BOOL ok = [text writeToFile:path
                    atomically:YES
                    encoding:NSUTF8StringEncoding
                    error:&err];
```

- `writeToFile:` — путь файла.
- `atomically:YES` — **писать атомарно**. Метод сначала пишет во временный файл рядом, и только когда всё легло — одним движением переименовывает его поверх целевого. Если на середине записи программу убьют или сядет батарея, старый файл останется целым, а не превратится в полузаписанный огрызок. Цена — нужно временно вдвое больше места. Для важных данных всегда YES.

- `encoding:NSUTF8StringEncoding` — **как переводить символы в байты**. UTF-8 — стандарт, бери его по умолчанию. Кодировка при чтении должна совпасть, иначе кириллица превратится в кашу.
- `error:&err` — причина, если вернулся NO.

Отличие от Си. В Си запись строки — это `fputs(text, f)` или `fwrite`, и понятия «кодировка» у тебя нет: пишутся ровно те байты, что лежат в `char *`. Что это было — UTF-8, latin-1 или мусор — Си не знает и не проверит. `NSString` всегда хранит *символы*, а при записи ты явно говоришь, в какие байты их перевести. Поэтому кириллица, эмодзи и иероглифы сохраняются корректно и предсказуемо.

Прочитать строку

```
NSString *back = [NSString stringWithContentsOfFile:path
                                     encoding:NSUTF8StringEncoding
                                     error:&err];
```

Читает весь файл и собирает из байтов `NSString`. При сбое (нет файла, не та кодировка) вернёт `nil` и заполнит `err`. Проверь именно на `nil`, а не на `err`.

Сырые байты: NSData

Когда содержимое не текст (картинка, архив, что угодно), работаем с `NSData` — мешком байтов из главы 18:

```
NSData *data = [text dataUsingEncoding:NSUTF8StringEncoding];
[data writeToFile:binPath atomically:YES];

NSData *raw = [NSData dataWithContentsOfFile:binPath];
NSString *fromData = [[NSString alloc] initWithData:raw
                                     encoding:NSUTF8StringEncoding];
```

- `dataUsingEncoding:` — строка → байты.
- `writeToFile:atomically:` у `NSData` — то же атомарное сохранение, но кодировка не нужна (байты и так байты).
- `dataWithContentsOfFile:` — прочитать файл в `NSData`.
- `initWithData:encoding:` — собрать строку из байтов обратно.

Пример `code/19-readwrite.m` пишет строку и в текстовый файл, и в бинарный, читает обоими способами и сверяет. Вывод:

```
строку записали в objc-gw.txt
прочитали символов: 43
совпало с исходным? 1
первая строка файла: Первая строка
в строке 78 байт (длиннее символов из-за кириллицы)
прочитали 78 байт, байты совпали? 1
из байтов восстановили первую строку: Первая строка
временные файлы удалены
```

Обрати внимание: **43 символа, но 78 байт**. Кириллическая буква в UTF-8 занимает два байта, а перевод строки и латиница — по одному. Вот почему `length` строки и `length` её `NSData` — разные числа (об этом была речь в главе 15).

Property list: сохраняем коллекции

Часто на диск нужно положить не одну строку, а целую структуру: массив, словарь, вложенные коллекции. Для этого у Apple есть формат **property list** (сокращённо **plist**) — текстовый (XML) или бинарный файл, в котором живут именно коллекции и простые значения.

Самое удобное: `NSArray` и `NSDictionary` умеют писать себя в plist **одним вызовом**.

```
NSArray<NSDictionary *> *tasks = @[
    @{ @"title": @"Купить кофе", @"done": @YES, @"priority": @1 },
    @{ @"title": @"Написать главу", @"done": @NO, @"priority": @3 },
];

NSURL *url = [NSURL fileURLWithPath:path];
NSError *err = nil;
BOOL ok = [tasks writeToURL:url error:&err];
```

`writeToURL:error:` сериализует массив в plist и пишет в файл. Обратное:

```
NSArray *back = [NSArray arrayWithContentsOfURL:url error:&err];
```

`fileURLWithPath:` превращает строку-путь в файловый `NSURL` — именно его ждут современные методы `writeToURL:` / `arrayWithContentsOfURL:`. (Старые `writeToFile:atomically:` и `arrayWithContentsOfFile:` тоже существуют, но без `error:`, поэтому новые предпочтительнее.)

Главное ограничение plist: внутри могут лежать только **plist-совместимые типы**. Их ровно шесть: `NSString`, `NSNumber`, `NSDate`, `NSData`, `NSArray`, `NSDictionary` (ключи словаря — только строки). Сунешь туда свой класс `Task` или `NSNull` — запись провалится: `writeToURL:error:` вернёт `NO` и положит причину в `err`. Для своих объектов нужна архивация (следующий раздел).

Под капотом всем этим заведует класс **NSPropertyListSerialization** — низкоуровневый мотор, который превращает коллекцию в NSData нужного формата и обратно:

```
NSData *plistData = [NSPropertyListSerialization
    dataWithPropertyList:tasks
        format:NSPropertyListXMLFormat_v1_0
        options:0
        error:&err];

NSArray *obj = [NSPropertyListSerialization
    propertyListWithData:plistData
        options:NSPropertyListImmutable
        format:NULL
        error:&err];
```

`dataWithPropertyList:format:options:error:` даёт байты (можно выбрать XML- или бинарный формат), `propertyListWithData:...` разбирает их обратно. Методы `writeToURL:` у коллекций — просто удобная обёртка над ним. Знать этот класс полезно, когда нужен контроль над форматом или сериализация в память, а не в файл.

JSON: NSJSONSerialization

JSON — самый ходовой формат обмена данными между программами и серверами. Foundation работает с ним через класс **NSJSONSerialization**: два метода, туда и обратно.

Объект → JSON

```
NSData *json = [NSJSONSerialization
    dataWithJSONObject:tasks
        options:NSJSONWritingPrettyPrinted
        error:&err];
```

`dataWithJSONObject:options:error:` берёт коллекцию и возвращает NSData с её JSON-представлением. `options:`

- 0 — компактный JSON в одну строку;
- `NSJSONWritingPrettyPrinted` — с отступами, удобно читать глазами;
- `NSJSONWritingSortedKeys` — ключи по алфавиту (стабильный вывод).

Что можно превращать в JSON. Корень — NSArray или NSDictionary. Значения — NSString, NSNumber (числа и @YES/@NO → true/false), вложенные массивы/словари и NSNull (станет null).

Ключи словаря — только строки. `NSDate`, `NSData` или свой класс напрямую не лезут — их сначала надо привести к строке/числу.

Осторожно: здесь `NSJSONSerialization` ведёт себя не так, как при чтении. Неподходящий объект внутри коллекции — это не `nil` с ошибкой, а **исключение** `NSInvalidArgumentException` («Invalid type in JSON write»), и программа падает. Если не уверен в данных, спроси заранее: `[NSJSONSerialization isValidJSONObject:tasks]` вернёт `YES` или `NO`.

JSON → объект

```
NSData *raw = [NSData dataWithContentsOfFile:path];
NSArray *parsed = [NSJSONSerialization JSONObjectWithData:raw
                                                         options:0
                                                         error:&err];
```

`JSONObjectWithData:options:error:` разбирает байты JSON в коллекцию. Что в JSON было объектом `{}` — станет `NSDictionary`, массивом `[]` — `NSArray`, и так далее. `options:0` — обычный режим (получаем неизменяемые коллекции). Если нужно потом менять результат, передай `NSJSONReadingMutableContainers`.

Ошибки — через возврат, не через краш

Это ключевое отличие от ручного разбора. Дай методу битый JSON — он не уронит программу, а вернёт `nil` и положит причину в `err`:

```
NSData *broken = [@"{ это не json }"
                 dataUsingEncoding:NSUTF8StringEncoding];
id bad = [NSJSONSerialization JSONObjectWithData:broken
                                              options:0
                                              error:&err];
// bad == nil, err.code == 3840 (ошибка разбора)
```

Всегда проверяй результат на `nil` перед тем, как им пользоваться: данные из сети или из чужого файла ломаются регулярно.

Отличие от Си. В Си «разобрать JSON» означает либо писать парсер руками (символ за символом), либо тащить стороннюю библиотеку — а потом следить, чтобы битый ввод не привёл к выходу за границы массива и краху. `NSJSONSerialization` встроен, разбирает любой корректный JSON одним вызовом, а на некорректном аккуратно возвращает `nil` с описанием ошибки. Никакой ручной работы с байтами и никаких сегфолтов.

Строим с нуля: список задач в JSON-файл

Соберём всё вместе. Задача: есть список дел — **массив словарей**. Сохраним его в JSON-файл во временной папке, прочитаем обратно и поработаем с прочитанными данными.

Шаг 1. Данные

```
NSArray<NSDictionary *> *tasks = @[
    @{ @"title": @"Купить кофе",    @"done": @YES, @"priority": @1 },
    @{ @"title": @"Написать главу", @"done": @NO,  @"priority": @3 },
    @{ @"title": @"Прогуляться",    @"done": @NO,  @"priority": @2 },
];
```

Каждая задача — словарь с тремя ключами. Все значения JSON-совместимы: строка, булево @YES/@NO (это NSNumber), число. Корень — массив.

Шаг 2. В файл и обратно

```
NSData *json = [NSJSONSerialization
                dataWithJSONObject:tasks
                options:NSJSONWritingPrettyPrinted
                error:&err];
[json writeToFile:jsonPath atomically:YES];

NSData *raw = [NSData dataWithContentsOfFile:jsonPath];
NSArray *parsed = [NSJSONSerialization JSONObjectWithData:raw
                options:0
                error:&err];
```

Превратили массив в байты, записали, прочитали байты назад, разобрали в массив. Четыре вызова — и данные совершили круг через диск.

Шаг 3. Работаем с прочитанным

parsed — обычный NSArray словарей, с ним работают приёмы из главы 17. Отберём невыполненные и отсортируем по приоритету:

```
NSArray *open = [parsed filteredArrayUsingPredicate:
    [NSPredicate predicateWithFormat:@"done == NO"]];
NSArray *sorted = [open sortedArrayUsingDescriptors:@[
    [NSSortDescriptor sortDescriptorWithKey:@"priority" ascending:NO] ]];
```

Полный файл — `code/19-json.m` (там же показан plist и ловля битого JSON). Вывод:

```
plist: прочитали задач: 3, первая: Купить кофе
```

```
JSON-файл: 269 байт
```

```
содержимое файла:
```

```
[
  {
    "done" : true,
    "priority" : 1,
    "title" : "Купить кофе"
  },
  {
    "done" : false,
    "priority" : 3,
    "title" : "Написать главу"
  },
  {
    "done" : false,
    "priority" : 2,
    "title" : "Прогуляться"
  }
]
```

```
невыполненные задачи по приоритету:
```

```
  [3] Написать главу
```

```
  [2] Прогуляться
```

```
битый JSON -> nil (код ошибки 3840)
```

```
временные файлы удалены
```

@YES/@NO стали true/false, кириллица сохранилась, а битый JSON честно вернул nil. Список задач пережил запись на диск и чтение обратно.

Архивация объектов: NSCoder

JSON и plist хороши для коллекций простых значений. Но как сохранить **свой объект** — экземпляр класса Task со всеми его полями? Превращать его руками в словарь и обратно утомительно и легко ошибиться. Foundation предлагает **архивацию**: объект сам описывает, как себя записать и как восстановиться.

Механизм построен на протоколе **NSCoding** (его строгий потомок — **NSSecureCoding**) и архиваторе **NSKeyedArchiver**.

Учим класс кодировать себя

Чтобы объект можно было заархивировать, его класс принимает протокол `NSSecureCoding` и реализует два метода: «запиши себя» и «восстанови себя».

```
@interface Task : NSObject <NSSecureCoding>
@property (nonatomic, copy) NSString *title;
@property (nonatomic, assign) NSInteger priority;
@property (nonatomic, assign) BOOL done;
@end

+ (BOOL)supportsSecureCoding { return YES; }

- (void)encodeWithCoder:(NSCoder *)coder {
    [coder encodeObject:self.title forKey:@"title"];
    [coder encodeInteger:self.priority forKey:@"priority"];
    [coder encodeBool:self.done forKey:@"done"];
}

- (instancetype)initWithCoder:(NSCoder *)coder {
    self = [super init];
    if (self) {
        _title = [coder decodeObjectOfClass:[NSString class] forKey:@"title"];
        _priority = [coder decodeIntegerForKey:@"priority"];
        _done = [coder decodeBoolForKey:@"done"];
    }
    return self;
}
```

Разберём по частям.

- **+ (BOOL)supportsSecureCoding { return YES; }** — обязательное согласие класса на «безопасное» разархивирование. Без него `NSKeyedUnarchiver` откажется работать в безопасном режиме.
- **encodeWithCoder:** — здесь объект записывает каждое поле под собственным **ключом** (строкой). `encodeObject:forKey:` — для объектов, `encodeInteger:forKey:` — для целых, `encodeBool:forKey:` — для булевых. Есть и `encodeDouble:`, `encodeInt:` и другие под разные типы.
- **initWithCoder:** — обратный метод: достаёт поля по тем же ключам. `decodeIntegerForKey:` и `decodeBoolForKey:` возвращают значение напрямую. А для объекта используется **decodeObjectOfClass:forKey:** — и вот тут проявляется «безопасность» `NSSecureCoding`: ты заранее

объявляешь, *какого класса* объект ожидаешь (NSString). Если в архиве под этим ключом окажется что-то другое (например, подменённый злонамеренный объект), декодирование провалится, а не создаст что попало.

Отличие от Си. В Си нет понятия «объект сам себя сохраняет». Чтобы положить структуру на диск, ты вручную пишешь каждое её поле: fwrite числа, fwrite длины строки, fwrite байтов строки — и так же вручную читаешь обратно в строгом порядке. Один сдвиг или забытое поле — и весь файл рассыпается. Здесь объект сам перечисляет поля по **именованным ключам**: порядок не важен, новое поле добавил — старые архивы всё ещё читаются (просто новый ключ вернёт nil/0).

Архивируем и восстанавливаем

Когда класс умеет кодироваться, превратить объект в байты — один вызов:

```
NSError *err = nil;
NSData *data = [NSKeyedArchiver archivedDataWithRootObject:task
                                     requiringSecureCoding:YES
                                     error:&err];
```

archivedDataWithRootObject:requiringSecureCoding:error: берёт объект (корень графа) и возвращает NSData — готовый архив. requiringSecureCoding:YES требует, чтобы класс поддерживал NSSecureCoding (наш поддерживает). Эти байты можно писать в файл как любой NSData:

```
[data writeToFile:path atomically:YES];
```

Обратно:

```
NSData *raw = [NSData dataWithContentsOfFile:path];
Task *back = [NSKeyedUnarchiver unarchivedObjectOfClass:[Task class]
                                     fromData:raw
                                     error:&err];
```

unarchivedObjectOfClass:fromData:error: поднимает объект из байтов. Ты снова указываешь **ожидаемый класс корня** ([Task class]) — та же защита. Если архив повреждён или класс не тот, вернётся nil и заполнится err.

Защита в действии

Попробуем распаковать наш архив Task как NSDate — заведомо не тот класс:

```
Task *wrong = [NSKeyedUnarchiver unarchivedObjectOfClass:[NSDate class]
                                     fromData:raw
```

```
error:&err];  
// wrong == nil, err != nil – аккуратная ошибка, не краш
```

Полный пример — code/19-archive.m. Вывод:

```
исходный: <Task 'Заархивировать объект' p5 готово>  
архив занял 284 байт  
восстановили: <Task 'Заархивировать объект' p5 готово>  
title совпал? 1  
распаковка как NSDate -> nil (есть ошибка? 1)  
архивный файл удалён
```

Объект совершил круг: Task → NSData → файл → NSData → Task, и все три поля вернулись на место. А попытка прочитать его «не тем классом» аккуратно дала nil с ошибкой.

Какой формат когда

- **Строка/байты** (writeToFile:/NSData) — простой текст, лог, готовый файл. Никакой структуры.
- **Property list** — коллекции из встроенных типов, настройки, мелкие структуры. Только plist-совместимые типы, родной формат Apple.
- **JSON** — обмен с сетью и другими программами. Коллекции простых значений, читается всем миром.
- **Архив (NSKeyedArchiver)** — свои объекты со своими полями, графы объектов со связями. Формат «для своих»: читается в основном кодом на Apple-платформах.

Проверяем

Все четыре файла собираются и запускаются так (имя меняй):

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \  
code/19-filemanager.m -o /tmp/t && /tmp/t
```

Файлы главы:

- code/19-filemanager.m — папки, файлы, атрибуты, удаление.
- code/19-readwrite.m — строки и NSData в файл и обратно.
- code/19-json.m — plist и JSON, список задач через диск.
- code/19-archive.m — класс Task с NSSecureCoding, архивация.

Каждый собирается без предупреждений, пишет только во временную папку и сам удаляет за собой файлы. Дата, время и пути в выводе будут свои.

Частые ошибки

- **Не та кодировка при чтении.** Записал в `NSStringEncoding`, читаешь в другой — кириллица превратится в мусор или чтение вернёт `nil`. Пиши и читай в одной кодировке, по умолчанию UTF-8.
- **Не проверил результат.** `stringWithContentsOfFile:`, `JSONObjectWithData:`, `unarchivedObjectOfClass:` возвращают `nil` при сбое. Используешь результат без проверки — поедешь дальше с `nil` и получишь пустоту или странности. Проверяй возврат, потом смотри `err`.
- **Несовместимый тип в plist/JSON.** Положил в массив свой объект, `NSDate` (для JSON) или `NSNull` (для plist) — сериализация провалится. Plist вернёт `NO/nil` с ошибкой, а `dataWithJSONObject:` бросит исключение и уронит программу — проверяй `isValidJSONObject:`. В plist/JSON идут только разрешённые типы; свои объекты — через архивацию.
- **Забыл `supportsSecureCoding` или `decodeObjectOfClass:`.** Без `+supportsSecureCoding` → YES архивация с `requiringSecureCoding:YES` не удастся: вернётся `nil` и ошибка «Class ‘Task’ does not adopt it». А если в `initWithCoder:` для объекта взять старый `decodeObjectForKey:` вместо `decodeObjectOfClass:forKey:`, безопасный распаковщик не узнает, какой класс ты ждёшь по этому ключу, и сверит значение только со списком классов корня. По нашим экспериментам строку он ещё пропустит, а `NSDate` или массив — уже нет: вся распаковка вернёт `nil` с ошибкой «value for key ... was of unexpected class».
- **Пишешь не во временную/Documents папку.** В приложении с песочницей запись в произвольный путь провалится (`writeToFile:` вернёт `NO`, а если не проверять возврат, ты этого и не заметишь). Спрашивай папку у системы: `NSTemporaryDirectory()` или `NSSearchPathForDirectoriesInDomains`.
- **`atomically:NO` для важных данных.** Прервётся запись — получишь полуфайл. Для всего, что жалко потерять, ставь YES.

Упражнения

1. Допиши `code/19-filemanager.m`: после создания файлов посчитай их суммарный размер, пройдя `contentsOfDirectoryAtPath:` и складывая `NSFileSize` каждого.
2. Сохрани словарь «город → население» (`{ @"Алматы": @2000000, ... }`) в plist через `writeToURL:error:`, прочитай назад `dictionaryWithContentsOfURL:error:` и выведи только города-миллионники.

3. Возьми список задач из `19-json.m`, добавь каждой ключ `@"tags"` с массивом строк. Убедись, что JSON по-прежнему пишется и читается (вложенные массивы JSON разрешены).
4. Попробуй разобрать JSON из файла, которого нет (`dataWithContentsOfFile:` вернёт `nil`). Поймай ситуацию до вызова `JSONObjectWithData:` (с `nil` вместо данных он бросит исключение) и выведи понятное сообщение, не уронив программу.
5. Добавь классу `Task` поле `NSDate *createdAt`. Закодируй его `encodeObjectForKey:`, декодируй `decodeObjectOfClass:[NSDate class] forKey:`. Проверь, что дата переживает архивацию.
6. Сделай массив из трёх `Task` и заархивируй его целиком одним вызовом `archivedDataWithRootObject:`. При распаковке используй `unarchivedObjectOfClasses:` с набором `[NSArray class]` и `[Task class]`. Подумай, почему классов теперь два.

Что мы получили

Теперь твои данные умеют переживать запуск программы. Ты управляешь файлами и папками через `NSFileManager`, пишешь и читаешь строки и `NSData` одним вызовом, сохраняешь целые коллекции в **plist** и **JSON** и поднимаешь их назад, а свои объекты учишь сохранять себя через **NSSecureCoding** и `NSKeyedArchiver`. И ты увидел общий принцип Foundation: ошибки приходят через возврат `nil/NO` и `NSError`, а не через краш, — поэтому битый файл или чужой JSON программу не роняют.

В отличие от си-шного `fopen/fwrite`, где ты ворочал байтами и сам изобретал формат, здесь весь объект или коллекция ложатся на диск и встают обратно готовыми вызовами. Дальше, в главе 20, разберём KVC и KVO — как читать и менять свойства объекта по имени и как подписываться на их изменения; механика `valueForKey:`, мелькавшая в главе 17 (и неявно работавшая здесь в `NSPredicate` и `NSSortDescriptor`), развернётся в полную силу.

Документация Apple

- `NSFileManager` — developer.apple.com/documentation/foundation/filemanager
- `NSString` (запись/чтение файла) — developer.apple.com/documentation/foundation/nsstring
- `NSData` — developer.apple.com/documentation/foundation/nsdata
- `NSJSONSerialization` — developer.apple.com/documentation/foundation/jsonserialization
- `NSPropertyListSerialization` — developer.apple.com/documentation/foundation/propertylistserialization
- `NSKeyedArchiver` — developer.apple.com/documentation/foundation/nskeyedarchiver
- `NSKeyedUnarchiver` — developer.apple.com/documentation/foundation/nskeyedunarchiver
- `NSSecureCoding` — developer.apple.com/documentation/foundation/nssecurecoding
- `NSCoding` — developer.apple.com/documentation/foundation/nscoding

- Archives and Serializations Programming Guide (архив Apple) — [developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ Archiving/Archiving.html](https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/Archiving/Archiving.html) (plist, архивы и кодирование объектов одним обзором).

Глава 20. KVC и KVO

До сих пор мы добирались до данных объекта прямо: `acc.balance`, `[acc setOwner:@"Серик"]`. Имя свойства было вписано в код, и компилятор знал его заранее. А что, если имя свойства станет известно только во время выполнения — придёт из формы, из конфигурации, из сетевого ответа? В Си такое невозможно в принципе: поле структуры читается только по имени прямо в исходнике. Objective-C даёт ровно эту возможность из коробки — через два механизма, выросших из runtime главы 14.

Первый — **KVC** (Key-Value Coding, кодирование «ключ-значение»): доступ к свойствам по строковому имени. Второй — **KVO** (Key-Value Observing, наблюдение «ключ-значение»): автоматическое уведомление о том, что свойство изменилось. KVO стоит на KVC, а под капотом — на той самой подмене `isa`, которую мы разобрали в главе 14. Построим оба с нуля.

Что мы сделаем

- Прочитаем и запишем свойство по строковому имени: `valueForKey:` и `setValue:forKey:`.
- Поймём, что KVC ищет под капотом (метод доступа -> голый `ivar`).
- Пройдём по вложенным объектам через `valueForKeyPath: (@"address.city")`.
- Применим KVC к массиву: соберём значения и посчитаем операторами `@count`, `@sum`, `@avg`, `@max`.
- Увидим, как KVC сам упаковывает примитивы в `NSNumber` (привет главе 16).
- Подпишемся на изменение свойства через `addObserver:forKeyPath:....`
- Своими глазами увидим `isa-swizzling`: после подписки `isa` объекта меняется на сгенерированный подкласс.
- Докажем, что изменение через сеттер уведомление шлёт, а прямое присваивание `ivar` — нет.
- Разберём ручные уведомления и `+automaticallyNotifiesObserversForKey:`.
- Снимем наблюдателя (`removeObserver:`) и узнаем, что будет, если забыть или снять дважды.

Часть 1. KVC — доступ по строковому имени

Разбираем задачу руками

Представь форму редактирования профиля. Полей десятки: имя, город, телефон, баланс. Писать на каждое поле по строчке `obj.name = ...`, `obj.city = ...` — скучно и негибко: добавили поле — правь код в трёх местах. Хочется иначе: пробежать по словарю `@{ @"name": @"Серик", @"balance": @1500.50 }` и присвоить каждое значение свойству с **таким же именем**. Имя свойства тут — обычная строка, известная только в момент выполнения.

Вот для этого и есть KVC. Его сердце — два метода, которые `NSObject` даёт **любому** объекту бесплатно:

- `setValue:forKey:` — «запиши это значение в свойство с этим именем»;
- `valueForKey:` — «верни значение свойства с этим именем».

Шаг 1. Читаем и пишем свойство по имени

Заведём класс `Account` с владельцем и балансом — самый обычный, без единой строчки «под KVC». Включать ничего не нужно: способность работать по ключу встроена в `NSObject`.

```
@interface Account : NSObject
@property (nonatomic, copy) NSString *owner;
@property (nonatomic) double balance;      /* примитив double */
@end

@implementation Account
@end
```

Теперь обращаемся к свойствам не через точку, а по строке:

```
Account *acc = [[Account alloc] init];

[acc setValue:@"Серик" forKey:@"owner"];
[acc setValue:@1500.50 forKey:@"balance"]; /* @1500.50 – NSNumber */

NSString *owner = [acc valueForKey:@"owner"];
id balance      = [acc valueForKey:@"balance"];

NSLog(@"владелец = %@", owner);
NSLog(@"баланс   = %@ (класс %@)", balance, [balance class]);
NSLog(@"баланс как double = %.2f", [balance doubleValue]);
```

Разберём по частям.

- `setValue:forKey:` принимает **объект-значение** и **строку-ключ**. Первый аргумент всегда объект (`id`), поэтому число пришлось завернуть в `NSNumber` литералом `@1500.50`. Ключ — обычная `NSString`.
- `valueForKey:` принимает ключ-строку и возвращает `id` — объект. Для `owner` это `NSString`, как и было. А вот для `balance` интереснее: само свойство — примитив `double`, объектом не является. KVC **сам** обернул его в `NSNumber`, чтобы вернуть как `id`. Поэтому переменная `balance` у нас типа `id`, а не `double`.
- `[balance doubleValue]` распаковывает `NSNumber` обратно в `double` — ровно как в главе 16.

Запускаем (`code/20-kvc.m`):

```
владелец = Серик
баланс   = 1500.5 (класс __NSCFNumber)
баланс как double = 1500.50
через свойство: owner=Серик balance=1500.50
```

Смотри на вторую строку, в скобках: класс значения — `__NSCFNumber`. Это внутренняя реализация `NSNumber`. То есть `valueForKey:@"balance"` действительно вернул объект-обёртку, хотя в классе `balance` объявлен как голый `double`. Это **автоупаковка KVC** (см. главу 16): примитивы `int`, `double`, `BOOL`, `C`-структуры KVC на входе распаковывает, а на выходе упаковывает в `NSNumber/NSNumber`. Тебе не нужно делать это руками — главное помнить, что `valueForKey:` для числового свойства вернёт именно `NSNumber`, а не `double`.

Отличие от Си. В Си поле структуры читается только по имени, вписанному в исходник: `acc.balance`. «Дай мне поле, имя которого лежит в переменной-строке» там нельзя — после компиляции от имён полей не остаётся ничего, спрашивать некого. Чтобы такое сделать в Си, пришлось бы вручную строить свою таблицу «строка -> смещение поля». В Objective-C KVC даёт это бесплатно: имена свойств живут в runtime, и `valueForKey:` находит нужное во время выполнения.

Шаг 2. Что KVC делает под капотом

Откуда `valueForKey:@"owner"` знает, как взять значение? Он не лезет сразу в память — он действует по строгому порядку поиска. Для чтения ключа `key` (упрощённо; полный порядок — в *Key-Value Coding Programming Guide*, раздел *Accessor Search Patterns*):

```
[obj valueForKey:@"balance"]
|
1. ищет геттер: -getBalance, -balance, -isBalance, -_balance
|      нашёл -> зовёт его, результат (если примитив) упаковывает
```

```

2. нет геттера? и разрешён прямой доступ к ivar?
|      ищет поле: _balance, потом _isBalance, balance, isBalance
|      нашёл -> читает поле напрямую, упаковывает
3. ничего не нашли -> -valueForKey: -> исключение
    NSUnknownKeyException

```

Для записи `setValue:forKey:` зеркально: сначала ищет сеттер `-setBalance:` (или `-_setBalance:`), и только если его нет (и разрешён прямой доступ) — пишет прямо в ivar `_balance`. Разрешение прямого доступа к ivar даёт метод класса `+accessInstanceVariablesDirectly` — по умолчанию он возвращает YES.

Шаг с ivar легко показать. Заведём класс с **голым полем без свойства** — ни геттера, ни сеттера у него нет:

```

@interface Box : NSObject {
    NSString *_secret;                /* только ivar */
}
@end

@implementation Box
@end

```

И всё равно достучимся до `_secret` по ключу "secret":

```

Box *box = [[Box alloc] init];
[box setValue:@"пароль" forKey:@"secret"]; /* пишет прямо в _secret */
NSLog(@"секрет из ivar = %@", [box valueForKey:@"secret"]);

```

В выводе:

```
секрет из ivar = пароль
```

Сеттера нет, а запись прошла — KVC сам нашёл ivar `_secret` по ключу `secret` (отбросив подчёркивание) и записал в него. Это и есть третий шаг алгоритма. Полезно для отладки, но в обычном коде на это не опираются: лезть в чужие приватные поля по строке — плохая привычка.

Шаг 3. Key path — путь сквозь вложенные объекты

Ключ адресует одно свойство. А если нужно свойство **внутри** свойства? Например, у человека есть адрес (`Address`), а у адреса — город. Для таких цепочек есть **key path** (путь по ключам): ключи, разделённые точками, и метод `valueForKeyPath:`.

```

@interface Address : NSObject
@property (nonatomic, copy) NSString *city;
@property (nonatomic, copy) NSString *street;
@end

@interface Person : NSObject
@property (nonatomic, copy) NSString *name;
@property (nonatomic, strong) Address *address;    /* вложенный объект */
@end

```

Собираем человека с адресом и читаем сквозь точку:

```

Person *p = [[Person alloc] init];
p.name     = @"Серик";
p.address = addr;          /* addr.city = @"Алматы", addr.street = @"Абая" */

NSLog(@"name          = %@", [p valueForKeyPath:@"name"]);
NSLog(@"address.city  = %@", [p valueForKeyPath:@"address.city"]);
NSLog(@"address.street = %@", [p valueForKeyPath:@"address.street"]);

```

```

name          = Серик
address.city   = Алматы
address.street = Абая

```

Точка в @"address.city" читается как «возьми у p свойство address, а у него — свойство city». KVC раскручивает путь слева направо: сначала valueForKey:@"address", потом у результата valueForKey:@"city". Цепочка может быть любой длины. Заметь: valueForKeyPath: с обычным ключом без точки (@"name") работает как valueForKey: — путь из одного звена.

Отличие от Си. В Си обращение к вложенному полю — это p.address->city, и весь путь зашит в исходник на этапе компиляции. Строка @"address.city", по которой во время выполнения находят вложенное поле, в Си немыслима без собственного парсера и таблиц. Здесь это одна строка и один вызов.

Шаг 4. KVC и коллекции

KVC раскрывается во всю силу на массивах. Два разных приёма.

Первый: valueForKey: на массиве собирает значения. Если послать valueForKey: не одиночному объекту, а NSArray, он применит ключ к **каждому** элементу и вернёт массив результатов. Возьмём корзину товаров. Product — простой класс со свойствами NSString *title и double

price и фабричным методом +title:price:, который создаёт и заполняет товар (целиком — в code/20-keypath.m):

```
NSArray<Product *> *cart = @[
    [Product title:@"Книга"    price:3500.0],
    [Product title:@"Наушники" price:9900.0],
    [Product title:@"Чехол"    price:1200.0],
];

NSArray *titles = [cart valueForKey:@"title"];
NSLog(@"названия = %@", titles);
```

```
названия = (
    "\U041a\U043d\U0438\U0433\U0430",
    "\U041d\U0430\U0443\U0448\U043d\U0438\U043a\U0438",
    "\U0427\U0435\U0445\U043e\U043b"
)
```

Из массива из трёх Product мы одной строкой достали массив их названий. (Кириллица в выводе показана как \U... — так NSArray печатает вложенные строки в своём description; сами строки целые, это лишь форма логга. Если хочется читаемо — склей массив через componentsJoinedByString:, как мы делали в главе 12.) Удобно для «вытащить из списка объектов список одного поля».

С числовым свойством то же самое, и вот тут видно кое-что интересное:

```
NSArray *prices = [cart valueForKeyPath:@"price"];
NSLog(@"цены = %@", prices);
```

```
цены = (
    3500,
    9900,
    1200
)
```

Свойство price объявлено как double — примитив, не объект. Но в массив попали объекты: KVC **сам завернул** каждое число в NSNumber, потому что в коллекции примитив положить нельзя. Это общее правило KVC: на границе он упаковывает примитивы в NSNumber, а структуры — в NSValue (обе обёртки мы разбирали в главе 16). Именно поэтому чуть ниже @sum.price вернёт нам NSNumber, а не double.

Второй: операторы коллекций. Это особые ключи, начинающиеся с @: @count, @sum, @avg, @max, @min, @distinctUnionOfObjects и другие. Они считают агрегат по всему массиву. Синтаксис: @оператор.ключ (кроме @count, которому ключ не нужен).

```
NSNumber *count = [cart valueForKeyPath:@"@count"];
NSNumber *sum   = [cart valueForKeyPath:@"@sum.price"];
NSNumber *avg   = [cart valueForKeyPath:@"@avg.price"];
NSNumber *max   = [cart valueForKeyPath:@"@max.price"];
NSNumber *min   = [cart valueForKeyPath:@"@min.price"];
```

```
NSLog(@"@count      = %@", count);
NSLog(@"@sum.price  = %@", sum);
NSLog(@"@avg.price  = %.2f", [avg doubleValue]);
NSLog(@"@max.price  = %@", max);
NSLog(@"@min.price  = %@", min);
```

```
@count      = 3
@sum.price   = 14600
@avg.price   = 4866.67
@max.price   = 9900
@min.price   = 1200
```

Разбор операторов:

- @"@count" — сколько элементов в массиве. Ключ после точки не нужен: считаем сами объекты, а не их поле.
- @"@sum.price" — сложить значения свойства price всех элементов. $3500 + 9900 + 1200 = 14600$.
- @"@avg.price" — среднее значение price. $14600 / 3 \approx 4866.67$.
- @"@max.price" / @"@min.price" — наибольшее и наименьшее значение price.

Все операторы возвращают **объект**, а не голое число: @count — это NSNumber, @sum и @avg по нашим экспериментам приходят как NSDecimalNumber (подкласс NSNumber), а @max/@min возвращают само значение свойства — здесь упакованный NSNumber. Поэтому среднее мы печатаем через [avg doubleValue]. Это та же автоупаковка: свойство price — double, а наружу приходит число-объект.

Запусти code/20-keypath.m целиком — увидишь и key path, и оба приёма с коллекциями.

Зачем вообще KVC

KVC — не самоцель, это фундамент. На нём стоит многое: обобщённый код (заполнение форм, сериализация объекта в словарь и обратно, маппинг JSON на модель — всё это перебор ключей строками); NSPredicate и сортировка (фильтры вида @"price > 1000" читают свойства через KVC); Cocoa bindings на macOS; и главное для нас — **KVO**, наблюдение за изменениями, к которому мы переходим. Без KVC оно бы не работало.

Часть 2. KVO — наблюдение за изменением

Разбираем задачу руками

Баланс счёта меняется — а где-то на экране висит его сумма, которую надо обновить. Можно после каждого изменения вручную звать «обнови экран», но изменений много, и легко забыть в одном месте. Хочется наоборот: один раз сказать «следи за balance» — и пусть уведомляют при любом изменении, кто бы его ни сделал. Это и есть KVO. Три действия:

1. **подписаться:** addObserver:forKeyPath:options:context;;
2. **получать уведомления** в методе observeValueForKeyPath:ofObject:change:context;;
3. **обязательно отписаться:** removeObserver:forKeyPath:context:.

Шаг 5. Подписываемся и ловим изменение

Наблюдаемый объект — наш Account. Добавим ему один «нечестный» метод (зачем — увидим на шаге 7):

```
@interface Account : NSObject
@property (nonatomic) double balance;
- (void)setBalanceSneaky:(double)value;
@end

@implementation Account
- (void)setBalanceSneaky:(double)value {
    _balance = value;          /* прямое присваивание полю, в обход сеттера */
}
@end
```

Наблюдатель — отдельный класс, который переопределяет метод-приёмник уведомлений. Этот метод объявлен в NSObject, так что просто переопределяем его:

```
@interface Watcher : NSObject
@end
```

```

@implementation Watcher
- (void)observeValueForKeyPath:(NSString *)keyPath
    ofObject:(id)object
    change:(NSDictionary<NSKeyValueChangeKey, id> *)change
    context:(void *)context {
    if (context == &kBalanceContext) {
        NSLog(@" [наблюдатель] %@: %@ -> %@",
            keyPath,
            change[NSKeyValueChangeOldKey],
            change[NSKeyValueChangeNewKey]);
    } else {
        [super observeValueForKeyPath:keyPath ofObject:object
            change:change context:context];
    }
}
@end

```

Разберём параметры этого метода — runtime зовёт его при каждом замеченном изменении:

- `keyPath` — какой ключ изменился (`@"balance"`).
- `object` — у какого объекта.
- `change` — словарь с подробностями. Ключи в нём — константы: `NSKeyValueChangeOldKey` (старое значение) и `NSKeyValueChangeNewKey` (новое). Значения там — объекты, поэтому `double` приходит как `NSNumber` (снова автоупаковка).
- `context` — тот самый «маркер», который мы передали при подписке. Сравнивая его, отличаем нашу подписку от чужих.

Теперь подписка и изменения в `main`:

```

Account *acc = [[Account alloc] init];
Watcher *w  = [[Watcher alloc] init];

[acc addObserver:w
    forKeyPath:@"balance"
    options:NSKeyValueObservingOptionNew | NSKeyValueObservingOptionOld
    context:&kBalanceContext];

acc.balance = 100.0;                /* через сеттер */
[acc setValue:@250.0 forKey:@"balance"]; /* через KVC-сеттер */
[acc setBalanceSneaky:999.0];        /* напрямую в ivar */

```

```
[acc removeObserver:w forKeyPath:@"balance" context:&kBalanceContext];
```

Разберём подписку `addObserver:forKeyPath:options:context::`

- `addObserver:` — кто будет получать уведомления (наш `w`). Важно: объект-наблюдатель KVO **не удерживает** — за временем его жизни следишь ты сам.
- `forKeyPath:` — за каким ключом следим. Можно и `key path` (`@"address.city"`).
- `options:` — что положить в словарь `change`. Мы объединили | две опции из `NSKeyValueObservingOptions`: `...OptionNew` (класть новое значение) и `...OptionOld` (класть старое). Есть ещё `...OptionInitial` (прислать уведомление сразу при подписке) и `...OptionPrior` (слать ещё и перед изменением).
- `context:` — произвольный указатель-маркер. Мы передаём адрес статической переменной `kBalanceContext` — он уникален во всей программе. В приёмнике сверяем `context == &kBalanceContext` и точно знаем, что это наша подписка, а не чья-то ещё.

Где взялся `kBalanceContext` — это просто статическая переменная на уровне файла, её адрес и служит уникальным маркером:

```
static char kBalanceContext; /* значение неважно — важен его адрес */
```

Шаг 6. Запускаем и видим `isa-swizzling`

Перед подпиской и сразу после неё распечатаем две вещи: что отвечает объект на `-class` и что лежит в его настоящем `isa` (через `object_getClass` из главы 14):

```
NSLog(@"до подписки: -class=%@ настоящий isa=%s",
      [acc class], class_getName(object_getClass(acc)));
/* ...addObserver... */
NSLog(@"после подписки: -class=%@ настоящий isa=%s",
      [acc class], class_getName(object_getClass(acc)));
```

Запускаем `code/20-kvo.m` целиком:

```
до подписки: -class=Account настоящий isa=Account
после подписки: -class=Account настоящий isa=NSKVONotifying_Account
--- меняем через сеттер ---
[наблюдатель] balance: 0 -> 100
[наблюдатель] balance: 100 -> 250
--- меняем напрямую через ivar ---
итоговый баланс = 999.00
```

Вот он, фокус из главы 14, вживую. До подписки и `-class`, и настоящий `isa` — это `Account`. **После** `addObserver: -class` по-прежнему отвечает `Account`, а настоящий `isa` стал `NSKVONotifying_Account` — класса, которого мы не писали.

Что произошло. В момент подписки runtime:

1. на лету создал **динамический подкласс** `Account` с именем `NSKVONotifying_Account` (см. главу 14, `isa-swizzling`);
2. переопределил в нём сеттер `setBalance:` так, что тот вокруг настоящего присваивания шлёт уведомления `willChange.../didChange...`;
3. переопределил `-class`, чтобы он **врал** и возвращал прежний `Account` — иначе твой код запутался бы, увидев незнакомый класс;
4. **подменил `isa`** объекта `acc` на этот новый подкласс.

Объект снаружи остался «тем же `Account`», но его реальный `isa` теперь ведёт на сгенерированный подкласс с «говорящими» сеттерами. Именно поэтому KVO «бесплатно» ловит изменения: каждое присваивание через сеттер проходит через переопределённый метод подкласса.

Отличие от Си. В Си, чтобы заметить изменение переменной, пришлось бы оборачивать каждое присваивание своей функцией-«сеттером» и вручную звать всех заинтересованных — везде, во всём коде, не забыв ни одного места. KVO делает это само, подменив класс объекта на лету (`isa-swizzling`, глава 14). Переменная как была полем — так и осталась, а наблюдение появилось из воздуха. В Си «слежки за переменной» нет и взяться ей неоткуда.

Шаг 7. Сеттер уведомляет, прямой `ivar` — нет

Теперь главный урок KVO. Посмотри на вывод ещё раз: уведомления [наблюдатель] пришли **дважды** — на `acc.balance = 100.0` и на `[acc setValue:@250.0 forKey:@"balance"]`. Оба пути идут через сеттер `setBalance:` (KVC-запись тоже зовёт сеттер), а сеттер у нас теперь — переопределённый, «говорящий».

А вот на `[acc setBalanceSneaky:999.0]` уведомления **нет**. Хотя баланс честно стал 999 (последняя строка это подтверждает), наблюдатель промолчал. Почему: `setBalanceSneaky:` пишет прямо в поле `_balance`, минуя сеттер. Подменённый класс ничего не оборачивал вокруг прямого доступа к `ivar` — ему просто негде вклиниться. KVO видит изменения **только через сеттер**.

Отсюда железное правило: **меняй наблюдаемые свойства через сеттер или `setValue:forKey:`, никогда не присваивай `ivar` напрямую** (внутри своих методов — `self.balance = ...`, а не `_balance = ...`). Иначе наблюдатели не узнают об изменении, и на экране останется старое значение. Это одна из самых частых и коварных ошибок с KVO.

Шаг 8. Ручные уведомления и `automaticallyNotifies...`

Автоматику KVO мы видели: подменённый сеттер сам шлёт пару «перед/после». Иногда нужно слать уведомления **вручную** — например, свойство вычисляемое и сеттера у него нет, или одно изменение должно уведомить сразу о нескольких ключах. Для этого есть пара методов:

```
[self willChangeValueForKey:@"balance"];    /* «сейчас изменю» */
_balance = value;                           /* собственно изменение */
[self didChangeValueForKey:@"balance"];      /* «изменил» */
```

`willChangeValueForKey:` запоминает старое значение, `didChangeValueForKey:` берёт новое и шлёт уведомление наблюдателям. Сравнивать KVO не станет: даже если значение не поменялось, наблюдатель получит уведомление «5 -> 5» (мы проверили — так же ведёт себя и автоматический сеттер, если присвоить то же число). Хочешь молчать при равных значениях — проверь это сам до `willChange....` Между парой — само изменение. Звать методы надо парой и в этом порядке.

Чтобы автоматика не сработала вдобавок к ручной (двойное уведомление), для такого ключа её отключают, переопределив метод класса:

```
+ (BOOL)automaticallyNotifiesObserversForKey:(NSString *)key {
    if ([key isEqualToString:@"balance"]) {
        return NO;          /* для balance уведомляем вручную */
    }
    return [super automaticallyNotifiesObserversForKey:key];
}
```

Вернул `NO` для ключа — runtime больше не оборачивает его сеттер сам, и вся ответственность на твоих `willChange.../didChange....` Для всех остальных ключей возвращаем решение `super` (по умолчанию `YES`). В обычной жизни автоматика хватает; ручные уведомления — для зависимых и вычисляемых свойств.

Шаг 9. `removeObserver` — обязательно

Последняя строка примера — не формальность:

```
[acc removeObserver:w forKeyPath:@"balance" context:&kBalanceContext];
```

Документация Apple требует снимать наблюдателя, пока оба объекта ещё живы: KVO не удерживает наблюдателя. Долгие годы нарушение этого правила кончалось падением: `Watcher` уничтожен, подписка осталась, и при следующем изменении `balance` уведомление уходило по мёртвому адресу.

По нашим экспериментам на свежей macOS (26) падения уже нет: Foundation сам обнуляет ссылку на умершего наблюдателя, и уведомление просто никуда не уходит. Но это недокументированное поведение, на старых системах оно другое, а сама запись о подписке так и висит на объекте. Поэтому правило не меняется: подписался — отпишись.

Снимай подписку там, где это логично: для долгоживущих объектов — в `dealloc` наблюдателя; в нашем коротком примере — прямо перед концом, пока `acc` и `w` оба на месте. Парность простая: на каждый `addObserver:` ровно один `removeObserver:`. А вот снять несуществующую подписку — гарантированная ошибка на любой системе, исключение `NSRangeException`:

```
Cannot remove an observer <Watcher 0x...> for the key path "balance"
from <Account 0x...> because it is not registered as an observer.
```

Так что не отписывайся дважды.

А как в Swift: `NSKeyValueObservation`

У классического KVO есть болевые точки: строковый ключ (опечатку поймает только в рантайме), приёмник в отдельном методе вдали от подписки, ручной `removeObserver:`. В Swift их убрали: метод `observe(_:options:changeHandler:)` принимает проверяемый компилятором key path (`\.balance`) и замыкание, а возвращает объект `NSKeyValueObservation` — пока он жив, подписка действует, а при его уничтожении снимается сама. Механика под ним — тот же `isa-swizzling`.

В Objective-C такого блочного API у Foundation **нет** (в заголовках `SDKNSKeyValueObservation` не встречается): здесь KVO — это только `addObserver:forKeyPath:options:context:` и `observeValueForKeyPath:...`, которые мы разобрали. Блочные обёртки над ними в проектах на Objective-C пишут сами или берут готовые сторонние библиотеки.

Полная программа

Три самостоятельных файла, каждый собирается и запускается отдельно:

- `code/20-kvc.m` — `valueForKey:/setValue:forKey:`, доступ к `ivar`, автоупаковка в `NSNumber`.
- `code/20-keypath.m` — `valueForKeyPath:`, key path сквозь вложенные объекты, KVC на колллекциях, операторы `@count/@sum/@avg/@max`.
- `code/20-kvo.m` — подписка, `isa-swizzling`, разница «сеттер vs `ivar`», снятие наблюдателя.

Проверяем

Каждый файл собирается одной командой (подставь имя):

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/20-kvc.m -o t && ./t
```

Все три собираются **без предупреждений** под `-Wall -Wextra`. Ожидаемый вывод приведён по ходу главы (дата, имя программы и числа в скобках будут свои). Контрольные точки:

- в `20-kvc.m` класс значения `balance` — `__NSCFNumber` (упаковка);
- в `20-kvo.m` после подписки настоящий `isa` — `NSKeyValueNotifying_Account`, а `-class` всё ещё `Account`;
- в `20-kvo.m` ровно **два** уведомления (на оба сеттера) и **ни одного** на `setBalanceSneaky:`.

Частые ошибки

- **Меняешь ivar напрямую — наблюдатель молчит.** `_balance = x` в обход сеттера KVO не заметит. Внутри методов пиши `self.balance = x` или `setValue:forKey:`.
- **Забыл `removeObserver:`.** Наблюдатель умер, подписка осталась. На старых системах следующее изменение слало сообщение мёртвому объекту и роняло программу; на свежей macOS, по нашим экспериментам, уже нет, но документация по-прежнему требует отписки. Парность строгая: один `addObserver:` — один `removeObserver:`.
- **Отписался дважды или без подписки.** Исключение `NSRangeException`. Снимай подписку ровно один раз.
- **Не сверяешь `context`.** Если на объект подписан и твой код, и суперкласс, без проверки `context` поймашь чужие уведомления. Всегда передавай уникальный `context` и сверяй его, а чужое отдавай `super`.
- **Опечатка в ключе.** `@“balance”` соберётся, но в рантайме даст `setValue:forUndefinedKey:` -> исключение `NSUnknownKeyException` («this class is not key value coding-compliant for the key balance»): ключи — строки, компилятор их не проверяет.
- **Ждёшь `double`, а пришёл `NSNumber`.** И из `valueForKey:`, и из словаря `change` числа приходят упакованными — распаковывай через `doubleValue/intValue`.

Упражнения

1. Добавь в `Account` свойство `NSString *currency` и в `20-kvc.m` заполни оба свойства, пробежав словарём `{ @"owner": @"Серик", @"currency": @"KZT" }` в цикле `for...in` по его ключам, вызывая `setValue:forKey:`. Это и есть «обобщённое заполнение формы».
2. В `20-keypath.m` добавь оператор `@“distinctUnionOfObjects.title”` и посмотри, чем он отличается от `valueForKey:@"title”` (подсказка: убирает дубликаты). Добавь в корзину второй товар «Книга» и сравни.

3. Сделай в `Person` свойство-массив `friends` (тоже `Person`) и достаь `valueForKeyPath:@"friends.name"` — массив имён друзей.
4. В `20-kvo.m` добавь к подписке опцию `NSKeyValueObservingOptionInitial` и убедись, что уведомление приходит сразу при `addObserver:`, ещё до первого изменения.
5. Сделай класс `Thermostat` с вычисляемой температурой и **ручными** уведомлениями (`willChangeValueForKey:/didChangeValueForKey: +automaticallyNotifiesObserversForKey:->NO`). Подпишись и проверь, что уведомление приходит ровно один раз.
6. Намеренно продублируй в `20-kvo.m` строку `removeObserver:` — отпишись дважды. Поймай падение и прочитай текст исключения `NSRangeException` — потом узнаешь такую ошибку сразу. Затем убери обе отписки, а наблюдателя `w` заверни в отдельную область видимости `{ ... }`, чтобы он умер раньше `acc`, и поменяй баланс: придёт ли уведомление и упадёт ли программа на твоей системе?

Что мы получили

Мы научили программу работать со свойствами по их строковому имени. **KVC** дал чтение и запись по ключу (`valueForKey:/setValue:forKey:`), проход сквозь вложенные объекты (`valueForKeyPath:,@"address.city"`), агрегаты по коллекциям (`@count,@sum,@avg,@max`) и бесплатную упаковку примитивов в `NSNumber`. На нём стоят формы, сериализация, `NSPredicate` и сортировка — весь обобщённый код, которому имя свойства известно лишь во время выполнения.

KVO добавил автоматическое наблюдение: подписался на ключ — и получаешь уведомления при каждом изменении через сеттер. Под капотом мы своими глазами увидели механику главы 14: `runtime` создаёт динамический подкласс `NSKVONotifying_Account`, подменяет `isa` объекта и переопределяет сеттеры. Отсюда два правила KVO: менять только через сеттер (иначе изменение незаметно) и снимать наблюдателя ровно один раз.

В Си ничего подобного нет: там поле читается лишь по имени из исходника, а следить за переменной пришлось бы вручную в каждой точке кода. Здесь и доступ по строке, и наблюдение выросли из одной идеи — живого `runtime`, где объект знает свой класс, а класс можно подменить на лету. Дальше поднимемся к многопоточности и увидим, как несколько потоков делят одни объекты.

Документация Apple

- Key-Value Coding Programming Guide (архив Apple) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/KeyValueCoding/index.html (Accessor Search Patterns, Collection Operators, Key Paths).

- Key-Value Observing Programming Guide (архив Apple) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/KeyValueObserving/KeyValueObserving.html (Registering, Receiving, Manual Change Notification).
- NSObject, разделы Key-Value Coding и Key-Value Observing — developer.apple.com/documentation/objectivec/nsobject-swift.class (valueForKey:, setValue:forKey:, valueForKeyPath:, addObserver:forKeyPath:options:context:, observeValueForKeyPath:ofObject:change:context:, removeObserver:).
- NSKeyValueObservingOptions — developer.apple.com/documentation/foundation/nskeyvalueobservingoptions
- NSKeyValueChangeKey — developer.apple.com/documentation/foundation/nskeyvaluechangekey
- NSKeyValueObservation (только Swift) — developer.apple.com/documentation/foundation/nskeyvalueobservation

Глава 21. Многопоточность, GCD и run loop

До сих пор наши программы делали одно дело за раз: строка за строкой, сверху вниз. Так удобно рассуждать, но в реальном приложении этого мало. Пока ты «скачиваешь» большой файл, интерфейс не должен замирать. Пока считаешь что-то тяжёлое — кнопки должны нажиматься. Для этого работу раскладывают на несколько потоков, и кто-то должен ими управлять.

Разберёмся, что такое поток простыми словами, познакомимся с **GCD** (Grand Central Dispatch) — главным инструментом Apple для фоновой работы, научимся ждать группу задач, безопасно инициализировать синглтон, увидим вживую **гонку данных** и починим её. Потом поднимемся на уровень выше — `NSOperation`, заглянем в `NSThread`, разберём **run loop** с таймером и закончим рассылкой уведомлений через `NSNotificationCenter`.

Что мы сделаем

- Поймём, что такое поток и зачем не блокировать главный.
- Запустим долгую «загрузку» в фоне и вернём результат на главную очередь.
- Дождёмся нескольких загрузок сразу через `dispatch_group`.
- Сделаем потокобезопасный синглтон через `dispatch_once`.
- Увидим гонку данных и починим тремя способами.
- Соберём цепочку зависимых задач на `NSOperationQueue`.
- Запустим таймер на `run loop` и разошлём уведомления подписчикам.

Поток — это что

Когда система запускает твою программу, она даёт ей один **поток выполнения** (thread). Поток — это как читающий палец, который ведёт по коду: дошёл до строки — выполнил, перешёл к следующей. Один палец = одно дело за раз.

Этот первый поток называют **главным** (main thread). На нём в приложении рисуется интерфейс и обрабатываются нажатия. И тут засада: если на главном потоке запустить что-то долгое — скачать файл, перемолоть мегабайт текста, — палец застрянет на этой строке. Пока он застрял, интерфейс не перерисовывается, кнопки не реагируют. Пользователь видит зависшее приложение.

Решение — **несколько потоков**. Пусть один палец (главный) занимается интерфейсом, а другой в это время скачивает файл. Они идут по коду параллельно. Закончил второй — отдал результат первому.

Управлять потоками вручную тяжело и опасно: их надо создавать, ждать, синхронизировать, не забыть закрыть. Поэтому Apple даёт инструмент, где о самих потоках думать почти не нужно, — GCD.

Отличие от Си. В чистом Си многопоточность — это библиотека POSIX-потоков (pthread). Ты сам зовёшь pthread_create, передаёшь указатель на функцию, потом pthread_join, чтобы дождаться, а для защиты общих данных вручную заводишь pthread_mutex_t и не забываешь lock/unlock. Это «как управлять потоком». GCD переворачивает мысль: ты пишешь блок «что сделать» (см. блоки в главе 12) и кладёшь его в очередь. Сколько завести потоков и как их переиспользовать — решает система. Меньше кода, меньше способов ошибиться.

GCD: очереди задач

Grand Central Dispatch (или просто GCD, в коде — dispatch_*) — это система очередей. Ты не создаёшь потоки. Ты кладёшь **задачу** (блок) в **очередь** (dispatch_queue_t), а GCD сам берёт поток из общего пула и выполняет на нём твой блок.

Очереди бывают двух характеров:

- **Последовательная** (serial): берёт по одной задаче, следующую начинает только когда закончит предыдущую. Порядок гарантирован.
- **Параллельная** (concurrent): может запустить сразу несколько задач на разных потоках. Порядок завершения не гарантирован.

Готовые очереди, которые всегда под рукой:

- **Главная** — dispatch_get_main_queue(). Последовательная, привязана к главному потоку. Всё, что положишь сюда, выполнится на главном потоке. Сюда возвращают результат, чтобы трогать интерфейс.
- **Глобальные фоновые** — dispatch_get_global_queue(qos, 0). Параллельные, общие на всю программу. Сюда кладут долгую работу. qos (quality of service, «класс качества

обслуживания») — подсказка системе о важности: `QOS_CLASS_USER_INITIATED` (пользователь ждёт результат), `QOS_CLASS_DEFAULT`, `QOS_CLASS_UTILITY` (долгая фоновая), `QOS_CLASS_BACKGROUND` (совсем неспешная). Второй аргумент — флаги, всегда `0`.

А свою очередь создаёшь так:

```
dispatch_queue_t q =
    dispatch_queue_create("com.example.work", DISPATCH_QUEUE_SERIAL);
```

Первый аргумент — метка (строка-имя, видна в отладчике, бери в стиле обратного домена). Второй — `DISPATCH_QUEUE_SERIAL` (последовательная) или `DISPATCH_QUEUE_CONCURRENT` (параллельная).

`dispatch_queue_t` — это объект. Под ARC ты не освобождаешь его руками: компилятор сам убирает очередь, когда на неё не останется ссылок.

Шаг 1: долгая загрузка в фоне

Поставим задачу. Надо «скачать» отчёт (это занимает время) и показать его. Скачивание имитируем паузой `[NSThread sleepForTimeInterval:0.5]` — полсекунды как будто ждём сеть.

Самая наивная мысль — сделать всё в лоб, прямо на главном потоке:

```
NSString *report = downloadReport(); // полсекунды программа стоит
NSLog(@"%@", report);
```

Работает, но полсекунды поток заморожен. В консольной программе это незаметно, в приложении — зависший экран. Унесём загрузку в фон.

```
dispatch_queue_t bg =
    dispatch_get_global_queue(QOS_CLASS_USER_INITIATED, 0);

dispatch_async(bg, ^{
    NSString *report = downloadReport();
    NSLog(@"загрузка закончена в фоне");
});

NSLog(@"главная очередь свободна");
```

`dispatch_async(queue, block)` — «положи блок в очередь и **не жди**, иди дальше» (`async` = `asynchronous`, асинхронно). Главный поток сразу печатает «свободна», а блок крутится где-то на фоновом потоке.

Но есть проблема: блок печатает результат сам, прямо на фоновом потоке. В реальном приложении трогать интерфейс с фонового потока **нельзя** — это приведёт к сбоям. Результат надо вернуть на главную очередь. И вторая беда: консольная программа дойдёт до конца `main` и закроется раньше, чем фоновый блок успеет отработать. Соберём правильную версию.

Файл `code/21-async.m`:

```
#import <Foundation/Foundation.h>

// «Скачиваем» данные: имитируем долгую работу паузой.
static NSString *downloadReport(void) {
    [NSThread sleepForTimeInterval:0.5];    // как будто ждём сеть
    return @"отчёт за июнь: 1000 строк";
}

int main(void) {
    @autoreleasepool {
        NSLog(@"1. старт на главной очереди");

        // Глобальная фоновая очередь — туда кладём долгую работу.
        dispatch_queue_t bg =
            dispatch_get_global_queue(QOS_CLASS_USER_INITIATED, 0);

        dispatch_async(bg, ^{
            // Этот блок выполняется НЕ на главном потоке.
            NSString *report = downloadReport();
            NSLog(@"2. загрузка закончена в фоне");

            // Результат возвращаем на главную очередь.
            dispatch_async(dispatch_get_main_queue(), ^{
                NSLog(@"3. показываем результат: %@", report);
                exit(0);           // явно завершаем программу
            });
        });

        NSLog(@"4. главная очередь свободна, ждём событий");

        // Запускаем обработку главной очереди. Без этого блоки,
        // отправленные в dispatch_get_main_queue(), не выполнятся.
```

```

    dispatch_main();
}
return 0;    // сюда мы не дойдём: программу закроем exit(0)
}

```

Разберём новое.

- `dispatch_async(bg, ^{ ... })` — кладем блок в фоновую очередь и тут же идём дальше (печатаем пункт 4).
- Вложенный `dispatch_async(dispatch_get_main_queue(), ^{ ... })` — изнутри фонового блока «перепрыгиваем» обратно на главную очередь, уже с готовым report. Это классический рисунок GCD: **тяжёлое — в фон, результат — на главную очередь**.
- `dispatch_main()` — запускает диспетчеризацию главной очереди и **никогда не возвращается**. В обычной программе с интерфейсом эту роль играет цикл событий приложения; в консольной мы зовём `dispatch_main` сами, иначе блоки для главной очереди никому выполнить.
- `exit(0)` — раз `dispatch_main` не возвращает управление, выходим из программы явно, когда напечатали результат. Иначе она висела бы вечно.

Соберём и запустим:

```

clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/21-async.m -o async
./async

```

Вывод:

```

2026-06-28 12:41:49.861 async[63752:1281799] 1. старт на главной очереди
2026-06-28 12:41:49.862 async[63752:1281799] 4. главная очередь свободна, ждём событий
2026-06-28 12:41:50.366 async[63752:1281800] 2. загрузка закончена в фоне
2026-06-28 12:41:50.366 async[63752:1281800] 3. показываем результат: отчёт за июнь: 1000 строк

```

Посмотри на числа в скобках после имени программы — это процесс:поток. У пунктов 1 и 4 одинаковый номер потока (1281799) — это главный. У пункта 2 другой (1281800) — фоновый.

А почему у пункта 3 тоже 1281800, ведь его блок мы положили на главную очередь? Это особенность консольной программы с `dispatch_main()`. По документации он «паркует» главный поток, и дальше блоки главной очереди выполняет поток из пула GCD — по одному за раз, как и положено последовательной очереди. По нашим экспериментам `[NSThread isMainThread]` внутри такого блока отвечает NO. В приложении с интерфейсом (где крутится run loop главного потока) главная очередь всегда выполняется именно на главном потоке — туда и возвращают результат для экрана.

И обрати внимание на порядок: сначала напечаталось «4», и только потом «2». Так и должно быть: `dispatch_async` не ждёт, поэтому главный поток ушёл вперёд, пока фон полсекунды «скачивал».

`dispatch_sync` и дедлок

У `dispatch_async` есть брат — `dispatch_sync` (`sync = synchronous`): он кладёт блок в очередь и **ждёт**, пока тот выполнится, и только потом возвращает управление. Удобно, когда результат нужен прямо сейчас.

Но `dispatch_sync` опасен одним: **нельзя звать его на той же последовательной очереди, на которой сейчас работаешь**. Получится **дедлок** (deadlock, взаимная блокировка). Представь: ты на главной очереди и говоришь «положи задачу на главную очередь и подожди её». Но главная очередь сейчас занята — тобой. Она освободится, когда ты закончишь, а ты не закончишь, пока она не освободится. Оба ждут друг друга вечно. Программа повисает.

```
// НЕ ДЕЛАЙ ТАК на главном потоке: мгновенный дедлок.  
dispatch_sync(dispatch_get_main_queue(), ^{  
    NSLog(@"я никогда не выполняюсь");  
});
```

Правило простое: `dispatch_sync` — только на **другую** очередь, не на текущую последовательную. Для возврата на главную очередь почти всегда берут `dispatch_async`, а не `sync`.

`dispatch_after`: сделать позже

Иногда задачу надо выполнить не сейчас, а через паузу — например, спрятать сообщение через полторы секунды. Для этого есть `dispatch_after`:

```
dispatch_after(  
    dispatch_time(DISPATCH_TIME_NOW, (int64_t)(1.5 * NSEC_PER_SEC)),  
    dispatch_get_main_queue(),  
    ^{ NSLog(@"прошло 1.5 секунды"); });
```

`dispatch_time(DISPATCH_TIME_NOW, наносекунды)` строит момент «сейчас плюс столько-то». `NSEC_PER_SEC` — число наносекунд в секунде, поэтому `1.5 * NSEC_PER_SEC` — это полторы секунды. Второй аргумент — очередь, третий — блок. Это не «усыпить поток на 1.5 с», а «через 1.5 с положить блок в очередь» — поток всё это время свободен.

Шаг 2: ждём несколько загрузок — `dispatch_group`

Теперь задача сложнее. Надо скачать **три** файла параллельно и продолжить, только когда готовы все три. Если запустить три `dispatch_async`, они уйдут в фон вразнобой — как понять, что закончились все? Считать вручную флажки — путь к ошибкам. GCD даёт готовый инструмент: **группу** `dispatch_group_t`.

Идея: создаём группу, каждую задачу «приписываем» к ней, а потом одной строкой ждём, пока группа опустеет.

Файл `code/21-group.m`:

```
#import <Foundation/Foundation.h>

// Имитируем загрузку одного файла: пауза + результат.
static NSString *downloadFile(NSString *name) {
    [NSThread sleepForTimeInterval:0.3];
    return [NSString stringWithFormat:@"%s готов", name];
}

int main(void) {
    @autoreleasepool {
        NSArray<NSString *> *names = @[ @"картинки", @"видео", @"музыка" ];

        dispatch_group_t group = dispatch_group_create();
        dispatch_queue_t bg =
            dispatch_get_global_queue(QOS_CLASS_USER_INITIATED, 0);

        // Потокбезопасный сбор результатов: пишем в массив на своей
        // последовательной очереди, чтобы не было гонки (см. 21-race.m).
        NSMutableArray<NSString *> *results = [NSMutableArray array];
        dispatch_queue_t guard =
            dispatch_queue_create("results.guard", DISPATCH_QUEUE_SERIAL);

        NSLog(@"запускаем параллельно загрузок: %lu",
              (unsigned long)names.count);

        for (NSString *name in names) {
            // group_async: задача «приписана» к группе.
            dispatch_group_async(group, bg, ^{
```



```

        NSString *done = downloadFile(name);
        dispatch_sync(guard, ^{ [results addObject:done]; });
    });
}

// Ждём, пока ВСЕ задачи группы завершатся (блокируем main).
dispatch_group_wait(group, DISPATCH_TIME_FOREVER);

NSLog(@"все загрузки завершены, собрано результатов: %lu",
      (unsigned long)results.count);
for (NSString *r in results) {
    NSLog(@"  - %@", r);
}
}
return 0;
}

```

Новое:

- `dispatch_group_create()` — создаём группу-счётчик незавершённых задач.
- `dispatch_group_async(group, queue, block)` — как `dispatch_async`, но счётчик группы растёт сразу, в момент постановки блока в очередь, а падает, когда блок завершился. Поэтому `wait` сразу после цикла не проскочит мимо ещё не начатых задач.
- `dispatch_group_wait(group, DISPATCH_TIME_FOREVER)` — **блокирует текущий поток**, пока счётчик не обнулится. `DISPATCH_TIME_FOREVER` — «жди сколько угодно». Здесь блокировка уместна: главному потоку всё равно нечем заняться, а в конце нужны все результаты. (Можно дать и таймаут — тогда `wait` вернёт ненулевое число, если не дождались.)
- Запись в общий `results` обёрнута в `dispatch_sync(guard, ...)` на последовательную очередь `guard`. Это защита от гонки: три фоновых потока пишут в один массив, и без защиты можно повредить его. Подробно — в следующем разделе.

Собираем, запускаем:

```

clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/21-group.m -o group
./group

```

```

2026-06-28 12:41:52.184 group[63757:1281834] запускаем параллельно загрузок: 3
2026-06-28 12:41:52.489 group[63757:1281834] все загрузки завершены, собрано результатов: 3
2026-06-28 12:41:52.489 group[63757:1281834]  - картинки готов

```

```
2026-06-28 12:41:52.489 group[63757:1281834] - видео готов
2026-06-28 12:41:52.489 group[63757:1281834] - музыка готов
```

Три загрузки по 0.3 с прошли **параллельно** — общая пауза вышла около 0.3 с, а не 0.9 с. А вот **порядок** в списке от запуска к запуску меняется: какой поток успел добавить результат раньше, тот и первый. Параллельная очередь не обещает порядок — если он важен, его надо обеспечить самому (например, писать результат в `results[i]` по индексу, а не `addObject:`).

group_notify вместо ожидания

`dispatch_group_wait` останавливает поток. Часто это не нужно — хочется просто «когда всё будет готово, выполни вот это, а пока иди дальше». Для этого есть `dispatch_group_notify`:

```
dispatch_group_notify(group, dispatch_get_main_queue(), ^{
    NSLog(@"все загрузки готовы - обновляю экран");
});
// поток НЕ блокируется, идёт дальше
```

Блок-уведомление поставится в указанную очередь автоматически, как только группа опустеет. В приложении с интерфейсом берут именно `notify` на главную очередь — поток не замораживается. В нашей консольной программе проще `wait`, потому что иначе пришлось бы крутить `dispatch_main`.

dispatch_barrier — кратко

Ещё один член семьи — **барьер**. На **своей параллельной** очереди `dispatch_barrier_async(q, ^{ ... })` ставит блок, который выполнится **в одиночку**: GCD дождётся завершения всех задач, запущенных до барьера, выполнит барьерный блок, не пуская параллельно никого, и только потом продолжит остальные. Классическое применение — «много читателей, один писатель»: чтения идут параллельно, а запись — через барьер, чтобы никто не читал в момент изменения. Барьер работает только на очереди, которую ты создал сам через `DISPATCH_QUEUE_CONCURRENT`; на глобальных очередях он просто ведёт себя как обычный `async`.

Шаг 3: одноразовая инициализация — dispatch_once

Частая задача — **синглтон**: объект, который должен существовать в одном экземпляре на всю программу (настройки, общий кэш). Наивно:

```
+ (instancetype)shared {
    static Settings *instance = nil;
    if (instance == nil) {                // ОПАСНО в многопоточе
```

```

        instance = [[Settings alloc] init];
    }
    return instance;
}

```

static внутри метода значит «одна переменная на все вызовы, живёт до конца программы» (то же static, что в Си). Но проверка if (instance == nil) небезопасна: если shared зовут два потока разом, оба могут увидеть nil и создать **по объекту**. Синглтон перестал быть единственным.

GCD решает это одной строкой — dispatch_once:

Файл code/21-once.m:

```

#import <Foundation/Foundation.h>

@interface Settings : NSObject
@property (nonatomic, copy) NSString *theme;
+ (instancetype)shared;
@end

@implementation Settings

+ (instancetype)shared {
    static Settings *instance = nil;
    static dispatch_once_t onceToken;
    // Блок выполнится РОВНО ОДИН раз за всё время работы программы,
    // даже если shared зовут из десятка потоков одновременно.
    dispatch_once(&onceToken, ^{
        instance = [[Settings alloc] init];
        instance.theme = @"тёмная";
        NSLog(@" >>> Settings создан (это видно один раз)");
    });
    return instance;
}

@end

int main(void) {
    @autoreleasepool {

```

```

dispatch_group_t group = dispatch_group_create();
dispatch_queue_t bg =
    dispatch_get_global_queue(QOS_CLASS_DEFAULT, 0);

// Зовём shared из 8 параллельных задач сразу.
for (int i = 0; i < 8; i++) {
    dispatch_group_async(group, bg, ^{
        Settings *s = [Settings shared];
        (void)s;    // нам важен лишь факт обращения
    });
}
dispatch_group_wait(group, DISPATCH_TIME_FOREVER);

Settings *a = [Settings shared];
Settings *b = [Settings shared];
NSLog(@"a == b ? %@", (a == b) ? @"да, один объект" : @"нет");
NSLog(@"тема: %@", a.theme);
}
return 0;
}

```

Разбор:

- `static dispatch_once_t onceToken;` — служебный флажок-токен. Должен быть `static` (один на все вызовы) и обнулён по умолчанию — `static` это гарантирует.
- `dispatch_once(&onceToken, ^{ ... })` — передаём **адрес** токена (&, как в Си) и блок. GCD выполнит блок ровно один раз за жизнь программы; все остальные вызовы (хоть из разных потоков одновременно) тихо подождут завершения первого и пройдут мимо. Безопасно и быстро.
- `(void)s;` — приём «использовать, чтобы компилятор не ругался на неиспользуемую переменную». Без него `-Wextra` выдал бы предупреждение.

Запуск:

```

clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/21-once.m -o once
./once

```

```

2026-06-28 12:41:54.113 once[63762:1281909]    >>> Settings создан (это видно один раз)
2026-06-28 12:41:54.113 once[63762:1281883] а == b ? да, один объект
2026-06-28 12:41:54.113 once[63762:1281883] тема: тёмная

```

Несмотря на 8 параллельных обращений, строка «Settings создан» напечаталась **один раз**, а а == b дало «да, один объект». Синглтон действительно единственный.

Отличие от Си. В Си для одноразовой инициализации в многопоточе заводят флаг и мьютекс руками: при каждом вызове берёшь pthread_mutex_lock, проверяешь флаг if (!done) { init(); done = 1; }, отпускаешь pthread_mutex_unlock. Это работает, но громоздко, и легко ошибиться (забыть unlock, проверить флаг без блокировки). dispatch_once делает то же самое одной строкой, потокобезопасно и без замка на горячем пути: после первого раза проверка почти бесплатна. (В POSIX есть похожий pthread_once, но dispatch_once ещё проще и быстрее.)

Гонка данных вживую

Гонка данных (race condition) — это когда два потока без согласования лезут к одной переменной или коллекции, и результат зависит от того, кто кого опередил. Покажем беду на самом простом примере: два потока миллион раз увеличивают один счётчик.

Казалось бы, bad++ — одна операция. На деле это три шага: **прочитать** текущее значение, **прибавить 1**, **записать** обратно. Если два потока прочитали одно и то же значение, оба прибавили 1 и оба записали — один инкремент потерян. На миллионах операций таких потерь будут сотни тысяч. (Формально по правилам Си одновременная запись без защиты — неопределённое поведение, так что «сколько потеряется» не обещает никто.)

Файл code/21-race.m:

```
#import <Foundation/Foundation.h>

#define ITERATIONS 1000000

int main(void) {
    @autoreleasepool {
        dispatch_queue_t concurrent =
            dispatch_get_global_queue(QOS_CLASS_DEFAULT, 0);

        // --- Версия с гонкой: два потока бьются за одну переменную ---
        __block long bad = 0;
        dispatch_apply(2, concurrent, ^(size_t i) {
            (void)i;
            for (int k = 0; k < ITERATIONS; k++) {
                bad++;          // НЕ атомарно: чтение, +1, запись
            }
        })
    }
}
```

```

});
NSLog(@"гонка:   ожидали %d, получили %ld",
      ITERATIONS * 2, bad);

// --- Починка через последовательную очередь ---
__block long good = 0;
dispatch_queue_t guard =
    dispatch_queue_create("counter.guard", DISPATCH_QUEUE_SERIAL);
dispatch_apply(2, concurrent, ^(size_t i) {
    (void)i;
    for (int k = 0; k < ITERATIONS; k++) {
        dispatch_sync(guard, ^{ good++; });
    }
});
NSLog(@"очередь: ожидали %d, получили %ld",
      ITERATIONS * 2, good);

// --- Починка через @synchronized ---
__block long locked = 0;
NSObject *lock = [NSObject new];
dispatch_apply(2, concurrent, ^(size_t i) {
    (void)i;
    for (int k = 0; k < ITERATIONS; k++) {
        @synchronized (lock) { locked++; }
    }
});
NSLog(@"@synchronized: ожидали %d, получили %ld",
      ITERATIONS * 2, locked);
}
return 0;
}

```

Новое:

- `#define ITERATIONS 1000000` — макрос-константа (из Си): препроцессор подставит `1000000` всюду, где встретит `ITERATIONS`.
- `dispatch_apply(count, queue, ^(size_t i){ ... })` — запускает блок `count` раз, параллельно, и **ждёт**, пока все повторы закончатся. Удобно для «сделай N одинаковых кусков работы». `size_t i` — номер повтора (0 и 1). Нам номер не нужен, гасим его `(void)i`.

- `@synchronized` (объект) `{ ... }` — встроенная в язык блокировка. Пока один поток внутри блока, остальные с тем же объектом-«ключом» ждут у входа. Объект `lock` тут — просто метка замка, его роль — быть общим ориентиром для всех потоков.

Запуск:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/21-race.m -o race
./race
```

```
2026-06-28 12:41:55.446 race[63767:1281956] гонка:   ожидали 2000000, получили 1185916
2026-06-28 12:42:03.461 race[63767:1281956] очередь: ожидали 2000000, получили 2000000
2026-06-28 12:42:04.013 race[63767:1281956] @synchronized: ожидали 2000000, получили 2000000
```

Первая строка — катастрофа: ждали 2 000 000, получили 1 185 916. Около 40 % инкрементов потеряно в гонке. И число **каждый раз разное** — это и есть коварство гонок: баг плавающий, ловится тяжело.

Обе починки дают ровно 2 000 000:

- **Последовательная очередь** `guard`: каждый `good++` идёт через `dispatch_sync(guard, ...)`, а `serial`-очередь пускает по одной задаче. Два потока физически не могут менять `good` одновременно.
- **@synchronized**: вход в блок защищён замком, внутри всегда один поток.

Третий способ — **NSLock**, объект-замок с явными `lock/unlock`:

```
NSLock *lock = [[NSLock alloc] init];
[lock lock];
shared++;          // защищённый участок
[lock unlock];
```

`NSLock` гибче `@synchronized` (можно, например, `tryLock`), но требует дисциплины: забыл `unlock` — повесишь программу. У всех трёх способов есть цена: защита сериализует доступ, и поэтому «починенные» версии в примере считались заметно дольше гонки. Посмотри на время в выводе: вариант с очередью шёл около 8 секунд, `@synchronized` — полсекунды. Два миллиона `dispatch_sync` ради одного `++` — дорогое удовольствие. Защищай **только** общие данные и **короткие** участки — не оборачивай в замок всё подряд.

Какой способ выбрать? Последовательная очередь GCD хороша, когда к общему объекту обращаются не миллионы раз подряд, а время от времени: весь доступ к нему живёт в одном месте, явных замков нет, а через `dispatch_async` на неё можно даже не ждать. Для частой защиты пары строк — `@synchronized` или `NSLock` (второй быстрее и умеет `tryLock`). А для одного

голового счётчика в реальном коде берут атомарные операции Си из `<stdatomic.h>` — они вообще обходятся без замка.

NSOperation: объектная надстройка над GCD

GCD — это блоки и очереди. Иногда хочется большего: отменять задачи, задавать зависимости («B не раньше A»), ограничивать число одновременных задач. Для этого Foundation даёт **NSOperation** — задачу как объект, и **NSOperationQueue** — очередь таких задач. Под капотом — тот же GCD, но с удобной объектной обвязкой.

Самый простой способ создать операцию — `NSBlockOperation` из блока:

```
NSOperationQueue *queue = [[NSOperationQueue alloc] init];
queue.maxConcurrentOperationCount = 2;    // не больше 2 задач разом

NSBlockOperation *download = [NSBlockOperation blockOperationWithBlock:^(
    [NSThread sleepForTimeInterval:0.2];
    NSLog(@"1. скачали данные");
)];

NSBlockOperation *process = [NSBlockOperation blockOperationWithBlock:^(
    NSLog(@"2. обработали (после скачивания)");
)];

[process addDependency:download];    // process ждёт download

[queue addOperations:@[download, process] waitUntilFinished:YES];
NSLog(@"3. очередь пуста");
```

Что здесь нового:

- `maxConcurrentOperationCount` — потолок параллельности. 1 сделает очередь последовательной.
- `addDependency:` — «эта операция не начнётся, пока не завершится та». Так строят порядок: сначала скачали, потом обработали. С голым GCD пришлось бы вкладывать блоки или городить группу.
- `addOperations:waitUntilFinished:` — добавляет операции и (с YES) ждёт их завершения. Это для нашей консольной программы, чтобы она не закрылась раньше.

Запуск этого фрагмента печатает стабильно:

```
2026-06-28 12:42:27.935 op[63785:1282325] 1. скачали данные
2026-06-28 12:42:27.936 op[63785:1282325] 2. обработали (после скачивания)
2026-06-28 12:42:27.936 op[63785:1282313] 3. очередь пуста
```


Зависимость сработала: «обработали» всегда после «скачали», хотя очередь параллельная.

Главная фишка `NSOperation`, которой нет у простого `dispatch_async`, — **отмена**. У операции есть метод `cancel` и свойство `isCancelled`:

```
[someOperation cancel];           // просим отмениться
// внутри длинной операции периодически проверяют:
if (operation.isCancelled) return;
```

`cancel` лишь **ставит флаг**; операция должна сама время от времени смотреть `isCancelled` и прекращать работу. Ещё не начатую операцию очередь после `cancel` просто не запустит.

Когда что брать. Нужна простая «отправь в фон и забудь», разовый прыжок на главную очередь, группа загрузок — бери **GCD**, он легче. Нужны отмена, зависимости между задачами, ограничение параллельности, переиспользуемые операции-объекты — бери **NSOperation**.

NSThread — самый низкий уровень

Под GCD и `NSOperation` лежит **NSThread** — обёртка над «сырым» потоком. Поток можно создать и запустить вручную:

```
[NSThread detachNewThreadSelector:@selector(work)
                toTarget:self
                withObject:nil];
```

или классом-наследником `NSThread` с переопределённым `main`. Ещё у `NSThread` есть полезные мелочи, которые мы уже использовали: `[NSThread sleepForTimeInterval:]` (усыпить текущий поток), `[NSThread currentThread]`, `[NSThread isMainThread]`.

На практике **создавать NSThread руками почти никогда не нужно**: GCD и `NSOperation` делают то же безопаснее и эффективнее (переиспользуют пул потоков, а не плодят новые). Знать о нём стоит, выбирать — в последнюю очередь.

Run loop: цикл ожидания событий

Вернёмся к главному потоку. Откуда он знает, что пора обработать нажатие или сработавший таймер? Внутри него крутится **run loop** (`NSRunLoop`, цикл выполнения) — бесконечный цикл, который **спит, пока нет событий**, а как событие появилось (таймер сработал, пришли данные) — будит поток, выполняет нужный код и снова засыпает. Без `run loop` поток просто дошёл бы до конца кода и завершился.

В приложении run loop главного потока запускает система — это он бесконечно ждёт нажатия. В консольной программе run loop не крутится сам; если он нам нужен (например, для таймера), его запускают руками.

Зачем он нам сейчас? Из-за **NSTimer**. Таймер — это объект, который через заданные промежутки шлёт сообщение. Но таймер **сам не считает время в вакууме**: он регистрируется в run loop, и именно run loop в нужный момент его «дёргает». Нет работающего run loop — таймер не работает ни разу. Это типичная ловушка новичка.

Создают повторяющийся таймер так:

```
[NSTimer scheduledTimerWithTimeInterval:0.2    // каждые 0.2 секунды
                                target:self
                                selector:@selector(fire:)
                                userInfo:nil
                                repeats:YES];
```

`scheduledTimer...` и создаёт таймер, и сразу регистрирует его в run loop текущего потока. Каждые 0.2 с run loop пошлёт `self` сообщение `fire:`, передав сам таймер аргументом. `repeats:YES` — повторять; `NO` — сработать один раз. Остановить таймер — `[timer invalidate]` (снимает его с run loop).

Шаг 4: таймер рассылает уведомления

Соберём всё в одной программе и заодно введём последнюю тему главы — **NSNotificationCenter**. Это «радиостанция» внутри приложения: один объект **рассылает** уведомление по имени, а сколько угодно других **подписчиков** его ловят. Отправитель не знает, кто слушает, — он просто «вещает в эфир».

Отличие от делегата. Делегат (глава 10) — связь **один-к-одному**: у объекта ровно один делегат, и он шлёт сообщения только ему. **NSNotificationCenter** — связь **один-ко-многим**: одно уведомление получают все, кто на него подписан, и отправитель их не перечисляет. Делегат — когда нужен один конкретный ответчик; уведомления — когда событие интересно многим и заранее неизвестно кому.

Построим: объект `Clock` каждые 0.2 с бьёт таймером и **рассылает** уведомление; объект `Display` на него **подписан** и реагирует. Связи между ними напрямую нет — только через центр уведомлений.

Файл `code/21-notification.m`:

```
#import <Foundation/Foundation.h>

// Имя уведомления вынесем в константу, чтобы не опечататься.
static NSString *const TickNotification = @"TickNotification";

// Подписчик: получает уведомления и реагирует.
@interface Display : NSObject
@property (nonatomic) NSInteger seen;
@end

@implementation Display

- (instancetype)init {
    self = [super init];
    if (self) {
        // Подписываемся: при уведомлении TickNotification зови onTick:.
        [[NSNotificationCenter defaultCenter]
            addObserver:self
            selector:@selector(onTick:)
            name:TickNotification
            object:nil];
    }
    return self;
}

- (void)onTick:(NSNotification *)note {
    self.seen += 1;
    NSNumber *count = note.userInfo[@"count"];
    NSLog(@"    Display получил тик #%@ (всего видел %ld)",
        count, (long)self.seen);
}

- (void)dealloc {
    // С macOS 10.11/iOS 9 центр сам забывает умерших подписчиков,
    // но явная отписка – хорошая привычка (см. разбор ниже).
    [[NSNotificationCenter defaultCenter] removeObserver:self];
    NSLog(@"    Display освобождён, отписался");
}
```

```
@end

// Источник тиков: таймер бьёт каждые 0.2 с и шлёт уведомление.
@interface Clock : NSObject
@property (nonatomic) NSInteger tick;
@property (nonatomic, strong) NSTimer *timer;
@end

@implementation Clock

- (void)start {
    self.timer =
        [NSTimer scheduledTimerWithTimeInterval:0.2
                                     target:self
                               selector:@selector(fire:)
                               userInfo:nil
                               repeats:YES];
}

- (void)fire:(NSTimer *)timer {
    self.tick += 1;
    NSLog(@"Clock: тик %ld, рассылаю уведомление", (long)self.tick);
    [[NSNotificationCenter defaultCenter]
        postNotificationName:TickNotification
                      object:self
        userInfo:@{ @"count": @(self.tick) }];
    if (self.tick >= 3) {
        [timer invalidate]; // снимаем таймер с run loop
    }
}

@end

int main(void) {
    @autoreleasepool {
        Display *display = [[Display alloc] init];
        Clock *clock = [[Clock alloc] init];
    }
}
```

```
[clock start];

NSLog(@"крутим run loop 1 секунду...");
// Без работающего run loop таймер НЕ сработает ни разу.
[[NSRunLoop currentRunLoop]
    runUntilDate:[NSDate dateWithTimeIntervalSinceNow:1.0]];

NSLog(@"готово: Display видел тиков: %ld", (long)display.seen);
}
return 0;
}
```

Разберём центр уведомлений по методам.

- `[NSNotificationCenter defaultCenter]` — общий на всю программу центр. Через него и подписываются, и рассылают.
- `addObserver:selector:name:object:` — подписка. `addObserver:` — кто слушает (`self`). `selector:` — какой метод позвать (`@selector(onTick:)`), селектор из главы 5; метод обязан принимать один аргумент `NSNotification *`. `name:` — имя интересующего уведомления. `object:` — от кого слушать; `nil` значит «от любого отправителя».
- `postNotificationName:object:userInfo:` — рассылка. `object:` — кто шлёт (обычно `self`). `userInfo:` — словарь с попутными данными; здесь кладем номер тика, подписчик достаёт его через `note.userInfo[@"count"]`.
- `removeObserver:` — отписка. Раньше её в `dealloc` звали строго обязательно: подписчик умер, не отписавшись, — и центр слал сообщение по «протухшему» адресу. Для программ под macOS 10.11 / iOS 9 и новее, как сказано в документации `addObserver:selector:name:object:`, центр сам забывает умерших подписчиков такого вида. Явная отписка в `dealloc` не вредит и нужна, если твой код должен работать и на старых системах. А вот блочную подписку `addObserverForName:object:queue:usingBlock:` снимать надо всегда: она возвращает объект-токен, и его передают в `removeObserver:`.

Про run loop в main:

- `[NSRunLoop currentRunLoop]` — run loop текущего (главного) потока.
- `runUntilDate:` — запускает цикл и крутит его до указанного момента, обрабатывая по дороте события (в том числе срабатывания таймера). `[NSDate dateWithTimeIntervalSinceNow:1.0]` — «сейчас плюс 1 секунда». Эта строка — то, что **даёт таймеру ожить**: целую секунду run loop работает, и таймер успевает ударить три раза по 0.2 с. Без неё main закрылся бы мгновенно, и тиков не было бы вовсе.

Запуск:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/21-notification.m -o notification
./notification
```

```
2026-06-28 12:42:13.783 notification[63774:1282130] крутим run loop 1 секунду...
2026-06-28 12:42:13.985 notification[63774:1282130] Clock: тик 1, рассылаю уведомление
2026-06-28 12:42:13.985 notification[63774:1282130]    Display получил тик #1 (всего видел 1)
2026-06-28 12:42:14.185 notification[63774:1282130] Clock: тик 2, рассылаю уведомление
2026-06-28 12:42:14.185 notification[63774:1282130]    Display получил тик #2 (всего видел 2)
2026-06-28 12:42:14.385 notification[63774:1282130] Clock: тик 3, рассылаю уведомление
2026-06-28 12:42:14.385 notification[63774:1282130]    Display получил тик #3 (всего видел 3)
2026-06-28 12:42:14.786 notification[63774:1282130] готово: Display видел тиков: 3
2026-06-28 12:42:14.786 notification[63774:1282130]    Display освобождён, отписался
```

Цепочка видна целиком: таймер на run loop бьёт три раза, каждый удар Clock превращает в уведомление, Display его ловит и считает. Никто из них не держит ссылку друг на друга — связь только через центр. В конце блока @autoreleasepool переменная display выходит из области видимости, ARC отпускает объект, срабатывает dealloc и честная отписка. Заметь: всё на одном потоке (1282130) — NSTimer и уведомления здесь синхронные, никакой многопоточности тут нет, только run loop.

Уведомления, кстати, по умолчанию доставляются **синхронно и на том же потоке**, где их послали: postNotificationName: не вернётся, пока все подписчики не отработают onTick:. Это просто прямые вызовы под капотом, а не «фон».

Частые ошибки

- **Тронул интерфейс с фонового потока.** UIKit/AppKit можно дёргать только с главного потока. Готов результат в фоне — возвращайся `dispatch_async(dispatch_get_main_queue(), ...)` и обновляй экран там.
- **dispatch_sync на текущую последовательную очередь** (особенно на главную из главного потока) — мгновенный дедлок. Возвращайся через `async`.
- **Ждал результат, а программа закрылась.** В консольных примерах фон не успевает: нужен `dispatch_group_wait`, `dispatch_main + exit`, или `runUntilDate:`. В приложении эту роль играет вечный run loop.
- **Таймер не срабатывает.** Почти всегда — нет работающего run loop, или таймер создан на потоке без него. На главном потоке приложения он есть; в своём фоновом — заводи и крути сам.

- **Забыл `removeObserver:`**. Для `addObserver(selector:...` на macOS 10.11+/iOS 9+ это уже не падение, но на старых системах — крах при следующей рассылке. Токен блочной подписки (`addObserverForName:...usingBlock:`) снимай всегда: иначе блок будет вызываться и дальше, а всё, что он захватил, не освободится.
- **Защитил замком слишком много**. Если весь код под `@synchronized`, потоки выстраиваются в очередь и параллельность пропадает. Защищай только короткий доступ к общим данным.
- **`dispatch_once_t` не `static`**. Токен обязан быть один на все вызовы и обнулён. Документация прямо говорит: токен в обычной локальной переменной, в куче или в `ivar` — неопределённое поведение. Локальная переменная создаётся заново при каждом вызове (и без `static` даже не обнулена), так что никакого «один раз» уже не будет.

Упражнения

1. В `21-async.m` добавь `dispatch_after` на главную очередь, который через 1 секунду печатает «таймаут». Перенеси `exit(0)` из блока с результатом в блок таймаута — иначе программа закроется раньше, чем он сработает. Проследи порядок: результат через 0.5 с, таймаут через 1 с.
2. Перепиши `21-group.m` так, чтобы порядок результатов **совпадал** с порядком имён: цикл сделай по индексу `i`, заранее заполни `results` тремя `[NSString null]` (в `NSMutableArray` нельзя писать за его конец) и пиши `results[i] = done` вместо `addObject:`. Подумай, нужна ли тут защита (подсказка: `NSMutableArray` не потокобезопасен, даже если потоки пишут в разные ячейки).
3. Замени в `21-group.m` `dispatch_group_wait` на `dispatch_group_notify` с `exit(0)` внутри и добавь `dispatch_main`. Сравни поведение.
4. В `21-race.m` добавь четвёртый вариант счётчика на `NSLock` и проверь, что он тоже даёт ровно 2 000 000.
5. В `21-notification.m` заведи второй подписчик `Logger`, который тоже слушает `TickNotification` и пишет своё. Убедись, что одно уведомление получают **оба** (один-ко-многим).
6. Сделай на `NSOperationQueue` три операции: «скачать А», «скачать В» и «склеить», где «склеить» зависит от обеих загрузок (`addDependency:` дважды). Проверь, что склейка всегда последняя.

Что мы получили

Теперь ты умеешь уносить долгую работу с главного потока и возвращать результат обратно. Разобрали GCD: очереди (главная, глобальные, свои `serial/concurrent`), `dispatch_`

async/dispatch_sync с предупреждением о дедлоке, dispatch_after, группы (group_async + wait/notify), барьер и потокобезопасный dispatch_once для синглтона. Увидели гонку данных живьём и починили её последовательной очередью, @synchronized и NSLock. Поднялись до NSOperation с зависимостями и отменой, заглянули в NSThread, завели таймер на run loop и связали объекты через NSNotificationCenter по схеме «один-ко-многим».

Везде мы держали вывод **детерминированным** — ждали задачи явно (wait, dispatch_main + exit, runUntilDate:), а не наугад. Это рабочая привычка: асинхронный код должен сам решать, когда он закончен.

Дальше — глава 22 про современный Objective-C: что в языке появилось за последние годы и как он выглядит в сегодняшнем коде.

Документация Apple

- Concurrency Programming Guide (архив Apple) — developer.apple.com/library/archive/documentation/General/Conceptual/ConcurrencyProgrammingGuide/Introduction/Introduction.html (обзор GCD, NSOperation, очередей и потоков).
- Threading Programming Guide, глава Run Loops (архив Apple) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/Multithreading/RunLoopManagement/RunLoopManagement.html
- Dispatch (GCD) — developer.apple.com/documentation/dispatch (dispatch_async, dispatch_sync, dispatch_group, dispatch_once, dispatch_after, барьеры).
- NSOperation — developer.apple.com/documentation/foundation/operation
- NSOperationQueue — developer.apple.com/documentation/foundation/operationqueue
- NSThread — developer.apple.com/documentation/foundation/thread
- NSRunLoop — developer.apple.com/documentation/foundation/runloop
- NSTimer — developer.apple.com/documentation/foundation/timer
- NSNotificationCenter — developer.apple.com/documentation/foundation/notificationcenter

Глава 22. Современный Objective-C

Язык, который ты прошёл за двадцать одну главу, написан в конце 1980-х. Но писать на нём сегодня — совсем не то же самое, что писать на нём тогда. За годы Apple добавила в Objective-C целый слой удобств и страховок: короткие литералы вместо громоздких вызовов, типобезопасные перечисления вместо голых `enum`, аннотации `nullable/nonnull`, чтобы описать, где допустим `nil`, дженерики у коллекций, проверку версии ОС прямо в коде. Часть этого уже мелькала в прошлых главах поодиночке. Теперь соберём «современное лицо» языка в одном месте и разберём каждый знак.

Главное, что объединяет всё в этой главе: **компилятор начинает ловить больше ошибок за тебя**. Раньше многое всплывало только в рантайме падением или мусором на экране — теперь `clang` ругается ещё на этапе сборки. В конце главы мы возьмём наивный класс из ранних глав и «осовременим» его, наблюдая, как с каждой аннотацией компилятор становится строже.

Что мы сделаем

- Сведём в одну таблицу литералы и субскрипты и увидим, что это сахар над обычными методами; добавим субскрипт в свой класс.
- Заменяем голый `enum` на `NS_ENUM` и битовые флаги на `NS_OPTIONS`.
- Опишем контракт по `nil` через `nullable/nonnull` и макросы `NS_ASSUME_NONNULL_BEGIN/END`.
- Навесим на коллекции дженерики `NSArray<NSString*> *`.
- Вспомним `instancetype` и пометим назначенный инициализатор `NS_DESIGNATED_INITIALIZER`.
- Проверим версию ОС через `@available`.
- Подключим Foundation модулем `@import`.
- Соберём всё это в одном осовремененном классе.

Литералы и субскрипты — сводка

Литерал-строку @"..." ты используешь с главы 1, а литералы коллекций @[] и @{} — с главы 9 (подробно они разобраны в главе 17). Здесь важно понять одну мысль: **каждый литерал и каждый субскрипт — это просто короткая запись обычного вызова метода**. Компилятор разворачивает её сам.

Вот вся «короткая запись» в одном списке:

```
NSNumber *n      = @42;           // [NSNumber numberWithInt:42]
NSNumber *flag   = @YES;          // [NSNumber numberWithBool:YES]
NSNumber *pi     = @3.14;         // [NSNumber numberWithDouble:3.14]
NSString *s      = @"строка";    // объект NSString
NSArray *arr     = @[@"a", @"b"]; // массив
NSDictionary *d  = @{@"one": @1};  // словарь
NSNumber *boxed  = @(x + y);      // @() — упаковка любого выражения
```

Знак @ здесь каждый раз говорит: «то, что справа, — это объект Objective-C, а не голое значение». @42 — это не число int, а полноценный объект NSNumber, которому можно слать сообщения.

Субскрипты — квадратные скобки доступа — тоже сахар:

```
arr[1]           // = [arr objectAtIndexedSubscript:1]
m[1] = @"b";     // = [m setObject:@"b" atIndexedSubscript:1]
d[@"one"]        // = [d objectForKeyedSubscript:@"one"]
md[@"k"] = @1;   // = [md setObject:@1 forKeyedSubscript:@"k"]
```

Здесь m — это NSMutableArray *, а md — NSMutableDictionary *. У неизменяемых NSArray/NSDictionary метода записи нет, и строка arr[1] = @"b"; не соберётся: expected method to write array element not found on object of type 'NSArray *'. Для чтения у NSArray метод objectAtIndexedSubscript: делает то же, что привычный objectAtIndex:, а objectForKeyedSubscript: у словаря — то же, что objectForKey:.

То есть скобки бывают двух видов, и компилятор различает их по типу ключа:

- **индексный субскрипт** (arr[i], ключ — целое число) раскрывается в objectAtIndexedSubscript: для чтения и setObject:atIndexedSubscript: для записи;
- **ключевой субскрипт** (d[key], ключ — объект) раскрывается в objectForKeyedSubscript: и setObject:forKeyedSubscript:.

Отличие от Си. В Си arr[i] — это адресная арифметика: *(arr + i), прямой доступ к байтам в памяти, без всякой проверки. В Objective-C arr[i] для объекта —

это вызов метода: runtime найдёт `objectAtIndexedSubscript:` и выполнит его, со всеми проверками границ внутри. Одинаковые скобки, совершенно разная механика.

Свой субскрипт

Раз субскрипт — это просто пара методов, мы можем поддержать его в любом своём классе. Достаточно объявить нужные методы — и `obj[0]` заработает. Сделаем «полку» с ячейками:

```
@interface Shelf : NSObject
- (instancetype)initWithCapacity:(NSUInteger)capacity;
- (id)objectAtIndexedSubscript:(NSUInteger)index;
- (void)setObject:(id)object atIndexedSubscript:(NSUInteger)index;
@end
```

Объявив именно эти два метода с такими именами, мы включаем синтаксис `shelf[i]`. Реализация хранит элементы в обычном `NSMutableArray`, но с проверкой границ:

```
- (id)objectAtIndexedSubscript:(NSUInteger)index {
    if (index >= _slots.count) {
        return @"(нет такой полки)";    // не падаем на выходе за границу
    }
    return _slots[index];
}

- (void)setObject:(id)object atIndexedSubscript:(NSUInteger)index {
    if (index < _slots.count) {
        _slots[index] = object;
    }
}
```

Теперь снаружи класс ведёт себя как массив:

```
Shelf *shelf = [[Shelf alloc] initWithCapacity:3];
NSLog(@"новая полка[0] = %@", shelf[0]);    // objectAtIndexedSubscript:
shelf[0] = @"книга";                        // setObject:atIndexedSubscript:
shelf[1] = @"чашка";
NSLog(@"полка[9] = %@", shelf[9]);          // выход за границы — не падаем
```

Полный файл — `code/22-literals-subscript.m`. Реальный вывод:

```
число 42, флаг 1, пи 3.14
строка: строка
arr[1] = b
```

```
d["two"] = 2
сумма в коробке: 12
новая полка[0] = (пусто)
полка[0] = книга, полка[1] = чашка
полка[9] = (нет такой полки)
```

Обрати внимание: @YES напечаталось как 1 — внутри NSNumber булево хранится как число. А shelf[9] не уронил программу, потому что мы сами проверили границу внутри метода-субскрипта.

NS_ENUM — типобезопасные перечисления

В главе 7 NS_ENUM упоминался только мимоходом, как обещание. Теперь разберём, что это и зачем.

Раньше состояния кодировали голым си-enum:

```
enum { TaskTodo, TaskInProgress, TaskDone }; // старый способ
```

Проблема в том, что у такого enum **нет своего типа**. Это просто набор именованных целых. Переменную под него ты заведёшь как int или NSInteger, и компилятор не будет следить, что туда кладут именно значения этого перечисления. Можно по ошибке присвоить 42, и никто не возразит.

NS_ENUM чинит это наполовину. Он одновременно задаёт **имя типа** и **тип хранения**:

```
typedef NS_ENUM(NSInteger, TaskStatus) {
    TaskStatusTodo,          // 0
    TaskStatusInProgress,    // 1
    TaskStatusDone,          // 2
};
```

Разберём строку NS_ENUM(NSInteger, TaskStatus):

- первый аргумент NSInteger — **на чём хранить** значения (целое машинного размера);
- второй аргумент TaskStatus — **имя нового типа**.

Теперь переменная TaskStatus st; имеет настоящий тип. Что это даёт:

1. **Документирование**. По сигнатуре – (void)setStatus:(TaskStatus)st сразу видно, какие значения уместны.
2. **Проверка switch**. Если в switch по TaskStatus забыть один из случаев, компилятор предупредит (-Wswitch). Для голого int он молчит — ему не из чего узнать полный список.

3. **Мост в Swift.** NS_ENUM приезжает в Swift настоящим enum (TaskStatus.inProgress), а голый си-enum — нет. Подробнее о мосте — в главе 23.

Одного NS_ENUM не делает: запретить присвоить «чужое» число он не может. Строка TaskStatus st = 42; соберётся даже с -Wall -Wextra без единого предупреждения (мы проверили) — правила Си для enum никуда не делись. Тип помогает читать код и проверять switch, но не стережёт присваивания.

Превращаем статус в текст и пользуемся защитой switch:

```
static NSString *statusName(TaskStatus status) {
    switch (status) {
        case TaskStatusTodo:      return @"в очереди";
        case TaskStatusInProgress: return @"в работе";
        case TaskStatusDone:      return @"готово";
    }
    return @"?";
}
```

Если ты допишешь в перечисление TaskStatusCancelled, но забудешь добавить его в этот switch, clang скажет прямо при сборке:

```
warning: enumeration value 'TaskStatusCancelled' not handled in switch
```

То есть компилятор напоминает обновить все разборы статуса. С голым int такого напоминания нет — ошибка дожидается до рантайма.

Отличие от Си. В Си enum — это просто способ дать имена целым числам; под капотом это int, и никакой типобезопасности нет: любое целое влезет в enum-переменную без возражений. NS_ENUM поверх того же си-механизма добавляет настоящий именованный тип, на который опираются и компилятор (проверка switch), и Swift (честный enum при импорте). Сама форма записи остаётся си-совместимой — NS_ENUM разворачивается в typedef enum с явным типом хранения: enum TaskStatus : NSInteger { ... } плюс служебный атрибут для Swift.

NS_OPTIONS — битовые флаги

Иногда значение — не одно из списка, а **набор** одновременно: права «чтение И запись», стороны выравнивания «слева И сверху». Это битовые маски, и для них есть NS_OPTIONS.

Выглядит почти как NS_ENUM, но значения — степени двойки, чтобы каждое занимало свой отдельный бит:

```
typedef NS_OPTIONS(NSUInteger, Permissions) {
    PermissionNone    = 0,          // 0000
    PermissionRead    = 1 << 0,    // 0001
    PermissionWrite    = 1 << 1,    // 0010
    PermissionDelete   = 1 << 2,    // 0100
    PermissionAll      = PermissionRead | PermissionWrite | PermissionDelete,
};
```

Запись $1 \ll 0$, $1 \ll 1$, $1 \ll 2$ — это **сдвиг влево**: берём единицу и двигаем её на 0, 1, 2 разряда. Получаем 0001, 0010, 0100 — по одному поднятому биту в каждом флаге. Тип хранения здесь `NSUInteger` — беззнаковое целое, потому что биты, а не «номер из списка».

Два оператора работают с такими флагами:

- **| (ИЛИ)** — **собрать** флаги вместе: `Read | Write` поднимет оба бита.
- **& (И)** — **проверить**, поднят ли бит: `perms & Read` даст ненулевое значение, если чтение разрешено.

Соберём права и проверим их по отдельности:

```
// 0001 | 0010 = 0011, то есть 3
Permissions perms = PermissionRead | PermissionWrite;

if (perms & PermissionRead) { NSLog(@"можно читать"); }
if (perms & PermissionDelete) { /* не сработает: бита нет */ }
```

Добавить флаг — `|=`. Снять флаг — `&=` с инверсией `~` (она переворачивает все биты, так что `& ~Write` гасит именно бит записи):

```
perms |= PermissionDelete; // выдали право на удаление
perms &= ~PermissionWrite; // отозвали право на запись
```

Полный пример — `code/22-enum-options.m`. Реальный вывод:

```
статус: в работе (код 1)
маска прав: 3
можно читать
удалять нельзя
после выдачи delete маска: 7
полные права? да
после отзыва write маска: 5
```

Проследи числа: `Read|Write = 3` (биты 0011); после `|= Delete` стало 7 (0111) — это и есть `PermissionAll`; после `&= ~Write` осталось 5 (0101 — чтение и удаление). Маска — это просто число, где каждый бит = один флаг.

`NS_OPTIONS` от `NS_ENUM` отличается ровно намерением: первое — «можно комбинировать через |», второе — «ровно одно значение из списка». Компилятор и Swift это намерение учитывают (в Swift `NS_OPTIONS` приезжает как `OptionSet`).

Nullability — где допустим nil

Указатель на объект может быть `nil`. Но из объявления — `(NSString *)name`; не видно: метод **гарантирует** строку или **может** вернуть `nil`? Долгие годы это знание жило только в голове автора и в документации. Аннотации **nullability** позволяют записать его прямо в коде.

Два главных слова:

- `nonnull` — «здесь `nil` не бывает». Контракт: и не передавай `nil`, и не жди его в ответ.
- `nullable` — «здесь `nil` допустим». Надо проверять.

Есть ещё два, реже:

- `null_unspecified` — «не указано» (нейтральное состояние, как было до аннотаций);
- `null_resettable` — особый случай для свойств: геттер всегда вернёт значение, но сеттеру **можно** передать `nil`, чтобы сбросить к значению по умолчанию (классика — `tintColor` в `UIKit`).

Применяют их к свойствам и параметрам:

```
@property (nonatomic, copy, nonnull) NSString *name; // всегда есть
@property (nonatomic, copy, nullable) NSString *email; // может быть nil

- (void)sendTo:(nonnull NSString *)address
            body:(nullable NSString *)body;
```

В типах-указателях внутри сложных объявлений (например, в параметрах функций Си или в указателях на указатели) пишут подчёркнутые варианты `_Nonnull` и `_Nullable` — они ставятся прямо рядом со звёздочкой:

```
NSString * _Nullable findName(NSDictionary * _Nonnull dict);
```

Размечать каждый указатель утомительно. Поэтому почти весь код по умолчанию считают `nonnull`, а явно помечают лишь исключения. Для этого есть пара макросов-«скобок»:

```

NS_ASSUME_NONNULL_BEGIN
// ... здесь всё nonnull по умолчанию,
// помечаем только то, что nullable ...
NS_ASSUME_NONNULL_END

```

Между BEGIN и END каждый объектный указатель без явной пометки считается nonnull. Это резко сокращает шум: вместо десятка nonnull ты пишешь один nullable там, где nil действительно возможен.

Зачем всё это, если в рантайме nil всё равно проглатывается молча?

1. **Контракт в коде.** Объявление само рассказывает, чего ждать. Не надо лезть в документацию.
2. **Предупреждения компилятора.** Передал nil туда, где объявлен nonnull, — clang предупредит. Это ловит ошибку до запуска.
3. **Мост в Swift.** Это главный приз. nonnull-тип приезжает в Swift как обычный String, а nullable — как опционал String?. Без аннотаций Swift вынужден импортировать всё как «неявно развёрнутый опционал» String! — небезопасно. Аннотируя Objective-C, ты делаешь его удобным и безопасным со стороны Swift (тизер главы 23).

Отличие от Си. В Си указатель либо валидный, либо NULL, и язык никак не помогает отличить одно от другого — разыменование NULL просто роняет программу в рантайме. Nullability добавляет к указателю «настроение»: компилятор знает, где nil ожидаем, а где нет, и предупреждает заранее. Сам указатель в памяти при этом не меняется — это чистая подсказка на этапе компиляции.

Lightweight generics — дженерики у коллекций

NSArray хранит id — любые объекты. Из объявления NSArray *names; не видно, что внутри строки. Достанешь элемент — получишь id, и компилятор пропустит к нему любое сообщение, даже бессмысленное.

Lightweight generics (облегчённые дженерики) позволяют указать тип элементов в угловых скобках:

```

NSArray<NSString *> *names;           // массив строк
NSMutableArray<NSNumber *> *scores;   // изменяемый массив чисел
NSDictionary<NSString *, NSNumber *> *salaries; // ключ-строка → число
NSSet<NSURL *> *links;                // множество URL

```

Что это даёт:

- При чтении элемент имеет конкретный тип, а не `id:NSString *first = names[0];` — компилятор знает, что это строка, и подскажет её методы.
- При записи проверяется тип: положить `NSNumber` в `NSArray<NSString *>` — предупреждение.

Например:

```
NSMutableArray<NSString *> *skills = [NSMutableArray array];
[skills addObject:@"Swift"];    // ок
[skills addObject:@42];        // warning: несовместимый тип элемента
```

Слово «облегчённые» здесь ключевое. **Эти аннотации существуют только для компилятора и стираются в рантайме.** В памяти `NSArray<NSString *>` — тот же самый `NSArray`, что хранит `id`. Никакой проверки типов во время выполнения дженерики не добавляют: если очень захотеть, через `id` или приведение можно положить «не тот» объект, и рантайм это пропустит. Дженерики — это страховка на этапе сборки, не больше.

Свой класс тоже можно сделать обобщённым, объявив параметр типа в угловых скобках у `@interface`. Параметр иногда помечают `__covariant` (ковариантный) — это говорит компилятору, что `Box<NSString *> *` можно использовать там, где ждут `Box<id> *` (по той же логике, по которой `NSArray<NSString *> *` подходит под `NSArray<id> *`):

```
@interface Box<__covariant T> : NSObject
- (void)put:(T)value;
- (T)get;
@end
```

Здесь `T` — заполнитель: при объявлении `Box<NSNumber *> *b` компилятор подставит вместо `T` тип `NSNumber *` и будет проверять `put:/get` именно по нему. Но, как и у коллекций, в рантайме `T` исчезает.

Отличие от Си. В Си нет ни коллекций-объектов, ни дженериков — массив там это кусок одинаковых байтов фиксированного типа, и точка. А `NSArray<NSString *>` — это не шаблон в духе `C++ std::vector<T>`, который порождает отдельный машинный код под каждый тип. Это лишь подсказка компилятору, стираемая после проверки: один и тот же класс `NSArray` в рантайме, разные аннотации в исходнике.

instancetype вместо id

`instancetype` мы подробно разбирали в главе 7, здесь — короткое напоминание в общем ряду «современных» вещей.

У инициализаторов и фабричных методов тип возврата пишут не `id`, а `instancetype`:

```
- (instancetype)initWithName:(NSString *)name;  
+ (instancetype)personWithName:(NSString *)name;
```

`instancetype` означает «объект **того самого** класса, у которого вызвали метод». Разница с `id` — в точности типа на этапе компиляции.

Тонкость, которую легко упустить: для методов семейства `init` (и для `alloc`, `new`) clang выводит точный тип сам, даже если написано `id`. Мы проверили: с `-(id)init` строка `[[Person alloc] initWithName:@"A"]` всё равно даёт ошибку `no visible @interface for 'Person' declares the selector 'length'`. А вот у фабрики вроде `+(id)personWithName:` такого вывода нет: `[[Person personWithName:@"A"] length]` соберётся молча и упадёт уже в рантайме. С `+(instancetype)personWithName:` та же строка станет ошибкой компиляции. Поэтому правило простое: для всего, что создаёт и возвращает экземпляр своего класса, пиши `instancetype` — и у `init`, и у фабрик, чтобы не держать в голове, где вывод сработает.

NS_DESIGNATED_INITIALIZER — назначенный инициализатор

Назначенный инициализатор (`designated initializer`) — тот главный `init`, через который проходит вся настройка объекта; остальные инициализаторы обязаны звать его. Понятие из главы 7. Современный синтаксис позволяет **позначить** такой инициализатор, и компилятор начнёт следить за правилами:

```
- (instancetype)initWithName:(NSString *)name  
    status:(EmployeeStatus)status NS_DESIGNATED_INITIALIZER;
```

Что проверяет компилятор, увидев эту пометку:

- любой другой инициализатор класса должен в итоге вызвать назначенный (а не `[super init]` напрямую) — иначе предупреждение;
- назначенный инициализатор должен вызвать назначенный инициализатор родителя через `super`;
- назначенный инициализатор родителя (у `NSObject` это `init`) нужно переопределить в своём классе — или объявить недоступным. Иначе clang предупредит: `method override for the designated initializer of the superclass '-init' not found`.

Последнее правило и объясняет частую пару: рядом с назначенным инициализатором запрещают «голый» `init`, чтобы заставить пользоваться полноценным конструктором:

```
- (instancetype)init NS_UNAVAILABLE;
```

NS_UNAVAILABLE помечает метод недоступным. Теперь `[[Person alloc] init]` не скомпилируется — компилятор направит к `initWithName:status:`. Это прямой пример темы главы: больше ошибок ловится при сборке.

@available — проверка версии ОС

Новые классы и методы появляются в конкретных версиях macOS/iOS. Если твоя программа должна работать и на старых системах, нельзя слепо звать свежий API — на старой ОС его просто нет. Проверку версии делают прямо в коде через `@available`:

```
if (@available(macOS 12.0, iOS 15.0, *)) {
    // здесь можно звать API, появившиеся в macOS 12 / iOS 15
} else {
    // запасной путь для старых систем
}
```

Разберём `@available(macOS 12.0, iOS 15.0, *)`:

- `macOS 12.0, iOS 15.0` — минимальные версии для каждой платформы;
- `*` — «на всех остальных платформах (tvOS, watchOS, будущих) условие считается выполненным». Звёздочка обязательна — она про платформы, которые ты явно не перечислил.

Условие истинно, когда программа выполняется на версии не ниже указанной. Пара с `@available` — атрибут `API_AVAILABLE(...)`, которым помечают собственные методы/классы, доступные только с определённой версии; тогда компилятор предупредит о каждом вызове, не обернутом в `@available('Future' is only available on macOS 30.0 or newer)`.

Проверим на живом примере (`code/22-available.m`). Создание `NSISO8601DateFormatter` (появился в macOS 10.12) обернём в проверку:

```
if (@available(macOS 10.12, *)) {
    NSISO8601DateFormatter *fmt = [[NSISO8601DateFormatter alloc] init];
    NSString *now = [fmt stringFromDate:[NSDate date]];
    NSLog(@"сейчас по ISO-8601: %@", now);
} else {
    NSLog(@"ISO-8601 форматтер недоступен на этой ОС");
}
```

Реальный вывод (дата будет своя):

```
ОС достаточно свежая: можно звать новые API
macOS пока младше 99 — используем старый код
сейчас по ISO-8601: 2026-06-28T07:45:33Z
```

Первая ветка сработала (система свежее macOS 12), вторая — проверка на заведомо будущую macOS 99 — ушла в `else`, третья создала форматтер, потому что 10.12 давно позади.

Модули: `@import` вместо `#import`

Весь код книги начинается с `#import <Foundation/Foundation.h>`. Есть более современная запись — **импорт модуля**:

```
@import Foundation;
```

Одна строка вместо подключения заголовка. Разница не только в краткости:

- `#import` текстово вставляет заголовок и заново разбирает его в каждом файле; модуль компилируется один раз и кешируется — сборка идёт **быстрее**;
- модуль несёт информацию о том, с какой библиотекой линковаться, поэтому флаг `-framework Foundation` при `@import` часто не нужен — **автолинковка**;
- можно импортировать отдельный подмодуль: `@import Foundation.NSString`;

У `@import` есть нюанс компиляции: ему нужен флаг `-fmodules`. Без него clang говорит прямым текстом:

```
file.m:1:1: error: use of '@import' when modules are disabled
  1 | @import Foundation;
    | ^
```

В Xcode модули включены по умолчанию, а в нашей «голой» команде из терминала их надо включить явно:

```
clang -fobjc-arc -fmodules -framework Foundation -O2 file.m -o file
```

Поэтому файлы-примеры в книге оставлены на привычном `#import` — чтобы собираться той же командой, что и везде. Но знать `@import` полезно: в реальном Xcode-проекте он встречается часто, и теперь он тебя не удивит.

Отличие от Си. `#import` (как и си-шный `#include`) — это работа препроцессора: грубая текстовая подстановка содержимого файла. `@import` — другое: компилятор подключает уже разобранный, заранее собранный образ библиотеки. В чистом Си модулей нет — там только текстовое включение заголовков.

KVO и dispatch блоками — пара слов

Две темы из прошлых глав — и обе упираются в блоки.

KVO (наблюдение за свойствами, глава 20) в Objective-C так и остался на методе `observeValueForKeyPath:ofObject:change:context:` — с ручным `context`-указателем и одним общим обработчиком на все ключи. Блочного KVO в Foundation для Objective-C нет. Удобный вариант с замыканием — `observe(_:options:changeHandler:)`, возвращающий `NSKeyValueObservation`, — существует только в Swift. Если в objc-коде хочется обработчик-блок прямо у подписки, берут уведомления `NSNotificationCenter`: у него есть метод `addObserverForName:object:queue:usingBlock:`.

```
id token = [[NSNotificationCenter defaultCenter]
    addObserverForName:@"StatusChanged"
                object:nil
                queue:nil
                usingBlock:^(NSNotification *note) {
    NSLog(@"статус изменился: %@", note.userInfo[@"status"]);
}];
```

Обработчик-блок (глава 12) лежит прямо рядом с подпиской и замыкает нужные переменные. Возвращённый `token` потом передают в `removeObserver:`, чтобы отписаться.

Dispatch — Grand Central Dispatch из главы 21 — изначально построен на блоках: `dispatch_async(queue, ^{ ... })` принимает блок с работой. Старый стиль с `NSThread` и отдельными методами-точками входа уступил место очередям и блокам. Общий мотив один: **современный Objective-C тяготеет к блокам** — код-обработчик пишут на месте, а не отдельным методом где-то в стороне.

Строим с нуля: осовремениваем класс

Соберём всё вместе. Возьмём наивный `Person` из главы 7 — у него был голый `init`, `ivars` без проверок и никаких аннотаций — и шаг за шагом сделаем из него современный класс, наблюдая, как растёт строгость компилятора.

Шаг 1. Что было

Исходный класс из главы 7 выглядел так:

```
@interface Person : NSObject {
    NSString *_name;
    NSInteger _age;
}
- (instancetype)init;
```

```
– (void)describe;
@end
```

Никаких подсказок: компилятор не знает, может ли `name` быть `nil`, какие бывают статусы, что лежит в коллекциях. Всё это всплыло бы только в рантайме.

Шаг 2. Типы для статуса и прав

Добавим `NS_ENUM` для статуса сотрудника и `NS_OPTIONS` для прав доступа — вместо того чтобы хранить их безымянными числами:

```
typedef NS_ENUM(NSInteger, EmployeeStatus) {
    EmployeeStatusActive,
    EmployeeStatusVacation,
    EmployeeStatusFired,
};

typedef NS_OPTIONS(NSUInteger, AccessRights) {
    AccessNone = 0,
    AccessRead = 1 << 0,
    AccessWrite = 1 << 1,
    AccessAdmin = 1 << 2,
};
```

Шаг 3. Nullability и дженерики в интерфейсе

Оборачиваем интерфейс в `NS_ASSUME_NONNULL_BEGIN/END`, помечаем единственное необязательное свойство `email` как `nullable`, а коллекциям даём типы элементов:

```
NS_ASSUME_NONNULL_BEGIN

@interface Person : NSObject

@property (nonatomic, copy) NSString *name;           // nonnull по умолчанию
@property (nonatomic, copy, nullable) NSString *email; // может быть nil
@property (nonatomic, assign) EmployeeStatus status;
@property (nonatomic, assign) AccessRights rights;

@property (nonatomic, strong) NSMutableArray<NSString*> *skills;
@property (nonatomic, strong)
```

```

    NSDictionary<NSString *, NSNumber *> *salaryByYear;

- (instancetype)initWithName:(NSString *)name
    status:(EmployeeStatus)status NS_DESIGNATED_INITIALIZER;
- (instancetype)init NS_UNAVAILABLE;

- (NSString *)objectAtIndexedSubscript:(NSUInteger)index; // person[0]
- (void)describe;

@end

NS_ASSUME_NONNULL_END

```

За один интерфейс мы навесили на класс пять видов современных аннотаций. Реализация назначенного инициализатора привычна, но теперь email законно стартует с nil, а коллекции — пустыми:

```

- (instancetype)initWithName:(NSString *)name status:(EmployeeStatus)status {
    self = [super init];
    if (self) {
        _name = [name copy];
        _status = status;
        _email = nil; // nullable — это законно
        _rights = AccessRead;
        _skills = [NSMutableArray array];
        _salaryByYear = @{};
    }
    return self;
}

```

Субскрипт по навыкам — тот же приём, что у «полки» выше:

```

- (NSString *)objectAtIndexedSubscript:(NSUInteger)index {
    if (index >= self.skills.count) {
        return @"(нет навыка)";
    }
    return self.skills[index];
}

```

Шаг 4. Смотрим, как компилятор ловит ошибки

Класс собран (code/22-nullability-generics.m). Теперь — главное ради чего всё затевалось. Допиши в файл функцию с четырьмя намеренными ошибками, собери — и почитай, что скажет компилятор. Ни одну из них старый наивный класс из главы 7 не заметил бы.

```
Person *p1 = [[Person alloc] init];
```

```
error: 'init' is unavailable
```

NS_UNAVAILABLE запретил голый init, и это **ошибка**: файл не соберётся, пока не позовёшь initWithName:status:. Единственная из четырёх, которая останавливает сборку.

```
p.name = nil;
```

```
warning: null passed to a callee that requires a non-null argument
        [-Wnonnull]
```

name объявлен nonnull (по умолчанию внутри NS_ASSUME_NONNULL_BEGIN), и передача nil ловится прямо в месте присваивания.

```
[p.skills addObject:@42];
```

```
warning: incompatible pointer types sending 'NSNumber *' to parameter
        of type 'NSString * _Nonnull' [-Wincompatible-pointer-types]
```

Дженерик NSMutableArray<NSString *> не пускает число в массив строк — и, обрати внимание, в тексте предупреждения виден и дженерик, и подставленный _Nonnull из наших аннотаций.

```
switch (p.status) {
    case EmployeeStatusActive: break;
    case EmployeeStatusVacation: break;
}
```

```
warning: enumeration value 'EmployeeStatusFired' not handled in switch
        [-Wswitch]
```

NS_ENUM заставляет switch покрыть все статусы и по имени называет забытый.

Заметь: **три из четырёх диагностик — предупреждения, а не ошибки**. Программа с ними соберётся и запустится. Компилятор не запрещает тебе писать так — он предупреждает, что ты нарушаешь контракт, который сам же и объявил. Поэтому в серьёзных проектах предупреждения не терпят: собирают с -Werror, превращающим их в ошибки, и правят сразу. В этом и смысл «современного» Objective-C: ты записываешь свои намерения в коде — а компилятор следит, чтобы их не нарушали.

Шаг 5. Запускаем правильную версию

Корректное использование собирается и работает:

```
Person *p = [[Person alloc] initWithName:@"Айгуль"
              status:EmployeeStatusActive];

p.email = @"aigul@example.kz";
p.rights = AccessRead | AccessWrite | AccessAdmin;
[p.skills addObject:@"Objective-C"];
[p.skills addObject:@"Swift"];
[p.skills addObject:@"SQL"];
[p describe];
NSLog(@"первый навык: %@", p[0]);    // субскрипт
```

Реальный вывод полного файла `code/22-nullability-generics.m` (там после этого ещё читается несуществующий навык `p[9]`, зарплата из словаря и создаётся второй сотрудник, «Аноним»):

```
Айгуль [работает], aigul@example.kz, навыков: 3
  -> администратор
первый навык: Objective-C
навык №9: (нет навыка)
зарплата 2025: 620000
Аноним [в отпуске], почта не указана, навыков: 0
  -> только чтение
```

Заметь две детали: у «Анонима» `email` равен `nil`, и `describe` аккуратно подставил «почта не указана» через `?:` — потому что мы заранее знали (и пометили), что почта `nullable`. А `p[0]` достал первый навык через наш субскрипт. Класс из главы 7 ничего из этого не умел.

Проверяем

Файлы главы:

- `code/22-literals-subscript.m` — литералы, субскрипты, свой субскрипт.
- `code/22-enum-options.m` — `NS_ENUM` и `NS_OPTIONS`.
- `code/22-nullability-generics.m` — осовремененный `Person`.
- `code/22-available.m` — `@available`.

Компиляция и запуск каждого — той же командой, что и всю книгу:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/22-enum-options.m -o /tmp/t && /tmp/t
```

Все четыре собираются без единого предупреждения. Чтобы попробовать `@import Foundation;` из текста, добавь флаг `-fmodules`.

Частые ошибки

- **Флаги `NS_OPTIONS` не степени двойки.** Если задать `PermissionWrite = 3`, его биты пересекутся с `Read` и `Write`, и проверки & начнут врать. Каждому флагу — свой отдельный бит ($1 \ll n$).
- **Ждать от дженериков рантайм-проверки.** `NSArray<NSString*>` не бросит исключение, если через `id` туда попадёт число, — аннотация стирается в рантайме. Это страховка компилятора, а не гарантия памяти.
- **`@available` без звёздочки.** `if (@available(macOS 12.0))` без `*` не скомпилируется: звёздочка для неперечисленных платформ обязательна.
- **`NS_ASSUME_NONNULL_BEGIN` без `END`.** Макросы идут парой. Забыл `END` — поедет nullability в остальной части файла и посыплются странные предупреждения.
- **`@import` без `-fmodules`.** Вне Xcode появится ошибка `use of '@import' when modules are disabled`. Либо добавь флаг, либо вернись к `#import`.
- **Думать, что аннотации меняют рантайм.** `nullable`, дженерики, `instancetype` — всё это работает на этапе компиляции. Объект в памяти от них не меняется ни на байт.

Упражнения

1. Добавь в `NS_ENUM TaskStatus` новое значение `TaskStatusCancelled` и собери `code/22-enum-options.m`, **не** дописывая `switch`. Прочитай предупреждение компилятора, потом добавь недостающий `case`.
2. Дай классу `Shelf` ещё и ключевой субскрипт: реализуй `objectForKeyedSubscript:` и `setObject:forKeyedSubscript:`, чтобы работало `shelf[@"полка-A"]`.
3. В `Person` объяви `email` как `nonnull` (убери `nullable`) и попробуй присвоить ему `nil`. Найди, на что ругается компилятор, и верни `nullable`.
4. Заведи свой обобщённый класс `Stack<T>` с методами `push:/pop` и проверь, что `Stack<NSString*>` не даёт положить `NSNumber`.
5. Оберни вызов любого свежего метода `Foundation` в `@available` и намеренно укажи будущую версию (`macOS 99.0, *`) — убедись, что срабатывает ветка `else`.
6. Перепиши `code/22-available.m` на `@import Foundation;` и собери его с `-fmodules`. Сравни, нужен ли при этом `-framework Foundation`.

Что мы получили

Мы собрали «современное лицо» Objective-C: литералы и субскрипты как сахар над методами (и свой субскрипт в придачу), типобезопасные `NS_ENUM` и битовые `NS_OPTIONS`, аннотации `nullability`, облегчённые дженерики у коллекций, `instancetype` и `NS_DESIGNATED_INITIALIZER`, проверку версии `@available` и импорт модулей `@import`. Сквозная мысль одна: **ты записываешь намерения в коде, а компилятор следит за их соблюдением** — и ловит при сборке то, что раньше падало в рантайме.

Почти каждая из этих вещей придумана не в последнюю очередь ради дружбы со Swift: `NS_ENUM` становится Swift-перечислением, `nullable` — опционалом, дженерики — настоящими дженериками Swift. Это и есть мостик к финальной главе. В главе 23 разберём, как два языка живут в одном проекте: как Swift видит твой Objective-C-код и как Objective-C дотягивается до Swift.

Документация Apple

- Programming with Objective-C (литералы, субскрипты, инициализаторы) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/Introduction/Introduction.html
- Adopting Modern Objective-C (`NS_ENUM`, `NS_OPTIONS`, `instancetype`, designated initializers) — developer.apple.com/library/archive/releasenotes/ObjectiveC/ModernizationObjC/AdoptingModernObjectiveC/AdoptingModernObjectiveC.html
- Designating Nullability in Objective-C APIs (аннотации `nullability` и мост в Swift-опционалы) — developer.apple.com/documentation/swift/designating-nullability-in-objective-c-apis
- Using Imported Lightweight Generics in Swift (обобщённые коллекции и свои обобщённые классы) — developer.apple.com/documentation/swift/using-imported-lightweight-generics-in-swift
- Marking API Availability in Objective-C (`@available`, `API_AVAILABLE`) — developer.apple.com/documentation/swift/marking-api-availability-in-objective-c
- Modules / `@import` (документация clang, не Apple) — clang.llvm.org/docs/Modules.html

Глава 23. Взаимодействие со Swift

Во введении мы договорились честно: новый проект с чистого листа сегодня обычно начинают на Swift, а Objective-C учат, чтобы работать с существующим кодом и понимать платформу изнутри. Эти два мира живут в одном приложении, в одних файлах проекта, и зовут друг друга по именам. Эта глава — про мост между ними: как написать Objective-C так, чтобы из Swift он выглядел красиво и безопасно, и как устроена сама связь.

Глава во многом **концептуальная**. Полноценный Swift-компилятор и проект Xcode мы тут собирать не будем. Уговор такой: Objective-C-сторону мы пишем по-настоящему, компилируем и запускаем (вывод в книге — реальный), а Swift-куски показываем рядом как **иллюстрацию** — «вот как этот же класс выглядит в Swift». В code/ они не лежат. Но это не догадки: при подготовке главы мы скормили наши objc-заголовки настоящему компилятору Swift и сверили каждую показанную подпись с тем, что он реально видит. Главное ты увидишь: какие аннотации в objc-коде делают Swift-сторону аккуратной.

Что мы сделаем

- Поймём, зачем вообще нужен мост и чем он отличается от обычной линковки.
- Разберём, как Objective-C виден из Swift (bridging header, module map).
- Разберём, как Swift виден из Objective-C (-Swift.h, @objc).
- Пройдём по маппингу типов: NSString ↔ String, NSArray ↔ [T], nullable ↔ опционал, NSError** ↔ throws; научимся управлять именами через NS_SWIFT_NAME.
- Соберём настоящий objc-класс, который мостится в Swift «как родной».

Зачем это вообще нужно

Представь живой проект Apple-экосистемы. Ему восемь лет. Ядро — сеть, модели, база — на Objective-C, потому что в год старта Swift только появился. Новые экраны команда пишет на Swift. А под ними обоими лежат системные библиотеки — Foundation, UIKit, — тоже во многом Objective-C. Три слоя, и все должны спокойно звать друг друга:

```
Swift-экраны (новое)
  | зовут
  ▼
Objective-C-ядро (старое, рабочее)
  | зовут
  ▼
Foundation / UIKit (системное, ObjC)
```

Переписать всё на Swift одним движением невозможно — годы работы и риск сломать рабочее. Поэтому языки живут вместе: новый код на Swift, старый на Objective-C, и между ними — официально поддерживаемый двусторонний мост. Без знания, как он устроен, ты не подключишь старый класс к новому экрану.

Отличие от Си. Си и так линкуется с чем угодно: у него есть двоичный интерфейс (ABI), и любая программа может вызвать сишную функцию, зная её имя и сигнатуру. Но линковка переносит только адрес и голые байты аргументов — «понимания» типов в ней нет. Мост Objective-C ↔ Swift делает больше: он переносит **имена методов, типы, опциональность и способ сообщать об ошибках**. Это семантический мост, а не просто «нашли функцию по символу». Поэтому Swift видит не `id` и `void *`, а `String`, `[Product]` и `throws` — компилятор Swift по-настоящему понимает, что приехало с другой стороны.

Как Objective-C виден из Swift

Swift умеет напрямую читать заголовочные файлы Objective-C (.h). Ему не нужен переходник на каждый метод — компилятор Swift сам разбирает `@interface`, видит свойства и методы и строит из них Swift-API. Вопрос только в том, **какие** заголовки ему показать. Тут два сценария.

Сценарий 1: один модуль (приложение) — **bridging header**

Когда Objective-C и Swift лежат в одном таргете (обычное приложение), Xcode заводит специальный файл — **bridging header**, «мост-заголовок». Имя по умолчанию: `ИмяПроекта-Bridging-Header.h`. Когда ты впервые добавляешь Swift-файл в objc-проект (или наоборот), Xcode предлагает создать его автоматически.

Внутри — просто `#import` тех objc-заголовков, которые ты хочешь видеть из Swift:

```
// MyApp-Bridging-Header.h
#import "ShoppingCart.h"
```

```
#import "NetworkClient.h"
#import "LegacyImageCache.h"
```

Всё, что подключено здесь, становится видимым из любого Swift-файла этого таргета — без `import`, без обёрток. Добавил класс в проект — дописал строку в `bridging header` — зовёшь его из Swift.

Сценарий 2: фреймворк — `umbrella header` и `module map`

Если Objective-C-код оформлен как **фреймворк** (отдельный переиспользуемый модуль), `bridging header` не используют. Вместо него у фреймворка есть **`umbrella header`** («зонтичный заголовок») — один публичный `.h`, который подключает все остальные публичные заголовки модуля:

```
// PaymentKit.h — umbrella header фреймворка PaymentKit
#import <PaymentKit/ShoppingCart.h>
#import <PaymentKit/PaletteColor.h>
```

И **`module map`** — описание модуля для компилятора (обычно его генерирует Xcode автоматически):

```
framework module PaymentKit {
    umbrella header "PaymentKit.h"
    export *
    module * { export * }
}
```

После этого из Swift фреймворк подключается одной строкой, как любой другой модуль:

```
import PaymentKit // и все публичные ObjC-классы PaymentKit доступны
```

Разница простая: **`bridging header`** — для «своих» файлов внутри приложения; **`umbrella header`** + **`module map`** — для упакованного фреймворка, который импортируют по имени.

Как Swift виден из Objective-C

Обратное направление чуть хитрее. Objective-C не умеет читать `.swift` напрямую. Поэтому компилятор Swift **сам генерирует** заголовков — `ИмяПродукта-Swift.h`. Это автоматический `.h`, в котором Swift-классы описаны на языке Objective-C. Ты его не пишешь и не редактируешь; ты его только подключаешь там, где зовёшь Swift из `objc`:

```
// в любом .m-файле, где нужен Swift-класс
#import "MyApp-Swift.h"

// теперь можно
OrderViewModel *vm = [[OrderViewModel alloc] init];
[vm reload];
```

Но генерируется этот заголовок не сам по себе для всего подряд. Swift по умолчанию **закрывает** от Objective-C. Чтобы Swift-класс или метод попал в `-Swift.h`, его надо явно пометить.

@objc и @objcMembers

Атрибут `@objc` говорит компилятору Swift: «этот класс/метод/свойство покажи Objective-C». Чтобы класс вообще был виден из `objc`, он должен наследоваться от `NSObject`. Помечать каждый член по отдельности утомительно — `@objcMembers` ставит `@objc` сразу на весь класс:

```
// Swift-сторона (иллюстрация)
@objcMembers class OrderViewModel: NSObject {
    var title: String = "" // автоматически @objc, видно из ObjC
    func reload() { /* ... */ } // автоматически @objc
    @objc func ping() { } // можно и точно, без @objcMembers
}
```

Что из Swift в принципе НЕ видно из Objective-C

Часть возможностей Swift не имеет аналога в Objective-C, и мостить их нечем. Их нельзя пометить `@objc`, и из `objc` они недоступны в принципе:

- **struct** — значимые типы Swift. В объектной модели Objective-C для них нет места, поэтому в `objc` они не видны вовсе.
- **enum с ассоциированными значениями** — `enum Result { case ok(Int); case fail(String) }`. Обычный «целочисленный» `@objc enum` мостится, а с ассоциированными значениями — нет.
- **Дженерики Swift** — свои обобщённые типы (`Stack<T>`) в `objc` не показать.
- **Кортежи** (`(Int, String)`), протоколы с `associatedtype` и прочее «чисто-swift'овое».

Правило в голове: **из Objective-C видно только то, что выражается в объектной модели Objective-C**. Классы-наследники `NSObject`, их методы, свойства и обычные перечисления — да. Уникальные swift-конструкции — нет.

Отличие от Си. В Си «видимость» — это вопрос наличия объявления в заголовке: есть прототип функции — можешь звать. Тут видимость определяется ещё и тем,

выразима ли вообще конструкция в чужой системе типов. Swift-структуру нельзя «объявить» в objc-заголовке не потому, что забыли, а потому, что в объектной модели Objective-C для неё нет места.

Маппинг типов: что во что превращается

Самое полезное в мосте — автоматический перевод типов. Один и тот же объект на стороне Objective-C и на стороне Swift называется по-разному, но это один объект. Вот основные пары.

Objective-C	Swift
<code>NSString *</code>	<code>String</code>
<code>NSArray<Product *> *</code>	<code>[Product]</code>
<code>NSDictionary<NSString *, T *></code>	<code>[String: T]</code>
<code>NSSet<T *> *</code>	<code>Set<T></code>
<code>NSInteger / NSUInteger</code>	<code>Int / UInt</code> (см. ниже)
<code>CGFloat / double</code>	<code>CGFloat / Double</code>
<code>BOOL</code>	<code>Bool</code>
<code>nullable NSString *</code>	<code>String?</code> (опционал)
<code>nonnull NSString *</code>	<code>String</code> (НЕ опционал)
<code>void (^)(NSString *)</code>	<code>(String) -> Void</code> (замыкание)
<code>- (...)error:(NSError **)</code>	<code>throws</code>

Строка про `NSUInteger` с подвохом. В системных фреймворках Apple `NSUInteger` приезжает в Swift как `Int` (поэтому у массивов `count: Int`), а в твоих собственных заголовках — как `UInt`. Мы проверили это компилятором Swift на классе из этой главы: `itemAtIndex: (NSUInteger)` пришёл как `item(at index: UInt)`.

NSString ↔ String. `NSString *` в параметре или возврате — это в Swift `String`. Мост конвертирует между двумя представлениями сам, руками ничего делать не надо.

NSArray<Product *> ↔ [Product]. Вот ради чего в главе 22 мы писали дженерик-параметр у коллекций. Без него `NSArray *` приехал бы в Swift как `[Any]` — бесполезный мешок с приведением каждого элемента к типу. С `NSArray<Product *> *` Swift видит честный `[Product]`. То же с `NSDictionary<K, V> → [K: V]`.

nullable/nonnull ↔ опционал. Это второй большой повод для главы 22. Опционал в Swift — это тип, который либо хранит значение, либо пуст (`nil`). `String?` (со знаком вопроса) — «строка или ничего». `String` (без вопроса) — «строка точно есть». Компилятор Swift заставляет проверять опционалы перед использованием, и в этом его безопасность.

Наши аннотации переводятся в это различие напрямую: `nullable NSString * → String?` (распаковать перед работой), `nonnull NSString * → String` (пользоваться сразу).

А что, если у objc-метода **нет** аннотации `nullability`? Тогда Swift не знает, бывает ли там `nil`, и подставляет **неявно развёрнутый опционал** — `String!`. Это «опционал, который Swift разрешает использовать без проверки». Удобно, но опасно: если там всё-таки прилетит `nil`, приложение упадёт. Поэтому неаннотированный objc-API в Swift выглядит россыпью восклицательных знаков — некрасиво и небезопасно. Аккуратные `NS_ASSUME_NONNULL_BEGIN/END` и точечные `nullable` убирают эти !.

Отличие от Си. В Си указатель либо ненулевой, либо `NULL`, и компилятор не следит, проверил ты его или нет — забыл проверку, получил разыменование нуля и падение. Опционал Swift поднимает это на уровень типа: «может быть пусто» вписано в тип, и компилятор не даст попользоваться значением, не распаковав его. Наши `nullable/nonnull` — это ровно тот мостик, по которому знание «бывает ли тут `nil`» переезжает из objc-заголовка в систему типов Swift.

Блоки ↔ замыкания. Блок Objective-C (`^{ }` из главы 12) — это и есть замыкание Swift. `void (^)(NSString *)` приезжает как `(String) -> Void`.

NSError ↔ throws.** Метод, который принимает последним параметром `NSError **`, возвращает `nil/NO` при неудаче и заполняет `*error`, Swift распознаёт и делает **throwing** — бросающим ошибку. Вместо разбора `NSError` ты пишешь `try` и ловишь ошибку в `catch`. Покажем на живом классе.

Собираем класс, который хорошо мостится

Хватит теории — напишем настоящую objc-сторону: корзину покупок `ShoppingCart`. Задача — аннотировать её так, чтобы в Swift она выглядела как написанный на Swift класс. Файл `code/23-bridgeable.m`.

```
#import <Foundation/Foundation.h>

NS_ASSUME_NONNULL_BEGIN    // всё ниже nonnull, кроме явного nullable

@interface ShoppingCart : NSObject

// nonnull -> в Swift String, НЕ String?
@property (nonatomic, copy, readonly) NSString *ownerName;

// дженерик -> в Swift [String], а не [Any]
```

```

@property (nonatomic, copy, readonly) NSArray<NSString *> *items;

// nullable -> в Swift String? (опционал)
@property (nonatomic, copy, nullable) NSString *promoCode;

// initWithOwner: -> в Swift init(owner:)
- (instancetype)initWithOwner:(NSString *)ownerName
    NS_DESIGNATED_INITIALIZER;

// запрещаем пустой init: в Swift его не будет видно
- (instancetype)init NS_UNAVAILABLE;

- (void)addItem:(NSString *)item;

// nullable-возврат -> в Swift item(at:) -> String?
- (nullable NSString *)itemAtIndex:(NSUInteger)index;

// блок; NS_NOESCAPE -> не «убегающий»
- (void)enumerateItemsUsingBlock:
    (void (NS_NOESCAPE ^)(NSString *item, NSUInteger index))block;

// error: последним -> в Swift throws
- (nullable NSNumber *)checkoutWithPricePerItem:(double)price
    error:(NSError **)error;

@end

NS_ASSUME_NONNULL_END

```

NS_ASSUME_NONNULL_BEGIN / END

Эта пара (мы видели её в главе 22) включает «по умолчанию всё ненулевое» для всех указателей между ними. Внутри блока помечаешь только **исключения** — то, что действительно может быть nil, словом nullable. И читается легче, и в Swift даёт чистый API без лишних !.

Свойства: ownerName, items, promoCode

Три свойства из листинга в Swift выглядят так (иллюстрация, не компилируем):

// Swift видит ShoppingCart примерно так:

```
class ShoppingCart: NSObject {
    var ownerName: String { get }      // nonnull -> не опционал
    var items: [String] { get }        // дженерик -> массив строк
    var promoCode: String?             // nullable -> опционал
}
```

ownerName — String, потому что он nonnull (мы внутри ASSUME_NONNULL). items — [String], потому что у NSArray указан параметр <NSString *>. promoCode — String?, потому что помечен nullable. Каждая аннотация на своём месте — и Swift-сторона выглядит идеально.

Инициализаторы: initWithOwner: и запрет init

Имя initWithOwner: Swift автоматически превращает в init(owner:) — часть «With» отрезается, а первый аргумент получает метку из остатка. А NS_UNAVAILABLE на пустом init говорит Swift: этого инициализатора нет, не предлагай его. В итоге:

```
let cart = ShoppingCart(owner: "Алия")    // есть
let bad  = ShoppingCart()                 // error: 'init()' is unavailable
```

Отличие от Си. В Си «создание» — это malloc плюс ручное заполнение полей; никаких именованных параметров. Мост переносит именно **имена аргументов**: initWithOwner: → init(owner:). Имя метода как часть API — этого в Си нет вовсе.

Блок с NS_NOESCAPE

NS_NOESCAPE помечает блок как **не убегаящий**: он отработает прямо внутри метода и не сохранится где-то на потом. Для Swift это важно: не убегаящее замыкание приезжает без пометки @escaping, и внутри него можно обращаться к свойствам и методам своего класса без явного self.. Без NS_NOESCAPE тот же параметр пришёл бы в Swift как @escaping (String, UInt) -> Void. В Swift:

```
cart.enumerateItems { item, index in    // без @escaping
    print("\(index) \([item])")
}
```

Метод с NSError** → throws

Шаблон «возвращаем nil + заполняем *error» Swift распознаёт и делает из метода throwing. Заметь, что nullable-возврат NSNumber * в Swift становится **не**-опциональным: раз о провале теперь сообщает throw, значению-результату опционал больше не нужен.

```
do {
    // NSNumber, не опционал
    let total = try cart.checkout(withPricePerItem: 1500)
    print("К оплате: \(total) ₸")
} catch {
    print("Ошибка checkout: \(error.localizedDescription)")
}
```

Имя метода тоже причесалось: хвост `error`: Swift убирает (он стал `throws`), а `checkoutWithPricePerItem`: Swift разрезал по предлогу `With`: получилось `checkout(withPricePerItem:)`. Если хочется короче, `checkout(pricePerItem:)`, — это задаётся руками через `NS_SWIFT_NAME`, о нём чуть ниже.

Запускаем objc-сторону

Swift мы не компилируем, но `objc`-класс — настоящий и рабочий. В `main` (он есть в файле) мы создаём корзину, добавляем товары, перебираем их блоком и дважды зовём `checkout` — на полной и пустой корзине. Собираем:

```
clang -fobjc-arc -framework Foundation code/23-bridgeable.m -o cart
./cart
```

Реальный вывод:

```
2026-06-28 12:41:47.247 cart[63741:1281717] Владелец: Алия
2026-06-28 12:41:47.247 cart[63741:1281717] Товаров: 3
2026-06-28 12:41:47.248 cart[63741:1281717] Товар 0: Клавиатура
2026-06-28 12:41:47.248 cart[63741:1281717] Товар 99: (null)
2026-06-28 12:41:47.248 cart[63741:1281717] [0] Клавиатура
2026-06-28 12:41:47.248 cart[63741:1281717] [1] Мышь
2026-06-28 12:41:47.248 cart[63741:1281717] [2] Коврик
2026-06-28 12:41:47.248 cart[63741:1281717] К оплате: 4500 ₸
2026-06-28 12:41:47.248 cart[63741:1281717] Ошибка checkout: Корзина
пуста — нечего оформлять
```

Товар 99: `(null)` — наш `nullable`-возврат: вне диапазона метод вернул `nil` (в Swift — `nil` опционала). Последняя строка — сработавший `NSError**`: на стороне Swift она прилетела бы в `catch`. Один код — две стороны моста.

Управляем именами: NS_SWIFT_NAME и компания

Иногда автоперевод имени выходит неуклюжим, и хочется задать Swift-имя руками. Для этого есть набор макросов — соберём их в файле `code/23-annotations.m`.

NS_SWIFT_NAME — переименовать для Swift

```
@interface PaletteColor : NSObject
@property (nonatomic, readonly) double red;
@property (nonatomic, readonly) double green;
@property (nonatomic, readonly) double blue;

// без аннотации Swift увидел бы фабрику rgb(withRed:green:blue:),
// с NS_SWIFT_NAME она становится обычным инициализатором
+ (instancetype)rgbWithRed:(double)red
                    green:(double)green
                    blue:(double)blue
    NS_SWIFT_NAME(init(red:green:blue:));

// длинное имя ужимаем до hexString()
- (NSString *)hexStringRepresentation NS_SWIFT_NAME(hexString());
@end
```

`NS_SWIFT_NAME(...)` принимает Swift-подпись метода. Для фабричного метода `rgbWithRed:green:blue:` мы просим Swift показать его как **инициализатор** `init(red:green:blue:)` — так в Swift привычнее создавать объекты. Для метода `hexStringRepresentation` просто даём короткое имя `hexString()` (скобки означают «это метод»).

Любопытная деталь, которую мы поймали на проверке. Сначала фабрика называлась `colorWithRed:green:blue:` — и Swift сам, без всякого `NS_SWIFT_NAME`, показал её как `init(red:green:blue:)`. Правило такое: если имя фабрики начинается с последнего слова имени класса (`color...` у `PaletteColor`), Swift считает её инициализатором. Именно так `+UIColor colorWithRed:green:blue:alpha:]` превращается в `UIColor(red:green:blue:alpha:)`. Для имени `rgbWithRed:...` правило не срабатывает, и тут `NS_SWIFT_NAME` уже по-настоящему нужен.

```
// Swift-сторона (иллюстрация)
// вместо PaletteColor.rgb(withRed:green:blue:)
let lime = PaletteColor(red: 0.6, green: 0.9, blue: 0.2)
// вместо hexStringRepresentation()
let hex  = lime.hexString()
```

NS_SWIFT_NAME работает не только на методах — им переименовывают классы, свойства, перечисления, даже сишные функции и типы. Этим инструментом системные фреймворки Apple причёсывают свой objc-API под современный Swift.

error: → throws и NS_SWIFT_NOTHROW

```
@interface ConfigLoader : NSObject

// error: последним -> в Swift throwing: try loader.load(from: text)
- (nullable NSDictionary<NSString *, NSString *> *)
    loadFromString:(NSString *)text
      error:(NSError **)error
    NS_SWIFT_NAME(load(from:));

// тоже принимает error:, но BOOL здесь — настоящий результат,
// а не «успех». NS_SWIFT_NOTHROW оставляет метод обычным,
// возвращающим Bool.
- (BOOL)isValidConfig:(NSString *)text
      error:(NSError **)error NS_SWIFT_NOTHROW;

@end
```

Первый метод — классический «загрузить или ошибка», и throws ему к лицу. Заодно NS_SWIFT_NAME(load(from:)) даёт короткое имя.

Второй — ловушка. Он принимает error:, и Swift по умолчанию сделал бы его throwing. Но BOOL здесь — **смысловой результат** («валиден ли конфиг»), а не сигнал успеха. NS_SWIFT_NOTHROW отменяет автоперевод: метод останется обычным и вернёт Bool.

```
// Swift-сторона (иллюстрация)
// throwing, результат [String : String]
let cfg = try loader.load(from: "host=localhost;port=5432")
// Bool, не throws
let ok = loader.isValidConfig("a=b", error: nil)
```

Рядом с этими тремя (NS_SWIFT_NAME, NS_NOESCAPE, NS_SWIFT_NOTHROW) есть и другие — NS_SWIFT_UNAVAILABLE, NS_REFINED_FOR_SWIFT и прочие, — но перечисленные закрывают подавляющее большинство случаев.

Запускаем второй файл

```
clang -fobjc-arc -framework Foundation code/23-annotations.m -o annot
./annot
```

Реальный вывод:

```
2026-06-28 12:41:48.573 annot[63746:1281768] Цвет в hex: #99E633
2026-06-28 12:41:48.573 annot[63746:1281768] host = localhost, port = 5432
2026-06-28 12:41:48.573 annot[63746:1281768] Ошибка: Пустая строка
конфигурации
2026-06-28 12:41:48.573 annot[63746:1281768] Конфиг валиден: да
```

Объектная логика та же, что и без аннотаций: макросы вроде `NS_SWIFT_NAME` разворачиваются в атрибуты (`__attribute__((swift_name))`), которые на машинный код objc-программы никак не влияют. Их единственная работа — подсказать **Swift-стороне**, как назвать и подать API. Поэтому вывод обычный, а вся польза проявилась бы в соседнем Swift-файле.

Что мостится плохо: подводные камни

Мост хорош, но не всесилен — несколько мест, где он буксует.

Композиция протоколов. Objective-C спокойно объявляет `id<Drawable, Serializable>` — объект, реализующий два протокола сразу. В Swift это `any Drawable & Serializable`, и такой простой случай переезжает нормально (мы проверили). Но сложные сигнатуры — класс плюс протоколы, протоколы внутри блоков и коллекций — стоит проверять: как именно приехал тип.

Перегрузка по типам аргументов. В Objective-C методы различаются полным именем с двоеточиями, «перегрузки» по типам как в Swift нет. Несколько @objc-методов с одним базовым именем должны различаться в objc-имени — иначе конфликт селекторов. На границе моста имена важнее типов.

Указатели и небезопасные типы. Сырые сишные указатели (`void *`, `int *`), C-структуры (не Foundation), указатели на функции мостятся в Swift как «небезопасные» (`UnsafePointer` и пр.) или не мостятся вовсе. Метод, торчащий наружу сырым указателем, в Swift неудобен и опасен — прячь такое за объектным API.

Селекторы и KVC. Динамические механизмы Objective-C из Swift доступны, но через специальный синтаксис. Селектор (имя сообщения, см. главу 5) в Swift записывают через `#selector`:

```
// Swift-сторона (иллюстрация)
button.addTarget(self, action: #selector(handleTap), for: .touchUpInside)
```

А чтобы метод вообще годился в `#selector`, он должен быть `@objc` — ведь `#selector` опирается на `objc-runtime`. Так же и KVC (`value(forKey:)`, глава 20) работает из Swift только по `@objc`-членам: без `objc`-видимости runtime просто не найдёт свойство по строковому имени.

Отличие от Си. В Си нет ни селекторов, ни доступа к полю по строке-имени: всё прибито на этапе компиляции. `#selector` и KVC живут только потому, что под Swift в этих местах работает `objc-runtime`. Граница со Swift — это ещё и граница между «всё известно заранее» (Си) и «имена разрешаются во время выполнения» (`objc-runtime`).

Современная связка: как смешать языки в проекте

Соберём по шагам, концептуально (Xcode сделает большую часть сам).

Добавить Swift-файл в старый Objective-C-проект:

1. В Xcode File → New → File → Swift File. Xcode заметит, что проект `objc`-шный, и предложит создать `bridging header` — **соглашайся**.
2. В появившийся `ИмяПроекта-Bridging-Header.h` впиши `#import` тех `objc`-заголовков, которые нужны Swift-коду.
3. Пиши Swift-класс. Чтобы потом звать его из Objective-C, наследуй от `NSObject` и помечай `@objc` (или `@objcMembers`).
4. В `.m`-файле, где зовёшь Swift, подключи `#import "ИмяПроекта-Swift.h"` — этот заголовок Xcode генерирует сам при каждой сборке.

Добавить Objective-C-файл в Swift-проект: File → New → File → Objective-C File, согласиться на `bridging header`, вписать туда `#import "ТвойКласс.h"` — и зови `objc`-класс из Swift напрямую (`import` не нужен). Аннотируй заголовок (`NS_ASSUME_NONNULL_BEGIN`, дженерики у коллекций, `NS_SWIFT_NAME`) — и Swift-сторона станет аккуратной, как мы разбирали всю главу.

Ключевая мысль: направление «ObjC → Swift» идёт через **bridging header** (или `module map` у фреймворка), а «Swift → ObjC» — через автогенерируемый **-Swift.h** плюс пометки `@objc`. Два канала, по одному в каждую сторону.

Проверяем

Файлы главы — `code/23-bridgeable.m` и `code/23-annotations.m`. Оба — настоящая `objc`-сторона моста. Каждый собирается отдельно, обычной командой (у каждого свой `main`, поэтому вместе в одну программу их не собрать):


```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/23-bridgeable.m -o cart && ./cart
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/23-annotations.m -o annot && ./annot
```

Ожидаемый вывод обоих показан выше (дата и числа в скобках будут свои). Swift-блоки в главе — иллюстрации: в code/ их нет, потому что для них нужен настоящий Swift-модуль, а не одна команда clang. Но именно ради них мы и расставляли все эти аннотации.

Частые ошибки

- **Забыл @objc на Swift-классе/методе** — и из Objective-C он не виден, -Swift.h про него молчит. Нужен @objc (или @objcMembers) и наследование от NSObject.
- **Не аннотировал nullability** — Swift подставит неявно развёрнутые опционалы (String!), и приложение упадёт на первом же неожиданном nil. Оборачивай заголовок в NS_ASSUME_NONNULL_BEGIN/END.
- **Забыл дженерик у коллекции** — NSArray * приедет как [Any], и в Swift каждый элемент придётся приводить к типу. Пиши NSArray<Product *> *.
- **Ждёшь, что Swift-struct/enum-с-ассоциированными-значениями увидит Objective-C** — не увидит. Эти типы невыразимы в objc; мости через класс-наследник NSObject.
- **Не пометил метод @objc, а зовёшь его в #selector или через KVC** — runtime его не найдёт. Динамика Objective-C требует objc-видимости.
- **Поставил NS_SWIFT_NOTHROW там, где error: действительно сигнализирует провал** — потерял удобный try/catch. Отменяй throws только когда BOOL/возврат — это смысловой результат, а не «успех».

Упражнения

1. Добавь в ShoppingCart свойство nullable NSString *giftMessage и метод – (void)clear;. Напиши рядом (в комментарии) иллюстративный Swift-блок: как эти члены выглядели бы в Swift.
2. В PaletteColor добавь метод – (PaletteColor *)blendedWith:(PaletteColor *)other; и подбери ему красивое Swift-имя через NS_SWIFT_NAME (например, blended(with:)). Скомпилируй objc-сторону.
3. Сделай метод – (nullable NSData *)exportConfig:(NSError **)error; и объясни в комментарии, почему в Swift он станет throwing и почему возврат перестанет быть опционалом.

4. Объясни своими словами, чем bridging header отличается от автогенерируемого `-Swift.h` и почему в смешанном проекте нужны оба.

Что мы получили

Ты увидел мост между Objective-C и Swift не как магию, а как набор понятных правил. Objective-C виден из Swift через bridging header (или umbrella header + module map у фреймворка) — Swift читает `.h` напрямую. Swift виден из Objective-C через автогенерируемый `-Swift.h`, и только то, что помечено `@objc` и живёт в объектной модели. Между сторонами автоматически переводятся типы: `NSString` ↔ `String`, дженерик-коллекции ↔ массивы и словари, `nullable/nonnull` ↔ опционалы, блоки ↔ замыкания, `NSError**` ↔ `throws`. А `NS_SWIFT_NAME`, `NS_NOESCAPE`, `NS_SWIFT_NOTHROW` позволяют точно причесать имена под Swift.

Теперь понятно и зачем была глава 22: nullability и дженерики — это не украшение заголовка, а материал, из которого строится безопасная и красивая Swift-сторона. Хорошо аннотированный Objective-C из Swift неотличим от написанного на Swift — в этом весь смысл моста.

На этом основная часть книги завершается: ты прошёл путь от первой строки на Си до того, как объект во время выполнения находит метод, и до стыка двух языков Apple. В главе 24 подведём итог и наметим, куда идти дальше. А в приложениях ждут дерево классов Foundation, шпаргалка по синтаксису и подготовка к собеседованиям — возвращайся к ним за справкой.

Документация Apple

- Importing Objective-C into Swift (bridging header, фреймворки) — developer.apple.com/documentation/swift/importing-objc-into-swift
- Importing Swift into Objective-C (`-Swift.h`, `@objc`) — developer.apple.com/documentation/swift/importing-swift-into-objc
- Migrating Your Objective-C Code to Swift (постепенный перенос кода) — developer.apple.com/documentation/swift/migrating-your-objc-code-to-swift
- Designating Nullability in Objective-C APIs — developer.apple.com/documentation/swift/designating-nullability-in-objc-apis
- Renaming Objective-C APIs for Swift (`NS_SWIFT_NAME`) — developer.apple.com/documentation/swift/renaming-objc-apis-for-swift
- About Imported Cocoa Error Parameters (`NSError **` → `throws`, `NS_SWIFT_NOTHROW`) — developer.apple.com/documentation/swift/about-imported-cocoa-error-parameters
- Objective-C and C Code Customization (весь набор макросов: `nullability`, `NS_ENUM`, `NS_REFINED_FOR_SWIFT`, `NS_SWIFT_UNAVAILABLE` и др.) — developer.apple.com/documentation/swift/objc-c-and-c-code-customization

Глава 24. Что дальше

Ты дошёл до конца. Это не дежурная фраза — за плечами целая книга, от первой переменной до отправки сообщений во время выполнения. Эта глава без новых конструкций и без «обязательного» кода. Она про то, что у тебя теперь в руках, куда с этим идти и как не растерять набранную скорость.

Сядем и спокойно подведём итог. А в конце соберём в одном месте ссылки на первоисточники, которые встречались по всей книге, — чтобы было куда возвращаться.

Что ты теперь умеешь

Давай честно, без поглаживаний, пройдемся по аркам книги. Не «мы изучили» — что конкретно ты можешь сделать сам.

База Си. Ты понимаешь переменную, тип, указатель и память. Знаешь, что объект живёт в куче, а у тебя на руках адрес — и поэтому объектные типы всегда со звёздочкой. Видишь разницу между `char *` и настоящим объектом-строкой. Это фундамент: без него объектная часть Objective-C повисает в воздухе, а у тебя она стоит на твёрдом.

Объектная модель. Ты пишешь свой класс с нуля: `@interface`, `@implementation`, методы с `-` и `+`, `self` и `super`. Объявляешь свойства через `@property` с правильными атрибутами (`nonatomic`, `copy`, `strong`, `weak`) и понимаешь, почему `copy` у `NSString`-свойства — не каприз, а защита. Строишь наследование, описываешь протоколы и делегаты, дописываешь чужие классы категориями, носишь с собой кусочки кода в блоках. И главное — ты усвоил саму идею: в Objective-C ты не вызываешь метод, ты **посылаешь сообщение**, а получатель сам во время выполнения решает, что на него ответить.

Память и runtime. Ты знаешь, как объект рождается и умирает: `alloc/ init`, счётчик ссылок, `dealloc`. Понимаешь, что делает ARC и от чего он тебя избавил, и при этом умеешь читать старый код с ручными `retain/ release`. Видишь цикл удержания (`retain cycle`) в блоке и умеешь его разорвать через `weak`. А под капотом ты уже не в темноте: `isa`, метаклассы, дерево клас-

сов до самого `NSObject`, как `objc_msgSend` находит метод, как работает пересылка сообщений и интроспекция. Это та самая «магия», которая после главы про runtime перестала быть магией.

Foundation. Ты свободно обращаешься со строками, числами через `NSNumber`, коллекциями (`NSArray`, `NSDictionary`, `NSSet` и их изменяемыми вариантами), датами, бинарными данными, файлами. Знаешь KVC/KVO и базовую многопоточность. Это не отдельные фокусы — это рабочий инструментарий, на котором держатся настоящие приложения.

Если коротко: ты умеешь читать почти любой Objective-C-код и писать свой. Это и была цель.

Куда идти по экосистеме Apple

Книга про **язык**. Мы намеренно не трогали графический интерфейс — окна, кнопки, экраны. Не потому что это сложно или неважно, а потому что GUI — это отдельные большие библиотеки со своей логикой, и впихивать их в книгу о языке значило бы сделать плохо и то и другое. Зато теперь, когда язык у тебя есть, эти библиотеки открываются как нормальный текст, а не как шифр.

Cocoa и AppKit — приложения для macOS

AppKit — это набор классов для настольных приложений Mac: окна, кнопки, таблицы, меню, текстовые поля. Связка AppKit + Foundation исторически называется **Cocoa**. Имена ты узнаешь с первого взгляда: `NSWindow`, `NSView`, `NSButton`, `NSTableView` — тот же префикс `NS`, та же модель. Кнопка шлёт сообщение, делегат отвечает, контроллер хранит свойства. Всё, что ты выучил про сообщения, свойства, делегаты и память, работает здесь один в один — просто классов больше.

UIKit — приложения для iPhone и iPad

UIKit — то же самое, но для iOS. Префикс другой, `UI`: `UIView`, `UIViewController`, `UIButton`, `UITableView`. Идея ровно та же. Когда во введении я говорил, что «целая эпоха мобильной разработки прошла на этом языке», — это про UIKit, написанный во многом на Objective-C. Делегаты, которые ты разбирал на маленьких примерах, в UIKit повсюду: `UITableViewDataSource`, `UITableViewDelegate` — это те же протоколы, что в главе про протоколы, только с боевыми именами.

Почему мы их не проходили — теперь понятно: это про **интерфейс**, а книга про **язык**. Но порог входа ты уже перешагнул. Открой любой учебник по UIKit или AppKit — и ты будешь спотыкаться не о синтаксис, а только о новые имена классов. А имена учатся быстро, когда грамматика уже своя.

Core Foundation и toll-free bridging

Под Foundation лежит ещё один слой — **Core Foundation**. Это библиотека на чистом Си, с типами вроде `CFStringRef`, `CFArrayRef`, `CFDictionaryRef`. Зачем она, если есть Foundation? Она ниже уровнем, работает там, где объектов Objective-C нет, и её зовут из самых системных мест.

Самое любопытное — **toll-free bridging** («бесплатный мост»): некоторые типы Core Foundation и Foundation — это буквально одно и то же в памяти, и их можно приводить друг к другу без копирования. `CFStringRef` и `NSString *` — пара. Поэтому в коде иногда встречаешь приведение с ключевым словом `__bridge`:

```
CFStringRef cfName = (__bridge CFStringRef)nsName;
```

Тебе не нужно знать это наизусть сегодня. Нужно — узнать в лицо, когда встретишь, и не испугаться: это всё та же строка, просто на этаж ниже. Знание про `isa` и про то, что объект — это адрес в памяти, тут очень кстати: мост работает именно потому, что в памяти лежит совместимая структура.

Где пригодится знание runtime

Глава про runtime — не музейный экспонат. Она окупается в реальной работе:

- **Отладка.** Когда приложение падает с `unrecognized selector sent to instance`, ты теперь читаешь это сообщение как осмысленный текст: объекту послали сообщение, на которое у него нет метода. Ты знаешь, где это ловить.
- **Интроспекция.** Спросить у объекта его класс, проверить `respondToSelector:`, перебрать методы класса — всё это живые приёмы, а не теория.
- **Method swizzling.** Подмена реализации метода во время выполнения — мощный и опасный инструмент. Им пользуются библиотеки аналитики, логирования, тестовые моки. Даже в проектах на Swift под этим лежит Objective-C-runtime.

Мост к Swift

Логичный следующий язык — **Swift**. В главе 23 мы уже навели мост и показали, как два языка живут в одном проекте. Здесь скажу, зачем тебе туда идти и что ты возьмёшь с собой.

Зачем учить Swift: новое в платформе Apple выходит в первую очередь, а часто и только, для него. Новый проект с чистого листа сегодня обычно начинают на Swift — об этом мы честно говорили ещё во введении. Но идёшь ты туда не с пустыми руками.

Что перенесётся почти без потерь:

- *Объектная модель.* Классы, наследование, методы, инициализаторы — всё на месте, только синтаксис мягче.
- *Foundation.* NSString, NSArray, NSDictionary, NSDate, NSURL — во многом те же классы, в Swift у них просто более удобные обёртки (String, Array, Dictionary). Имена и поведение узнаваемы.
- *ARC.* Подсчёт ссылок и циклы удержания — ровно те же, с тем же weak. Свою главу про память будешь вспоминать дословно.
- *Протоколы и делегаты.* Тот же паттерн, который ты разбирал, — в Swift он ещё и центральный приём всего языка.

Что в Swift устроено иначе — и к чему готовиться:

- *Типы-значения (value types).* В Swift struct и enum копируются при передаче, а не передаются по ссылке. В Objective-C объект — всегда ссылка. Это другая модель мышления, и к ней надо привыкнуть.
- *Опционалы (optionals).* Вместо «можно послать сообщение nil, и ничего не случится» Swift заставляет явно отмечать, где значение может отсутствовать. Это строже и безопаснее, но непривычно после вольницы nil.
- *Настоящие дженерики (generics).* В Objective-C обобщения (NSArray<NSString *> *) — подсказка компилятору, а в runtime всё равно id. В Swift дженерики честные, с проверкой типов по-настоящему.

Совет простой: не учи Swift «вместо» Objective-C. Учи его как соседний язык той же платформы. Половину пути ты уже прошёл здесь.

Где сегодня применяют Objective-C

Во введении мы об этом говорили — повторю не дословно, а с высоты пройденной книги, потому что теперь эти слова значат для тебя больше.

Поддержка и развитие. В мире живут миллионы строк рабочего Objective-C-кода. Их не переписывают одним движением — их читают, чинят, дополняют. Тот, кто умеет это делать, нужен и будет нужен ещё долго.

Смешанные проекты. Большие кодовые базы годами живут на двух языках сразу: новое пишут на Swift, старое остаётся на Objective-C, и они разговаривают через мост. Чтобы быть полезным в такой команде, надо знать оба берега.

Понимание системы. Foundation, AppKit, многое в UIKit написаны на Objective-C. Когда из Swift ты дёргаешь привычный API, под ним часто работает objc-механика. Знать её — значит понимать платформу по-настоящему, а не только её современный фасад.

Runtime-трюки. Swizzling, динамическая диспетчеризация, интроспекция — там, где нужно влезть глубже обычного, дорога ведёт в Objective-C-runtime, даже из Swift-проекта.

Так что не думай, что выучил «вчерашний язык». Ты выучил язык, на котором держится фундамент целой платформы.

Как расти дальше

Книга кончилась, а учиться нет. Несколько привычек, которые отличают того, кто застрял, от того, кто растёт.

Читай заголовки SDK. Файлы `.h` системных фреймворков — это не страшная магия, а просто объявления. В Xcode по `⌘`-клику на имя класса ты проваливаешься прямо в его заголовок и видишь все его методы и свойства. Привыкни туда заглядывать: это самый точный и самый честный источник о том, что класс умеет.

Читай чужой код. Эпоха Objective-C оставила горы открытого кода — библиотеки, приложения, примеры Apple тех лет. Открывай, разбирай, спорь с ним. Чужой код учит идиомам быстрее любого учебника: видишь, как взрослые люди раскладывают делегаты, именуют методы, разрывают циклы удержания.

Документация Apple — первоисточник. По всей книге я просил тебя заглядывать в `developer.apple.com`, и не зря. Учебники устаревают, форумы врут, а официальная документация — самая близкая к истине. Заведи рефлекс: встретил незнакомый класс или метод — проверь его на сайте Apple, а не догадывайся.

Привычка сверяться. Лучшее, что ты можешь унести из этой книги, — не список фактов, а манера их проверять. Не уверен, как ведёт себя метод? Напиши пять строк, скомпилируй, посмотри вывод. Мы так делали всю книгу: наивная версия → запуск → проблема → фикс. Это не школьное упражнение, это рабочий метод. Сомнение, проверенное на живой программе, превращается в знание.

Документация Apple

Здесь собраны ключевые первоисточники, которые встречались по всей книге. Это твоя стартовая полка — возвращайся.

Язык:

- Programming with Objective-C (объекты, сообщения, свойства, протоколы, категории, блоки — общее введение в язык) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/Introduction/Introduction.html

- Objective-C Feature Availability Index (что в какой версии языка появилось) — developer.apple.com/library/archive/releasenotes/ObjectiveC/ObjCAvailabilityIndex/index.html

Runtime:

- Objective-C Runtime Programming Guide (Messaging, Dynamic Method Resolution, Forwarding) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ObjCRuntimeGuide/Introduction/Introduction.html
- Objective-C Runtime (API reference: `object_getClass`, `class_getName`, `class_copyMethodList`, `class_addMethod`, `method_exchangeImplementations`, `sel_getName`) — developer.apple.com/documentation/objectivec

Память:

- Advanced Memory Management Programming Guide (полная модель владения: `retain/release/autorelease`, пулы) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/MemoryMgmt/Articles/MemoryMgmt.html
- Transitioning to ARC Release Notes (что делает ARC, `__strong`/`__weak`/`__unsafe_unretained`, циклы удержания) — developer.apple.com/library/archive/releasenotes/ObjectiveC/RN-TransitioningToARC/Introduction/Introduction.html

Foundation:

- Foundation (корень всех справочников по классам Foundation) — developer.apple.com/documentation/foundation
- NSObject — developer.apple.com/documentation/objectivec/nsobject
- NSString — developer.apple.com/documentation/foundation/nsstring
- NSArray — developer.apple.com/documentation/foundation/nsarray (NSDictionary, NSSet — соседние страницы того же справочника).
- Collections Programming Topics (массивы, словари, множества одним обзором) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/Collections/Collections.html

Дальше по экосистеме:

- AppKit — developer.apple.com/documentation/appkit (интерфейс macOS).
- UIKit — developer.apple.com/documentation/uikit (интерфейс iOS).
- Core Foundation — developer.apple.com/documentation/corefoundation (нижний си-слой и toll-free bridging).

Мост к Swift:

- The Swift Programming Language — docs.swift.org/swift-book (официальная книга про язык Swift, твой следующий шаг).

- Importing Objective-C into Swift (как два языка живут в одном проекте; мы это разбирали в главе 23) — developer.apple.com/documentation/swift/importing-objc-into-swift

Из классики — не как обязаловку, а по-человечески — стоит назвать книги Аарона Хиллегасса по Cocoa и Objective-C: они учили целое поколение разработчиков Apple и до сих пор отлично объясняют идиомы. Если зайдёт стиль «строим руками», эти книги тебе понравятся.

Спасибо, что дошёл

Серьёзно — спасибо. Дочитать книгу про язык до конца, разбирая каждый знак @ и каждую скобку, умеют немногие. Ты начал с того, что такое переменная, а закончил тем, как `objc_msgSend` находит метод во время выполнения. Это большой путь, и ты прошёл его сам.

Язык, который ты выучил, — не учебная игрушка. На нём написана целая эпоха Apple: NeXTSTEP, Mac OS X, первые iPhone, библиотеки, под которыми и сегодня работает половина платформы. Теперь этот язык у тебя в руках — не как набор правил, а как понятный, родной инструмент.

Иди пиши что-нибудь своё. Открывай заголовки, читай чужой код, проверяй сомнения на живой программе. У тебя теперь есть всё, чтобы продолжать без меня. Удачи.

Приложение А. Иерархия классов Foundation

Это справочник, а не глава: дерево классов Foundation с короткими пометками. Держи его под рукой, когда читаешь чужой код или ищешь, от кого унаследован незнакомый класс.

У дерева Foundation **два корня**. Из первого, `NSObject`, растёт почти всё — строки, числа, коллекции, даты, сетевые классы, твои собственные типы. Второй, `NSProxy`, — отдельный тонкий корень для объектов-заместителей; он не наследует `NSObject` и в дереве стоит сам по себе. Механику обоих корней (поле `isa`, метаклассы, поиск метода, пересылка сообщений) мы разобрали в главе 14 — здесь её не повторяем, только показываем, кто от кого происходит.

Как читать дерево ниже. Отступ и ветка `└─/└─` у класса означают «наследник». Строки без префикса `NS` («Строки и атрибутированный текст», «Коллекции» и т.д.) — не классы, а тематические группы: каждый класс первого уровня внутри группы наследует прямо `NSObject`. Пометка (`NSObject`) у части классов — то же самое, сказанное явно. Каждая связь «родитель → наследник» в дереве сверена с runtime: мы прошли по всем классам функцией `class_getSuperclass` на macOS 26. Имена даны как в SDK (с префиксом `NS`); в актуальном Swift у многих из них есть «голые» имена без префикса (`String`, `URLRequest`), но в Objective-C пишем с `NS`.

Дерево классов Foundation

```
NSObject (корневой класс — базовое поведение: alloc/init, retain count,
|         isKindOfClass:, respondsToSelector:, description, пересылка)
|
└─ Строки и атрибутированный текст
|   └─ NSString
|       └─ NSMutableString             изменяемая строка
|           └─ NSAttributedString     строка + атрибуты (шрифт, цвет, ссылки)
|               └─ NSMutableAttributedString
```

└─	Значения и числа	
└─	└─ NSValue	обёртка над любым C-значением (NSRange, CGPoint, указатель ...)
└─	└─ └─ NSNumber	обёртка над числом/BOOL
└─	└─ └─ └─ NSDecimalNumber	десятичная арифметика без потерь
└─	Коллекции	
└─	└─ NSArray	упорядоченный список
└─	└─ └─ NSMutableArray	
└─	└─ NSDictionary	пары ключ→значение
└─	└─ └─ NSMutableDictionary	
└─	└─ NSSet	множество (уникальные элементы)
└─	└─ └─ NSMutableSet	
└─	└─ └─ └─ NSCountedSet	множество со счётчиком повторов
└─	└─ NSOrderedSet	уникальные + сохранён порядок
└─	└─ └─ NSMutableOrderedSet	
└─	└─ NSEnumerator	курсор обхода коллекции (nextObject)
└─	└─ └─ NSDirectoryEnumerator	обход содержимого каталога
└─	Данные и время	
└─	└─ NSData	неизменяемый буфер байтов
└─	└─ └─ NSMutableData	
└─	└─ NSDate	момент времени (точка на оси)
└─	└─ NSCalendar	(NSObject) календарь, разбивка даты на компоненты
└─	└─ NSDateComponents	(NSObject) год/месяц/день/час ... по отдельности
└─	└─ NSTimeZone	(NSObject) часовой пояс
└─	└─ NSFormatter	абстрактный «текст ≠ объект»
└─	└─ └─ NSDateFormatter	дата ≠ строка
└─	└─ └─ NSNumberFormatter	число ≠ строка (валюта, проценты)
└─	Разбор текста	
└─	└─ NSCharacterSet	набор символов (фильтр/триммер)
└─	└─ └─ NSMutableCharacterSet	
└─	└─ NSScanner	(NSObject) последовательный разбор строки
└─	└─ NSRegularExpression	(NSObject) регулярные выражения
└─	└─ └─ NSDataDetector	извлечение ссылок, дат, телефонов

└─ URL и сеть	
└─ NSURL	(NSObject) адрес ресурса (http, file, ...)
└─ NSURLComponents	(NSObject) сборка/разбор URL по частям
└─ NSURLQueryItem	(NSObject) пара имя=значение строки запроса
└─ NSURLRequest	описание запроса
└─ NSMutableURLRequest	
└─ NSURLResponse	(NSObject) ответ сервера (общая часть)
└─ NSHTTPURLResponse	HTTP-ответ (коды, заголовки)
└─ NSURLSession	(NSObject) современная загрузка по сети
└─ NSURLSessionTask	(NSObject) абстрактная задача сессии
└─ NSURLSessionDataTask	приём данных в память
└─ NSURLSessionUploadTask	отправка тела запроса
└─ NSURLSessionDownloadTask	загрузка в файл
└─ NSURLSessionStreamTask	потокое TCP-соединение
└─ NSURLCredential	(NSObject) логин/пароль/сертификат
└─ NSURLCache	(NSObject) кэш ответов
└─ NSURLConnection	(NSObject) УСТАРЕЛО – заменён NSURLSession
└─ Файлы и окружение	
└─ NSFileManager	(NSObject) операции с файлами и каталогами
└─ NSFileHandle	(NSObject) низкоуровневое чтение/запись дескриптора
└─ NSBundle	(NSObject) доступ к ресурсам приложения/фреймворка
└─ NSProcessInfo	(NSObject) сведения о процессе, аргументы, env
└─ NSUserDefaults	(NSObject) хранилище пользовательских настроек
└─ Уведомления (NotificationCenter)	
└─ NSNotification	(NSObject) событие: имя + объект + userInfo
└─ NotificationCenter	(NSObject) рассылка уведомлений подписчикам
└─ Потоки, очереди, синхронизация	
└─ NSThread	(NSObject) отдельный поток выполнения
└─ NSOperation	(NSObject) единица работы для очереди
└─ NSBlockOperation	оборачивает блок(и) ^{ }
└─ NSInvocationOperation	оборачивает вызов метода
└─ NSOperationQueue	(NSObject) очередь, исполняющая NSOperation
└─ NSRunLoop	(NSObject) цикл обработки событий потока
└─ NSTimer	(NSObject) отложенный/повторяющийся вызов
└─ (четыре блокировки ниже не наследуют друг друга – их роднит	

```

|   |   только протокол NSLocking)
|   |   └─ NSLock          (NSObject) обычный мьютекс
|   |   └─ NSRecursiveLock (NSObject) рекурсивный мьютекс
|   |   └─ NSCondition     (NSObject) мьютекс + условие
|   |   └─ NSConditionLock (NSObject) блокировка по «номеру условия»
|
|   └─ Архивация и сериализация
|       └─ NSCoder          абстрактный кодер/декодер объектов
|           └─ NSKeyedArchiver  объект → данные (по ключам)
|               └─ NSKeyedUnarchiver  данные → объект
|       └─ NSPropertyListSerialization (NSObject) plist ≠ объекты
|       └─ NSJSONSerialization  (NSObject) JSON ≠ объекты
|
|   └─ Запросы и сортировка
|       └─ NSPredicate        (NSObject) условие фильтрации (как WHERE)
|           └─ NSComparisonPredicate  сравнение «левое op правое»
|               └─ NSCompoundPredicate  AND/OR/NOT над предикатами
|       └─ NSExpression       (NSObject) выражение внутри предиката
|       └─ NSSortDescriptor   (NSObject) правило сортировки (ключ + порядок)
|
|   └─ Интроспекция вызова (см. главу 14)
|       └─ NSInvocation       (NSObject) упакованное сообщение целиком
|       └─ NSMethodSignature (NSObject) описание сигнатуры метода
|
|   └─ Прочее, часто встречающееся
|       └─ NSError            (NSObject) ошибка: домен + код + userInfo
|       └─ NSException        (NSObject) исключение (бросается @throw)
|       └─ NSNull             (NSObject) «ничего» там, где nil нельзя
|                               (внутри коллекций);
|                               синглтон [NSNull null]
|       └─ NSUUID             (NSObject) 128-битный уникальный идентификатор
|
| NSProxy (ВТОРОЙ, отдельный корень – НЕ наследует NSObject)
|   абстрактный суперкласс для объектов-заместителей: реализует
|   минимум и заставляет наследника определить пересылку
|   (–forwardInvocation: / –methodSignatureForSelector:).

```

| На нём строят ленивую загрузку и удалённые прокси.
└─ твои прокси-классы

Дерево не исчерпывающее: в Foundation сотни классов, плюс к ним добавляют свои AppKit, UIKit, Core Data и т.д. Здесь — костяк, который встречается в повседневном коде. Если нужного класса нет, открой его страницу на developer.apple.com: вверху всегда указана строка Inherits From.

Пары «неизменяемый → изменяемый»

Foundation последовательно разделяет данные на неизменяемые (immutable) и изменяемые (mutable). Изменяемый класс всегда **наследник** неизменяемого и добавляет методы правки на месте (addObject:, appendString: ...). Базовый класс хранения общий, поэтому NSMutableArray — это и есть NSArray, которому разрешили меняться.

Неизменяемый	Изменяемый
NSString	NSMutableString
NSAttributedString	NSMutableAttributedString
NSArray	NSMutableArray
NSDictionary	NSMutableDictionary
NSSet	NSMutableSet
NSOrderedSet	NSMutableOrderedSet
NSData	NSMutableData
NSCharacterSet	NSMutableCharacterSet
NSURLRequest	NSMutableURLRequest

Практический вывод: метод, который принимает NSArray *, спокойно примет и NSMutableArray * (наследник подходит везде, где ждут предка). Обратное неверно. И помни про сору у свойств-строк (см. главу 8): сору неизменяемого свойства фиксирует значение, даже если ему передали мутабельный экземпляр.

Ключевые протоколы Foundation

Иерархия типов — это не только классы. Многое в поведении задают **протоколы** (см. главу 10): класс не наследует их, а *принимает* и обязуется реализовать. Самые важные:

- **NSObject (протокол)**. Не путать с одноимённым классом. Это корневой протокол: class, superclass, isKindOfClass:, isEqual:, hash, respondsToSelector:, description,

retain/release (под MRR). Его принимают **оба** корня — и класс NSObject, и NSProxy, — поэтому объекты обоих деревьев умеют отвечать на эти сообщения. Подробности в главе 14.

- **NSCopying.** Требуется `-copyWithZone:`. Объект умеет отдать свою копию (у пар «неизменяемый/изменяемый» — **неизменяемому**); вызывается через `[obj copy]`. Нужен, чтобы объект можно было класть ключом в NSDictionary.
- **NSMutableCopying.** Требуется `-mutableCopyWithZone:`. Отдаёт **изменяемую** копию; вызывается через `[obj mutableCopy]`.
- **NSCoding.** Требуется `-initWithCoder:` и `-encodeWithCoder:`. Объект умеет сериализоваться через NSCoder (архивация в данные и обратно).
- **NSSecureCoding.** Расширяет NSCoder и добавляет `+supportsSecureCoding`. Защищает от подмены класса при разборе архива; обязателен в современных API (например, при передаче объектов между процессами).
- **NSFastEnumeration.** Даёт объекту поддержку быстрого цикла `for (id x in collection)`. Его принимают все коллекции (NSArray, NSSet, NSDictionary, NSOrderedSet).
- **NSLocking.** Требуется `-lock` и `-unlock`. Его принимают NSLock, NSRecursiveLock, NSCondition, NSConditionLock — поэтому их можно использовать единообразно.
- **Сравнение через compare:**. Отдельного протокола «Comparable» в Objective-C нет, но по соглашению классы, которые можно упорядочивать (NSNumber, NSString, NSDate), реализуют `-compare:` и возвращают NSComparisonResult (NSOrderedAscending/NSOrderedSame/NSOrderedDescending). На это соглашение опираются `sortedArrayUsingSelector:` и NSSortDescriptor.

И один **класс**, который логически относится сюда же:

- **NSEnumerator** — не протокол, а класс (наследник NSObject). Курсор обхода: `-nextObject` отдаёт следующий элемент и nil в конце. Его возвращают `-objectEnumerator` и `-reverseObjectEnumerator` коллекций. Сам NSEnumerator принимает NSFastEnumeration, поэтому по нему тоже работает `for (id x in enumerator)`.

Предупреждение: toll-free bridging с Core Foundation

Часть классов Foundation имеет «близнеца» в Core Foundation — это си-уровневый фреймворк с типами вроде CFStringRef, CFArrayRef, CFDictionaryRef. Такие пары **toll-free bridged** («мост без пошлины»): в памяти это один и тот же объект, и указатель можно приводить от одного типа к другому без преобразования.

NSString	<-->	CFStringRef
NSArray	<-->	CFArrayRef
NSDictionary	<-->	CFDictionaryRef

NSData	<-->	CFDataRef
NSDate	<-->	CFDateRef
NSNumber	<-->	CFNumberRef

То есть `CFStringRef` можно передать туда, где ждут `NSString *`, и наоборот — но под ARC приведение требует ключевого слова `__bridge` (или `__bridge_transfer`/`__bridge_retained`, когда нужно передать владение). Core Foundation считает ссылки вручную (`CFRetain`/`CFRelease`), ARC за его типами не следит — отсюда и нужда в явных мостах. Это узкая тема; здесь достаточно знать, что мост существует. Подробности — в документации Apple «Toll-Free Bridged Types». Само слово `__bridge` ты уже встречал в главах 8 и 14 — там им приводили объект к `void *`, чтобы напечатать адрес.

Документация Apple

- Foundation (обзор фреймворка и полный список классов) — developer.apple.com/documentation/foundation
- Foundation → Numbers, Data, and Basic Values; Strings and Text; Collections; Dates and Times; Filtering and Sorting; URL Loading System — тематические разделы того же справочника.
- NSObject (класс) — developer.apple.com/documentation/objectivec/nsobject
- NSObject (протокол) — developer.apple.com/documentation/objectivec/nsobjectprotocol
- NSProxy — developer.apple.com/documentation/foundation/nsproxy
- Toll-Free Bridged Types (Core Foundation ↔ Foundation) — developer.apple.com/library/archive/documentation/CoreFoundation/Conceptual/CFDesignConcepts/Articles/tollFreeBridgedTypes.html
- Механику `isa`, метаклассов и пересылки см. в главе 14 этой книги.

Приложение В. Шпаргалка по синтаксису Objective-C

Это справочник, а не глава. Здесь всё коротко: листинг плюс одна-две строки пояснения, чтобы быстро вернуться за синтаксисом. Если что-то непонятно — соответствующая глава объясняет это подробно.

Компиляция

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 file.m -o prog
./prog
```

Основной режим: ARC включён, подключён Foundation, предупреждения видны.

- `-fobjc-arc` — включить ARC (автоподсчёт ссылок). Память объектов считает компилятор.
- `-framework Foundation` — подключить библиотеку NSString, NSArray, NSLog и прочих. Старые версии clang без неё ругались Undefined symbols; свежий Apple clang (по нашей проверке — clang 21 из Xcode 26) для .m-файлов подставляет её сам. В книге пишем флаг явно.
- `-Wall -Wextra` — показать предупреждения. Чистый код собирается без них.
- `-O2` — оптимизация. `-O0` или без флага — отладочная сборка.
- `-o prog` — имя результата. Без `-o` получится a.out.

```
# ручная память (MRR):
clang -fno-objc-arc -framework Foundation file.m -o prog
# чистый Си (.c):
clang -std=c11 -Wall -Wextra -O2 file.c -o prog
```

`-fno-objc-arc` — отключить ARC, тогда retain/release/autorelease зовёшь сам (глава 13). Си-файлы компилируются как обычный Си.

Скелет программы

```
#import <Foundation/Foundation.h>

int main(void) {
    @autoreleasepool {
        NSLog(@"%@", @"Привет");
    }
    return 0;
}
```

`#import` подключает заголовок (не вставит дважды, в отличие от `#include`). Объектная работа идёт внутри `@autoreleasepool` — пула автоосвобождения для временных объектов.

Отправка сообщений

```
[receiver message];           // без аргументов
[receiver message:a];         // один аргумент
[receiver message:a with:b];   // имя метода — message:with:
[obj doThis:[other doThat]];   // вложенный вызов: внутренний первый
id result = [array objectAtIndex:0];
SEL sel = @selector(setName:); // селектор как значение
```

Квадратные скобки — отправка сообщения. Двоеточия — части имени метода (`message:with:`). `@selector(name:)` берёт само имя метода значением типа `SEL`. Сообщение к `nil` безопасно и возвращает `nil/0`.

Отличие от Си. В Си `foo(obj, 5)` — адрес функции известен при компиляции. `[obj foo:5]` — runtime ищет метод во время выполнения.

Объявление класса

```
// Counter.h
@interface Counter : NSObject {
    NSInteger _hidden;           // ivar (поле экземпляра), необязательно
}
@property (nonatomic, assign) NSInteger value; // свойство
- (void)increment;               // метод экземпляра
+ (instancetype)counter;        // метод класса
```

```

@end

// Counter.m
@implementation Counter
- (void)increment { self.value += 1; }
+ (instancetype)counter { return [[self alloc] init]; }
@end

```

@interface ... @end — объявление (что есть), @implementation ... @end — реализация (как работает).
 - — метод экземпляра, + — метод класса. Ivar’ы для свойств генерируются автоматически как _value.

Атрибуты @property

```

@property (nonatomic, copy)    NSString *name;
@property (nonatomic, strong)  NSArray *items;
@property (nonatomic, weak)    id<MyDelegate> delegate;
@property (nonatomic, assign)  NSInteger count;
@property (nonatomic, readonly) BOOL ready;
@property (nonatomic, getter=isEnabled) BOOL enabled;

```

Атрибут	Когда применять
strong	владеющая ссылка на объект (по умолчанию для объектов)
weak	невладеющая, обнуляется при удалении — для delegate, циклов
copy	для NSString, блоков, изменяемых-в-неизменяемые — хранит копию
assign	для скаляров (NSInteger, BOOL, double), без владения
atomic	потокобезопасный доступ к геттеру/сеттеру (по умолчанию)
nonatomic	без блокировок, быстрее — почти всегда ставят его
readonly	только геттер; readwrite (по умолчанию) — геттер и сеттер
getter=name	задать имя геттера, частый случай — isEnabled для BOOL
setter=name:	задать имя сеттера

copy для NSString-свойства обязателен: иначе кто-то передаст NSMutableString и поменяет её у тебя за спиной.

Создание объектов

```
Counter *c = [[Counter alloc] init];           // alloc + init
NSString *s = [[NSString alloc] initWithFormat:@"%x=%d", 5];
NSArray *a = [NSArray arrayWithObjects:@"a", @"b", nil]; // фабрика
Counter *c2 = [Counter counter];              // своя фабрика
```

alloc выделяет память, init... настраивает объект — всегда парой. Фабричные методы (+) делают то же одним вызовом. Возвращаютinstancetype — «объект этого же класса», поэтому тип сохраняется.

```
- (instancetype)initWithName:(NSString *)name {
    self = [super init];           // сначала инициализирует базовый класс
    if (self) {                    // проверка: super мог вернуть nil
        _name = [name copy];       // пишем напрямую в ivar
    }
    return self;
}
```

Канонический инициализатор: self = [super init], проверка if (self), заполнение полей, return self. Назначенный инициализатор зовёт [super init...]; остальные (удобные) зовут назначенный.

Наследование

```
@interface Dog : Animal           // Dog наследует Animal
@end

@implementation Dog
- (NSString *)sound {
    NSString *base = [super sound]; // вызов версии родителя
    return [base stringByAppendingString:@" гав"]; // переопределение
}
@end
```

: Super после имени — наследование. [super message] зовёт реализацию родителя. Переопределение — просто объявить метод с тем же именем.

Протоколы

```
@protocol Drawing <NSObject>
- (void)draw;                      // @required по умолчанию
@optional
- (void)drawHighlighted;          // @optional – можно не реализовывать
@required
- (CGFloat)area;
@end

@interface Shape : NSObject <Drawing> // класс принимает протокол
@end

id<Drawing> thing = someShape;        // «любой объект, умеющий Drawing»

if ([obj respondsToSelector:@selector(drawHighlighted)]) {
    [obj drawHighlighted];           // проверка @optional
}

if ([obj conformsToProtocol:@protocol(Drawing)]) { ... } // принят ли протокол
```

Протокол — список методов-обязательств. <Drawing> после класса — «реализует». id<Drawing> — тип «любой объект с этим протоколом». @optional-методы перед вызовом проверяй через respondsToSelector:.

Категории и расширения

```
// Категория: добавить методы к существующему классу (даже чужому)
@interface NSString (Reversing)
- (NSString *)reversedString;
@end

@implementation NSString (Reversing)
- (NSString *)reversedString { ... }
@end

// Расширение класса (анонимная категория) – приватные свойства/методы,
// объявляется в .m своего класса
@interface Counter ()
@property (nonatomic, assign, readonly) BOOL ready; // readonly наружу
```

```
- (void)privateHelper;
@end
```

Категория `Class (Name)` дописывает методы к готовому классу. Расширение `Class ()` — для приватной начинки своего класса, добавляет и `ivar`’ы.

Отличие от Си. В Си нельзя дописать функцию-метод к чужому `struct`. Категория добавляет методы к любому классу, включая `NSString`.

Блоки

```
int (^add)(int, int) = ^(int a, int b) { return a + b; }; // переменная-блок
int r = add(2, 3);                                     // вызов: 5
```

```
typedef void (^Completion)(BOOL success); // имя для типа блока
- (void)loadWithCompletion:(Completion)done;
```

```
NSArray *sorted = [arr sortedArrayUsingComparator:
    ^NSComparisonResult(id x, id y) { return [x compare:y]; }];
```

`^` объявляет блок — функцию с захватом окружающих переменных. По умолчанию захваченные переменные доступны только для чтения.

```
__block int total = 0; // __block — можно менять внутри блока
void (^addUp)(int) = ^(int x) { total += x; };
addUp(2);
addUp(3); // total == 5
```

```
__weak typeof(self) weakSelf = self; // разрыв retain cycle
self.handler = ^{ [weakSelf doWork]; }; // блок не удерживает self
```

`__block` — позволяет блоку менять переменную. `weakSelf`-паттерн нужен, когда объект хранит блок, а блок зовёт `self`: иначе `retain cycle`.

Память

```
@property (nonatomic, strong) Node *next; // владеет: держит живым
@property (nonatomic, weak) Node *parent; // не владеет: обнулится в nil
```

```
__strong id keep = obj;    // владеющая локальная (по умолчанию)
__weak id watch = obj;    // слабая: станет nil, когда obj удалят
```

ARC сам считает strong-ссылки: пока есть хоть одна — объект жив. weak не считается и автоматически становится nil после удаления цели.

Признаки retain cycle (два объекта держат друг друга strong и не освобождаются): родитель↔ребёнок, объект хранит блок, зовущий self, delegate объявлен strong. Лечение — сделать одну из ссылок weak.

```
for (int i = 0; i < 1000000; i++) {
    @autoreleasepool {      // пул внутри цикла, а не снаружи
        NSString *s = [NSString stringWithFormat:@"%d", i];
        (void)s;             // ... работа с s ...
        // временные объекты освобождаются на каждой итерации
    }
}
```

Литералы

```
NSString *s = @"text";
NSNumber *i = @42;           // = [NSNumber numberWithInt:42]
NSNumber *b = @YES;
NSNumber *f = @3.14;
NSNumber *e = @(1 + 2);      // @(выражение)
NSArray *a = @[@"a", @"b", @"c"];
NSDictionary *d = @{@"key": @1, @"name": @"Bob"};
```

```
NSString *first = a[0];      // = [a objectAtIndexedSubscript:0]
NSNumber *one = d[@"key"];   // = [d objectForKeyedSubscript:@"key"]
NSMutableArray *m = [a mutableCopy];
m[0] = @"z";                 // = [m setObject:@"z" atIndexedSubscript:0]
```

Знак @ строит объект из обычного значения. Субскрипты [i]/[key] — короткая запись доступа к элементам.

Интроспекция

```
[obj isKindOfClass:[NSString class]];    // obj – NSString ИЛИ потомок
[obj isKindOfClass:[NSString class]];    // ровно NSString, без потомков
[obj respondsToSelector:@selector(length)]; // есть ли такой метод
[obj class];                             // объект Class данного экземпляра
[a isEqual:b];                           // сравнение по значению
[s isEqualToString:@"x"];                // быстрее для строк
```

`isKindOfClass:` учитывает наследников, `isKindOfClass:` — нет. `isEqual:` сравнивает содержимое (`a ==` — адреса/указатели).

Частые типы

```
NSString *s = @"hi";
NSUInteger n = s.length;
NSString *up = [s uppercaseString];
NSString *c = [s stringByAppendingString:@"!"];
BOOL same = [s isEqualToString:@"hi"];

NSMutableString *ms = [@"a" mutableCopy];
[ms appendString:@"b"];                // изменяемая строка
```

```
NSNumber *num = @42;
int iv = num.intValue;
double dv = num.doubleValue;
BOOL bv = num.boolValue;
```

```
NSArray<NSString *> *a = @[@"x", @"y"];
NSString *x = a[0];
NSUInteger cnt = a.count;
BOOL has = [a containsObject:@"x"];
```

```
NSMutableArray<NSString *> *ma = [a mutableCopy];
[ma addObject:@"z"];
[ma removeObjectAtIndex:0];
```

```
NSDictionary<NSString *, NSNumber *> *d = @{@"k": @1};
NSNumber *v = d[@"k"];
```



```

NSSet<NSString *> *set = [NSSet arrayWithArray:a];    // уникальные элементы
NSMutableSet *mset = [set mutableCopy];
[mset addObject:@"q"];

NSData *data = [@"hi" dataUsingEncoding:NSUTF8StringEncoding];
NSUInteger bytes = data.length;

NSDate *now = [NSDate date];
NSTimeInterval secs = [now timeIntervalSince1970];

```

Паттерн NSError

```

- (BOOL)doWork:(NSError **)error {    // BOOL-результат + NSError**
    if (failed) {
        if (error) {
            *error = [NSError errorWithDomain:@"MyDomain"
                                           code:42
                                           userInfo:nil];
        }
        return NO;
    }
    return YES;
}

NSError *err = nil;
if (![obj doWork:&err]) {    // проверяй РЕЗУЛЬТАТ, не err
    NSLog(@"%@", err.localizedDescription);
}

```

Метод возвращает BOOL/объект, ошибку пишет в *error. Признак неудачи — возвращённое NO/nil, а не сам error.

Современный Objective-C

```

typedef NS_ENUM(NSUInteger, Direction) { // перечисление
    DirectionNorth,
    DirectionEast,
    DirectionSouth,

```

```
};

typedef NS_OPTIONS(NSUInteger, Options) { // битовые флаги
    OptionNone = 0,
    OptionBold = 1 << 0,
    OptionItalic = 1 << 1,
};

Options o = OptionBold | OptionItalic;
```

NS_ENUM — типобезопасное перечисление, NS_OPTIONS — флаги под |.

```
NS_ASSUME_NONNULL_BEGIN // всё внутри по умолчанию nonnull
@interface User : NSObject
@property (nonatomic, copy) NSString *name; // nonnull
@property (nonatomic, copy, nullable) NSString *nick; // может быть nil
- (nullable User *)friendNamed:(NSString *)name;
@end
NS_ASSUME_NONNULL_END
```

Nullability помечает, где nil допустим. Между _BEGIN/_END всё считается nonnull, исключения метим nullable.

```
NSArray<NSString *> *names; // дженерик: массив строк
NSDictionary<NSString *, NSNumber *> *scores; // строка → число
NSMutableArray<User *> *users;
```

Дженерики уточняют тип элементов — компилятор проверит и подскажет.

Спецификаторы формата NSLog

```
NSLog(@"%@", obj); // любой объект (зовёт description)
NSLog(@"%d", 42); // int
NSLog(@"%ld", (long)bigInt); // long / NSInteger — кастуй к long
NSLog(@"%lu", (unsigned long)u); // NSUInteger — кастуй к unsigned long
NSLog(@"%f", 3.14); // double / float
NSLog(@"%zu", sizeof(int)); // size_t (sizeof, strlen)
NSLog(@"%p", obj); // адрес объекта
NSLog(@"%@ - %ld", name, (long)age);
```

`%@` — для объектов, остальное — как в `printf`. `NSInteger/NSUInteger` кастуй к `long/unsigned long`. Объект в `%p` можно передать как есть: с `-Wall -Wextra` clang молчит (ворчит только `-Wformat-pedantic`). `(__bridge void *)obj` нужен лишь тогда, когда объект явно приводишь к `void *` — без `__bridge` ARC такое приведение не пропустит.

Частые грабли — одной строкой

- `==` сравнивает указатели, для содержимого строк — `isEqualToString:`.
- `NSString`-свойство объявляй `copy`, иначе словят мутацию через `NSMutableString`.
- KVO-наблюдателя снимай `removeObserver:` до того, как он умрёт (иначе крэш).
- `delegate` всегда `weak` — `strong` даёт `retain cycle`.
- Нельзя менять коллекцию внутри `for (x in collection)` — будет исключение.
- Блок, хранимый объектом и зовущий `self`, — `retain cycle`: бери `weakSelf`.
- `%d` для `NSInteger` ломается на 64 битах — кастуй к `(long)` и пиши `%ld`.
- Изменяемую копию делай `mutableCopy`, не `copy` (тот вернёт неизменяемую).
- Литерал `@[...]@{...}` не принимает `nil` среди элементов — упадёт.

Документация Apple

- Programming with Objective-C (язык целиком) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/Introduction/Introduction.html
- Working with Blocks (раздел про блоки того же руководства) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/WorkingwithBlocks/WorkingwithBlocks.html
- Objective-C Runtime — developer.apple.com/documentation/objectivec
- Foundation — developer.apple.com/documentation/foundation

Приложение С. Как готовиться к собеседованию по Objective-C

Книга кончилась, а вопросы на собеседовании только начинаются. Это приложение — мостик от «я понимаю язык» к «я могу про него внятно рассказать вслух, под взглядом человека на-против». Это два разных умения. Можно годами писать рабочий код и поплыть на простом «а почему свойство NSString объявляют через `copy?`». Здесь мы это закроем.

Приложение прозаическое — кода почти нет. Сам код, который надо писать руками, лежит в следующих трёх разделах (D, E, F) и в папке `code/`. А тут — карта местности: где сегодня вообще собеседуют на Objective-C, что спрашивают на каждом уровне, в каких формах задают вопросы, какие устные ловушки повторяются из раза в раз и как ко всему этому подготовиться так, чтобы не зубрить, а понимать.

Где сегодня собеседуют на Objective-C

Скажу честно, без розовых очков — ровно как говорил во введении и в последней главе. Новый проект с чистого листа в 2020-х обычно начинают на Swift. Если ты ждёшь вакансию «чистый Objective-C, зелёное поле, пишем с нуля» — таких почти не осталось.

Зато живёт и нанимает другое:

- **Поддержка и развитие существующих iOS/macOS-проектов.** Миллионы строк рабочего кода написаны на Objective-C, и их не переписывают одним движением. Их читают, чинят, дополняют. Тут язык нужен не «для общего развития», а каждый день.
- **Смешанные команды Swift + Objective-C.** Большие кодовые базы годами живут на двух языках сразу: новое — на Swift, старое — на Objective-C, и они разговаривают через мост. От тебя ждут, что ты свободен на обоих берегах.
- **Платформенная и инфраструктурная работа.** Библиотеки, SDK, инструменты, где важен runtime: аналитика, логирование, отладочные обвязки. Там Objective-C-механика всплывает даже в Swift-проектах.

Из этого следует важная мысль, которую держи в голове всё собеседование. **Чаще всего вакансии звучит как «iOS-разработчик», а не «Objective-C- разработчик».** Язык спрашивают не сам по себе, а как **фундамент платформы.** Интервьюер хочет понять, понимаешь ли ты, как Apple устроена изнутри: подсчёт ссылок, сообщения, runtime, Foundation. Objective-C здесь — самый честный способ это проверить, потому что в нём вся механика видна руками, без сахара Swift сверху.

Поэтому не удивляйся, если на собеседовании «по Objective-C» половина вопросов будет про вещи, общие со Swift: ARC, циклы удержания, делегаты, многопоточность. Это не подвох. Это и есть фундамент.

Что проверяют по уровням

Дальше идут три раздела с задачами — D, E и F. Они разбиты по уровням: Junior, Middle, Senior. Деление условное (на реальном собеседовании вопросы перемешаны), но оно показывает, **как растёт глубина.** Вот карта, что обычно ждут на каждом уровне.

Junior — раздел D

От джуна хотят уверенного синтаксиса и базовой картины памяти. Не глубины, а отсутствия дыр в основах.

- **Синтаксис сообщений.** [receiver message:arg] — что это, чем отличается от вызова функции, что такое селектор, что такое получатель. Уметь прочитывать вложенное сообщение [[obj foo] bar] и не запутаться в скобках.
- **Классы, свойства, инициализация.** @interface/@implementation, методы с - и +, @property, пара alloc/init, что такое назначенный инициализатор (designated initializer), зачем self = [super init].
- **Базовая память.** Что такое ARC и от чего он избавил. Разница strong и weak на пальцах. Почему объектные типы всегда со звёздочкой.
- **Коллекции.** NSArray, NSDictionary, NSSet и их изменяемые варианты. Литералы @[], @{}, @(42). Чем immutable отличается от mutable и что будет, если мутировать immutable.
- **Поведение nil.** Можно ли слать сообщение nil, что вернётся, почему это не падение.

Middle — раздел E

С миддла начинается настоящий разговор про память и про то, как язык устроен чуть глубже поверхности.

- **Память глубже.** Цикл удержания (retain cycle) — как возникает, как его увидеть, как разорвать через weak. Что такое autorelease pool и когда он сливается. `__unsafe_unretained` (ближайший аналог в Swift — `unowned(unsafe)`) и чем он опаснее weak.
- **Блоки и захват.** Как блок захватывает переменные, почему `self` внутри блока — это сильная ссылка и как из-за неё рождается цикл, что делает `__block`, зачем пишут `__weak typeof(self) weakSelf = self`.
- **Протоколы и делегаты.** Паттерн делегирования, почему делегат держат через weak, `@required` против `@optional`, проверка `respondsToSelector:` перед вызовом опционального метода.
- **Категории.** Что можно и чего нельзя добавить категорией, чем она отличается от наследования, что такое расширение класса (class extension) и чем оно отличается от категории.
- **Сору-семантика.** Зачем сору у свойств, разница сору и `mutableCopy`, поверхностная и глубокая копия, протоколы `NSCopying` / `NSMutableCopying`.
- **KVC/KVO базово.** Что такое доступ к свойствам по строковому ключу (`valueForKey:`), идея наблюдения за изменениями (`addObserver:forKeyPath:options:context:`).

Senior — раздел F

Сеньора проверяют на понимание runtime и на умение проектировать, а не только писать.

- **Runtime.** Указатель `isa`, классы и метаклассы, дерево классов до `NSObject`, как `objc_msgSend` находит метод, динамическое разрешение методов, пересылка сообщений (forwarding), `method swizzling`. Не «слышал слова», а «могу объяснить шаги и нарисовать».
- **Потокобезопасность и GCD.** Очереди (serial/concurrent), `dispatch_async`, барьеры, гонки данных, почему `atomic` не спасает коллекцию, как защитить общий ресурс.
- **Проектирование API.** Именование методов в стиле Cocoa, immutable- против mutable-интерфейсов, что выносить в публичный заголовок, а что прятать в class extension.
- **Дизайн-паттерны Cocoa.** Делегирование, target-action, наблюдатель (KVO/Notification Center), синглтон через `dispatch_once`, MVC так, как его понимает Apple.
- **Interop со Swift.** Как класс Objective-C виден в Swift и наоборот, что такое `NS_ASSUME_NONNULL`, аннотации `nullable/nonnull`, «облегчённые дженерики» (`NSArray<NSString*> *`) и почему в runtime от них ничего не остаётся.
- **Краевые случаи.** Поведение `nil` в арифметике, tagged pointer, toll-free bridging, что возвращает сообщение `nil` для структуры.

Типичные форматы вопросов

Полезно заранее узнать формы, в которых вопрос прилетает. Их немного.

«**Что выведет этот код**». Тебе показывают листинг и просят сказать результат, не запуская. Проверяют, держишь ли ты в голове модель выполнения: порядок вызовов, поведение `nil`, когда срабатывает геттер, что напечатает `%@` для конкретного объекта. Читай такой код медленно, строка за строкой, проговаривай вслух, что происходит.

«**Найди утечку / найди баг**». Дают кусок кода с retain cycle, с сообщением освобождённому объекту, с мутацией immutable-коллекции, с гонкой данных. Твоя задача — увидеть проблему и объяснить, **почему** она проблема и **как** чинить. Первое, что ищи глазами, — сильные ссылки по кругу (особенно `self` внутри блока, который хранит сам объект).

«**Объясни разницу X и Y**». Любимый жанр. Короткий устный вопрос на понимание. Классические пары:

- `strong` против `copy` для свойства;
- `isEqual:` против `==`;
- `isKindOfClass:` против `isKindOfClass:`;
- `id` против `instancetype`;
- категория против наследования;
- `copy` против `mutableCopy`;
- `nil` против `Nil` против `NULL` против `NSNull`.

Хороший ответ — это **одно точное предложение про суть плюс пример, когда выбор важен**. Не лей воду.

«**Реализуй с нуля**». Просят написать `isEqual:` и `hash` для своего класса, простой синглтон, потокобезопасный кэш, разобрать строку. Здесь важен не только результат, но и то, как ты рассуждаешь по дороге. Проговаривай ход мысли.

Устные концептуальные. Без кода вообще: «расскажи, как работает ARC», «что происходит при отправке сообщения», «что такое autorelease pool». Тут проверяют, умеешь ли ты объяснять — структурно, от общего к частному, своими словами, а не цитатой из документации.

Классические устные ловушки Objective-C

Это вопросы, которые задают снова и снова. Если ты ответишь на них чётко и по-человечески, половина собеседования уже за тобой. Привожу с короткими правильными ответами — не заучивай дословно, а пойми и перескажи своими словами.

Почему свойство `NSString` объявляют через `copy`, а не `strong`? Потому что вместо `NSString` тебе могут передать `NSMutableString` — это его наследник, он подойдёт по типу. Если хранить его через `strong`, ты удержишь **тот же самый** изменяемый объект, и внешний код может поменять его у тебя за спиной — твоё «неизменяемое» имя вдруг стало другим. `copy` снимает неизменяе-

мый снимок в момент присваивания, и он уже никуда не уплывёт. Для изменяемых строк сору вернёт неизменяемую копию — это и есть желаемое поведение свойства.

Чем категория отличается от наследования (subclass)? Категория добавляет методы к **уже существующему** классу, и они появляются сразу у **всех** его экземпляров во всём приложении — даже у тех, что создаёт система. Своих хранимых переменных (ivar) категория добавить не может. Наследование создаёт **новый** тип: ему можно дать новые поля, и изменения видны только у экземпляров подкласса. Коротко: категория — «дописать поведение всем», подкласс — «сделать новую разновидность».

Что такое tagged pointer? Оптимизация runtime на 64-битных системах. Маленькие значения — короткие строки, небольшие числа, даты — система не кладёт в кучу отдельным объектом, а упаковывает **прямо внутрь самого указателя**. То есть указатель перестаёт быть адресом и сам несёт данные, а биты-метки говорят runtime, какого это класса. Выгода — нет аллокации и освобождения, доступ мгновенный. Для тебя как программиста это прозрачно: объект ведёт себя как обычный, просто под капотом его «нет» в куче. Поэтому, кстати, на tagged-pointer-объекты бессмысленно смотреть как на адрес и опасно завязываться на их retainCount.

Можно ли слать сообщение nil? Да, и это не падение — фундаментальное свойство языка. Сообщение nil ничего не делает и возвращает «нулевое» значение: для объекта — nil, для чисел — 0, для BOOL — NO, для структуры — обнулённую структуру. Это и удобно (меньше проверок на nil), и коварно: баг с молчаливым nil не упадёт, а тихо вернёт ноль, и ищи потом, почему «ничего не происходит».

Отличие от Си. В Си разыменовать нулевой указатель — неопределённое поведение, на практике обычно мгновенный крах. В Objective-C отправка сообщения nil — штатная безопасная операция, которая просто возвращает ноль. Механику диспетчера сообщений мы разбирали в главе 14.

Зачем делегат держат через weak? Чтобы не было цикла удержания. Обычно объект А создаёт объект В и держит его сильно, а В хочет сообщать А о событиях через свойство delegate. Если delegate сильный, А держит В, В держит А — оба никогда не освободятся, утечка. weak разрывает круг: делегат указывает на владельца, но не удерживает его, и автоматически становится nil, когда владелец умирает.

Как работает autorelease pool? Это отложенное освобождение. Когда объект помечен autorelease, он не освобождается сразу — его регистрируют в текущем пуле, а пул сольёт (освободит) всех своих в конце блока @autoreleasepool { ... }. В приложении с UI такой пул автоматически оборачивает каждую итерацию главного цикла событий (run loop), поэтому временные объекты живут до конца обработки события и потом исчезают пачкой. Свой @autoreleasepool руками заводят в тяжёлых циклах, чтобы пиковая память не росла: сливаешь пул на каждой итерации — временные объекты не копят.

Почему `atomic`-свойства не дают потокобезопасности коллекции? Потому что `atomic` гарантирует атомарность только **одного** обращения — геттер вернёт целый, не разорванный пополам указатель, а сеттер запишет его целиком. И всё. `atomic` ничего не знает про **содержимое** объекта и про **составные** операции. Если два потока одновременно зовут `[array addObject:]` у одного `NSMutableArray`, само свойство атомарно отдаёт указатель на массив, но мутация массива не защищена — гонка данных, порча массива и рано или поздно падение. Потокобезопасность коллекции делается синхронизацией (очередь GCD, блокировка), а не атрибутом свойства.

В чём разница `id` и `instancetype`? `id` — «какой-то объект, тип неизвестен», компилятор не проверяет, что ты ему шлёшь. `instancetype` — «объект класса получателя», и компилятор это знает. Разница видна на фабричных методах: если фабрика объявлена как `+(instancetype)person`, компилятор понимает, что `[Person person]` — это именно `Person *`, и ловит ошибки типов; если она возвращает `id`, проверки нет. Тонкость: для методов семейств `alloc/init/new` компилятор выводит тип сам даже при `id`, а для фабрик — нет. Поэтому в инициализаторах и фабриках пишут `instancetype`, а не `id`: единообразно и без сюрпризов.

Что такое `method swizzling` и чем он опасен? Это подмена реализации метода во время выполнения: ты меняешь местами реализации двух методов (через `method_exchangeImplementations`), и там, где система звала оригинал, теперь выполняется твой код. Применяют в аналитике, логировании, тестовых обёртках. Опасен тем, что:

- меняет поведение **глобально** — для всех экземпляров класса, включая системные;
- зависит от порядка и времени применения (где и когда сделать подмену);
- легко конфликтует с другим `swizzling` и с категориями;
- ломает интуицию при отладке: код в исходнике один, а выполняется другой.

Правильный ответ на собеседовании — «знаю, как сделать, но применяю редко и осторожно, потому что это глобальная и трудноотлаживаемая магия».

Как готовиться

Подготовка к такому собеседованию — не зубрёжка карточек. Это та самая привычка из всей книги: сомнение проверяй на живой программе.

Гоняй код в `clang` сам. Любой вопрос «что выведет» проверяется за минуту: набери пять строк, собери `clang -fobjc-arc -framework Foundation -Wall file.m -o file`, запусти, посмотри вывод. Не верь на слово ни учебнику, ни себе — верь компилятору. Память так укладывается намертво: ты не помнишь правило, ты помнишь, как оно повело себя у тебя на экране.

Читай заголовки SDK. Файлы `.h` системных фреймворков — самый честный источник о том, что класс умеет. В Xcode по `⌘`-клику на имя класса проваливаешься прямо в заголовок и видишь

все методы, свойства, атрибуты, аннотации nullable. Привыкни туда заглядывать: половина «каверзных» вопросов снимается одним взглядом на объявление.

Проговаривай вслух. Это отдельный навык. Знать ответ и **сказать** его связно под взглядом человека — разные вещи. Возьми список ловушек выше и объясни каждую вслух, будто человеку напротив, без подглядывания. Споткнулся — значит, понимаешь хуже, чем казалось. Запишись на диктофон, послушай: вода и «эээ» сразу слышны.

Сверяйся с документацией Apple. Учебники устаревают, форумы врут, официальная документация ближе всего к истине. Встретил спорное утверждение про tagged pointer, autorelease или swizzling — проверь его на `developer.apple.com`, а не запоминай чужую байку. На собеседовании ценится не «я где-то читал», а «я знаю первоисточник».

Как разбирать задачи в этом разделе

Разделы D, E и F устроены одинаково, и пользоваться ими нужно правильно, иначе пользы не будет.

Сначала сам, потом решение. Перед каждой задачей закрой решение и дай себе настоящую попытку. Напиши код, прогони его в clang, ошибись, почини. Только потом открывай разбор и сравнивай. Если сразу читать ответ, мозг кивает «логично» и ничего не запоминает — это иллюзия понимания, которая разваливается на собеседовании. Польза рождается в момент, когда ты **застрял и выкарабкался сам**.

Где лежат решения. Задачи с однозначным, корректным решением вынесены отдельными файлами в папку `code/` — их можно собрать и запустить ровно как код глав книги. В разборе задачи указано имя файла. Сначала попробуй написать своё, потом сверься с эталоном: важны не только результат, но и стиль — именование, атрибуты свойств, обработка nil.

Каверзы с UB живут только в тексте. Часть вопросов специально про **неопределённое поведение** (undefined behavior) и про то, чего делать нельзя: сообщение освобождённому объекту, опора на `retainCount`, гонка данных, мутация коллекции во время перебора. Такой код мы **не** кладём в `code/` и не предлагаем «запустить и посмотреть» — потому что у неопределённого поведения нет «правильного вывода»: на твоей машине оно может случайно сработать, на собеседовании — упасть, а в проде — испортить данные. Эти задачи разбираем словами: почему так нельзя, что произойдёт в худшем случае, как написать правильно. Если очень хочется увидеть падение своими глазами — это нормально, но держи в голове, что увиденный результат ничего не доказывает, кроме того, что так делать нельзя.

Напоследок

Собеседование по Objective-C — это не экзамен на память, а разговор о том, понимаешь ли ты платформу. Ты уже прошёл всю книгу: от первой переменной до того, как `objc_msgSend` находит метод во время выполнения. Тебе есть что сказать.

Не пытайся выглядеть тем, кто знает всё, — таких не бывает. Гораздо сильнее звучит честное «вот это знаю и могу объяснить, а вот тут не уверен, проверил бы в коде». Именно так работает хороший разработчик: не угадывает, а проверяет. Эту привычку мы тренировали всю книгу — она и есть твоё главное преимущество в комнате.

Открывай раздел D и начинай с джуновских задач, даже если метишь выше: основы должны отлетать от зубов, иначе на глубине ты будешь спотыкаться о мелочи. Пиши код, ошибайся, проговаривай вслух. Удачи — ты готов лучше, чем тебе кажется.

Документация Apple

Ключевые первоисточники, по которым стоит готовиться. Это та же стартовая полка, что в конце книги, — но отобранная под собеседование.

Язык и идиомы:

- Programming with Objective-C (объекты, сообщения, свойства, протоколы, категории, блоки; база для Junior/Middle-вопросов) — [developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ ProgrammingWithObjectiveC/Introduction/Introduction.html](http://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/Introduction/Introduction.html)
- Coding Guidelines for Cocoa (именование методов и классов так, как этого ждут на собеседовании про проектирование API) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/CodingGuidelines/CodingGuidelines.html

Память:

- Transitioning to ARC Release Notes (что делает ARC, `__strong`/`__weak`/`__unsafe_unretained`, циклы удержания — основа Middle-вопросов про память) — [developer.apple.com/library/archive/releasenotes/ObjectiveC/ RN-TransitioningToARC/Introduction/Introduction.html](http://developer.apple.com/library/archive/releasenotes/ObjectiveC/RN-TransitioningToARC/Introduction/Introduction.html)
- Advanced Memory Management Programming Guide (полная модель владения: `retain`/`release`/`autorelease`, пулы автоосвобождения) — [developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ MemoryMgmt/Articles/MemoryMgmt.html](http://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/MemoryMgmt/Articles/MemoryMgmt.html)

Runtime (Senior):

- Objective-C Runtime Programming Guide (Messaging, Dynamic Method Resolution, Message Forwarding — то, что спрашивают про `objc_msgSend` и пересылку) — developer.apple.com/library/archive/documentation/ObjCRuntime/Articles/ObjCRuntime.html

[com/library/archive/documentation/Cocoa/Conceptual/ObjCRuntimeGuide/Introduction/Introduction.html](https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ObjCRuntimeGuide/Introduction/Introduction.html)

- Objective-C Runtime (API reference) — developer.apple.com/documentation/objectivec (object_getClass, class_getName, method_exchangeImplementations, sel_getName — для вопросов про интроспекцию и swizzling).

Многопоточность:

- Dispatch (Grand Central Dispatch) — developer.apple.com/documentation/dispatch (очереди, dispatch_async, барьеры, dispatch_once — потокобезопасность и синглтон).
- Concurrency Programming Guide (очереди против потоков, общая модель) — developer.apple.com/library/archive/documentation/General/Conceptual/ConcurrencyProgrammingGuide/Introduction/Introduction.html

Коллекции и значения:

- Foundation — developer.apple.com/documentation/foundation (корень справочников по классам).
- Collections Programming Topics (массивы, словари, множества, копирование, NSCopying) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/Collections/Collections.html

Interop со Swift:

- Importing Objective-C into Swift (как два языка видят друг друга, аннотации nullability, облегчённые дженерики) — developer.apple.com/documentation/swift/importing-objective-c-into-swift

Приложение D. Задачи собеседования: Junior

Этот раздел — тренажёр. Здесь собраны вопросы, которые реально задают на собеседованиях уровня Junior по Objective-C: про синтаксис, объекты, базовую память и коллекции. Ничего нового по сравнению с основными главами тут нет — это проверка того, уложилось ли уже пройденное.

Формат у каждой задачи один. Сначала — условие. Если это вопрос «что выведет программа», не беги сразу к ответу: закрой нижнюю часть рукой, проговори ответ вслух или запиши, и только потом читай разбор. Так проверка работает, а простое чтение — нет.

Многие задачи сопровождаются компилируемым файлом в code/. Собрать и запустить любой можно одной командой:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/junior-01-nil.m -o /tmp/t && /tmp/t
```

Весь показанный вывод — настоящий, снят с этой команды. Дата, время и числа в скобках у тебя будут свои — их добавляет NSLog (см. главу 1). Часть задач намеренно про падения программы (выход за границу массива, addObject:nil). Такой код мы разбираем словами, но в code/ не кладём — чтобы проверка примеров оставалась зелёной.

Задача 1. Что выведет [nil length]

Дан код:

```
NSString *s = nil;
NSUInteger len = [s length];
NSString *up = [s uppercaseString];
```

```
B00L      has = [s hasPrefix:@"a"];
NSLog(@"%lu / %@ / %d", (unsigned long)len, up, has);
```

Программа упадёт? Если нет — что напечатает?

Разбор. Программистов из Си здесь подлавливают чаще всего. В Си проход по нулевому указателю — как правило, мгновенный крах (segmentation fault). Кажется, что и тут [nil length] обязано уронить программу. Не уронит.

В Objective-C **отправка сообщения nil — безопасная пустая операция**. Runtime видит, что получатель nil, и просто ничего не делает. А в ответ отдаёт «нулевой» результат — какой именно, зависит от типа возврата метода:

- метод возвращает число (NSUInteger, NSInteger, int...) → получишь 0;
- метод возвращает объект (NSString *, id...) → получишь nil;
- метод возвращает B00L → получишь N0.

Поэтому вывод (файл code/junior-01-nil.m):

```
[nil length]      = 0
[nil uppercaseString] = (null)
[nil hasPrefix:@"a"] = 0
```

(null) — это так NSLog печатает nil-объект. 0 в конце — это N0.

Это удобно (не нужно обвешивать каждый объект проверкой if (x != nil)), но и опасно: если объект случайно оказался nil, ошибка не выстрелит на месте, а тихо даст 0/nil/N0, и баг всплывёт позже и не там. Где результат важен — проверяй if (x == nil) явно.

Отличие от Си. *p при p == NULL в Си роняет программу. [nil message] в Objective-C — узаконенная пустая операция с нулевым результатом. Это сознательное решение языка, а не случайность.

Задача 2. == или isEqualToString:

Что напечатает каждая строка?

```
NSString *a = @"hello";
NSString *b = [NSString stringWithFormat:@"%s", @"hel", @"lo"];
NSLog(@"%d", (a == b));
NSLog(@"%d", [a isEqualToString:b]);
```

Разбор. Текст у a и b одинаковый — hello. Но вопрос не в тексте.

- `a == b` сравнивает **указатели**: «это один и тот же объект в памяти?».
- `[a isEqualToString:b]` сравнивает **содержимое**: «одинаковый ли текст, буква за буквой?».

`a` — это строковый литерал, он живёт в неизменяемой области программы. `b` собран в рантайме методом `stringWithFormat:` — это **другой** объект. Значит, указатели разные, а текст одинаковый:

```
a == b           : 0
[a isEqualToString:b]: 1
адрес a = 0x10e7b1038
адрес b = 0xd146d585ee0ba9ab
```

(Адреса у тебя будут свои, важно одно — они разные.) Вывод файла `code/junior-02-string-equal.m`. Странный «адрес» `b` — не опечатка: короткую строку runtime упаковал прямо в указатель (tagged pointer, подробнее в задаче 3), в куче такой строки нет вовсе.

Правило на всю жизнь: **строки сравнивай по содержимому через `isEqualToString:`, никогда через `==`**. Две внешне одинаковые строки `==` легко сочтёт разными. `==` для объектов уместно, только когда нужна именно **идентичность** — «тот же самый объект?»: проверить, не `nil` ли это, или сравнить с объектом-одиночкой вроде `[NSNull null]` (задача 9).

Отличие от Си. В Си строки тоже не сравнивали через `==` (это сравнило бы адреса `char *`) — для содержимого был `strcmp`. Здесь та же логика, только вместо `strcmp` — сообщение `isEqualToString:`.

Задача 3. Два `NSNumber` и `==`

```
NSNumber *a = @42;
NSNumber *b = @42;
NSNumber *x = [NSNumber numberWithDouble:12345.6789];
NSNumber *y = [NSNumber numberWithDouble:12345.6789];
NSLog(@"%d %d", (a == b), [a isEqualToString:b]);
NSLog(@"%d %d", (x == y), [x isEqualToNumber:y]);
```

Что выведет?

Разбор. Логика та же, что у строк: содержимое чисел сравнивает `isEqualToNumber:`, а `==` сравнивает указатели. Но с числами есть ловушка хитрее. Реальный вывод (`code/junior-03-number-equal.m`):

```
@42 == @42          : 1
[@42 isEqualToNumber:@42] : 1
x == y              : 0
[x isEqualToNumber:y]  : 1
```

Смотри: @42 == @42 дало 1 (правда!), а x == y для двух одинаковых дробных — 0 (ложь!). Как так?

Маленькие целые NSNumber рантайм на 64-битных системах хранит **прямо внутри указателя** (это называется tagged pointer — «помеченный указатель»): само число 42 закодировано в биты «адреса», отдельный объект в памяти не создаётся. Поэтому два @42 получают одинаковый «адрес», и == случайно оказывается прав. А 12345.6789 так не упаковывается — создаются два разных объекта, и == честно возвращает 0.

Вывод: == для чисел **иногда случайно прав, иногда врёт** — на него нельзя полагаться вообще. Сравнивай значения через isEqualToNumber: (a isEqualToNumber: верен в обоих случаях — обе строки с ним дали 1).

Задача 4. Что делают alloc и init и что будет, если забыть init

```
@interface Box : NSObject
@property (nonatomic) NSInteger value;
@end

@implementation Box
- (instancetype)init {
    self = [super init];
    if (self) { _value = 100; }
    return self;
}
@end

// ...
Box *raw = [Box alloc];
Box *ok  = [[Box alloc] init];
```

Чему равны raw.value и ok.value?

Разбор. Объект в Objective-C рождается в два шага, и у каждого своя работа:

- `alloc` (`allocate`, выделить) — просит у системы память под новый объект и **обнуляет** все его поля. Объект уже существует, но «сырой»: настройка ещё не выполнена.
- `init` (`initialize`, инициализировать) — приводит объект в рабочее состояние. Здесь `init` задаёт `_value = 100` (значение по умолчанию).

Если `init` забыть, останется сырой объект с нулевыми полями. Вывод (`code/junior-04-alloc-init.m`):

```
после alloc:      value = 0
после alloc + init: value = 100
```

`raw` прошёл только `alloc` → `value` нулевой. `ok` прошёл `alloc` и `init` → `value` равен 100, как задумано в инициализаторе. Поэтому почти всегда их пишут слитной парой: `[[Box alloc] init]`.

Разбери и сам инициализатор — это шаблон, который встречается постоянно:

- `self = [super init];` — сначала даём родителю настроить свою часть объекта и **присваиваем результат `self`** (родитель имеет право вернуть другой объект или `nil`);
- `if (self) { ... }` — настраиваем свои поля, только если родитель не отдал `nil`;
- `return self;` — обязательно возвращаем готовый объект.

Частый баг новичка — забыть `return self;` или `self = [super init]`. Без `return self;` инициализатор вернёт мусор (компилятор предупредит: `non-void function does not return a value`). Без `self = [super init]` у `NSObject` всё случайно сработает, но если родитель вернёт другой объект, ты будешь настраивать не тот (подробнее — задача 20).

Отличие от Си. В Си это было `malloc(sizeof(...))` плюс ручное заполнение полей. Здесь `alloc` выделяет память, а `init` заполняет — но заполняет сам объект, по своему чертежу. Освобождать руками не надо: память объектов считает ARC (глава 13).

Задача 5. `strong` против `copy` у свойства `NSString`

Дан класс с двумя свойствами-строками:

```
@property (nonatomic, strong) NSString *strongName;
@property (nonatomic, copy)   NSString *copiedName;
```

И код:

```
NSMutableString *m = [NSMutableString stringWithString:@"Анна"];
p.strongName = m;
p.copiedName = m;
```

```
[m appendString:@" Каренина"];
NSLog(@"%@ / %@", p.strongName, p.copiedName);
```

Что напечатает?

Разбор. Подвох в том, что NSString * может в действительности указывать на NSMutableString — изменяемую строку (NSMutableString — потомок NSString). Мы кладем в оба свойства **одну и ту же** изменяемую строку m, а потом меняем m.

- strong хранит **ту же ссылку**, что ему дали. Свойство и переменная m указывают на один объект. Изменили m — изменилось и то, что видит strongName.
- copy при присваивании делает **снимок** — неизменяемую копию. Свойство получает свой отдельный объект @"Анна", на который последующие правки m уже не влияют.

Вывод (code/junior-05-copy-property.m):

```
strong: Анна Каренина
copy:   Анна
```

strongName «поехал» вслед за m, а copiedName остался снимком. Вот почему **строковые свойства почти всегда объявляют с copy**: ты хочешь своё неизменное значение, а не чужую строку, которую кто-то снаружи поменяет у тебя за спиной. То же правило — для NSArray, NSDictionary, NSSet: их свойства тоже делают copy.

Задача 6. BOOL, YES, NO и ловушка с числом

Что выведет?

```
NSLog(@"%d %d", YES, NO);
int weird = 256;
BOOL fromInt = (BOOL)weird;
NSLog(@"%d", fromInt);
```

Разбор. BOOL — это objс-шный «да/нет» с двумя именованными значениями: YES (под капотом 1) и NO (под капотом 0). Печатаем их через %d, потому что это маленькие целые. Первая строка очевидна: 1 0.

А вот вторая — классическая ловушка, и ответ в ней **зависит от платформы**. На Intel-маке (x86_64) BOOL — это signed char, обычное однобайтовое целое. Число 256 в один байт не влезает: его младший байт равен нулю (256 = 0x100). При укладке в BOOL старшие биты отбрасываются, остаётся 0, то есть NO. Вывод, снятый на Intel-маке (code/junior-06-bool.m):

```
YES = 1, NO = 0
sizeof(B00L) = 1 байт
есть элементы: 1
B00L из числа 256 = 0
```

Число было ненулевым (256, «истина по смыслу»), а B00L получился 0, «ложь».

На Apple Silicon (arm64) и на iPhone B00L — это настоящий bool из Си. Там любое ненулевое число при приведении превращается в 1, и последняя строка покажет B00L из числа 256 = 1. Хороший ответ на собеседовании называет обе платформы: «на x86_64 — 0 из-за обрезки байта, на arm64 — 1». Код, который работает только на одной из них, — баг. Отсюда два правила:

1. **Не заталкивай произвольное число прямо в B00L.** Если хочешь «не ноль ли это», пиши явное сравнение: `B00L ok = (count > 0);`.
2. **Не сравнивай B00L с YES напрямую** (`if (x == YES)`). Там, где B00L — `signed char`, в `x` может попасть, скажем, 2 (например, `B00L x = flags & 2;`). Тогда `x` — истина в условии `if (x)`, но `x == YES` (то есть `2 == 1`) даст ложь. Пиши просто `if (x)` — это проверяет «истинность», а не точное равенство единице.

Привычка «не пихать числа в B00L и не сравнивать с YES» спасает на любой платформе.

Задача 7. `addObject:nil` — что произойдёт

```
NSMutableArray *a = [NSMutableArray array];
NSString *name = nil;
[a addObject:name];
```

Программа продолжит работать?

Разбор. Нет — она **упадёт** прямо на `addObject:`. И это надо чётко отделять от задачи 1.

Сообщение `nil` (получатель — `nil`) безопасно. Но здесь получатель — это живой массив `a`, а `nil` идёт **аргументом**. `NSMutableArray` не умеет хранить «ничего»: коллекции Foundation хранят только настоящие объекты. Получив `nil` аргументом, метод выбрасывает исключение вроде:

```
*** -[__NSArrayM insertObject:atIndex:]: object cannot be nil
```

и программа аварийно завершается. Тот же эффект даст литерал с `nil` внутри: `@[a, nil, b]` — крах на месте сборки массива.

Поэтому перед добавлением проверяй, что объект есть:

```
if (name != nil) {  
    [a addObject:name];  
}
```

А если нужно осмысленное «здесь значения нет» — кладут не `nil`, а объект-заглушку `NSNull` (см. задачу 9).

Запоминай разницу: **`nil`-получатель** — тихо и безопасно; **`nil`-аргумент у коллекции** — краш.

Задача 8. `objectAtIndex`: против `firstObject` на пустом массиве

```
NSArray *empty = @[];  
id a = [empty firstObject];  
id b = [empty objectAtIndex:0];
```

Что вернёт каждая строка?

Разбор. Они ведут себя **по-разному**, и в этом вся соль.

- `firstObject` (и `lastObject`) на пустом массиве **возвращает `nil`** — безопасно, без падения.
- `objectAtIndex:0` на пустом массиве **роняет программу**: индекс 0 за границей пустого массива, runtime выбрасывает исключение `NSRangeException — index 0 beyond bounds for empty array`.

Поэтому вторую строку в файл мы не кладём. Безопасную часть проверяем в `code/junior-07-firstobject.m`:

```
firstObject пустого: (null)  
lastObject  пустого: (null)  
firstObject:      10  
objectAtIndex:0: 10  
lastObject:       30
```

На непустом массиве `firstObject` и `objectAtIndex:0` дают одно и то же. Разница только на пустом: один отдаёт `nil`, другой падает.

Практический вывод: когда не уверен, что в массиве есть элементы, бери крайние через `firstObject/lastObject` — они прощают пустоту. `objectAtIndex`: используй, только когда индекс точно в пределах `0 ... count-1`.

Задача 9. nil-значение в словаре и NSNull

```
NSMutableDictionary *d = [NSMutableDictionary dictionary];
d[@"name"] = @"Анна";
d[@"middle"] = [NSNull null];
d[@"city"] = nil;
NSLog(@"%@ / %@ / %@ / %@",
      d[@"name"], d[@"middle"], d[@"city"], d[@"absent"]);
```

Что выведет — и не упадёт ли строка с `d[@"city"] = nil`?

Разбор. Сразу про `d[@"city"] = nil` — это **не** краш и **не** вставка пустого значения. Для изменяемого словаря присваивание `nil` по ключу через скобки означает **удаление ключа**. После этой строки ключа `city` в словаре просто нет.

Хранить `nil` как значение словарь не может (та же причина, что у массива в задаче 7). Когда надо явно сказать «значение здесь отсутствует, и это осмысленно», кладут объект-заглушку **NSNull** — это настоящий объект, синглтон, который получают сообщением `[NSNull null]`.

Вывод (`code/junior-08-nsnull.m`, без строки `city`, чтобы показать работающий результат):

```
name: Анна
middle: <null>
absent: (null)
middle — это NSNull, а не настоящее значение
```

Тонкость: `<null>` (в угловых скобках) — это как печатается объект `NSNull`. А `(null)` (в круглых) — это `nil`, то есть «ключа нет вообще» (`absent` мы не клали). Отличать их важно: за отсутствующим ключом словарь возвращает `nil`, а `NSNull` — это лежащее по ключу реальное значение, которое надо проверять сравнением `== [NSNull null]`.

Зачем вообще `NSNull`? Чаще всего он прилетает из разобранного JSON: поле с `null` в ответе сервера превращается именно в `NSNull`, и забыть про это — типичный источник багов («думал, там строка, а там `NSNull`»).

Задача 10. Почему нельзя `[array addObject:5]`

```
NSMutableArray *a = [NSMutableArray array];
[a addObject:5];           // ???
[a addObject:@5];          // ???
```

Какая строка скомпилируется, какая нет, и почему?

Разбор. Первая строка **не компилируется**. `addObject:` ждёт объект (`id`), а `5` — это голый `int`, примитив языка Си, не объект. Объекту коллекции дать примитив нельзя.

Чтобы число попало в коллекцию, его сначала **упаковывают** (`boxing`) в объект `NSNumber`. Делают это литералом:

- `@5` — упаковать целое `5`;
- `@3.14` — упаковать дробное;
- `@(10 + 5)` — упаковать значение **выражения** (скобки обязательны, если внутри не одна константа, а вычисление или переменная: `@(x)`).

Вторая строка (`@5`) поэтому в порядке. Достать число обратно (`unboxing`) — сообщением вроде `intValue / doubleValue`. Вывод `code/junior-09-nsnumber.m`:

```
храним объект:    42
распаковали int:  42
второй элемент:   15
double обратно:   3.14
```

Отличие от Си. В Си массив `int a[]` хранит числа напрямую — это ведь просто кусок памяти. `NSMutableArray` хранит **только объекты**, поэтому число приходится заворачивать в `NSNumber`. Зато один массив может держать вперемешку строки, числа и что угодно — все они объекты.

Задача 11. Быстрый перебор `for-in` и мутация во время него

```
NSMutableArray *nums = [@[1, 2, 3, 4] mutableCopy];
for (NSNumber *x in nums) {
    if (x.intValue % 2 == 0) {
        [nums removeObject:x]; // удаляем прямо в цикле
    }
}
```

Что случится?

Разбор. Программа **упадёт**. Цикл `for (x in nums)` — это быстрый перебор (`fast enumeration`). Он проходит коллекцию и одновременно следит, не менялась ли она. Если изменить коллекцию **прямо во время перебора** (`removeObject:`, `addObject:`), `runtime` ловит это и выбрасывает исключение `NSGenericException` (адрес у тебя будет свой):

```
*** Collection <__NSArrayM: 0x6000010e4270> was mutated while being
enumerated.
```

Менять то, что сейчас перебираешь, нельзя. Правильных приёмов два:

1. **Собрать кандидатов, удалить потом.** Проходим коллекцию только на чтение, копим в отдельный массив, что надо убрать, и удаляем уже после цикла.
2. Перебирать **копию** (`for (x in [nums copy])`), а менять оригинал.

Первый приём проверяем в `code/junior-10-forin.m`. Там два куска: сначала обычный перебор массива фруктов (только чтение), потом удаление чётных из `@1, @2, @3, @4, @5` по первому приёму:

```
фрукт: яблоко (длина 6)
фрукт: банан (длина 5)
фрукт: вишня (длина 5)
без чётных: (
    1,
    3,
    5
)
```

(яблоко — 6 символов: перебор отработал штатно, потому что массив `fruits` мы только читали.
) Чётные удалились корректно, потому что `removeObjectsFromArray`: вызван **после** цикла, а не внутри.

Отличие от Си. Перебор `for (int i=0; i<n; i++) a[i]` в Си про мутацию ничего не знает — массив фиксированный, «сломать перебор вставкой» там нечем. Foundation-коллекции растут и сжимаются, поэтому язык специально защищается от изменения во время обхода.

Задача 12. Полиморфизм: какой метод вызовется

```
@interface Animal : NSObject
- (NSString *)sound;
- (void)describe;
@end
@implementation Animal
- (NSString *)sound { return @"..."; }
```

```

- (void)describe {
    NSLog(@"%@ говорит: %@", NSStringFromClass([self class]), [self sound]);
}
@end

@interface Dog : Animal @end
@implementation Dog
- (NSString *)sound { return @"Гав"; }
@end

// аналогично Cat -> @"Мяу"

NSArray *zoo = @[[Animal new], [Dog new], [Cat new]];
for (Animal *a in zoo) { [a describe]; }

```

describe написан **один раз** в Animal. Что напечатают три вызова?

Разбор. Здесь работает полиморфизм. Переменная цикла объявлена как Animal *a, но это лишь обещание компилятору «тут будет животное». А какой метод реально выполнится, runtime выбирает по **фактическому классу** объекта в момент выполнения (динамическая диспетчеризация, глава 5).

describe унаследован всеми и сам по себе не переопределён — поэтому выполняется один и тот же его код. Но внутри он шлёт [self sound], и вот тут self — это конкретный объект: для Dog сработает Dog-овский sound, для Cat — кошачий. Вывод (code/junior-11-polymorphism.m):

```

Animal говорит: ...
Dog говорит: Гав
Cat говорит: Мяу

```

Один метод describe, а звук у каждого свой — потому что sound выбирается по реальному классу self, а не по типу переменной a. NSStringFromClass([self class]) подтверждает: класс у каждого свой.

Отличие от Си. В Си describe(animal) — обычная функция, она вызвала бы один и тот же sound, какой указали при компиляции. Здесь объект сам решает в рантайме, какой sound его — это и есть «отправка сообщения», а не «вызов функции».

Задача 13. isKindOfClass: против isKindOfClass:

Dog — потомок Animal. Что выведет?


```
Dog *d = [Dog new];
NSLog(@"%d", [d isKindOfClass:[Animal class]]);
NSLog(@"%d", [d isKindOfClass:[Animal class]]);
NSLog(@"%d", [d isKindOfClass:[Dog class]]);
```

Разбор. Оба сообщения спрашивают объект о его классе, но по-разному:

- `isKindOfClass:` — «объект относится к этому классу **или любому его потомку?**». Учитывает всю цепочку наследования.
- `isMemberOfClass:` — «объект относится **ровно** к этому классу?». Точное совпадение, потомки не считаются.

`d` — это `Dog`, а `Dog` — потомок `Animal`. Значит:

- `[d isKindOfClass:[Animal class]]` → YES (`Dog` — разновидность `Animal`);
- `[d isKindOfClass:[Animal class]]` → NO (`d` не ровно `Animal`, а `Dog`);
- `[d isKindOfClass:[Dog class]]` → YES (вот теперь точное совпадение).

Полный вывод `code/junior-12-isKindOfClass.m`:

```
isKindOfClass:Dog      = 1
isKindOfClass:Animal   = 1
isKindOfClass:NSObject = 1
isMemberOfClass:Dog    = 1
isMemberOfClass:Animal = 0
```

`isKindOfClass:NSObject` тоже 1 — `NSObject` корень всех классов, любой объект ему «родственник». На практике почти всегда нужен именно `isKindOfClass:` (он учитывает наследование). `isMemberOfClass:` берут редко — когда важно поймать строго один класс и отсеять потомков.

Задача 14. `instancetype` или `id` в фабричном методе

В чём разница между двумя объявлениями и почему первое лучше?

```
+ (instancetype)widgetWithTitle:(NSString *)title; // так
+ (id)widgetWithTitle:(NSString *)title;           // или так?
```

Разбор. Оба варианта вернут объект, и программа в обоих случаях заработает. Разница в том, **что знает компилятор** о типе результата.

- `id` — «какой-то объект, тип неизвестен». Компилятор не проверит, что ты потом с ним делаешь.

- `instancetype` — «объект **именно этого** класса» (того, у которого вызвали метод). Компилятор знает точный тип и проверит обращения к нему.

Зачем это нужно на практике:

```
NSString *s = [Widget widgetWithTitle:@"Кнопка"]; // (1)
NSUInteger n = [Widget widgetWithTitle:@"Кнопка"].title.length; // (2)
```

- (1) Ошибка: в строковую переменную кладём `Widget`. С `instancetype` компилятор это ловит: `incompatible pointer types initializing 'NSString *' with an expression of type 'Widget *'`. С `id` — молчит, и программа упадёт уже при запуске, когда `s` получит строковое сообщение.
- (2) С `instancetype` результат — `Widget *`, и точка `.title` сразу работает. С `id` это ошибка компиляции: `property 'title' not found on object of type 'id'`. Особенно это важно при наследовании: фабричный метод с `instancetype`, написанный в `Widget`, для подкласса `RoundWidget` вернёт уже `RoundWidget *`, а с `id` — просто «объект».

Внутри метода для этого пишут `[[self alloc] init]`, а не `[[Widget alloc] init]:self` в методе класса — это сам класс, по которому метод вызвали, поэтому подкласс получит объект своего класса. Проверка `code/junior-13-instancetype.m`:

```
создан Widget с title = Кнопка
```

Тонкость для тех, кто хочет блеснуть: у методов семейств `alloc`, `init`, `new` компилятор выводит тип результата сам, даже если там написано `id` (так называемый `related result type`). У фабрик вроде `widgetWithTitle:` такого вывода нет — там `instancetype` обязателен.

Правило: для `alloc/init` и фабричных методов («конструкторов») возвращай **`instancetype`**. `id` оставляй там, где тип результата и правда заранее не известен.

Задача 15. Реализуй стек поверх `NSMutableArray`

Напиши класс `Stack` с операциями `push:`, `pop`, `peek`, `isEmpty`, `count`. Стек — это «стопка тарелок»: кладём сверху и снимаем сверху (LIFO, `last in — first out`).

Разбор. Стек удобно построить поверх готового `NSMutableArray`, работая только с **одним** его концом. Берём конец массива за «верх стопки»: `addObject:` кладёт в конец, `lastObject` смотрит конец, `removeLastObject` снимает с конца. Все три операции — быстрые.

```
@interface Stack : NSObject
- (void)push:(id)object;
- (id)pop;
```

```
- (id)peek;
- (BOOL)isEmpty;
- (NSUInteger)count;
@end

@implementation Stack {
    NSMutableArray *_items;          // приватное хранилище
}

- (instancetype)init {
    self = [super init];
    if (self) { _items = [NSMutableArray array]; }
    return self;
}

- (void)push:(id)object { [_items addObject:object]; }

- (id)pop {
    if (_items.count == 0) return nil;    // защита от выхода за границу
    id top = [_items lastObject];
    [_items removeLastObject];
    return top;
}

- (id)peek { return [_items lastObject]; }    // на пустом вернёт nil
- (BOOL)isEmpty { return _items.count == 0; }
- (NSUInteger)count { return _items.count; }

@end
```

Главное в реализации:

- Хранилище `_items` объявлено в фигурных скобках `@implementation` — это приватное поле, снаружи его не видно. Класс отдаёт наружу только осмысленные операции, а на чём он построен — его дело (инкапсуляция).
- В `pop` стоит проверка на пустоту. Документация Apple обещает, что `removeLastObject` у пустого массива выбросит `NSRangeException`. По нашему запуску современный Foundation молча ничего не делает, но опираться на недокументированную мягкость нельзя — по-

этому сначала проверяем и возвращаем `nil`. `peek` через `lastObject` проверки не требует — `lastObject` на пустом сам отдаёт `nil`.

- `_items` создаём в `init`, иначе он останется `nil`, и `push`: будет тихо слать сообщения в `nil` (см. задачу 1) — элементы «добавлялись» бы в никуда.

Вывод `code/junior-14-stack.m`:

```
пустой в начале: 1
после трёх push, count = 3
peek (не снимая): c
pop: c
pop: b
после двух pop, count = 1
pop: a
pop пустого: (null)
пустой в конце: 1
```

Видно главное свойство LIFO: положили `a`, `b`, `c`, а снимаются они в обратном порядке — `c`, `b`, `a`. `pop` пустого стека отдал `nil` и не уронил программу.

Задача 16. Найди утечку: сильная ссылка на `delegate`

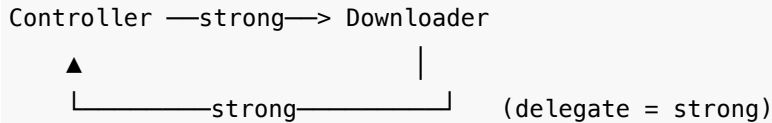
Что не так с этим кодом и к чему приведёт?

```
@interface Downloader : NSObject
@property (nonatomic, strong) id<DownloadDelegate> delegate; // strong
@end

@interface Controller : NSObject <DownloadDelegate>
@property (nonatomic, strong) Downloader *downloader;
@end

// контроллер создаёт загрузчик и ставит себя его делегатом:
self.downloader = [Downloader new];
self.downloader.delegate = self;
```

Разбор. Здесь **цикл удержания** (retain cycle) — обе стороны держат друг друга сильно, и ни одна не может освободиться:



Controller сильно держит Downloader (свойство `downloader`). Downloader сильно держит Controller обратно (свойство `delegate`). Когда Controller должен бы умереть, его всё ещё держит Downloader — а Downloader держит Controller. ARC видит, что у обоих ещё есть владделец, и не освобождает ни того, ни другого. Память утекла, `dealloc` не вызовется никогда.

Лечение — сделать `delegate` слабой ссылкой (`weak`):

```
@property (nonatomic, weak) id<DownloadDelegate> delegate;
```

`weak` указывает на объект, но **не удерживает** его. Цикл разрывается: сильная связь остаётся только одна (Controller → Downloader), и когда контроллер уходит, освобождаются оба. Файл `code/junior-15-delegate.m` доказывает это печатью из `dealloc`:

```
Controller: загрузка завершена
Controller освобождён
Downloader освобождён
после блока — оба объекта уже мертвы
```

Оба `dealloc` сработали — утечки нет. Будь `delegate` сильным, этих двух строк про «освобождён» в выводе не было бы вовсе.

Правило, которое спрашивают почти на каждом собеседовании: **свойство `delegate` всегда `weak`**. Делегат — это «тот, кому я докладываю», а не «тот, кем я владею»; владеть им я не должен.

Отличие от Си. В Си памятью управляешь сам (`malloc/free`), и «циклов удержания» нет — есть просто забытый `free` (утечка) или двойной `free` (крах). Здесь память считает ARC по числу сильных ссылок, и новый класс ошибок — два объекта, держащие друг друга. `weak` — штатное средство такие циклы рвать.

Задача 17. `mutableCopy` и `copy` — что можно менять

```
NSArray *a = @[a1, a2, a3];
NSMutableArray *m1 = [a copy];
NSMutableArray *m2 = [a mutableCopy];
[m2 addObject:a4]; // строка A
[m1 addObject:a5]; // строка B
```

Какая строка упадёт?

Разбор. Подвох в типе, который реально возвращают эти сообщения, а не в том, что написано слева.

- `copy` у массива возвращает **неизменяемый** `NSArray` — даже если ты присвоил его переменной типа `NSMutableArray *`. Компилятор поверит объявлению и пропустит код, но в рантайме объект-то неизменяемый.
- `mutableCopy` возвращает **изменяемый** `NSMutableArray` — настоящую редактируемую копию.

Поэтому:

- строка A (`[m2 addObject:@4]`) — в порядке: `m2` действительно изменяемый.
- строка B (`[m1 addObject:@5]`) — **краш**: `m1` в действительности неизменяемый `NSArray`, и в ответ на `addObject:` прилетит исключение `-[__NSArrayI addObject:]: unrecognized selector sent to instance` (имя класса в скобках зависит от того, как массив создан).

Бонус: `copy` у **неизменяемого** массива обычно даже не копирует — возвращает тот же объект. По нашему запуску `m1 == a` здесь истинно. Копировать то, что и так не меняется, незачем.

Тип переменной (`NSMutableArray *`) — это лишь обещание компилятору, а изменяемость определяет **реальный объект**, который вернуло сообщение. `copy` всегда отдаёт неизменяемый вариант, `mutableCopy` — изменяемый, как бы ты ни назвал переменную. Этот код мы не кладём в `code/`, потому что строка B роняет программу.

Задача 18. Объект-значение как ключ словаря

```
NSMutableDictionary *d = [NSMutableDictionary dictionary];
NSString *k1 = [NSString stringWithFormat:@"пользователь_%d", 7];
d[k1] = @"Анна";
NSString *k2 = [NSString stringWithFormat:@"пользователь_%d", 7];
NSLog(@"%@@", d[k2]);
```

`k1` и `k2` — разные объекты. Найдётся ли значение по `k2`?

Разбор. Да, напечатает Анна. И это важно понять: словарь ищет ключ **не по адресу, а по содержимому**.

Когда кладёшь и достаёшь по ключу, `NSDictionary` не сравнивает указатели через `==`. Он спрашивает у ключа две вещи: его `hash` (число-отпечаток) и `isEqual:` (равенство по содержимому).

У двух строк `user_7` хеш одинаковый и `isEqual:` вернёт YES, хотя это разные объекты в памяти. Поэтому ключ находится.

Отсюда следствие, о котором спрашивают чуть выше уровнем: если делаешь **свой** класс ключом словаря, ты обязан правильно реализовать у него пару `isEqual:` и `hash` (согласованно: равные объекты — одинаковый хеш). Иначе словарь не найдёт по ключу то, что в него положил. У готовых `NSString` и `NSNumber` это уже сделано за тебя — потому строки и числа и используют ключами чаще всего.

(Почему ключ длинный, кириллицей? Короткую ASCII-строку вроде `user_7 runtime` упакует в `tagged pointer` (задача 3), и `k1 == k2` окажется истиной — вопрос потеряет смысл.)

Ещё деталь: ключи словарь **копирует** (copy), поэтому ключ обязан поддерживать протокол `NSCopying`. Эта копия и защищает: даже если ключом дали `NSMutableString`, а потом поменяли её, внутри словаря лежит замороженный снимок, и значение по старому тексту находится. А вот `NSSet` свои элементы **не** копирует: положишь в множество изменяемую строку, поменяешь её — хеш собьётся, и множество не найдёт даже её саму.

Задача 19. Свойство `readonly` и приватная изменяемость

Снаружи класс отдаёт неизменяемый список, а внутри хочет его пополнять. Как так сделать и где тут типичная ошибка новичка?

```
@interface Playlist : NSObject
@property (nonatomic, copy, readonly) NSArray *songs;
- (void)addSong:(NSString *)song;
@end
```

Разбор. `readonly` означает «снаружи есть только геттер, сеттера нет». Это правильное оформление: чужой код может прочитать `playlist.songs`, но не подменит весь список и не полезет добавлять в него напрямую.

Ошибка новичка — отдать наружу **тот самый** внутренний `NSMutableArray`:

```
- (NSArray *)songs { return _songs; } // _songs изменяемый!
```

Тип в объявлении — `NSArray *`, но фактически вернётся изменяемый объект. Любой снаружи сможет привести его обратно к `NSMutableArray` и менять твой внутренний список за спиной — инкапсуляция сломана. Правильно — внутри держать `NSMutableArray`, а наружу отдавать **снимок**:

```
@implementation Playlist {
    NSMutableArray *_songs;    // своё поле: свой геттер у readonly-свойства
}                             // отключает автоматическое создание _songs
- (NSArray *)songs { return [_songs copy]; }
// ... init создаёт _songs, addSong: делает [_songs addObject:song]
@end
```

(Атрибут `copy` у `readonly`-свойства на геттер никак не влияет — он касается только сеттера, а сеттера снаружи нет. Поэтому снимок в геттере делаем руками.) А пополнять список — только через свой метод `addSong:`, который работает с приватным `_songs` напрямую. Так класс полностью контролирует, кто и когда меняет его данные. Это та же мысль, что в задачах 5 и 17: не отдавай наружу изменяемое нутро, отдавай неизменяемый снимок.

Задача 20. Что такое `self` в методе и зачем `[super init]`

```
- (instancetype)init {
    self = [super init];
    if (self) {
        _value = 100;
    }
    return self;
}
```

Объясни каждую строку: что такое `self`, что такое `super`, почему результат `[super init]` присваивают обратно `self`.

Разбор. Это шаблон назначенного инициализатора, и его просят разобрать очень часто.

- **self** — внутри любого метода это «текущий объект-получатель», тот, кому послали сообщение. Через `self` объект обращается к своим полям и шлёт сообщения сам себе.
- **super** — тот же самый объект `self`, но «глядя на него как на родителя»: сообщение через `super` ищет метод не в своём классе, а начиная с **родительского**. `[super init]` означает «пусть сначала родитель (`NSObject` и выше) настроит свою часть объекта».

Почему `self = [super init];`, а не просто `[super init];`? Потому что `init` родителя **имеет право вернуть другой объект** или `nil`. Это редко, но бывает (так устроены некоторые классы Foundation). Если родитель вернул другой объект, дальше настраивать надо именно его, поэтому результат кладут обратно в `self`. А `if (self)` страхует от случая, когда родитель вернул

`nil` (инициализация не удалась) — тогда мы не лезем настраивать поля у несуществующего объекта и сразу вернём `nil`.

Порядок строго такой: **сначала родитель** (`[super init]`), **потом своя настройка** (`_value = 100`). Наоборот нельзя — пока родитель не привёл объект в порядок, трогать его рано. И в конце обязательно `return self;` — без него инициализатор вернёт мусор. Рабочий пример этого шаблона — `code/junior-04-alloc-init.m` из задачи 4.

Отличие от Си. В Си не было ни `self`, ни `super` — была функция и переданные ей аргументы. Здесь объект знает сам себя (`self`) и умеет позвать «родительскую версию» метода (`super`) — это часть объектной модели, которой в чистом Си попросту нет.

Что повторить перед собеседованием

Короткий чек-лист по этому разделу. Если на любой пункт отвечаешь неуверенно — вернись к нужной задаче.

- Сообщение `nil`-получателю безопасно и даёт `0/nil/NO`; `nil` аргументом коллекции — краш (задачи 1, 7).
- Строки и числа сравнивают по содержимому: `isEqualToString:` / `isEqualToNumber:`, не `==` (задачи 2, 3).
- Объект рождается связкой `[[Класс alloc] init];` `init` строится по шаблону `self = [super init] ... return self` (задачи 4, 20).
- Строковые и коллекционные свойства объявляй с `copy`; наружу отдавай неизменяемый снимок, не внутренний `mutable` (задачи 5, 17, 19).
- `BOOL` — это `YES/NO`; не пихай в него произвольные числа и не сравнивай с `YES` напрямую (задача 6).
- Числа в коллекциях — только упакованные в `NSNumber` (`@5`, `@(x)`); отсутствие значения — `NSNull`, не `nil` (задачи 9, 10).
- Коллекцию нельзя менять во время `for-in` (задача 11).
- Метод выбирается по фактическому классу объекта — полиморфизм; `isKindOfClass:` учитывает наследование, `isMemberOfClass:` — нет (задачи 12, 13).
- Для `alloc/init` и фабрик возвращай `instancetype`, не `id` (задача 14).
- `delegate` всегда `weak` — иначе цикл удержания (задача 16).

Документация Apple

- Programming with Objective-C → Working with Objects (объекты, сообщения, nil, id) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/WorkingwithObjects/WorkingwithObjects.html
- Programming with Objective-C → Encapsulating Data (свойства, copy, strong/weak,instancetype) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/EncapsulatingData/EncapsulatingData.html
- NSObject Protocol (isEqual:, hash, isKindOfClass:, isKindOfClass:, respondsToSelector:) — developer.apple.com/documentation/objectivec/nsobjectprotocol
- NSString — developer.apple.com/documentation/foundation/NSString
- NSNumber — developer.apple.com/documentation/foundation/NSNumber
- NSArray / NSMutableArray — developer.apple.com/documentation/foundation/NSArray и developer.apple.com/documentation/foundation/NSMutableArray
- NSDictionary / NSMutableDictionary — developer.apple.com/documentation/foundation/NSDictionary
- NSNull — developer.apple.com/documentation/foundation/NSNull
- Advanced Memory Management Programming Guide (ARC, сильные/слабые ссылки, циклы удержания) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/MemoryMgmt/Articles/MemoryMgmt.html

Приложение Е. Задачи собеседования: Middle

Junior-раздел проверял, помнишь ли ты механику: сообщения, nil, свойства, простой ARC. Middle-уровень спрашивает иначе — не «что делает эта строка», а «что произойдёт с памятью», «что выведет этот код и почему», «как ты спроектируешь вот это». Здесь почти не бывает ответа в одну строку: за каждым вопросом стоит модель владения, поведение runtime или контракт класса.

Формат похож на Junior-раздел. Каждая задача — это **условие**, потом **подумай сам** (остановись и попробуй ответить), потом **разбор** по правилам Objective-C и, где задача про код, — **компилируемое решение** в code/middle-NN-имя.m. Каждый показанный вывод реальный: файлы собраны и запущены, dealloc-логи настоящие. Дата, имя программы и число в квадратных скобках, которые NSLog печатает в начале строки, здесь убраны — у тебя они будут свои.

Собрать и запустить любой файл:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/middle-01-retain-cycle.m -o /tmp/t && /tmp/t
```

Идём от простого к сложному: сперва блоки и память, потом копирование, протоколы и категории, в конце — KVC/KVO и контракты NSCopying, isEqual:/hash.

Задача 1. Что выведет блок: захват по значению и __block

Условие. Перед тобой три фрагмента. Скажи, что напечатает каждый.

```
int x = 10;
void (^plain)(void) = ^{ NSLog(@"%d", x); };
x = 99;
plain();                      // (1) ?
```

```

__block int y = 10;
void (^shared)(void) = ^{ NSLog(@"%d", y); };
y = 99;
shared();                                // (2) ?

NSMutableArray *blocks = [NSMutableArray array];
for (int i = 0; i < 3; i++) {
    [blocks addObject:^( NSLog(@"%d", i); )];
}
for (void (^b)(void) in blocks) { b(); }    // (3) ?

```

Подумай сам. Когда блок «снимает» значение переменной — в момент создания или в момент вызова?

Разбор. Обычный захват фиксирует **копию значения в момент создания блока** (глава 12). Дальнейшие изменения переменной блок не видит.

- (1) plain захватил x равным 10. Присваивание $x = 99$ идёт уже после создания блока — на захваченную копию оно не влияет. Печатает **10**.
- (2) __block меняет природу захвата: y становится **общей ячейкой**, на которую смотрят и внешний код, и блок. К моменту вызова shared в ячейке лежит 99. Печатает **99**.
- (3) Каждая итерация создаёт новый блок и захватывает **своё** значение i на тот момент: 0, 1, 2. Печатает **0 1 2**.

Третий пункт — частая ловушка для тех, кто пришёл из языков, где замыкание в цикле захватывает переменную «по ссылке» и все блоки потом видят финальное значение. В Objective-C без __block каждый блок держит свою копию. Сделай i объявленным как __block — и все три блока напечатают одно и то же финальное значение цикла: все блоки делят одну ячейку i , и после выхода из цикла в ней 3. Напечатается 3 3 3 (по нашему запуску).

Решение: code/middle-02-block-capture.m.

```

plain видит x = 10
shared видит y = 99
блок помнит i = 0
блок помнит i = 1
блок помнит i = 2

```

Задача 2. weak после смерти объекта и ловушка unsafe_unretained

Условие. Объект создан во вложенном блоке и умирает на выходе из него. Снаружи на него смотрит __weak-ссылка. Что напечатают две последние строки — и что было бы, объяви мы ссылку unsafe_unretained?

```
__weak Ticket *weakRef = nil;
@autoreleasepool {
    Ticket *t = [[Ticket alloc] init];
    t.code = @"A-17";
    weakRef = t;
} // t умирает здесь
NSLog(@"%@", weakRef); // ?
NSLog(@"%lu", (unsigned long)weakRef.code.length); // ?
```

Подумай сам. Что лежит в weakRef после смерти объекта — мусор, nil или старый адрес?

Разбор. weak — это ссылка-наблюдатель: она не удерживает объект, а когда тот умирает, ARC сам обнуляет её в nil (глава 13). Поэтому слабая ссылка либо показывает живой объект, либо честный nil — но никогда не висячий указатель.

- NSLog(@"%@", weakRef) после смерти t печатает (**null**).
- weakRef.code.length — это сообщение к nil. В Objective-C сообщение к nil безопасно и возвращает «нулевое» значение, для числа — 0.

Будь ссылка unsafe_unretained (или старое свойство assign), всё иначе: она **не обнуляется**. После смерти объекта она осталась бы указывать на освобождённую память — висячий указатель. Первая же NSLog(@"%@", ...) по нему — обращение к мёртвому объекту, неопределённое поведение: мусор или падение. Правило простое: нужна необременяющая ссылка «вверх» или на делегата — бери weak, а не unsafe_unretained, если только тебе не важна каждая наносекунда и ты не доказал, что объект переживёт ссылку.

Решение: code/middle-03-weak-nil.m.

```
объект жив: weakRef.code = A-17
dealloc Ticket (A-17)
объект мёртв: weakRef = (null)
сообщение к nil безопасно: длина = 0
```

Строка dealloc Ticket (A-17) встала ровно на закрывающей скобке вложенного блока — объект умер там, и сразу после слабая ссылка стала nil.

Задача 3. Найди утечку: команда и игроки

Условие. Команда владеет игроками (сильно держит их в массиве), а каждый игрок ссылается на свою команду обратной ссылкой. Найди ошибку, из-за которой `dealloc` не сработает, и почини её.

```
@class Team;    // Team объявлен ниже, а Player уже ссылается на него

@interface Player : NSObject
@property (nonatomic, copy) NSString *name;
@property (nonatomic, strong) Team *team;    // ← подозрительно
@end

@interface Team : NSObject
@property (nonatomic, strong) NSMutableArray<Player *> *players;
@end
```

Подумай сам. Кто кого держит и где замыкается кольцо?

Разбор. Считаем сильные ссылки. `Team.players` сильно держит каждого `Player` — это правильно, команда владеет составом. Но `Player.team` объявлен `strong` — игрок тоже сильно держит команду. Получается кольцо: команда → игрок → команда. Когда снаружи на оба объекта ссылок не останется, их счётчики не дойдут до нуля, `dealloc` не сработает — классическая утечка (глава 13).

Чинить надо ту ссылку, которая **не владеет по смыслу**. Команда владеет игроком — `Team.players` оставляем `strong`. Игрок командой не владеет, лишь смотрит «вверх» — значит `Player.team` должно быть **weak**. Меняем один атрибут, и кольцо разорвано:

```
@property (nonatomic, weak) Team *team;    // вверх не владеем
```

Это одна из трёх типовых пар, где рождаются циклы: `parent` ↔ `child` (разбирали в главе 13), объект ↔ блок (глава 12), объект ↔ `delegate` (глава 10). Лечение всегда одно — одну из двух ссылок сделать слабой, и слабой делает ту, что «вверх» или «вбок», а не «вниз к собственности».

Решение: `code/middle-01-retain-cycle.m` (уже с починкой — `weak`).

```
Асет играет за Кайрат
выходим из блока — ждём три dealloc...
dealloc Team (Кайрат)
dealloc Player (Асет)
dealloc Player (Болат)
блок закрыт — dealloc были выше, утечки нет
```

Три `dealloc` на месте — утечки нет. Верни `Player.team` обратно в `strong`, пересобери — и все три строки `dealloc` исчезнут. Вот так, по пропавшему `dealloc`, утечку и ловят.

Задача 4. Цикл удержания объект ↔ блок: что выведет `dealloc`

Условие. Объект хранит блок в своём свойстве, а блок внутри обращается к `self`. Два варианта метода — какой из них утечёт и почему? Сработает ли `dealloc`?

```
@interface Worker : NSObject
@property (nonatomic, copy) void (^job)(void);
@property (nonatomic, copy) NSString *name;
@end

- (void)setupBad {
    self.job = ^{ NSLog(@"%@", self.name); };           // A
}

- (void)setupGood {
    __weak typeof(self) weakSelf = self;
    self.job = ^{ NSLog(@"%@", weakSelf.name); };       // B
}
```

Подумай сам. Что захватывает блок, когда упоминает `self.name`?

Разбор. Блок захватывает объекты **сильно**. Стоит блоку упомянуть `self.name`, как он захватывает сам `self` — сильно. А `self` сильно держит блок в свойстве `job` (атрибут `copy`, для блоков это правильно). Кольцо: `self` → `job` → `self`.

- В варианте А (`setupBad`) кольцо замкнуто. После того как внешний код отпустит `Worker`, его счётчик останется 1 (держит блок), `dealloc` **не сработает**. Кстати, `clang` это кольцо видит и предупреждает: `capturing 'self' strongly in this block is likely to lead to a retain cycle [-Warc-retain-cycles]`. Глушить предупреждение нельзя — это и есть баг.
- В варианте В (`setupGood`) блок захватывает не `self`, а `weakSelf` — слабую копию ссылки. Сильно `self` блок больше не держит, кольца нет. Когда внешний код отпустит `Worker`, тот спокойно уничтожится, и `dealloc` **сработает**.

Полный рабочий пример с обоими вариантами и живыми `dealloc`-логами уже разобран в главе 12 (`code/12-retain-cycle.m`): для `bad` строки `dealloc` нет, для `good` — есть. Идиома `__weak typeof(self) weakSelf = self;` — твой главный инструмент против цикла «объект ↔ блок». Если внутри блока `self` нужен несколько раз и важно, чтобы он не исчез посреди работы, дела-

ют ещё `__strong typeof(weakSelf) strongSelf = weakSelf;` в начале блока — это «weak-strong dance», но базовый ответ на собеседовании — `weakSelf`.

Задача 5. Зачем `@autoreleasepool` внутри длинного цикла

Условие. Тебе говорят: «оберни тело тяжёлого цикла в `@autoreleasepool`». Объясни, что это даёт. Покажи на примере, что объект, которого держит только пул, живёт ровно до слива пула.

Подумай сам. Кто освобождает временные объекты, у которых нет явного владельца, и **когда**?

Разбор. Многие объекты живут «в долг»: их никто не держит сильно, и в живых их удерживает только **пул автоосвобождения** (глава 13). Пул сливается (`drain`) на своей закрывающей скобке и шлёт всем накопленным объектам отложенный `release`. Если ты в длинном цикле плодишь тысячи таких временных объектов, они копятся в пуле до самого его слива — память пухнет, хотя на каждом шаге тебе нужен лишь один. Свой `@autoreleasepool` внутри тела цикла сливается на каждой итерации и держит пик памяти низким.

Важная честная оговорка про **современный ARC**. Для объектов, которые метод возвращает «в долг», ARC умеет рукопожатие `objc_autoreleaseReturnValue / objc_retainAutoreleasedReturnValue`: если обе стороны его поддерживают, объект передаётся из рук в руки, вообще не попадая в пул. Поэтому объект твоего собственного ARC-класса, возвращённый из фабрики, в пул может и не попасть. Но полагаться на это нельзя: многие методы Foundation по-прежнему кладут результат в пул. По нашему замеру (macOS, Intel, -02) цикл из `2 000 000 [NSString stringWithFormat:]` без внутреннего пула поднял пик памяти до ~275 МБ, а с `@autoreleasepool` в теле цикла — всего до ~2,3 МБ.

Чтобы увидеть момент смерти объекта своими глазами (через `dealloc`), в примере нужен свой класс `Temp`. Его рукопожатие как раз увело бы мимо пула, поэтому мы кладем объект в пул явно через `CFAutorelease(CFBridgingRetain(t))` — эти функции отдают объект пулу в обход ARC-оптимизации.

Решение: `code/middle-04-autoreleasepool.m`.

```
=== объект живёт ровно до слива пула ===
внутри пула: Temp(1) ещё жив (dealloc не было)
    dealloc Temp(1)
после слива: Temp(1) уже мёртв (dealloc был выше)
=== тяжёлый цикл: пул на каждой итерации ===
итерация 0: объект создан
    dealloc Temp(100)
```



```
итерация 1: объект создан
    dealloc Temp(101)
итерация 2: объект создан
    dealloc Temp(102)
цикл закончен — пул не дал памяти расти
```

`dealloc Temp(1)` встал точно между «внутри пула» и «после слива» — объект дожид до закрывающей скобки пула и умер на ней. В цикле каждый `Temp` умирает на сливе своей итерации, а не копится до конца.

Задача 6. Поверхностная копия коллекции

Условие. Есть массив, внутри которого лежит **мутабельный** вложенный массив. Делаем `mutableCopy` и потом меняем вложенный массив. Что увидим в обеих коллекциях?

```
NSMutableArray *inner = [NSMutableArray arrayWithObjects:@"a", @"b", nil];
NSArray *original = @[inner, @"hello"];
NSMutableArray *shallow = [original mutableCopy];
[inner addObject:@"c"];
// original[0] = ?   shallow[0] = ?
```

Подумай сам. Что именно копирует `mutableCopy` — контейнер, его содержимое или и то и другое?

Разбор. `copy` и `mutableCopy` у коллекций делают **поверхностную копию** (`shallow copy`): создаётся новый контейнер, но в него кладутся **те же самые** объекты, что были в оригинале, — по ссылке, без копирования. Контейнеры теперь разные (`original != shallow`), а вот вложенный массив `inner` — **общий** на обе коллекции (`original[0] == shallow[0]`).

Поэтому `[inner addObject:@"c"]` меняет один объект, видимый из обеих коллекций: и `original[0]`, и `shallow[0]` покажут (a, b, c). Копия защитила тебя только от изменений **верхнего** уровня (добавь элемент в `shallow` — `original` не дрогнет), но не от изменений вложенных мутабельных объектов.

Когда нужна полная независимость, делают **глубокую копию** (`deep copy`). Простой способ — прогнать структуру через архиватор: `NSKeyedArchiver` → `NSKeyedUnarchiver`. После такой копии вложенные массивы тоже становятся отдельными. Ещё один ответ, который ждут на собеседовании, — `[[NSArray alloc] initWithArray:original copyItems:YES]`: он шлёт `copy` каждому элементу, но только на **один** уровень вглубь (и вложенный `NSMutableArray` при этом станет неизменяемым `NSArray`).

Решение: code/middle-05-shallow-copy.m.

```
разные контейнеры? да
общий вложенный массив? да
original[0] = (
    a,
    b,
    c
)
shallow[0] = (
    a,
    b,
    c
)
после deep-копии inner += d:
original[0] = (
    a,
    b,
    c,
    d
)
deep[0] = (
    a,
    b,
    c
)
```

После mutableCopy оба массива видят c. После глубокой копии добавленный d ушёл только в original — deep его не заметил, потому что у него свой, отдельный вложенный массив.

Задача 7. copy-семантика у NSString-свойства

Условие. Свойство объявлено strong, а присваивают ему NSMutableString. Потом исходную строку меняют. Что станет со значением свойства? Сравни со свойством copy.

```
@property (nonatomic, strong) NSString *strongName;
@property (nonatomic, copy)   NSString *safeName;
```

```
NSMutableString *src = [NSMutableString stringWithString:@"Айгуль"];
p.strongName = src;
p.safeName    = src;
[src appendString:@" Сергеевна"]; // меняем ИСХОДНУЮ строку
```

Подумай сам. Что хранит свойство `strong` — снимок строки или ссылку на ту же `src`?

Разбор. `NSMutableString` — наследник `NSString`, поэтому его можно присвоить свойству типа `NSString *`. Но дальше всё зависит от атрибута:

- `strong` сохраняет **ту же ссылку** на `src`. Изменили `src` — и `strongName` «поехал» вслед, ведь это один и тот же объект. После `appendString:` он покажет Айгуль Сергеевна. Объект мутирует за спиной владельца — источник коварных багов.
- `copy` в сеттере вызывает `[src copy]` и сохраняет **неизменяемый снимок**. Дальнейшие изменения `src` его не трогают: `safeName` так и останется Айгуль.

Отсюда правило, которое на собеседовании ждут почти всегда: **строковые свойства (`NSString`, `NSArray`, `NSDictionary` и любые типы с мутабельным наследником) объявляй `copy`**. Тогда снаружи никто не подсунет тебе мутабельный объект и не изменит его потом за твоей спиной. У `copy` для неизменяемой строки накладных расходов почти нет — `[immutableString copy]` часто возвращает тот же объект с поднятым счётчиком, а не настоящую копию.

Решение: `code/middle-08-string-copy.m`.

до мутации:

```
strongName = Айгуль
safeName    = Айгуль
```

после мутации `src`:

```
strongName = Айгуль Сергеевна
safeName    = Айгуль
```

Одно и то же присваивание, разные атрибуты — и `strong` «протёк» изменением исходной строки, а `copy` устоял.

Задача 8. Простой кэш на `NSMutableDictionary` и `copy`-ключи

Условие. Напиши простой кэш «ключ-значение» поверх `NSMutableDictionary`. Объясни, что произойдёт, если в качестве ключа передать `NSMutableString`, а потом изменить эту строку. Останется ли значение доступным?

Подумай сам. NSDictionary хранит ссылку на твой ключ или его копию?

Разбор. NSDictionary хранит **копию** ключа — поэтому ключ обязан поддерживать NSCopying, а при вставке словарь сам вызывает у ключа `copy`. Это сделано именно ради защиты: если бы словарь держал ссылку на твой мутабельный ключ, ты бы изменил строку — изменился бы и её `hash`, и словарь больше не нашёл бы значение по «тому же» ключу. Снимок-копия замораживает ключ в момент вставки.

Значит, наш кэш над `NSMutableDictionary` уже корректен: `setObject:forKey:` сам копирует ключ. Передали `NSMutableDictionary` `@\"user:1\"`, потом дописали к ней `-changed` — в словаре всё равно лежит замороженная копия `user:1`. Значение достаётся по литералу `@\"user:1\"`, а по изменённой строке — уже нет (другой ключ).

Когда кэш должен ещё и сбрасываться под нехватку памяти, вместо `NSMutableDictionary` берут `NSCache`: тот сам выкидывает элементы при давлении на память и потокобезопасен. Но ключи `NSCache`, в отличие от словаря, не копирует (просто держит сильно), поэтому для строковых ключей либо копируй их сам, либо используй неизменяемые.

Решение: `code/middle-13-cache.m`.

```
по 'user:1':      Асет
по изменённому ключу: (null)
объектов в кэше: 1
```

Значение нашлось по оригинальному ключу и не нашлось по изменённой строке — доказательство того, что словарь сохранил копию ключа, а не ссылку на твою мутабельную строку.

Задача 9. Спроектируй делегат с необязательными методами

Условие. Спроектируй протокол делегата для загрузчика: один метод обязательный («завершилось»), один необязательный («прогресс»). Покажи, как загрузчик зовёт необязательный метод безопасно, и объясни, почему свойство `delegate` обычно `weak`.

Подумай сам. Что будет, если позвать `@optional`-метод у делегата, который его не реализует?

Разбор. Протокол делит методы директивами `@required` (по умолчанию) и `@optional`. Обязательный метод протокол гарантирует — его можно звать без проверки. А необязательный делегат может и не реализовать; если позвать несуществующий метод напрямую — `unrecognized selector`, падение. Поэтому перед вызовом `@optional`-метода спрашивают `respondsToSelector:` — реализует ли делегат этот селектор:

```
if ([self.delegate respondsToSelector:@selector(downloaderDidProgress:)]) {
    [self.delegate downloaderDidProgress:0.5];
}
```

Свойство-делегат объявляют **weak** по той же причине, что в задаче 3: почти всегда владелец держит объект, а тот через delegate смотрит назад на владельца. Сделай delegate сильным — получишь цикл удержания владелец → объект → delegate → владелец. Слабая ссылка рвёт кольцо и заодно сама обнуляется в nil, если делегат умер раньше, — тогда вызовы к self.delegate просто молча уходят в nil.

Тип свойства — id<DownloaderDelegate>: «любой объект, лишь бы соответствовал протоколу». Протокол наследует <NSObject>, чтобы у делегата гарантированно был respondsToSelector:.

Решение: code/middle-06-delegate.m.

```
--- minimal (без optional) ---
Minimal: готово, DATA
--- full (с optional) ---
Full: прогресс 50%
Full: готово, DATA
```

Первый делегат реализует только обязательный метод — прогресс пропущен без падения. Второй реализует оба — прогресс вызван. Один и тот же загрузчик, поведение задаёт делегат.

Задача 10. Две категории с одинаковым методом — что вызовется

Условие. В двух разных категориях одного класса объявлен метод с **одинаковым** именем, но разной реализацией. Какая из них победит при вызове?

```
@interface NSString (A)
- (NSString *)tagline;
@end

@implementation NSString (A)
- (NSString *)tagline { return @"из A"; }
@end

@interface NSString (B)
- (NSString *)tagline;
@end

@implementation NSString (B)
```

```
- (NSString *)tagline { return @"из B"; }  
@end
```

Подумай сам. Гарантирует ли язык, какая реализация окажется в таблице методов?

Разбор. Это **неопределённое поведение**, и так на собеседовании и отвечают. Когда две категории одного класса определяют метод с одним именем, какая из них «ляжет» в таблицу методов класса последней — **не специфицировано**. Обычно побеждает категория, чей объектный файл runtime загрузил позже (зависит от порядка линковки), но опираться на это нельзя.

Хуже того — категория, переопределяющая метод **самого класса** (не другой категории, а, скажем, родной метод NSString), молча затирает оригинал, и вызвать прежнюю реализацию уже не получится: `super` ведёт к методу родителя, а не к затёртому оригиналу. Поэтому правила такие:

- **Не переопределяй существующие методы через категорию.** Категория — чтобы **добавлять** методы, а не подменять. Хочешь изменить поведение — делай подкласс.
- Имена методов в категориях **префиксируй** (`xyz_tagline`), чтобы не столкнуться ни с будущими методами Apple, ни с чужой категорией. Особенно это важно для категорий на классах Foundation.

Запускать такой код как «эксперимент» бессмысленно: результат у тебя может оказаться из А, у коллеги — из В, а после перестановки файлов в сборке — снова другой. Правильный ответ — «поведение не определено, так делать нельзя».

Задача 11. Категория не может добавить ivar — как обойти

Условие. Тебе нужно «прицепить» дополнительное поле к чужому классу (скажем, NSString) через категорию. Но категория не умеет добавлять ivar. Как сохранить данные на объекте?

Подумай сам. Где хранить значение, если места под ivar в объекте нет?

Разбор. Категория добавляет **методы**, но не **переменные экземпляра**: раскладку полей объекта runtime фиксирует, когда подключает сам класс, а категория подключается к уже готовому классу (у которого, возможно, уже есть живые экземпляры) и менять её не может. Расширение класса (class extension) ivar добавлять умеет — но только потому, что компилируется вместе с `@implementation` класса. Объявить в категории `@property` можно, но автоматического ivar под него не появится — будут только объявления геттера и сеттера, которые ты обязан написать сам. А хранить значение — через **связанные объекты** (associated objects) из runtime:

```
#import <objc/runtime.h>
static const char kTagKey;    // адрес переменной = уникальный ключ

- (void)setTag:(NSString *)tag {
    objc_setAssociatedObject(self, &kTagKey, tag,
                             OBJC_ASSOCIATION_COPY_NONATOMIC);
}
- (NSString *)tag {
    return objc_getAssociatedObject(self, &kTagKey);
}
```

`objc_setAssociatedObject` привязывает значение к объекту по **ключу** — им служит адрес статической переменной (`&kTagKey`), он гарантированно уникален. Политика `OBJC_ASSOCIATION_COPY_NONATOMIC` означает «храни копию, без атомарности» — это аналог атрибута `copy` у свойства. Runtime сам снимет связанные объекты, когда носитель умрёт, — следить за их освобождением не нужно.

Это ровно тот механизм, на котором, например, держат «обёрнутый» делегат или флаг на ячейке таблицы, не плодя подкласс.

Решение: `code/middle-07-associated.m`.

```
до: tag = (null)
после: tag = важный
```

До присваивания связанного значения нет — геттер возвращает `nil`. После — строка достаётся обратно с того же объекта.

Задача 12. KVC: доступ по строке, `keyPath` и операторы

Условие. Что напечатает этот код? Объясни `valueForKey:`, `valueForKeyPath:` и операторы `@sum/@avg/@max`.

```
[e setValue:@350000 forKey:@"salary"];
[e valueForKeyPath:@"address.city"];
[team valueForKeyPath:@"@sum.salary"];
[team valueForKeyPath:@"@avg.salary"];
```

Подумай сам. Как KVC превращает строку `@salary` в вызов сеттера, и что делает `@` перед именем оператора?

Разбор. KVC (Key-Value Coding, глава 20) даёт доступ к свойствам по **строковому имени**, известному только во время выполнения:

- `setValue:forKey:` ищет сеттер `setSalary:` и зовёт его. Свойство — примитив `NSInteger`, поэтому KVC сам **распаковывает** переданный `NSNumber` (`@350000`) в `NSInteger`. Обратно, `valueForKey:` для примитива сам **упаковывает** значение в `NSNumber`.
- `valueForKeyPath:@"address.city"` идёт по точке сквозь вложенный объект: сначала `address`, у него — `city`. Это **keyPath**, цепочка ключей.
- Применённый к **массиву**, `valueForKey:@"name"` собирает значения ключа из всех элементов в новый массив.
- `@sum`, `@avg`, `@max`, `@count` — **коллекционные операторы**. `@` перед именем оператора отличает его от обычного ключа. `"@sum.salary"` означает «просуммируй `salary` по всем элементам массива». Результат — `NSNumber`.

Решение: `code/middle-09-kvc.m`.

```
valueForKey name = Асет
после setValue salary = 350000
keyPath city = Алматы
все имена: (
    "\U0410\U0441\U0435\U0442",
    "\U0411\U043e\U043b\U0430\U0442",
    "\U0414\U0430\U043d\U0430"
)
@sum.salary = 1200000
@avg.salary = 400000
@max.salary = 500000
@count      = 3
```

Кириллица в массиве показана escape-кодами вида `\U0410` — так `NSArray` печатает строки в `description`, это нормально. Сумма трёх зарплат — 1 200 000, среднее — 400 000, максимум — 500 000.

Задача 13. KVO: что приходит в `observeValueForKeyPath:` и почему `ivar` молчит

Условие. Подписались на свойство `balance` через KVO. Меняем его то через сеттер, то напрямую через `ivar`. Когда придёт уведомление, а когда нет? Что лежит в словаре `change`? Что обязательно сделать в конце?

Подумай сам. На чём именно стоит KVO — на присваивании `ivar` или на вызове сеттера?

Разбор. KVO (Key-Value Observing, глава 20) уведомляет наблюдателя об изменении свойства. Подписка — `addObserver:forKeyPath:options:context:.` Уведомление приходит в `observeValueForKeyPath:ofObject:change:context:.`, и в словаре `change` лежат старое и новое значения под ключами `NSKeyValueChangeOldKey` и `NSKeyValueChangeNewKey` (если при подписке запросил опции `Old` и `New`).

Ключевой момент: **KVO перехватывает сеттер, а не присваивание `ivar`.** Под капотом runtime подменяет класс объекта на сгенерированный подкласс, чей сеттер `setBalance:` шлёт уведомления вокруг настоящего присваивания. Поэтому:

- `acc.balance = 100` идёт через сеттер — **уведомление придёт.**
- `acc->_balance = 999` пишет прямо в `ivar`, минуя сеттер, — **никакого уведомления.** Значение свойства при этом поменялось (getter вернёт 999), но KVO про такое изменение не знает. В примере `ivar _balance` объявлен `@public` явно — только ради демонстрации прямой записи.

И обязательное: пока наблюдатель жив и подписан, а наблюдаемый объект ещё меняется, подписку надо снять до смерти наблюдателя — `removeObserver:forKeyPath:.` KVO не удерживает наблюдателя и не обнуляет ссылку на него. Забудешь — при следующем уведомлении KVO допустится до уже мёртвого наблюдателя и уронит программу. На старых системах (до macOS 10.13 и iOS 11) падала и смерть **наблюдаемого** объекта с живыми подписками: «was deallocated while key value observers were still registered». Теперь в обычном случае это прощается, но правило «подписался — отпишись» от этого не отменяется.

Решение: `code/middle-10-kvo.m`.

```
--- через сеттер (KVC-совместимо) ---
изменилось 'balance': 0 -> 100
изменилось 'balance': 100 -> 250
--- прямое изменение ivar (в обход сеттера) ---
balance стал 999, но observe не вызвался
наблюдатель снят
```

Два изменения через сеттер дали два уведомления со старым и новым значением. Прямая запись в `ivar` прошла молча, хотя `balance` стал 999. В конце — `removeObserver:.`

Задача 14. Реализуй NSCopying

Условие. Сделай свой класс копируемым: реализуй `copyWithZone:` так, чтобы `[obj copy]` возвращал **независимую** копию (меняем копию — оригинал не дрогнул).

Подумай сам. Что должен вернуть `copyWithZone:` и как перенести в копию все поля, включая строковое?

Разбор. Чтобы объект отвечал на `copy`, класс объявляет протокол `<NSCopying>` и реализует единственный его метод — `copyWithZone:`. Сам `copy` без аргументов — это удобная обёртка из `NSObject`, которая зовёт `copyWithZone:nil`. Реализация создаёт новый объект и переносит в него состояние:

```
- (id)copyWithZone:(NSZone *)zone {
    Vec2 *copy = [[[self class] allocWithZone:zone] init];
    copy.x = self.x;
    copy.y = self.y;
    copy.label = self.label;    // свойство copy само снимет снимок строки
    return copy;
}
```

Тонкости, которые проверяют на собеседовании:

- `[[self class] allocWithZone:zone]`, а не `[[Vec2 alloc] ...]` — чтобы у **подкласса** копия получалась правильного класса.
- Строковое поле `label` объявлено свойством `copy`, поэтому присваивание `copy.label = self.label` само делает снимок — глубоко копировать строку вручную не нужно.
- Числа `x`, `y` — значения, копируются присваиванием.

Решение: `code/middle-11-nsCopying.m`.

```
оригинал: (1, 2) A
копия:    (100, 2) B
разные объекты? да
```

Изменили копии `x` и `label` — оригинал остался (1, 2) A. Объекты разные: `copy` вернул новый экземпляр, а не ту же ссылку.

Задача 15. Реализуй isEqual: и hash вместе

Условие. Реализуй сравнение объектов по значению: два «денежных» объекта равны, если равны их суммы. Что будет, если переопределить isEqual:, но **забыть** hash? Проверь поведение в NSSet.

Подумай сам. Зачем коллекциям-множествам нужен hash, если есть isEqual:?

Разбор. Контракт NSObject жёсткий: **если два объекта равны по isEqual:, у них обязан совпадать hash.** NSSet и NSDictionary сначала раскладывают объекты по «корзинам» по их hash и только внутри корзины сравнивают через isEqual:. Если переопределить isEqual:, но не hash, два «равных» объекта унаследуют дефолтный hash от NSObject (он основан на адресе) — у каждого свой. Они попадут в **разные корзины**, и множество их сравнивать не станет: оба окажутся внутри, а поиск по «равному» объекту провалится.

Поэтому переопределяют **обе** вещи согласованно:

```
- (BOOL)isEqual:(id)other {
    if (self == other) return YES;
    if (![other isKindOfClass:[Money class]]) return NO;
    return self.amount == ((Money *)other).amount;
}

- (NSUInteger)hash { return (NSUInteger)self.amount; }
```

hash должен зависеть **ровно от тех полей**, что участвуют в isEqual:. Для нескольких полей их хэши обычно комбинируют через XOR ($a \oplus b$) — это простейший вариант. У XOR есть слабость: он симметричен ($a \oplus b == b \oplus a$), и равные поля гасят друг друга ($a \oplus a == 0$), поэтому часто пишут $31 * a + b$. Главное — равным объектам одинаковый hash.

Решение: code/middle-12-isequal-hash.m.

```
=== сломанный (hash не согласован) ===
в множестве объектов: 2 (ждали 1)
containsObject(100)? НЕТ
=== правильный (isEqual + hash) ===
в множестве объектов: 1 (ждали 1)
containsObject(100)? да
```

Без согласованного hash множество приняло два «равных» объекта как разные (count = 2) и не нашло объект по значению. С правильным hash — дубликат отброшен (count = 1), поиск работает.

Задача 16. isEqual: против isEqualToString:

Условие. В чём разница между [a isEqual:b] и [a isEqualToString:b] для двух NSString? Что быстрее и что безопаснее?

Подумай сам. Какой из методов знает заранее, что оба аргумента — строки?

Разбор. Оба сравнивают строки по содержимому (не по адресу — для адреса был бы ==, и для строк он почти всегда неверен). Разница в типе и цене:

- `isEqual:` — общий метод `NSObject`. Он принимает `id` (любой объект), поэтому сначала **проверяет класс** аргумента: а строка ли это вообще? Только потом сравнивает содержимое. Универсален, чуть медленнее, безопасен к чужим типам — [str isEqual:someArray] спокойно вернёт NO.
- `isEqualToString:` — специализированный метод `NSString`. Он уже **предполагает**, что аргумент — строка, пропускает проверку класса и сравнивает содержимое напрямую. Быстрее на горячем пути.

Ловушка `isEqualToString::` если передать в него **не строку** (например, `nil` пройдёт молча и вернёт NO, а вот объект другого класса в некоторых случаях — неопределённое поведение), гарантий меньше. Практическое правило: когда **точно знаешь**, что оба — `NSString`, бери `isEqualToString:` (читаемее и быстрее). Когда тип под вопросом или сравниваешь разнородные объекты — `isEqual:`. И никогда не сравнивай строки через `==`: он сверяет указатели, а не текст, и «совпадёт» лишь случайно (на одинаковых литералах или на коротких строках, упакованных в tagged pointer, — см. задачу 3 раздела D).

Задача 17. Быстрый разбор: работает ли dealloc?

Условие. Для каждого мини-сценария скажи, работает ли `dealloc` объекта А к концу блока. Все свойства — под ARC.

1. А создан локально, в конце блока локальная ссылка выходит из области.
2. А сильно держит В, В сильно держит А.
3. А сильно держит В, В держит А через weak-свойство.
4. А хранит в свойстве сору-блок, блок упоминает `self.name`.
5. А хранит блок, блок упоминает `weakSelf.name`.

Подумай сам. В каждом случае посчитай, остаётся ли на А хоть одна сильная ссылка после выхода из блока.

Разбор.

1. **Сработает.** Единственная сильная ссылка вышла из области — счётчик 0, `dealloc`.
2. **Не сработает.** Классический цикл: каждый держит другого сильно, счётчики застряли на 1. Утечка.
3. **Сработает у обоих.** `weak` не считается за владение: когда исчезли локальные ссылки, счётчик A дошёл до нуля — `dealloc` у A, а он отпускает B, и тот освобождается следом.
4. **Не сработает.** Блок захватил `self` сильно, `self` держит блок — цикл «объект ↔ блок».
5. **Сработает.** Блок захватил `weakSelf`, сильного кольца нет.

Это сжатый обзор всего, что было выше: единственный вопрос на собеседовании по памяти — «остаётся ли на объект сильная ссылка». Если да и она по кругу ведёт к самому объекту — это цикл, `dealloc` молчит, и одну ссылку в кольце надо сделать `weak`.

Задача 18. Code review: найди и почини утечку в блоке

Условие. Тебе прислали на ревью этот метод. Объект `Timer` устанавливает повторяющийся обработчик, который пишет в свой `log`. Найди проблему и предложи правку.

```
@interface Timer : NSObject
@property (nonatomic, copy) void (^onTick)(void);
@property (nonatomic, strong) NSMutableArray *log;
@end

@implementation Timer
- (void)start {
    self.onTick = ^{
        [self.log addObject:@"tick"];    // ← что не так?
    };
}
- (void)dealloc { NSLog(@"dealloc Timer"); }
@end
```

Подумай сам. Что захватывает блок в строке `[self.log addObject:...]`?

Разбор. Блок упоминает `self.log` — а значит захватывает **self сильно**. Свойство `onTick` объявлено `copy` и сильно держит блок. Кольцо `self` → `onTick` → `self` замкнулось: `Timer` никогда не вызовет `dealloc`, сообщение «`dealloc Timer`» не появится. Это утечка, и компилятор честно предупредит про `-Warc-retain-cycles`.

Правка — слабая копия `self`:

```
- (void)start {
    __weak typeof(self) weakSelf = self;
    self.onTick = ^{
        [weakSelf.log addObject:@"tick"];
    };
}
```

Теперь блок захватывает `weakSelf`, сильного кольца нет, и когда внешний код отпустит `Timer`, тот спокойно уничтожится — `dealloc Timer` напечатается. Если внутри блока к `self` обращаются несколько раз и важно, чтобы он не исчез посреди работы блока, в начале блока берут сильную копию на время вызова:

```
self.onTick = ^{
    __strong typeof(weakSelf) strongSelf = weakSelf;
    if (strongSelf == nil) { return; } // объект уже умер — выходим
    [strongSelf.log addObject:@"tick"];
};
```

Это «weak-strong dance»: слабая ссылка рвёт цикл, а локальная сильная на время выполнения блока гарантирует, что объект не исчезнет между двумя строками. На собеседовании достаточно назвать причину (захват `self`) и базовую правку (`weakSelf`); упоминание `strongSelf` — плюс к ответу.

Что проверяет этот раздел

Middle-уровень — это уже не «знаю синтаксис», а «понимаю модель». Сведём ключевое в одну картину:

- **Память.** Сильная ссылка владеет, слабая наблюдает и сама становится `nil`, `unsafe_unretained` повисает. Цикл удержания — два владельца по кругу; лечится одним `weak`. ARC циклы не ловит — это на тебе.
- **Блоки.** Захват — копия значения в момент создания; `__block` — общая ячейка; упоминание `self` в блоке свойства — цикл, режется `weakSelf`.
- **Копирование.** `copy/mutableCopy` поверхностны; строковые свойства — `copy`; свой класс делаешь копируемым через `NSCopying`.
- **Контракты.** Равенство — это `isEqual:` плюс согласованный `hash`; словарь копирует ключи; делегат — `weak` и `respondsToSelector:` для `@optional`.

- **Runtime-приёмы.** Категория добавляет методы, но не ivar (выручают связанные объекты); две категории с одним методом — неопределённость; KVC даёт доступ по строке, KVO стоит на сеттере и требует `removeObserver:`.

Если на собеседовании ты на каждый такой вопрос отвечаешь не «по памяти», а выводом из модели владения и контрактов классов — это и есть Middle.

Документация Apple

- Programming with Objective-C → **Working with Blocks** — захват переменных, `__block`, циклы удержания с `self` — [developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ ProgrammingWithObjectiveC/WorkingwithBlocks/WorkingwithBlocks.html](http://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/WorkingwithBlocks/WorkingwithBlocks.html)
- **Transitioning to ARC Release Notes** — `strong/weak/ unsafe_unretained`, захват `self` в блоках, разрыв циклов — developer.apple.com/library/archive/releasenotes/ObjectiveC/RN-TransitioningToARC/Introduction/Introduction.html
- **Advanced Memory Management Programming Guide** — модель владения, `autorelease`, пулы автоосвобождения — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ MemoryMgmt/Articles/MemoryMgmt.html
- **Key-Value Coding Programming Guide** — `valueForKey:`, `valueForKeyPath:`, коллекционные операторы `@sum/@avg/@max` — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ KeyValueCoding/index.html
- **Key-Value Observing Programming Guide** — подписка, словарь `change`, ручные и автоматические уведомления — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ KeyValueObserving/KeyValueObserving.html
- `NSCopying` и `-[NSObject copy]` — developer.apple.com/documentation/foundation/nscopying
- `-[NSObject isEqual:]` и `-[NSObject hash]` (контракт равенства) — developer.apple.com/documentation/objectivec/nsobject
- `-[NSString isEqualToString:]` — developer.apple.com/documentation/foundation/nsstring
- **Associated Objects** — `objc_setAssociatedObject` / `objc_getAssociatedObject` — developer.apple.com/documentation/objectivec
- `NSCache` — developer.apple.com/documentation/foundation/nscache

Приложение F. Задачи

собеседования: Senior

Junior проверял механику, Middle — модель владения и контракты классов. На Senior спрашивают про то, что крутится **под капотом**: как устроены isa и метаклассы, как objc_msgSend ищет метод, три шанса пересылки, swizzling, связанные объекты, KVO изнутри, и отдельно — многопоточность: гонки, синглтоны, барьеры, дедлоки, пулы автоосвобождения в фоне. Здесь почти нет ответов в одну строку: за каждым вопросом стоит runtime или модель потоков.

Формат прежний. Сначала **условие**. Если задача про «что выведет» — остановись на **подумай сам** и ответь, прежде чем читать **разбор**. Где есть рабочий код, он лежит в code/senior-NN-имя.m, собран и запущен, и весь показанный вывод — настоящий. Дату, имя программы и число в скобках, которые NSLog печатает в начале строки, здесь убрали — у тебя они будут свои.

Собрать и запустить любой файл:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/senior-01-metaclass.m -o /tmp/t && /tmp/t
```

Часть задач — концептуальные, без файла в code/: они или уронили бы программу (дедлок, сломанный swizzling), или их результат не определён (две категории), или это разбор словами (исключения, пулы в фоне). Такие помечены прямо в тексте.

Идём от runtime к потокам и краевым случаям, от сложного к очень сложному.

Задача 1. Куда указывает isa класса и где живут методы класса

Условие. Есть класс Tool с методом экземпляра -use и методом класса +build. Ответь, не запуская:

- куда указывает isa самого класса Tool;
- в чьей таблице методов лежит -use, а в чьей +build;

- куда в итоге приходит цепочка `isa`, если идти объект → класс → метакласс →

Подумай сам. Если объект знает свой класс через `isa`, то что знает через свой `isa` сам класс?

Разбор. Объект — это структура, чьё первое поле `isa` указывает на класс (глава 14). Но **класс — тоже объект**, и у него тоже есть `isa`. Указывает он на **метакласс** (`metaclass`) — «класс класса».

Зачем метакласс вообще нужен. Методы экземпляра (с минусом, `-use`) лежат в таблице методов **класса**: шлешь сообщение объекту — `runtime` идёт по его `isa` в класс и ищет там. А методы класса (с плюсом, `+build`) надо где-то держать по той же схеме. Их кладут в **метакласс**. Когда шлешь сообщение классу (`[Tool build]`), `runtime` идёт по `isa` класса в метакласс и ищет `+build` там. Одна и та же машина поиска работает на двух уровнях.

Поэтому:

- `-use` лежит в таблице **класса** `Tool`, в метаклассе его нет;
- `+build` лежит в таблице **метакласса** `Tool`, в самом классе его нет.

Цепочка `isa` из примера: объект `t` → класс `Tool` → метакласс `Tool` → **корневой метакласс** (метакласс `NSObject`), а его `isa` указывает сам на себя — цепочка замыкается. Заметь: `isa` любого метакласса ведёт **сразу** в корневой метакласс, минуя метаклассы родителей. По родителям вверх идёт другой указатель — `superclass` (метакласс `Tool` → метакласс `NSObject` → класс `NSObject`). В выводе последняя строка — как раз этот прыжок: `isa` метакласса `Tool` ведёт в метакласс `NSObject`, чьё имя — `NSObject`.

Решение — `code/senior-01-metaclass.m`:

```
isa класса Tool указывает на: Tool (метакласс? да)
-use   в классе Tool?      да
-use   в метаклассе Tool?  нет
+build в классе Tool?      нет
+build в метаклассе Tool?  да
цепочка isa:
  объект t      isa -> Tool
  класс Tool    isa -> Tool
  метакласс Tool isa -> NSObject
```

Заметь тонкость вывода: у метакласса **то же имя**, что у класса, — `Tool`. `class_getName` для метакласса возвращает имя класса. Отличить их можно только через `class_isMetaClass` (в первой строке — «метакласс? да»). А `-use` нашёлся только в классе, `+build` — только в метаклассе: ровно там, где `runtime` их и держит.

Отличие от Си. В Си у типа нет «класса своего типа» — после компиляции от `struct Tool` не остаётся ничего, спрашивать не у кого. Здесь же у класса есть свой класс

(метакласс), а у того — свой (корневой метакласс): целая живая лесенка `isa` там, где в Си пустота.

Задача 2. `objc_msgSend`, `SEL`, `IMP`, `Method` — кто есть кто

Условие. У класса `Calc` есть метод `-(int)addTo:(int)x`, который возвращает `x + 100`. Объясни, во что компилятор превращает `[c addTo:5]`, и что напечатают три вызова — обычная отправка, вызов через `IMP` напрямую и вызов через `objc_msgSend`, приведённый к точной сигнатуре.

```
int viaMessage = [c addTo:5]; // (1)
int (*impFn)(id, SEL, int) = (void *)impFromMethod;
int viaImp = impFn(c, sel, 5); // (2)
int (*send)(id, SEL, int) = (void *)objc_msgSend;
int viaSend = send(c, sel, 5); // (3)
```

Подумай сам. Чем `SEL` отличается от `IMP`, а `Method` — от них обоих?

Разбор. Разложи отставку сообщения на детали (глава 14):

- **SEL (селектор)** — уникализированное имя сообщения. Не строка, а её зарегистрированный в runtime «номерок»: для `"addTo:"` во всей программе существует ровно один `SEL`, поэтому селекторы сравнивают через `==`.
- **IMP (implementation)** — указатель на си-функцию, которая и есть тело метода. Её первые два скрытых аргумента — `self` и `_cmd` (селектор), дальше параметры.
- **Method** — пара (`SEL`, `IMP`) плюс кодировка типов. Класс хранит таблицу таких пар.
- **`objc_msgSend(receiver, sel, ...)`** — функция, которая по `isa` получателя находит нужную `IMP` и передаёт ей управление.

`[c addTo:5]` компилятор превращает в `objc_msgSend(c, @selector(addTo:), 5)`. Все три пути в задаче ведут к одной и той же `IMP` и дают один и тот же результат — `105`. Разница лишь в том, **на каком уровне** тыходишь: скобки прячут `objc_msgSend`; `impFn` зовёт реализацию напрямую, минуя поиск; `send` — это сам `objc_msgSend`, приведённый к сигнатуре метода. Без приведения её не вызвать вообще: в современном SDK она объявлена как `void objc_msgSend(void)`, и вызов с аргументами — ошибка компиляции, а не предупреждение.

Кодировка типов `i20@0:8i16` (вывод с `x86_64/arm64`) читается так: возвращает `int (i)`; дальше аргументы — `@(self), : (_cmd), i` (наш `int x`); числа — смещения и размеры, их можно игнорировать. Аргументов «3» — это `self`, `_cmd` и `x`: два всегда скрытые.

Решение — `code/senior-02-msgsend.m`:

```

SEL как строка: addTo:
Method найден: да
кодировка типов метода: i20@0:8i16
число аргументов (с учётом self и _cmd): 3
IMP из Method и из класса совпали? да
[c addTo:5]          = 105
через IMP           = 105
через objc_msgSend = 105

```

IMP из Method (через `method_getImplementation`) и IMP из класса (через `class_getMethodImplementation`) совпали — это буквально один адрес кода. Все три вызова дали 105: квадратные скобки — лишь удобный синтаксис над `objc_msgSend`.

Отличие от Си. В Си `addTo(c, 5)` — переход по **фиксированному адресу**, выбранному компоновщиком на этапе сборки. В Objective-C тот же вызов — это поиск IMP по SEL **во время выполнения**, от `isa` получателя вверх. Один и тот же `[obj addTo: 5]` может выполнить разный код в зависимости от того, какой объект сейчас в `obj`.

Задача 3. Динамическое разрешение метода: сколько раз позовётся resolve

Условие. Класс `Ghost` не реализует ни `shout`, ни `twice`. Их выдаёт `+resolveInstanceMethod:` через `class_addMethod` при первом обращении. Сколько раз вызовется `+resolveInstanceMethod:`, если послать `shout` дважды и `twice`: один раз?

Подумай сам. Когда метод уже доращён в класс, нужен ли `resolve` при следующем таком же сообщении?

Разбор. Когда `objc_msgSend` не находит метод обычным поиском (по классу и вверх по `superclass`), он даёт классу первый шанс — зовёт `+resolveInstanceMethod:(SEL)sel:` «может, доопределишь?» Если ты внутри вызвал `class_addMethod` и вернул `YES`, runtime повторяет поиск — теперь метод на месте.

Ключевое: `class_addMethod` встраивает метод **в класс навсегда**. Поэтому `resolve` срабатывает **ровно один раз на селектор** — при самом первом сообщении. Второй `shout` идёт уже по готовой таблице, минуя разрешение.

Считаем вызовы `+resolveInstanceMethod::`

- первый `shout` → метода нет → `resolve` (раз), доращиваем, выполняем;

- второй shout → метод уже есть → resolve **не** зовётся;
- twice: → метода нет → resolve (два), доращиваем, выполняем.

Итого — **2 раза**. Кодировки типов в `class_addMethod`: "`@@:`" — вернёт объект (`@`), принимает `self` (`@`) и `_cmd` (`:`); "`@@:@`" — то же плюс ещё один объектный аргумент.

Решение — `code/senior-03-resolve.m`:

```
первый вызов shout:
[resolve] доопределили shout
Ghost кричит! (селектор был shout)
второй вызов shout (resolve уже не нужен):
Ghost кричит! (селектор был shout)
вызов twice:21 ->
[resolve] доопределили twice:
42
resolveInstanceMethod: позвался 2 раз(a)
```

[`resolve`] напечаталось дважды — по разу на каждый новый селектор. Между двумя `shout` его нет: класс буквально вырос на метод и больше в разрешении не нуждается.

Задача 4. Логирующий прокси на `forwardingTargetForSelector:`

Условие. Сделай прокси, который сам не реализует `name` и `compute:`, а пересылает их «настоящему» объекту `Service` и попутно считает каждую пересылку. Какой из трёх шансов пересылки тут самый дешёвый и почему?

Подумай сам. Что должен вернуть `forwardingTargetForSelector:`, чтобы runtime просто повторил сообщение другому объекту?

Разбор. Когда первый шанс (`resolveInstanceMethod:`) не сработал, runtime даёт **второй**: `-forwardingTargetForSelector:(SEL)sel`. Ты возвращаешь **другой объект**, и runtime повторяет отправку уже ему — буквально как `objc_msgSend(другойОбъект, sel, ...)`. Это самый дешёвый шанс: ни копирования аргументов, ни построения `NSInvocation`, сообщение просто едет к новому получателю. Тип возврата и аргументы сохраняются бесплатно.

На этом строят прокси: объект-обёртка хранит `_target` и в `forwardingTargetForSelector:` возвращает его, если тот умеет селектор. Заодно можно посчитать и залогировать каждую пересылку — то, ради чего обёртку часто и заводят (аналитика, трассировка).

Тонкость: `[proxy respondsToSelector:@selector(name)]` вернул бы **нет** — прокси и правда не реализует `name`, он его пересылает. Пересылка работает уровнем ниже, уже после того как обычный поиск метода провалился.

Решение — `code/senior-05-logging-proxy.m`:

```
[proxy] переслано: name (всего 1)
name      = Service
[proxy] переслано: compute: (всего 2)
compute = 81
[proxy] переслано: compute: (всего 3)
compute = 16
итого переслано сообщений: 3
```

Снаружи проху ведёт себя как `Service`: вернул имя и посчитал квадраты ($9^2 = 81$, $4^2 = 16$). Внутри — три залогированные пересылки. Один объект прозрачно подменил другой, добавив к нему лог.

Задача 5. Рассылающий прокси: почему тут нужен `forward Invocation`:

Условие. Нужен прокси, который **одно** сообщение `event:` доставляет **сразу нескольким** адресатам. Почему `forwardingTargetForSelector:` тут не годится и какой шанс пересылки придётся взять?

Подумай сам. Сколько объектов умеет вернуть `forwardingTargetForSelector:`?

Разбор. `forwardingTargetForSelector:` умеет вернуть **ровно одного** получателя — runtime повторит сообщение ему, и на этом всё. Доставить одно сообщение нескольким он не может в принципе.

Здесь нужен **третий шанс** — самый мощный и самый дорогой:

1. `-methodSignatureForSelector:` — runtime просит описание сигнатуры метода, чтобы понять, как упаковать аргументы. Берём сигнатуру у первого адресата, который знает селектор.
2. `-forwardInvocation:(NSInvocation *)inv` — runtime упаковывает всё сообщение целиком в объект `NSInvocation`: получатель, селектор, все аргументы. С ним можно делать что угодно. Мы в цикле зовём `[inv invokeWithTarget:t]` для **каждого** адресата — сообщение выполняется на всех по очереди.

NSInvocation строить дороже, чем просто повторить отправку, но только он даёт полный контроль: переслать многим, изменить аргументы, отложить, залогировать. Тип переменной — `id<Listener>`: компилятор разрешает слать `event:`, хотя `Broadcaster` его не реализует (динамика главы 5).

Решение — `code/senior-04-broadcast-proxy.m`:

шлём ОДНО сообщение прокси:

```
Анна получает событие: пожарная тревога
Борис получает событие: пожарная тревога
Вера получает событие: пожарная тревога
```

Одно `[group event:@"пожарная тревога"]` превратилось в три доставки. `forwardingTargetForSelector:` так не смог бы — он вернул бы лишь одного слушателя.

Задача 6. Ленивый прокси на `NSProxy`: зачем отдельный корень

Условие. «Тяжёлый» объект `Heavy` дорого создавать, и мы хотим отложить его создание до первого реального обращения. Сделай прокси-заместитель. Почему для этого берут `NSProxy`, а не `NSObject`?

Подумай сам. Если унаследовать прокси от `NSObject`, какие сообщения он обработает **сам**, не переслав их `Heavy`?

Разбор. В Objective-C два корня иерархии (глава 14). Основной — `NSObject`: он тащит гору готового поведения (`description`, `hash`, `isEqual:`, `respondsToSelector:`, десятки `NSObject`-методов). Второй, отдельный — `NSProxy`: тонкий абстрактный класс специально для заместителей. Он почти ничего не реализует сам и заставляет наследника определить пересылку (`methodSignatureForSelector: + forwardInvocation:`).

В этом и причина выбора. Прокси должен пересылать `Heavy` **всё**. Унаследуй он `NSObject` — сотни сообщений из `NSObject` и его категорий (`isKindOfClass:`, `respondsToSelector:`, `conformsToProtocol:`, `valueForKey:`, `performSelector:...`) прокси обработал бы **сам**, своим унаследованным кодом, и ответил бы про себя, а не про `Heavy`. У `NSProxy` своих ответов — горстка: по нашему эксперименту сам он отвечает на `class`, `description`, `hash`, `isEqual:` (плюс управление памятью), а `isKindOfClass:` и `respondsToSelector:` уже уходят в пересылку и честно отвечают за `Heavy`. Мимо пересылки проходит минимум.

Механика: `LazyProxy` хранит `_real` (поначалу `nil`). Метод `real` создаёт `Heavy` при первом обращении (лениво). `forwardInvocation:` зовёт `[inv invokeWithTarget:[self real]]` — и именно тут,

на первом реальном сообщении, Heavy наконец рождается. Создаётся прокси через [LazyProxy alloc] без init: yNSProxy своя схема.

Решение — code/senior-13-nsproxy.m:

```
создаём прокси (Heavy ещё НЕ создан):  
прокси готов, Heavy всё ещё не создан  
первое реальное обращение -> report:  
    Heavy: дорогой объект СОЗДАН  
ответ: тяжёлый отчёт готов
```

Прокси создан, а «Heavy СОЗДАН» появилось **только** на report — создание отложилось ровно до первого дела. Будь прокси на NSObject, часть служебных сообщений он перехватил бы сам, и прозрачной замены не вышло бы.

Задача 7. Method swizzling правильно: +load, dispatch_once, class_addMethod

Условие. Нужно подменить метод -tap класса Button так, чтобы перед оригиналом срабатывала аналитика. Где безопасно делать подмену, зачем тут dispatch_once, и почему «правильный» swizzle сначала пробует class_addMethod, а не сразу method_exchangeImplementations?

Подумай сам. Что будет, если метод tap достался классу **по наследству**, а ты обменяешь его реализацию через exchange?

Разбор. Swizzling — обмен реализациями двух методов (глава 14). Делать его надо аккуратно, иначе словишь трудноуловимый баг. Три правила из рабочего решения:

1. **Где.** Подмену кладут в +load категории. +load runtime зовёт один раз при загрузке образа, ещё до main, и потокобезопасно. Это надёжный момент: класс уже есть, но программа ещё не работает.
2. **dispatch_once.** Дополнительный пояс поверх подтяжек: гарантирует, что подмена выполнится строго один раз, даже если код как-то позовут повторно. Двойной swizzle вернул бы метод в исходное состояние — dispatch_once это исключает.
3. **Сначала class_addMethod.** Если tap достался Button **по наследству** (определён в родителе), голый method_exchangeImplementations полез бы менять реализацию **в родителе** — а это ломает поведение для всех остальных потомков. Поэтому сперва пробуем class_addMethod: она добавит метод **в наш класс** и вернёт YES, если метода тут ещё не было. Тогда обмен делаем уже внутри своего класса. И только если метод определён прямо в Button (class_addMethod вернула NO) — честный exchange.

Тело `analytics_tap` после обмена в конце зовёт `[self analytics_tap]` — на вид рекурсия, но это не она: после обмена имя `analytics_tap` указывает на **старый** код `tap`. Классический приём «обёртки».

Решение — `code/senior-06-swizzle.m`:

нажимаем кнопку:

```
[analytics] зафиксировали нажатие
Button: обычное нажатие
```

`[b tap]` теперь сначала логирует, потом выполняет исходное нажатие — мы обернули чужой метод, не трогая его исходник. Подмена глобальна: касается всех экземпляров `Button` в программе.

Задача 8. Найди тонкий баг: swizzling, который ломает KVO

(Концептуальная, без файла: сценарий из условия роняет программу. Мы воспроизвели его экспериментом.)

Условие. В проекте есть помощник, который вешает логирование на сеттер `setName:`. Чтобы «попасть в настоящий класс объекта», он берёт класс через `object_getClass(obj)` и свиззлит его по схеме задачи 7 при первом вызове. Работало месяцами. Потом в одном экране на тот же `Person` повесили KVO — раньше, чем сработал помощник. Теперь первое же `p.name = ...` роняет программу с `EXC_BAD_ACCESS` где-то внутри Foundation. Где баг?

Подумай сам. Что вернёт `object_getClass(obj)` для объекта, на который уже подписан KVO, — и чей сеттер тогда окажется в обмене?

Разбор. Вспомни, как устроен KVO (глава 20, задача 10 ниже). В момент `addObserver:` Foundation создаёт **динамический подкласс** (`NSKVONotifying_Person`), кладёт в него свой `setName:` и **подменяет isa** объекта на этот подкласс. `-class` при этом врёт, что это по-прежнему `Person`, а `object_getClass` говорит правду.

Теперь пройди по шагам помощника:

1. `object_getClass(p)` вернул **не** `Person`, а `NSKVONotifying_Person`. Свиззлим чужой, сгенерированный Foundation класс.
2. `class_addMethod(..., @selector(setName:), ...)` вернул `N0`: в KVO-подклассе `setName:` уже есть. Значит, идём в `method_exchangeImplementations`.
3. А `log_setName:` в KVO-подклассе **нет** — `class_getInstanceMethod` нашёл его у родителя `Person`. Обмен переписал метод **родителя**: теперь у всех `Person` под именем `log_setName:`

сидит KVO-сеттер. Это ровно та ошибка, от которой защищает `class_addMethod` в задаче 7, только с другой стороны.

4. KVO-сеттер — общая функция на все свойства. Какой ключ менять и чью реализацию звать дальше, он, судя по поведению, выводит из селектора `_cmd`. После обмена его зовут под чужим именем `log_setName:` — и его логика рушится. В нашем эксперименте это падение с `EXC_BAD_ACCESS` внутри `-[NSObject(NSKeyValueObservingPrivate) _changeValueForKey:...]`.

Отдельно — «плавающий» хвост. Пока KVO не подписан, `object_getClass` возвращает `Person`, и помощник подменяет сеттер у `Person`. Подписали KVO после этого — всё работает: KVO-сеттер в момент вызова зовёт `setName:` родителя и попадает в подмену (лог и уведомление — оба есть, мы проверили). Поэтому баг зависит от порядка: кто успел первым — KVO или помощник.

Как чинить:

- Свиззли **только свой класс, взятый статически** (`[Person class]` или `self` в `+load` категории), а не класс конкретного экземпляра. Под `object_getClass(obj)` может оказаться что угодно: KVO-подкласс, класс-кластер, прокси.
- Делай это **один раз в `+load`** с `dispatch_once` (задача 7) — до того, как кто-либо успеет подписаться на KVO.
- Не свиззли сеттеры, за которыми наблюдают через KVO, без крайней нужды. Хочешь лог изменений — подпишись на тот же KVO.

Короткий ответ на собеседовании: **`object_getClass` у наблюдаемого объекта — это `NSKVONotifying_...`; свиззлинг его сеттера подсовывает KVO-функции чужой `_cmd` и заодно портит метод родителя. Свиззли только свой класс, статически, в `+load`, один раз.**

Задача 9. Связанный объект: хранимое свойство в категории

Условие. `View` — будто бы чужой класс из фреймворка: исходник недоступен, `ivar` в него не добавить. Нужно прицепить к нему счётчик `badgeCount`. Категория хранить поля не умеет. Как сохранить значение **на объекте**, и почему у двух разных `View` оно своё?

Подумай сам. Если в объекте нет места под новый `ivar`, где `runtime` держит привязанное к нему значение?

Разбор. Категория добавляет **методы**, но не **`ivar`**: размер объекта фиксируется при компиляции класса. Объявить в категории `@property` можно, но автоматического `ivar` под него не будет — только обещание геттера и сеттера, которые ты обязан написать сам. А значение хранят через **связанные объекты** (`associated objects`) из `runtime`:

```
static const void *kBadgeKey = &kBadgeKey;    // адрес = уникальный ключ

- (void)setBadgeCount:(NSNumber *)v {
    objc_setAssociatedObject(self, kBadgeKey, v,
                             OBJC_ASSOCIATION_RETAIN_NONATOMIC);
}
- (NSNumber *)badgeCount {
    return objc_getAssociatedObject(self, kBadgeKey);
}
```

objc_setAssociatedObject привязывает значение **к конкретному объекту** по ключу. Ключом служит адрес статической переменной (&kBadgeKey) — он гарантированно уникален во всей программе. Привязка живёт у каждого экземпляра своя: runtime держит её в отдельной таблице, индексированной по объекту, а не в самом объекте. Поэтому a.badgeCount и b.badgeCount независимы. И следить за освобождением не нужно: связанные объекты runtime снимает сам, когда носитель умирает.

Политика OBJC_ASSOCIATION_RETAIN_NONATOMIC означает «держи сильно, без атомарности» — аналог strong, nonatomic у свойства (для строк взяли бы COPY_NONATOMIC).

Решение — code/senior-07-associated.m:

```
до: Почта badge=(null), Чаты badge=(null)
после: Почта badge=7, Чаты badge=0
у каждого объекта своё значение? да
```

До присваивания связанного значения нет — геттер возвращает nil ((null)). После — у «Почты» 7, у «Чатов» 0: значение у каждого объекта своё, хотя ivar мы не добавляли и класс не трогали.

Отличие от Си. В Си, чтобы прицепить данные к чужой структуре, пришлось бы менять её определение и пересобирать всех, кто её использует. Здесь runtime хранит привязку в стороне, по адресу объекта, — чужой класс можно «дорастить» данными, не имея его источника.

Задача 10. KVO под капотом: подмена isa и NSObjectifying_

Условие. Подписались через KVO на свойство balance объекта Account. Что произойдёт с **классом** объекта? Что покажет -class, а что — реальный isa (через object_getClass) до подписки, после неё и после отписки?

Подумай сам. Как одно лишь `addObserver:` заставляет обычный сеттер рассылать уведомления, не трогая исходный код `Account`?

Разбор. KVO стоит **на сеттере**, и реализован он через подмену `isa` (глава 20). В момент `addObserver:forKeyPath: runtime:`

1. генерирует **динамический подкласс** `NSKVONotifying_Account`;
2. переопределяет в нём `setBalance:` так, чтобы вокруг настоящего присваивания слались `willChangeValueForKey: / didChangeValueForKey:`;
3. **подменяет `isa`** объекта асс на этот новый подкласс.

С этого момента асс физически — экземпляр `NSKVONotifying_Account`, и его сеттер уведомляет наблюдателей. Но `-class` специально **врёт**, что это по-прежнему `Account`: KVO маскирует подмену, чтобы не пугать твой код. А настоящий `isa`, прочитанный напрямую через `object_getClass`, выдаёт правду — `NSKVONotifying_Account`. После `removeObserver: runtime` возвращает `isa` на место — снова `Account`.

Отсюда два важных факта (оба видны в выводе):

- Уведомление приходит только при изменении **через сеттер** (или через KVC `setValue:forKey:`, который его зовёт) — потому что перехвачен именно сеттер. Прямая запись в `ivar` минует его, и KVO молчит.
- В конце обязателен `removeObserver::addObserver:` не удерживает наблюдателя, и если тот умрёт раньше, следующее уведомление уйдёт по висячему указателю и уронит программу.

Решение — `code/senior-08-kvo-isa.m`:

```
до подписки:  -class=Account  настоящий isa=Account
после подписки: -class=Account  настоящий isa=NSKVONotifying_Account
меняем баланс через сеттер:
[наблюдатель] balance: 0 -> 100
[наблюдатель] balance: 100 -> 250
после отписки:  настоящий isa=Account
```

Вот доказательство «isa-swizzling»: `-class` всё время говорит `Account`, а настоящий `isa` после подписки стал `NSKVONotifying_Account` и вернулся обратно после отписки. Два изменения через сеттер дали два уведомления со старым и новым значением. Это та же машинерия, что в задаче 8, — вот почему swizzling наблюдаемых сеттеров и опасен.

Задача 11. Две категории с одинаковым методом — какая победит

(Концептуальная, без файла — результат не определён, запускать бессмысленно.)

Условие. На один класс навесили две категории, и в обеих объявлен метод `tagline` с **разной** реализацией. Какая из них вызовется?

```
@implementation NSString (A)
- (NSString *)tagline { return @"из A"; }
@end
@implementation NSString (B)
- (NSString *)tagline { return @"из B"; }
@end
```

Подумай сам. Гарантирует ли язык, какая реализация ляжет в таблицу методов класса?

Разбор. Это **неопределённое поведение**. Когда две категории одного класса определяют метод с одним именем, какая из них «победит» — **не специфицировано**.

Почему так, на уровне runtime. При загрузке образа runtime берёт методы категорий и **дописывает** их в таблицу методов класса. Методы категории кладутся так, что при поиске находятся **раньше** родных методов класса, — поэтому категория и «перебивает» оригинал. А вот порядок между **двумя категориями** определяется порядком, в котором runtime их подгрузил, то есть порядком линковки объектных файлов. По нашим наблюдениям обычно «побеждает» категория, загруженная **последней** (её метод оказывается в начале списка и находится первым), — но опираться на это нельзя: поменяется порядок файлов в сборке, версия компилятора или тулчейна — и результат сменится. Поэтому «эксперимент» тут ничего не доказывает: у тебя выйдет из A, а у коллеги с другим порядком файлов в проекте — из B.

Хуже того — категория, переопределяющая **родной** метод класса (не другой категории, а сам метод `NSString`), молча затирает оригинал, и вызвать прежнюю реализацию через `super` уже нельзя: у категории нет «родителя» с этим методом.

Правила, которые ждут в ответе:

- **Категория — чтобы добавлять методы, а не подменять.** Хочешь изменить поведение — делай подкласс или swizzling (осознанно, см. задачи 7–8).
- **Префиксируй имена методов в категориях** (`abc_tagline`), особенно на классах Foundation, — чтобы не столкнуться ни с будущим методом Apple, ни с чужой категорией.
- Правильный ответ на «какая победит» — **«поведение не определено, так делать нельзя»**, а не угадывание порядка линковки.

Задача 12. Гонка данных: `atomic` не делает коллекцию потокобезопасной

Условие. Свойство `value` объявлено `atomic`. Два потока параллельно по 100 000 раз делают `value = value + 1`. Ожидаем 200 000. Что получим — и почему `atomic` тут не спас?

Подумай сам. `atomic` делает неделимым **что** — отдельный `get/set` или всю операцию `value = value + 1`?

Разбор. `atomic` гарантирует, что **каждый отдельный** геттер и сеттер выполнится целиком, без того чтобы другой поток увидел свойство «наполовину записанным». Это всё, что он обещает.

A `value = value + 1` — это **две** раздельные операции:

1. `get` — прочесть текущее `value`;
2. `set` — записать `value + 1`.

Между ними другой поток успевает вклиниться: оба читают, скажем, 1000, оба пишут 1001 — один инкремент потерян. `atomic` сделал неделимыми **шаги**, но не **связку** `read-modify-write`. Это и есть гонка данных (`race condition`): часть инкрементов теряется, итог меньше ожидаемого.

Чинить надо так, чтобы **вся** связка `read-modify-write` была неделимой. В решении это последовательная (`SERIAL`) очередь: `dispatch_sync(guard, ^{ good++; })` — два потока физически не пересекутся внутри, очередь пропускает по одному. Альтернативы — мьютекс (`@synchronized`, `os_unfair_lock`) или атомарный счётчик из `<stdatomic.h>`.

Решение — `code/senior-09-atomic-race.m`:

```
atomic-свойство: ожидали 200000, получили 175216 (часть потеряна)
serial-очередь:  ожидали 200000, получили 200000
```

Точное число у тебя будет своё (гонка недетерминирована), но оно будет **меньше** 200 000 — инкременты теряются. Через последовательную очередь — ровно 200 000.

Вывод, который ждут на собеседовании: **`atomic` защищает доступ к одному свойству, а не последовательность операций и тем более не содержимое коллекции.** `atomic`-свойство типа `NSMutableArray` тоже не делает сам массив потокобезопасным — атомарна лишь подмена указателя на массив, а не `addObject:`. Для составных операций нужна явная синхронизация.

Задача 13. `dispatch_sync` на текущей последовательной очереди — дедлок

(Концептуальная, без файла — пример роняет программу.)

Условие. Что произойдёт с этим кодом, если он выполняется **на главном потоке**?

```
dispatch_sync(dispatch_get_main_queue(), ^{
    NSLog(@"привет");
});
```

А с этим — если он уже выполняется внутри блока на последовательной очереди `q`?

```
dispatch_sync(q, ^{
    // мы уже внутри блока на q
    dispatch_sync(q, ^{ ... });    // и снова синхронно на ту же q
});
```

Подумай сам. Что делает `dispatch_sync` с **текущим** потоком, пока блок не выполнится, — и кто на последовательной очереди должен этот блок выполнить?

Разбор. Оба случая — **дедлок** (взаимная блокировка): поток ждёт сам себя. На старых системах программа просто зависала навсегда. Современный `libdispatch` такую ситуацию распознаёт и роняет процесс: по нашему запуску на macOS оба примера падают с `EXC_BAD_INSTRUCTION` в функции `__DISPATCH_WAIT_FOR_QUEUE__` (в `crash-логе` будет строка «`dispatch_sync` called on queue already owned by current thread»). Это диагностика, а не спасение: код всё равно сломан.

Разбери `dispatch_sync` по словам: «поставь блок в очередь и **заблокируй текущий поток**, пока этот блок не выполнится до конца». Теперь наложи это на **последовательную** очередь — она выполняет блоки строго по одному, не начиная следующий, пока не закончит текущий.

Первый случай. Главная очередь последовательна, и обслуживает её главный поток. Ты на главном потоке зовёшь `dispatch_sync(main)`. Главный поток **блокируется** и ждёт, пока блок выполнится. Но выполнить блок может только главный поток — а он сейчас стоит и ждёт сам себя. Никто не сдвинется: зависание.

Второй случай — та же ловушка в общем виде. Ты **внутри** блока на очереди `q` синхронно ставишь на `q` ещё один блок и ждёшь его. Но `q` не начнёт второй блок, пока не доделает первый, — а первый стоит и ждёт второго. Круг замкнулся.

Ключевая формула: **`dispatch_sync` на ту очередь, на которой ты сейчас выполняешься (если она последовательная), — это всегда дедлок**. Сюда же частый прод-баг: код на главном потоке вызывает «удобную» обёртку, а та внутри делает `dispatch_sync(main)` «чтобы наверняка на главном» — и всё виснет, как только её случайно позвали уже с главного потока.

Как правильно:

- Если ты, возможно, **уже** на нужной очереди и нужен синхронный результат — не делай безусловный `dispatch_sync`; проверь, та ли это очередь, или выполни блок напрямую.
- Не нужен результат прямо сейчас — бери `dispatch_async`: он не блокирует текущий поток и дедлока не даёт.
- `dispatch_sync` на **другую** очередь (не текущую) — нормально; опасна именно отправка на самого себя.

В code/ этот пример не кладём намеренно: он либо падает, либо виснет — смотря по системе.

Задача 14. Потокобезопасный синглтон на `dispatch_once`

Условие. Шестнадцать потоков одновременно лезут за [Database shared]. Напиши +shared так, чтобы объект создался **ровно один раз** и все потоки получили один и тот же экземпляр. Почему именно `dispatch_once`, а не `if (instance == nil)`?

Подумай сам. Что будет, если два потока одновременно пройдут проверку `if (instance == nil)` раньше, чем хоть один успел присвоить `instance`?

Разбор. Наивный `if (instance == nil) { instance = [[self alloc] init]; }` в многопоточе ломается: два потока могут **одновременно** увидеть `nil`, оба создадут объект, и один экземпляр перезастрёт другой — синглтон перестает быть единственным. Это та же гонка, что в задаче 12, только цена выше: расходятся не числа, а сам объект.

`dispatch_once` решает это на корню. Он гарантирует, что блок выполнится **один раз за всё время работы программы**, даже под шквалом потоков: первый поток выполняет блок, остальные ждут его завершения и затем видят готовый результат. Токен `dispatch_once_t` обязан быть `static` (одна ячейка на всю программу) — по нему `dispatch_once` и помнит, что уже отработал.

Деталь: внутри пишут `[[self alloc] init]`, а не `[[Database alloc] init]`. `self` в методе класса — это сам класс, по которому метод позвали, поэтому подкласс получит синглтон своего класса.

Решение — `code/senior-10-singleton.m`:

```
>>> Database создан (видно один раз)
a == b ? да, один объект
path: /var/db/main
```

Строка «Database создан» напечаталась **один раз**, хотя за синглтоном ломились 16 потоков. `a == b` — один и тот же объект. `dispatch_once` сделал создание атомарным и единственным.

Задача 15. Потокобезопасный кэш: барьерная запись на concurrent-очереди

Условие. Нужен кэш «ключ-значение», в который пишут и из которого читают **много потоков сразу**. Хочется, чтобы чтения шли параллельно, а записи — эксклюзивно (никто не читает в момент записи). Как это устроить на GCD?

Подумай сам. Можно ли пускать читателей параллельно, но писателя — в одиночку, на одной и той же очереди?

Разбор. Это классическая схема «много читателей — один писатель» (readers-writer), и GCD даёт её почти даром через **барьер** на собственной **concurrent**-очереди:

- **Чтение** — `dispatch_sync(_queue, ^{ ... })` на **concurrent**-очереди: несколько чтений идут **параллельно**, не мешая друг другу, и `sync` возвращает прочитанное значение вызывающему.
- **Запись** — `dispatch_barrier_async(_queue, ^{ ... })`. Барьер заставляет GCD дожидаться завершения **всех текущих** операций на очереди, выполнить блок-писатель **в одиночку** (в этот момент на очереди больше ничего не идёт) и только потом пустить следующие. `async` — потому что писателю не нужно ждать результата, он лишь ставит барьер в очередь и сразу возвращается, не блокируя свой поток.

Важно: очередь должна быть **своя**, созданная `dispatch_queue_create(..., DISPATCH_QUEUE_CONCURRENT)`. На глобальной **concurrent**-очереди барьер не работает как барьер — она общая, и GCD не даёт на ней эксклюзивности.

Зачем `count` в конце читает через `dispatch_sync` на той же очереди: к моменту, когда `sync`-чтение дойдёт до выполнения, все ранее поставленные барьеры-записи на этой очереди уже отработают (очередь упорядочена) — поэтому увидим ровно 100 элементов, а не «сколько успели».

Решение — `code/senior-11-barrier-cache.m`:

```
в кэше 100 элементов (ожидали 100)
k7  = 49 (ожидали 49)
k12 = 144 (ожидали 144)
```

Сто параллельных писателей и пятьдесят параллельных читателей не подрались: барьер развёл записи с чтениями, и кэш собрал все 100 пар ($7^2 = 49$, $12^2 = 144$). Чтения при этом шли одновременно — барьер сдерживает только запись.

Задача 16. weak в многопоточности и пулы автоосвобождения в фоне

(Концептуальная, без файла — разбираем словами.)

Условие. Две части одного вопроса.

1. Безопасно ли читать __weak-ссылку из нескольких потоков? Что не так с таким кодом, если объект могут освободить в другом потоке?

```
if (self.weakDelegate) {  
    [self.weakDelegate doWork];    // между проверкой и вызовом?  
}
```

2. Ты запускаешь долгую работу на **своём** фоновом потоке (NSThread/ pthread), внутри в цикле создаёшь временные объекты. Почему память может потечь и runtime ругается «autorelease with no pool in place»?

Подумай сам. (1) Сколько раз этот код **читает** weakDelegate и что может случиться между чтениями? (2) Кто на свеже созданном потоке отвечает за слив пула автоосвобождения?

Разбор, часть 1 — weak и потоки. Само по себе чтение и запись __weak-ссылки **потокобезопасны**: машинерия ARC под капотом синхронизирована, и ты всегда получишь либо валидный, уже удержанный на время чтения объект, либо честный nil, но **никогда** не висячий указатель на освобождённую память. Это сильная гарантия.

Ловушка в другом — в **двойном чтении**. Код выше читает weakDelegate **дважды**: один раз в if, второй раз при вызове. Между ними другой поток может освободить объект. Тогда первое чтение даст живой объект (if пройдёт), а второе — уже nil. Само сообщение к nil безопасно (тихий по-ор), но логика «проверил — значит есть» нарушена, и при более сложном коде это даёт плавающие баги. Правильно — **прочитать один раз в сильную локальную переменную**, она удержит объект на время работы:

```
__strong id strong = self.weakDelegate;    // одно чтение, объект удержан  
if (strong) {  
    [strong doWork];                        // он точно жив до конца блока  
}
```

Это тот же «weak-strong dance», что и в блоках (приложение E): слабая ссылка рвёт цикл, а локальная сильная гарантирует, что объект не исчезнет **посреди** работы.

Разбор, часть 2 — пулы в фоне. Многие объекты Foundation возвращаются **автоосвобождёнными**: их держит в живых только **пул автоосвобождения** (autorelease pool), пока тот не сольётся (глава 13). Вопрос — кто этот пул создаёт.

- На **главном** потоке пул есть всегда — его крутит run loop приложения, сливая на каждом витке.
- На очередях **GCD** система оборачивает выполнение блоков в свой пул и периодически его сливает — поэтому под `dispatch_async` обычно пул заводить не нужно (но для долгого цикла внутри блока — всё равно стоит).
- На **своём** потоке (NSThread, голый pthread) **явного пула нет** — его никто не создал. Современный runtime на этот случай молча заводит пул сам и сливает его, **только когда поток завершается**. По нашему эксперименту (pthread, три автоосвобождённых объекта) все три `dealloc` пришли уже после выхода из функции потока. Для короткого потока это терпимо. Для долгоживущего потока с циклом — нет: временные объекты копятся часами, память растёт, будто течёт.
- Сообщение `object ... autoreleased with no pool in place - just leaking` из старых книг и форумов runtime печатает только при переменной окружения `OBJC_DEBUG_MISSING_POOLS=YES`. В этом режиме он пул не заводит, и объекты действительно текут — так удобно искать места без пула.

Лечение — обернуть тело потока (а в длинном цикле — тело итерации) в свой `@autoreleasepool`:

```
- (void)threadMain {
    @autoreleasepool {                // пул на весь поток
        while (working) {
            @autoreleasepool {        // и на каждую итерацию: пик памяти низкий
                ... временные объекты ...
            }
        }
    }
}
```

Короткий ответ на собеседовании: **weak потокобезопасен, но читай его один раз в сильную локальную; у своего фонового потока нет явного пула — runtime сольёт неявный только при выходе из потока, поэтому оборачивай работу (и каждую итерацию долгого цикла) в @autoreleasepool сам.**

Задача 17. Исключение через границу @autoreleasepool и в фоновом потоке

(Концептуальная, без файла — разбираем словами.)

Условие. Две ситуации.

1. Объект автоосвобождён **внутри** `@autoreleasepool { ... }`, и прямо оттуда бросают `NSException`, который ловят **снаружи** пула. Что станет с этим объектом и можно ли вернуть его наружу?
2. На фоновой GCD-очереди бросают `NSException` и **нигде** не ловят. Что будет с программой — упадёт только поток или весь процесс?

Подумай сам. Что происходит с пулом, когда исключение «разматывает» стек мимо его закрывающей скобки? И есть ли у фонового потока «верхний» ловец исключений, как у главного `run loop`?

Разбор, часть 1 — исключение через границу пула. Интуиция говорит: «при выходе из `{ }` пул сливается». Но спецификация ARC в Clang говорит прямо обратное: если блок `@autoreleasepool` покинули **исключением**, пул **не сливается**. Он остаётся в стеке пулов, и его объекты освобождает первый **внешний** пул, который сольётся нормально: при его сливе `runtime` заодно снимает и все забытые внутренние.

Мы проверили: объект автоосвобождён во внутреннем пуле, из него брошено исключение, `@catch` снаружи его поймал — `dealloc` пришёл только в конце внешнего `@autoreleasepool`, уже после `@catch`.

Практический вывод: **не опирайся на время жизни автоосвобождённого объекта, который пересёк границу пула исключением**. Он не умрёт сразу, но и сильной ссылки на него у тебя нет: исчезнет, когда сольётся какой-то внешний пул, — а этот момент от тебя не зависит. Если объект нужен после `@catch`, держи его в сильной переменной, объявленной **снаружи** `@try`. Добавь сюда, что код под ARC по умолчанию не готов к исключениям (флаг `-fobjc-arc-exceptions` выключен для `.m`): при разматывании ARC не отпускает сильные локальные переменные, и они текут.

Разбор, часть 2 — исключение в фоне. В Objective-C `NSException` — это сигнал о **программерской ошибке** (вышел за границу массива, послал нераспознанный селектор), а **не** механизм управления потоком. Восстановимые ошибки передают через `NSError`, а не бросают исключениями.

Что до фонового потока: **никакого «верхнего» ловца у него нет**. `AppKit` на главном потоке по умолчанию ловит исключения из обработчиков событий и пишет их в лог (`UIKit` так не делает — там приложение падает), но обычный фоновый поток и блок GCD такой защиты не имеют. Непойманное `NSException` доходит до самого верха стека потока, `runtime` зовёт обработчик непоzymanых исключений и затем `abort()` — **падает весь процесс**, а не «только поток». Поток в общей памяти процесса; уронив его исключением, ты роняешь всё приложение.

Отсюда правила:

- `@try/@catch` — **не** инструмент повседневной обработки ошибок и тем более не «глушилка» багов. Поймать `NSException`, чтобы продолжить как ни в чём не бывало, опасно: после программской ошибки состояние программы уже могло поехать (полусделанная мутация, повисший `lock`).
- Исключение, пересекающее `@autoreleasepool`, **не** сливает его: объекты доживут до слива внешнего пула, но владеть ими ты уже не можешь.
- В фоновом потоке/блоке GCD непойманное исключение валит **весь процесс**. Для ожидаемых сбоев возвращай `NSError`, а исключения оставь на невосстановимые ошибки программиста.

В `code/` это не кладём: пример с непойманным исключением сознательно завершает процесс и сломал бы проверку.

Задача 18. Tagged pointers: почему `==` для маленьких `NSNumber` иногда прав

Условие. Два одинаковых **маленьких** `NSNumber`, созданных в рантайме, сравнивают через `==` (указатели). Потом — два одинаковых **больших**. Где `==` даст «равны», а где «нет», и почему?

```
NSNumber *a = [NSNumber numberWithInt:5];
NSNumber *b = [NSNumber numberWithInt:5];           // a == b ?
NSNumber *c = [NSNumber numberWithInt:1234567890123456789LL];
NSNumber *d = [NSNumber numberWithInt:1234567890123456789LL]; // c == d ?
```

Подумай сам. Всегда ли `NSNumber` — это объект в куче со своим адресом?

Разбор. На 64-битных системах маленькие `NSNumber` runtime хранит **прямо внутри указателя** — это **tagged pointer** («помеченный указатель»): само число закодировано в биты «адреса», отдельный объект в куче **не создаётся**. Два одинаковых маленьких числа получают **один и тот же** tagged-указатель — поэтому `==` для них случайно оказывается прав (да).

Большое число (1234567890123456789) в биты указателя не влезает — runtime создаёт **настоящий объект в куче**. Два таких равны по значению, но это **два разных** объекта по разным адресам, и `==` честно даёт нет.

Любопытная деталь: `[a class]` для tagged-числа всё равно показывает `__NSCFNumber` — снаружи это полноценный `NSNumber`, упаковка в указатель скрыта. И адреса tagged-указателей **обфусцированы** (runtime подмешивает случайную соль на каждый запуск), поэтому в выводе они выглядят как огромные «случайные» числа и меняются от запуска к запуску: прочитать число прямо из «адреса» не выйдет.

Решение — code/senior-12-tagged.m:

```
маленькое 5: a == b (указатели)? да    класс=__NSCFNumber
           a=0x5012cb0de9e1ded7 b=0x5012cb0de9e1ded7 (свой в каждом запуске)
большое число: c == d (указатели)? нет    класс=__NSCFNumber
           c=0x6000007556c0 d=0x6000007556e0 (обычные адреса в куче)
isEqual: всегда верно: маленькие равны, большие равны
```

Смотри: маленькие 5 дали одинаковый указатель (== → да), большие — разные кучные адреса (== → нет). А isEqual: верен в **обоих** случаях.

Вывод, который и проверяют: == для NSNumber (и для объектов вообще) — лотерея, зависит от значения, разрядности и сборки. Сравнивай значения **только** через isEqual: / isEqualToNumber:. Tagged pointers — причина, по которой == иногда «случайно работает» и тем коварнее: на маленьких числах в тестах прошло, на больших в проде сломалось.

Отличие от Си. В Си int — это всегда само число в памяти, и == сравнивает значения. Здесь NSNumber — объект, == сравнивает указатели, и runtime ради экономии прячет маленькие числа в сам указатель. Поэтому «сравнение чисел» через == ведёт себя то как сравнение значений, то как сравнение адресов — и доверять ему нельзя.

Задача 19. Найди и почини утечку: цикл удержания через NSTimer

Условие. Объект Ticker заводит повторяющийся таймер на себя через scheduledTimerWithTimeInterval:target:selector:.... Объект выходит из области видимости, но dealloc не зовётся. Где цикл удержания и как починить?

```
- (void)startBad {
    self.timer = [NSTimer scheduledTimerWithTimeInterval:0.05
                                           target:self
                                           selector:@selector(tick:)
                                           userInfo:nil
                                           repeats:YES];
}
```

Подумай сам. Как NSTimer относится к своему target — слабо или сильно? И кто держит сам таймер?

Разбор. scheduledTimer...target: **сильно удерживает** target, то есть self. А self сильно держит таймер в свойстве timer. И вдобавок запланированный таймер держит **run loop**. Кольцо:

```
run loop —> timer —strong—> self —strong—> timer
```

Пока таймер не остановлен (`invalidate`), его держит `run loop`, таймер держит `self`, `self` держит таймер. Внешние ссылки на `Ticker` исчезли, а счётчик не падает до нуля — `dealloc` не сработает **никогда**. Коварство в том, что `weak` на свойстве `timer` тут не помог бы: сильную ссылку держит не оно, а сам таймер на `target`.

Чинить надо **связь таймер → self**. В современном API берут **блочный** таймер и захватывают `self` **слабо**:

```
- (void)startGood {
    __weak typeof(self) weakSelf = self;
    self.timer = [NSTimer scheduledTimerWithTimeInterval:0.05
                                              repeats:YES
                                              block:^(NSTimer *t) {

        Ticker *strong = weakSelf;
        if (!strong) { [t invalidate]; return; } // self ушёл – гасим таймер
        [strong tick:t];
    }];
}
```

Теперь таймер держит блок, а блок ссылается на `self` **слабо**. Когда внешний код отпускает `Ticker`, `weakSelf` становится `nil`, блок при очередном тике это видит и сам гасит осиротевший таймер (`invalidate`) — кольцо разорвано, `dealloc` срабатывает.

(Не путай: слабый захват освобождает `self`, но не сам таймер. Пока его не погасили, `run loop` держит таймер, таймер держит блок, и блок продолжает срабатывать впустую. Поэтому таймер всё равно гасят через `invalidate` — в нашем решении это делает сам блок, заметив `weakSelf == nil`, а в реальном коде обычно ещё и явно, когда таймер больше не нужен.)

Решение — `code/senior-14-timer-cycle.m`:

```
--- ПЛОХО: target/selector-таймер ---
    (dealloc для «bad» НЕ вызван – утечка через таймер)
--- ХОРОШО: блочный таймер + weak ---
    dealloc: Ticker «good» освобождён
    (dealloc для «good» вызван выше – цикл разорван)
```

У «bad» строки `dealloc` нет — объект застрял в кольце с таймером. У «good» `dealloc` напечатался: слабый захват `self` в блоке разорвал цикл, и осиротевший таймер сам погасился при следующем тике. По пропавшему `dealloc`, как обычно, утечку и ловят.

Отличие от Си. В Си «таймер» был бы структурой с указателем на callback-функцию — никаких счётчиков ссылок, никаких циклов удержания, только риск забыть free или позвать функцию по висячему указателю. Здесь память считает ARC, и появляется новый класс ошибок: объект и таймер, сильно держащие друг друга. Лечится тем же weak, что и циклы с блоками и делегатами.

Что повторить перед Senior-собеседованием

Если на любой пункт отвечаешь не из модели, а «по памяти» — вернись к задаче.

- **isa и метаклассы.** Объект → класс → метакласс → корневой метакласс (замкнут на себя). —методы живут в классе, +методы — в метаклассе (задача 1).
- **Отправка сообщения.** [obj foo:5] = objc_msgSend(obj, sel, 5); SEL — имя, IMP — код, Method — их пара; сначала кэш класса, потом поиск от isa вверх по superclass (задача 2).
- **Три шанса пересылки.** resolveInstanceMethod: (доращиваем метод) → forwardingTargetForSelector: (один другой получатель, дёшево) → methodSignatureForSelector: + forwardInvocation: (NSInvocation, много получателей, дорого) → doesNotRecognizeSelector: и падение (задачи 3, 4, 5).
- **Два корня.** NSObject (почти всё) и NSProxy (тонкий, для прокси — ничего лишнего не перехватывает) (задача 6).
- **Swizzling.** Только в +load, dispatch_once, сначала class_addMethod (не ломать родителя); свиззлить свой класс статически, а не object_getClass(obj) — там может оказаться KVO-подкласс (задачи 7, 8).
- **Связанные объекты.** Категория добавляет методы, но не ivar — данные цепляем через objc_setAssociatedObject по адресу-ключу (задача 9).
- **KVO изнутри.** Подмена isa на NSKVONotifying_..., перехват сеттера, -class врёт; обязателен removeObserver: (задача 10).
- **Категории-дубли.** Два метода с одним именем — поведение не определено; категория добавляет, а не подменяет; префиксы (задача 11).
- **Потоки.** atomic ≠ потокобезопасная коллекция и ≠ атомарный read-modify-write; синглтон — dispatch_once; кэш — concurrent-очередь + dispatch_barrier_async; dispatch_sync на текущую serial-очередь — дедлок, который современный GCD превращает в падение (задачи 12–15).
- **weak и пулы.** weak потокобезопасен, но читай один раз в сильную локальную; у своего фонового потока нет явного пула, неявный сливается лишь при выходе из потока — обращай в @autoreleasepool (задача 16).

- **Исключения.** Программерская ошибка, не управление потоком; пересекая пул, **не** сливают его; непойманное в фоне валит весь процесс (задача 17).
- **Tagged pointers.** Маленькие NSNumber живут в самом указателе — == «случайно» прав; сравнивай через isEqual: (задача 18).
- **NSTimer.** Держит target сильно — цикл через run loop; лечится блочным таймером с weak Self и invalidate (задача 19).

Документация Apple

- **Objective-C Runtime Programming Guide** — developer.apple.com: Messaging (objc_msgSend), Dynamic Method Resolution (resolveInstanceMethod:), Message Forwarding (forwardingTargetForSelector:, forwardInvocation:).
- **Objective-C Runtime (API reference)** — developer.apple.com/documentation/objectivec/objective-c-runtime (функции object_getClass, class_getName, class_isMetaClass, class_getInstanceMethod, method_getImplementation, class_addMethod, method_exchangeImplementations, objc_setAssociatedObject).
- **NSObject** — developer.apple.com/documentation/objectivec/nsobject-swift.class (class, +load, +resolveInstanceMethod:, -forwardingTargetForSelector:, -methodSignatureForSelector:, -forwardInvocation:, -doesNotRecognizeSelector:).
- **NSProxy** — developer.apple.com/documentation/foundation/nsproxy (второй корневой класс для заместителей).
- **NSInvocation** — developer.apple.com/documentation/foundation/nsinvocation (упакованное сообщение для пересылки многим).
- **Key-Value Observing Programming Guide** (подписка, словарь change, что KVO делает с isa) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/KeyValueObserving/KeyValueObserving.html
- **Concurrency Programming Guide** и **Dispatch (GCD)** — developer.apple.com/documentation/dispatch (dispatch_once, dispatch_sync/dispatch_async, dispatch_barrier_async, последовательные и параллельные очереди).
- **Advanced Memory Management Programming Guide** и **Transitioning to ARC Release Notes** (weak/strong, пулы автоосвобождения, циклы удержания, таймеры) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/MemoryMgmt/Articles/MemoryMgmt.html и developer.apple.com/library/archive/releasenotes/ObjectiveC/RN-TransitioningToARC/Introduction/Introduction.html
- **Automatic Reference Counting** в документации Clang — clang.llvm.org/docs/AutomaticReferenceCounting.html (раздел @autoreleasepool: пул не сливается при выходе исключением; -fobjc-arc-exceptions).

- **Exception Programming Topics** (NSException для программмерских ошибок, NSError для восстановимых) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/Exceptions/Exceptions.html

Приложение G. Objective-C глазами C++-разработчика

Остальная книга писалась для того, кто видит указатель впервые. Это приложение — для тебя, если ты пришёл из C++. Премиса простая: ты уже знаешь, что такое `vtable`, `RAII`, шаблоны, умные указатели и `RTTI`. Я не буду объяснять их заново. Я покажу, **как Objective-C решает те же задачи иначе** — где-то слабее, где-то неожиданно мощнее, — и где между языками проходят настоящие границы.

Формат — сравнения бок о бок. Для каждой темы: короткий тезис, блок «как в C++», блок «как в Objective-C», и разбор того, в чём именно разница. Листинги в тексте — выжимки; полные программы, которые компилируются и запускаются, лежат в `code/` с префиксом `g-`, команды сборки — в конце.

1. Камертон: статика против динамики

Оба языка — надстройки над Си, оба компилируются в нативный код, оба дают доступ к указателям и ручной памяти. На этом сходство кончается.

C++ — язык **компиляции**. Его идеал — `zero-overhead abstraction`: всё, что можно решить до запуска, решается до запуска. Типы, перегрузки, специализации шаблонов, `devirtualization`, инлайнинг — работа компилятора. В готовом бинарнике от системы типов не остаётся почти ничего; вызов функции — это переход по адресу, который выбрал компоновщик.

Objective-C — язык **рантайма**. Рядом с твоей программой живёт библиотека `libobjc`, и она работает всё время выполнения: хранит классы как объекты, ищет методы по имени, пересылает сообщения, добавляет методы на лету. Центральная операция — не «вызвать функцию по адресу», а «послать объекту сообщение, и пусть он сам в рантайме решит, какой код выполнить». Это медленнее прямого вызова, но из этой динамики растёт половина возможностей Cocoa: `KVO`, `target-action`, пересылка, категории, `NSCoding`, апповый цикл.

Держи этот контраст в голове всё приложение. Почти каждое различие ниже — его следствие.

2. Класс и файлы

И там, и там объявление отделяют от реализации. Но в C++ это твой выбор (можно всё в заголовке), а в Objective-C — жёсткое разделение на два ключевых слова.

```
// Greeter.hpp
class Greeter {
public:
    Greeter(std::string name);    // объявление
    std::string hello() const;
    int times = 0;                // поле прямо в классе
private:
    std::string name_;
};

// Greeter.cpp
Greeter::Greeter(std::string name) : name_(std::move(name)) {}
std::string Greeter::hello() const { return "Привет, " + name_; }
```

```
// Greeter.h
@interface Greeter : NSObject
@property (nonatomic, copy) NSString *name;    // объявление свойства
@property (nonatomic) NSInteger times;
- (instancetype)initWithName:(NSString *)name;
- (NSString *)hello;
@end

// Greeter.m
@implementation Greeter
- (instancetype)initWithName:(NSString *)name {
    self = [super init];
    if (self) { _name = [name copy]; }
    return self;
}
- (NSString *)hello {
    return [NSString stringWithFormat:@"Привет, %@", _name];
}
@end
```

В чём разница:

- В C++ `class` — одно ключевое слово, заголовок и `.cpp` — твоё соглашение. В Objective-C интерфейс (`@interface ... @end`) и реализация (`@implementation ... @end`) — это синтаксис языка. Интерфейс кладут в `.h`, реализацию в `.m`.
- В Objective-C **нет инлайна тела метода в заголовке**. Нельзя написать `-(int)x { return 5; }` в `@interface`. Тело всегда в `@implementation`. Заголовочного `inline`-метода как в C++ просто не существует — диспетчеризация всё равно динамическая, инлайнить нечего.
- Базовый класс указывают через двоеточие — `: NSObject`. Почти всё в Foundation растёт из `NSObject` (см. приложение A); своя иерархия без него — редкость.
- Шаблона нет: один заголовок — один конкретный класс, не специализация.

3. Вызов метода против отправки сообщения

Это главное различие, и у него есть драматическое практическое следствие.

```
Greeter *g = getGreeter();      // может вернуть nullptr
// g->hello() при g == nullptr — undefined behavior (разыменование).
// Защищаться надо РУЧНОЙ проверкой:
if (g != nullptr) {
    std::cout << g->hello();
}
```

```
Greeter *g = getGreeter();      // может вернуть nil
// [g hello] при g == nil — БЕЗОПАСНО. Вернётся nil, ничего не падает.
NSString *s = [g hello];       // если g == nil, s == nil — и всё
```

В C++ `g->hello()` для неvirtуального метода компилятор превращает в прямой `call` по известному адресу (для виртуального — в косвенный через `vtable`). В обоих случаях получатель обязан существовать: вызов у `nullptr` — UB. На практике виртуальный вызов почти наверняка упадёт, а неvirtуальный, не трогающий полей, может и «сработать» — что ещё хуже: баг прячется до поры.

В Objective-C `[g hello]` компилятор превращает в `objc_msgSend(g, @selector(hello))` (см. главу 14). Эта функция **первым делом проверяет получателя на nil** и, если он `nil`, тихо возвращает ноль нужного размера — `nil`, `0`, `NO`, `0.0` (см. главу 5). Программа не падает. Это сознательное решение языка, а не случайность.

Проверяется это парой `code/g-nil-vs-null.cpp` и `code/g-nil-vs-null.m`. C++ при `nullptr` обязан уклониться от вызова, иначе UB:

```
g == nullptr, метод НЕ вызван (иначе UB/краш)
length = 5, upper = HELLO
```

Objective-C спокойно шлёт сообщения `nil` и получает нули:

```
length      = 0
uppercaseStr = (null)
hasPrefix   = 0
real length = 5
real upper   = HELLO
```

Плюс: код Сосоа почти не обвешан проверками `if (x != nil)` — отправка `nil` безвредна. Минус: если объект случайно стал `nil`, ошибка не выстрелит сразу, а «размажется» нулями по логике, и баг всплывёт позже. C++-привычка «получил указатель — проверь» в Objective-C превращается в «получил неожиданный 0/nil — ищи, где по дороге был `nil`-получатель».

Отличие от C++. `obj->method()` — диспетчеризация на этапе компиляции (прямой вызов) или через `vtable` (`virtual`). `[obj method]` — поиск метода по имени в рантайме, начиная с защиты от `nil`. Та же операция «позвать поведение объекта», но в разных временах: у C++ — почти всегда `compile-time`, у Objective-C — всегда `run-time`.

4. Типы и полиморфизм

В C++ полиморфизм надо *включить*: общий базовый класс плюс `virtual`. В Objective-C он включён всегда, и общего предка не требует.

```
struct Animal {
    virtual ~Animal() = default;
    virtual std::string speak() const = 0;    // без virtual — статика
};
struct Dog : Animal { std::string speak() const override { return "гав"; } };

std::unique_ptr<Animal> a = std::make_unique<Dog>();
a->speak();           // динамически — потому что virtual + общий базовый
```

```
@interface Dog : NSObject @end    // общего предка со speak у них нет
@interface Duck : NSObject @end

id animal = someAnimal();
if ([animal respondsToSelector:@selector(speak)]) {
    [animal speak];              // динамически ВСЕГДА, общий предок не нужен
}
```

В чём разница:

- **Методы Objective-C «виртуальные» по умолчанию.** Нет ключевого слова `virtual`, нет `override`. Каждое сообщение идёт через `objc_msgSend`, то есть реализация выбирается в рантайме. Аналог «невиртуального вызова ради скорости» появился в Clang лишь недавно и как расширение: атрибут `__attribute__((objc_direct))` превращает метод в прямой вызов Си-функции — зато такой метод пропадает из рантайма (его не найти по селектору, не переопределить, не подменить). В обычном коде им почти не пользуются.
- Поэтому **полиморфизм не требует общей базы.** В C++ положить `Dog` и `Duck` в один контейнер и звать `speak()` через виртуальный вызов можно, только если у них общий полиморфный предок (обходные пути вроде `std::variant + std::visit` требуют заранее знать весь список типов). В Objective-C достаточно, чтобы оба отвечали на селектор `speak` — это `duck typing` в буквальном смысле (см. `code/g-poly.m`).
- Тип `id` — это «любой объект», стираемый указатель без статических гарантий. Под `id` компилятор пропускает любое объявленное где угодно сообщение. Это твой аналог `void*` + «доверься рантайму», но безопаснее: непонятое сообщение не UB, а контролируемое исключение (глава 14).

Запусти обе программы. C++ через `vtable`:

```
гав (это Dog)
мяу
```

Objective-C через сообщения к неродственным классам:

```
Dog -> гав
      (это Dog)
Duck -> кря
```

Нюанс про «нет `override`»: переопределение метода в подклассе работает, просто пишется без ключевого слова — ты объявляешь метод с тем же селектором, и он перекрывает родительский. Защиты от опечатки в имени (как `override` в C++) язык не даёт; ошибёшься в селекторе — получишь новый метод вместо переопределения, молча.

5. Шаблоны против `lightweight generics`

Здесь Objective-C проигрывает вчистую, и это надо понимать сразу.

```
template <typename T>
T max_of(T a, T b) { return a > b ? a : b; }

std::vector<int> v;           // настоящее инстанцирование под int
max_of(3, 7);                 // инстанцируется max_of<int>
```

```
// Lightweight generics: ТОЛЬКО подсказка компилятору.
NSArray<NSString *> *names = @[@"Серик"];
NSString *first = names.firstObject;    // компилятор знает тип -> без каста

// В рантайме параметр типа стёрт: это обычный NSArray из id.
```

В чём разница:

- C++-шаблоны — это **мономорфизация**: компилятор генерирует отдельный машинный код под каждый набор типов. `std::vector<int>` и `std::vector<double>` — разные типы с разным кодом. На шаблонах же стоят метапрограммирование, трейты, SFINAE и (с C++20) концепты.
- ObjC lightweight generics (`NSArray<NSString *> *`) — это **только аннотация для компилятора**, чтобы он проверял типы на этапе сборки и избавлял от лишних кастов. В рантайме параметр **стирается** (type erasure): остаётся обычный NSArray, хранящий id. Никакого специализированного кода не генерируется.
- Аналога шаблонного метапрограммирования в Objective-C **нет вообще**. Generic-обобщения по типу делают через id и динамику, а не через генерацию кода. Если тебе нужна compile-time-генерация — это территория C++ (и .mm-файлов, тема 17).

6. Наследование

```
// C++: множественное наследование + абстрактные классы.
struct Drawable { virtual void draw() const = 0; };
struct Serializable { virtual std::string encode() const = 0; };
class Widget : public Drawable, public Serializable { /* ... */ };

// Objective-C: одиночное наследование классов + НЕКОЛЬКО протоколов.
@protocol Drawable <NSObject> - (void)draw; @end
@protocol Serializable <NSObject> - (NSString *)encode; @end

@interface Widget : NSObject <Drawable, Serializable>
@end
```

В чём разница:

- Класс в Objective-C наследует **ровно один** суперкласс. Множественного наследования классов нет.
- Роль «множественности» играют **протоколы** (@protocol). Протокол ≈ чисто абстрактный класс / интерфейс из C++: набор объявлений методов (и свойств) без реализации. Класс

может принять сколько угодно протоколов — отсюда «много интерфейсов, один базовый класс».

- Ключевое отличие протокола от абстрактного класса C++: **у протокола нет состояния**. Он не несёт полей (ivars) и не даёт реализации по умолчанию. То есть это интерфейс в чистом виде, без «миксина с данными». Diamond-проблемы наследования данных просто не возникает — наследовать данные не от кого.
- В протоколе методы бывают @required (по умолчанию) и @optional. Опциональных методов в абстрактном классе C++ нет — там либо чисто виртуальный, либо нет. Перед вызовом опционального метода спрашивают respondsToSelector:.

Если нужна общая реализация для нескольких классов без общего предка — в C++ ты бы взял CRTP или миксин-наследование. В Objective-C категории на протокол нет (это возможность Swift, protocol extension): общую реализацию кладут в категорию на общий базовый класс (хоть на NSObject) или в свободную функцию; см. следующую тему.

7. Расширение классов: категории

Это приём, аналога которому в C++ нет, и он часто удивляет сильнее всего.

```
// C++: дописать метод в std::string НЕЛЬЗЯ.
// Только свободная функция или свой наследник.
std::string shout(const std::string &s) {
    std::string r = s;
    for (char &c : r)
        c = static_cast<char>(std::toupper(static_cast<unsigned char>(c)));
    return r + "!";
}
shout(name);      // вызов «снаружи», не name.shout()
```

```
// Objective-C: КАТЕГОРИЯ добавляет метод прямо в чужой класс – NSString.
@interface NSString (Shout)
- (NSString *)shout;
@end

@implementation NSString (Shout)
- (NSString *)shout {
    return [[self uppercaseString] stringByAppendingString:@"!"];
}
@end

[@"привет" shout];      // теперь ЛЮБАЯ строка умеет shout как родной метод
```


В чём разница:

- В C++ изменить готовый класс (тем более чужой/системный) невозможно: можно унаследоваться или написать свободную функцию. Метод появляется «снаружи», синтаксис вызова другой.
- Категория Objective-C добавляет метод **в сам класс** во время загрузки образа. После этого `shout` есть у любого `NSString` в программе, как встроенный, и зовётся обычным сообщением `[s shout]`. Подкласс не создаётся (см. главу 11).
- Цена гибкости: категория **не может добавить хранимое поле** (`ivar`) — только методы. Если нужно «прицепить» данные к чужому классу, используют `associated objects` из рантайма. Для своих новых полей есть расширения класса (`class extension`, безымянная категория в `.m`).
- Ещё цена — глобальность: категория меняет класс для **всей** программы. Два конфликтующих метода с одним селектором в разных категориях — и кто победит, не определено. Дисциплина та же, что со `swizzling`.

Запуск `code/g-category.m` показывает, что метод доступен и литералам, и строкам из недр Foundation:

```
ПРИВЕТ!
Т!
А!
ВВ!
ССС!
```

8. Память: ARC против RAII

Обе модели детерминированные (не трассирующий GC). Прямой аналог ARC в C++ — не RAII вообще, а `std::shared_ptr`: оба считают ссылки. Но устроены они по-разному.

```
{
    auto obj    = std::make_shared<Node>(); // сильная ссылка, refcount=1
    auto alias  = obj;                      // refcount=2
    std::weak_ptr<Node> w = obj;            // слабая, не держит
} // здесь Node разрушается ДЕТЕРМИНИРОВАННО, на закрывающей скобке
```

```
{
    Node *obj = [[Node alloc] init]; // strong по умолчанию, retain=1
    Node *alias = obj;              // retain=2 (ARC вставит retain)
    __weak Node *w = obj;           // слабая, не держит, занулится сама
} // ARC вставит release; объект освободится, когда retain дойдёт до 0
```

Сопоставление почти один в один:

C++	Objective-C (ARC)
<code>std::shared_ptr<T></code>	strong-ссылка
<code>std::weak_ptr<T></code>	weak-ссылка
цикл <code>shared_ptr</code> $\nrightarrow$ <code>shared_ptr</code> (утечка)	retain cycle
разрыв цикла через <code>weak_ptr</code>	разрыв через weak
деструктор <code>~T()</code>	<code>-dealloc</code>

Где аналогия ломается — и это важно для C++-головы:

- **В Objective-C объекты всегда в куче и всегда по указателю.** Нет value-объектов на стеке, нет `Node n`; с автоматическим хранением. Любой объект — это `Node *`, рождённый `alloc/init`. Отсюда нет `copy-ctor`, `move-ctor`, `RVO` — этих понятий просто негде применить к объектам.
- **Нет деструкторов, привязанных к области видимости.** `-dealloc` детерминирован по счётчику ссылок: он приходит, когда счётчик дошёл до нуля. Локальную `strong-`переменную ARC отпускает не позже `}`, но спецификация Clang разрешает ему сделать это **раньше** — сразу после последнего использования (если не пометить переменную `__attribute__((objc_precise_lifetime))`). Деструктор C++ гарантированно зовётся ровно на `}`. Поэтому RAII-идиома «ресурс жив ровно до конца scope» (`lock_guard`, `scope_guard`) на объектах Objective-C ненадёжна: ты управляешь не scope, а тем, кто держит `strong-`ссылку. Ресурс, который надо закрыть в точный момент, закрывают явным вызовом (`close`, `invalidate`).
- **weak сам зануляется.** Когда объект умирает, все `__weak`-указатели на него автоматически становятся `nil` (zeroing weak references). У `std::weak_ptr` иначе: сам объект `weak_ptr` не «обнуляется», ты узнаёшь о смерти только через `.lock()/expired()`. В Objective-C можно прямо проверить `if (w != nil)`. Но в многопоточном коде объект может умереть сразу после проверки, поэтому идиома та же, что с `.lock()`: прочитать weak один раз в сильную локальную и работать с ней.
- ARC — это не библиотека, а **работа компилятора**: он вставляет `retain/release/autorelease` за тебя по правилам владения (глава 13). Это ближе к «компилятор расставил `shared_ptr`-операции», чем к отдельному типу умного указателя.

9. Свойства

```
class Person {
public:
```

```

    const std::string &name() const { return name_; } // геттер вручную
    void setName(std::string n) { name_ = std::move(n); }
    int age = 0; // или публичное поле
private:
    std::string name_;
};
// KVO/наблюдения за изменением поля — нет в языке.

```

```

@interface Person : NSObject
@property (nonatomic, copy) NSString *name; // геттер+сеттер генерятся
@property (nonatomic, assign) NSInteger age;
@end
// person.name = @"..."; -> [person setName:@"..."] (это сообщение)
// person.name -> [person name]

```

В чём разница:

- @property — это не поле, а **декларация пары сообщений** геттер/сеттер плюс (обычно) скрытый ivar `_name`. Компилятор синтезирует и аксессоры, и хранилище. Точечный синтаксис `person.name` — синтаксический сахар над отправкой сообщения, а не доступ к памяти, как `obj.field` в C++.
- Атрибуты в скобках задают семантику владения и потокобезопасность: `copy/strong/weak/assign`, `nonatomic/atomic`, `readonly`. В C++ это ты пишешь руками в сеттере. `copy` у `NSString`-свойства — важная идиома (`immutable`-копия на входе), прямого аналога у `std::string` нет, потому что там присваивание и так копирует значение.
- Главный бонус, которого в C++ нет в языке: свойства автоматически участвуют в **KVC** (доступ по строковому ключу: `[obj valueForKey:@"name"]`) и **KVO** (наблюдение за изменением, глава 20). Это рантайм-магия поверх сеттеров-сообщений; в C++ такого встроенного механизма наблюдения нет — его собирают руками (сигналы/слоты, `observer`-паттерн).

10. Перегрузка: её нет

```

// C++: перегрузка функций и операторов — норма.
double area(double s) { return s * s; }
double area(double w, double h) { return w * h; } // то же имя, иные параметры
Vec2 operator+(Vec2 a, Vec2 b); // operator overloading

```

```
// Objective-C: НЕТ ни перегрузки функций/методов, ни перегрузки операторов.
// Каждый селектор уникален. «Разные сигнатуры» = разные ИМЕНА.
- (instancetype)initWithName:(NSString *)name;
- (instancetype)initWithName:(NSString *)name age:(NSInteger)age;
```

В чём разница:

- Имя метода Objective-C — это **селектор целиком**, со всеми двоеточиями (глава 5). `initWithName:` и `initWithName:age:` — два разных имени, а не одна перегруженная функция. Поэтому методы называют длинно и описательно: имя обязано быть уникальным и заодно объясняет каждый аргумент прямо в точке вызова.
- Перегрузки **функций/методов по типам аргументов нет**: нельзя завести два метода `setValue:` с разными типами — это один селектор. Если нужны варианты, придумывают разные имена (`setIntValue:`, `setStringValue:`).
- Перегрузки **операторов нет**: `+` и `==` для своих классов не переопределяются (`==` у объектов всегда сравнивает указатели). Сравнение по содержимому — это сообщение (`isEqual:`/`isEqualToString:`), сложение — метод (`arrayByAddingObject:`). Единственное исключение — сабскрипты: `obj[i]` и `obj[key]` компилятор превращает в сообщения `objectAtIndexedSubscript:`/`objectForKeyedSubscript:` (и парные `set...`), так что своему классу «`[]`» можно дать, просто реализовав эти методы. Но это соглашение об именах, а не настоящий `operator[]`.

Проверка C++-стороны — `code/g-overload.cpp`:

```
9
10
площадь фигуры X
(4, 6)
```

В Objective-C каждую из этих «перегрузок» пришлось бы назвать отдельным селектором.

11. Пространства имён: их тоже нет

```
namespace net { class Socket {}; }
namespace ui { class Socket {}; } // ок, разные пространства
net::Socket s;
```

```
// Objective-C: namespace НЕТ. Уникальность — через ПРЕФИКСЫ в имени класса.
@interface NSString : NSObject @end // NS = NeXTSTEP / Foundation
// CFArrayRef // CF = Core Foundation (Си-тип)
@interface SRKSocket : NSObject @end // свой трёхбуквенный префикс
```

В чём разница:

- В Objective-C **плоское глобальное пространство имён классов**: имя класса должно быть уникально на всю программу, включая все слinkованные фреймворки. namespace, вложенности, using — ничего этого нет.
- Поэтому коллизии разводят **префиксами**: NS (Foundation/AppKit исторически от NeXTSTEP), CF (Core Foundation), UI (UIKit), CA (Core Animation). Двухбуквенные префиксы Apple оставляет себе, поэтому свои классы префиксуют **тремя** буквами и больше — чтобы не столкнуться с будущими системными именами.
- Это касается и @protocol, и категорий, и даже Си-функций C-уровня рантайма. Дисциплина именования заменяет языковой механизм.

12. Интроспекция против RTTI

Здесь Objective-C берёт убедительный реванш за шаблоны.

```
// C++ RTTI: скромный набор, требует полиморфного типа (есть virtual).
Animal *a = get();
if (Dog *d = dynamic_cast<Dog *>(a)) { /* это Dog */ }
const std::type_info &t = typeid(*a);
std::cout << t.name();      // искажённое имя, без списка методов/полей

// Objective-C runtime: объект знает о себе почти всё (глава 14).
if ([obj isKindOfClass:[Dog class]]) { /* это Dog или его потомок */ }
if ([obj respondsToSelector:@selector(speak)]) { [obj speak]; }

unsigned int n;
Method *m = class_copyMethodList([obj class], &n);    // СПИСОК методов
// + class_copyIvarList, class_getSuperclass, swizzling,
// добавление методов на лету
free(m);
```

В чём разница:

- C++ RTTI — это typeid и dynamic_cast; динамический ответ они дают лишь для полиморфных типов. Ты можешь спросить «этот ли тип» и привести вниз по иерархии. Перечислить методы или поля класса в рантайме нельзя — после компиляции их структуры в программе нет.
- Objective-C-объект **носит знание о себе с собой** через поле isa и живой объект класса. В рантайме доступны: имя класса, цепочка суперклассов, полный список методов и ivar'ов, проверки isKindOfClass:/respondToSelector:/conformsToProtocol:, добавление методов

(class_addMethod), подмена реализаций (swizzling), пересылка сообщений. На этом построены KVO, сериализация, DI-фреймворки и половина отладочного инструментария (глава 14).

- Практически: где C++ требует знать тип статически или городить visitor-паттерн, Objective-C спрашивает объект напрямую во время работы. Цена — та самая динамика и чуть меньшая скорость.

13. Строки и контейнеры: семантика копирования

Тонкое место, на котором C++-разработчик спотыкается чаще всего.

```
std::vector<std::string> a{"apple", "banana"};
std::vector<std::string> b = a;    // ГЛУБОКАЯ КОПИЯ: b независим от a
b.push_back("cherry");
b[0] = "APPLE";
// a по-прежнему {apple, banana}; &a[0] != &b[0]
```

```
NSMutableArray *a = [@"apple", @"banana" mutableCopy];
NSMutableArray *b = a;           // копируется только УКАЗАТЕЛЬ: один объект
[b addObject:@"cherry"];
b[0] = @"APPLE";
// а ТОЖЕ изменился: a и b — это один и тот же массив в куче
```

В чём разница:

- `std::string`/`std::vector` — **value-типы**. Сам объект живёт на стеке или инлайн внутри владельца (буфер элементов — в куче, но им владеет только он), присваивание = глубокая копия, передача в функцию по значению копирует, по `const&` — нет. Семантика значения, как у `int`.
- `NSString`/`NSArray` — **объекты в куче по ссылке**. Переменная — это указатель; присваивание копирует **указатель**, а не содержимое. Два имени смотрят на один объект, и правка через одно видна через другое. Это *reference-семантика*, как у `shared_ptr<vector>`, а не у `vector`.
- Чтобы получить независимую копию, её **просят явно**: `[obj copy]` (immutable-копия) или `[obj mutableCopy]`. Это контракт протоколов `NSCopying`/`NSMutableCopying` (приложение А).
- Отсюда же пары *immutable/mutable*: `NSString`/`NSMutableString`, `NSArray`/`NSMutableArray`. В C++ изменяемость регулирует `const`, и тип один; в Objective-C это **два разных класса**, причём *mutable* — наследник *immutable*.

Сравни вывод. C++ (`code/g-copy-semantics.cpp`) — два независимых вектора:

```
a.size() = 2, a[0] = apple
b.size() = 3, b[0] = APPLE
&a[0] == &b[0]? no
```

Objective-C (code/g-copy-semantics.m) — один объект на два имени, пока не попросили mutable Copy:

```
a.count = 3, a[0] = APPLE
b.count = 3, b[0] = APPLE
a == b (один объект)? да
после mutableCopy: a.count = 3, c.count = 4
```

Отличие от C++. у `std::vector` — это копирование значения; у `NSArray *` — копирование указателя. Если в C++-голове сидит «присвоил — значит скопировал», в Objective-C это источник коварных багов: ты поменял `b`, а сломался `a`.

14. Исключения

Идиома обработки ошибок различается принципиально, и это не вкусовщина.

// C++: исключения — ШТАТНЫЙ механизм ошибок, дружат с RAII.

```
try {
    auto data = readFile(path);    // бросит при ошибке
    process(data);                // деструкторы раскрутят ресурсы
} catch (const std::runtime_error &e) {
    log(e.what());
}
```

*// Objective-C: recoverable-ошибки — через NSError **, а НЕ исключения.*

```
NSError *error = nil;
NSData *data = [NSData dataWithContentsOfFile:path
                                options:0
                                error:&error];
if (!data) {
    // вернул nil -> смотрим error
    NSLog(@"%@", error.localizedDescription);
    return;
}
// @try/@catch существует, но он для ПРОГРАММЕРСКИХ ошибок (баг в коде):
@try { [arr objectAtIndex:999]; }
@catch (NSException *e) { NSLog(@"баг: %@", e.reason); }
```

В чём разница:

- В C++ исключения — нормальный путь сообщения об ошибке, и они работают в связке с RAII: при раскрутке стека деструкторы автоматически освобождают ресурсы. throw/catch — повседневный инструмент.
- В Objective-C исключения (@throw/@try/@catch/NSException) зарезервированы под **ошибки программиста** — выход за границы массива, unrecognized selector, нарушение контракта. Их **не** ловят, чтобы «продолжить работу»; они сигнализируют о баге, который надо чинить.
- Ожидаемые, восстановимые ошибки (файла нет, сеть упала, JSON битый) передают через **NSError** ** — выходной параметр-указатель: метод возвращает nil/NO при неудаче и заполняет error (глава 18). Это ближе к C++ std::expected (C++23)/кодам возврата, чем к исключениям.
- Практическое следствие для C++-головы: код Objective-C не рассчитан на раскрутку исключением. Под ARC в .m по умолчанию даже сильные локальные переменные при @throw не отпускаются (нужен флаг -fobjc-arc-exceptions; в .mm он включён по умолчанию), а Сосоа-код между @throw и @catch может остаться в полусломанном состоянии. Поэтому исключениями не «рулят» потоком.

15. Блоки против лямбд

Близкие родственники, но с разным синтаксисом захвата.

```
int base = 10;
auto byValue = [base](int x) { return base + x; }; // [=]-стиль: копия
auto byRef    = [&base](int x) { return base + x; }; // [&]: по ссылке
auto counter = [n = 0]() mutable { return ++n; };   // изменяемое состояние
std::function<void(const std::string&)> f = [](auto s){ /*...*/ }; // тип
```

```
int base = 10;
int (^byCopy)(int) = ^(int x) { return base + x; }; // копия, как [=]
__block int counter = 0;                          // __block: по ссылке + mutable
void (^tick)(void) = ^{ counter++; };
typedef int (^IntTransform)(int);                  // тип блока через typedef
```

Сопоставление:

- ^{ ... } — это блок, замыкание Objective-C. По умолчанию он **копирует** захваченные переменные (как [=] в C++). Захваченное только читается; присвоить ему внутри нельзя.

- Чтобы менять захваченную переменную, её помечают `__block`. Это совмещает в себе и захват «по ссылке» (`[&]`), и право на мутацию (`mutable`) из C++ — одно ключевое слово вместо двух механизмов.
- Тип блока пишут как `returnType (^name)(args)` и обычно прячут за `typedef` — это твой аналог `std::function<...>/auto`. Блок, в отличие от не-захватывающей C++-лямбды, не конвертируется в указатель на функцию. (Обратное есть: в `.mm` под ARC C++-лямбду можно присвоить переменной блочного типа.)
- Память: блоки участвуют в ARC. Захват `self` сильной ссылкой в долгоживущем блоке — классический `retain cycle` (аналог двух `shared_ptr` друг на друга); лечится `__weak`-копией `self`, как `weak_ptr` в C++.

Вывод совпадает по смыслу. C++ (`code/g-blocks.cpp`):

```
byValue(5) = 15
byRef(5)    = 105
counter: 123
Привет, C++!
sum = 10
```

Objective-C (`code/g-blocks.m`):

```
addByCopy(5) = 15
counter = 3
sum = 10
```

16. const, ссылки и параметры

```
void greet(const std::string &name); // ссылка, без копии, const-correct
int &ref = x;                        // ссылки как полноценный тип
const Widget &w = makeWidget();     // const-correctness понижает API
```

```
– (void)greet:(NSString *)name; // объект ВСЕГДА указатель, ссылок нет
// NSString * const ptr – const на сам указатель, не на «глубину»
```

В чём разница:

- В Objective-C (в `.m`) **нет ссылок** (`&`). Объекты передаются и хранятся только как указатели. «Ссылочный» параметр-выход делают через указатель на указатель: `NSError **`, `id *` (тема 14).
- **const-correctness ограничен**. `const` применим к Си-типам и к самому указателю (`NSString * const p`), но «глубокой» константности объекта, как `const Widget&`, в

системе типов нет. Неизменяемость объекта выражают не `const`, а **отдельным классом** (`NSString` vs `NSMutableString`) — см. тему 13.

- Поэтому привычные C++-приёмы «передал по `const&`, чтобы без копии и без правки» в Objective-C не нужны и невыразимы: объект и так передаётся указателем (без копии), а защита от правки — это выбор `immutable`-класса.

(Ссылки `&` и `const Widget&` снова станут доступны в `.mm`-файлах — там работает полноценный C++, тема 17.)

17. Objective-C++ (.mm): главное практическое

Вот ради чего C++-разработчику вообще стоит читать про Objective-C: их можно **смешивать в одном файле**. Файл с расширением `.mm` компилируется как Objective-C++ (`clang++ -ObjC++`), и в нём доступны оба языка целиком: `std::string` и `NSString`, шаблоны и сообщения, RAII и ARC.

Можно держать C++-объект полем `ObjC`-класса и звать C++ из методов:

```
// .mm — C++-объекты как ivar, C++ вызывается из метода Objective-C.
@implementation Sample {
    Stats      _stats;    // обычный C++-класс с std::vector внутри
    std::string _label;    // std::string как ivar
}
- (void)addValue:(double)v { _stats.add(v); }    // зовём C++ из ObjC-метода
- (double)mean            { return _stats.mean(); }
@end
```

И наоборот — звать Objective-C из обычной C++-функции:

```
// .mm — Foundation внутри чистой C++-функции.
static std::string normalize(const std::string &input) {
    @autoreleasepool {
        NSString *s = [NSString stringWithUTF8String:input.c_str()];
        NSString *up = [[s stringByTrimmingCharactersInSet:
            [NSCharacterSet whitespaceCharacterSet]] uppercaseString];
        return std::string([up UTF8String]);    // NSString -> std::string
    }
}
```

Запусти оба файла. `code/g-interop.mm` — C++-контейнеры внутри `ObjC`-класса:

```
mean = 3.70
sample: mean=3.70, sorted=[1.0 1.5 3.0 4.0 9.0 ]
```

code/g-interop-call.mm — Objective-C из C++-функции:

```
'привет' -> 'ПРИВЕТ' (len=6)
'Objective-C++' -> 'OBJECTIVE-C++' (len=13)
'mix' -> 'MIX' (len=3)
```

Грабли, о которых надо знать заранее:

- **Деструкторы C++-полей.** Для класса с C++-ivar'ами компилятор генерирует скрытый метод `.cxx_destruct`, и runtime зовёт его, когда объект уничтожается (после `-dealloc`), — это тот самый метод, что мы видели в дампе методов в главе 14. Он есть и без ARC. То есть `std::vector/ std::string` как ivar честно разрушаются, без ручного кода. Симметрично есть `.cxx_construct` для их конструирования.
- **C++-ivar должен быть default-constructible.** Компилятор инициализирует ivar'ы при создании объекта, поэтому C++-поле обязано иметь конструктор по умолчанию. Тип без него (например, требующий аргумент) напрямую в ivar не положить — заворачивай в указатель, `std::optional` или `std::unique_ptr`.
- **Ссылку (&) в ivar положить нельзя.** Ссылка не имеет «пустого» состояния и не переназначается, а ivar'ы создаются до твоего `init`. Нужна косвенность — храни указатель.
- **Кодировка при %s.** %s в NSString/NSLog трактует Си-строку в системной кодировке, а не обязательно UTF-8 — кириллица из `std::string` через %s может исказиться. Конвертируй явно через `[NSString stringWithUTF8String:...]` и печатай %@.
- **Один .mm тянет за собой C++-рантайм.** Смешивать дёшево в плане кода, но .mm компилируется дольше и подключает `libc++`. Чисто ObjC-файлы оставляй .m, переводи в .mm только те, где реально нужен C++.

Именно так в экосистеме Apple живут кроссплатформенные C++-ядра под нативным UI: бизнес-логика и алгоритмы на C++ (общие с Android/Windows), а тонкий слой UI и интеграции с Cocoa — на Objective-C, и стыкуются они в .mm.

18. Когда что — и сводка

Грубое, но рабочее правило выбора:

- **C++** — когда нужен перформанс «к металлу», zero-overhead абстракции, value-семантика, шаблонное обобщение, кроссплатформенность, контроль над каждым байтом и каждым тактом.
- **Objective-C** — когда работаешь с Cocoa/Foundation, нужна динамика рантайма (KVO, target-action, пересылка, категории, сериализация), гибкость «спросить объект в рантайме» важнее последней наносекунды.

- **Вместе** — в экосистеме Apple их мешают через .mm: C++-ядро плюс ObjC-обвязка к системным фреймворкам.

Сводная таблица «C++ → аналог в Objective-C»:

C++	Objective-C
vtable / virtual call	objc_msgSend (всегда динамически)
template<T> (мономорфизация)	lightweight generics (стираются в рантайме)
std::shared_ptr	strong-ссылка (ARC)
std::weak_ptr	weak-ссылка (само-зануляющаяся)
~T() по выходу из scope	-dealloc по обнулению счётчика
dynamic_cast / typeid	isKindOfClass: / respondsToSelector: + runtime
абстрактный класс / интерфейс	@protocol (без состояния)
множественное наследование	один суперкласс + несколько протоколов
свободная функция над чужим типом	категория (@interface NSString (X))
namespace	префиксы имён (NS, CF, свой)
лямбда [](){}	блок ^{ }
[&] / mutable захват	__block
std::function<...>	typedef-тип блока
std::string (value)	NSString * (heap, по ссылке)
std::vector (value, копия при =)	NSArray * (по ссылке, сору явно)
const / immutable через const	отдельный immutable-класс
RAII	ARC
исключения как штатные ошибки	NSError ** (исключения — для багов)
перегрузка функций/операторов	нет — уникальные селекторы

Проверяем

Все файлы приложения лежат в code/ с префиксом g-. Команды сборки по языкам:

```
# Objective-C (.m)
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/g-ИМЯ.m -o /tmp/t && /tmp/t

# C++ (.cpp)
clang++ -std=c++17 -Wall -Wextra -O2 \
    code/g-ИМЯ.cpp -o /tmp/t && /tmp/t
```

```
# Objective-C++ (.mm)
clang++ -ObjC++ -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/g-ИМЯ.mm -o /tmp/t && /tmp/t
```

Файлы и что показывают:

- `g-nil-vs-null.cpp / g-nil-vs-null.m` — UB при вызове `y nullptr` против безопасного сообщения к `nil`.
- `g-copy-semantics.cpp / g-copy-semantics.m` — глубокая копия `std::vector` против копирования указателя у `NSMutableArray`.
- `g-blocks.cpp / g-blocks.m` — лямбды и захват против блоков и `__block`.
- `g-poly.cpp / g-poly.m` — виртуальная диспетчеризация против `duck typing` на `id + respondsToSelector:`.
- `g-overload.cpp` — перегрузка функций и операторов (в Objective-C её нет).
- `g-category.m` — категория добавляет метод прямо в `NSString`.
- `g-interop.mm` — C++-объекты (`Stats`, `std::string`) как `ivar` ObjC-класса; C++ зовётся из ObjC-методов.
- `g-interop-call.mm` — Objective-C/Foundation внутри чистой C++-функции.

Все двенадцать собираются под `-Wall -Wextra` **без единого предупреждения**. Дата, время и числа в скобках у строк `NSLog` будут твои.

Документация Apple

- Programming with Objective-C (язык: объекты, сообщения, протоколы, категории, свойства) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/Introduction/Introduction.html
- Отдельной главы про Objective-C++ в актуальных руководствах Apple нет (она была в снятом с сайта «The Objective-C Programming Language»). Правила смешивания, ARC в `.mm`, `-fobjc-arc-exceptions`, `objc_precise_lifetime` — в спецификации Clang: clang.llvm.org/docs/AutomaticReferenceCounting.html.
- Objective-C Runtime — developer.apple.com/documentation/objectivec/objective-c-runtime (`objc_msgSend`, `class_copyMethodList`, `isKindOfClass:`, `respondsToSelector:` — механика главы 14).
- Transitioning to ARC Release Notes (ARC, `strong/weak`, `retain cycle`; см. главу 13) — developer.apple.com/library/archive/releasenotes/ObjectiveC/RN-TransitioningToARC/Introduction/Introduction.html

- Working with Blocks (блоки, `__block`, циклы удержания; см. главу 12) — developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/WorkingwithBlocks/WorkingwithBlocks.html
- Error Handling Programming Guide / NSError — developer.apple.com/documentation/foundation/nerror (идиома NSError **: см. главу 18).
- Using Imported Lightweight Generics in Swift — developer.apple.com/documentation/swift/using-imported-lightweight-generics-in-swift (как устроены NSArray<NSString *> * и что из них видит Swift).
- Атрибуты Clang (`objc_direct`, `objc_precise_lifetime`) — clang.llvm.org/docs/AttributeReference.html.